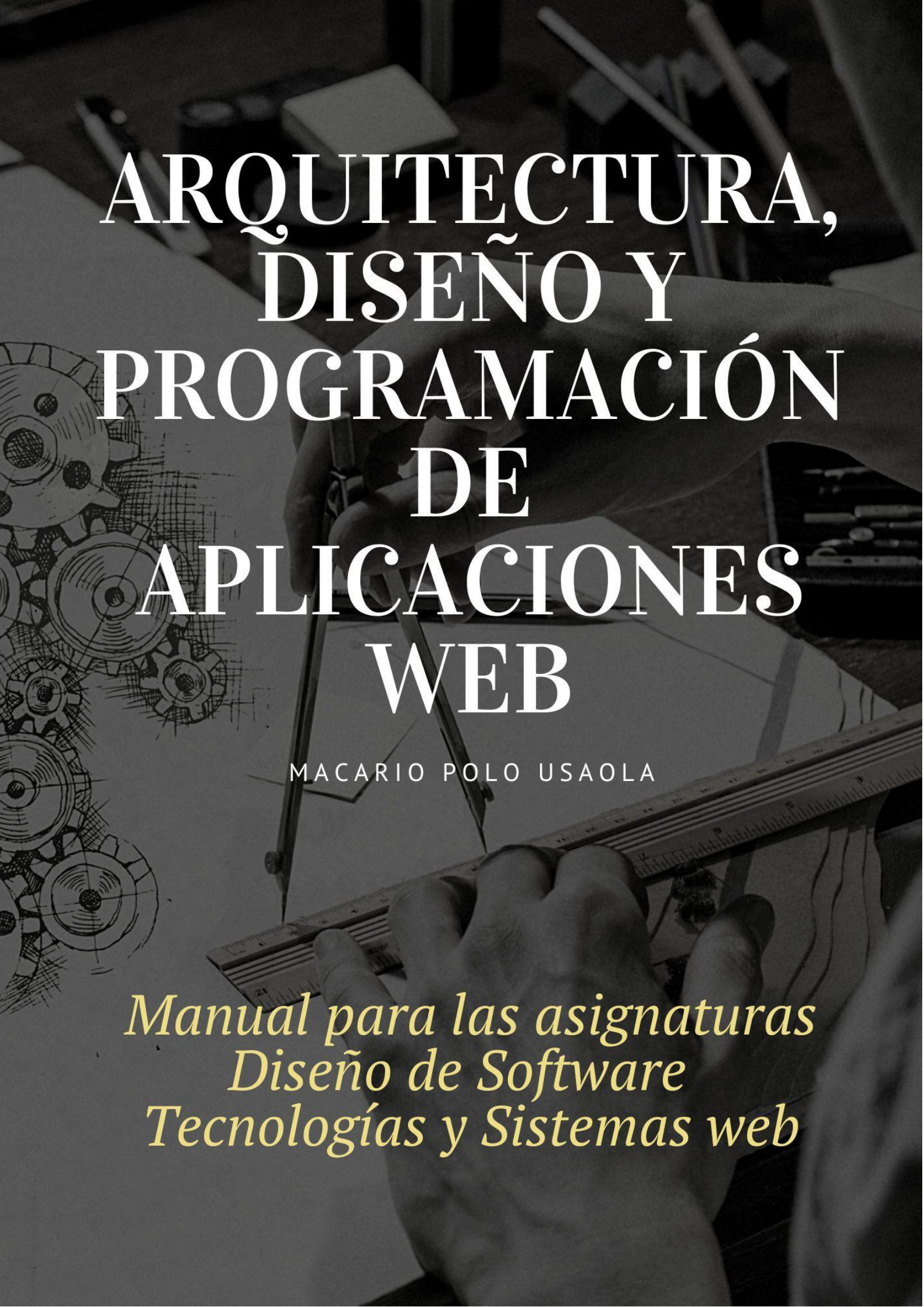


ARQUITECTURA, DISEÑO Y PROGRAMACIÓN DE APLICACIONES WEB

MACARIO POLO USAOLA

Manual para las asignaturas
Diseño de Software
Tecnologías y Sistemas web

© Macario Polo Usaola, 2023
Diseño de portada: Prado Polo García, pradete2000@gmail.com © 2023.

Macario Polo Usaola es catedrático de Lenguajes y Sistemas Informáticos en la Escuela Superior de Informática de la Universidad de Castilla-La Mancha.

Dedicado a Macario Polo Hervás

Primera parte: ¿Qué es la arquitectura de un sistema software? 13

Capítulo 1. Estilos arquitectónicos 15
1. Concepto de arquitectura software 15
2. Modelo 4+1 de Kruchten 16
2.1. Vista lógica 16
2.2. Vista de procesos 16
2.3. Vista de desarrollo 16
2.4. Vista física 16
2.5. La quinta vista, la de escenarios 17
2.6. Adaptación del modelo 4+1 17
3. Arquitectura cliente-servidor 17
3.1. Presentación remota 17
3.2. Presentación distribuida 18
3.3. Sistema de datos remoto 18
3.4. Lógica distribuida 18
3.5. Datos distribuidos 19
4. Arquitectura cliente-servidor en entornos web 19
4.1. Vista física de un servidor web 20
4.2. Estilos arquitectónicos en aplicaciones web 20
5. Arquitectura multicapa 22
6. Arquitectura Modelo-Vista-Controlador (MVC) 22
7. Arquitectura hexagonal 23
8. Arquitectura de *pipes and filters* (tuberías y filtros) 25
9. Microservicios 25

Segunda parte: Protocolos y comunicación en sistemas web 27

Capítulo 2. Comunicación en sistemas web 29
1. Clientes y servidores 29
1.1. Clientes de tipo navegador 29
1.2. Otros tipos de clientes 31
2. Comunicación básica por *http* 32
2.1. Interacción manual e interacción con un navegador 33
3. Cabecera *Content-Type* 35
4. Partes y contrapartes 36
5. Códigos de respuesta 38
6. Tipos de peticiones 39
6.1. Respuestas esperadas 40
6.2. Cambio de intenciones 41
7. Cookies 42
7.1. Cookies de terceros 43
8. Protocolo *ws* 44

Tercera parte: Desarrollo de un sistema web sencillo 47

Capítulo 3. Descripción del sistema 49
1. Descripción de la versión 1 del proyecto 49
2. Vista arquitectónica de alto nivel 49

Capítulo 4. Desarrollo de *Order the numbers* 53
1. Introducción a Angular 53
2. Creación de un proyecto básico 53
2.1. Ejecución 56
2.2. Primer bindeo 57
2.3. Eliminación de código 58
3. Componentes en Angular 58
4. Creación de un componente para el juego *Order the numbers* 59
4.1. Microarquitectura de *Order the numbers* 61
4.2. Creación de la clase *Otn* 62
4.3. Conexión visual de *Otn* con la vista 65

4.4. Conexión del comportamiento 66
4.5. Adaptación a la descripción del juego 72
4.6. Ligera adecuación del aspecto visual 74

Capítulo 5. Desarrollo del *backend* para *Word race* con *Test-Driven Development* 77
1. Introducción 77
2. Un esbozo de la arquitectura del *backend* 77
3. Configuración de Eclipse 78
3.1. Instalación de un JDK versión 17 78
4. Creación y preparación del proyecto 78
4.1. El fichero *application.properties* 80
5. Visión del proyecto como desarrolladores *Test Driven* 81
6. Dependencias de Spring 84
7. Primer test en el proceso *Test Driven* 87
7.1. Uso de un servidor falso 88
7.2. Implementación del controlador 91
7.3. Paso del mensaje al servicio 95
7.4. Implementación del servicio 97
7.5. Creación del repositorio y de la entidad 98
7.6. Creación de la base de datos 104
7.7. Inserción de palabras en la base de datos 105
7.8. Compleción del caso de prueba 106
8. Desarrollo de la funcionalidad *checkWord* 109
8.1. Escritura de los casos de prueba 110
8.2. Implementación del código de producción 111
8.3. Ejecución de los casos 114
9. Alternativas de implementación 115

Capítulo 6. Implantación de la base de datos real en un *docker* 117
1. Introducción 117
2. *Dockerización* de la base de datos 117
3. Instalación de un cliente para SQL Server 119
4. Creación de la base de datos 120
5. Uso de la base de datos desde el código de producción 121
6. Ejecución del código de producción 123
7. Inserción de datos en la tabla *word* 124
7.1. El patrón *Singleton* 129

Capítulo 7. Ejecución de pruebas con la base de datos real 133
1. Cambios en configuración de ejecución 133
2. Escritura de un caso de prueba 133

Capítulo 8. Desarrollo del *front-end* para *Word race* 135
1. Planteamiento del *front-end* 135
2. Primeros pasos 135
3. Conexión con el *backend* 136
3.1. Importación del *HttpClient* 138
3.2. Creación del campo *wrService* en *WrComponent* 138
3.3. Creación del campo *client* en *WRService* 138
3.4. Envío de mensajes al servidor 139
3.5. Implementación de *WrComponent (I)* 140
3.6. Implementación de *WR* 141
3.7. Implementación de *WrComponent (II)* 141
3.8. Implementación del servicio *WRService (I)* 142
3.9. Enlace de la vista con el modelo (I) 144
3.10. Adición de *Word race* a la pantalla principal 144
4. Ejecución 145
4.1. Depuración del cliente 145
4.2. Corrección de la vista 148
5. Compleción del código del cliente 148
5.1. Procesamiento del observable 153
5.2. Comentarios a la tercera solución 154

Capítulo 9. Introducción a las pruebas funcionales con Selenium 157
1. Introducción 157
2. Instalación 157
3. Grabación de un sencillísimo *script* 158
4. Preparación del fichero Java de test 159
5. Escritura de un escenario sencillo para *Order the numbers* 160
6. Pruebas de *Word race* 163
6.1. Ejecución 164
6.2. Control del tiempo en Selenium 166

Capítulo 10. Pruebas de rendimiento con JMeter (I) 169
1. Introducción 169
2. Diseño de una prueba sencilla 169
2.1. Preparación 169
2.2. Ejecución con 5 usuarios 171
2.3. Ejecución con 50 usuarios 172

Capítulo 11. Uso de un *profiler* 177
1. Introducción 177
2. *Profiling* del servidor 177
2.1. *Profiling* con un uso manual 178
2.2. *Profiling* con JMeter 179
3. Refactorización del código 179
3.1. Tests de rendimiento tras la refactorización 181
4. Otra posible solución: utilizar una caché 182
4.1. Costes y beneficios de la caché 185

Capítulo 12. Reejecución de pruebas 187
1. Pruebas de regresión 187
2. Reejecución de los casos de prueba 187

Capítulo 13. Pruebas de rendimiento con JMeter (II) 189
1. Continuamos con el escenario de pruebas 189
2. Desarrollo del escenario 189
2.1. Adición de una petición a *checkWord* 189
2.2. Envío de la propia palabra recibida 190
2.3. Pseudocódigo del plan de pruebas 191
2.4. Diseño del plan de pruebas 192
3. Ejecución y análisis de los resultados 193
4. Posibles refactorizaciones para mejorar los tiempos 194
4.1. Opción A: creación de un índice en la tabla 194
4.2. Opción B: buscar en la caché 196

Cuarta parte: *put, post, delete, options*, peticiones CORS 199

Capítulo 14. Modificación del sistema 201
1. Nuevos requisitos 201
2. Análisis y diseño de los cambios 201
2.1. Estereotipado de las asociaciones 204
2.2. El jugador virtual 205
2.3. Inclusión del *Word race* 206
3. Plan de desarrollo 207

Capítulo 15. Integración del *router* en el lado cliente 209
1. El *router* 209
2. Actualización del *front-end* 210

Capítulo 16. Gestión de usuarios en el servidor 213
1. Caso de uso *register* 213
1.1. Un caso de prueba para el escenario normal 213
1.2. Implementación del controlador 214
1.3. Implementación del servicio 220
1.4. Implementación de la entidad *User* 221
1.5. Implementación del repositorio *UserDAO* 222
1.6. Ejecución del caso de prueba *testRegister200* 222

1.7. Reejecución de *testRegister200* 224
1.8. Prueba de escenarios alternativos 225
2. Caso de uso *login* 232
3. Caso de uso *removeUser* (I) 233
4. Modificación del caso de uso *login* 234
4.1. Inyección de la sesión en los servicios web 236
5. Caso de uso *removeUser* (II) 237
5.1. Ejecución del caso *testDelete* 238
5.2. Ejecución de *testDelete* 239

Capítulo 17. Mejora de los casos de prueba 241
1. Medida de la cobertura 241
2. Técnicas para la creación de buenos datos de prueba 242
2.1. Clases de equivalencia 243
2.2. Valores límite 243
2.3. Conjetura de errores 243
2.4. Obtención de todas las salidas 244
3. Criterios de cobertura para los datos de prueba 244
3.1. *Each use* (cada valor) 244
3.2. *Pairwise* (todos los pares) 245
3.3. *All combinations* (todas las combinaciones) 249
4. Otros criterios de cobertura para código fuente 250
5. Obtención de casos de prueba a partir de máquinas de estado 252
5.1. Todos los estados y todos los caminos interesantes 253
5.2. Todas las transiciones 253
5.3. Todos los pares de entrada y salida 253
5.4. Escritura de oráculos 254
5.5. Escritura de casos de prueba. Refactorización de código 254

Capítulo 18. *login* y *register* en el *front-end* 261
1. Justificación 261
2. Interpretación de los resultados de las pruebas del Capítulo 16 261
3. Preparación 262
4. Implementación del servicio 263
5. Caso de uso *register* 264
5.1. Prueba manual 265
5.2. Pruebas con Selenium 266
6. Caso de uso *login* 268
6.1. Prueba manual 269
6.2. Más casos de prueba 270

Capítulo 19. El caso de uso *removeUser*. Peticiones CORS 275
1. Caso de uso *removeUser* 275
1.1. Prueba manual del caso de uso 275
2. Diagnóstico del error 276
3. Peticiones CORS (*Cross-Origin Resource Sharing)* 278
3.1. Envío de la petición *login* 279
3.2. Envío de la petición *removeUser* 280
4. Reejecución de las pruebas 281
5. Filtrado de métodos 281
6. Filtrado de cabeceras 282
6.1. Personalización de la *response* 282
6.2. Tratamiento de la *response* y personalización de la *request* 283
6.3. Tratamiento de la *request* en el servidor 285
7. Pruebas del caso de uso 286

Quinta parte: WebSockets para los juegos multijugador 287

Capítulo 20. *Otn* para dos jugadores: *backend* (I) 289
1. Análisis y diseño de la solución 289
2. Implementación del *GamesController* 289
2.1. Alternativa 1: sala de espera de jugadores 289
2.2. Alternativa 2: sala de espera de partidas 292

3. Implementación de *Match*. Programación reflexiva 293
3.1. Creación reflexiva de *Match* para la alternativa 1 295
3.2. Creación reflexiva de *Match* para la alternativa 2 295
4. Elección de alternativa 296
5. Prueba del recurso web *play* 297

Capítulo 21. WebSockets 301
1. Comunicación mediante protocolo *ws* 301
2. WebSockets en el lado cliente 301
3. WebSockets en el lado servidor 302
4. Establecimiento de la conexión 302
5. Implementación del WebSocket en el servidor 304
5.1. ¿Funciona el WebSocket? 305

Capítulo 22. *Otn* para dos jugadores: *backend* (II) 307
1. Comportamiento deseado 307
1.1. Establecimiento de la conexión *ws* 307
2. Diseño de un Web Socket cliente para los casos de prueba 308
3. Implementación del WebSocket cliente 311
4. Implementación de la nueva clase de prueba 312
5. Recepción de la conexión en *WSGames* 314
5.1. Enlace de las dos sesiones (I). Patrón *Wrapper* 314
6. Enlace de las dos sesiones (II). Cambios en *WSGames* 317
7. Envío de movimientos desde el cliente 319
7.1. Ejecución de *TestOtn2Real* 320
7.2. Tratamiento del cierre de sesiones *ws* 323
7.3. Tratamiento del cierre de sesiones *http* 324
8. Desarrollo de una partida de *Order the numbers* 325
8.1. Planteamiento de un posible escenario 325
8.2. Procesamiento del movimiento en el servidor. Patrón *Template- method* 327
8.3. Diseño de los subtipos de *Match* 329
8.4. Clases abstractas 331
8.5. Implementación de las clases concretas 332
8.6. *OtnBoard* (III) 341
8.7. *Match* 342
9. Ejecución del escenario 345
9.1. Primer intento 346
9.2. Continuación de la partida 348
9.3. Fin de la partida 350
9.4. Control de excepciones: probando un movimiento ilegal 352

Capítulo 23. *Otn* para dos jugadores: lado *front-end* 355
1. Reorganización del *front* 355
1.1. *app.component.ts* 355
1.2. *app.component.html* 355
1.3. Estructura de clases 356
2. Componentes de los juegos multijugador 358
3. Cambios en *login* 359
4. Inicio del juego multijugador: mensajes *http* y *ws* 360
5. Alternativas de diseño del componente del juego 362
6. Implementación de la alternativa 6 366
6.1. Cambios en *MultiplayerComponent* 366
6.2. Recepción del evento en *App* 367
6.3. Emisión del evento al *OtnMultiplayerComponent* 368
6.4. *OtnMatch:* qué clase tan sencilla 369
6.5. Implementación de los estados 369
7. Prueba manual del escenario de inicio de partida 372
7.1. Un error típico 373
7.2. Una solución 374
7.3. Reejecución del escenario 378

Sexta parte: Integración de servicios de terceros 379

Capítulo 24. Correo electrónico 381
1. Nuevos casos de uso 381
2. Suscripción a un servidor de correo electrónico 382
3. Integración del servicio en nuestro *backend* 383
3.1. Fase 1: creación de la cuenta, generación del token y envío del correo 384
3.2. Fase 2: confirmación de la cuenta 387
4. Envío de un redireccionamiento 388

Capítulo 25. Integración de servicios de pago 389
1. Descripción del escenario 389
2. Aplicación a las partidas multijugador 390
2.1. Creación de una cuenta en Stripe 390
2.2. Necesidades para procesar el flujo de mensajes 391
2.3. Implementación del controlador de pagos (I) 391
2.4. Comprobación de las partidas pagadas 393
2.5. Procesamiento en el cliente de la respuesta 406 394
2.6. Envío de la petición de prepago 396
2.7. Carga del formulario de pago 397
2.8. Ejecución del pago 399

Séptima parte: Introducción a los microservicios y a la seguridad 403

Capítulo 26. Filtros *http* en Spring 405
1. Introducción 405
2. Un filtro sencillo 405
3. Jerarquía de los filtros 406
4. Manipulación de la cadena de filtros 409

Capítulo 27. Introducción a los microservicios 413
1. Migración a una arquitectura de microservicios 413
2. Elección de alternativa 414
3. Creación del servidor de gestión de cuentas 414
3.1. Instalación de MySQL 416
3.2. Diseño del servidor 418
3.3. Implementación de *auth* 420
4. Cambios en el cliente para el control de acceso 426
5. Cambios en el sistema *games* 427
6. Cambios para partidas multijugador 428
6.1. Coexistencia de dos sesiones *http* 429
6.2. Adaptación del flujo de pago 432
6.3. Ejercicio 437
6.4. Nuevos problemas para iniciar partida 437

Capítulo 28. Introducción de una capa de seguridad 441
1. Un usuario desleal 441
2. Protección de la base de datos 442
3. Introducción a Spring Security 443
3.1. Cabecera *Authorization : BASIC* 444
3.2. Envío de peticiones con *Postman* 445

Capítulo 29. Filtros de seguridad (I): autorización de todas las peticiones 449
1. Autorización de cualquier petición 449
2. Preparación de pruebas 450
3. Prueba del registro 451
3.1. Registro con Postman 451
3.2. Registro con el navegador 452
4. Prueba de la confirmación de cuenta: una curiosidad 452
5. Prueba del login. De Postman a CURL 453

Capítulo 30. Creación manual de un token de seguridad 455
1. Compra de partidas 455
2. Creación de un hilo 456
3. Construcción de un cliente asíncrono 457

Capítulo 31. Filtros de seguridad (II): filtrado de peticiones 463

1. Filtrado de peticiones: protección de *setCredit* en *auth* 463
2. Con *Authorization : BASIC + credenciales* 463
3. Con *Authorization : BEARER + token* 464
3.1. Obtención del token 465
3.2. Modificación de la configuración de seguridad 466
3.3. Obtención segura del token 467

Octava parte: otros aspectos 469

Capítulo 32. Jugadores virtuales 471
1. Justificación 471
2. Planteamiento del problema 471
3. Un *Timer* trivial 471
4. Alternativas de diseño para el jugador virtual 472
4.1. Alternativa 1: asignación de clientes *http* y *ws* 472
4.2. Alternativa 2: acceso directo a la partida 474
4.3. Elección de alternativa 474
5. Comportamiento del *VirtualUser* 475
6. Posibles implementaciones del *VHttpClient* 475
6.1. Alternativa 1: colocando pausas 475
6.2. Alternativa 2: creando *callbacks* 476
6.3. Elección de alternativa 477
7. Implementación con *callbacks* 477
7.1. Conexión del *WebSocket* 479
7.2. Solo falta seguir... 480

Capítulo 33. Dibujo de gráficos: formato SVG 481
1. Cálculo de π con el método de Montecarlo 481
2. Implementación del cálculo 481
3. Tirando y dibujando los dardos 482
3.1. Dibujo del cuadrado de 2x2 483
3.2. Dibujo del círculo de radio 1 483
3.3. Dibujo de los dardos 484

Capítulo 34. Manejo de objetos binarios 485
1. Los usuarios, con foto 485
2. Tratamiento del binario en el *front* 485
3. Tratamiento del binario en el *back* 487

Índice de términos 491

PRIMERA PARTE: ¿QUÉ ES LA ARQUITECTURA DE UN SISTEMA SOFTWARE?

Capítulo 1. Estilos arquitectónicos

1. Concepto de arquitectura software

Existen muchas definiciones del término «arquitectura software». Booch, Rumbaugh y Jacobson, creadores de UML y del Proceso de Unificado de Desarrollo, la definen como «un conjunto de decisiones significativas acerca de la organización de un sistema software, la selección de sus elementos estructurales y las interfaces que componen el sistema, junto a la descripción del comportamiento y las colaboraciones entre dichos elementos, así como la composición de estos elementos estructurales y de comportamiento en subsistemas».

Para Clements y Kazman (2003), la arquitectura es «la instanciación de un determinado estilo arquitectónico (multicapa, P2P, cliente-servidor, etc.)». Según estos autores, el diseño arquitectónico de un sistema permite:

1) Analizar el grado en que este diseño se ajusta a los requisitos.
2) Considerar otras alternativas en un momento en el que los cambios en el diseño son aún relativamente sencillos.
3) Reducir los riesgos asociados a la construcción del software.

En su libro de 2005 (*Ingeniería del Software: un enfoque práctico)*, Pressman afirma que el diseño arquitectónico:

1) Facilita la comunicación entre los diferentes *stakeholders*.
2) Permite resaltar las decisiones de diseño que tendrán un impacto importante en las etapas de desarrollo subsiguientes.
3) Constituye un modelo relativamente sencillo y entendible de cómo se estructurará el sistema y de cómo colaborarán sus diferentes componentes.

Sin el ánimo de dar una definición formal ni completa, sino más bien una definición práctica que permita entender el concepto, podemos decir que la arquitectura es el diseño a muy alto nivel de la estructura o comportamiento de un sistema software. En otras palabras, y aunque pueda resultar cuestionable decirlo en este libro que pretende ser técnico, pintar en una servilleta 3 o 4 cajas unidas por flechas es una forma de diseño arquitectónico (Figura 1).

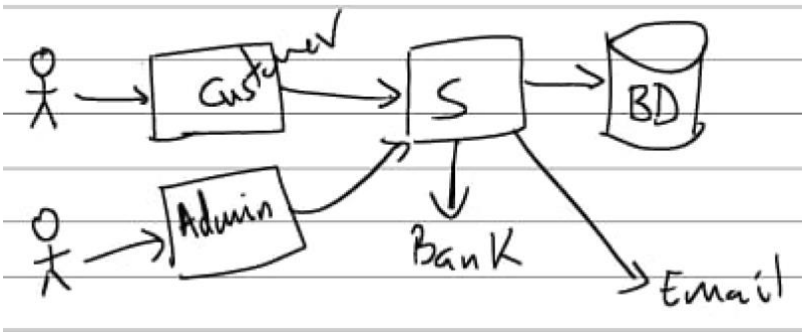


Figura 1. Una vista arquitectónica de un sistema

2. Modelo 4+1 de Kruchten

Lo cierto es que la semidefinición que acabamos de dar coincide muy bien con la de Philippe Kruchten[1], quien dice que «La arquitectura software está relacionada con el diseño y la implementación de estructuras de alto nivel del software».

De acuerdo con este autor, la arquitectura de un sistema puede describirse desde varios puntos de vista, en lo que se llama el Modelo 4+1, que se compone de las vistas lógica, de procesos, de desarrollo y física, más una vista adicional de escenarios que complementa a las cuatro anteriores.

2.1. Vista lógica

La vista lógica se enfoca en describir la estructura a alto nivel del sistema. Para representar esta vista se utilizan principalmente diagramas de clase, si bien pueden utilizarse también modelos de datos, como diagramas entidad-interrelación o relacionales. En este libro, en el que se utiliza Spring como *framework* para la gestión de la persistencia, no aparece ningún diagrama entidad-interrelación, pues UML tiene, con respecto al modelo de datos, aproximadamente el mismo poder expresivo que el entidad-interrelación, con lo que solamente se usan diagramas de clases.

2.2. Vista de procesos

En esta vista se considera que el sistema es un conjunto de procesos que se comunican de alguna manera (mediante una red, por ejemplo). Para describir la vista de procesos se toman en cuenta los requisitos no funcionales (rendimiento o disponibilidad, por ejemplo) y aspectos como la concurrencia, distribución tolerancia a fallos o integridad del sistema, así como la forma en que las principales abstracciones de la vista lógica se ajustan a los procesos identificados.

Puesto que se trata de una vista dinámica, que describe el flujo de procesos o tareas, suelen utilizarse diagramas de estado o de actividad (ambos en UML) o diagramas de flujo de datos.

2.3. Vista de desarrollo

La vista de desarrollo se ocupa de la organización modular del software en el entorno de desarrollo. El sistema se divide en subsistemas, con la idea de que cada subsistema lo pueda desarrollar un grupo de desarrolladores diferente. Se recomienda que se dé una estructura de capas a cada subsistema, de manera que cada capa pueda comunicarse únicamente con las capas adyacentes, a las que debe ofrecer interfaces bien definidas.

Esta vista es útil para la asignación de recursos al desarrollo, planificación del proyecto, monitorización del avance del proyecto e identificación de elementos reutilizables. De UML, se pueden utilizar los diagramas de paquetes.

2.4. Vista física

Representa la distribución física de los procesos y clases en los diferentes nodos sobre los que el sistema se vaya a ejecutar, así como las conexiones entre ellos. Por ello, se utilizan los diagramas de despliegue de UML para representarla (Figura 2). De hecho, en castellano se la llama también “vista de despliegue”.

[1] El artículo original está disponible en https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf

Figura 2. Un diagrama de despliegue

2.5. La quinta vista, la de escenarios

La quinta vista es la vista de escenarios, que complementa a todas las demás y, probablemente, sea la única de la que no puede prescindirse. Los escenarios son abstracciones de los requisitos más importantes, pues muestran interacciones entre procesos, entre objetos o entre subsistemas.

De acuerdo con Kruchten, esta vista es redundante con respecto a las demás, pero sirve principalmente dos objetivos:

1) Es un medio para descubrir elementos arquitectónicos durante el propio proceso de diseño arquitectónico.
2) Sirve como validador e ilustrador del mismo diseño arquitectónico, y para la preparación de casos de prueba.

Para representar esta vista, se utilizan principalmente diagramas de casos de uso y diagramas de secuencia.

2.6. Adaptación del modelo 4+1

No todos los diseños arquitectónicos necesitan todas las vistas: si el sistema es muy pequeño, probablemente las vistas lógica y de desarrollo sean muy parecidas; o, si el software correrá en un único procesador, seguramente no haga falta la vista física. No obstante, y como ya se ha apuntado, la vista de escenarios siempre resulta útil. De hecho, en las páginas que nos quedan podremos comprobarlo, pues utilizaremos con mucha frecuencia diagramas de casos de uso y de secuencia.

3. Arquitectura cliente-servidor

Con un diseño arquitectónico cliente-servidor, las responsabilidades se distribuyen entre los suministradores del servicio (el *servidor)* y sus solicitantes (*clientes)*.

Por lo general, es el cliente el que inicia la comunicación con el servidor mediante una *request*. Ambos sistemas tienen que estar de acuerdo en el protocolo de comunicación utilizado y en el formato de los mensajes intercambiados.

Normalmente, todos los sistemas tienen responsabilidades de presentación (interfaz de usuario), de procesamiento y de persistencia de datos, y existen diversos estilos arquitectónicos en función de la distribución de responsabilidades entre nodos. A continuación se ilustran algunos de ellos simplemente a título ilustrativo, pues puede haber muchas otras combinaciones.

3.1. Presentación remota

En este modelo de distribución de responsabilidades (Figura 3), el cliente dispone únicamente de responsabilidades de presentación (es decir, de mostrar los datos); el procesamiento y almacenamiento se lleva a cabo en el servidor. Sería el caso de los llamados «terminales tontos». Un cajero automático es un ejemplo de este sistema.

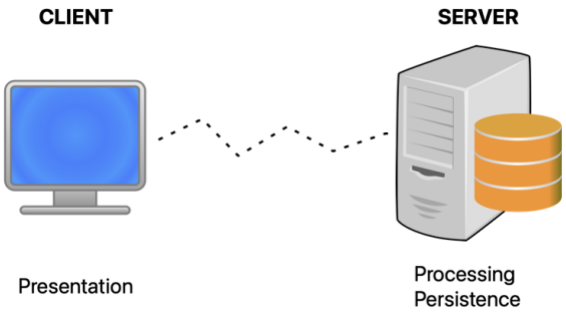


Figura 3. Presentación remota

3.2. Presentación distribuida

En el modelo de presentación distribuida el cliente es un "terminal tonto", aunque también se dispone de ventanas para interactuar con los datos en el servidor (Figura 4).

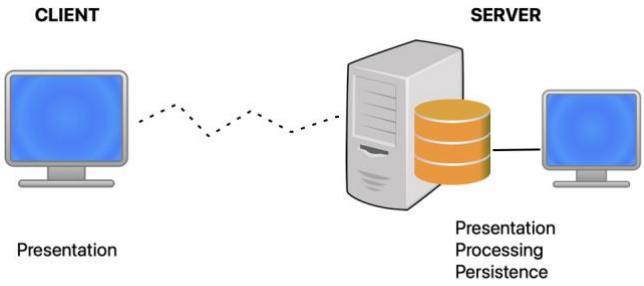


Figura 4. Presentación distribuida

3.3. Sistema de datos remoto

En el sistema de datos remoto (Figura 5), el servidor se encarga únicamente de almacenar la información. Todo el procesamiento reside en el cliente. Una aplicación web que recibe datos y que, sin mayor validación ni procesamiento, la guarda como llega en la base de datos, es un ejemplo de un sistema de este estilo.

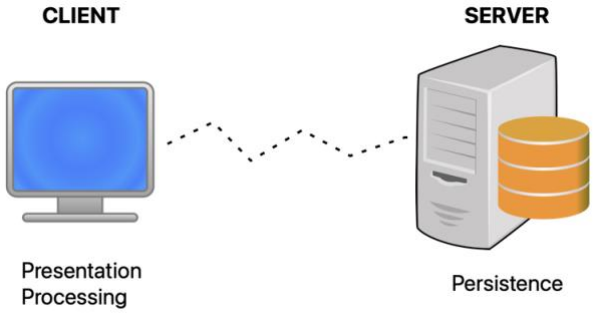


Figura 5. Sistema de datos remoto

3.4. Lógica distribuida

En este modelo (Figura 6), tanto el cliente como el servidor tienen responsabilidades de procesamiento de los datos a nivel de negocio. Una aplicación web en la que hay algún tipo de programa en JavaScript que procesa los datos, los envía después al servidor, que los reprocesa y valida antes de almacenarlos, es un ejemplo de un sistema de este tipo.

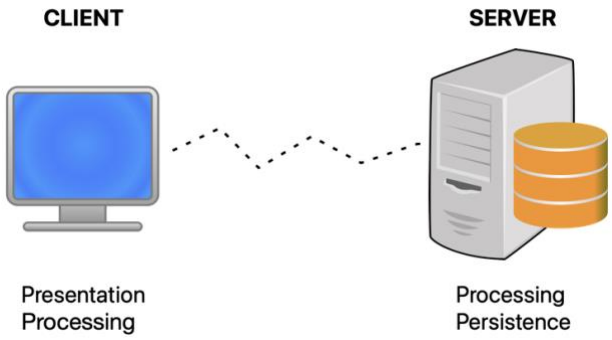


Figura 6. Sistema de lógica distribuida[Error! Bookmark not defined.,Error! Bookmark not defined.]

3.5. Datos distribuidos

En este modelo (Figura 7), hay datos que residen tanto en el cliente como en el servidor: quizás los datos de uso más frecuente residan en el cliente a modo de caché, mientras que el resto residen en el servidor, que los entrega al cliente solamente en el momento en que lo necesitan.

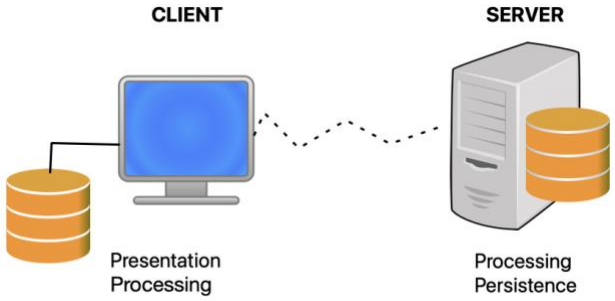


Figura 7. Sistema de datos distribuidos[Error! Bookmark not defined.,Error! Bookmark not defined.]

4. Arquitectura cliente-servidor en entornos web

Esencialmente, una aplicación web es un sistema cliente-servidor en el que ambos subsistemas (el cliente y el servidor) se comunican, en principio, mediante protocolo *http*. El cliente (*user agent*) es normalmente un navegador que, por un lado, renderiza las respuestas recibidas del servidor y, por otro, posee un motor que interpreta el código JavaScript que recibe del servidor. El cliente, no obstante, puede ser una aplicación nativa para dispositivos, por ejemplo, Android o iPhone: lo importante para considerar que se trata de una aplicación web es que se comuniquen por *http*.

Normalmente, la comunicación se inicia mediante una petición GET desde el cliente hacia el servidor (Figura 8), en la que el cliente solicita algo del servidor.

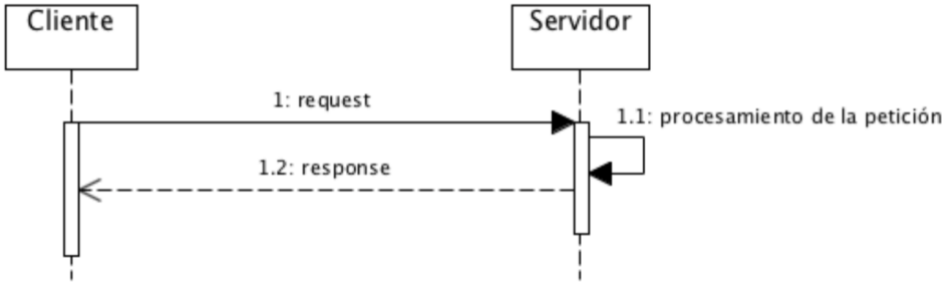


Figura 8. Inicio de una comunicación entre un cliente y un servidor

La respuesta devuelta, como veremos en el Capítulo 2 y, con mucho más detalle, a lo largo de todo el libro, lleva cabeceras, un código de respuesta (que, resumiendo, indica si ha sido o no satisfactoria) y puede llevar un cuerpo. Cuando el cliente es un navegador, sabe cómo manejar el cuerpo de la respuesta en función del valor de la cabecera *Content-Type*. Algunos valores habituales de esta cabecera son *text/plain, text/html, text/css, application/pdf, application/json, image/jpg* o *video/mp4*.

4.1. Vista física de un servidor web

En la Figura 9 se ilustra la estructura general de un contenedor o servidor de aplicaciones web (abreviadamente, servidor web), como Tomcat, JBoss o IIS (*Internet Information Server*): cuando el servidor se lanza, se pone a la escucha en un puerto (normalmente el 80 si es *http* o el 443 si es *https*).

El servidor de la figura ofrece dos aplicaciones: *facturas* y *games*. Cuando los clientes se conectan, no hacen sino enviar ristras de bytes al puerto de escucha, que son capturadas por una serie de interceptores que tienen los servidores. En función de la ristra de bytes que llegue al servidor, se construye un objeto de tipo *Request* (para *http*) o *Session* (para el *ws* otro protocolo con el que también trabajaremos en este libro), se analiza la URL y el contenedor reenvía la petición a la aplicación y recurso solicitados. Así, si la ristra de bytes que se recibe es conforme:

1) Al protocolo *ws*, el servidor web entiende que debe crear una *Session* y reencaminarla, en esta figura, al elemento etiquetado *WebSocket* que aparece a la derecha, dentro de la aplicación *games*.
2) Si corresponde a una petición *http*, se crea una *Request* y se envía al recurso web que corresponda.

Como se ve, el servidor web puede estar conectado a otros servidores: en la figura, está conectado a dos servidores de bases de datos (un servidor de MySQL y otro de MongoDB), que pueden o no correr en la misma máquina física.

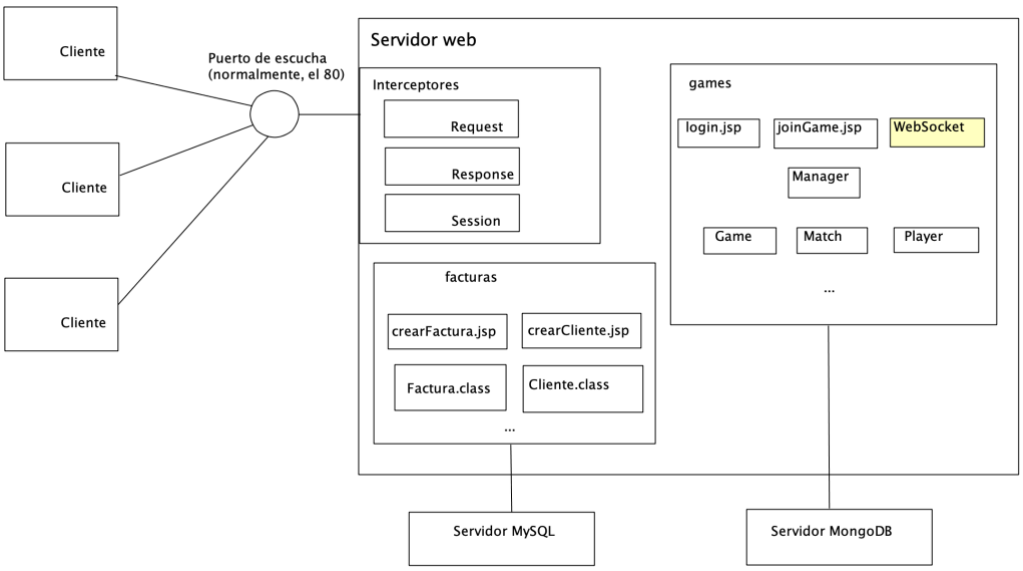


Figura 9. Un servidor web que ofrece dos aplicaciones: facturas *y* games

4.2. Estilos arquitectónicos en aplicaciones web

Los navegadores modernos incorporan potentes motores de ejecución de JavaScript, que es un lenguaje de programación orientado a objetos y de

propósito general, aunque se utiliza fundamentalmente en el lado cliente de las aplicaciones web (también en el lado servidor, con *frameworks* como *NodeJS).*

Antiguamente los navegadores eran meros "renderizadores" del texto HTML que los servidores le entregaban. Haciendo un símil con la *Presentación remota* que vimos en la sección «Presentación remota» (página 17), a Figura 10 ilustra el hecho de que el servidor recibía la petición y, en la práctica, se encargaba todo el procesamiento de la información, incluyendo la generación de la vista, que era enviada al cliente para su mero renderizado.

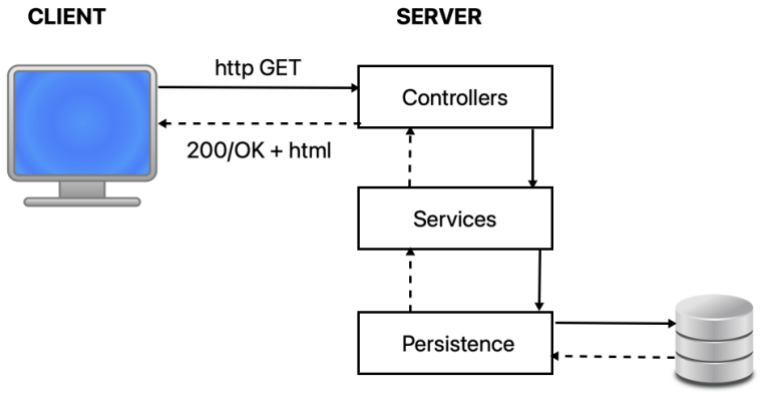


Figura 10. Modelo arquitectónico primitivo de una aplicación web

Una aplicación web actual se parece más al modelo que mostramos en la Figura 11, en la que el cliente posee un motor de ejecución JavaScript (*JS Engine)* y un sistema de almacenamiento local (almacenes denominados *sessionStorage* y *localStorage).* La información viaja desde el cliente hacia el servidor y viceversa en forma de objetos JSON (una forma de representar objetos en forma de cadenas de caracteres), con lo que el tráfico de datos puede limitarse a, exclusivamente, la información estrictamente necesaria para realizar una acción. Esto fomenta la construcción del lado cliente de las aplicaciones como SPA (*Single Page Application,* o Aplicación de Página Sencilla): con la primera petición *GET*, el servidor envía al cliente todo el HTML y todo el JavaScript que el cliente necesitará; si, en algún momento, el servidor debe actualizar al cliente (por una *response* enviada tras la recepción de una *request,* por ejemplo), el servidor le envía únicamente el dato justo para que se actualice la vista. Este dato será recibido en el cliente e interpretado por el *JS Engine,* que actualizará la vista a partir de los elementos del DOM (*Document Object Model*) de la página web.

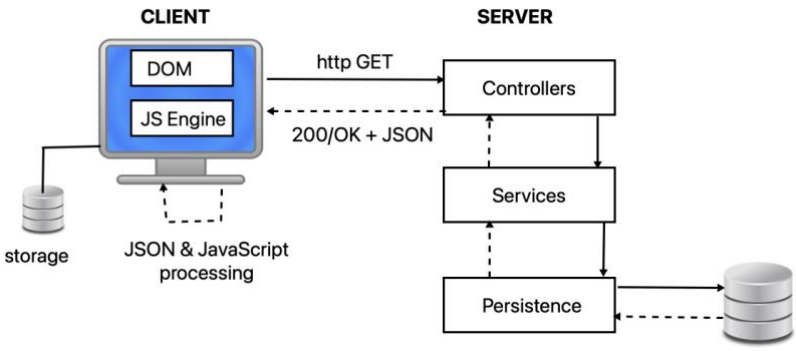


Figura 11. Arquitectura de una aplicación web moderna

5. Arquitectura multicapa

En un sistema desarrollado con un diseño arquitectónico multicapa, las responsabilidades se distribuyen en capas, identificándose habitualmente tres capas:

1) La de presentación, en la que se colocan las ventanas (vistas) de la aplicación con las que interactúa el usuario.
2) La de dominio o negocio (en la que reside la lógica de la aplicación y en la que están los elementos más importantes que aparecen en el enunciado del problema).
3) La de persistencia, que contiene las clases encargadas de gestionar el almacenamiento de los datos.

Idealmente, cada capa interactúa solamente con sus capas adyacentes: presentación con dominio, y dominio con presentación y con persistencia. No es admisible acceder directamente a la capa de persistencia desde la de presentación.

Si nos fijamos en la Figura 11, ese diseño arquitectónico cliente-servidor de «una aplicación web moderna» ilustra también que, en el lado del servidor, hay una arquitectura con una capa de controladores, otra de servicios y otra de persistencia, cada una de las cuales se comunica solo con sus adyacentes.

6. Arquitectura Modelo-Vista-Controlador (MVC)

El patrón Modelo-Vista-Controlador (MVC) es un patrón de arquitectura software en el que se separa el dominio (*Modelo,* en el contexto del patrón) de la vista (*Vista* en el contexto del patrón, valga la redundancia), mediante un elemento intermedio al que se llama *Controlador*.

Habitualmente, el usuario interactúa con el sistema mediante una ventana (que forma parte de la vista) y, a través de ella, solicita la ejecución de alguna funcionalidad. La vista enviará un mensaje al controlador que, a su vez, lo pasará hacia el modelo (insistimos, aunque lo repitamos, en que el modelo es el dominio), que es el que se encargará de servir la funcionalidad solicitada mediante, por ejemplo, acceso a una base de datos, a un sistema externo, realizando un cálculo, etcétera. Cuando el modelo ha terminado el cómputo, le devuelve el resultado al controlador que, a su vez, lo comunica a la vista. Gráficamente:

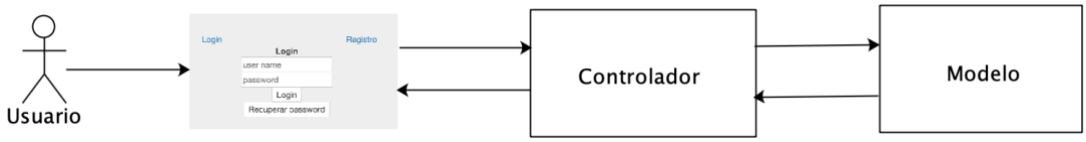


Figura 12. Estructura básica del patrón MVC

Es posible que el modelo tenga que acceder a una base de datos, o que reciba mensajes de un sistema externo. Como se muestra en la Figura 13, este sistema externo puede también modificar con sus mensajes el modelo de nuestro sistema: en este caso, el modelo actualizaría el controlador y éste, deseablemente, también la vista.

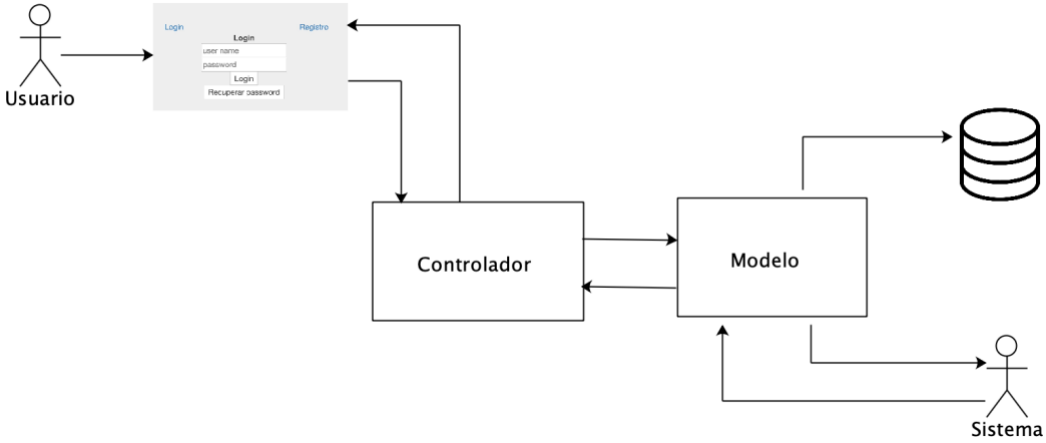


Figura 13. Interacción del modelo con el exterior

Pero ni la comunicación desde el modelo hacia el exterior se hace directamente, ni la comunicación desde el sistema externo se hace directamente sobre nuestro modelo, sino que en ambos casos se usan elementos intermedios.

En la figura anterior, el modelo envía datos al sistema externo, del que puede también recibir información. A alto nivel, que es de lo que tratamos en este capítulo, el diseño arquitectónico del sistema lo podríamos redibujar como en la Figura 14: así como entre el modelo y la base de datos colocamos una capa DAO, también colocamos una "capa de servicios" para enviar información desde el modelo hacia el sistema exterior. Y, para que el modelo reciba datos desde el sistema externo, le colocamos otro controlador.

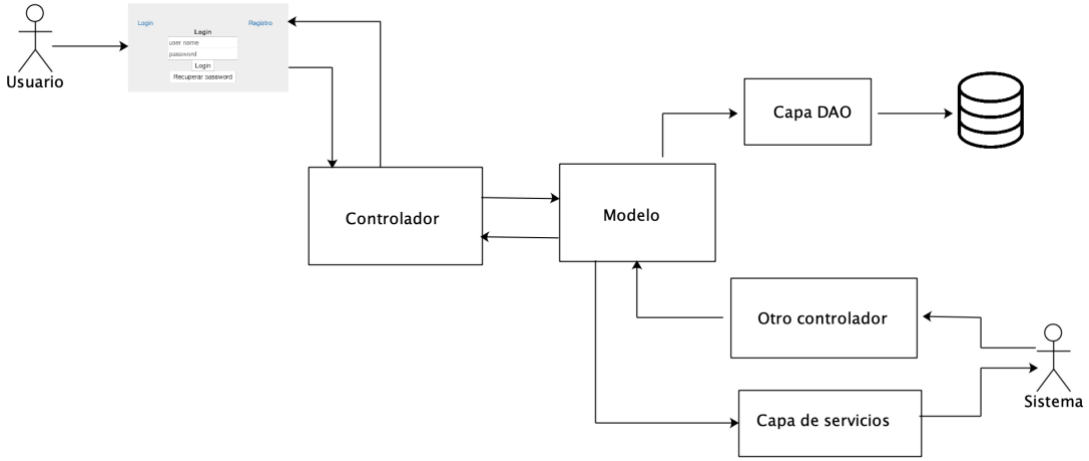


Figura 14. Adición de otras capas para desacoplar el modelo del exterior

7. Arquitectura hexagonal

Casi-casi que el modelo de arquitectura hexagonal se ha explicado con las últimas figuras.

En este estilo arquitectónico, también llamado de *puertos y adaptadores,* la aplicación se dibuja como un hexágono: a la izquierda del hexágono se sitúan los *actores primarios* (aquellos que arrancan la ejecución de casos de uso en el sistema); a la derecha se colocan los *actores secundarios* (aquellos a los que el sistema envía mensajes). Evidentemente, un actor puede tanto enviar como

recibir mensajes: se considera que un actor es primario cuando inicia la comunicación con nuestro sistema.

Los actores de la izquierda envían sus mensajes al sistema a través de *adaptadores de entrada* (se los llamada *driving adapters* y se encuentran en la frontera del sistema: podrían ser los controladores de la sección anterior); los adaptadores de entrada, a su vez, conocen a *puertos,* que son interfaces que se implementan en la capa de aplicación.

La parte derecha del hexágono es aproximadamente simétrica con respecto de la parte izquierda: cuando la capa de aplicación quiere enviar un mensaje a un actor (por ejemplo, a una base de datos o a un servidor web externo), lo envía a un *puerto* (también una interfaz) que estará implementada por algún *adaptador de salida* ("driven adapters").

En la Figura 15 se muestra un ejemplo de este estilo arquitectónico aplicado al backend de un supuesto sistema:

- El actor primario *Cliente* puede ser, por ejemplo, una aplicación que corre en un navegador. En este caso, el adaptador al que envía mensajes podría ser un controlador REST. El controlador conoce a uno o más *puertos,* que son interfaces que se implementan en la capa de aplicación.
- Supongamos que el actor primario *Reloj* manda periódicamente al sistema una señal mediante *rmi* para que se mande un correo electrónico a aquellos usuarios que no hayan modificado su contraseña en los últimos tres meses. Bien, pues el adaptador al que el reloj envía sus mensajes sería un Servidor *rmi*, que a su vez conocería a una interfaz que se implementaría en la capa de aplicación. La implementación de esta interfaz utilizaría los puertos de conexión a la base de datos y al servidor SMTP para realizar las tareas indicadas. Al revés de lo que ocurre con los puertos de entrada (que se implementan en la capa de aplicación), los puertos de salida se implementan en los adaptadores de salida.

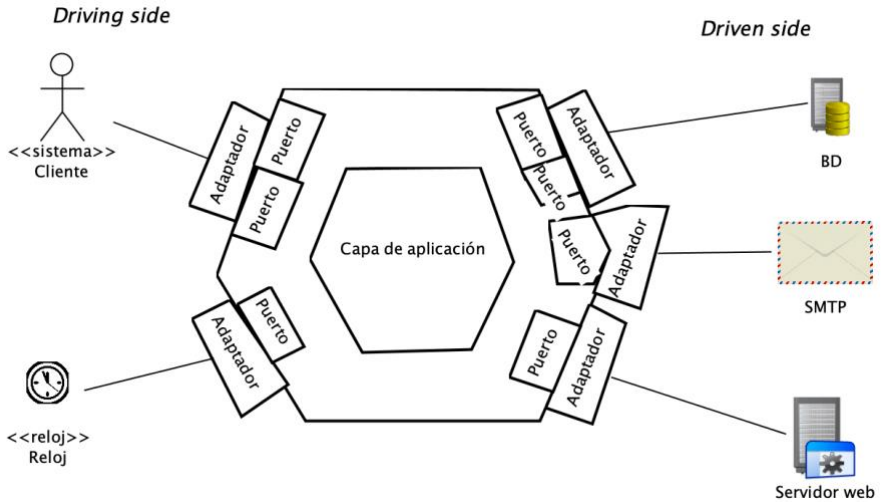


Figura 15. Un ejemplo de arquitectura hexagonal

8. Arquitectura de *pipes and filters* (tuberías y filtros)

En este estilo arquitectónico, el sistema se entiende como un grafo cuyos nodos son componentes (a los que se llama *filtros*) y cuyos arcos representan entradas y salidas (*pipes* o *tuberías).*

Cada filtro recibe datos, los transforma y produce salidas que envía al siguiente filtro. Los filtros solo dependen de las entradas que reciben (no conocen a sus vecinos), de modo que no dependen de la implementación de otros filtros y pueden ser fácilmente sustituibles.

Es relativamente fácil entender de un vistazo el funcionamiento general de un sistema de este tipo, su mantenimiento es sencillo y es fácil detectar problemas como cuellos de botella. Se aplica casi exclusivamente a procesos *batch:* un compilador, por ejemplo, se describe bien con una arquitectura de este tipo: se recibe el código fuente, se hace el análisis léxico y sintáctico, se construye el árbol de sintaxis abstracta, se compila, se enlaza si es preciso, se genera el ejecutable, etcétera.

El envío de correos a los usuarios que no han cambiado la password en los últimos tres meses y que citábamos en la sección de la arquitectura hexagonal puede describirse como un proceso de pipes and filters (Figura 16). Ahora bien: es importante considerar que, si el proceso se describe así, su implementación ha de ajustarse también a este modelo.

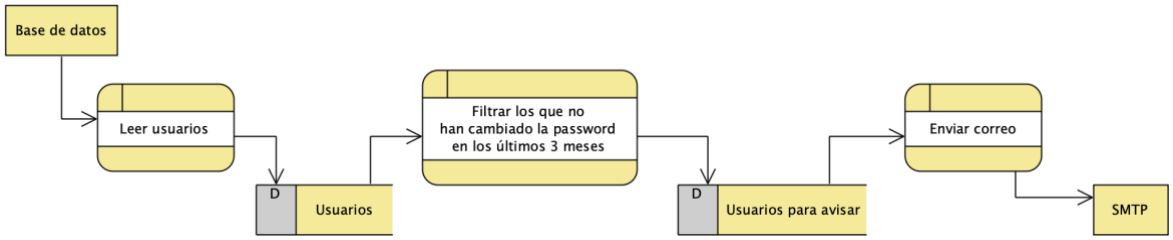


Figura 16. Un proceso descrito mediante tuberías y filtros

9. Microservicios

En un diseño arquitectónico basado en microservicios, la aplicación se entiende, se construye y se despliega como un conjunto de pequeños servicios, cada uno de los cuales se ejecuta de forma autónoma en un proceso separado. Cada servicio es altamente cohesivo, en el sentido de que ofrece una funcionalidad o conjunto de funcionalidades bien clara y delimitada.

Cada servicio se puede desplegar independientemente de los demás y pueden estar construidos con distintos lenguajes de programación y pilas tecnológicas. Los servicios se comunican entre sí mediante peticiones a la API que cada uno ofrece (*http*, por ejemplo), transmisión de sucesos e intermediarios de mensajes.

SEGUNDA PARTE: PROTOCOLOS Y COMUNICACIÓN EN SISTEMAS WEB

Capítulo 2. Comunicación en sistemas web

1. Clientes y servidores

Un servidor web es una máquina con un *socket* abierto que escucha constantemente en un puerto y que espera recibir mensajes en el formato especificado por el protocolo que corresponda (en principio *http).*

Los clientes, a los que también se llama *user-agents,* pueden ser aplicaciones de cualquier tipo, pero han de utilizar el mismo protocolo que utiliza el servidor para comunicarse con él. En muchas ocasiones, el *user-agent* es un cliente de tipo navegador (Chrome, Safari, Opera, Edge, etcétera), pero no tiene por qué ser así.

1.1. Clientes de tipo navegador

Un navegador es una aplicación especialmente preparada para mantener comunicaciones con servidores web y que dispone de la capacidad de procesar recursos de muy diversos tipos, como texto en JavaScript o HTML, audios, imágenes, etcétera.

El diseño arquitectónico de las primeras aplicaciones web seguía en casi todos los casos el modelo de presentación remota (Figura 10): inicialmente, todo el procesamiento de la información se ejecutaba en el servidor, que entregaba al navegador directamente el texto HTML con la respuesta. Así, los navegadores eran casi meros *renderizadores* del texto recibido.

Un ejemplo muy significativo de este comportamiento se daba en los formularios de registro de usuarios de muchos sitios web: aparece normalmente una lista de *widgets* para que el usuario escriba su nombre, correo, contraseña, etcétera. En muchas ocasiones aparecen también varios desplegables para elegir el país, región, provincia y municipio. En esas primeras aplicaciones, cuando el usuario elegía el país, el navegador detectaba el evento de cambio en el *widget* de selección del país y enviaba toda la información que se tuviera hasta el momento en el formulario; el servidor la leía, recuperaba las regiones del país seleccionado y devolvía al navegador la misma página cargada con la misma información que había recibido, más el *widget* de regiones cargado con las regiones seleccionables. Al seleccionar la región sucedía lo mismo: se enviaba toda la información al servidor, que devolvía la misma página, pero ya incluyendo las provincias. La Figura 17 ilustra este proceso:

1) Pepe se conectaba con su navegador a cierta página de registro de usuarios.
2) El navegador enviaba la petición al servidor (mensaje 2), que recuperaba el recurso (mensaje 3) y se lo devolvía en forma de página HTML (mensaje 4), quizás con un <*select*> precargado con los países que existiesen en aquel momento. Luego, el navegador *renderizaba* la página, escribiendo con letra más grande lo que pudiera tener la etiqueta <*h1*>, poniendo cierto a una tabla con el atributo <*border*>, etcétera.
3) Sobre esa página, Pepe escribía sus datos (mensaje 6) y, en el 7, seleccionaba España en el <*select*> de países. El navegador detectaba el cambio mediante alguna función asociada al manejador de eventos *onchange* (mensaje 8) y enviaba todos los datos del formulario al

servidor, bloqueando la interacción con el usuario, que debía esperar a que llegase la respuesta.

4) El servidor, con una consulta previa a la base de datos, elaboraba una versión nueva de la página web (mensajes 10 a 12), que devolvía al *user-agent*. Así, lo que este hacía era recargar por completo la página web.
5) El proceso continuaba de la misma forma con la elección de la región.

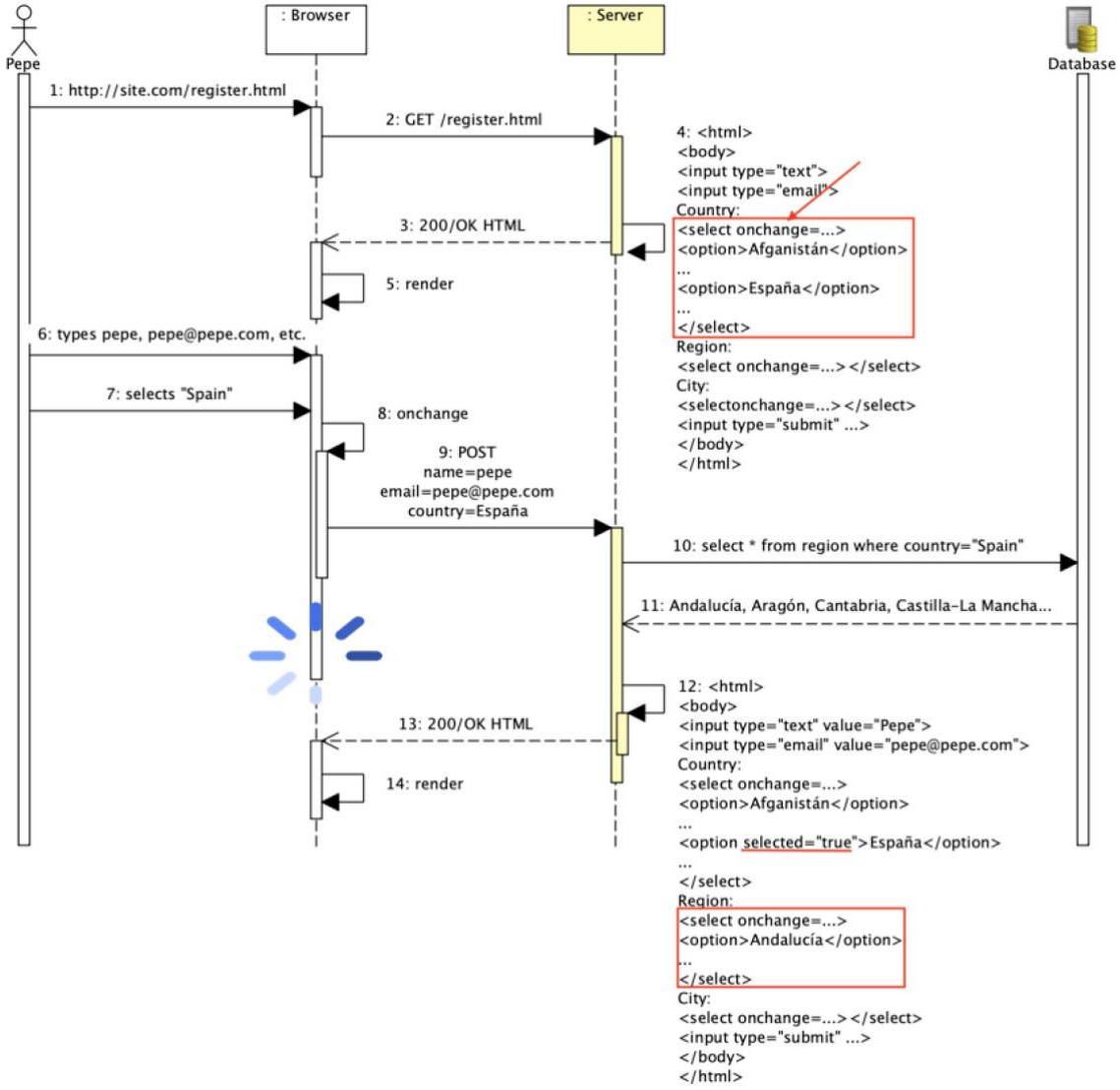


Figura 17. Modelo de paso de mensajes con presentación remota pura

Pero en 2006, el *World Wide Web Consortium* (W3C) publicó el primer estándar con la especificación de la *XMLHttpRequest,* un objeto que ya se venía utilizando, y que permite que el navegador envíe peticiones asíncronas a un servidor. Una petición asíncrona se envía (y se recoge su respuesta) en segundo plano (es decir, sin bloquear la interfaz de usuario).

En la Figura 18 el usuario se conecta al mismo recurso del mismo servidor, que elabora la página HTML. Igual que antes, Pepe escribe sus credenciales y vuelve a seleccionar España como país, y el navegador responde también ejecutando la función asociada al manejador de eventos *onchange*. Pero es aquí donde se produce un cambio interesante del comportamiento: mientras llega la respuesta, Pepe puede seguir manipulando la página y corregir "Pepe" por "José" (mensaje 12), lo que significa que la interfaz de usuario no se bloquea. Además,

la petición se envía a un recurso *getRegions* que no ha de elaborar de nuevo la página HTML, sino que simplemente devuelve al navegador la lista de regiones de España. El navegador, entonces, redibuja solamente el *<select>* con las regiones.

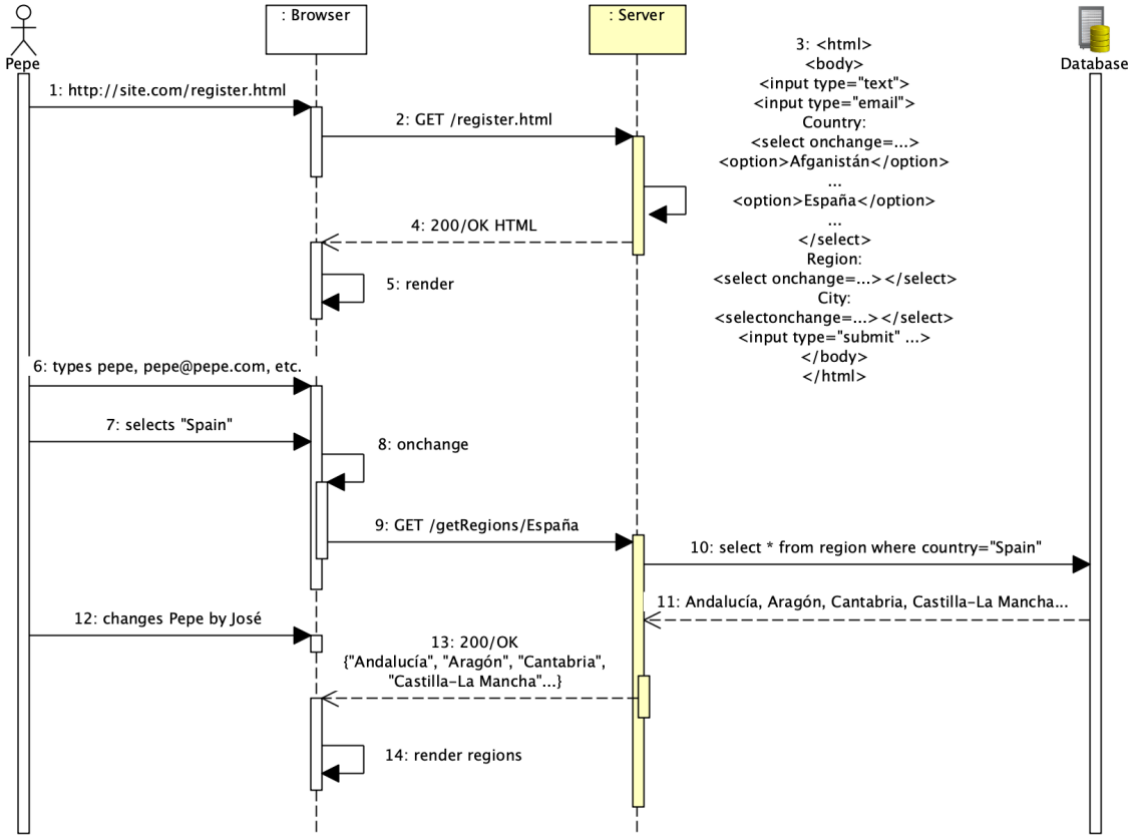


Figura 18. Paso de mensajes con una petición asíncrona

1.2. Otros tipos de clientes

Como ya apuntábamos, no todos los clientes de aplicaciones web tienen por qué ser navegadores, sino que una aplicación web puede tener un cliente de cualquier tipo siempre y cuando la comunicación entre ambos se establezca y mantenga por uno de los protocolos que se usan en la web.

Un servidor web puede, por ejemplo, ofrecer una funcionalidad que pueda ser invocada desde un cliente, independientemente de que éste sea o no un navegador. El lado izquierdo de la siguiente figura, por ejemplo, ilustra una sencilla aplicación móvil en Android que, sin embargo, invoca un servicio ofrecido por un servidor web para obtener el tipo de un triángulo: en su *request*, el cliente envía las longitudes de los tres lados; al recibir la petición, el servidor construye una *response* (por ejemplo, *{"resultado" : "SCALENE"})* que no contiene (como en el ejemplo anterior, de la recuperación de las regiones de un país) información "visual", sino que solo contiene un objeto que ha de ser procesado adecuadamente por la aplicación cliente para mostrar el resultado en la pantalla de la forma que se desee (en el ejemplo, mostrando el texto *Scalene* debajo del botón).

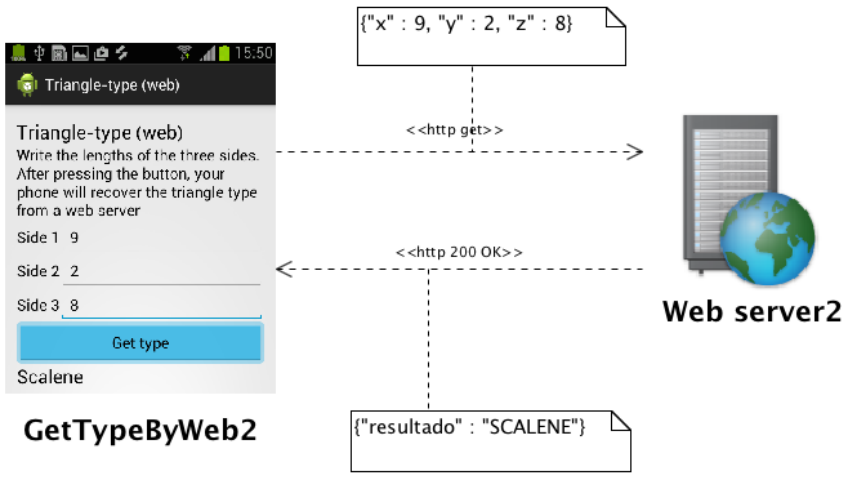


Figura 19. Una aplicación web cuyo cliente no es un navegador

Sin ánimo de ser pesados, sino de que quede bien clara la idea, el protocolo de comunicación entre cliente y servidor es *http,* pero el servidor no entrega al cliente texto HTML, ni JavaScript, ni CSS: lo que le envía de vuelta en la *response* es la cadena mencionada (*{"resultado" : "SCALENE"}*), que ha de ser *parseada* por el cliente para interpretarla y mostrar adecuadamente el resultado que corresponda. En el ejemplo se utiliza el formato *JSON* tanto para formatear la petición (un objeto con tres campos numéricos *x, y, z*) que envía el cliente como la respuesta que devuelve el servidor (un objeto con un solo campo de tipo texto).

Realmente, cualquier aplicación puede ser cliente web de cualquier otra. Cuando hablábamos tan brevemente de los microservicios en la sección 9 del capítulo anterior, decíamos que «Los servicios se comunican entre sí mediante peticiones a la API que cada uno ofrece (*http*, por ejemplo)», lo que significa que un servidor puede también enviar peticiones *http* a otro. El *user-agent* no es ni una aplicación móvil ni un navegador, sino un completo *backend* que corre en alguna máquina. Volveremos a esto, sobre todo, a partir del Capítulo 24.

2. Comunicación básica por *http*

Una *request* es la petición que el cliente envía al servidor. En *http*, una *request* consta de:

1) Una línea inicial que incluye: un método (también llamado "verbo", que indica el tipo de petición), una url con el recurso solicitado y una versión del protocolo que se está utilizando.
2) Una o más cabeceras (*headers)*. Una cabecera es una cadena formada por un par *nombre : valor*.
3) Una línea en blanco que indica el fin de la sección de cabeceras.
4) Tal vez, un cuerpo del mensaje con la información de carga (lo que se llama *payload* y, con menor frecuencia, *body)*. Si la petición, por ejemplo, sube un fichero a un servidor, los bytes que componen el propio fichero serán los que viajen en el *payload*

Cuando el servidor recibe la *request,* elabora la *response* con un formato parecido:

1) Una *línea de estado* con la versión del protocolo, un código de respuesta y, a veces, un mensaje explicativo.
2) Una o más cabeceras con información del servidor y de la respuesta.
3) Una línea en blanco que indica el fin de la sección de cabeceras.

4) El cuerpo de la respuesta, que podrá ser texto en HTML, código JavaScript, una imagen, etcétera.

2.1. Interacción manual e interacción con un navegador

Para meternos, un poquito al menos, en las tripas de la comunicación web, podemos descargar e instalar un servidor en nuestro propio equipo, como Apache Tomcat, un servidor multiplataforma gratuito que se utiliza en multitud de sistemas reales.

Cuando lo lanzamos, se pone a escuchar por defecto en el puerto 8080: si en un navegador escribimos *http://localhost:8080* o *http://127.0.0.1:8080*, nos aparecerá la página principal del servidor (Figura 20). El hecho de escribir esa url en la barra de direcciones y pulsar *Enter* supone que el navegador envíe una petición GET al servidor.

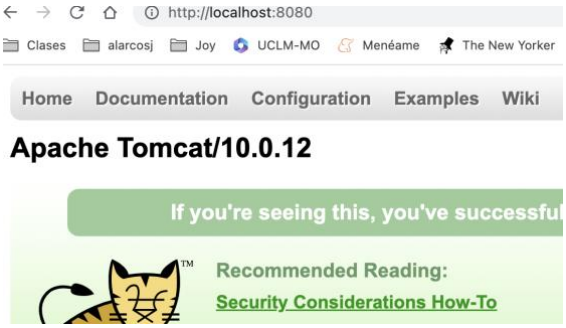


Figura 20. Página principal de Tomcat en localhost

También podemos enviar manualmente esa misma petición si abrimos una consola o terminal y ejecutamos el comando *telnet* (es posible que en Windows deba activarse escribiendo en una consola: *pkgmgr /iu:"TelnetClient"*).

telnet es un cliente del protocolo TCP, con lo que nos sirve para conectarnos a cualquier tipo de servidor: otra cosa, evidentemente, es que el servidor entienda lo que le digamos si no le escribimos en su idioma (es decir, en su protocolo).

En nuestro caso, vamos a usar *telnet* para hablar con nuestro servidor: en la consola escribimos un primer mensaje que se ajuste al formato de una *request*. Ya sabemos: método o verbo (que será GET en este primer ejemplo), url del recurso que solicitamos y versión del protocolo; añadiremos la cabecera *Host* con la dirección *ip* de nuestro servidor, y no mandaremos ninguna *payload*. En la Figura 21:

A) En (1), ejecutamos *telnet*, solicitando conectarnos al puerto 8080 de la máquina que esté escuchando en la dirección 127.0.0.1.

B) Cuando se conecta, podemos comenzar a dirigirnos al servidor hablándole en su idioma: en (2) escribimos la petición tal y como hemos descrito en la sección anterior:
   - El verbo, en este caso, es GET.
   - El recurso es el recurso que el servidor tenga en la raíz (/).
   - El protocolo es *http* versión 1.1.

   Nótese en (3) que, conforme al protocolo, añadimos la línea en blanco que debe colocarse después de la sección de cabeceras.

C) En (4), (5) y (6) se describe la respuesta que se recibe:
   - La línea de estado (4) con la versión del protocolo y el código de respuesta: 200 significa que la petición se ha procesado correctamente.

- Después, en (5) vienen tres cabeceras, de las que la más significativa es la primera, *Content-Type,* que el servidor utiliza para indicarle al cliente el tipo de elemento que se le está enviando. En este caso, le dice que es texto en HTML (*text/html).*
- La segunda cabecera (*Transfer-Encoding: chunked)* no es demasiado importante en este momento, pero lo explicaremos para que no parezca que queremos escaquearnos. *chunked* le indica al cliente que la respuesta le va a llegar por trozos (*chunks)*, y el 2000 que aparece en (6) es el tamaño en bytes de cada trozo.
- El cuerpo de la respuesta está en (7), y es el mismo texto HTML que compone la vista mostrada en la figura anterior.

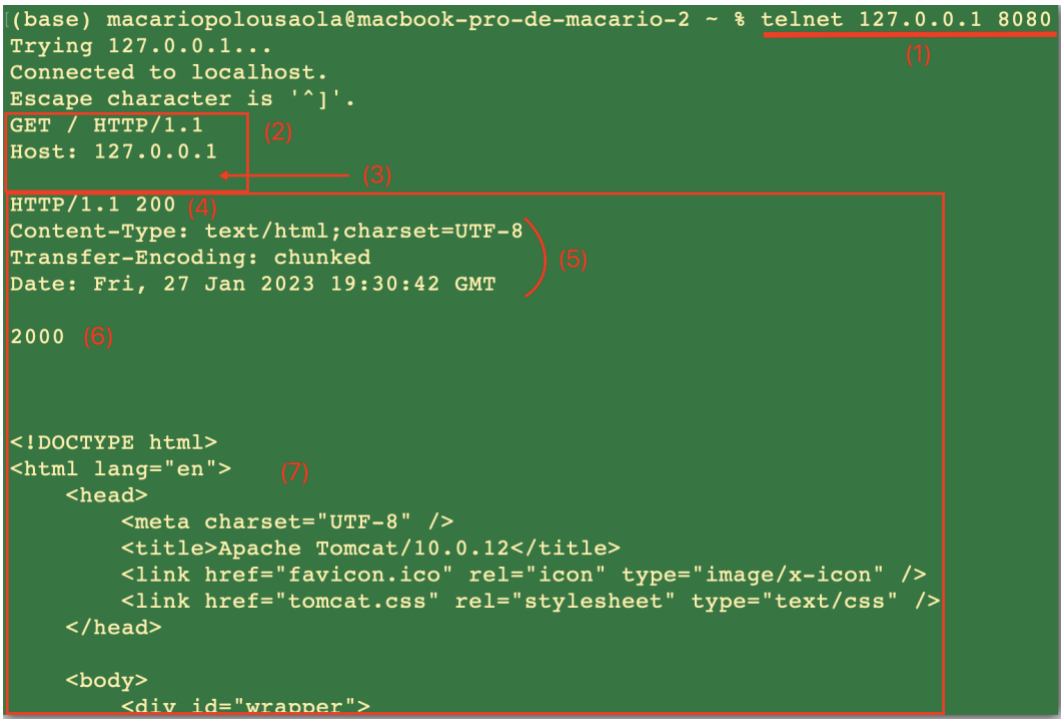


Figura 21. Envío manual de la petición y obtención de la respuesta

La respuesta que recibe *telnet* a la petición que ha enviado a *localhost:8080* (*GET / HTTP/1.1:* o sea, «me dirijo a ti en tu idioma, dame por favor tu recurso raíz») es, en efecto, exactamente la misma que ha recibido el navegador en la Figura 20. Sucede que el navegador es un *user-agent* especializado en interpretar no solo *http* (protocolo en el nivel 7 de OSI), sino también el texto HTML que recibe; *telnet,* sin embargo, es un cliente TCTP que lo único que hace es mostrar por la consola los bytes que le llegan, sin darles más interpretación.

Aquí es en donde hay una grandísima diferencia entre *telnet* y el navegador: este interpreta mientras recibe el texto en HTML y, por ejemplo, encuentra en el cuerpo de la respuesta una línea como *<img id="tomcat-logo" src="tomcat.svg" alt="[tomcat logo]"*>. Mientras que *telnet* no hace nada, el navegador sabe que debe enviar una nueva petición para recuperar el elemento *tomcat.svg*. Esto se ilustra en la Figura 22, en la que se ve cómo, tras la petición inicial, el navegador envía muchas otras peticiones para *renderizar* la respuesta.

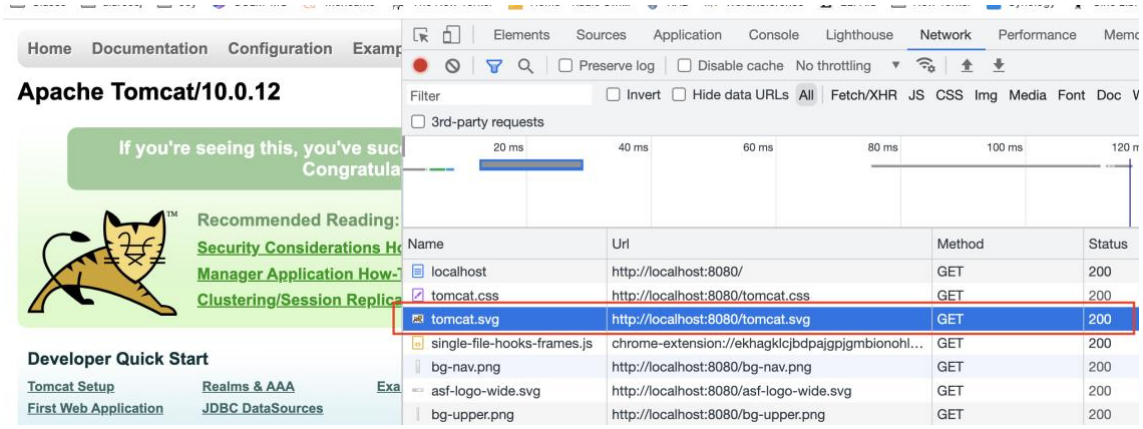


Figura 22. En el navegador, una petición desencadena muchas otras peticiones

Cada una de estas peticiones se envía en la forma de una *request*, y el servidor, en su *response*, devuelve el contenido solicitado, que puede ser un fichero *css*, una imagen, un programa en JavaScript, un vídeo, etcétera.

3. Cabecera *Content-Type*

Como se acaba de decir, el servidor puede devolver una muy diversa cantidad de tipos de contenido en sus respuestas, y conviene que el cliente sepa qué tipo de respuesta se le está enviando a cada una de sus *requests*, para que pueda *renderizarlas* adecuadamente: si no se le dice, el cliente no sabrá qué hacer con ella.

Este es precisamente el propósito de la cabecera *Content-Type* que el servidor debe incluir en cada una de las respuesta que envía. Los valores de esta cabecera son los llamados *MIME Types* (*Multipurpose Internet Mail Extensions*). Entre otros muchos, están los que aparecen en la Tabla I.

Content-Type	**Descripción**
`text/plain`	Texto plano
`text/html`	Texto HTML
`text/css`	Texto CSS
`text/csv`	Texto separado por comas
`text/x-c`	Texto del lenguaje de programación C
`application/pdf`	Fichero PDF
`application/json`	Texto (objetos) en formato JSON
`application/rtf`	Fichero RTF
`image/bmp`	Imagen bmp
`image/gif`	Imagen GIF
`image/png`	Imagen PNG
`video/3gpp`	Vídeo en formato 3gpp
`video/mp4`	Vídeo en formato mp4
`video/mpeg`	Vídeo en formato mpeg
`video/ogg`	Vídeo en formato ogg

Tabla I. Algunos tipos MIME

Si el navegador recibe un contenido con la primera cabecera (*text/plain)*, lo interpreta en efecto como texto plano y lo mostrará de esta manera, aunque sea realmente texto en HTML. En la Figura 23 se muestra el distinto comportamiento de Chrome (izquierda) y Firefox (derecha) ante una repuesta con un valor de *Content-Type* que no es compatible con el cuerpo del mensaje.

Figura 23. Diferente comportamiento ante una Content-Type *incompatible con el cuerpo de la respuesta*

4. Partes y contrapartes

Así como un cliente web envía *requests* y recibe *responses,* el servidor recibe esas *requests* y prepara y envía las *responses* adecuadas. Ambos sistemas (el cliente y el servidor) manipulan entonces objetos de los tipos *request* y *response* aunque, evidentemente, la estructura y operaciones de ambos es diferente en uno y en otro:

1) El cliente, que es quien manipula la *request,* tiene principalmente métodos para colocar información en la *request:* añadir cabeceras, añadir el *payload,* etcétera. El servidor tiene un objeto *request* como contraparte de la *request* del cliente, pero que tendrá, sobre todo, métodos para leer la información contenida en la petición: lectura de cabeceras, del *payload*, etcétera.
2) Del mismo modo, el servidor, que es quien crea las respuestas, manipula objetos de tipo *response* mediante métodos con los que puede colocar información en ella. El cliente, cuando recibe una respuesta, accede a sus contenidos mediante métodos de consulta, pero no puede manipularla.

Así pues, tanto el cliente como el servidor manipulan *requests* y *responses,* pero de manera completamente diferente:

1) En el servidor (la Figura 24 muestra la estructura de las dos interfaces proporcionadas en el paquete *javax.servlet.http* para manipular objetos de estos dos tipos en el servidor, que son conformes a la descripción dada por el W3C), la *request* contiene casi exclusivamente métodos de consulta (tipo *get* o *is*), pues es un objeto recibido del cliente que no tiene sentido modificar. La *response*, sin embargo, es la respuesta que el servidor elabora para enviársela al cliente, por lo que, al ser un objeto que crea y manipula el servidor, ofrece principalmente operaciones para modificarle el estado (*add* y *set*).

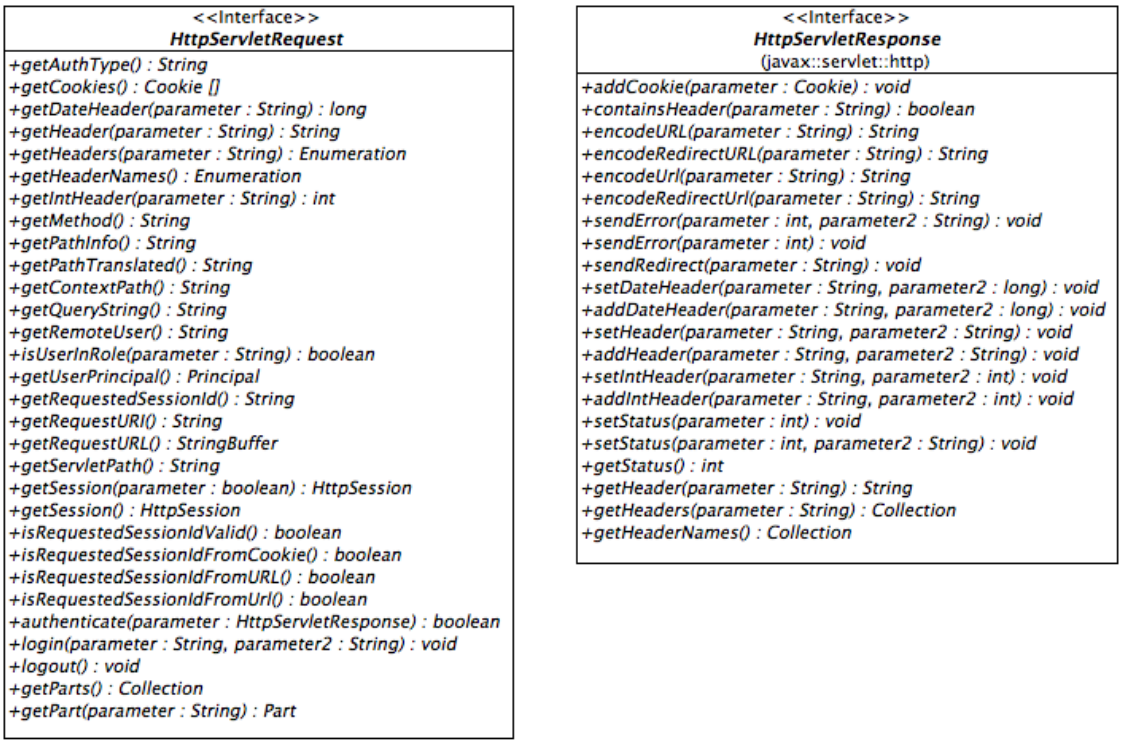


Figura 24. Estructura de una Request y una Response en el servidor (paquete javax.servlet.http)

2) En el cliente también se manipulan objetos de ambos tipos, pero su implementación es completamente diferente. La *request* se crea en el cliente y se envía al servidor, por lo que el cliente debe ofrecer operaciones para crearla, prepararla y enviarla; la *response* es el resultado obtenido del servidor, por lo que sólo se dispone de operaciones para consultarla. La Figura 25 muestra la especificación del W3C de una XMLHttpRequest (una *request* asíncrona, pero *request* al fin y al cabo). Como se ve en el código, tiene cuatro secciones:
    a. En la sección *request* aparecen las operaciones necesarias para crear el objeto y ponerlo en el estado deseado: *open*, por ejemplo, toma como parámetros el método (GET, POST, etc.) y la url con el recurso al que deseamos enviar la petición; con *setRequestHeader* establecemos las cabeceras que sean necesarias; con *send* se envía la petición, quizás con *payload*.
    b. En la sección *response* se define de forma explícita la estructura de la *response* recibida del servidor como respuesta a esta *request*. Como se ve, todas sus operaciones son de consulta: *getAllResponseHeaders()* devuelve las cabeceras recibidas desde el cliente; *getResponseHeader(ByteString)* devuelve una cabecera concreta; *responseText* contiene el texto de respuesta; *status* contiene el código de respuesta (200 si ha ido bien); *statusText,* el texto de respuesta (típicamente, *OK* si el *status* es 200).
    c. Además, hay una sección (*states)* en la que se definen cinco constantes con los posibles estados por los que puede pasar la petición (no enviada, abierta, cabeceras recibidas, cargando y terminada). El estado actual de la petición se almacena en el

atributo de sólo lectura llamado *readyState*, y sus valores admisibles serán uno de los cinco que se acaban de mencionar.

d. Por último, y aunque aparece en primer lugar, la *request* dispone de un manejador de eventos (*onreadystatechange*) que apunta a una función que debe ejecutarse cuando cambie el estado re la respuesta. Típicamente, en esta función de manejo del evento habrá una o más instrucciones condicionales en las que se preguntará por el campo *readyState:* en función de sus valores, se podrá ir mostrando al usuario una u otra información.

```
interface XMLHttpRequest : XMLHttpRequestEventTarget {
  // event handler
  attribute EventHandler onreadystatechange;

  // states
  const unsigned short UNSENT = 0;
  const unsigned short OPENED = 1;
  const unsigned short HEADERS_RECEIVED = 2;
  const unsigned short LOADING = 3;
  const unsigned short DONE = 4;
  readonly attribute unsigned short readyState;

  // request
  void open(ByteString method, [EnsureUTF16] DOMString url);
  void open(ByteString method,
   [EnsureUTF16] DOMString url, boolean async,
      optional [EnsureUTF16] DOMString? username = null,
      optional [EnsureUTF16] DOMString? password = null);
  void setRequestHeader(ByteString header, ByteString value);
  attribute unsigned long timeout;
  attribute boolean withCredentials;
  readonly attribute XMLHttpRequestUpload upload;
  void send(optional (ArrayBufferView or Blob or Document or
      [EnsureUTF16] DOMString or FormData)? data = null);
  void abort();

  // response
  readonly attribute unsigned short status;
  readonly attribute ByteString statusText;
  ByteString? getResponseHeader(ByteString header);
  ByteString getAllResponseHeaders();
  void overrideMimeType(DOMString mime);
  attribute XMLHttpRequestResponseType responseType;
  readonly attribute any response;
  readonly attribute DOMString responseText;
  readonly attribute Document? responseXML;
};
```

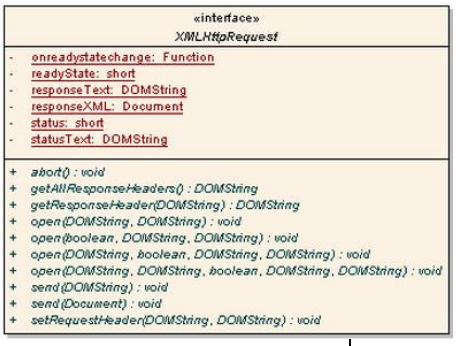


Figura 25. Estructura de una Request y una Response en el cliente

5. Códigos de respuesta

La primera línea de la respuesta contiene la versión del protocolo, un código numérico y, a veces, un texto explicativo. Hay cinco categorías para los códigos en función de cuál sea su primer dígito:

1) 1xx (Informativo): se recibió la petición y se continúa el proceso.
2) 2xx (Éxito): la petición se recibió, se entendió y se aceptó.
3) 3xx (Redirección): se requieren más acciones para completar la *request*.
4) 4xx (Error en el cliente): la sintaxis de la *request* está mal o no se puede ejecutar.
5) 5xx (Error en el servidor): el servidor falló al procesar una *request* aparentemente válida (por ejemplo, se intenta convertir a número una cadena con letras nada más recibir la *request).*

La especificación de *http*[2] es bastante explícita en cuanto a las situaciones en que se debe enviar cada código de respuesta, y qué información debe acompañar a dicho código.

Por ejemplo, el código 401 (*Unauthorized*) se devuelve cuando la petición no ha podido servirse debido a que el cliente no tiene credenciales suficientes para acceder al recurso solicitado. Según la especificación, cuando el servidor devuelve este código de respuesta debe añadir una cabecera llamada *WWW-Authenticate* en la que se informe al cliente de los posibles mecanismos para verificar la identidad. El código 403 (*Forbidden)* es parecido, pero no obliga a enviar ninguna cabecera. Así por ejemplo, si intentamos acceder al banco BBVA con unas credenciales incorrectas, el servidor nos debería enviar un 401, pero sin embargo nos envía directamente un 403 (Figura 26).

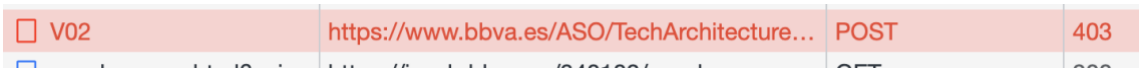


Figura 26. El 403 sustituye a menudo al 401

En la Tabla II se listan los códigos de respuesta más frecuentes.

Código	Texto explicativo	Código	Texto explicativo
100	Continue	405	Method Not Allowed
101	Switching Protocols	406	Not Acceptable
200	OK	407	Proxy Authentication Required
201	Created	408	Request Time-out
202	Accepted	409	Conflict
203	Non-Authoritative Information	410	Gone
204	No Content	411	Length Required
205	Reset Content	412	Precondition Failed
206	Partial Content	413	Payload Too Large
300	Multiple Choices	414	URI Too Long
301	Moved Permanently	415	Unsupported Media Type
302	Found	416	Range Not Satisfiable
303	See Other	417	Expectation Failed
304	Not Modified	426	Upgrade Required
305	Use Proxy	500	Internal Server Error
307	Temporary Redirect	501	Not Implemented
400	Bad Request	502	Bad Gateway
401	Unauthorized	503	Service Unavailable
402	Payment Required	504	Gateway Time-out
403	Forbidden	505	HTTP Version Not Supported
404	Not Found		

Tabla II. Códigos de respuesta http

6. Tipos de peticiones

El primer token que enviamos en una petición identifica la clase de solicitud que estamos haciendo. A este token se lo llama también *método* o *verbo,* y es la primera fuente de información respecto de lo que el cliente solicita del servidor.

La Tabla III muestra los tipos de peticiones más habituales:

[2] https://www.rfc-editor.org/rfc/rfc9110.html

Método	Descripción
GET	Solicita un recurso al servidor. Se espera que la petición no modifique el estado de los recursos.
HEAD	Envía una petición similar a GET, pero el servidor no envía el cuerpo de la respuesta, sino sólo la línea de estado y las cabeceras.
OPTIONS	Se utiliza para conocer los métodos de comunicación disponibles con cierto recurso del servidor. Estudiaremos más a fondo este método a partir del Capítulo 19.
POST	Envía al servidor cualquier cantidad de datos en el *payload*. La intención de POST es crear algún tipo de objeto nuevo en el servidor. Cuando un usuario se crea una cuenta en un sistema, lo habitual es hacerlo mediante POST.
PUT	Como POST, envía también información al servidor en el *payload* de la petición, pero con la intención de modificar algún recurso existente.
PATCH	Este método es muy similar a PUT y, de hecho, se utiliza muy poco, pues su propósito es prácticamente el mismo.
DELETE	Borra un recurso.

Tabla III. Métodos http *de uso más frecuente*

Haciendo un símil con las operaciones CRUD (*Create, Read, Update and Delete*), en bases de datos, los métodos valen para:

- GET para leer (*Read*).
- POST para crear (*Create).*
- PUT (y PATCH) para actualizar (*Update).*
- DELETE para borrar (*Delete).*

6.1. Respuestas esperadas

A pesar de lo dicho en la sección 5 y en la Tabla II, el código de respuesta a una petición puede ser el que el programador del servidor prefiera: por ejemplo, si me equivoco al escribir la url en el navegador y escribo *rregisterr* en lugar de *register*, la respuesta habitual es un código 404 (NOT FOUND), pero el programador podría escribir el servidor para que este devuelva un 403, un 500 o incluso un 200 que simulase que todo ido bien.

No obstante, se espera que las respuestas recibidas sean coherentes con la situación acontecida en el servidor por lo que, asumiendo que el cliente envía la petición con credenciales suficientes, los códigos de respuesta habituales son los siguientes, según el tipo de petición:

Método	Respuesta si OK	Respuesta si error
GET	200 (OK)	404, *Not found*: no se ha encontrado el recurso solicitado
POST	201 (Created)	409, *Conflict:* por ejemplo, si existe un recurso con el mismo *id*.
PUT/PATCH	200 (OK)	404, *Not found*: no se ha encontrado el recurso que se desea modificar. 409, *Conflict:* algo (una restricción de integridad en la base de datos, por ejemplo) impide actualizar el recurso
DELETE	200 (OK)	404, *Not found*: no se ha encontrado el recurso que se desea modificar.

Tabla IV. Códigos de respuesta habitualmente esperados

6.2. Cambio de intenciones

Pero no todo el monte es orégano, y suele (y muchas veces debe) ocurrir que una petición GET, que se supone que no tiene efectos colaterales en el servidor, los tenga: al fin y al cabo, las peticiones las envían los usuarios, pero su código lo escriben los programadores y programadoras.

De este modo, una petición a un recurso *login* que recibe un nombre y una contraseña puede enviarse mediante cualquier tipo de método, también con un GET: en la Figura 21 enviamos las credenciales de Pepe (un usuario al que conoceremos en profundidad en estas páginas) como parámetros en la url en una petición GET.

```
macariopolousaola@macbook-pro-de-macario-2 ~ % telnet 127.0.0.1 8080
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
GET /login?nombre=Pepe&pwd=pepe1234 HTTP/1.1
Host: 127.0.0.1

HTTP/1.1 200
Content-Type: ...
```

Figura 27. Envío de una petición para loguearnos mediante GET

Del mismo modo, las credenciales se podrían enviar en un POST, en un PUT o incluso en un DELETE, pues al final es el programador o el jefe de proyecto quien decide cómo se va a tratar la información.

Además, la información que enviamos no tiene por qué ir en el *payload*, sino que puede viajar en las cabeceras de la petición (con dos restricciones: los valores han de ser cadenas y no han de superar la longitud máxima que permita el servidor).

En la Figura 28, el servidor (o *backend*) ha atendido correctamente (devuelve un 200) una petición a su recurso *login*, que recibe las credenciales en dos cabeceras *Name* y *Pwd,* y además con una petición de tipo DELETE. Aunque una petición de *login* nunca va con el verbo DELETE (que implícitamente significa que queremos borrar algo), el programador puede hacer que las peticiones se envíen y se reciban como quiera.

```
macariopolousaola@macbook-pro-de-macario-2 ~ % telnet 127.0.0.1 8080
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
DELETE /login HTTP/1.1
Host: 127.0.0.1
Name: Pepe
Pwd: pepe1234

HTTP/1.1 200
```

Figura 28. Solicitamos loguearnos con DELETE y con las credenciales en las cabeceras

La moraleja de esta sección es que, si bien *http* describe con bastante claridad el propósito de cada verbo y de cada código de respuesta, a veces el programador no es completamente fiel a las descripciones dadas en los estándares. A veces esto sucede por desconocimiento, pero otras se hacen a conciencia. Por ejemplo, una petición GET (que, repetimos, se espera que no tenga efectos colaterales en el servidor) para ver una imagen es muy probable que, en el servidor, anote que esa imagen ha sido vista una vez más: es decir, sí que hay un efecto colateral en el servidor, pues se guarda en la base de datos que se ha solicitado de nuevo ese recurso. Igualmente, cuando buscamos algo en un buscador de Internet enviamos una petición GET, pero cuando el servidor la procesa incrementa el número de búsquedas de ese término (Figura 29).

Figura 29. Términos más buscados en España en Google en 2022

7. Cookies

No lo hemos dicho, pero *http* es un protocolo sin estado (*stateless),* lo que significa que el cliente envía una *request* r_1, el servidor le devuelve la respuesta y, si el cliente envía una nueva *request* r_2, el servidor ya no sabe que esta segunda petición viene del mismo cliente que envió r_1. El curioso y desocupado lector[3] dirá que esto es mentira, porque uno introduce sus credenciales en su banco y, en las subsiguientes peticiones, el sistema sabe que estas proceden del mismo lugar que la primera. Y la curiosa y desocupada lectora[3] lleva parcialmente razón, pero sucede que las *cookies* son un mecanismo especial que permite que el servidor sepa de dónde procede una petición.

En efecto, una *cookie* es una cabecera algo especial que el servidor crea y envía al cliente. El nombre que el servidor le da a esta cabecera es *Set-Cookie,* y su particularidad es que las cookies que nos envía un servidor se envían nuevamente a ese mismo servidor cada vez que le enviemos una nueva petición.

[3] https://cvc.cervantes.es/literatura/clasicos/quijote/edicion/parte1/prologo/default.htm

Para conocernos durante una sesión de navegación (en el banco, por ejemplo), el servidor crea una cookie especial de sesión, a la que asigna un valor aleatorio ("1234", por ejemplo). Cuando el usuario se identifica correctamente en su banco, el navegador recibe esta cookie y la guarda en algún medio de almacenamiento local que el propio navegador implemente. Cuando, acto seguido, ese mismo usuario solicita consultar el saldo de su cuenta, envía una petición con una cabecera *Cookie* con ese valor aleatorio ("1234"), que el servidor utiliza para saber que esta *request* r_2 proviene del mismo *user-agent* que envió la r_1.

7.1. Cookies de terceros

Además de para mantener la sesión, un uso habitual de las cookies es la gestión de publicidad personalizada, que se ilustra en la Figura 30:

1) Un usuario visita el agregador de noticias de *meneame.net*. Este sitio utiliza cookies para insertar publicidad personalizada.
2) El mismo usuario abandona *menéame* y entra en *atrapalo.com*, en donde busca vuelos de Madrid a nueva York.
3) El mismo usuario abandona la web de *atrápalo* y regresa a *menéame*. Como se ve en el lado derecho de la figura, el anuncio ahora está personalizado para ese usuario.

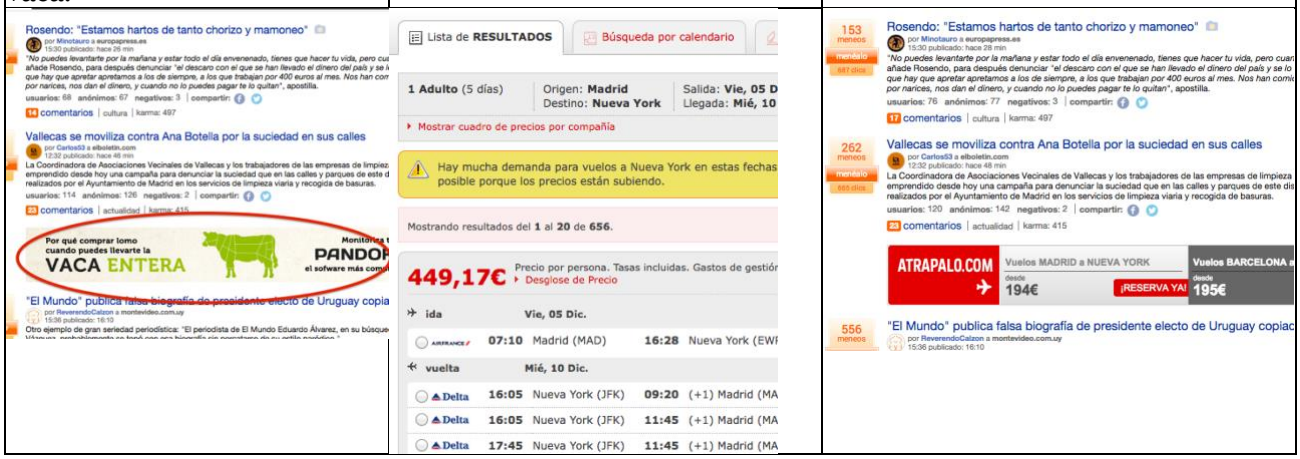
Momento 1: el usuario se conecta a *meneame.net*. Le aparece un anuncio con una vaca.	**Momento 2:** el usuario busca un vuelo Madrid-Nueva York en *atrapalo.com*	**Momento 3:** el usuario regresa a *meneame.net*.
		

Figura 30. Compartición de cookies entre diversos servidores

Para que el escenario que se acaba de describir suceda, intervienen al menos tres servidores (Figura 31):

1) El usuario escribe en su navegador la url de *menéame,* que le devuelve texto (mensaje 2) en HTML que, ya sabemos por la Figura 22, puede incluir código que el navegador interprete para lanzar otras peticiones.
2) Al interpretar ese HTML, el navegador envía la petición del mensaje 3 al servidor de publicidad (*ads* en este ejemplo).
3) *ads* lee las posibles cookies que el cliente le haya podido enviar. En este escenario es la primera vez que el usuario se conecta, por lo que el servidor le crea una cookie con *id=1234*, la cual le envía junto al anuncio aleatorio de la vaca en el mensaje 6. El navegador almacena esta cookie, y volverá a enviarla a *ads* en las nuevas peticiones que le envíe.

4) En el mensaje 5, el mismo usuario busca en *atrápalo* el vuelo. Este servidor le devuelve su respuesta, en la que se incluye una petición al mismo servidor de publicidad que utiliza *menéame*.
5) Cuando el usuario recibe la respuesta, su navegador envía sin intervención del usuario el mensaje 7, en el que le dice que quien tenga asignada la cookie "1234" ha buscado un vuelo Madrid-Nueva York. Obviamente, *ads* toma buena nota de esto en el mensaje 8.
6) Cuando el usuario regresa a *menéame,* el mensaje 12 replica la petición del mensaje 3, pero ahora con la cookie "1234". *ads* la procesa y dice «Anda, pero si este usuario buscó vuelos de Madrid a Nueva York», y entonces le prepara y envía un anuncio en el mensaje 13.

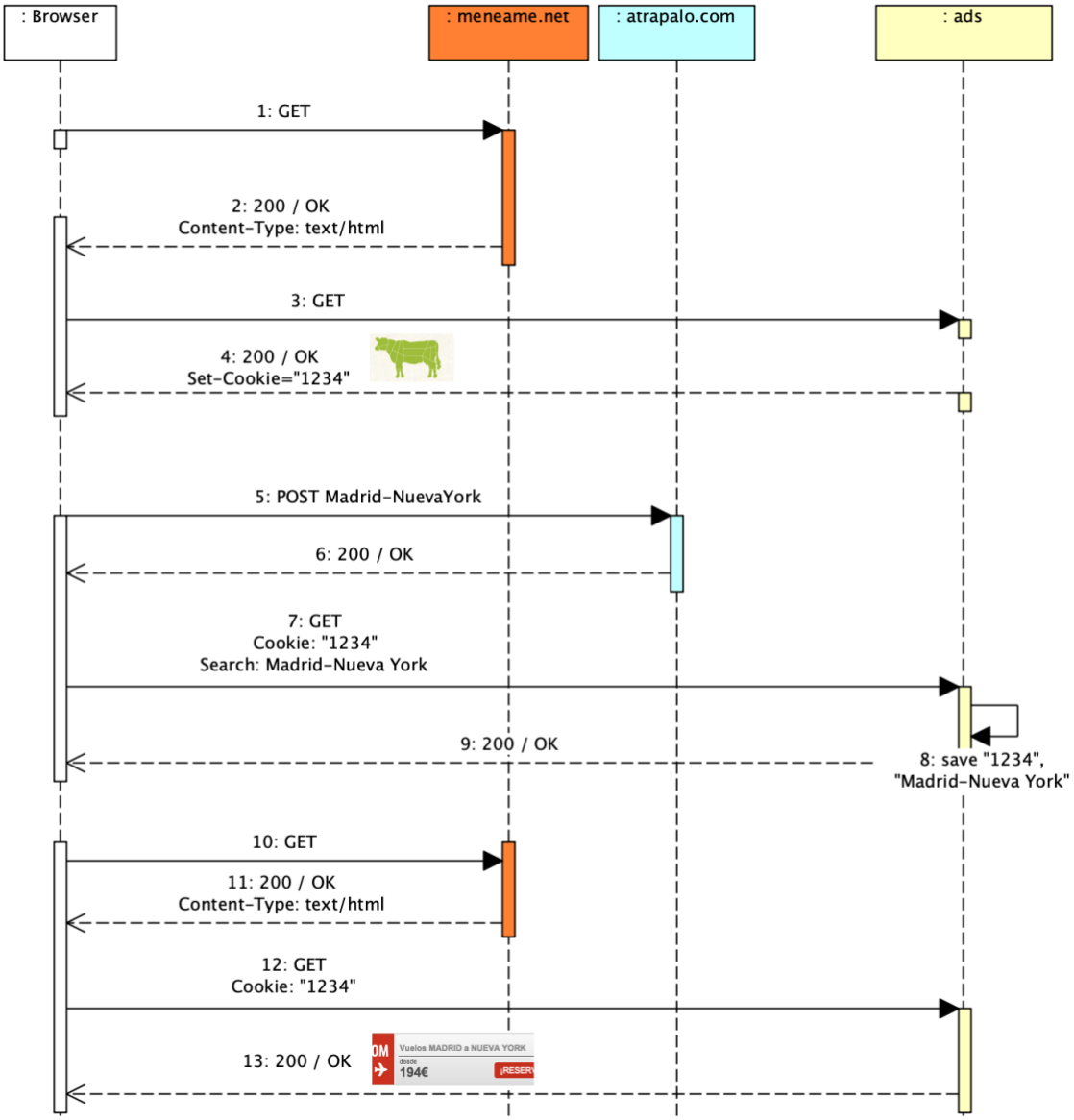


Figura 31. Un escenario con cookies de terceros

8. Protocolo *ws*

A pesar de la argucia de la cookie de sesión, el servidor web no conoce directamente a sus clientes: el truco de las cookies le sirve al servidor para saber si quien le habla le habló ya en algún momento anterior; pero el servidor no puede

decirle cosas a los clientes sin que estos se lo pidan: dicho de otro modo, el servidor no puede enviar *responses* si no hay *requets*.

Para habilitar esta posibilidad se creó el protocolo *ws* (*WebSocket*), que permite que un servidor envíe mensajes a los clientes siempre que estos se hayan anunciado con anterioridad.

ws se utiliza, por ejemplo, en los sistemas de chat: si el usuario Pepe quiere saludar a Ana mediante *ws*, ambos deben: (1) disponer de un WebSocket en su cliente y (2) haberse conectado al mismo WebSocket en el servidor. Si es el caso, Pepe le dirá al servidor que quiere saludar a Ana; el servidor buscará al *user-agent* de Ana y, sin que este haya hecho otra cosa más que anunciarse, el servidor podrá localizarlo y reenviarle el mensaje.

Así como hay partes y contrapartes de especificación y abierta y completamente estandarizadas para las *request* y las *responses* de *http,* también existe lo mismo para los WebSockets y su protocolo *ws*. Los trabajaremos en profundidad a partir del Capítulo 21.

TERCERA PARTE: DESARROLLO DE UN SISTEMA WEB SENCILLO

Capítulo 3. Descripción del sistema

1. Descripción de la versión 1 del proyecto

En esta primera versión del proyecto, se nos ha pedido desarrollar una aplicación web que permita a sus usuarios jugar individualmente a juegos de tablero.

De momento se nos ha planteado el desarrollo de dos juegos: el *Word race* y el *Order the numbers:*

1) En el primero, el jugador recibe las letras desordenadas de una palabra y debe colocarlas hasta encontrar la palabra. Por ejemplo, recibe SACOT y, si pulsa en dos letras distintas, estas se intercambian: si pulsa en la S y la C, la cadena pasa a ser CASOT; si luego pulsa en A y S, obtiene CSAOT; de este modo, va intercambiando letras hasta adivinar cualquiera de las palabras que se puedan formar con esas letras (en este ejemplo serían válidas CASTO, TOCAS y TACOS). Cuando las letras que ha ordenado forman una palabra válida, le llega una nueva ristra de letras desordenadas.
   En este juego, es el *backend* quien escoge y desordena la palabra con la que debe jugar el usuario. Además, cada vez que el usuario hace un intercambio de letras, la secuencia de caracteres que se ha formado se envía al servidor, que comprueba si la tiene en su diccionario.
2) En el *Order the numbers,* el usuario recibe desordenados los números del 1 al 8 en un cuadrado de 3x3 casillas, estando vacía la última casilla. El objetivo es colocar los números ordenados del 1 al 8: si pulsa en un número adyacente (en horizontal o en vertical) a la casilla vacía, el número y la casilla vacía se intercambian. Cuando los consigue ordenar crecientemente del 1 al 8, se le propone un nuevo cuadrado con números desordenados.
   En este juego, el propio *front-end* puede generar la secuencia desordenada de números.

De momento, en esta primera versión no se requiere que los usuarios tengan cuenta en el sistema: es decir, no se requiere registro.

2. Vista arquitectónica de alto nivel

El proyecto se nos ha encargado como aplicación web, por lo que podemos entenderlo como dos subproyectos: el *front-end* por un lado, y el *backend* por otro. Evidentemente, ambos subsistemas deben definir un formato con el que intercambiarse información.

La Figura 32 muestra una primera vista del sistema a muy alto nivel: realmente, esta figura sirve para describir prácticamente cualquier sistema cliente-servidor. No hay que olvidar que una aplicación web tiene una arquitectura cliente-servidor, con el cliente implementado en el *front-end,* y el servidor en el *back,* comunicándose ambos con los protocolos de Internet, principalmente *http* y *ws*.

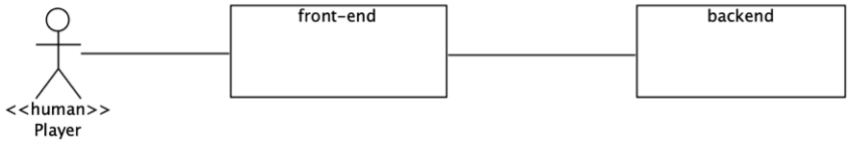


Figura 32. Entendemos el proyecto como dos subproyectos

Para uno de los juegos, se nos ha dicho que el *backend* debe seleccionar y desordenar la palabra, y también comprobar con un diccionario si la secuencia creada por el jugador es correcta. Podemos asumir que este diccionario residirá en una base de datos servida por un Sistema Gestor de Bases de Datos (Figura 33). Quizá más adelante se decida que el diccionario resida en un fichero o en un conjunto de ficheros: de momento reservamos para este elemento una posición distinguida en esta vista de alto nivel.

Figura 33. El servidor manejará probablemente una base de datos

Tanto el *front* como el *back* deben ofrecer diferentes funcionalidades según el juego al que se esté jugando:

1) El lado servidor no ofrece nada para el *Order the numbers,* pues toda su lógica reside en el lado cliente.
2) Para el *Word race,* sin embargo, el lado servidor debe ofrecer al cliente, al menos, una funcionalidad para entregarle una palabra desordenada y otra para comprobar la corrección de la palabra recibida.

Podemos enriquecer la figura anterior añadiendo casos de uso en los dos subsistemas:

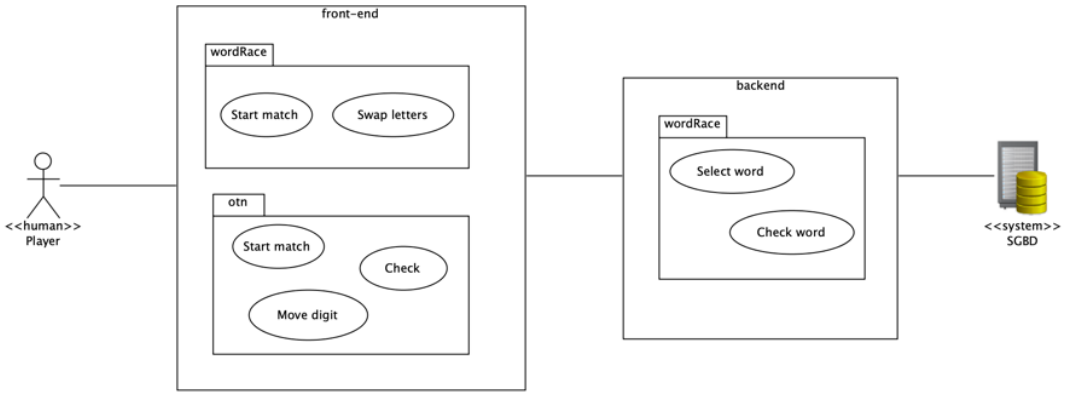


Figura 34. Adición de casos de uso en ambos lados

Además, podemos establecer relaciones entre los actores y los casos de uso, y entre los propios casos de uso de los dos subsistemas. En la Figura 35, se ha sustituido la línea que conecta al actor *Player* con el subsistema *front-end* por asociaciones dirigidas (flechas), y lo mismo se ha hecho con las que conectan ambos subsistemas y con la que une el *back* con la base de datos.

En los diagramas de casos de uso, una asociación dirigida denota qué elemento inicia la comunicación, independientemente de que esta pueda ser luego bidireccional. Así, el jugador arranca la funcionalidad de empezar una

partida de cualquiera de los dos juegos en su *front-end*; igualmente, podemos entender que, cuando el jugador inicia una partida de *Word race,* se ejecuta la funcionalidad *Select word* en el *backend,* y que cuando intercambia las letras en el lado cliente, se envía un mensaje al servidor para que compruebe si la palabra es correcta. En el *Order the numbers,* sin embargo, no hay comunicación con el *backend,* pues todo él corre en el lado cliente. Sí que hay una relación de inclusión entre los casos de uso *Move digit* y *Check:* el caso de uso *Check* comprueba si los números están correctamente ordenados. Es una funcionalidad distinguida y por eso decidimos separarla en un caso de uso adicional.

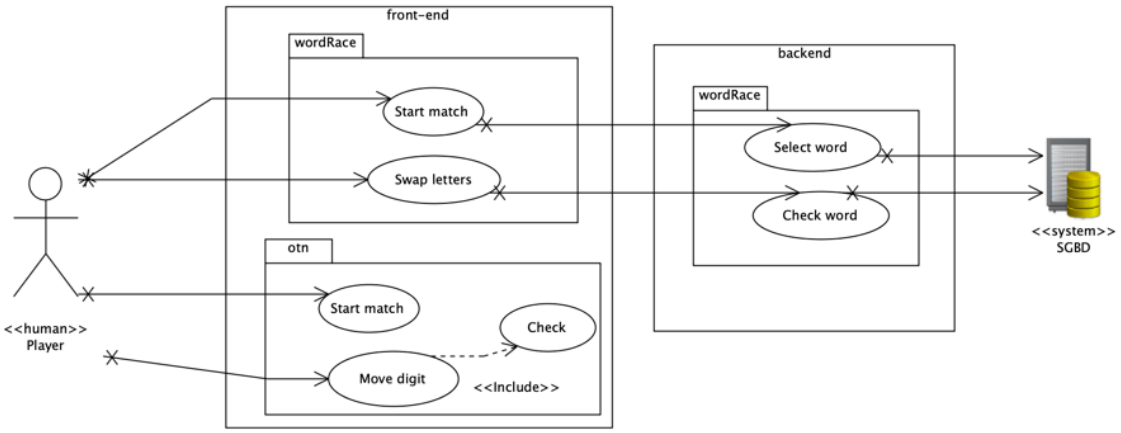


Figura 35. Relaciones entre actores y casos de uso

Capítulo 4. Desarrollo de *Order the numbers*

1. Introducción a Angular

Angular es un *framework* para el desarrollo del lado cliente de aplicaciones web. Utiliza TypeScript como lenguaje de script. TypeScript es algo así como un "dialecto" de JavaScript, pero con control de tipos. Al final, un programa escrito en TypeScript se traduce a JavaScript, de manera que los motores de ejecución de los navegadores pueden interpretarlo y ejecutarlo correctamente.

2. Creación de un proyecto básico

Como IDE (*Integrated Development Environment:* Entorno Integrado de Desarrollo) utilizaremos *Visual Studio Code* (VSC), un potentísimo sistema de Microsoft que permite desarrollar aplicaciones en cualquier lenguaje, conectar a bases de datos o a otros tipos de sistemas gracias a la multitud de *plugins* de los que dispone.

Cuando arrancamos VSC, encontraremos en su menú *Terminal* una opción *New Terminal,* que nos abrirá una terminal integrada en el propio IDE (véase la parte inferior de la Figura 36):

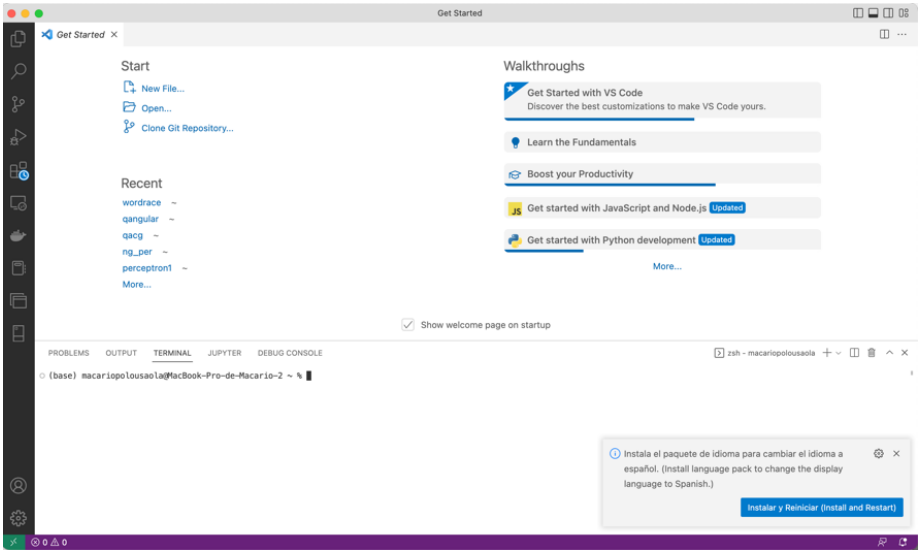


Figura 36. Apertura de una terminal en Visual Studio Code

Para crear el proyecto, escribiremos en la terminal el comando *ng new fejuegos:*

- *ng* viene de aNGular.
- *fejuegos* es el nombre de nuestro proyecto, y resume el hecho de que aquí vamos a colocar todo el *front-end de los juegos:* más adelante iremos añadiendo el lado cliente del *Word race*.

Angular nos pregunta si queremos añadir un *routing* (aceptamos la opción por defecto: *Yes)* y nos pide el tipo de hoja de estilo (aceptamos *CSS)*. Entonces, nos genera una estructura de carpetas ciertamente compleja, que nos va resumiendo en la salida de la consola (Figura 37).

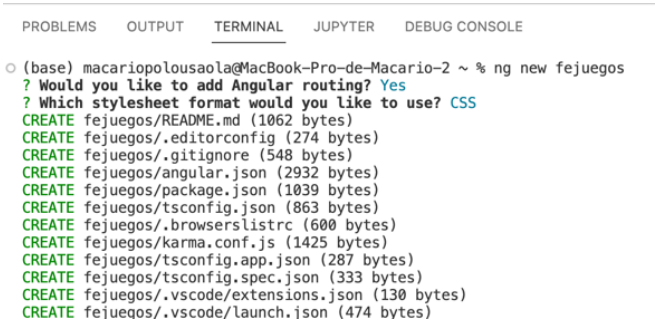


Figura 37. Creación de un proyecto en Angular

Cuando ha terminado, podemos abrir la carpeta *fejuegos* que, en la figura anterior, se ha creado en el directorio del usuario (*home):* elegimos *File, Open folder* y buscamos la carpeta *fejuegos*. Como vemos en la Figura 38, Angular nos ha creado varios ficheros y varias carpetas.

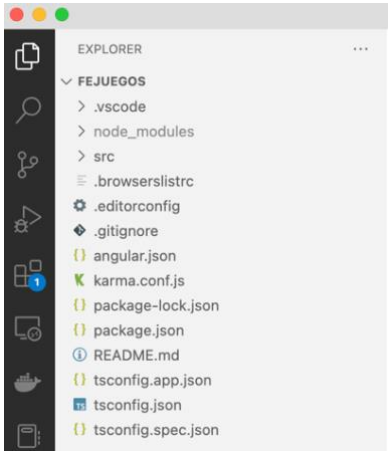


Figura 38. Estructura creada para el proyecto fejuegos

De momento, nos interesa sobre todo la carpeta *src ,* en la que hay también una subcarpeta *app,* dentro de la cual se incluye el código que, de momento, nos interesa. También en esta carpeta *app* colocaremos el código que nosotros vayamos construyendo.

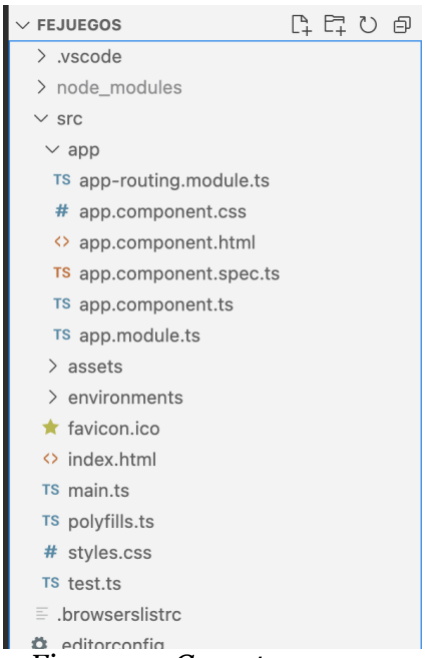


Figura 39. Carpeta src

Como vemos en la figura anterior, Angular nos ha generado dentro de *app,* entre otros, un fichero *app-component.html* y otro *app.component.ts*. Hay más, pero no nos interesan de momento.

La aplicación que nos ha generado Angular sigue un patrón MVC (Modelo-Vista-Controlador):

- La vista es el fichero con extensión *html.*
- El modelo es el que acaba en *.ts* (extensión de los ficheros TypeScript).
- El controlador es el propio Angular.

Por entendernos sin perder generalidad, podemos asumir que, con el patrón MVC, si algo cambia en la vista, el modelo se entera; y, si algo cambia en el modelo, la vista también se entera. Gráficamente, podemos dibujarlo del siguiente modo:

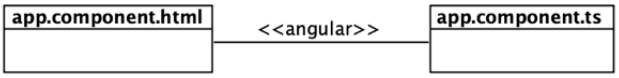


Figura 40. MVC con Angular

El usuario interactúa con la vista (el fichero *html*) y, cuando hace un cambio (por ejemplo, pulsa un botón o escribe en una caja de texto), este es inmediatamente transmitido al modelo (el fichero *ts)*, que ejecuta la operación que corresponda.

Del mismo modo, si el modelo reciba un mensaje desde el *backend,* tal vez la vista deba actualizarse de alguna manera: por ejemplo, si en este sistema tuviésemos implementado un chat, podría ocurrir que el fichero *ts* recibiese un mensaje de otro usuario: en este caso, Angular (que actúa de controlador, y así lo denotamos con el estereotipo *«angular»),* percibe el cambio en el modelo y lo transmite a la vista.

A estas relaciones entre vista y modelo y entre modelo y vista, que son instantáneas y provocan cambios de estado, se las llama *binding*. Es posible que

usemos este término en estas páginas, o adaptaciones al castellano mediante el verbo inventado “bindear”.

2.1. Ejecución

El proyecto está en este momento listo para ser ejecutado. Para hacerlo, abrimos una nueva terminal (es posible que VSC nos la haya cerrado) y escribimos el comando *ng serve,* como en la Figura 41. Nótese que en esta terminal, estamos dentro de la carpeta del proyecto.

Figura 41. Ejecución del proyecto

Angular incluye un pequeño servidor web que, por defecto, escucha en el puerto 4200. Para dirigirnos a él, abrimos un navegador y escribimos *localhost:4200* en la barra de direcciones. En nuestro caso, y así lo aconsejamos cuando estamos desarrollando, lo abriremos en modo incógnito, de manera que no dispongamos ni de cookies ni de información almacenada en la caché, que podrían enturbiar y molestarnos.

Si todo va bien, y así debería ser porque no hemos hecho ningún cambio respecto del código generador, el aspecto del navegador debería ser algo así como:

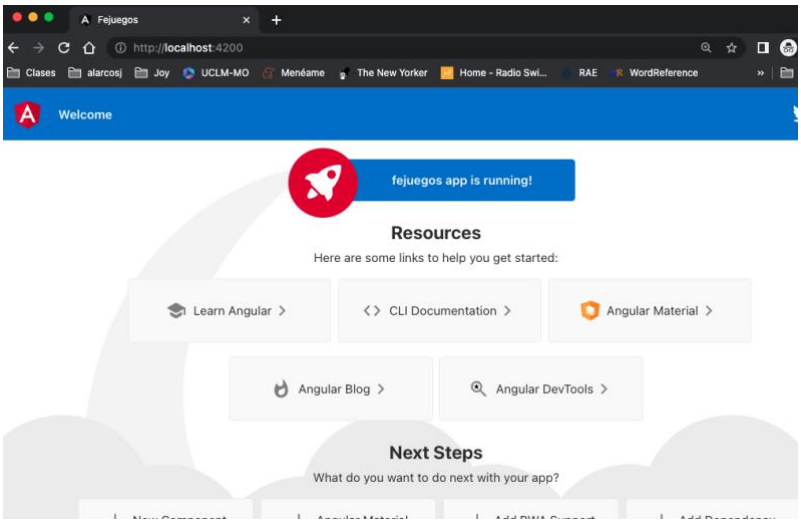


Figura 42. Ejecución del proyecto recién creado

Lo que vemos en la Figura 42 es el contenido de la página *app.component.html*. Si la abrimos en VSC, veremos que sus contenidos son:

1) Unas líneas de comentario.
2) Unas cuantas instrucciones de estilo (entre *<style>* y *</style>).*
3) El código *html* propiamente dicho que muestra el contenido y que tiene:
   a. Un *div* con *class=toolbar* (la franja superior azul).

b. Un *div* con *class=content*, en el que aparece la figura del cohete y el texto *qmesa app is running!*
c. Un *h2* con el texto "Resources", debajo del cual aparece un *div class="card-container",* en el que aparecen varios *anchor* (etiqueta <*a*>) con los textos "Learn Angular", "CLI documentation", etcétera.
d. Un *h2* con el texto "Next Steps" y, debajo, otro *div class="card-container"* con varios *button*.
e. Un *div class="terminal",* que muestra esa especie de figura con una terminal.
f. Otro *div class="container"* con varios logos.
g. Una sección *footer*.
h. Un *svg* con *id="clouds"*.
i. Unos comentarios.
j. Una etiqueta <*router-outlet*>.

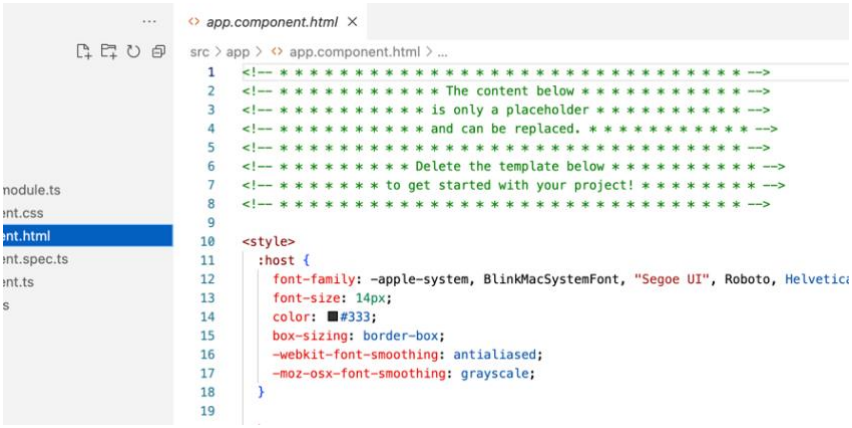


Figura 43. Código generado para app.component.html

2.2. Primer bindeo

"Bindeo" de "bindear", ya se ha advertido que castellanizaríamos el término *binding*, que podemos traducir como "enlazamiento", "vinculación" o algo así.

El caso es que, hacía la línea 344 del código de la vista (*app.component.html)* nos encontramos el siguiente código:

```
343
344     <span>{{ title }} app is running!</span>
345
```

Figura 44. El campo title *del modelo está bindeado en la vista*

Ese elemento *{{ title }}* hace en realidad referencia al campo *title* del modelo asociado. En efecto:

1) En el modelo (fichero *ts,* cuyo código en TypeScript se ofrece en la parte superior izquierda de la Figura 45) existe un campo *title* cuyo valor es "fejuegos".
2) En la vista (fichero *html,* del cual se muestra la línea 344) hay un *binding* a ese campo.
3) Lo que el usuario ve es el valor de ese campo seguido del texto "app is running".

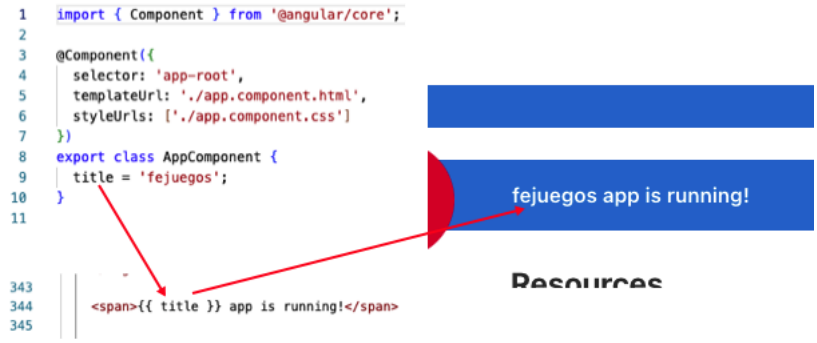


Figura 45. Binding entre app.component.html *(vista)* y *app.component.ts* (modelo)

2.3. Eliminación de código

El código de la vista que nos ha generador Angular en *app.component.html* está muy bien para hacernos una idea de las posibilidades que nos ofrece, pero vamos a limpiarlo para adecuarlo a lo que deseamos.

Eliminaremos las primeras líneas, que solo tienen comentarios, y también desde la línea 305 (después de la etiqueta de cierre *</style>*) hasta antes de la última línea que ha añadido Angular. Es decir, nos quedamos así:

```
291        visibility: hidden;
292      }
293    }
294  </style>
295
296  <router-outlet></router-outlet>
297
```

Figura 46. Eliminación de código

Si, tras borrar todo ese código, guardamos el cambio en VSC, el navegador se recarga automáticamente y nos mostrará una página en blanco.

3. Componentes en Angular

Una aplicación Angular se estructura en componentes. Un componente está formado por una vista (*html)*, un modelo (*ts)* y, quizás, una hoja de estilo (*css)*. La aplicación que se nos ha generado al escribir el comando *ng new fejuegos* consta de un único componente formado por la vista, su modelo y una hoja de estilo llamada *app.component.css*.

En la Figura 47 se muestra el código del modelo generado para el componente *app.component.ts:* es decir, el fragmento del componente que, "de alguna manera", Angular sabe que está bindeado a la vista *app.component.html*.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'fejuegos';
}
```

Figura 47. Código de app.component.ts

Ese "de alguna manera" que hemos entrecomillado en el párrafo anterior viene determinado porque la clase *AppComponent* (véase el propio nombre de la

clase en la Figura 47: *expor class AppComponent)* está anotada con *@Component*. Una anotación en TypeScript es muy semejante a un estereotipo en UML. La anotación *@Component* requiere que la clase a la que anota tenga tres valores (que, de nuevo haciendo un símil con UML, se corresponderían con los *tagged values* de los estereotipos):

1) Un *selector* que, en este caso, es *app-root*.
2) Una vista (*app.component.html).*
3) Una hoja de estilo (*app.component.ts).*

En la Figura 48 se resume gráficamente la estructura de este componente. Obsérvese que: (1) ya no hacemos referencia al fichero *app.component.ts,* sino a la clase definida en ese fichero (*AppComponent* en este caso), y que (2) la clase es un *Component* de los que Angular define en el paquete *angular/core*.

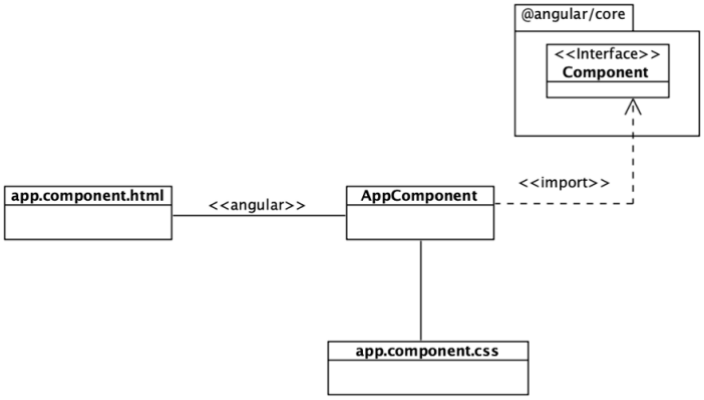


Figura 48. El componente app *que nos ha generado Angular*

Otra forma de representar el diagrama anterior se muestra en la Figura 49, en el que hemos estereotipado con *Component* la clase *AppComponent*. Esta notación es más compacta y proporciona tanta información como la anterior.

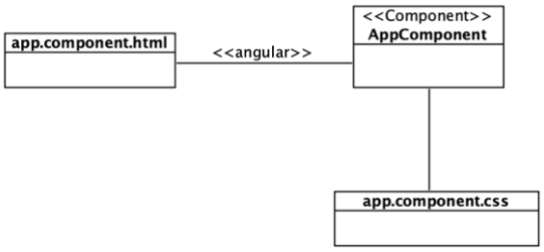


Figura 49. El componente app *generado por Angular, ahora estereotipado*

4. Creación de un componente para el juego *Order the numbers*

De momento nos gustaría que, al conectarnos a *localhost:4200,* el navegador nos mostrara en la vista una cuadrícula de 3x3 con 8 dígitos desordenados y, como se decía en la descripción de este juego (página 49), la última casilla en blanco.

Generaremos para esto un componente al que, por abreviar, llamaremos *otn,* siglas de *Order the numbers*. Para ello, detenemos el servidor que está sirviendo Angular en el puerto 4200 pulsando Control-C y escribimos el comando *ng generate component otn:*

```
✔ Compiled successfully.
^C
(base) macariopolousaola@MacBook-Pro-de-Macario-2 fejuegos % ng generate component otn
CREATE src/app/otn/otn.component.css (0 bytes)
CREATE src/app/otn/otn.component.html (18 bytes)
CREATE src/app/otn/otn.component.spec.ts (578 bytes)
CREATE src/app/otn/otn.component.ts (263 bytes)
UPDATE src/app/app.module.ts (463 bytes)
(base) macariopolousaola@MacBook-Pro-de-Macario-2 fejuegos %
```

Figura 50. Creación de un componente para Order the numbers

Dentro de *src/app,* Angular nos ha generado una carpeta *otn* con cuatro ficheros (Figura 51): el primero es la hoja de estilo; el segundo, la vista; el tercero (*otn.component.spec.ts)* se utiliza para la fase de pruebas y no lo trataremos en este libro, por lo que podemos eliminarlo; en el cuarto implementaremos el modelo que conectaremos a la vista y será el punto de acceso a la lógica del juego: recordemos que Angular, de alguna manera, y si somos fieles a su principios, nos impone o nos invita a usar el patrón MVC, por lo que en la vista solo han de residir responsabilidades relacionadas con mostrar los datos; estos, como ya vamos sabiendo, se calculan en el modelo asociado.

Figura 51. Ficheros generados en la carpeta otn

En la Figura 52 se muestra el código de la clase *OtnComponent,* que reside en el fichero *src/app/otn/otn.component.ts*. Como vemos, el valor del campo *selector* de su anotación *@Component* es *app-otn*.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-otn',
  templateUrl: './otn.component.html',
  styleUrls: ['./otn.component.css']
})
export class OtnComponent implements OnInit {

  constructor() { }

  ngOnInit(): void {
  }
}
```

Figura 52. Código de la clase OtnComponent

Bien, pues haremos uso de ese valor del campo *selector* para que este componente se incluya en la página principal de la aplicación que estamos sirviendo en *localhost:4200*.

Lo único que necesitamos hacer es incluir una etiqueta *<app-otn></app-otn>* en la vista del componente que la aplicación carga cuando arranca: en la Figura 46 mostrábamos cómo quedaba el código del componente de arranque tras eliminar el código sobrante. Ahora, añadimos la citada etiqueta, que es válida para Angular puesto que se ha generado como un componente:

```
294   </style>
295
296   <app-otn></app-otn>
297   <router-outlet></router-outlet>
298
```

Figura 53. Inclusión del componente recién creado en el componente de arranque

Si nos fijamos en el código de *otn.component.html* (es decir, en la vista del componente que acabamos de crear), vemos que solo incluye la línea * otn works! *. Si lanzamos el proyecto mediante *ng serve* y recargamos el navegador, veremos simplemente que *Otn funciona*.

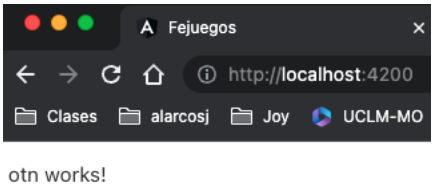


Figura 54. ¡Otn funciona!

4.1. Microarquitectura de *Order the numbers*

Entendemos por "microarquitectura" el diseño arquitectónico que realizamos para una pequeña parte del sistema. Mantener patrones microarquitectónicos es importante en equipos en los que trabaja mucha gente, porque así cada uno podrá saber en qué parte del sistema se encuentra cada responsabilidad.

En este caso, consideraremos que los dos elementos principales que tenemos en el componente (la vista y el modelo: respectivamente el *html* y el *ts*) constituyen el punto de acceso a la lógica del juego: podríamos, desde luego, implementar con JavaScript toda la lógica del juego en la propia vista (o sea, incrustar en el *html* las funciones JavaScript necesarias para que el usuario pudiese jugar), pero es fácilmente entendible, como ingenieros de software, que esta no es una solución "elegante"; podríamos también implementar con TypeScript (que, ya sabemos, posteriormente, y por unos mecanismos que no nos interesan demasiado, se traducen a JavaScript) el juego en la clase *OtnComponent* (Figura 52), pero entenderemos, también con nuestro buen criterio y buen gusto que nos da la Ingeniería de Software, que tampoco es una solución profesionalmente idónea.

De hecho, asumiremos que el jefe de proyecto nos impone la utilización del componente (el *html* y el *ts*) como una mera interfaz entre el usuario y la lógica de dominio. De alguna manera, la figura siguiente resume esta orden del jefe: el jugador interactúa con la vista (*otn.component.html*) y los eventos que produce se pasan al modelo (*OtnComponent,* que reside en el fichero *otn.component.ts)*; el modelo, a su vez, interactúa con la clase *Otn*, que ha implementar la lógica del juego.

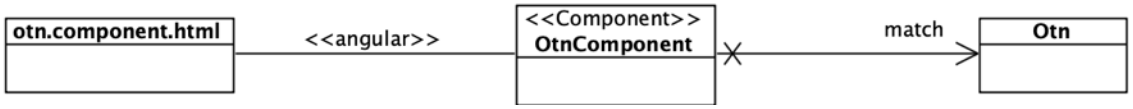


Figura 55. La lógica del juego residirá en una clase aparte

Tal vez otro jefe de proyecto nos hubiera permitido incluir todo en *OtnComponent,* pero hemos tenido la mala suerte de estar en un libro de

Arquitectura, Diseño y Programación y esta solución no se nos permite: la clase *Otn* debe implementar toda la lógica del juego y, de alguna manera, ha de conseguir que los cambios se transmitan a su modelo y desde aquí a la vista asociada en la etiqueta *selector* del componente.

4.2. Creación de la clase *Otn*

La clase *Otn* representa realmente una partida (de ahí que, en la Figura 55, hayamos anotado con *match,* "partida", el nombre del campo) de un jugador. Según la imposición que nos ha hecho nuestro jefe de proyecto, en esta clase debe residir toda la lógica necesaria para poder jugar a *Order the numbers*.

Para crear esta clase, y dentro de Visual Studio Code, damos con el botón derecho en *src/app/otn* y elegimos *New File:*

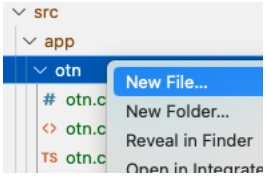


Figura 56. Creación de un fichero en una carpeta

Le damos como nombre *otn.ts* y VSC mostrará un editor en blanco. Una partida de este juego tal y como se ha descrito tiene, como campos, una matriz de 3x3 casillas en las que hay números del 1 al 8 más una casilla vacía. La descripción inicial de la clase puede ser la que se muestra en la Figura 57, si bien el compilador de TypeScript integrado en VSC nos advierte de que hay un error, pues estamos declarando el campo *numbers* como un array bidimensional de objetos de clase *number* (un tipo de TypeScript) sin que estemos asignándole a este campo ningún valor inicial.

```
src > app > otn > TS otn.ts > ...
1   export class Otn {
2       numbers : number[][]
3   }
```

Figura 57. Creación de la clase Otn

Para evitar este error, o bien asignamos los valores directamente en la misma línea en que declaramos la variable (Figura 58)...

```
src > app > otn > TS otn.ts > ...
1   export class Otn {
2       numbers : number[][] = [
3           [-1, -1, -1],
4           [-1, -1, -1],
5           [-1, -1, -1]
6       ]
7   }
8
```

Figura 58. Inicialización de una variable en su misma declaración

...o bien los asignamos en el constructor de la clase. Esto será lo que hagamos, puesto que debemos descolocar los números. Para colocarlos descolocados en la matriz, llenaremos un vector unidimensional con los números del 1 al 8, intercambiaremos sus posiciones varias veces al azar y, luego, los colocaremos en la matriz. En otras palabras (Figura 59):

```
export class Otn {
	numbers : number[][]

	constructor() {
		let values = [1, 2, 3, 4, 5, 6, 7, 8]

		let a, b, aux
		for (let i=0; i<100; i++) {
			a = Math.floor(Math.random()*values.length)
			b = Math.floor(Math.random()*values.length)
			aux = values[a]
			values[a] = values[b]
			values[b] = aux
		}

		this.numbers = []
		let cont = 0
		for (let i=0; i<3; i++) {
			this.numbers[i] = []
			for (let j=0; j<3; j++) {
				if (cont<9)
					this.numbers[i].push(values[cont++])
			}
		}
		console.log(this.numbers)
	}
}
```

Figura 59. Creación del tablero de Order the numbers

En la figura anterior, generamos un tablero desordenado y, en su última línea, lo mostramos para comprobar si ha quedado bien. Para que, al cargar la página en el navegador, el tablero se cree, debemos instanciar el campo *match* que apuntábamos en la Figura 55 y que resaltamos en la Figura 60.

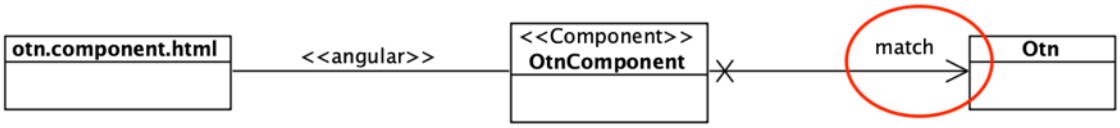


Figura 60. Tenemos que instanciar el objeto match

Para conseguir dicha instanciación y ser fieles a la Figura 60, debemos declarar en *OtnComponent* un campo *match* de tipo *Otn* y, además, llamar a su constructor en el propio constructor de *OtnComponent*. Es decir:

```
import { Component, OnInit } from '@angular/core';
import { Otn } from './otn';

@Component({
  selector: 'app-otn',
  templateUrl: './otn.component.html',
  styleUrls: ['./otn.component.css']
})
export class OtnComponent implements OnInit {

  match : Otn

  constructor() {
    this.match = new Otn()
  }

  ngOnInit(): void {
  }
}
```

Figura 61. Declaración del campo match *en el componente, e instanciación en su constructor*

Si el servidor sigue activo, el navegador se habrá recargado; si no, escribimos nuevamente el comando *ng serve* y nos dirigimos con el navegador a *localhost:4200*. Si no hacemos nada no veremos nada distinto de lo que ya veíamos en la Figura 54; para ver si, en efecto, se muestra por consola el tablero desordenado, mostramos las opciones de desarrollo en el navegador que estemos utilizando. En Google Chrome, pulsamos con el botón derecho en la página que se nos esté mostrando y elegimos "Inspeccionar" (Figura 62).

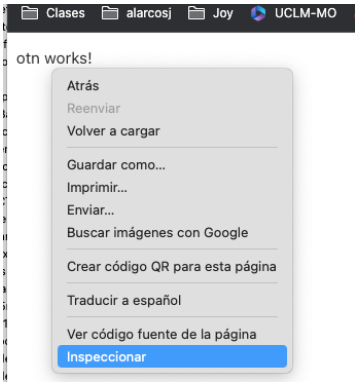


Figura 62. Opciones de desarrollo en Chrome

Si, en estas opciones, elegimos la solapa *Sources* y mostramos la consola de JavaScript (habiendo elegido *Sources,* la consola aparece y desaparece pulsando la tecla ESC), veremos lo siguiente:

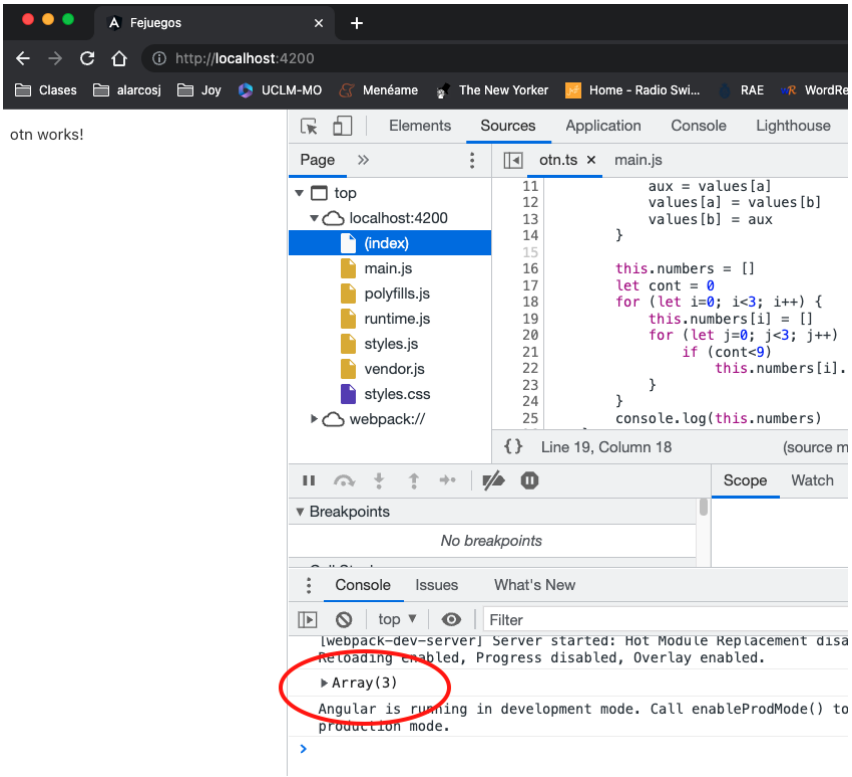


Figura 63. Consola de JavaScript

Nótese, en la zona de la consola, ese elemento que resaltamos con un óvalo rojo en el que pone *Array(3):* podemos expandirlo y veremos que, como esperábamos, el tablero del *Order the numbers* está formado por 3 filas y 3

columnas con los números del 1 al 8 desordenados, y en cuya última celda no hay valor (realmente, hay un valor sin definir, *undefined):*

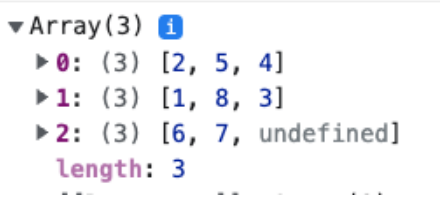


Figura 64. Un tablero desordenado que se muestra por consola

4.3. Conexión visual de *Otn* con la vista

Reproducimos nuevamente una figura anterior para ilustrar el hecho de que la vista y el modelo se conocen gracias al control que establece Angular, con lo que todo lo que cambie en el modelo (*OtnComponent)* puede ser conocido por la vista (el *html).*

Como vemos en la Figura 65, el *html* conoce al *ts* (clase *OtnComponent)*, y uno de los campos de *OtnComponent* es el objeto *match,* de tipo *Otn*. Por tanto, desde *otn.component.html* debemos ser capaces de algún modo de observar el estado de *match,* puesto que este (*match)* es un miembro del modelo (*OtnComponent)* asociado a la vista.

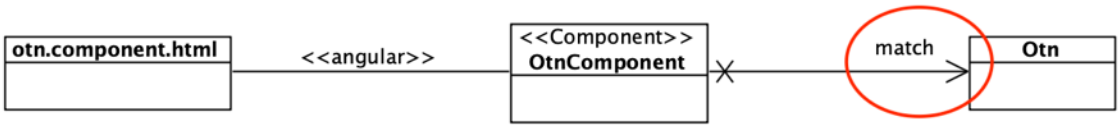


Figura 65. Conexión entre los diferentes elementos

En el *html* podemos observar cualquier cambio que se produzca en el modelo asociado, entre los que se encuentran los cambios en el objeto *match*. Claro, que aquí debemos traducirlos a instrucciones *html* que el navegador sea capaz de interpretar y de "renderizar[4]".

Desde el *html* podemos hacer referencia a cualquier elemento del modelo. Por ir probando, desde el *html* haremos referencia al elemento *match*. Para ello, modificamos el código de *otn.component.html* para que se muestre el valor de *match*. En la Figura 66 hemos añadido un elemento *div* en cuyo cuerpo hemos colocado el valor la *match* que hay en el modelo asociado a esta vista. Como dijimos, el código TypeScript se traduce a JavaScript, que es el lenguaje que realmente entiende el navegador; por eso, el resultado que muestra es un objeto en forma de cadena, que es algo tan poco entendible como la cadena *[object Object]*.

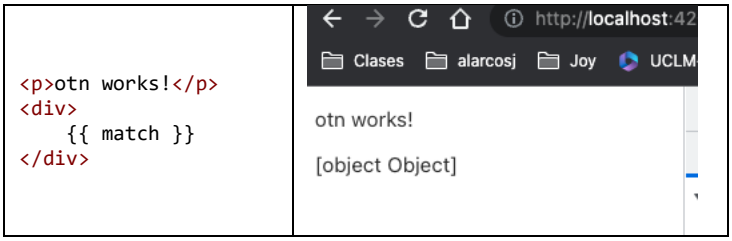


Figura 66. Bindeamos a match *en la vista*

[4] En nuestro contexto, se llama *renderizar* a la acción que lleva a cabo el navegador cuando dibuja, pinta o escribe un resultado.

La partida (objeto *match)* contiene, como sabemos, un campo *numbers* que, a su vez, incluye un objeto *numbers,* que hemos declarado como un array bidimensional. Con el siguiente código, accedemos desde la vista al campo *numbers* de *match,* e iteramos (directiva **ngFor)* sobre cada fila contenida en *match*. Como se ve en la Figura 67, iteramos sobre cada fila (*row)* del array bidimensional que teníamos en el campo *numbers* de la *match;* para cada fila, escribimos su longitud mediante *{{ row.length }}*.

```
<p>otn works!</p>

<div *ngFor="let row of match.numbers">

        <div>

          {{ row.length }}

        </div>

</div>
```

```
otn works!

3
3
3
```

Figura 67. Recuperamos la longitud de cada fila

En la siguiente figura, iteramos también sobre los elementos de cada columna, para que se nos muestre la palabra "Hola". Así, esperamos obtener una matriz de 9x9 y que en todas las celdas podamos leer "Hola":

```
<p>otn works!</p>

<div *ngFor="let row of match.numbers">
    <span *ngFor="let col of row">
        Hola
    </span>
</div>
```

```
otn works!

Hola Hola Hola
Hola Hola Hola
Hola Hola Hola
```

Figura 68. Iteramos sobre cada fila y columna

Para que en el cliente se muestren los números desordenados que se han generado en el elemento *match* basta con que, en lugar de escribir "Hola", bindeemos al valor del campo *col:* véase, en la figura anterior, que cada elemento *row* del primer **ngFor* representa una fila de la matriz contenida en *match.numbers;* y que, en el segundo **ngFor,* cada elemento *col* representa un elemento de los contenidos en la *row* actual: como cada elemento *col* es uno de los valores de la *row,* podemos recuperar su valor sustituyendo la palabra "Hola" por *{{ col }}*:

```
<p>otn works!</p>

<div *ngFor="let row of match.numbers">
    <span *ngFor="let col of row">
        {{ col }}
    </span>
</div>
```

```
Clases

otn works!

8 7 4
2 6 5
1 3
```

Figura 69. Mostramos el valor de cada celda de la matriz

4.4. Conexión del comportamiento

En el epígrafe anterior hemos visto cómo conectar el modelo (entendido de forma ampliada, pues hemos hecho referencia a la clase *Otn,* que está definida separadamente de *OtnComponent,* elemento este que sí está completamente conectado al *html)* con la vista para que esta pueda mostrar su estado. De alguna manera, y siguiendo el orden de la Figura 65, transmitimos el estado del elemento de la derecha (el modelo) hacia la izquierda (la vista).

Pero ahora nos interesa transmitir el estado de izquierda a derecha: es decir, cuando el usuario pulse sobre un dígito que esté junto a la posición vacía, ambas posiciones deben intercambiarse. Así, por ejemplo, el tablero desordenado que se mostraba por consola en la Figura 64 debería mostrarse al usuario como en la Figura 70.

2	5	4
1	8	3
6	7	

Figura 70. Tablero correspondiente a la Figura 64

Si, en la figura anterior, el usuario hace clic sobre el número 3, la posición vacía debería pasar a ocupar la posición del 3, y este número debería bajar, de forma que el tablero quedase de la siguiente forma:

2	5	4
1	8	
6	7	3

Figura 71. Tablero después de hacer clic sobre el número 3

Para esto, debemos conseguir que el modelo reciba los eventos provenientes de la vista. Es decir, nos gustaría completar el código de la Figura 69 para que, al hacer clic sobre un dígito, el modelo tomase nota e hiciese el intercambio de posiciones correspondiente.

En el diagrama de clases de la Figura 65 teníamos tres clases: vista, modelo y un objeto del nivel de dominio, *Otn,* en la que debe residir la lógica del juego. Con esta estructura de clases, un posible diagrama de secuencia en UML para este escenario tan concreto, referido a hacer clic sobre el número 3 de la Figura 70 (ubicado en la fila 1, columna 2), puede ser el siguiente:

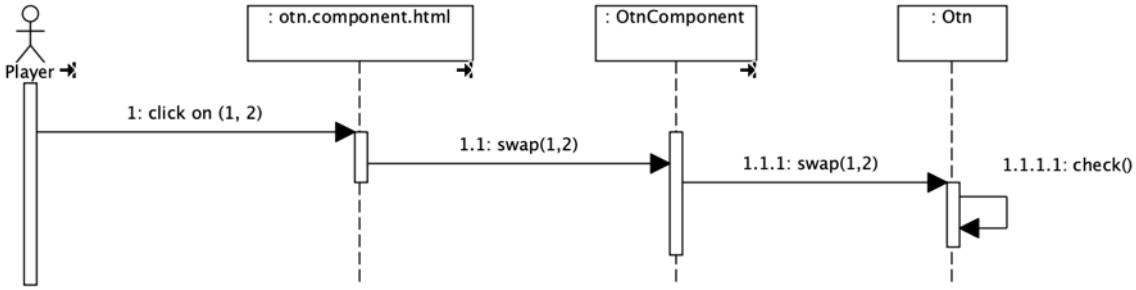


Figura 72. El jugador hace clic en la posición (1, 2)

En la figura anterior, el usuario hace clic en la posición (1, 2); inmediatamente, el modelo asociado a la vista recibe el mensaje y ejecuta su método *swap,* que recibe la fila y la columna como parámetros, y lo pasa al objeto de dominio asociado. Este comprobará que la posición sobre la que se ha hecho clic sea adyacente a la posición vacía y, como el diagrama de secuencia de la Figura 72 está describiendo el escenario concreto en que el intercambio de posiciones sí puede realizarse, se actualiza la vista de alguna manera.

Obviamente:

1) Hay que transmitir el evento *click* desde la vista al modelo (el evento es *click,* pues proviene del inglés, aunque en el texto usemos su traducción al castellano, "clic"[5]).

[5] https://dle.rae.es/clic

2) En el modelo, hay que implementar el receptor del tal evento *click* mediante un método al que, según la Figura 72, llamaremos *swap* (en castellano, "intercambiar").
3) Tal método *swap* del modelo llama al método *swap* de la instancia de dominio *match* asociada. Aquí se llamará, según el diagrama de secuencia, a otro método *check* (que podrá ser privado) y que, según el conocimiento que tenemos del problema, ha de comprobar que la posición sobre la que se ha hecho clic sea adyacente a la posición vacía.

4.4.1. Emisión del evento click desde la vista

Para el primer punto (transmitir el evento *click* desde la vista al modelo), modificamos ligeramente el código de la Figura 69. En la Figura 73 indicamos que, cuando se haga clic sobre este *span* de la vista, se ejecute *swap* sobre el modelo asociado. Recuérdese (Figura 47) que, como esta vista forma parte de un componente (Figura 52), Angular los mantiene a ambos enlazados.

```
<p>otn works!</p>

<div *ngFor="let row of match.numbers">
    <span (click)="swap(row, col)" *ngFor="let col of row">
        {{ col }}
    </span>
</div>
```

Figura 73. Con (click)*, enviamos el evento clic desde la vista hacia el modelo*

4.4.2. Recepción del evento en el modelo

Para el segundo punto (recibir en el modelo el evento *click)* tenemos que implementar en el modelo un método *swap* que ha de recibir la fila y la columna sobre la que se ha hecho clic. Así a bote pronto, podríamos añadir el método a *otn.component.ts* y dejarlo como en la Figura 74, en la que solo pedimos que se muestre en la consola la fila y columna sobre las que hemos hecho clic. Más tarde lo pasaremos a *match,* el objeto de dominio de clase *Otn*. El código de esta figura no compila, porque debemos declarar el tupo de los parámetros *row* y *col*.

```
import { Component, OnInit } from '@angular/core';
import { Otn } from './otn';

@Component({
  selector: 'app-otn',
  templateUrl: './otn.component.html',
  styleUrls: ['./otn.component.css']
})
export class OtnComponent implements OnInit {

  match : Otn

  constructor() {
    this.match = new Otn()
  }

  ngOnInit(): void {
  }

  swap(row, col) {
    console.log(row + ", " + col)
  }
}
```

Figura 74. Una versión de swap *que no nos sirve*

row y *col* serán los números de fila y de columna, por lo que los declaramos como *number* en *swap:*

```
swap(row : number, col : number) {
  console.log(row + ", " + col)
}
```

Figura 75. Declaramos como number *los parámetros de* swap

Sin embargo, cuando guardamos y se recarga el proyecto en *localhost:4200,* el compilador de Angular nos informa de un error. En la Figura 76, el compilador de TypeScript integrado en Visual Studio Code nos informa de que el elemento que pasamos como primer parámetro al método *swap* (que, en la declaración formal que hemos hecho en la Figura 75, es de tipo *number)* no se puede convertir a un array *number[].*

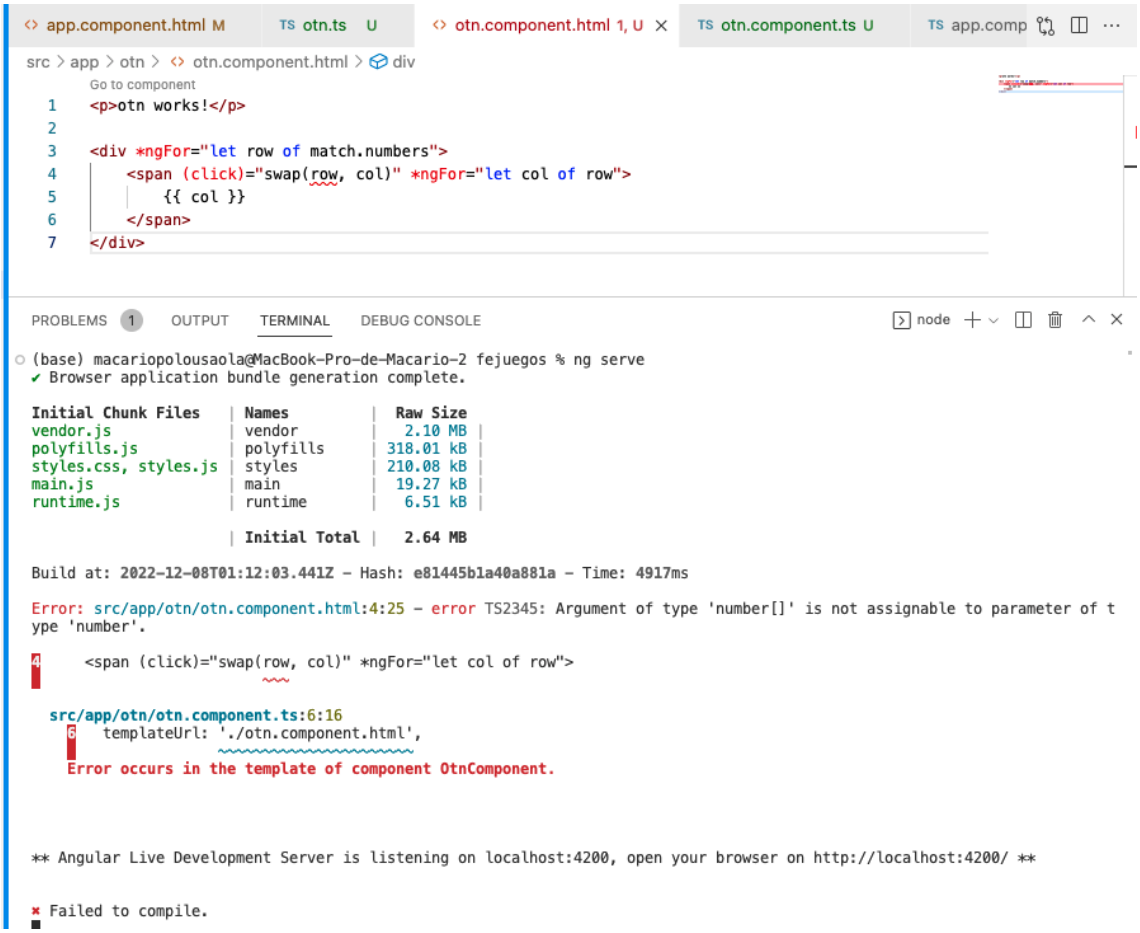


Figura 76. Un error de compilación, posiblemente inesperado

Y es que, en efecto, el primer parámetro *row* que pasamos desde la línea 4 de la vista es un array de *number*, y no un objeto de clase *number* como podía esperarse: es decir, el parámetro *row* cuyo valor pasamos es un *number[].* Podemos solucionarlo de dos maneras:

1) Declarar el primer parámetro de *swap* como *number[]:*

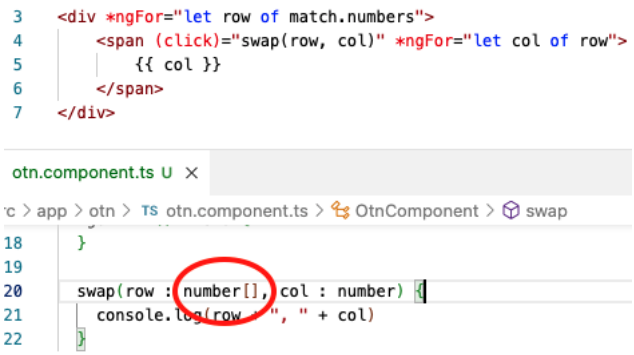


Figura 77. Declaración del parámetro formal como number[]

2) Pasar como valores a *swap* los números de fila y columna.

Elegiremos la segunda opción, ya que parece más natural que *swap* reciba como parámetros las posiciones de la fila y la columna sobre las que el usuario ha hecho clic. Para esto, podemos ampliar las dos directivas **ngFor* del *html*: **ngFor* itera sobre una colección y dispone de una variable *index* que podemos recuperar: en la Figura 78 asignamos a la variable *i* el índice del primer **ngFor,* y a *j* el índice del segundo. Además, en la llamada a *swap,* pasamos los valores de *i* y de *j*.

```
<div *ngFor="let row of match.numbers; let i = index">
    <span (click)="swap(i, j)" *ngFor="let col of row; let j = index">
        {{ col }}
    </span>
</div>
```

Figura 78. Recuperación del índice de la iteración

Ahora, la aplicación responde correctamente, mostrándose por consola los valores deseados de la fila y la columna según hacemos clic en cada número:

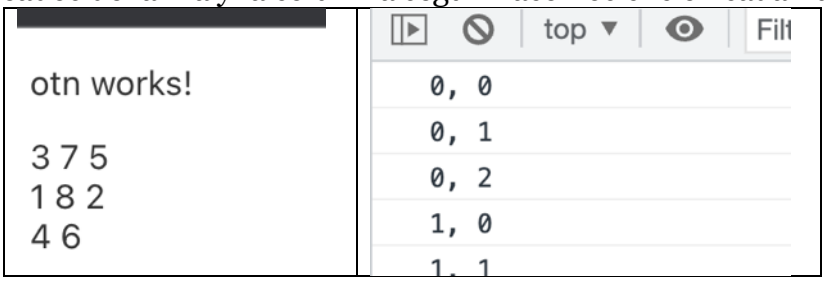


Figura 79. El sistema va funcionando

4.4.3. Paso del mensaje al objeto de dominio

En el modelo de este componente (clase *OtnComponent)* tenemos declarado (véase el diagrama de clases de la Figura 65 y el código que lo representa en la Figura 74) un objeto *match* de clase *Otn,* que es a quien debemos delegar la responsabilidad de comprobar que el movimiento sea correcto.

Para ello, reescribimos el método *swap* como sigue:

```
20  swap(row : number, col : number) {
21    this.match.swap(row, col)
22  }
```

Figura 80. Nueva implementación de swap *en* OtnComponent

Puesto que el método *swap* no existe en la clase del objeto al que lo estamos enviando (*this.match,* de tipo *Otn*) debemos crearlo. El menú *Quick fix* (Figura 81, izquierda) nos da la opción de crear el método *swap* directamente en la clase *Otn* (Figura 81, centro); una vez aceptada esta opción, podemos ir directamente a su código (derecha).

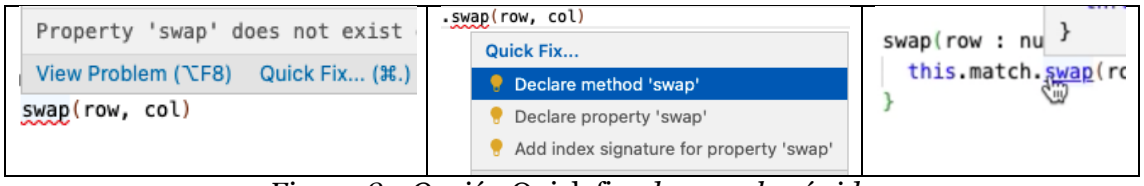


Figura 81. Opción Quick fix, *de arreglo rápido*

VSC nos ha creado un método *swap(row: number, col : number)* al comienzo de la clase *Otn*. Su implementación simplemente lanza un error que indica que el método no está implementado. Primero, organizamos el código, llevando este método después del constructor de la clase (es una buena costumbre que los campos aparezcan al principio de la clase, luego el constructor y luego los métodos) y eliminando la instrucción de que se lance el error. Es decir, dejamos la clase *Otn* como sigue:

```
export class Otn {
    numbers : number[][]

    constructor() {
        let values = [1, 2, 3, 4, 5, 6, 7, 8]

        let a, b, aux
        for (let i=0; i<100; i++) {
            a = Math.floor(Math.random()*values.length)
            b = Math.floor(Math.random()*values.length)
            aux = values[a]
            values[a] = values[b]
            values[b] = aux
        }

        this.numbers = []
        let cont = 0
        for (let i=0; i<3; i++) {
            this.numbers[i] = []
            for (let j=0; j<3; j++) {
                if (cont<9)
                    this.numbers[i].push(values[cont++])
            }
        }
        console.log(this.numbers)
    }

    swap(row: number, col: number) {

    }
}
```

Figura 82. Código actual de la clase de dominio Otn

El método *swap* debe comprobar que la posición sobre la que hacemos clic sea adyacente a la posición vacía que, inicialmente, se encuentra en las coordenadas (2, 2). Como esta irá cambiando, podemos declarar un campo *emptyPosition* en la clase, en la que iremos colocando la posición vacía. Declararemos el tipo de este campo como *any*, con lo que admitirá cualquier tipo de valor: en nuestro caso, será un JSON con dos campos *row* y *col*. Podemos inicializarlo en el constructor o bien en su misma declaración, como hacemos en la Figura 83.

```
1   export class Otn {
2       numbers : number[][]
3       emptyPosition : any = { row: 2, col : 2 }
4
5       constructor() {
6           let values = [1, 2, 3, 4, 5, 6, 7, 8]
```

Figura 83. Declaración e inicialización del campo emptyPosition

Ahora, cuando se ejecute el método *swap*, debemos comprobar si los parámetros que se reciben (*row* y *col*) son adyacentes con los valores contenidos en los campos *row* y *col* de *emptyPosition*. Si lo son, colocamos en la posición vacía el número sobre el que hemos hecho clic, colocamos un valor vacío en la posición sobre la que se hizo clic y actualizamos el campo *emptyPosition*. En la Figura 84 se muestra el código de *swap* y del método auxiliar *isAdjacent*. Obsérvese que, en *swap*, asignamos el valor 0 a la nueva posición vacía.

```
swap(row: number, col: number) {
    if (this.isAdjacent(row, col)) {
        this.numbers[this.emptyPosition.row][this.emptyPosition.col] =
                this.numbers[row][col]
        this.numbers[row][col] = 0
        this.emptyPosition = {
            row : row,
            col : col
        }
    }
}

private isAdjacent(row : number, col : number) {
    return this.emptyPosition.row-1==row && this.emptyPosition.col==col
        || this.emptyPosition.row+1==row && this.emptyPosition.col==col
        || this.emptyPosition.row==row && this.emptyPosition.col-1==col
        || this.emptyPosition.row==row && this.emptyPosition.col+1==col
}
```

Figura 84. Métodos swap *e* isAdjacent

Si probamos la interfaz de usuario, vemos que el número sobre el que hacemos clic se intercambia con la posición vacía cuando es adyacente. Sin embargo, tras el primer movimiento aparece el 0 que hemos asignado a la posición vacía.

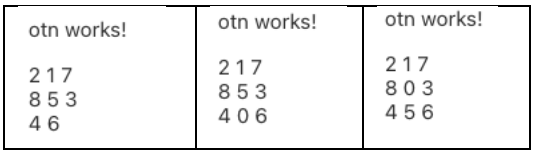


Figura 85. El juego va avanzando

4.5. Adaptación a la descripción del juego

Con el "gorro" de desarrolladores que tenemos ahora mismo, tanto el aspecto visual del juego como el hecho de que aparezca el 0 son asuntos menores que resolveremos después. Ahora, debemos tener en cuenta que, en la descripción del juego (página 49), se decía que "Cuando los consigue ordenar crecientemente del 1 al 8, se le propone un nuevo cuadrado con números desordenados". Bien, pues esto es lo que debemos completar, y lo haremos añadiendo algo de código al método *swap* para que, tras actualizar la posición vacía, se compruebe si los números están ordenados (Figura 86).

```
swap(row: number, col: number) {
    if (this.isAdjacent(row, col)) {
        this.numbers[this.emptyPosition.row][this.emptyPosition.col] =
                this.numbers[row][col]
        this.numbers[row][col] = 0
```

```
        this.emptyPosition = {
            row : row,
            col : col
        }
        if (this.numbersAreOrdered())
            console.log("Ordenados")
    }
}

private numbersAreOrdered() : boolean {
    let numbers = []
    for (let i=0; i<3; i++) {
        for (let j=0; j<3; j++) {
            if (this.numbers[i][j])
                numbers.push(this.numbers[i][j])
        }
    }
    let last = numbers[0]
    for (let i=1; i<numbers.length; i++)
        if (numbers[i]!=last+1)
            return false
        else
            last = numbers[i]
    return true
}
```

Figura 86. Comprobamos si los números están ordenados

Ya conseguimos determinar si la secuencia de números está ordenada. De hecho, para probarlo sin tener que jugar una partida, podemos modificar el código del constructor de *Otn* para que los números no se desordenen (basta con poner *i<0* en la condición de finalización del primer *for*).

Ahora, y según la descripción del juego, debemos proponer un nuevo cuadrado con los números desordenados. Para eso, modificamos la instrucción *console.log("Ordenados")* de la Figura 86 por una llamada *this.disorder(),* un método que implementaremos reutilizando casi en su totalidad el código del constructor. Además, también sustituiremos el código del constructor por una llamada a *this.disorder():*

```
export class Otn {
    numbers : number[][]
    emptyPosition : any = { row: 2, col : 2 }

    constructor() {
        this.disorder()
    }

    private disorder() {
        let values = [1, 2, 3, 4, 5, 6, 7

        let a, b, aux
        for (let i=0; i<100; i++) {
            a = Math.floor(Math.random()*
            b = Math.floor(Math.random()*
            aux = values[a]
            values[a] = values[b]
            values[b] = aux
        }

        this.numbers = []
        let cont = 0
        for (let i=0; i<3; i++) {
            this.numbers[i] = []
            for (let j=0; j<3; j++) {
                if (cont<9)
                    this.numbers[i].push(values[cont++])
            }
        }
    }

    swap(row: number, col: number) {
        if (this.isAdjacent(row, col)) {
            this.numbers[this.emptyPosition.row][this.emptyPosition.col] = this.numbers[row][col]
            this.numbers[row][col] = 0
            this.emptyPosition = {
                row : row,
                col : col
            }
            if (this.numbersAreOrdered())
                this.disorder()
        }
    }
```

Figura 87. Llamamos a this.disorder() *en el constructor y si los números están ordenados*

Observamos en la Figura 87 el nuevo método *disorder* y las dos llamadas que hemos incluido. También, en la línea 2, el compilador nos advierte de un error (Figura 88), y es que el campo *numbers* no está recibiendo valor ni en la misma línea en la que lo estamos declarando, ni en el constructor de la clase.

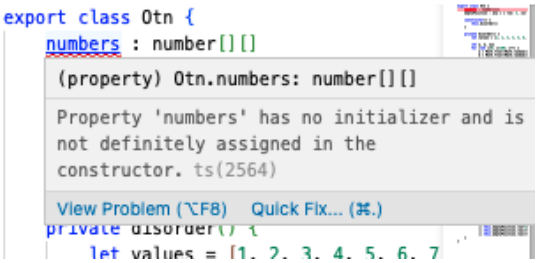


Figura 88. Debemos dar valor a los campos, bien en su declaración, bien en el constructor

Para resolverlo, basta con que inicialicemos el campo en el constructor (*this.numbers=[]*). Igualmente, podemos eliminar esta misma instrucción de la línea 21.

4.6. Ligera adecuación del aspecto visual

Ciertamente, el juego aparece como muy poco atractivo. Aunque no conseguiremos una maravilla visual, sí que haremos algunos cambios para adecuar su aspecto trabajando, obviamente, sobre la vista (el *html).*

Sustituiremos el *span* en el que mostramos cada número por un botón (Figura 89): como vemos, el botón correspondiente a la posición vacía sale "feo" la primera vez (pues su valor inicial es *undefined)* y con un 0 la segunda y sucesivas veces.

```
<p>otn works!</p>
<div *ngFor="let row of match.numbers; let i = index">
    <span (click)="swap(i, j)" *ngFor="let col of row; let j = index">
        {{ col }}
    </span>
</div>
```

```
<p>otn works!</p>
<div *ngFor="let row of match.numbers; let i = index">
    <button (click)="swap(i, j)" *ngFor="let col of row; let j = index">
        {{ col }}
    </button>
</div>
```

Figura 89. Cambiamos el span *por un* button

Usaremos la directiva **ngIf* para que, cuando *col* (que es la variable que estamos bindeando y mostrando mediante *{{ col }}*) tenga valor, este se muestre y que, cuando no lo tenga, se muestre un espacio en blanco: es importante notar que tanto el valor *undefined* como 0 hacen cierta la condición *!col.*

```
<p>otn works!</p>

<div *ngFor="let row of match.numbers; let i = index">
    <button (click)="swap(i, j)" *ngFor="let col of row; let j = index">
        <span *ngIf="col">{{ col }}</span>
        <span *ngIf="!col"> </span>
    </button>
</div>
```

Figura 90. Uso de *ngIf

Capítulo 5. Desarrollo del *backend* para *Word race* con *Test-Driven Development*

1. Introducción

Para el desarrollo del lado servidor del *Word race,* utilizaremos el entorno de desarrollo Eclipse.

En este capítulo implementaremos el lado servidor del *Word race* con un enfoque TDD. Cuando el sistema se ponga en producción, el diccionario residirá en una tabla de una base de datos con las palabras del diccionario ofrecida por un servidor de SQL Server que correrá en un *docker,* si bien en este capítulo utilizaremos, para apoyar nuestro desarrollo dirigido por pruebas, una base de datos en memoria.

2. Un esbozo de la arquitectura del *backend*

Desarrollaremos este subsistema como un proyecto Maven y utilizaremos Spring Boot. Spring Boot es un *framework* para la construcción de aplicaciones Java, que se apoya en Spring y que acelera el desarrollo de los sistemas mediante un conjunto muy rico de anotaciones.

En Spring hay cuatro tipos fundamentales de clases:

1) Controladores, que reciben información del exterior, posiblemente la validen y deleguen la responsabilidad a un servicio.
2) Servicios, que implementan operaciones de negocio.
3) Entidades, que representan objetos contenidos en la base de datos.
4) Repositorios, que acceden a la base de datos.

De forma general, la comunicación entre estos tipos de clases es la que se muestra en la Figura 91: el controlador puede ser una clase que reciba mensajes *http* desde un navegador; cuando recibe el mensaje, pasa la petición al servicio, que es quien realmente implementa la operación que deba ejecutarse. Muchas veces, el servicio necesita almacenar o recuperar información de la base de datos: si con la información recibida debe, por ejemplo, guardar cierta información nueva en la base de datos, creará una instancia de la entidad y, mediante un repositorio, la guardará; si, por ejemplo, la petición consiste en recuperar cierta información de la base de datos, utilizará el repositorio para crear una instancia de la entidad, que será la que se devuelva al controlador y, desde este, viajará al cliente que la solicitó.

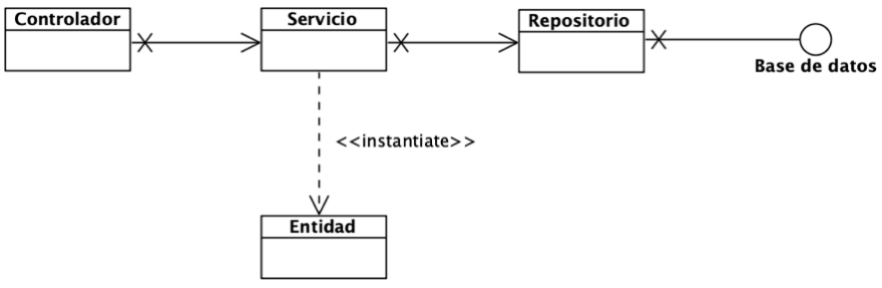


Figura 91. Comunicación entre clases en Spring

3. Configuración de Eclipse

Disponemos de muchas versiones de Spring Boot, siendo quizá la más utilizada la 2.0.5. Recientemente se ha liberado la versión 3.0.1, que introduce bastantes modificaciones respecto de las anteriores. En particular: (1) requiere la versión 17 del JDK (*Java Development Kit*) de Java, y (2) no utiliza (ya lo mencionaremos más adelante) el paquete *javax*, que se ha sustituido por el paquete *jakarta*.

En este libro se utilizará la versión 3.0.1 y, puntualmente, se dará información respecto del uso de Spring Boot con versiones anteriores: a nivel de código, las diferencias son realmente muy pequeñas.

3.1. Instalación de un JDK versión 17

Utilizaremos un *Open JDK* que podemos descargar de https://jdk.java.net/archive/. Una vez instalado, debemos indicarle a Eclipse dónde se encuentra. Para ello, vamos a las preferencias de Eclipse (en Windows y Linux, en el menú *Window;* en Mac, en el menú *Eclipse)*, lo añadimos y lo marcamos para que se use por defecto:

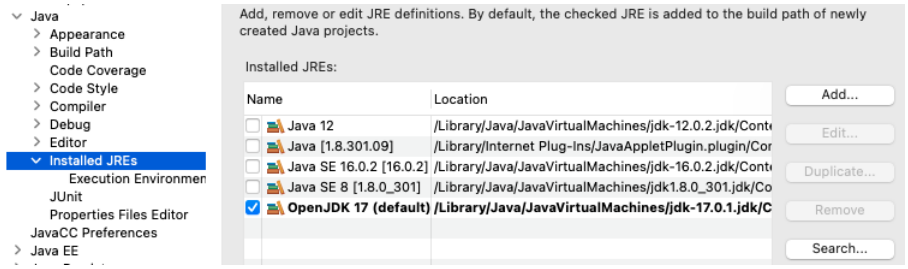


Figura 92. Elegimos el JDK 17

4. Creación y preparación del proyecto

En Eclipse (Figura 93), elegimos crear un nuevo *Maven Project,* dejamos las opciones por defecto en el segundo paso del asistente y, en el tercero, elegimos el arquetipo *quickstart*.

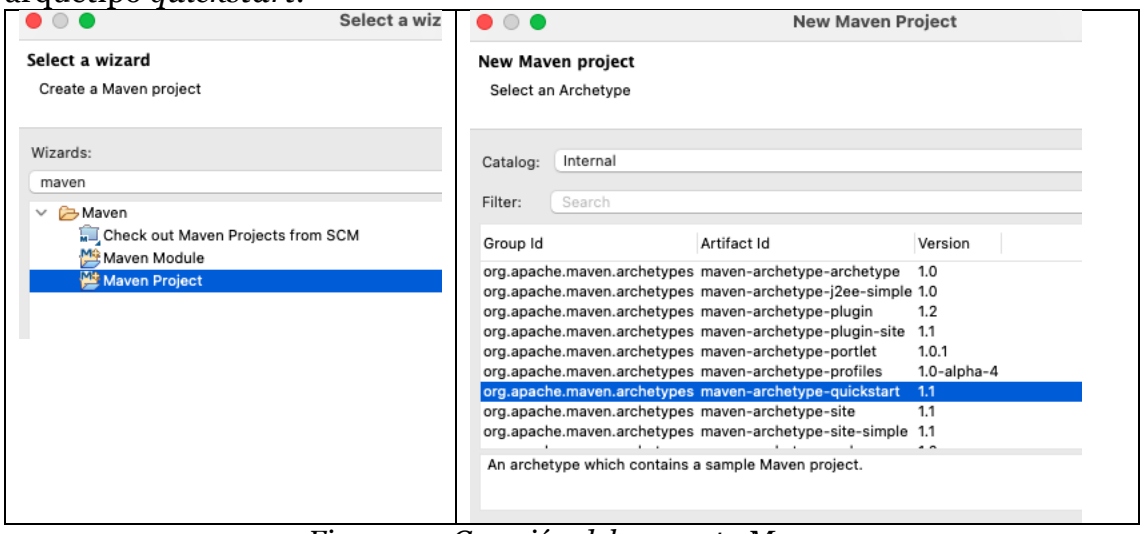


Figura 93. Creación del proyecto Maven

Un arquetipo Maven describe la estructura de carpetas y ficheros del proyecto que se va a crear: muchos proyectos tienen estructuras parecidas (en cuanto a arquitectura, por ejemplo), con una distribución similar de ficheros en

carpetas, etcétera. El arquetipo genera una estructura predeterminada de este estilo.

El arquetipo seleccionado es muy simple, pero nos servirá para ir construyendo manualmente todos los elementos que vayamos necesitando.

En el siguiente paso, el asistente nos solicita que indiquemos algunos parámetros del proyecto, y los rellenaremos como en el lado izquierdo de la Figura 94. Tras pulsar *Finish,* se nos habrán creado las carpetas y ficheros que se muestran en el lado derecho.

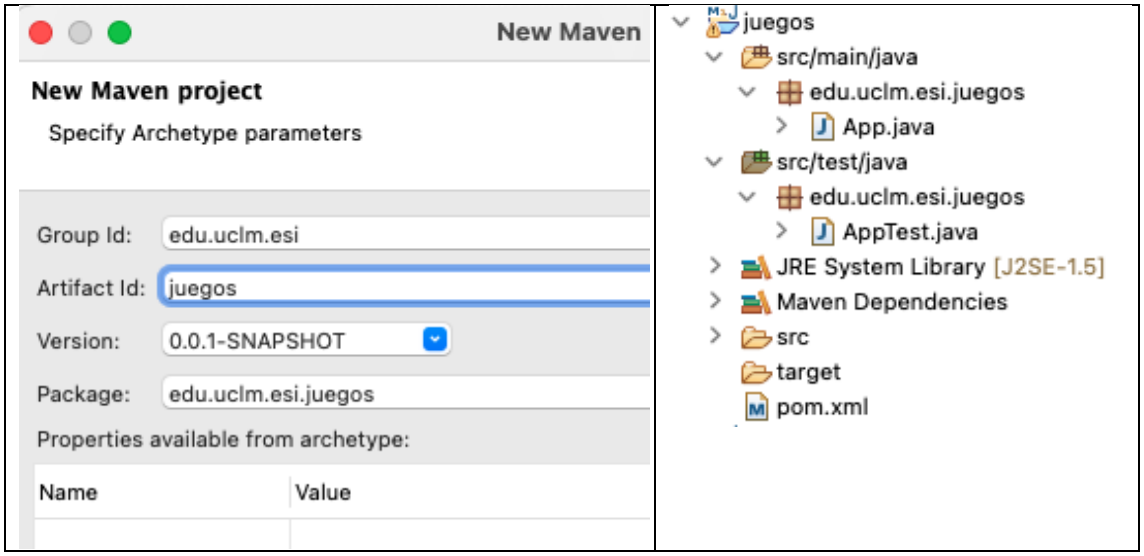


Figura 94. Creamos el proyecto edu.uclm.esi.juegos

Uno de los ficheros que se nos ha generado es el *pom.xml* (*Project Object Model)*, en el que se indican, entre otros elementos, las dependencias que se incluirán en el proyecto. Este arquetipo nos ha incluido solamente una (Figura 95) que hace referencia a la versión 3.8.1 de JUnit.

Figura 95. Inclusión de una dependencia a JUnit 3.8.1

Esta versión es demasiado antigua y deseamos sustituirla por JUnit Júpiter, por lo que haremos la modificación correspondiente en el *pom.xml*. Para saber qué debemos escribir exactamente, podemos buscar la dependencia en, por ejemplo, *mvnrepository* (Figura 96).

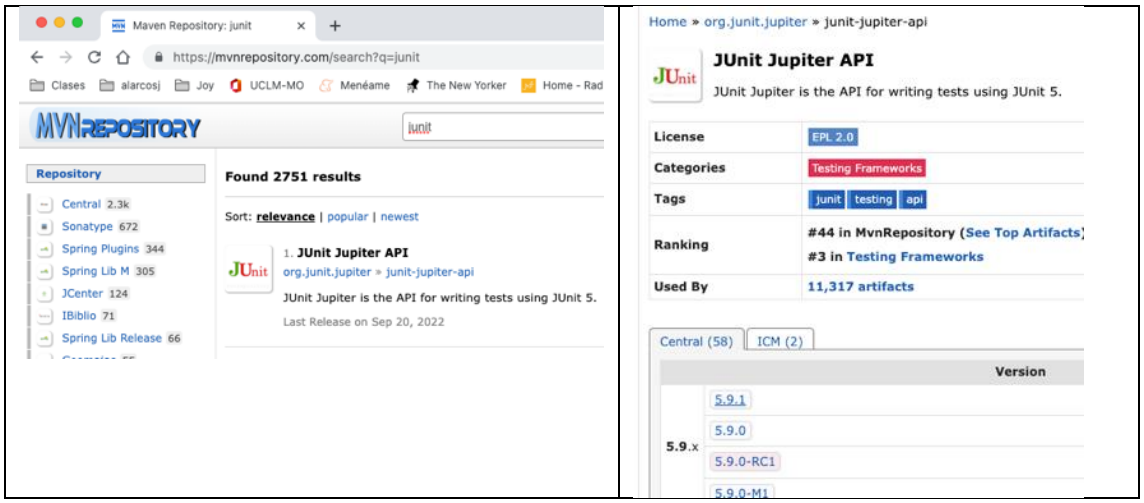


Figura 96. Búsqueda en mvnrepository.com

Copiamos el código XML que se incluye y lo pegamos en el *pom.xml*, sustituyendo la dependencia de JUnit 3.8.1. Realmente, añadiremos algunas dependencias más, todas ellas de JUnit (Figura 97).

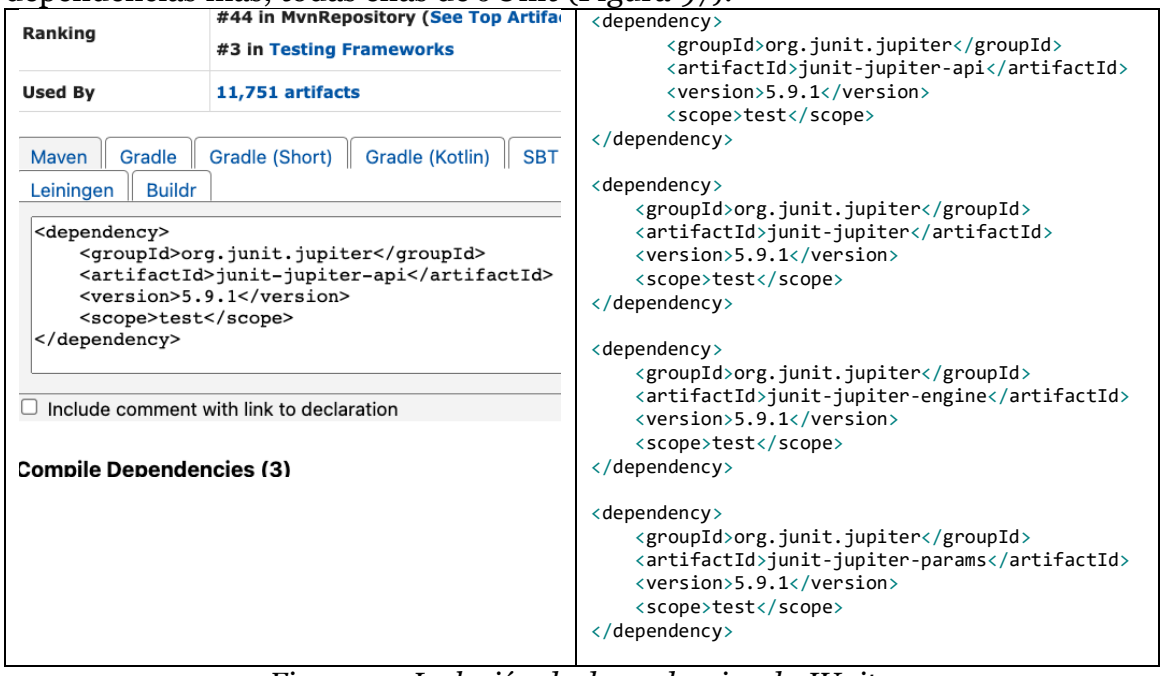


Figura 97. Inclusión de dependencias de JUnit

Tras este cambio, el fichero *AppTest.java* que nos había creado el arquetipo presenta errores de compilación, que se deben a la sustitución de la versión 3.8. por la 5.9.1. No importa: podemos eliminar con tranquilidad este fichero.

4.1. El fichero *application.properties*

Los proyectos de Spring Boot necesitan un fichero de configuración llamado *application.properties,* en el que se definen algunos parámetros importantes de ejecución: puerto en el que escuchará el servidor *http,* URL de la base de datos, así como otros elementos que pueden ser necesarios.

Este fichero se ubica normalmente en la carpeta *src/main/resources,* si bien puede haber otro en *src/test/resources,* para definir elementos de configuración aplicables solo al entorno de pruebas.

Pulsando con el botón derecho sobre el proyecto, podemos crear una nueva *Source folder* (Figura 98) llamada *resources*.

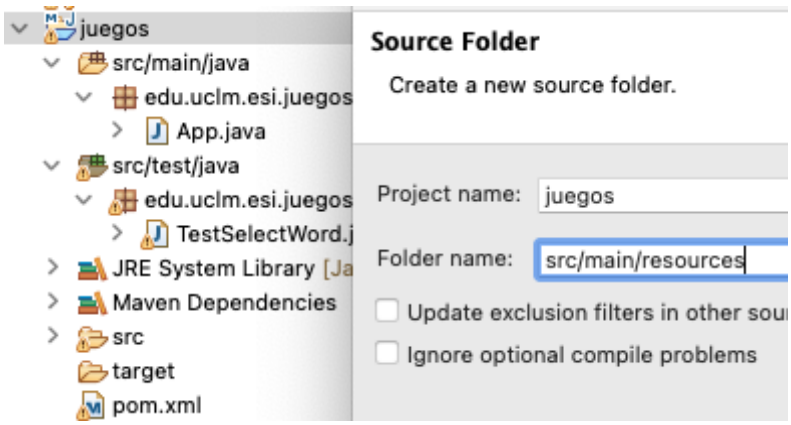


Figura 98. Creación de la carpeta resources

Cuando lo necesitemos, que será pronto, crearemos en esta carpeta el fichero *application.properties,* que puede crearse vacío de momento.

5. Visión del proyecto como desarrolladores *Test Driven*

El enfoque de desarrollo básico con TDD pasa por la construcción progresiva del código de producción mediante la descripción previa de escenarios de uso del sistema, que se escriben como casos de prueba. Obviamente, no todo vale: que el caso de prueba se supere es condición necesaria, pero no suficiente, para dar por bueno el código de producción.

Entre otros elementos, el desarrollador TD debe tener presente las funcionalidades y el diseño arquitectónico que vaya a tener el sistema. Por eso, velaremos en todo momento por que el software que vayamos construyendo se adapte al estilo arquitectónico esbozado en la Figura 91.

Las funcionalidades que debe ofrecer el *backend* para el juego *Word race* son, según el diagrama de casos de uso que ya construimos (y que reproducimos de nuevo en la Figura 99):

1) *Select word,* que debe devolver una palabra al azar de las contenidas en el diccionario.
2) *Check word,* que comprueba si la combinación de letras que se recibe es una palabra del diccionario.

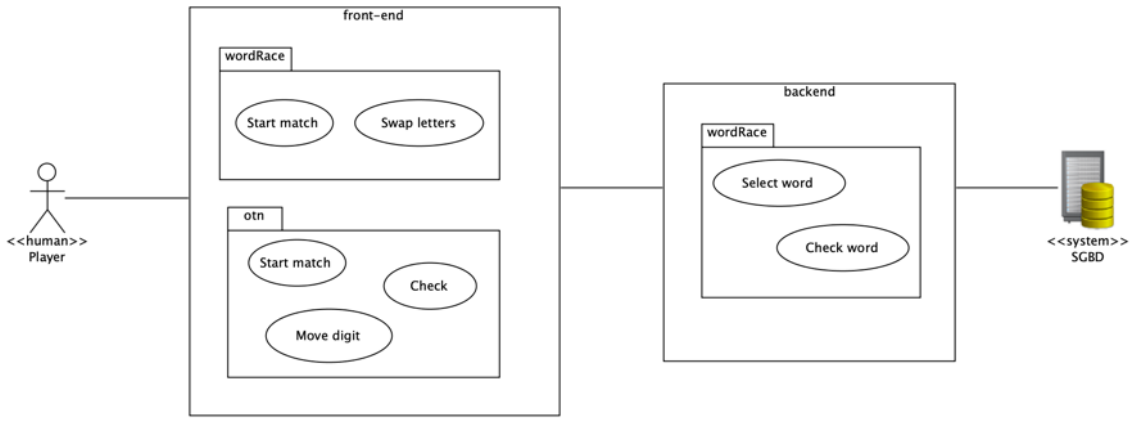


Figura 99. Casos de uso

Ambas funcionalidades pueden ofrecerse como servicios web desde un mismo controlador al que, por ejemplo, podremos llamar *WordRaceController:*

1) De acuerdo con el estilo arquitectónico ya comentado, cuando este controlador reciba una petición de ejecutar la funcionalidad *selectWord*, enviará un mensaje a un servicio (*WordRaceService,* por ejemplo) que, utilizando un repositorio (*WordDAO*), devolverá una instancia de la entidad *Word*.
2) De manera parecida, cuando el controlador reciba una petición de comprobar una palabra, le pedirá al servicio que, a su vez, le pregunte al repositorio que si tal palabra existe.

En la siguiente figura se ilustra el posible paso de mensajes que puede suceder cuando al *WordRaceController* le llega, desde el navegador, una petición de ejecución para que se le devuelva una palabra aleatoria de 5 letras:

1) El mensaje 1 representa la petición que recibe el controlador. Se ha estereotipado con *«get»* porque se trata de una petición *http* de este tipo, pues lo que se desea es, simplemente, realizar una consulta. El mensaje recibido es *selectWord?length=5*, en donde *length* es un parámetro y 5 es el valor.
2) El mensaje 1.1 representa la llamada que el controlador hace al servicio. Le pasa como valor del parámetro el 5 que ha recibido desde el navegador.
3) El servicio utiliza el DAO (*Data Access Object:* objeto de acceso a datos) para recuperar palabras con el mensaje 1.1.1. Como no conocemos ni imaginamos de momento de qué forma recuperaremos una palabra aleatoria, lo dejamos anotado simplemente con "select word".
4) El DAO, en el mensaje 1.1.1.1, envía una petición al gestor de base de datos en el lenguaje que este entienda: si es SQL, será probablemente una instrucción del tipo *select * from word where length(word)=5,* o algo por el estilo.
5) El DAO devolverá la respuesta al servicio que, en el mensaje 1.1.2, crea una instancia *word* de la entidad *Word,* que será la que se devuelva al controlador.
6) Finalmente, en el mensaje 1.1.3.1, el controlador envía al navegador la respuesta solicitada, tal vez en formato JSON. En el ejemplo, envía un JSON con un campo *word* cuyo valor consiste en las letras de la palabra TACOS desordenadas. Este mensaje lo hemos estereotipado con *«200/OK»* para denotar que se al cliente le llega un código de respuesta correcto (es decir, que la petición se ha resuelto correctamente).

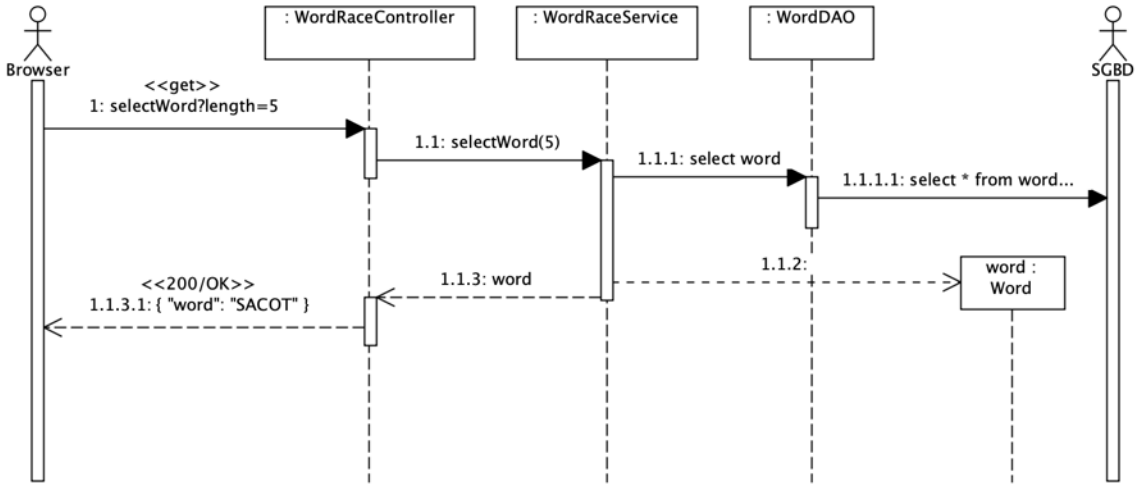


Figura 100. Un posible flujo de mensajes en el backend

En realidad, la petición no viaja directamente desde el *user-agent* (el navegador, en este caso) hasta el controlador, sino que hay una serie de "interceptores" que se ejecutan antes de llegar al controlador. Algunos de estos interceptores son del propio *framework* de Spring, pero otros son del servidor de aplicaciones (Tomcat o JBoss, por ejemplo) que se esté utilizando. Es decir, que realmente, antes de llegar al *WordController*, han pasado otras cosas (Figura 101):

1) La petición la recibe el servidor web en el formato estandarizado del protocolo *http* (Cuadro I): una línea con el método (*get)*, la URI del recurso, etcétera.
2) El servidor web la analiza y procesa y envía a los interceptores de Spring en el mensaje 1.1, que dejamos intencionadamente en blanco porque no sabemos qué es lo que reciben exactamente.
3) Los interceptores de Spring buscan el controlador solicitado (un controlador cuyo nombre público es *words:* véase la primera parte de la URL del mensaje 1: */words)* y, dentro de este, un recurso cuyo nombre público sea *selectWord* y que acepte peticiones de tipo *get*. Además, y puesto que se recibe un parámetro *length=5* en la URL, los interceptores de Spring comprobarán que el método que implementa el recurso *selectWord* tenga declarado un parámetro llamado *length* en el que pueda "inyectarse" el valor 5. Si se puede, se ejecuta el mensaje 1.1.1, sucederá lo que ya se ha explicado en la Figura 100 y, luego, el controlador devolverá la respuesta por el mismo camino por el que ha venido la petición: interceptores de Spring y envío de lo que corresponda al servidor web. Este formateará la respuesta al protocolo *http* (Figura 101) y la entregará al cliente.

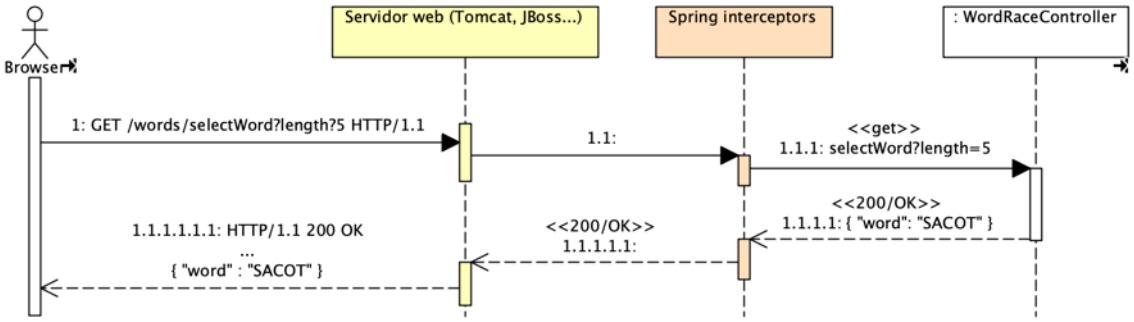


Figura 101. Lo que pasa antes y después de ejecutar el servicio definido en el controlador

Cuadro I

6. Dependencias de Spring

Como tenemos clara la arquitectura del proyecto (los *controladores* reciben la información, puede que la validen, y la pasan a un servicio que ya se encarga del resto) y la tecnología que vamos a usar (Spring Boot), debemos añadir a nuestro fichero *pom.xml* las dependencias necesarias para usar las etiquetas que declaren que ciertas clases serán controladores, servicios, entidades, repositorios, etcétera.

El código de *pom.xml* que, de momento, vamos a necesitar para que el proyecto comience a compilar y a funcionar, es el de la Figura 102, del cual nos interesa, sobre todo, la sección de dependencias:

1) En un primer bloque de la sección *dependencies* se han añadido las librerías necesarias para que la aplicación arranque desde Spring Boot. Alguna dependencia (la segunda, cuyo *artifactId* es *spring-boot-starter-test*, tiene como campo *scope* el valor *test,* lo cual le indica a Spring que solo debe incluir esta dependencia cuando ejecute casos de prueba: es decir, no debe incluirla en el código de producción). Si volvemos a mirar la Figura 94, el arquetipo que hemos utilizado para crear el proyecto ha generado dos carpetas importantes: *src/main/java,* en la que se escribiremos el código de producción; *src/test/java,* en la que escribiremos nuestros casos de prueba.
2) En el segundo bloque están las dependencias de JUnit versión Júpiter que ya incluimos anteriormente.
3) Antes de *dependencies* tenemos también un elemento *parent,* que indica qué versión de Spring Boot estamos utilizando.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>edu.uclm.esi</groupId>
    <artifactId>juegos</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>jar</packaging>

    <name>juegos</name>
    <url>http://maven.apache.org</url>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
```

```xml
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.0.1</version>
    </parent>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-websocket</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-api</artifactId>
            <version>5.9.1</version>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.9.1</version>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-engine</artifactId>
            <version>5.9.1</version>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-params</artifactId>
            <version>5.9.1</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

Figura 102. Nuevas dependencias añadidas a pom.xml

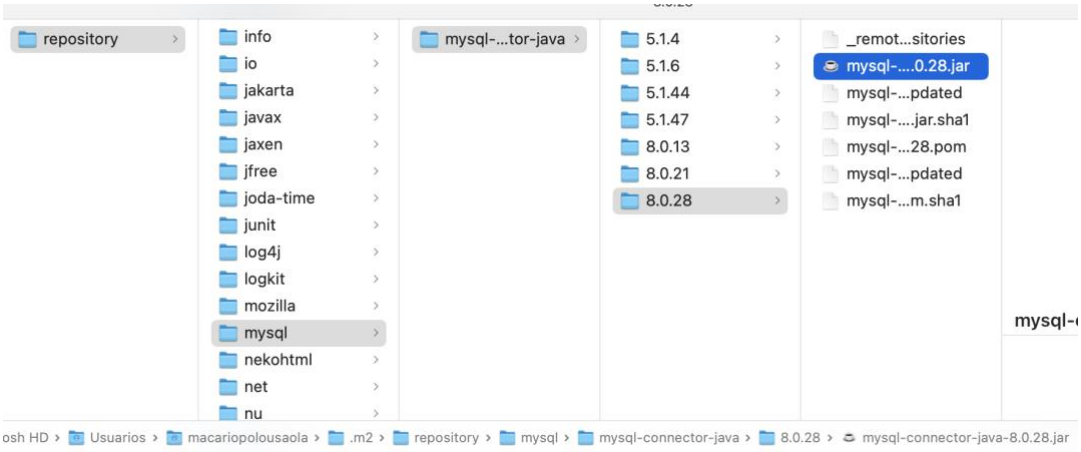
En la figura anterior se ha resaltado el elemento *parent,* en que se indica que la versión de *org.springframework.boot* que deseamos utilizar es la *3.0.1*. Más abajo, en la sección *dependencies,* hay muchas dependencias cuyo *groupId* es también *org.springframework.boot:* cuando se procese este fichero *pom.xml,* Maven sabrá que la versión de estas dependencias es la de su "padre": también la 3.0.1.

Dependencias del *pom.xml*

Cada dependencia incluida en el fichero *pom.xml* incluye al menos un atributo *groupId* y otro *artifactId*. Algunos tienen también un campo *version* y un *scope*.

A medida que vamos añadiendo dependencias en el fichero, estás se descargan del repositorio remoto en el que se encuentren. Si no se indica ninguno (en el *pom* pueden añadirse repositorios remotos adicionales con *repositories*), Maven busca las dependencias en https://repo.maven.apache.org/maven2/, y las descarga en un repositorio local, habitualmente en un directorio llamado *.m2/repository* que cuelga del directorio del usuario.

En la figura se muestra parte del repositorio local: el fichero *jar* de a derecha se encuentra en.*m2/repository/mysql/mysql-connector-java/8.0.28,* lo que indica que su *groupId* es *mysql,* su *atifactId* es *mysql-connector-java* y su campo *version* es *8.0.28*. Como se ve, el desarrollador tiene en su disco varias versiones de las librerías de *mysql* (5.1.4, 5.1.6, etcétera).

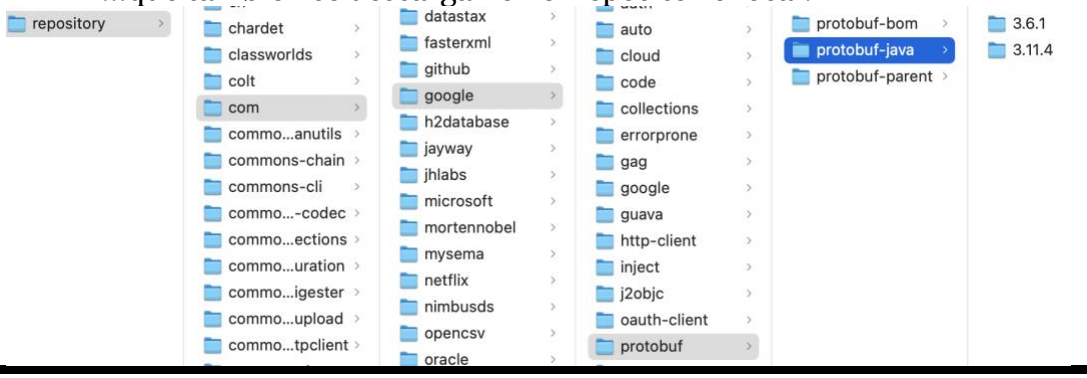


Una dependencia no implica que se descargue un único fichero *jar*. En la carpeta *8.0.28* de la figura anterior vemos que hay un fichero con extensión *pom,* en el que se pueden incluir otras librerías que también deberán descargarse. Por ejemplo, en el fichero *.pom* incluye dos dependencias…

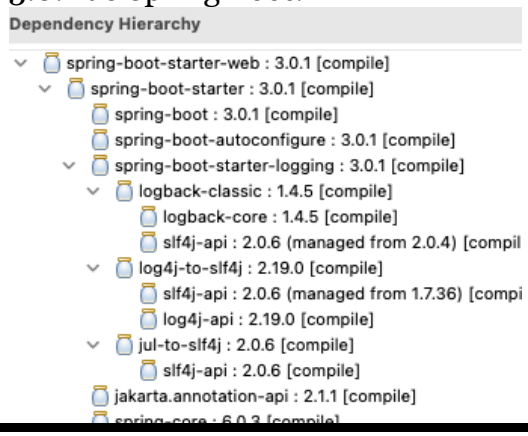
```
<dependencies>
  <dependency>
    <groupId>com.google.protobuf</groupId>
    <artifactId>protobuf-java</artifactId>
    <version>3.11.4</version>
  </dependency>

  <dependency>
    <groupId>com.oracle.oci.sdk</groupId>
    <artifactId>oci-java-sdk-common</artifactId>
    <version>2.3.0</version>
    <optional>true</optional>
  </dependency>
</dependencies>
```

…que también se descargan en el repositorio local:



En Eclipse puede verse la jerarquía de librerías representada en el fichero *pom.xml*. En la siguiente figura, se muestra parte de las dependencias si usamos la versión 3.0.1 de Spring Boot:



Cuadro II. Dependencias del pom.xml

Es posible que, tras modificar el fichero *pom.xml,* Eclipse nos muestre un aspa roja en el árbol del proyecto (Figura 103, izquierda), que suele desaparecer si sobre el proyecto pulsamos el botón derecho y elegimos *Maven, Update Project*.

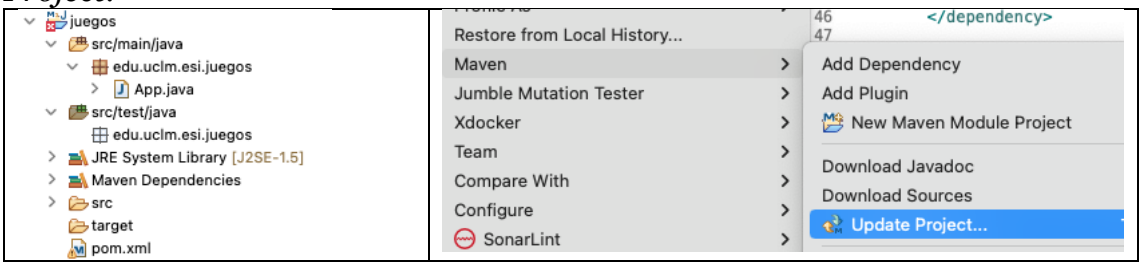


Figura 103. Error extraño al modificar el fichero pom.xml

7. Primer test en el proceso *Test Driven*

Reproducimos de nuevo el paso de mensajes correspondiente a la búsqueda de una palabra aleatoria, pues comenzaremos por aquí la implementación del servidor.

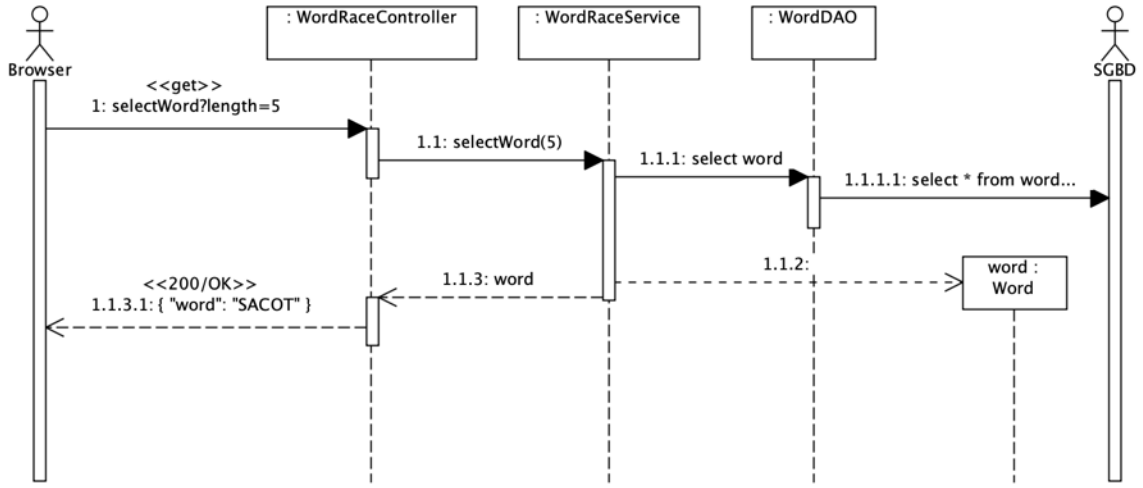


Figura 104. Un posible flujo de mensajes en el backend

En nuestro primer test, simularemos que el usuario, desde su navegador, envía al *WordRaceController* la petición indicada en el mensaje 1. De acuerdo con Kent Beck, el ciclo de desarrollo con TDD consiste en:

1) Crear un caso de prueba.
2) Ejecutar todos los casos de prueba de los que disponemos hasta la fecha. El nuevo caso de prueba debe fallar.
3) Escribir código de producción para que el nuevo caso de prueba se supere.
4) Refactorizar el código de producción, para que sea sencillo de leer y sencillo de mantener.

7.1. Uso de un servidor falso

Como bien anticipaba Kent Beck, nuestro primer caso de prueba va a fallar, pues ejercitará una funcionalidad que aún no existe. Además, tampoco tenemos creada ninguna infraestructura complementaria: ni servidor web, ni base de datos con palabras, ni nada.

Por ello, escribiremos nuestro primer caso de prueba utilizando un servidor falso: un objeto *mock* (literalmente: de imitación, sustituto), que simulará el comportamiento de otro objeto del que, por algún motivo, no disponemos en la etapa en que nos encontremos, o no podemos utilizar, o cualquier otra causa. Existen mocks predefinidos para muchos objetos habituales, aunque también podemos crear los que necesitemos.

Comenzamos añadiendo una clase nueva, a la que llamaremos *TestSelectWord,* a la carpeta del proyecto que contendrá los tests:

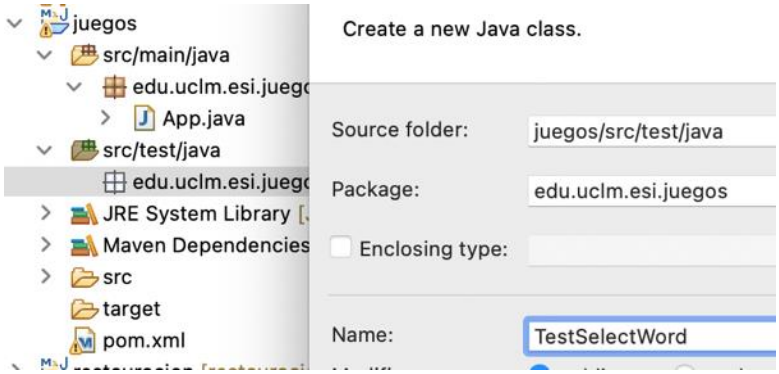


Figura 105. Adición de la clase TestSelectWord

En esta clase escribimos el siguiente código (Figura 106), en el que encontramos varias líneas interesantes:

1) La clase está anotada con *@SpringBootTest*: esto le indica a Spring que la aplicación puede arrancar por aquí.
2) El campo *server* es de tipo *MockMvc,* lo que indica que este servidor es un servidor "de mentira": no correrá realmente sobre un servidor web.
3) Disponemos de un método *test1* anotado con *@Test:* esto le indica a JUnit que este método implementa un caso de prueba.

```java
package edu.uclm.esi.juegos;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.junit.jupiter.SpringExtension;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@ExtendWith(SpringExtension.class)
@AutoConfigureMockMvc
public class TestSelectWord {

   @Autowired
   private MockMvc server;

   @Test
   void test1() {

   }
}
```

Figura 106. Código inicial de TestSelectWord

Si hacemos clic con el botón derecho sobre el fichero *TestSelectWord* y elegimos *Run As,* aparecerá la opción *JUnit Test:* es decir, podemos ejecutar este fichero con JUnit. Si le damos a esta opción, sin embargo, nos aparecerá por la consola de Eclipse una excepción bastante grande (Figura 107), que viene a decirnos que no hemos indicado la URL en la que se encuentra la base de datos.

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Failed to configure a DataSource: 'url' attribute is not specified and no embedded datasource could be configured.

Reason: Failed to determine a suitable driver class


Action:

Consider the following:
        If you want an embedded database (H2, HSQL or Derby), please put it on the classpath.
        If you have database settings to be loaded from a particular profile you may need to activate it (no profiles
```

Figura 107. Error obtenido al intentar ejecutar TestSelectWord

Y es cierto: hemos creado la carpeta *src/main/resources,* pero no hemos creado el fichero *application.properties* en el que debemos especificar este valor. El lector curioso se preguntará que, bueno, tal vez alguna aplicación no necesite base de datos. Por supuesto, puede ocurrir así, y bastaría entonces, para que el caso de prueba se ejecutara, con eliminar o comentar la dependencia a *jpa (Java Persistence API)* que tenemos en el fichero *pom.xml:*

```
41              <scope>runtime</scope>
42          </dependency>
43          <!-- <dependency>
44              <groupId>org.springframework.boot</groupId>
45              <artifactId>spring-boot-starter-data-jpa</artifactId>
46          </dependency> -->
47
48          <dependency>
```

Figura 108. La dependencia a JPA requiere un acceso a una base de datos

Ahora sí, el único caso de prueba que tenemos se ejecuta y supera (claro, porque su implementación está vacía):

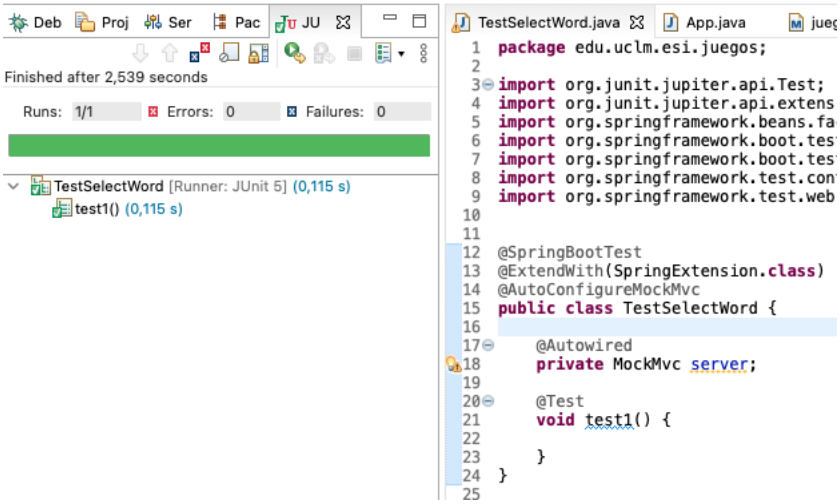


Figura 109. Sin JPA, el caso se ejecuta y se supera

Siguiendo el flujo de mensajes de la Figura 104, enviaremos al *WordRaceController* una petición a su recurso *selectWord* con el parámetro *length=5*. La petición se enviará desde *TestSelectWord* a través del falso *server* que hemos definido.

En la librería mock disponemos del objeto *MockMvcRequestBuilders,* que permite crear una petición *http* de cualquier tipo. En nuestro caso, será una petición de tipo *get* dirigida a la url *words/selectWord?length=5*.

En la Figura 110 se muestra el código completo de la clase. En particular, nos interesa el código del caso de prueba implementado en el método *test1:* se construye una petición de tipo *get* mediante la llamada a *MockMvcRequestBuilders.get*, que recibe como parámetro la url a la que se dirige la petición. La última línea de este método envía la petición y comprueba que el código de respuesta recibido sea un 200, lo que denotaría que la petición se ha servido correctamente.

```
package edu.uclm.esi.juegos;

import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.junit.jupiter.SpringExtension;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.RequestBuilder;
import org.springframework.test.web.servlet.request.MockMvcRequestBuilders;

@SpringBootTest
@ExtendWith(SpringExtension.class)
@AutoConfigureMockMvc
public class TestSelectWord {

   @Autowired
   private MockMvc server;

   @Test
   void test1() throws Exception {
      RequestBuilder request = MockMvcRequestBuilders.
         get("/words/selectWord?length=5");

      this.server.perform(request).andExpect(status().isOk());
   }
}
```

Figura 110. Nuevo código de TestSelectWord

Obviamente, no tenemos todavía ningún controlador, por lo que el caso de prueba emitirá veredicto de fallo (es decir, JUnit mostraría la barra roja) si lo ejecutáramos.

7.2. Implementación del controlador

Para que el caso de prueba se supere, debemos crear un recurso que sea accesible mediante la url *words/selectWord* y que tome un parámetro numérico llamado *length*.

Además, y retomando el esbozo de diseño arquitectónico para *backends* desarrollados con Spring que apuntábamos en la Figura 91 (página 77), este recurso será accesible mediante un controlador, que pasará la operación a un servicio.

Supondremos que el jefe de proyecto, igual que nos imponía un determinado diseño de clases para el *front-end*, nos ha impuesto el diseño arquitectónico de la Figura 91 para el *back*. Para adecuarnos a este diseño, crearemos varias carpetas en la zona del código de producción de nuestro proyecto Eclipse (Figura 111):

1) Carpeta *http* para los controladores.
2) Carpeta *services* para los servicios.
3) Carpeta *entities* para las entidades.
4) Carpeta *dao* para los repositorios.

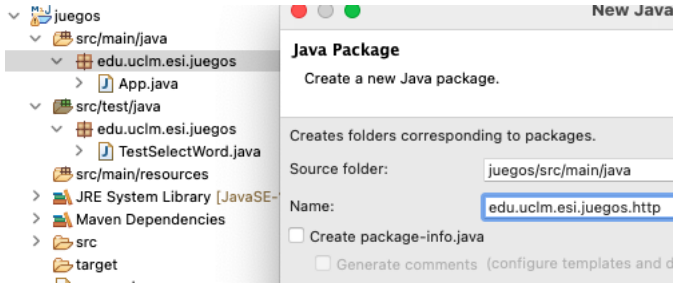


Figura 111. Creación del paquete http *en nuestro backend*

En el paquete *http* crearemos el controlador que se muestra en la Figura 112:

1) La anotación *@RestController* le indica a Spring que esta clase recibe mensajes *http* y devuelve respuestas con cuerpo (véase estructura de una *response* en el Cuadro I, página 84).
2) La anotación *@RequestMapping("words")* indica el nombre público con el que este controlador se da a conocer en la red. Hay que tener en cuenta que este controlador está contenido en un servidor de aplicaciones, que a su vez estará corriendo en alguna *ip* en algún puerto: el nombre completo de este controlador será la *ip* o el nombre del servidor, el puerto y, finalmente, la palabra *words*.
3) El método *selectWord* está anotado con *@GetMapping("/selectWord")*, lo que hace que este método se exponga como un servicio web de tipo *get* (por la anotación *@GetMapping)* cuyo nombre público es *selectWord* (realmente, irá concatenado después de la *ip* o al nombre del servidor y del nombre del controlador: *.../words/selectWord*).
4) El único parámetro del método (*int length)* está anotado con *@RequestParam:* esto les indica a los interceptores de Spring (Figura 101) que, cuando reciba una petición para este recurso, busque en la URL

un parámetro llamado *length* y que, si este existe, lo intente convertir a un *int* y lo "inyecte" en el parámetro.

```
package edu.uclm.esi.juegos.http;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("words")
public class WordRaceController {

    @GetMapping("/selectWord")
    public String selectWord(@RequestParam int length) {
        return "SACOT";
    }

}
```

Figura 112. Código inicial del WordRaceController

Si el parámetro *length* no viniese en la URL (si la petición fuese, por ejemplo, *words/selectWord* o *words/selectWord?longitud=5*), la petición pasaría por los interceptores de Tomcat (vuelva a mirarse la Figura 101), que pasaría a los de Spring, pero los de Spring la rechazarían antes de que el método llegase a ejecutarse. Probemos esto utilizando cualquier navegador*:*

1) Primero, y ya que tenemos el controlador, debemos lanzar la aplicación en modo de producción, no de tests. Para ello, modificamos el contenido de la clase *App* (contenida en *src/main/java)* y escribimos el siguiente código (Figura 113).

```
package edu.uclm.esi.juegos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.servlet.ServletComponentScan;
import org.springframework.boot.web.servlet.support.SpringBootServletInitializer;

@SpringBootApplication
@ServletComponentScan
public class App extends SpringBootServletInitializer {

    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder builder) {
        return builder.sources(App.class);
    }
}
```

Figura 113. Clase App*, por la que arrancará la aplicación*

2) En Eclipse, pulsamos con el botón derecho sobre la clase *App* n derecho sobre la clase *App* y elegimos *Run As, Java Application*. La consola de Eclipse nos anunciará que la aplicación está escuchando en el puerto 8080 (Figura 114).

```
pping  : Mapped URL path [/**] onto handler of type [c
       : LiveReload server is running on port 35729
       : Registering beans for JMX exposure on startup
erver  : Tomcat started on port(s): 8080 (http) with c
       : Started App in 1.896 seconds (JVM running for
```

Figura 114. El puerto de escucha por defecto es el 8080

3) En un navegador cualquiera, escribimos *localhost:8080/words/selectWord* en la barra de direcciones. El resultado que obtendremos lo devuelven los interceptores de Spring y es el que se muestra en la Figura 115.

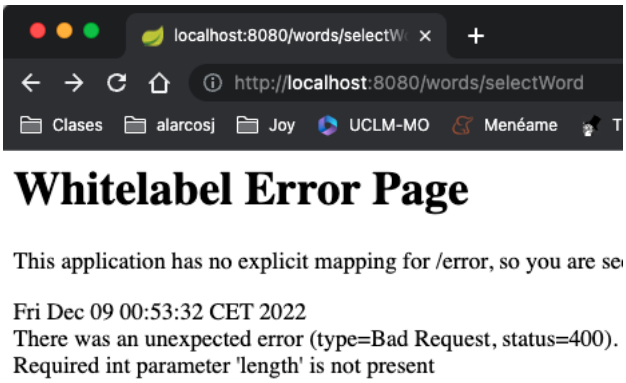


Figura 115. Error que obtenemos al no pasar el parámetro length

4) Si la URL que escribimos en el navegador incluye el parámetro *length* con cualquier valor numérico, se nos devuelve la palabra SACOT (Figura 116), que es la respuesta que hemos programado en la Figura 112.

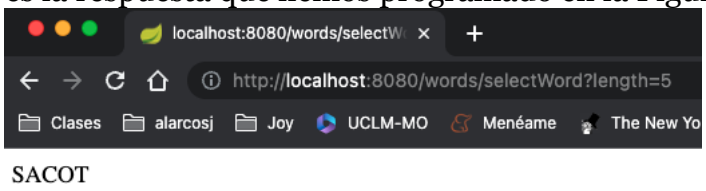
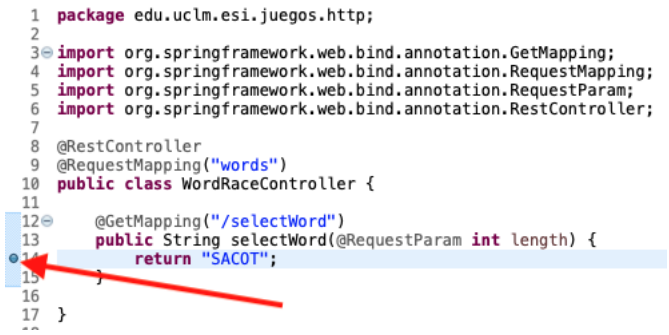


Figura 116. Respuesta obtenida al enviar la petición con un valor entero para el parámetro length

Observando la pila de interceptores

Este ejemplo nos viene bien para ilustrar esos interceptores de Tomcat y Spring que ilustrábamos en el diagrama de secuencia de la Figura 101 (página 84): paremos el servidor en Eclipse y pongamos un punto de interrupción en la línea 14 de *WordRaceController* (Figura 117).



```
1  package edu.uclm.esi.juegos.http;
2
3  import org.springframework.web.bind.annotation.GetMapping;
4  import org.springframework.web.bind.annotation.RequestMapping;
5  import org.springframework.web.bind.annotation.RequestParam;
6  import org.springframework.web.bind.annotation.RestController;
7
8  @RestController
9  @RequestMapping("words")
10 public class WordRaceController {
11
12     @GetMapping("/selectWord")
13     public String selectWord(@RequestParam int length) {
14         return "SACOT";
15     }
16
17 }
```

Figura 117. Colocamos un punto de interrupción en selectWord

Ahora, lancemos la aplicación en modo depuración: botón derecho sobre la clase *App* y, en lugar de *Run As,* elegimos *Debug As, Java Application* (Figura 118).

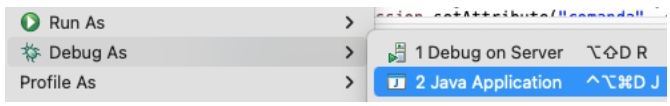


Figura 118. Lanzamos la aplicación en modo depuración

Al haber colocado un punto de interrupción en esa línea, esperamos que la ejecución se detenga en ese punto para poder comprobar qué es lo que está sucediendo.

1) Si escribimos *localhost:8080/words/selectWord* en la barra de direcciones del navegador, ¿se detendrá ahí la ejecución de la aplicación? La respuesta es que no: la petición salta los interceptores de Tomcat y pasa a los de Spring, pero estos no son capaces de inyectar nada en el parámetro *length,* puesto que la URL solicitada no lo incluye.
2) Si escribimos una URL válida, como *http://localhost:8080/words/selectWord?length=5,* la petición pasa los interceptores del servidor de aplicaciones, pasa los de Spring y llega al método *selectWord* del controlador. La Figura 119 ilustra este resultado: en el lado derecho, la ejecución se ha detenido en la sentencia sobre la que hemos colocado el *breakpoint;* en su lado izquierdo aparece la pila de llamadas que se ha ido construyendo según se iba procesando la petición: se ve que la ejecución está detenida en *WordRaceController.selectWord(int)*, pero que previamente se han ejecutado múltiples operaciones de otras clases que, en principio, nos interesan.

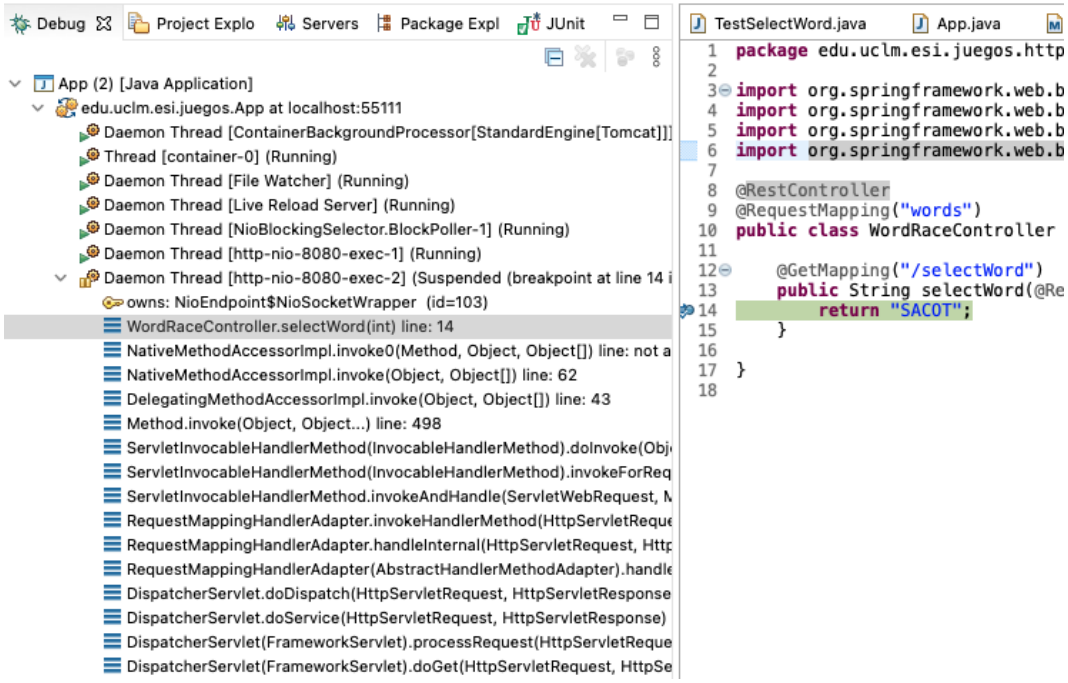


Figura 119. Con una petición válida, la petición pasa por todos los interceptores y llega al controlador

Cuadro III. Observando la pila de interceptores

Ahora, si ejecutamos el caso de prueba con JUnit, se nos muestra la barra verde, indicando que este se está superando (Figura 120).

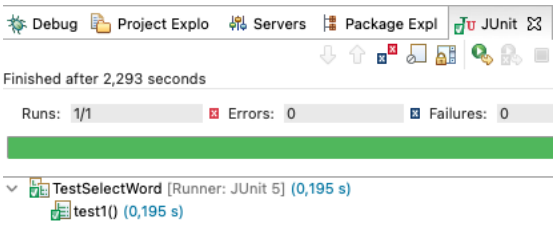


Figura 120. Ahora, el caso de prueba contenido en TestSelectWord *se supera*

El hecho de que un caso de prueba se supere no indica que el código de producción no contenga errores, sino que no se viola la condición o condiciones establecidas en dicho caso. En nuestro ejemplo, el código del caso de prueba solo requiere que se obtenga un código 200 (OK, "todo ha ido bien") para una petición sencilla:

```
    void test1() throws Exception {
        RequestBuilder request = MockMvcRequestBuilders.
            get("/words/selectWord?length=5");

        this.server.perform(request).andExpect(status().isOk());
    }
```

Figura 121. El caso se supera porque solo pregunta si se ha recibido un código 200

7.3. Paso del mensaje al servicio

De acuerdo con nuestro estilo arquitectónico y con el paso de mensajes, el controlador debe pasar la búsqueda de la palabra a un servicio. En la siguiente figura volvemos a mostrar el flujo de mensajes que deseamos.

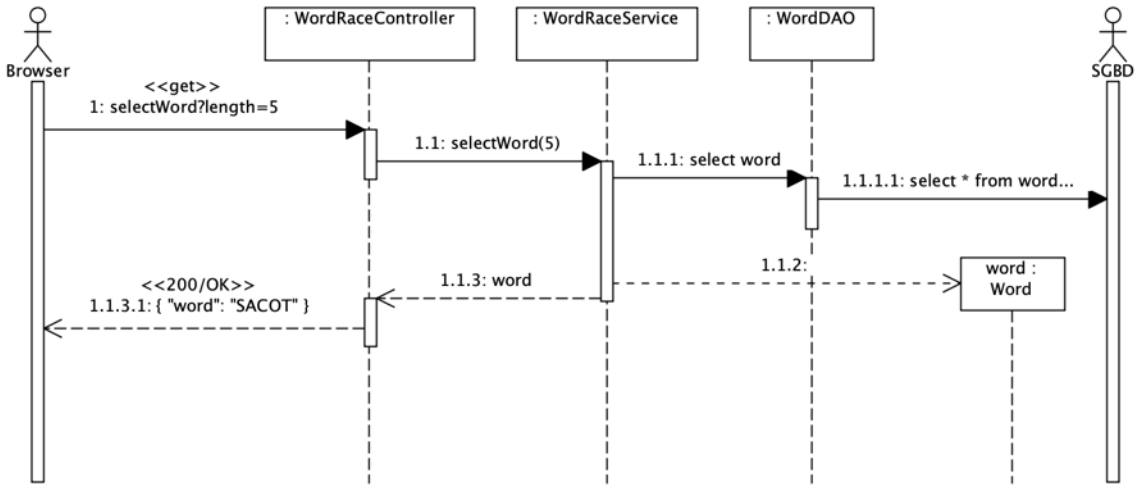


Figura 122. Flujo de mensajes para el servicio selectWord

Como se ve, el *WordRaceController* conoce a un *WordRaceService*, sobre el que ejecuta la operación *selectWord,* cuyo resultado es el que se devuelve al *browser*. De momento, haremos la siguiente implementación del *WordRaceController* (Figura 123):

1) El método *selectWord* recibe como parámetro *length* la longitud deseada de la palabra que debe devolverse aleatoariamente. Como vemos, el controlador le pide directamente al servicio (línea 17) que haga este cómputo.
2) El servicio está declarado en la línea 13 como el campo *wordRaceService* de tipo *WordRaceService,* y está anotado como *@Autowired*.

3) Si todo fuera bien (hay tres errores de compilación que debemos subsanar), el método *selectWord* de este controlador devolvería una palabra aleatoria al *front-end*.

```
 8  @RestController
 9  @RequestMapping("words")
10  public class WordRaceController {
11
12      @Autowired
13      private WordRaceService wordRaceService;
14
15      @GetMapping("/selectWord")
16      public String selectWord(@RequestParam int length) {
17          return this.wordRaceService.selectWord(length);
18      }
19
20  }
```

Figura 123. El controlador delega la responsabilidad al servicio

Sucede que ni tenemos la clase *WordRaceService* (línea 13), ni en esta el método *selectWord* al que llamamos (línea 17) ni sabemos qué quiere decir lo de *@Autowired*. Esto se explica en el cuadro siguiente.

Cómo funciona Spring Boot (I)

El siguiente trozo de código muestra una clase habitual de arranque de una aplicación con Spring Boot: la función *main* nos permite arrancar la aplicación desde consola o desde nuestro entorno de desarrollo; la que aparece más abajo se utiliza cuando la aplicación se va a desplegar en un servidor de aplicaciones en forma de *war*.

Fijándonos en el ciclo iniciado con el *main* (es similar en el otro caso), cuando la aplicación arranca, se llama al método estático *SpringApplication.run*, que recibe como parámetro la clase que, precisamente, contiene la propia función *main*.

Spring, entonces, utilizando programación reflexiva, observa el paquete en el que la clase está definida (*edu.uclm.esi.juegos* en este caso), e inspecciona todas las clases que encuentre en ese paquete y en sus subpaquetes.

```
p package edu.uclm.esi.juegos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.servlet.ServletComponentScan;
import org.springframework.boot.web.servlet.support.SpringBootServletInitializer;

@SpringBootApplication
@ServletComponentScan
public class App extends SpringBootServletInitializer {

    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder builder) {
        return builder.sources(App.class);
    }
}
```

Spring Boot inspecciona cada clase que encuentre en esta estructura, y comprueba si la clase tiene alguna anotación (*@RestController, @Service* , etcétera*):* en caso negativo, pasa de ella; en caso afirmativo, la "inspecciona".

"Inspeccionar la clase" significa inspeccionar sus miembros. Si, por ejemplo, en un *@RestController* se encuentra un campo anotado como *@Autowired*, Spring comprueba que el tipo de ese campo sea una clase "autowireable".

Esto es exactamente lo que estábamos añadiendo cuando hemos llegado a este cuadro, que reproducimos de nuevo:

```
 8  @RestController
 9  @RequestMapping("words")
10  public class WordRaceController {
11
12      @Autowired
13      private WordRaceService wordRaceService;
14
15      @GetMapping("/selectWord")
16      public String selectWord(@RequestParam int length) {
17          return this.wordRaceService.selectWord(length);
18      }
19
20  }
```

En el *WordRaceController,* que es un *@RestController,* declaramos un campo *@Autowired* de tipo *WordRaceService* (clase que anotaremos con *@Service).* Cuando la aplicación arranca, Spring revisa este controlador (porque está anotado con *@RestController)* y encuentra el campo de tipo *WordRaceService*. Como el campo está anotado con *@Autowired* y el tipo es (o va a ser, pues aún no lo hemos implementado) un *@Service,* Spring Boot llamará automáticamente al constructor sin parámetros de este servicio.

Si olvidásemos anotar el servicio con *@Service,* el objeto no podría ser creado automáticamente, por lo que Spring Boot detendría la ejecución y terminaría.

Cuadro IV

7.4. Implementación del servicio

Dentro de la zona del código de producción (*src/main/java)* crearemos la clase *WordRaceService* en el paquete *edu.uclm.esi.juegos.services*. Por ahora, podemos desplazar la devolución de la palabra SACOT a la nueva operación *selectWord* del servicio:

```
package edu.uclm.esi.juegos.services;

import org.springframework.stereotype.Service;

@Service
public class WordRaceService {

   public String selectWord(int length) {
      return "SACOT";
   }
}
```

Figura 124. Primera implementación del servicio

Si ejecutamos el caso de prueba, este se sigue superando, pues lo único que seguimos comprobando es que se obtenga una respuesta con código 200.

Volviendo al paso de mensajes de la Figura 122, el servicio debe ahora acceder a la base de datos a través de un repositorio (la clase *WordDAO,* que aun debemos implementar). De acuerdo con esa figura, tras recuperar la información de la base de datos, el servicio debe crear una instancia de *Word* (que será una entidad) y devolverla con sus letras desordenadas al controlador.

7.5. Creación del repositorio y de la entidad

Podemos asumir que, en la base de datos, dispondremos de una tabla llamada *word* en la que estarán todas las palabras (o muchas) en castellano. En este punto debemos tomar una pequeña decisión técnica: cada fila de la tabla *word* tendrá dos columnas: un entero autonumérico que será la clave principal, y una columna de tipo *varchar* que contendrá el texto de la palabra.

La entidad *Word* será tan sencilla como esa tabla: será una clase que se llamará *Word* y tendrá dos campos, un *Integer* para el *id* (en las entidades se prefiere usar *Integer, Double,* etcétera en lugar de *int, double* por la posible existencia de valores *null* en la tabla asociada a la entidad), y un campo *word* para el texto. Las entidades son clases anotadas con *@Entity* y deben tener siempre uno o más campos que se corresponderán con la clave principal de la tabla asociada. Además, deben tener métodos *get* y *set* para uno de los campos que correspondan a columnas de la tabla asociada.

De momento, el código de la entidad *Word* será el de la siguiente figura, en el que observamos que se nos marcan dos errores, ya que Eclipse no conoce ni la anotación *@Entity* ni *@Id*. Tampoco se nos da opción a importar estas anotaciones de ningún lugar, y es porque, hace unas páginas (en la Figura 108), comentamos en el fichero *pom.xml* la dependencia JPA, que es precisamente en donde reside la definición de estas dos anotaciones.

```
1  package edu.uclm.esi.juegos.entities;
2
3  @Entity
4  public class Word {
5      @Id
6      private Integer id;
7      private String word;
8
9      public Integer getId() {
10         return id;
11     }
12
13     public void setId(Integer id) {
14         this.id = id;
15     }
16
17     public String getWord() {
18         return word;
19     }
20
21     public void setWord(String word) {
22         this.word = word;
23     }
24 }
25
```

Figura 125. Código inicial de Word

Así pues, regresamos al *pom.xml*, descomentamos la dependencia y, ya sí, podemos importar las anotaciones *@Entity* e *@Id* como en la Figura 126. Si usásemos una versión anterior de Spring, importaríamos de *javax.persistence* en lugar de *jakarta.persistence,* como se muestra en la Figura 127.

```
1  package edu.uclm.esi.juegos.entities;
2
3  import jakarta.persistence.Entity;
4  import jakarta.persistence.Id;
5
6  @Entity
7  public class Word {
8      @Id
9      private Integer id;
10     private String word;
```

Figura 126. @Entity *e* @Id *residen en la dependencia JPA*

En versiones anteriores de **Spring Boot 3.0.1** se utilizaba el paquete *javax* en lugar del *jakarta*. El único cambio necesario es sustituir los *import*:

```
1  package edu.uclm.esi.juegos.entities;
2
3  import javax.persistence.Entity;
4  import javax.persistence.Id;
5
6  @Entity
7  public class Word {
8      @Id
9      private Integer id;
10     private String word;
```

Figura 127. Spring 3.0.1 utiliza el paquete jakarta *en lugar de* javax

Recordemos, por otro lado, que eliminamos la dependencia JPA porque la aplicación no arrancaba: si esta dependencia está presente, Spring Boot busca una base de datos a la que conectarse y, como no la tenemos de momento, detiene el arranque de la aplicación y muestra el error de la Figura 107 (página 89).

Al haber vuelto a añadir la dependencia y no disponer de base de datos, la aplicación tampoco arrancará; pero de momento no nos importa: sigamos con nuestro enfoque TDD, con el que perseguimos escribir código de producción para superar un buen caso de prueba (ya lo completaremos: no podemos conformarnos con obtener solamente un código 200) adhiriéndonos al estilo arquitectónico que se nos haya impuesto o que hayamos elegido, y a su paso de mensajes asociado.

En su última implementación, el método *selectWord* del servicio devolvía directamente la palabra SACOT. Ahora, y de acuerdo con el flujo de mensajes de la Figura 122, el servicio debe acceder a la base de datos a través del repositorio *WordDAO*. La estructura de clases de esta parte del sistema será la siguiente:

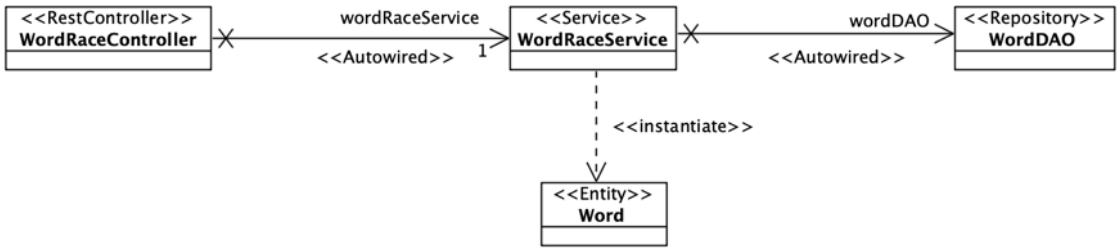


Figura 128. Relación entre los diferentes elementos del Word race

Debemos crear el repositorio en el paquete *edu.uclm.esi.juegos.dao*. Como vemos en la siguiente figura, un repositorio no es una clase, sino una interfaz que, en principio, no requiere de la declaración de ningún método.

Como vemos también, el repositorio toma dos tipos como parámetros: *Word*, que es la entidad que maneja este repositorio, e *Integer*, que es el campo de la entidad (anotado con *@Id*) que se corresponde con la clave principal.

```
package edu.uclm.esi.juegos.dao;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import edu.uclm.esi.juegos.entities.Word;

@Repository
public interface WordDAO extends JpaRepository <Word, Integer> {

}
```

Figura 129. Código inicial del repositorio WordDAO

A pesar de su triste contenido, el *WordDAO* es un elemento potentísimo, que incluye multitud de operaciones para manipular las entidades de tipo *Word:* si nos fijamos, *WordDAO* es un *JpaRepository* (una interfaz de Spring) que, a su vez, incluye operaciones heredadas de otras interfaces. Como se ve en la Figura 130, nuestro repositorio de gestión de entidades de tipo *Word* incluye todas las operaciones de:

1) *CrudRepository:* entre otras, están las operaciones *save*, que guarda el objeto en la tabla asociada a la entidad mediante un *insert* o un *update,* según la instancia sea nueva o ya existiera; *findById,* que busca en la tabla asociada una fila cuya clave principal coincida con el valor pasado como parámetro; *count,* que devuelve el número de filas en la tabla, etcétera.
2) *PagingAndSortingRepository,* que permite recuperar datos de una tabla paginadamente (es decir, de varios en varios).
3) *QueryByExampleExecutor,* que permite realizar cómodamente varios tipos de consultas.
4) *JpaRepository,* que ofrece otro amplio conjunto de métodos.

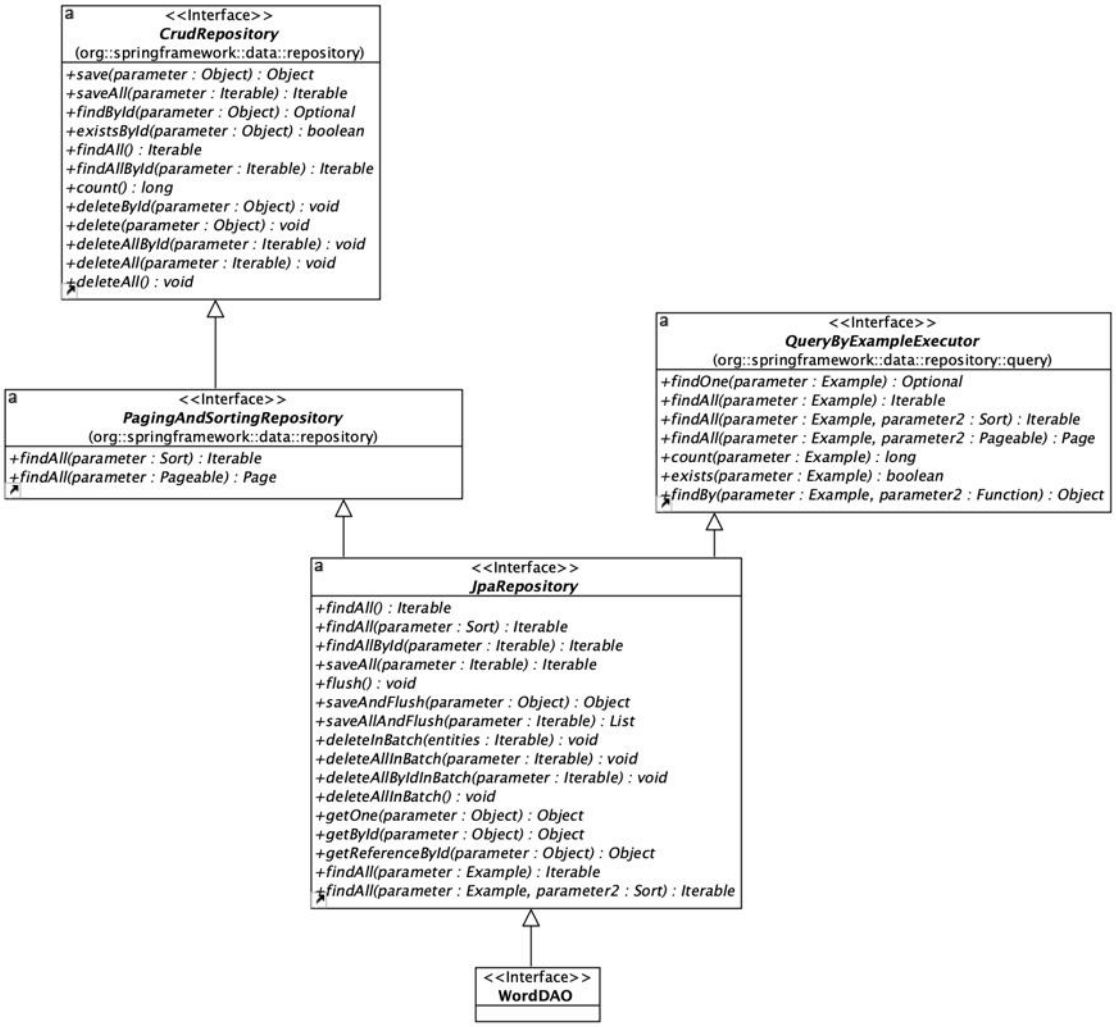


Figura 130. El WordDAO *hereda multitud de operaciones*

Para terminar de modelar el sistema y dejarlo como en la Figura 128, debemos declarar el campo *wordDAO* de tipo *WordDAO* en el *WordRaceService,* y anotarlo como *@Autowired*. Todavía tenemos pendiente decidir la forma en

que seleccionaremos la palabra aleatoria desde la tabla en la que almacenemos las palabras: de momento, y aunque es un mecanismo altamente ineficiente y puede mejorarse, desde el servicio recuperaremos una lista con todas las palabras de la tabla, eliminaremos aquellas cuya longitud sea distinta de la deseada, seleccionaremos una palabra al azar de entre las restantes, la desordenaremos y la devolveremos al controlador, para que este a su vez la entregue al *user-agent* que la solicitó.

Todo esto es lo que se implementa en la Figura 131:

1) Declaramos el *wordDAO*.
2) Recuperamos todas las filas de la tabla *word* mediante el método *findAll* que el repositorio *WordDAO* hereda de *JpaRepository*.
3) Recorremos la lista de palabras de arriba abajo y vamos eliminado aquellas cuya longitud no sea la deseada.
4) Guardamos en *pos* un número aleatorio entre 0 y el número de palabras que nos quedan.
5) Recuperamos la palabra que está en la posición *pos* de la lista.
6) Desordenamos la palabra.
7) La devolvemos.

```
@Service
public class WordRaceService {

   @Autowired
   private WordDAO wordDAO;

   public Word selectWord(int length) {
      List<Word> words = this.wordDAO.findAll();

      for (int i=words.size()-1; i>=0; i--)
         if (words.get(i).getWord().length()!=length)
            words.remove(i);

      int pos = new SecureRandom().nextInt(words.size());
      Word word = words.get(pos);
      word.disorder();

      return word;
   }

}
```

Figura 131. Código (provisional) de selectWord

Del código de la figura anterior destacamos dos cosas:

1) El tipo devuelto por el método ya no es *String*, sino *Word*. En el controlador, también deberemos recuperar este resultado en forma de *Word*.
2) La desordenación de las letras de la palabra la estamos delegando a un método *disorder* de la entidad *Word*. Crear este método en *Word* es, por supuesto, perfectamente legal y válido, pero las entidades suelen ser objetos "POJO" (*Plain Old Java Object)* que disponen solamente de métodos *get* y *set* para sus campos. Puesto que el acto de desordenar la palabra no es una operación *get/set*, y si queremos mantener la entidad como un POJO, tal vez deberíamos sustituir la sentencia *word.disorder()* por *this.disorder(word):* es decir, en lugar de que sea la palabra la responsable de desordenar sus propias letras, delegamos esta responsabilidad al servicio. Elegiremos esta segunda opción para mantener *Word* como un POJO. Por tanto, reescribimos el código de *WordRaceService* y lo dejamos como en la Figura 132.

```
@Service
public class WordRaceService {

    @Autowired
    private WordDAO wordDAO;

    public Word selectWord(int length) {
        ...

        Word word = words.get(pos);
        this.disorder(word);

        return word;
    }

    private void disorder(Word word) {
        char[] disordered = word.getWord().toCharArray();
        int length = disordered.length;
        char aux;
        SecureRandom dice = new SecureRandom();
        for (int i=0; i<length*5; i++) {
            int posA, posB;
            do {
                posA = dice.nextInt(length);
                posB = dice.nextInt(length);
            } while (posA==posB);

            aux = disordered[posA];
            disordered[posA] = disordered[posB];
            disordered[posB] = aux;
        }
        word.setDisordered(new String(disordered));
    }

}
```

Figura 132. Inclusión del método disorder(Word) *en el servicio*

La última sentencia de *disorder* ejecuta *setDisordered* sobre la instancia *word*. Este método no existe (la clase *Word* es un POJO y solo tiene dos métodos *get* y dos métodos *set,* que leen y asignan valor a los dos campos que tiene, y que se corresponden con las columnas de la tabla *word* de la base de datos), pero vamos a crearlo: *setDisordered* asigna a un nuevo campo de la clase *Word* (al que llamaremos *disordered)* las letras desordenadas de la palabra. Habrá que crear el campo en la clase y también un método *getDisordered*.

Este nuevo campo *disordered*, sin embargo, no se corresponderá con ninguna columna de la tabla *word* (las letras desordenadas se calculan en tiempo de ejecución y no hay necesidad de que se almacenen en la tabla), por lo que anotaremos el campo como *@Transient*. De este modo, la entidad *Word* queda ahora como sigue:

```java
package edu.uclm.esi.juegos.entities;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Transient;

@Entity
public class Word {
   @Id
   private Integer id;
   private String word;

   @Transient
   private String disordered;

   public Integer getId() {
      return id;
   }

   public void setId(Integer id) {
      this.id = id;
   }

   public String getWord() {
      return word;
   }

   public void setWord(String word) {
      this.word = word;
   }

   public void setDisordered(String disordered) {
      this.disordered = disordered;
   }

   public String getDisordered() {
      return disordered;
   }
}
```

Figura 133. Nueva implementación de Word

Tenemos un error pendiente de arreglar en el *WordRaceController:* el servicio *selectWord* devuelve un *String,* pero ahora debe devolver un *Word*. Bien, cambiamos el tipo devuelto por el método y asunto arreglado.

```java
@GetMapping("/selectWord")
public Word selectWord(@RequestParam int length) {
   return this.wordRaceService.selectWord(length);
}
```

Figura 134. El servicio selectWord *entrega un* Word *al cliente*

En el siguiente cuadro se detalla la forma en que se consigue enviar el objeto de respuesta (de tipo *Word)* al cliente en un formato entendible por este.

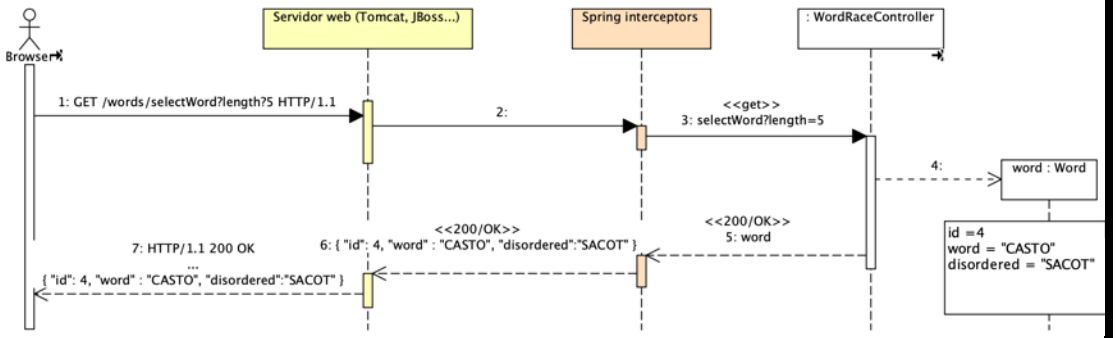
Cómo funciona Spring Boot (II)

Como ya sabemos, antes de que la petición llegue al *WordRaceController,* pasa por una serie de elementos previos a los que hemos llamado interceptores. Estos interceptores transforman progresivamente la petición hasta hacerla entendible y procesable por el método implementado en el controlador.

Cuando se termina de procesar la petición y se va a entregar la respuesta, esta realiza el camino inverso, y va retrocediendo (hasta que llegue al navegador) por todos los interceptores, en el mismo orden.

La figura siguiente ilustra la llegada de la petición al servidor web, de ahí a los interceptores de Spring y, de ahí, a nuestro controlador. De alguna manera (obviamos los detalles del servicio, repositorio, etcétera) se construye una instancia de tipo *Word,* que es el tipo del dato que se devolverá al cliente.

Obviamente, el tipo *Word* pertenece al *backend,* está definido en su paquete *edu.uclm.esi.juegos.entities.Word* y, para que sea enviable al cliente y este pueda entenderlo, debe ser transformado a algún formato de intercambio válido.



Cuando el controlador termina de ejecutarse, entrega a los interceptores de Spring el objeto *word* de clase *Word*. Estos interceptores se encargan de convertir el objeto a ese formato válido para viajar hasta el cliente.

Por defecto, los interceptores convierten a un JSON el objeto que han de devolver, llamando a todos los métodos públicos que comiencen por *get*. Así, cuando los interceptores van a convertir la instancia de *Word* mostrada en el lado derecho de la figura de arriba, ejecutan sobre ella los métodos *getId, getWord* y *getDisordered,* y colocan en el JSON que van a devolver el resultado de estos tres métodos.

Cuadro V. Cómo funciona Spring Boot (II)

7.6. Creación de la base de datos

Como tenemos incluida la dependencia JPA y no tenemos base de datos, Spring interrumpe el arranque de la aplicación, porque no encuentra base de datos a la que conectarse.

Estando como estamos aplicando un enfoque TDD, podemos crear una base de datos que se utilice solo durante la fase de pruebas, que tenga unas pocas palabras y que nos facilite la creación de casos de prueba.

Utilizaremos para esto una base de datos en memoria, que no requiere la instalación de ninguna aplicación ni acceso a ningún sistema físico. Para ello, incluiremos en el fichero *pom.xml* una dependencia para poder utilizar una base de datos de este tipo (Figura 135). Obsérvese en la figura que restringimos su aplicación a la fase de *test*.

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
</dependency>
```

Figura 135. Dependencia para usa la base de datos h2

h2 es una base de datos que entiende SQL. Para poder utilizarla, debemos especificar algunos parámetros en el fichero de configuración *application.properties* que, en este caso, colocaremos en la carpeta *src/test/resources* (Figura 136, izquierda) con los contenidos que se muestran en el lado derecho:

1) Primero, indicamos que el puerto de escucha del servidor será el 80.
2) Luego, especificamos la URL de la base de datos.
3) Con *spring.jpa.hibernate.ddl-auto = create* le decimos a Spring que cree la base de datos cuando la aplicación arranque. ¿Cómo la crea? Bien, pues revisa todas las clases que tengan la anotación *@Entity*, explora cada una y genera automáticamente la instrucción DDL (*Data Definition Language)* que corresponda, como *create table word…*
4) Utilizamos las dos últimas líneas para que Spring nos muestre las instrucciones SQL que vaya generando.

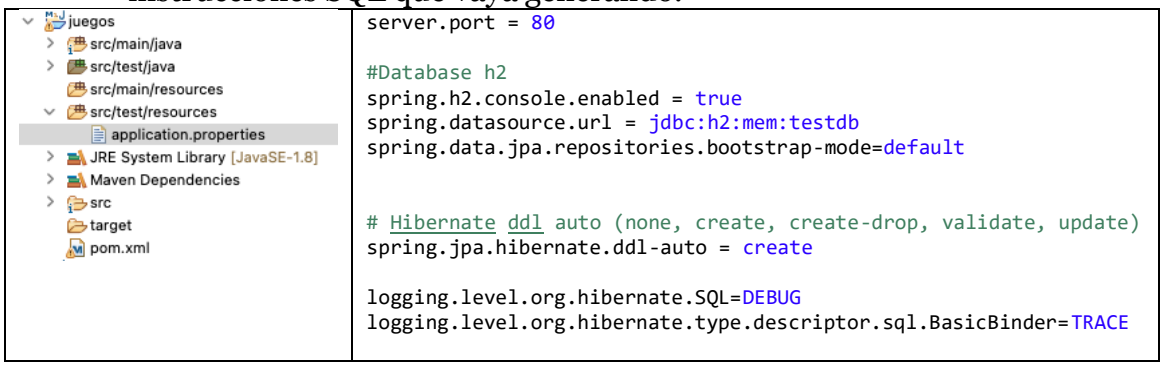


Figura 136. application.properties *para las pruebas*

Ahora, si ejecutamos el caso de prueba, Spring arranca correctamente la aplicación, pues encuentra en *application.properties* toda la información que necesita para conectar a la base de datos. De hecho, en la consola de Eclipse vemos dos líneas interesantes: una que borra (si es que ya existe) la tabla *word* y otra que la crea (Figura 137). Si el valor asignado a *spring.jpa.hibernate.ddl-auto* hubiese sido, por ejemplo, *none,* la tabla no se habría creado. Igualmente, las dos instrucciones SQL que se resaltan en la figura aparecen en la consola por que así lo hemos especificado en las dos últimas líneas de *application.properties*.

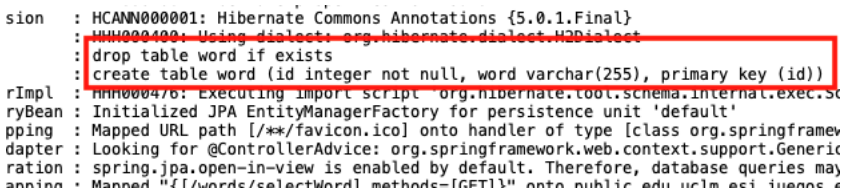


Figura 137. Creación de la tabla de palabras

No obstante, y aunque Spring arranque, el caso de prueba falla: claro, no hay palabras en la base de datos y el caso de prueba está solicitando una de 5 letras. Se llega a llamar al método *selectWord* del servicio, que obtiene una lista de palabras vacía y, claro, cuando poco después genera un número aleatorio entre 0 y el número de palabras de la lista, el método falla.

7.7. Inserción de palabras en la base de datos

Para que el caso de prueba se supere debemos disponer de, al menos, una palabra de 5 letras en la tabla *word*. Lo que haremos será insertar algunas palabras en la base de datos antes de ejecutar el caso de prueba. Para ello,

añadimos un método que anotamos con *@BeforeEach,* para que se ejecute antes de cualquier caso de prueba (es decir: antes de cualquier método anotado con *@Test).*

```
@SpringBootTest
@ExtendWith(SpringExtension.class)
@AutoConfigureMockMvc
public class TestSelectWord {

   @Autowired
   private MockMvc server;

   @Autowired
   private WordDAO wordDAO;

   @BeforeEach
   void setUp() {
      String[] ss = new String[] {
            "ALFA", "BETA", "CASA", "COSA", "CASTO", "LUCERO", "MACARIO"
      };
      for (int i=0; i<ss.length; i++) {
         Word word = new Word();
         word.setId(i);
         word.setWord(ss[i]);
         this.wordDAO.save(word);
      }
   }

   @Test
   void test1() throws Exception {
      RequestBuilder request = MockMvcRequestBuilders.
         get("/words/selectWord?length=5");

      this.server.perform(request).andExpect(status().isOk());
   }
}
```

Figura 138. Un método anotado con @BeforeEach, *que se ejecuta antes de cualquier* @Test

Ahora, podemos ejecutar *TestSelectWord* y comprobaremos que el caso de prueba muestra la barra verde, indicándonos que se está superando.

7.8. Compleción del caso de prueba

Sin embargo, y como vemos en el código del método *test1* de la figura anterior, el oráculo del caso de prueba (el oráculo es la instrucción que determina si el caso de prueba se supera o no) solo comprueba el código de respuesta (200, OK).

Con la nueva implementación que hemos dado al código de producción, sabemos que al *user-agent* le llega como respuesta un objeto de clase *Word* en formato JSON (recuérdese el Cuadro V, página 104) que tendrá tres campos: el *id,* una cadena con la palabra ordenada (campo *word)* y otro con la cadena desordenada (campo *disordered).* Como en la tabla *word* de la base de datos en memoria estamos insertando solamente una palabra de 5 letras (CASTO), en el oráculo del caso podemos comprobar que en el campo *word* de la respuesta se recibe la palabra CASTO y que, en el campo *disordered,* se recibe esta palabra con las letras desordenadas.

En el Cuadro I (página 84) se listaban los elementos de la *response:*

1) Línea de estado con:
    a. Versión del protocolo.
    b. Código de respuesta y mensaje explicativo.

2) Cabeceras.
3) Línea en blanco que indica el fin de la sección de cabeceras.
4) Cuerpo de la respuesta.

Bien, pues a todos ellos podemos acceder desde la biblioteca de objetos de test que nos ofrece Spring, y que tenemos incluida en una de las librerías del fichero *pom.xml*.

Reescribamos poco a poco el caso de prueba, cuyo código inicial reproducimos de nuevo:

```
void test1() throws Exception {
  RequestBuilder request = MockMvcRequestBuilders.
          get("/words/selectWord?length=5");


  this.server.perform(request).andExpect(status().isOk());
}
```

El objeto *request* representa la petición que se va a enviar.

perfom envía la petición que se pasa como parámetro. El resultado es un objeto de tipo *ResultActions* que usaremos para inspeccionar la *response*.

Figura 139. El caso de prueba que tenemos hasta ahora, explicado

Como decimos en la explicación de la figura anterior, *perform* envía la petición y devuelve un *ResultActions,* que nos da acceso a la *response* obtenida. Optaremos, en la Figura 140, por usar *andExpect,* lo que nos permitirá conocer el código de respuesta.

```
@Test
void test1() throws Exception {
    RequestBuilder request = MockMvcRequestBuilders.
        get("/words/selectWord?length=5");

    ResultActions resultActions = this.server.perform(reques
    resultActions.and
                  andDo(ResultHandler handler) : ResultActions - Resu
                  andExpect(ResultMatcher matcher) : ResultActions -
                  andReturn() : MvcResult - ResultActions
```

Figura 140. Algunos métodos del tipo ResultActions

Como vemos en la Figura 141, disponemos de un amplio conjunto de operaciones para recuperar el código de respuesta.

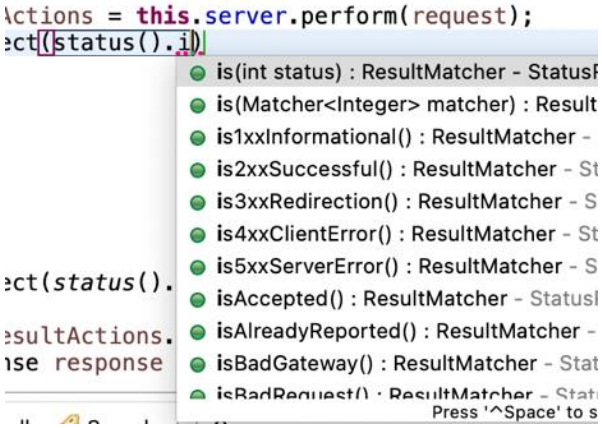


Figura 141. Recuperación del código de respuesta

Si nos fijamos, en la Figura 140 se puede ver que la operación *andExpect* que hemos utilizado devuelve el propio objeto *ResultActions* sobre el que se ha ejecutado la operación. Es decir, en el siguiente ejemplo, las instancias

resultActions y *copyOfResultActions* apuntan al mismo objeto, puesto que *andExpect* es una operación idempotente.

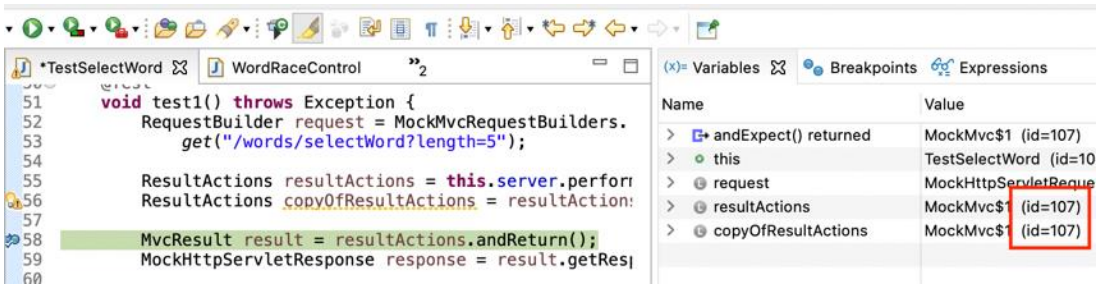
```
@Test
void test1() throws Exception {
   RequestBuilder request = MockMvcRequestBuilders.
      get("/words/selectWord?length=5");

   ResultActions resultActions = this.server.perform(request);
   ResultActions copyOfResultActions = resultActions.andExpect(status().is(200));
   ...
```

Figura 142. La operación andExpect *es idempotente*

Idempotencia de una operación

Podemos comprobar que las dos instancias resaltadas en la Figura 142 son la misma utilizando el depurador. En esta figura se ha colocado un punto de interrupción después de asignar valor a ambas variables, y ahí se ha detenido la ejecución. En el lado derecho vemos el estado de las variables y, en el recuadro, el *id* de cada una, el 107 en este caso, lo que demuestra que las dos instancias apuntan a la misma referencia.



Cuadro VI. Idempotencia de una operación

A partir de este objeto *ResultActions* queremos llegar al cuerpo de la respuesta propiamente dicho, pues es en donde esperamos encontrar el JSON devuelto por el *backend,* que contendrá los campos *id, word* y *disordered*. La *response* es un objeto de tipo *MockHttpServletResponse* que obtendremos a partir del *resultActions* llamando primero a *andReturn* (que devuelve un *MvcResult*) y, luego, a *getResponse* (Figura 143):

```
@Test
void test1() throws Exception {
   RequestBuilder request = MockMvcRequestBuilders.
      get("/words/selectWord?length=5");

   ResultActions resultActions = this.server.perform(request);
   MvcResult result = resultActions.andExpect(status().is(200)).andReturn();
   MockHttpServletResponse response = result.getResponse();

   ...
```

Figura 143. Poco a poco llegamos a la response

A partir de la *response,* podemos recuperar el cuerpo de la respuesta mediante su método *getContentAsString* y utilizar el resultado devuelto para construir el JSON que utilizaremos para comprobar que ha llegado la palabra CASTO en el campo *word,* y una combinación desordenada de estas letras en *disordered*:

```
@Test
void test1() throws Exception {
   RequestBuilder request = MockMvcRequestBuilders.
      get("/words/selectWord?length=5");

   ResultActions resultActions = this.server.perform(request);
   MvcResult result = resultActions.andExpect(status().is(200)).andReturn();
   MockHttpServletResponse response = result.getResponse();

   String responseBody = response.getContentAsString();

   JSONObject jso = new JSONObject(responseBody);

   assertTrue(jso.getString("word").equals("CASTO"));
   assertTrue(this.containsAll("CASTO", jso.getString("disordered")));
}
```

Figura 144. El caso de prueba ahora comprueba el cuerpo recibido de la response

Con el código de la figura anterior, el oráculo del caso sigue comprobando, en primer lugar, que el código de la respuesta es 200; luego, en los dos *assertTrue* del final, se comprueba que el campo *word* contiene CASTO y, mediante el método auxiliar *containsAll(String, String),* que debemos programar nosotros (Figura 145), que el campo *disordered* contiene esas mismas letras.

```
private boolean containsAll(String a, String b) {
   for (int i=0; i<a.length(); i++) {
      if (!this.contains(b, a.charAt(i)))
         return false;
   }
   return true;
}


private boolean contains(String b, char c) {
   for (int i=0; i<b.length(); i++)
      if (b.charAt(i)==c)
         return true;
   return false;
}
```

Figura 145. Métodos auxiliares

Obviamente, el código del caso de prueba puede resumirse y dejarlo, por ejemplo, como en la figura siguiente:

```
@Test
void test1() throws Exception {
   RequestBuilder request = MockMvcRequestBuilders.
      get("/words/selectWord?length=5");

   JSONObject jso = new JSONObject(this.server.perform(request).
      andExpect(status().is(200)).
      andReturn().getResponse().
      getContentAsString());

   assertTrue(jso.getString("word").equals("CASTO"));
   assertTrue(this.containsAll("CASTO", jso.getString("disordered")));
}
```

Figura 146. El caso de prueba de la Figura 144, pero bastante resumido

8. Desarrollo de la funcionalidad *checkWord*

Para implementar esta funcionalidad seguiremos un enfoque similar al de *selectWord*, aunque ya no lo explicaremos tan detalladamente, pues se espera que algo haya aprendido el lector en este punto. Entonces:

1) Seguiremos teniendo presente la arquitectura del sistema y el paso de mensajes que Spring y el director técnico nos imponen (*controlador* que le pasa el mensaje al *servicio,* que será quien acceda a la base de datos mediante un *repositorio* creando, si es preciso, una *entidad)* y
2) No olvidaremos la idea de escribir primero el caso y, luego, el código de producción para que se supere.

En primer lugar, para esta funcionalidad, los desarrolladores del *front* y del *back* deben ponerse de acuerdo sobre el formato de la *request:*

1) Si es de tipo *get,* el lado cliente puede enviar la petición de la forma */checkWord?word=CSTAO* (o sea, por ejemplo, pasando la palabra que coloca el usuario en un parámetro *word* de la URL) o como */checkWord/CSTAO* (es decir, haciendo que el valor del parámetro forme parte de la URL).
2) Podría ser de tipo *post, put* o *patch:*
    a. Las peticiones de tipo *post* suelen utilizarse para crear información nueva en el servidor: por ejemplo, podríamos tener un *front-end* para que el administrador del sistema cree nuevas palabras, que enviaría mediante peticiones de este tipo. La nueva palabra viajaría en el *payload* (recuérdese el Cuadro I, página 84) de la petición. No parece que esta opción sea la más correcta, puesto que no vamos a crear información nueva, sino simplemente a consultar si existe en el diccionario la palabra que pasamos como parámetro.
    b. *put* y *patch* se utilizan para actualizar información. Igual que con las peticiones *post,* la información viaja en el *payload*. De todos modos, no queremos actualizar información, sino recuperar, por lo que descartamos cualquiera de estos tipos de método.

Total, que nos decantamos por enviar la petición con *get* y, por variar y aprender otra forma de enviar información, esperaremos recibir la petición como */checkWord/CSTAO*.

8.1. Escritura de los casos de prueba

En esta ocasión escribiremos dos casos:

1) Uno para comprobar que si se pasa la palabra con las letras mal colocadas (por ejemplo, CSTAO) se devuelve, por ejemplo, un código 404 (NOT FOUND: no encontrado).
2) Otro para que si se pasa una palabra existente, se obtiene un código 200 (OK: todo ha ido bien).

La Figura 147 muestra un ejemplo del primer caso: se envía la petición y se espera que el servidor responda con un código 404. Podríamos haber sustituido la llamada a *isNotFound()* por *is(404)*, o por una llamada a *is4xxClientError* (Figura 148).

```
@Test
void test2() throws Exception {
   RequestBuilder request = MockMvcRequestBuilders.
        get("/words/checkWord/CSTAO");

   this.server.perform(request).
      andExpect(status().isNotFound());
}
```

Figura 147. Esperamos que el backend *devuelva un código 404 si en el diccionario no se encuentra la palabra CSTAO*

Figura 148. is4xxClientError *se verifica con cualquier error de la serie 4*

Para el segundo caso, podemos pensar que basta con enviar al servidor una de las combinaciones válidas que se pueden formar con las letras C, S, T, A y O, como TOCAS, CASTO, COSTA, TACOS o TOSCA, pero, teniendo en cuenta que estamos trabajando con la base de datos de prueba, en memoria, que solo dispone de la palabra CASTO, enviaremos precisamente esta cadena, CASTO:

```
@Test
void test3() throws Exception {
   RequestBuilder request = MockMvcRequestBuilders.
         get("/words/checkWord/CASTO");

   this.server.perform(request).
      andExpect(status().isOk());
}
```

Figura 149. Esperamos que checkWord *devuelva 200 con la palabra CASTO*

8.2. Implementación del código de producción

El código del servicio web que responde a la petición *checkWord/PALABRA* puede ser un método que, igual que el servicio, se llame también *checkWord*. Lo incluiremos en el mismo *WordRaceController* y puede tener, a primera vista, ese aspecto:

```
@GetMapping("/checkWord/{word}")
public void checkWord(@PathVariable String word) {
   this.wordRaceService.checkWord(word);
}
```

Figura 150. Posible código del servicio web checkWord

Como vemos, es un método que se ofrece en un servicio llamado *checkWord*. En la anotación *@GetMapping* añadimos el valor que puede variar (*{word}*, en este caso, entre llaves). Vemos también que el método no devuelve nada (es de tipo *void);* por tanto, debemos programar este método para que:

1) Devuelva 200 si la cadena *word* que recibe como parámetro existe en el diccionario.
2) Devuelva 404 si no la encuentra.

Estas dos respuestas (200 y 404) son las que hemos especificado respectivamente en los casos de prueba *test3* y *test2*.

Sustituiremos el código de *checkWord* en *WordRaceService* para que devuelva un booleano: *true* si la palabra existe y *false* en caso contrario. Entonces, en el servicio web devolveremos 200 o 404 según el valor emitido por *WordRaceService*. En la Figura 151 se muestra el nuevo código de este método. Por facilitar la legibilidad del código, se ha cambiado el nombre del método *checkWord* de *WordRaceService* por *existsWord,* pues este da más sensación de que se devolverá un valor booleano.

Como vemos, el método sigue devolviendo un *void,* pero lanza una *ResponseStatusException* con valor *NOT_FOUND* (404) si la palabra no existe. Si sí existe, el método termina sin lanzar excepción, y los interceptores de Spring

enviarán al cliente un código 200. El cliente actuará de un modo u otro en función de este código.

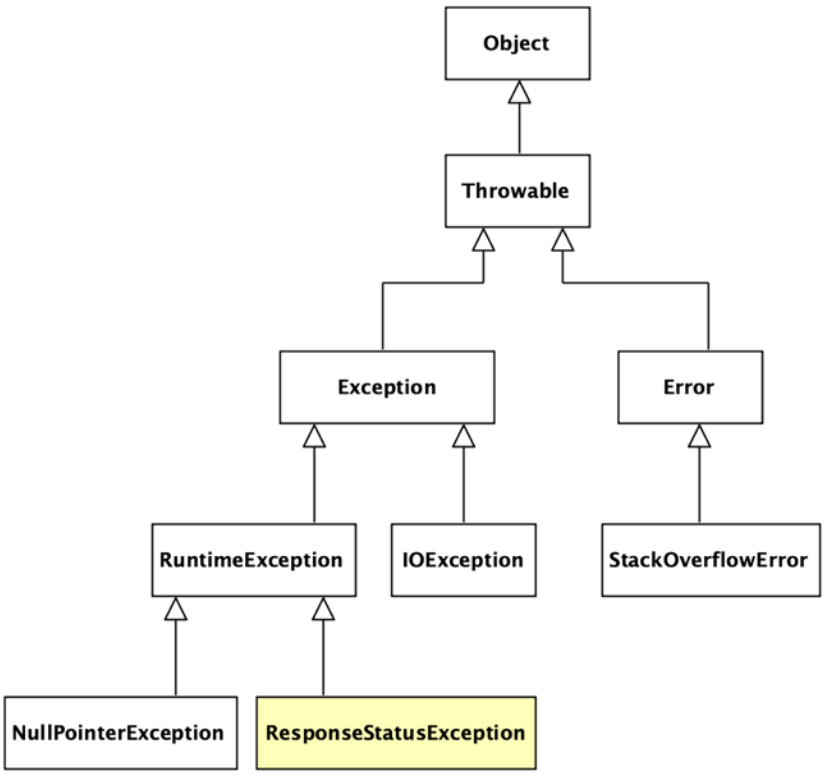
```
@GetMapping("/checkWord/{word}")
public void checkWord(@PathVariable String word) {
   if (!this.wordRaceService.existsWord(word))
      throw new ResponseStatusException(HttpStatus.NOT_FOUND);
}
```

Figura 151. Reescritura del servicio web checkWord

Jerarquía de excepciones en Java

La *ResponseStatusException* es una especialización de *RuntimeException*. Las *RuntimeException* no necesitan ser capturadas con bloques *try catch* ni arrojadas con *throws*.

En este punto, es interesante que recordemos la jerarquía de excepciones de Java, que se muestra en la siguiente figura: las *RuntimeException* no necesitan ser capturadas. En sus especializaciones está la *ResponseStatusException* que acabamos de utilizar y también hay otras que suceden a veces de forma inesperada y que el compilador tampoco nos obliga a tratar expresamente, como la *NullPointerException*. Otras excepciones, como la *IOException* (que puede lanzarse cuando, por ejemplo, abrimos un fichero) sí que debe tratarse con un *try catch* o lanzándola con un *throws*.



Cuadro VII. Jerarquía de excepciones en Java

El método *existsWord* en el servicio recibe una cadena como parámetro y la buscará en la base de datos a través del repositorio, devolviendo *true* o *false* según la encuentre o no. Daremos a este método la implementación de la Figura 152:

1) Utiliza una operación *findByWord* del repositorio *wordDAO* que, como vemos, devuelve un *Optional<Word>*. El tipo *java.util.Optional<T>* es un tipo parametrizable que puede o no contener en su interior un objeto

del tipo parametrizado (*T,* en este ejemplo). Es decir, dentro de *optWord* puede o no haber una instancia de tipo *Word*.

2) Devuelve el resultado de consultar si hay algo dentro del *optional*: si hay algo, *isPresent* devolverá *true*, y *false* si no. Si necesitásemos acceder al valor contenido en *optWord,* usaríamos *optWord.get()*.

```
public boolean existsWord(String word) {
   Optional<Word> optWord = this.wordDAO.findByWord(word);
   return optWord.isPresent();
}
```

Figura 152. existsWord *en* WordRaceService

findByWord no existe en el repositorio (su código, sin este método, aparece en la Figura 129, página 99), por lo que el Eclipse nos indicará que tenemos un error en la primera línea del método. El repositorio, sin embargo, sí que fue definido como un *JpaRepository<Word, Integer>*, que le indica a Spring que este repositorio gestiona la persistencia de entidades de tipo *Word,* cuya clave principal es un *Integer*.

En la entidad *Word* tenemos un campo *word*. En los repositorios de Spring podemos utilizar algunas convenciones de nombrado para que se generen de forma automática sentencias de acceso a la base de datos. Por defecto, todos los repositorios incluyen la operación *findById* (que se hereda de la interfaz *CrudRepository* mostrada en la Figura 130), que genera cuando se ejecuta una instrucción tipo *select * from TABLA where COLUMNA_PK=…* (si la base de datos que se ataca es SQL; si fuese una base de datos no-SQL, como MongoDB, se generaría otro tipo de instrucción).

Como *word* es un campo de la entidad *Word,* la llamada, por ejemplo, al método *findByWord("CASTO")* generará una instrucción de tipo *select * from word where word="CASTO":*

1) La parte *select* * se genera porque el nombre del método empieza por *findBy*.
2) El fragmento *from word* se añade porque la entidad que gestiona el repositorio es *Word*.
3) Finalmente, *where word="CASTO"* se genera porque, en el nombre del método, después del *By* aparece *Word,* que Spring mapea a un nombre de columna.

Lo único que necesitamos para usar este método en el repositorio es crearlo. No es necesario darle ninguna implementación.

```
package edu.uclm.esi.juegos.dao;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import edu.uclm.esi.juegos.entities.Word;

@Repository
public interface WordDAO extends JpaRepository <Word, Integer> {

   Optional<Word> findByWord(String word);

}
```

Figura 153. Nuevo código del repositorio WordDAO

Convenciones de nombrado en Spring

En los repositorios pueden escribirse muchos métodos que, adecuadamente nombrados, permiten lanzar operaciones contra la base de datos sin que tengamos que escribir código de persistencia (SQL, por ejemplo).

En la siguiente tabla, tomada de https://docs.spring.io/spring-data/jpa/docs/1.6.0.RELEASE/reference/html/jpa.repositories.html, se muestran algunos ejemplos.

Keyword	Sample	JPQL snippet
And	findByLastnameAndFirstname	… where x.lastname = ?1 and x.firstnar
Or	findByLastnameOrFirstname	… where x.lastname = ?1 or x.firstname
Is,Equals	findByFirstname, findByFirstnameIs, findByFirstnameEquals	… where x.firstname = 1?
Between	findByStartDateBetween	… where x.startDate between 1? and ?2
LessThan	findByAgeLessThan	… where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	… where x.age <= ?1
GreaterThan	findByAgeGreaterThan	… where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	… where x.age >= ?1
After	findByStartDateAfter	… where x.startDate > ?1
Before	findByStartDateBefore	… where x.startDate < ?1
IsNull	findByAgeIsNull	… where x.age is null
IsNotNull, NotNull	findByAge(Is)NotNull	… where x.age not null
Like	findByFirstnameLike	… where x.firstname like ?1
NotLike	findByFirstnameNotLike	… where x.firstname not like ?1
StartingWith	findByFirstnameStartingWith	… where x.firstname like ?1 (parameter bo
EndingWith	findByFirstnameEndingWith	… where x.firstname like ?1 (parameter bo
Containing	findByFirstnameContaining	… where x.firstname like ?1 (parameter bo
OrderBy	findByAgeOrderByLastnameDesc	… where x.age = ?1 order by x.lastname
Not	findByLastnameNot	… where x.lastname <> ?1
In	findByAgeIn(Collection<Age> ages)	… where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> age)	… where x.age not in ?1
True	findByActiveTrue()	… where x.active = true
False	findByActiveFalse()	… where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	… where UPPER(x.firstame) = UPPER(?1)

Cuadro VIII

8.3. Ejecución de los casos

Si ejecutamos los casos, obtendremos para todos ellos la barra verde, lo que indica que el código de producción que hemos escrito está sirviendo correctamente los escenarios de ejecución que hemos descrito en los tres casos de prueba:

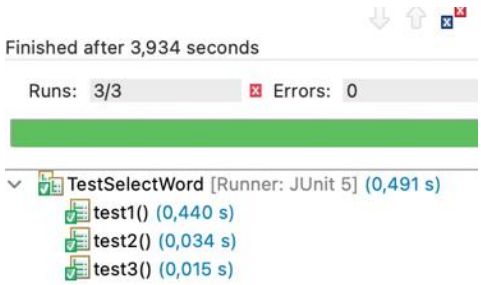


Figura 154. El sistema va estando bien construido

9. Alternativas de implementación

El diseño del sistema que hemos construido se corresponde bastante fielmente con el diagrama de clases de la Figura 155.

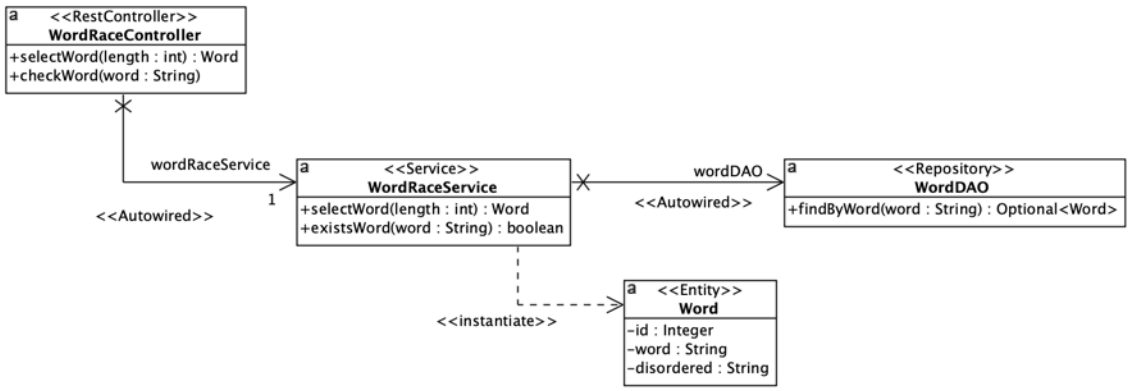


Figura 155. Diseño actual del sistema

Tal vez algún ingeniero de software opine que podríamos eliminar el servicio, que actúa de intermediario entre *WordRaceController* y el repositorio, dejando el sistema como en la Figura 146: en este diseño, la implementación de las operaciones que teníamos en el servicio pasa a los métodos del controlador. Por supuesto, el sistema podría exhibir exactamente el mismo comportamiento, pero este diseño no se ajustaría a la arquitectura genérica que se propuso en la Figura 91 (página 77).

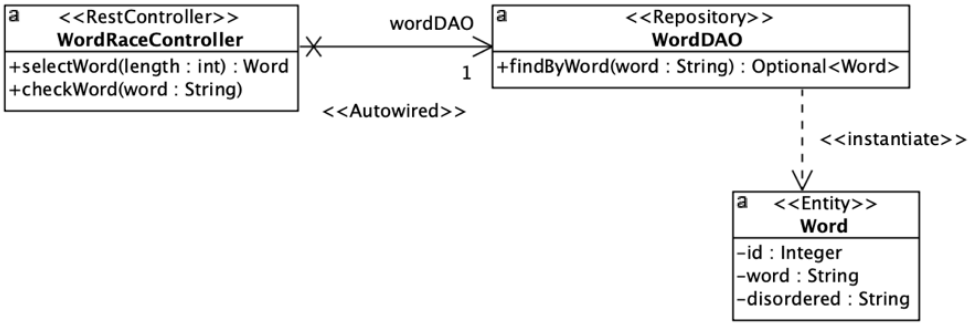


Figura 156. Diseño alternativo

Capítulo 6. Implantación de la base de datos real en un *docker*

1. Introducción

Por algún motivo, se ha decidido implantar la base de datos en un servidor SQL Server que, sin embargo, no corre de forma nativa en ordenadores Mac. Por esta razón se utilizará un *docker*, en el que se instalará una imagen de SQL Server con la base de datos.

Aunque el lector disponga de un ordenador con Windows o Linux, en los que SQL Server funciona de manera nativa, se le invita a que se familiarice con la tecnología de contenedores y utilice *dockers*.

2. *Dockerización* de la base de datos

En el docker se instalan imágenes que luego pueden ponerse en ejecución. En nuestro caso, buscamos en Internet con la cadena, por ejemplo, "sql server docker mac" y encontraremos varias páginas de ayuda.

1) En primer lugar, debemos descargar la imagen de SQL Server, para lo que se nos propone el comando: *sudo docker pull mcr.microsoft.com/mssql/server:2019-latest*
2) Una vez descargada la imagen, aparecerá en la lista de imágenes de *Docker Desktop* (Figura 157). Si pulsamos el botón *Run* se arrancará la ejecución de la imagen, pero se interrumpirá porque faltan algunos parámetros que debemos dar.

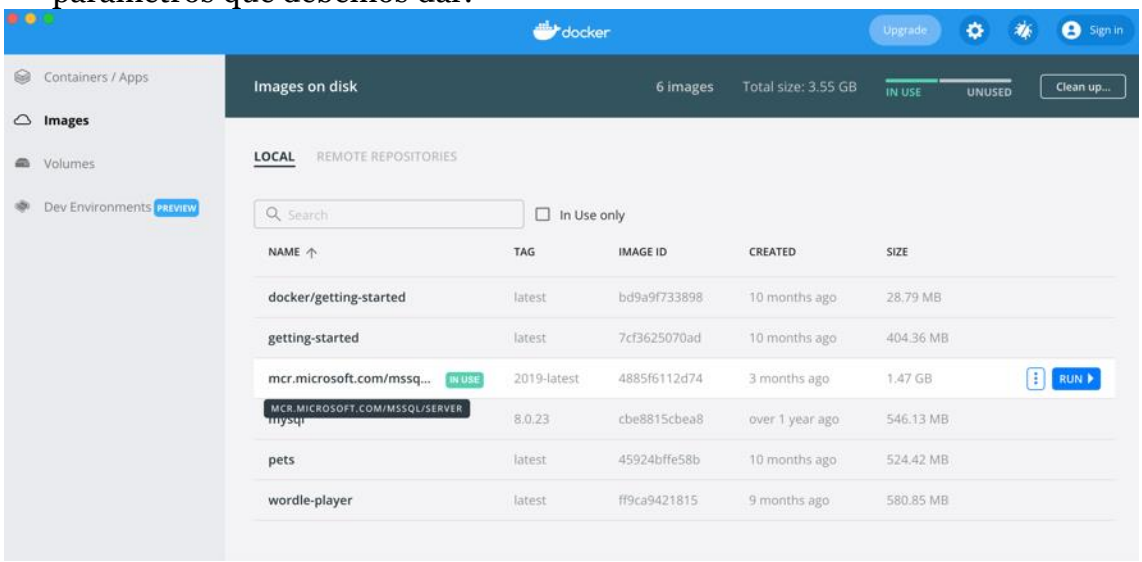


Figura 157. Imágenes almacenadas en el docker

3) Escribiremos en una terminal el siguiente comando: *sudo docker run -e 'ACCEPT_EULA=Y' -e 'SA_PASSWORD=DiSoft2022 ' -p 1433:1433 -d mcr.microsoft.com/mssql/server:2019-latest*. Desmenuzamos el significado de cada fragmento de este comando en la siguiente figura:

sudo docker run	Ejecuta una imagen

-e 'ACCEPT_EULA=Y'	*-e* denota que se está asignando valor a una variable de entorno. En este caso, asignamos *Y* (yes) a *ACCEPT_EULA,* lo que indica que aceptamos el Acuerdo de Licencia del Usuario Final (End-User License Agreement).
-e 'SA_PASSWORD=DiSoft2022 '	Aquí asignamos valor a otra variable de entorno. En este caso a *SA_PASSWORD*, que es la contraseña del usuario *sa*. En SQL Server, *sa* es el "superadministrador".
-p 1433:1433	Con *-p* asignamos valor a dos puertos: el primero es el puerto externo es en el que se va a poner a escuchar el docker para recibir peticiones desde el exterior; el otro es el puerto interno al que el docker redirigirá el tráfico que le llegue al puerto externo. Por defecto, SQL Server escucha en el 1433. Co *-p 1433:1433* estamos diciéndole al docker que, lo que le llegue al puerto 1433, lo redirija al puerto interno 1433.
-d mcr.microsoft.com/mssql/server:2019-latest	*-d* indica que la imagen se ejecute en segundo plano.

Figura 158. El comando de ejecución del docker, explicado cachito a cachito

Tras ejecutar el último comando, se nos habrá creado en el *Docker Desktop* un contenedor, que ahora podemos lanzar a ejecutar. En la Figura 159, el propio comando *docker* ha asignado un nombre aleatorio a este contenedor (*modest_gates).* Podríamos haber asignado un nombre más significativo con el parámetro *–name*.

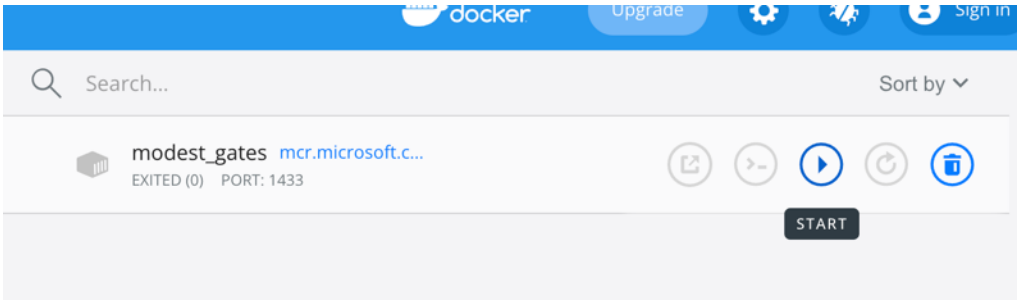


Figura 159. Se ha creado un contenedor que podemos ejecutar

Si hacemos clic en el botón *Start* de la figura anterior y, a continuación, sobre el nombre del contenedor, aparecerá la consola de arranque del proceso asociado a este contenedor (Figura 160).

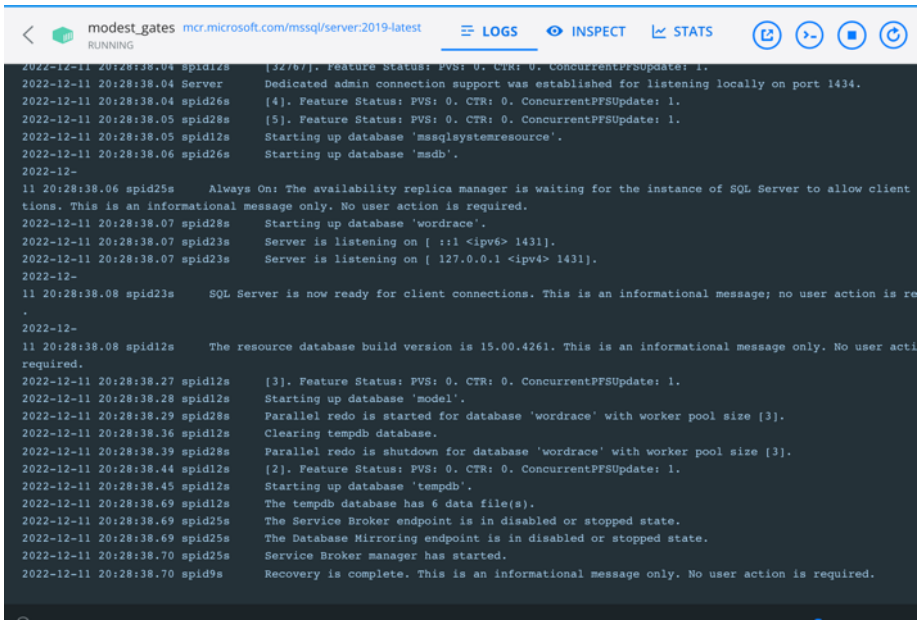


Figura 160. Consola de arranque del proceso

El gestor de base de datos nos escucha en el puerto 1433, y atenderá peticiones para SQL Server, que serán adecuadamente servidas si se utiliza su protocolo de comunicación: igual que *http* es un protocolo perfectamente estandarizado, que define bien la estructura de las peticiones y respuestas, cada fabricante de sistemas gestores de bases de datos define su protocolo de comunicación. Así como cualquier navegador envía peticiones en formato *http*, para comunicarnos con una base de datos SQL Server necesitamos un *user-agent* que emita peticiones en el formato que entiende el servidor de SQL Server.

3. Instalación de un cliente para SQL Server

Azure Data Studio es un cliente de SQL Server que podemos utilizar en cualquier sistema operativo. Su aspecto nos recordará mucho al de Visual Studio Code, que ya hemos utilizado para el desarrollo de *Order the numbers*.

Una vez descargado e instalado, crearemos una conexión nueva al servidor que tenemos corriendo en el docker, y que está escuchando en el puerto 1433 de *localhost* (suponiendo que tengamos todo en la misma máquina).

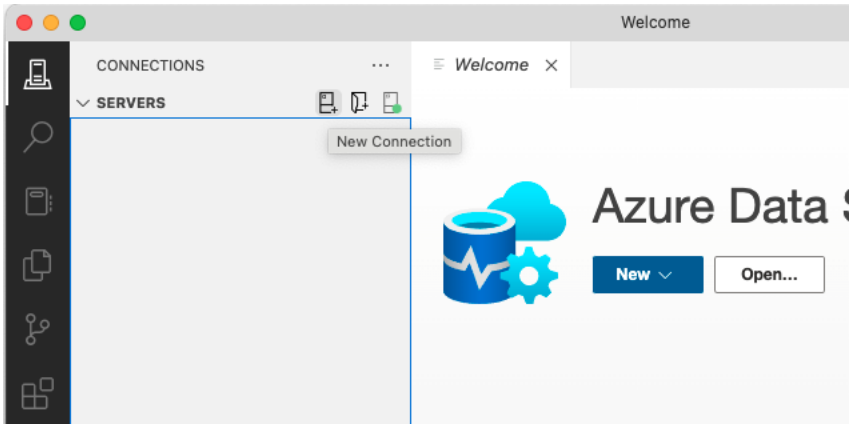


Figura 161. Azure Data Studio recién instalado y ejecutado

Debemos establecer los parámetros de conexión: servidor (*localhost)*, tipo de autenticación (de momento, *SQL Login*), nombre del usuario que se conecta (*sa*) y contraseña (la que hemos establecido al crear el contenedor: *DiSoft2022);* además, indicaremos que de momento no usaremos encriptación.

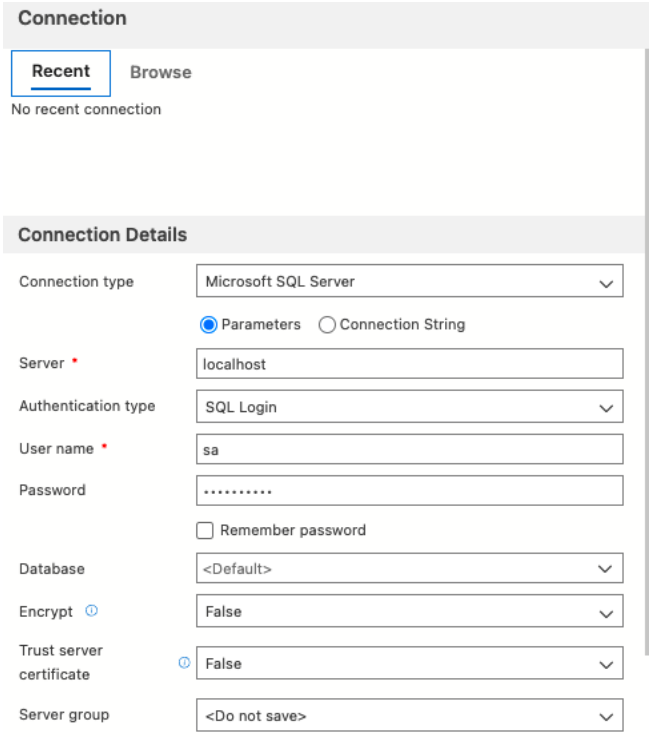


Figura 162. Parámetros de conexión a la base de datos

Si todo va bien, nos aparecerán todas las bases de datos ofrecidas por este servidor que acabamos de instalar, todas ellas relacionadas con el propio funcionamiento del servidor: *master, model, msdb* y *tempdb*.

4. Creación de la base de datos

Lo primero será crear la base de datos en la que almacenaremos, de momento, la lista de palabras y, más adelante y cuando cambien los requisitos, los datos que necesitemos de los usuarios y, probablemente, de otros juegos.

En el menú *File* encontramos la opción *New Query,* con la que se abre una consola en la que escribiremos el comando para crear una base de datos que se llame *games* (Figura 163).

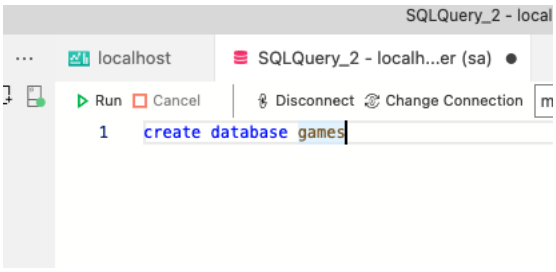


Figura 163. Una query *para crear la base de datos* games

Pulsamos el botón *Run* para ejecutar la sentencia DDL (*Data Definition Language,* el subconjunto de SQL que se utiliza para crear bases de datos, tablas, modificarlas, etcétera). En la solapa *localhost* encontraremos un botón *Refresh*

que recargará las bases de datos ubicadas en *localhost* y, entre ellas, encontraremos *games* (Figura 164).

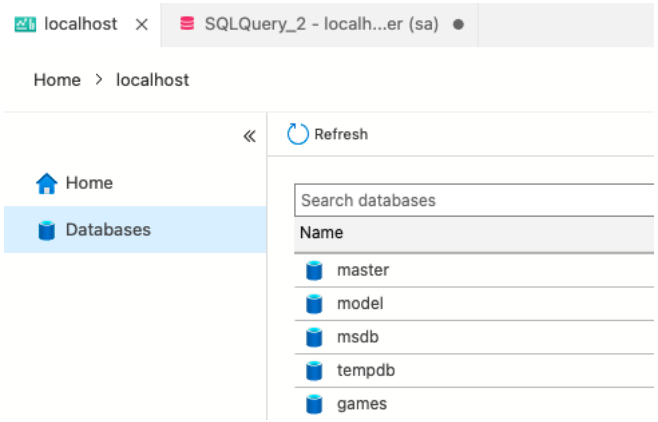


Figura 164. Ya aparece la base de datos que acabamos de crear

5. Uso de la base de datos desde el código de producción

En el proyecto Eclipse tenemos código de producción y código de test y, además, dos carpetas *resources* con los recursos que utilizamos para la versión de producción y la versión de *test* (Figura 165).

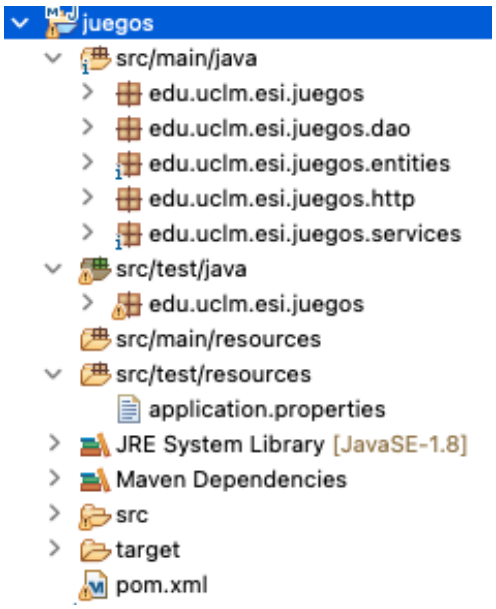


Figura 165. Estructura de carpetas y ficheros del proyecto

En el fichero *src/test/resources/application.properties* tenemos la configuración de ejecución del proyecto Spring para cuando estamos ejecutando en modo de prueba; debemos crear un fichero parecido en *src/main/resources* con la configuración de ejecución para la versión de producción.

Además, en el *pom.xml* tenemos algunas dependencias marcadas con la etiqueta *<scope>test</scope>*: por ejemplo, la base de datos en memoria. Pero, para la versión de producción, cuyo *scope* será *runtime,* utilizaremos la base de datos SQL Server que acabamos de crear.

En primer lugar, añadimos al *pom.xml,* después de la dependencia a *h2,* otra a SQL Server, que nos permitirá conectar a un servidor SQL Server desde una aplicación Java (Figura 166). Nótese la etiqueta *<scope>runtime</scope>*, que

indica que esta dependencia debe utilizarse solo cuando se ejecute el código de producción.

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
</dependency>

<dependency>
     <groupId>com.microsoft.sqlserver</groupId>
     <artifactId>mssql-jdbc</artifactId>
     <scope>runtime</scope>
</dependency>
```

Figura 166. Inclusión de una dependencia a SQL Server

Ahora crearemos el fichero *application.properties* en *src/main/resources:* podemos copiar y pegar el que tenemos en *test*, pero debemos modificar algunas cosas. En la Figura 167:

1) El puerto sigue siendo el 80.
2) Cambiamos la URL en la que escucha el servidor de base de datos: ya no es *jdbc:h2:mem:testdb,* sino que ahora apunta a la ubicación de un sistema real. El formato de la URL depende de cada proveedor. Microsoft, fabricante de SQL Server, nos dice que, tras el protocolo (*jdbc)*, subprotocolo (*sqlserver)* y dirección del servidor (*localhost* en este caso, pero podría haber sido *juegos.com:1800* si el servidor estuviese en el puerto 1800 de *juegos.com)*, debemos especificar el nombre de la base de datos (*games* en este caso) y si usamos encriptación en la comunicación (hemos marcado *false*).
3) Indicamos también el nombre y la contraseña del usuario que accede a la base de datos desde el *backend*.
4) Debemos especificar una clase que contenga la implementación del *driver* utilizado para conectar al servidor (*com.microsoft.sqlserver.jdbc.SQLServerDriver*) y un "dialecto". Mientras que el *driver* debe ser proporcionado por el fabricante (*com.microsoft…),* el dialecto es una clase de un tercero (*org.hibernate…)* que traduce al formato esperado por la versión del servidor de base de datos que estemos utilizando las instrucciones SQL que se vayan generando. Con la versión de SQL Server que hemos instalado en el docker y el *driver* que especificamos en el *application.properties,* el dialecto debe ser el que se implemente en la clase *SQLServer2009Dialect*.
5) El resto de líneas las mantenemos igual: en las dos últimas indicamos que se nos muestre por la consola el código de acceso a la base de datos que Spring vaya generando; en la antepenúltima, mantenemos también que, al arrancar la aplicación (ahora en modo de producción), se ejecuten las instrucciones DDL necesarias para crear las tablas, columnas, posibles relaciones entre tablas, etcétera.

```
server.port = 80

spring.datasource.url = jdbc:sqlserver://localhost;databaseName=games;encrypt=false;
spring.datasource.username = sa
spring.datasource.password = DiSoft2022

spring.datasource.driverClassName=com.microsoft.sqlserver.jdbc.SQLServerDriver
spring.jpa.hibernate.dialect=org.hibernate.dialect.SQLServer2009Dialect

# Hibernate ddl auto (create, create-drop, validate, update)
spring.jpa.hibernate.ddl-auto = create

logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

Figura 167. application.properties *en* src/main/resources

6. Ejecución del código de producción

Arrancaremos la aplicación por la clase *App* que tenemos en el paquete *edu.uclm.esi.juegos* y cuyo código ya comentamos y vimos en la Figura 113 (página 92): botón derecho sobre *App,* elegimos *Run As,* luego *Java Application* y Spring lanza la aplicación.

Como sabemos, Spring revisa las clases que tengan anotaciones (controladores, servicios, entidades, etcétera). Además, como tenemos incluida la dependencia a JPA, intenta también conectar a la base de datos indicada en el campo *spring.datasource.url* del fichero *application.properties* de la zona de producción (*src/main/resources).* Igualmente, como hemos puesto a *true* el valor *spring.jpa.hibernate.ddl-auto* (Figura 167), Spring lanza algunas sentencias DDL para crear la base de datos. La primera es *drop table word*, que intenta eliminar de la base de datos *games* alguna tabla que se llame *word*. Como no existe, veremos por consola una gran excepción que a primera vista asusta (Figura 168).

```
--- [  restartedMain] org.hibernate.Version                  : HHH000412: Hibernate Core {5.2
--- [  restartedMain] org.hibernate.cfg.Environment          : HHH000206: hibernate.propertie
--- [  restartedMain] o.hibernate.annotations.common.Version : HCANN000001: Hibernate Commons
--- [  restartedMain] org.hibernate.dialect.Dialect          : HHH000400: Using dialect: org.
--- [  restartedMain] org.hibernate.SQL                      : drop table word
--- [  restartedMain] o.h.t.s.i.ExceptionHandlerLoggedImpl   : GenerationTarget encountered e

dAcceptanceException: Error executing DDL via JDBC Statement
.internal.exec.GenerationTargetToDatabase.accept(GenerationTargetToDatabase.java:67) ~[hibernat
.internal.SchemaDropperImpl.applySqlString(SchemaDropperImpl.java:375) [hibernate-core-5.2.17.F
.internal.SchemaDropperImpl.applySqlStrings(SchemaDropperImpl.java:359) [hibernate-core-5.2.17.
.internal.SchemaDropperImpl.dropFromMetadata(SchemaDropperImpl.java:241) [hibernate-core-5.2.17
.internal.SchemaDropperImpl.performDrop(SchemaDropperImpl.java:154) [hibernate-core-5.2.17.Fina
```

Figura 168. Error al ejecutar drop table word

No obstante, Spring no interrumpe en este caso la ejecución sino que, en la misma consola, podemos ver un poco más abajo que ha lanzado la sentencia DDL de la Figura 169 para crear la tabla.

```
    : create table word (id int not null, word varchar(255), primary key (id))
mpl : HHH000476: Executing import script 'org.hibernate.tool.schema.internal.exec
Bean : Initialized JPA EntityManagerFactory for persistence unit 'default'
ping : Mapped URL path [/**/favicon.ico] onto handler of type [class org.springfra
pter : Looking for @ControllerAdvice: org.springframework.boot.web.servlet.context
```

Figura 169. Creación de la tabla word

¿Por qué genera Spring esa instrucción tan concreta: *create table word…?* Porque, como hemos dicho varias veces, Spring revisa el código cuando arranca. En nuestro caso, encuentra la clase *Word* anotada como *Entity* y, como en *application.properties* le hemos dicho que cree la base de datos, revisa la clase

Word y genera una instrucción DDL acorde a la estructura de esta entidad, y por eso crea la columna *id* como de tipo *int* (porque el campo *id* es *Integer* en la entidad), *word* como *varchar* (por el campo *word,* de tipo *String,* al que asigna la longitud máxima, 255; esto es demasiado y habrá que acortarlo a la palabra más larga que tengamos en nuestro diccionario) e indica (instrucción *primary key (id)*) que la clave principal de la tabla que está creando es la columna que anotamos con *@Id*.

La Figura 170 ilustra esta equivalencia entre la clase y la tabla. Para el campo *disordered* no se ha creado columna alguna porque está anotado como *@Transient*.

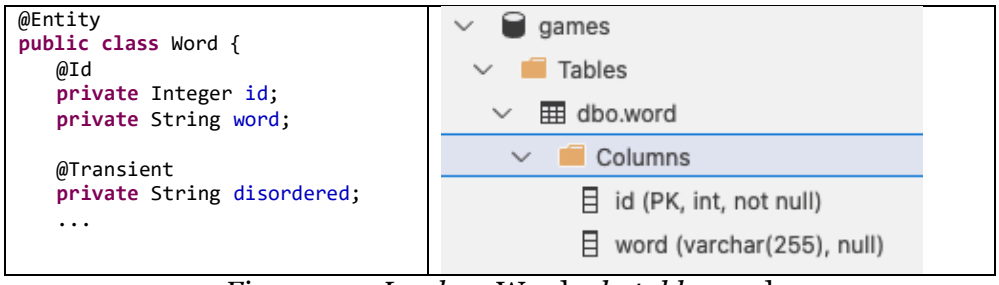


Figura 170. La clase Word *y la tabla* word

7. Inserción de datos en la tabla *word*

Hasta ahora estamos insertando datos (gracias al método anotado con *@BeforeEach)* en la base de datos del modo de test, pero no tenemos nada en la base de datos de producción que reside en SQL Server. Claramente, tenemos que meter algunas palabras: a ser posibles, todas las del idioma español.

Si buscamos en Internet con la cadena "base de datos palabras español", encontramos pronto un enlace a https://scriptinside.blogspot.com/2012/01/listado-de-palabras-en-castellano.html que nos ofrece una base de datos en MySQL con todas las palabras en castellano. No sabemos si están todas-todas, pero sí que hay muchas-muchas.

Se nos descarga un fichero *rar* que, según la descripción, contiene más de 55.000 palabras en castellano en una única tabla con cuatros campos. Textualmente, dice que la tabla contiene: «la clave primaria y 3 campos de texto con la misma palabra (todas en minúscula). La diferencia entre los 3 campos es que el primero llamado palabra es un varchar con lo cual en las consultas SQL no distingue entre acentos o ñ y n, los otros dos campos son binarios por tanto si distinguen, uno de ellos tiene ñ pero no tiene acentos y el otro tiene acentos y ñ para una comparativa más exacta».

El archivo, descomprimido, contiene un fichero *db_diccionario.sql* que, si ejecuta, crea una base de datos en un servidor de *MySQL* con una tabla *palabras,* en la que inserta las 55.000 palabras que contiene (Figura 171).

```
CREATE DATABASE /*!32312 IF NOT EXISTS*/`db_diccionario` /*!40100 DEFAULT CHARACTER SET utf8 */;

USE `db_diccionario`;

/*Table structure for table `palabras` */

DROP TABLE IF EXISTS `palabras`;

CREATE TABLE `palabras` (
  `palabra_id` smallint(5) unsigned NOT NULL AUTO_INCREMENT,
  `palabra` varchar(25) NOT NULL,
  `sin_acentos` varbinary(25) NOT NULL,
  `sensible` varbinary(25) NOT NULL,
  PRIMARY KEY (`palabra_id`),
  UNIQUE KEY `PALABRA_UNICA` (`palabra`)
) ENGINE=InnoDB AUTO_INCREMENT=58009 DEFAULT CHARSET=utf8;

/*Data for the table `palabras` */

insert  into `palabras`(`palabra_id`,`palabra`,`sin_acentos`,`sensible`) values (1,'a','a','a'),(2,'ababa'
insert  into `palabras`(`palabra_id`,`palabra`,`sin_acentos`,`sensible`) values (16454,'de','de','de'),(16
insert  into `palabras`(`palabra_id`,`palabra`,`sin_acentos`,`sensible`) values (31907,'integrante','integ
insert  into `palabras`(`palabra_id`,`palabra`,`sin_acentos`,`sensible`) values (48291,'revén','reven','re
```

Figura 171. Algunas líneas del código para MySQL que nos hemos bajado

Tenemos varias formar de llevarnos estos datos a nuestra tabla *word* (recuérdese que solo tenemos dos columnas: *id* y *word*) de SQL Server. Podemos optar por instalar (si no le tenemos) el servidor de MySQL, ejecutar el fichero que hemos descargado y, luego, exportarlo de alguna manera para importar los datos en SQL Server.

Podemos también refinar la búsqueda que hemos hecho en Internet, y añadir *txt* a la cadena de búsqueda:

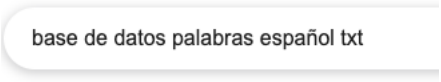


Figura 172. Buscamos las palabras en castellano en formato txt

Con esta última, encontramos un fichero llamado *0_palabras_tocas.txt*[6] que podemos bajar a nuestra carpeta de descargas. Podemos abrirlo directamente con una hoja de cálculo; en el caso del redactor de este libro, con Excel.

Lamentablemente, cuando lo abrimos, nos encontramos un pequeño problema con las tildes:

[6] Ubicado en https://github.com/JorgeDuenasLerin/diccionario-espanol-txt, de Jorge Dueñas Lerín.

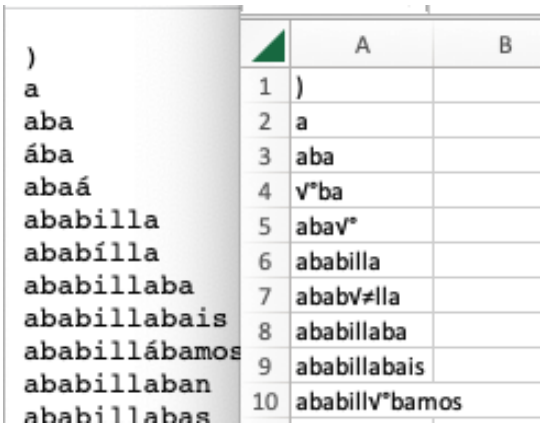


Figura 173. Nuestro programa de hoja de cálculo no procesa bien las tildes

No pasa nada. Cerramos el archivo, lo abrimos con la opción *Abrir* y marcamos el formato *UTF-8* (Figura 174). Luego, terminamos de importar los datos y nos aparecerá una hoja Excel con 646.616 filas.

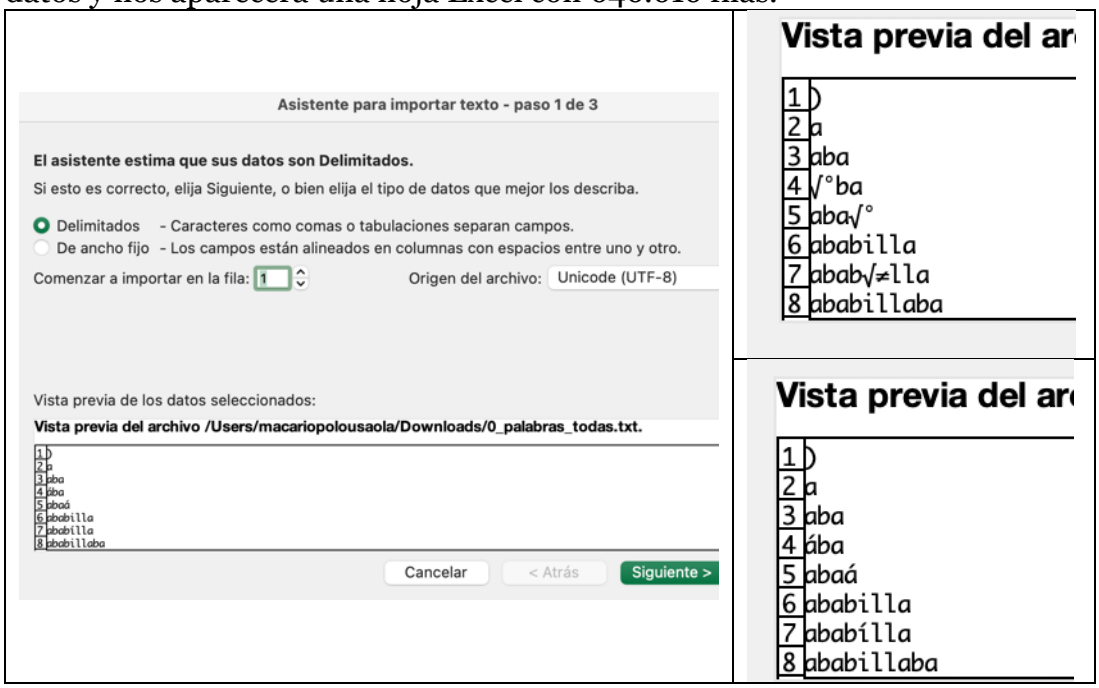


Figura 174. Importamos el archivo como UTF-8 *para que sus datos salgan bien*

La primera de las filas, que contiene un extraño paréntesis, no nos sirve, por lo que la eliminamos. Nos quedamos entonces con 646.615 palabras:

Figura 175. Últimas filas del fichero con las palabras en castellano

De alguna manera, tenemos que insertar todas esas palabras en la tabla *word* de la base de datos del servidor SQL Server que acabamos de instalar y que

escucha en el puerto 1433 del docker. En la tabla tenemos dos columnas: *id,* que es un entero, y *word,* que es un *varchar*. Para insertar la última palabra, *zuzón,* tendríamos que escribir algo como *insert into word (word) values ('zuzón').*

Para conseguir una expresión de este tipo, vamos a la fila 1 de la hoja de cálculo y, en la celda B1, escribimos una fórmula que concatene el texto que hay en A1 con algo más:

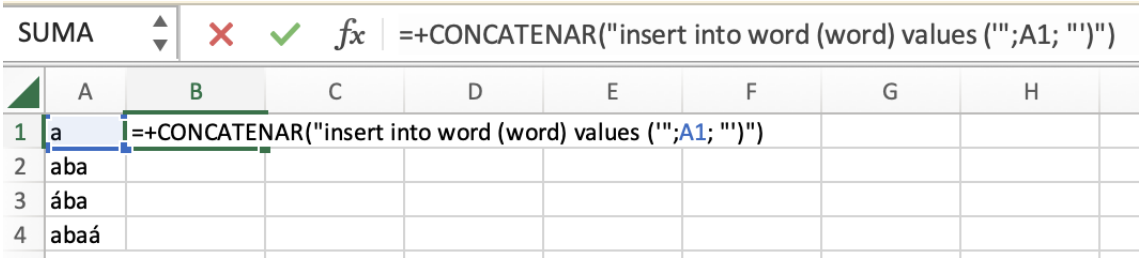


Figura 176. Generamos una instrucción insert into *por cada palabra del fichero*

Si copiamos y pegamos la fórmula desde B1 hasta B646615, la columna B se nos va llenando con sentencias *insert into* (Figura 177).

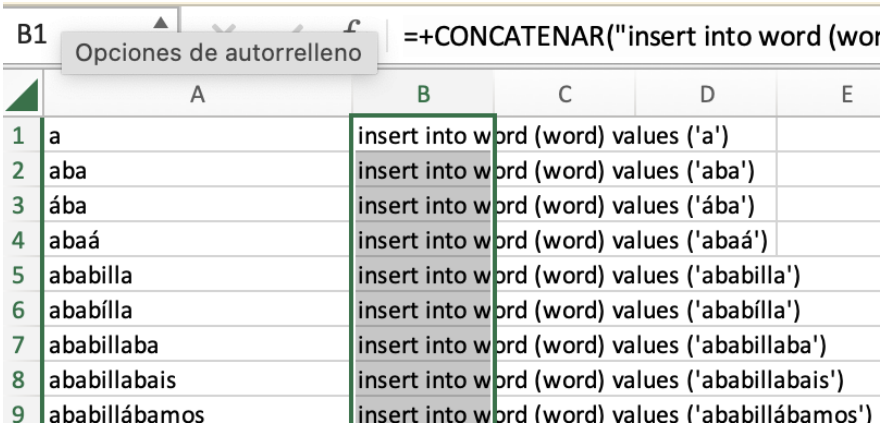


Figura 177. Generamos cientos de miles de sentencias SQL para insertar las palabras

Lanzar seiscientas y pico mil sentencias puede tener algún inconveniente: incluso copiarlas y pegarlas tal vez provoque algún problema de memoria, por lo que optaremos por otra solución:

1) Lanzamos la aplicación para que se nos cree automáticamente la tabla *word*, como sucedió en la Figura 169, o bien la creamos manualmente (Figura 178).

Figura 178. Creamos la tabla word *usando el cliente de SQL Server*

2) Con la tabla creada, escribiremos un pequeño programita que recorra el fichero de palabras y las vaya insertando, una a una, en la tabla *word*.

Este segundo punto puede hacerse de varias maneras. Optaremos por crear un método *insertWords* en la clase *App* al que llamaremos cuando la aplicación haya arrancado:

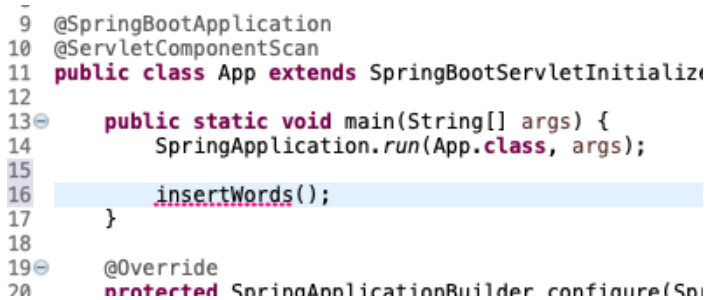


Figura 179. Llamaremos a insertWords *para que inserte las palabras del fichero en la tabla* words *de la base de datos*

Escribiremos el método *insertWords* de modo que recorra el fichero con las palabras y, haciendo uso de una instancia del repositorio *wordDAO* de tipo *WordDAO*, vaya guardando en la base de datos cada una de ellas. Añadamos el método *insertWords* a la clase *App*, que queda ahora de la siguiente manera:

```
@SpringBootApplication
@ServletComponentScan
public class App extends SpringBootServletInitializer {

   @Autowired
   private WordDAO wordDAO;

    public static void main(String[] args) {
      SpringApplication.run(App.class, args);

      insertWords();
    }

    private static void insertWords() {
      FileInputStream fis;
      InputStreamReader isr;
      BufferedReader reader;

      try {
         fis = new FileInputStream
            ("/Users/macariopolousaola/Downloads/0_palabras_todas.txt");
         isr = new InputStreamReader(fis, StandardCharsets.UTF_8);
         reader = new BufferedReader(isr);
         String line = reader.readLine();

         int cont = 1;
         while (line != null) {
            Word word = new Word();
            word.setId(cont);
            word.setWord(line);
            wordDAO.save(word);
            System.out.println(cont  + "\t" + line);
            line = reader.readLine();
            cont++;
         }
```

```
            reader.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
        ...
}
```

Figura 180. Posible implementación de insertWords

En el código anterior hemos añadido un campo *wordDAO* a la clase, que usamos en la línea resaltada en amarillo dentro de *insertWords*. El compilador nos informa de un error: no podemos hacer referencia a un campo no estático desde un método que sí lo es. En este caso, el método *insertWords* es estático, pero el campo *wordDAO* es un campo de instancia.

Podemos modificar la declaración *private WordDAO wordDAO* por *private* ***static*** *WordDAO wordDAO,* pero Spring lanzará un error al arrancar, porque no puede haber campos que sean simultáneamente estáticos y *@Autowired*. Hay que buscar otra manera de poder tener una referencia a la base de datos desde esta clase de arranque en la que nos encontramos.

7.1. El patrón *Singleton*

Un patrón es una solución buena a un problema frecuente. Y, en el desarrollo de software, un problema habitual es la necesidad de disponer de algún elemento que sirva como punto de acceso común desde diferentes lugares de la aplicación.

El patrón *Singleton* resuelve bien este problema. Un *singleton* es una clase de la que, por su propia implementación, solo podrá crearse una única instancia, y a la que podremos acceder desde cualquier lugar.

En nuestro ejemplo, crearemos una clase a la que llamaremos *Manager*, que será un *singleton* y que dispondrá de acceso al repositorio. En lugar de declarar en la clase *App* un *wordDAO,* lo declararemos en este *Manager,* que tendremos disponible después de que Spring haya analizado y arrancado la aplicación desde el método *main*.

Gráficamente, tendremos lo siguiente (Figura 181). Respecto de la clase *singleton,* esta tiene el constructor con el modificador *private* (el signo menos en UML), lo que impide que podamos crear instancias desde fuera de ella misma. Cuando necesitamos crear la única instancia existente de esta clase llamamos a su método estático *get*.

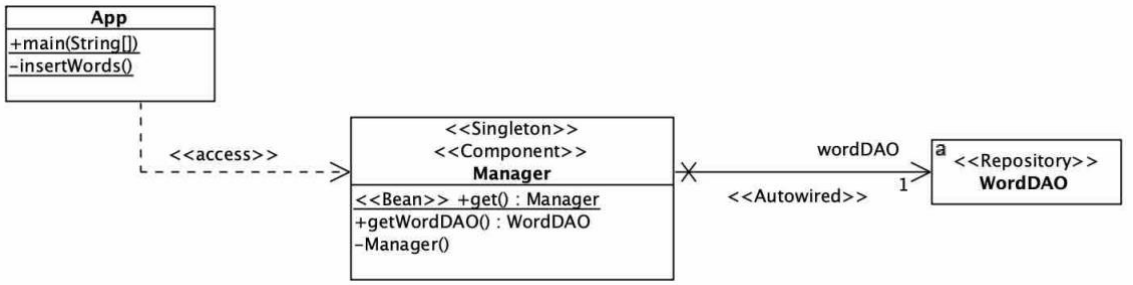


Figura 181. Inclusión de un singleton *con acceso al repositorio*

Hay varias formas de implementar un *singleton,* alguna de las cuales puede fallar en caso de que haya una alta concurrencia de llamadas a *get*. La Figura 182 es una implementación bastante segura del *singleton* representado en el diagrama de clases anterior. Obsérvese que:

1) La clase está anotada con la anotación *@Component* de Spring, de manera que Spring la revisa e instancia al arrancar la aplicación.
2) Dispone de un campo *@Autowired* de tipo *WordDAO*: cuando Spring arranca e instancia este *@Component,* instancia también este campo.
3) Dispone del método *get*, que devuelve la única instancia existente de este objeto.
4) Dispone del método *getWordDAO*, que devuelve una instancia al repositorio.

```
package edu.uclm.esi.juegos.auxi;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.stereotype.Component;

import edu.uclm.esi.juegos.dao.WordDAO;

@Component
public class Manager {

   @Autowired
   private WordDAO wordDAO;

   private Manager() {}

   private static class ManagerHolder {
      static Manager singleton=new Manager();
   }

   @Bean
   public static Manager get() {
      return ManagerHolder.singleton;
   }

   public WordDAO getWordDAO() {
      return wordDAO;
   }
}
```

Figura 182. Una implementación del Singleton

Si recordamos, el método *main* en *App* llama primero a *SpringApplication.run(App.class, args)*, que comprueba las clases anotadas, conecta a la base de datos, etcétera; si todo va bien, entonces ejecuta su segunda sentencia, que es la llamada a *insertWords*. En la Figura 180 resaltábamos el uso del campo *wordDAO* que, ahora, sustituiremos por una llamada al *getWordDAO* que tenemos en el *Manager*. Es decir, el bucle *while* de *insertWords* queda de este modo:

```
while (line != null) {
   Word word = new Word();
   word.setId(cont);
   word.setWord(line);
   Manager.get().getWordDAO().save(word);
   System.out.println(cont  + "\t" + line);
   line = reader.readLine();
   cont++;
}
```

Figura 183. Pequeño cambio para acceder al wordDAO *a través del* Manager, *que en este punto ya estará instanciado*

Ya podemos ejecutar la aplicación para que se pueble la tabla de palabras. Son más de seiscientas mil filas, por lo que llevará un rato. Para acelerarlo un

poquito más, podemos comentar las dos últimas filas de *application.properties*, de modo que no se muestre el código SQL por la consola. Además, y puesto que ya tenemos creadas la base de datos y la tabla, podemos también establecer a *none* el campo *ddl-auto:*

```
...
spring.jpa.hibernate.ddl-auto = none

#logging.level.org.hibernate.SQL=DEBUG
#logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

Figura 184. Pequeños cambios en application.properties

Por supuesto, el docker debe estar arrancado y ejecutando el contenedor de SQL Server.

Tengamos ahora cuidado, porque hay un par de palabras en la lista que hemos bajado que empiezan por paréntesis: las podemos eliminar ejecutando, en Azure Data Studio, la sentencia de la Figura 185.

```
delete from word where LEFT(word, 1)='('
```

Figura 185. Eliminamos dos palabras incorrectas

Capítulo 7. Ejecución de pruebas con la base de datos real

1. Cambios en configuración de ejecución

El código del método *insertWords* habrá tardado un rato largo en ejecutarse. Además, el hecho de que le hayamos incluido una instrucción *System.out.println* para que nos vaya mostrando cada palabra insertada, ralentiza aún más la ejecución.

Pero el caso es que tenemos ya la base de datos con su tabla *word* completamente llena y limpia, y podemos aprovechar para probar el comportamiento del *backend* con la nueva base de datos.

Antes de seguir, recordemos que estamos llamando a *insertWords* desde la función *main* de la clase de arranque *App:* eliminemos o comentemos la llamada al método, para evitar que se vuelvan a insertar todas las palabras cuando volvamos a lanzar la aplicación.

Para probar con la base de datos real, necesitamos que la zona de tests del proyecto (carpeta *src/test/java)* utilice la configuración de base de datos definida en el *application.properties* de la zona de producción (ubicado en *src/main/resources).* Para que esto suceda, lo único que necesitamos es, por ejemplo, renombrar el *application.properties* de *src/test/resources* añadiéndole *.old* al nombre (Figura 186).

Figura 186. Cambiamos el nombre del fichero para que Spring no lo considere

2. Escritura de un caso de prueba

Ahora que las pruebas usarán la base de datos de producción, crearemos un caso de prueba que envíe una petición para recuperar una palabra aleatoria de, por ejemplo, 15 letras de longitud. Si todo va bien, recibiremos un JSON con tres campos: *id, word* y *disordered.* A continuación, enviaremos otra petición para comprobar que el servidor responde con un código 404 si comprobamos la palabra recibida en *disordered;* luego, intercambiaremos las letras primera y última de la palabra y volveremos a comprobar que obtenemos un 404; finalmente, probaremos con la palabra buena, que es la recibida en el campo *word,* con la que esperamos obtener un 200.

El caso de prueba lo escribiremos en otro fichero al que llamaremos *TestSelectWordReal* (Figura 187).

```
package edu.uclm.esi.juegos;

import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
import org.json.JSONObject;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
```

```java
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.junit.jupiter.SpringExtension;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.RequestBuilder;
import org.springframework.test.web.servlet.request.MockMvcRequestBuilders;

@SpringBootTest
@ExtendWith(SpringExtension.class)
@AutoConfigureMockMvc
public class TestSelectWordReal {

   @Autowired
   private MockMvc server;

   @Test
   void test1() throws Exception {
      RequestBuilder request = MockMvcRequestBuilders.
         get("/words/selectWord?length=15");

      JSONObject jso = new JSONObject(this.server.perform(request).
         andExpect(status().is(200)).
         andReturn().getResponse().
         getContentAsString());

      String word = jso.getString("word");
      String disordered = jso.getString("disordered");


      request = MockMvcRequestBuilders.get("/words/checkWord/" + disordered);
      this.server.perform(request).
         andExpect(status().isNotFound());

      char c0 = disordered.charAt(0);
      char c14 = disordered.charAt(14);
      disordered = c14 + disordered.substring(1, 14) + c0;

      request = MockMvcRequestBuilders.get("/words/checkWord/" + disordered);
      this.server.perform(request).
         andExpect(status().isNotFound());


      request = MockMvcRequestBuilders.get("/words/checkWord/" + word);
      this.server.perform(request).
         andExpect(status().isOk());
   }
}
```

Figura 187. Código para probar con la base de datos real

Capítulo 8. Desarrollo del *front-end* para *Word race*

1. Planteamiento del *front-end*

En la vista funcional del sistema que mostramos en la Figura 35 (página 51), habíamos definido dos casos de uso para este juego en el lado cliente y otros dos en el servidor. Los de este ya los tenemos implementados y probados. Nos falta implementar los del lado cliente.

Igual que para *Order the numbers,* utilizaremos Angular, y trataremos de mantener el mismo estilo de diseño y arquitectura en ambos juegos.

Como se recordará, en el proyecto *fejuegos* (desarrollado en el Capítulo 4) disponemos de un componente de arranque (*app*) que tiene su vista (*app.component.html)* y su modelo *(app.component.ts)*. La vista (Figura 53, página 61) incluye una etiqueta <*app-otn*></*app-otn*> que indica que en la vista principal de la aplicación se incluya el componente de *Order the numbers*. Ahora, trataremos de conseguir lo mismo con *Word race*. Esto se ilustra en la Figura 188, que muestra un posible primer diseño del sistema tras haber incluido el nuevo juego.

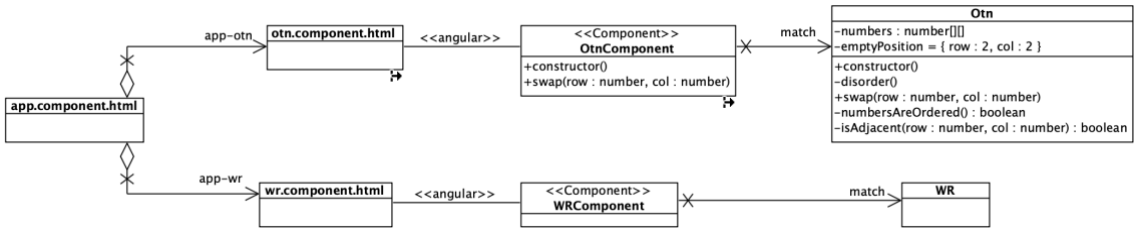


Figura 188. Incluiremos un componente wr *con el* Word race

Como vemos, igual que *OtnComponent* conoce a una instancia *match* de tipo *Otn,* también *WRComponent* conoce a una instancia *match* de tipo WR. En los dos casos, *Otn* y *WR* representan la lógica de cada uno de los juegos. Del mismo modo, igual que el usuario juega a *Order the numbers* mediante la vista *otn.component.html*, también jugará a *Word race* mediante *wr.component.html.* Y, así como el movimiento de un número en la vista provoca la ejecución de *swap* en el modelo, también los eventos que se recojan en *wr.component.html* se transmitirán al modelo *wr.component.ts*.

2. Primeros pasos

Como hemos comentado, daremos a los elementos de este juego una estructura paralela al de *Order the numbers*. Al usuario le aparecerán las letras desordenadas de una palabra y podrá hacer clic en dos de ellas, que se intercambiarán. La validez de la palabra intercambiada se comprobará con el servidor. En la clase *WR*, que es la *match* asociada al modelo, almacenaremos una cadena con la palabra que vaya componiendo el usuario.

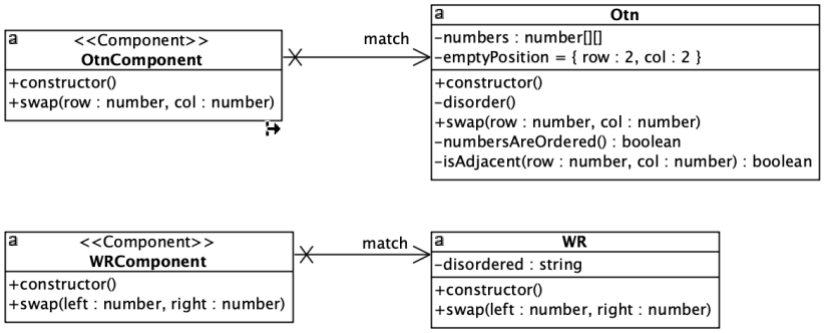


Figura 189. Estructura "paralela" de los dos juegos

3. Conexión con el *backend*

Una de las diferencias del primer juego con respecto a este segundo radica en que el primero no necesitaba acceder al servidor, pero este segundo sí. De alguna manera, tenemos que llegar desde el *front* hasta el *back* por *http:*

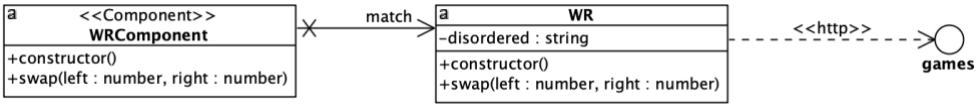


Figura 190. De algún modo debemos llegar al backend

Igual que en el desarrollo del *backend* hemos explicado que suele seguirse un determinado estilo arquitectónico y de paso de mensajes, en Angular también hay ciertos estilos habituales de diseño. Así, y aunque es posible acceder al servidor desde la clase *WR,* lo habitual es hacerlo desde el modelo del componente a través de un servicio. En cualquier caso, asumiremos que nuestro jefe de proyecto nos prohíbe acceder al servidor desde la clase de dominio, y nos obliga a hacerlo desde el modelo *WRComponent*.

Por tanto, en lugar de acceder de la forma mostrada en la figura anterior, accederemos de este otro modo:

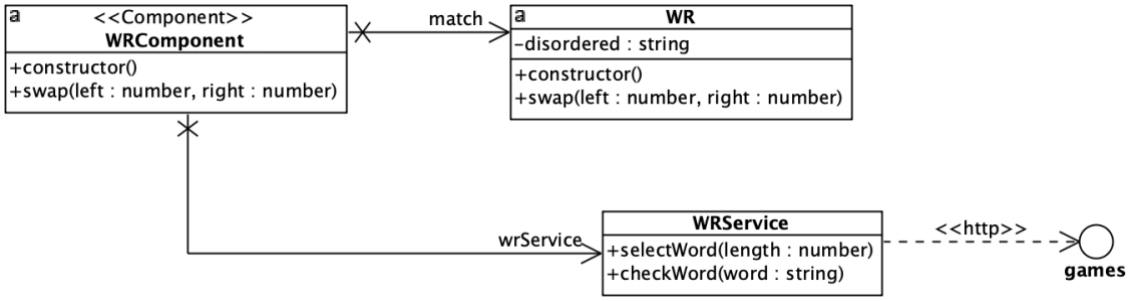


Figura 191. Acceso al servidor mediante un servicio

Para ello necesitamos crear un servicio de acceso al servidor. Crearemos un *WRService* con el comando *ng generate service WR,* que nos genera lo siguiente:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class WRService {

  constructor() { }
}
```

Figura 192. Código generado para WRService

El *WRService* accederá al servidor utilizando un *import { BrowserModule } from '@angular/platform-browser';*
import { NgModule } from '@angular/core';

import { AppComponent } from './app.component';
import { HttpClientModule } from '@angular/common/http';

@NgModule({
declarations: [
AppComponent
],
imports: [
BrowserModule,
HttpClientModule
],
providers: [],
bootstrap: [AppComponent]
})
export class AppModule { }, que es un módulo de Angular. Como vimos en el diagrama de clases anterior, *WRService* ofrece dos operaciones *selectWord* y *checkWord*, que enviarán peticiones *http* a los dos recursos homónimos del *backend* utilizando el citado *HttpClient,* para el cual declararemos un campo *client* de este tipo en *WRService*. Gráficamente se observa que, quien realmente accede al servidor, es el *HttpClient,* jugando el *WRService* el papel de un intermediario entre el componente y el cliente.

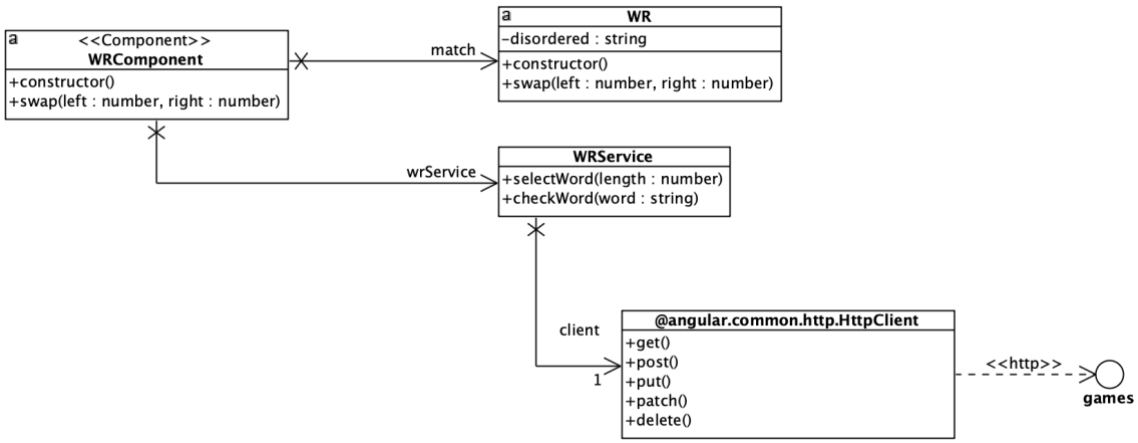


Figura 193. Es el HttpClient *quien realmente accede al servidor*

3.1. Importación del *HttpClient*

Para usar el *HttpClient* debemos declararlo en el fichero *app.module.ts,* para lo que necesitamos añadir las dos instrucciones que resaltamos en la siguiente figura:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http'

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { OtnComponent } from './otn/otn.component';
import { WrComponent } from './wr/wr.component';

@NgModule({
  declarations: [
    AppComponent,
    OtnComponent,
    WrComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    HttpClientModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Figura 194. Cambios en app.module.ts *para utilizar* HttpClient

Continuemos ahora creando la estructura de clases de la Figura 193.

3.2. Creación del campo *wrService* en *WrComponent*

En el diagrama de clases anterior, declaramos un campo *wrService* de tipo *WRService* en el *WrComponent*. Lo habitual sería escribir una línea declarando *wrService : WRService* en la zona de campos de la clase, pero las referencias a servicios se crean mediante la inyección del servicio en el constructor de la clase que lo utiliza. Es decir:

```
...
export class WrComponent implements OnInit {
  match : WR

  constructor(private wrService : WRService) {
    this.match = new WR()
  }
   ...
```

Figura 195. Inyección del servicio en el componente, que declara el campo

3.3. Creación del campo *client* en *WRService*

La creación de este campo se hace exactamente de la misma manera: declarándolo como un parámetro del constructor. Angular se encargará de pasar el objeto.

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class WRService {

  constructor(private client : HttpClient) { }
}
```

Figura 196. Inyección del cliente en el servicio

3.4. Envío de mensajes al servidor

Ya utilizamos esta figura en algún lugar del Capítulo 5, para ilustrar lo que sucede en el *backend* cuando llega un mensaje desde el navegador:

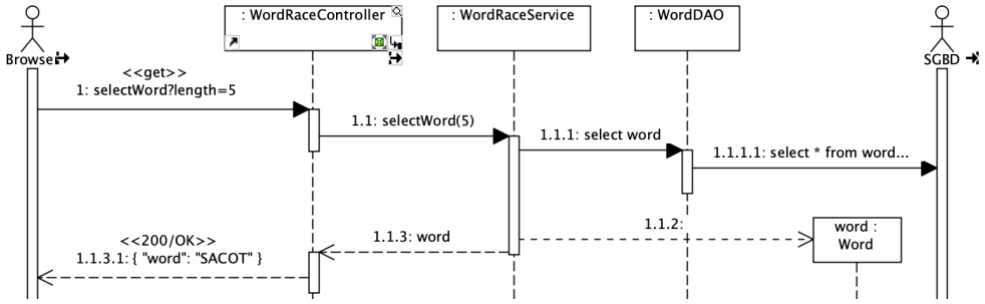


Figura 197. Cómo responde el backend *a la llegada de un mensaje*

Lo que ahora nos interesa es diseñar e implementar la forma en que los mensajes salen del lado cliente y llegan las respuestas del servidor. Por seguir con el ejemplo de la Figura 197, queremos describir qué sucede desde que el usuario hace clic en dos letras descolocadas en la vista de este juego hasta que el servidor le responde con un 200 o un 404, pasando por supuesto por la llamada al servicio web *checkWord*.

Un posible paso de mensajes se muestra en la Figura 198:

1) El usuario, usando la vista, hace clic en la primera letra de, supongamos, la cadena SCATO.
2) Esto supone que sobre el objeto *WrComponent* asociado a la vista se ejecute el método *select(1)*.
3) El usuario hace clic sobre la segunda letra; el componente detecta que ya había una letra seleccionada y entonces invoca al *swap(1,2)* de *match*.
4) Además, desde el componente debe llegarse al *WordRaceController* para preguntarle si la cadena CSATO es una palabra del diccionario. El componente le pregunta al servicio, que hace uso del *client* para enviar la petición *http*. El controlador, en este ejemplo, devuelve un error 404, lo que indica que esa cadena no se ha encontrado en el diccionario.

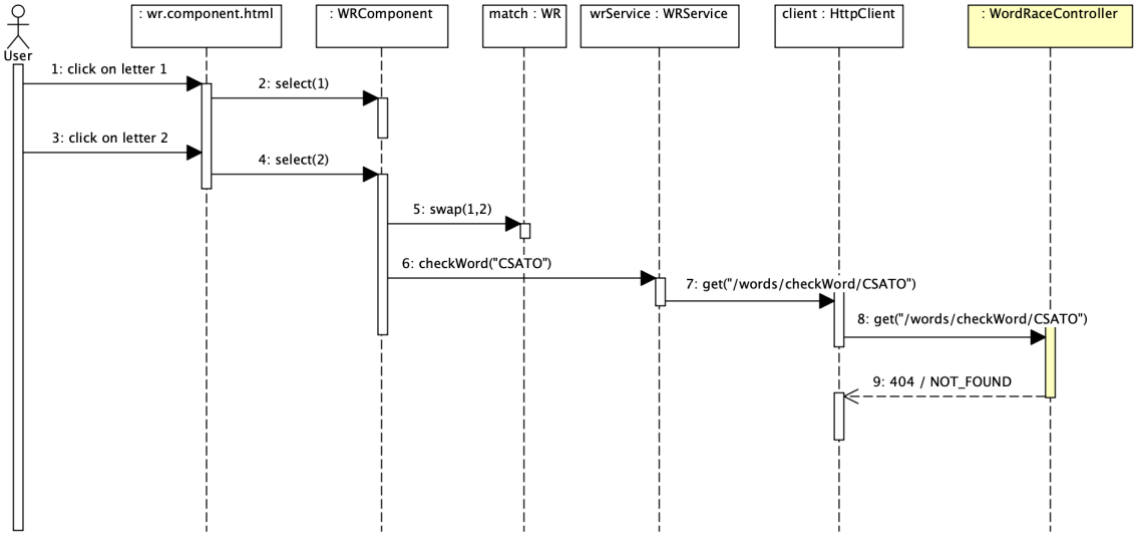


Figura 198. Paso de mensajes para cierto escenario en el lado cliente

Se habrá notado que, ahora, no está la operación *swap* que habíamos previsto como miembro de *WRComponent* en el diagrama de clases de la Figura 193. No pasa nada: al profundizar y hacer un diseño más detallado, hay cosas que aparecen y cosas que desaparecen.

3.5. Implementación de *WrComponent (I)*

De acuerdo con el diagrama de secuencia de la figura anterior, *WrComponent* debe tener un método *select* que reciba la posición de la letra sobre la que el usuario ha hecho clic. Si es la segunda vez que se pulsa, se hace otra cosa (llamar al método *swap* de *match*). Para tomar esta decisión, guardaremos en una variable *firstPos* el valor de la primera posición sobre la que se hace clic. La declararemos como *firstPos? : number*. La interrogación indica que la variable puede no tener valor (es decir, ser *undefined)*.

De momento, el código de la clase será el siguiente: en el método *select*, si no hay valor en *firstPos,* lo asignamos; si sí lo hay, llamamos a *swap* y lo desasignamos estableciéndolo a *undefined*.

```
export class WrComponent implements OnInit {
  match : WR
  firstPos? : number

  constructor(private wrService : WRService) {
    this.match = new WR()
  }

  ngOnInit(): void {
  }

  select(pos : number) {
    if (!this.firstPos) {
      this.firstPos = pos
    } else {
      this.match.swap(this.firstPos, pos)
      this.firstPos = undefined
    }
  }
}
```

Figura 199. Código de WrComponent

3.6. Implementación de *WR*

WR representa en el cliente la partida de *Word race*. Según el diagrama de clases de la Figura 193, dispone de:

1) Un campo *disordered* de tipo *string* que contiene la cadena que se recibe del servidor y que va recomponiendo el usuario.
2) Un constructor.
3) Un método *swap(left : number, right : number)* que hay que implementar, y que intercambiará las posiciones *left* y *right* de la cadena *disordered*.

En la siguiente figura se muestra una posible implementación de esta clase, en la que hay algún detalle interesante:

1) El campo está declarado como *disordered?,* lo que indica que puede tomar un valor *undefined*. Si quitásemos la interrogación se nos advertiría de un error de compilación: o le asignamos valor directamente en la declaración (por ejemplo: *disordered : string = "CSATO")* o en el constructor.
2) En la primera del método *swap* hacemos referencia a *this.disordered!* (termina en una admiración). Esto se utiliza cuando estamos seguros de que en ese punto la variable tendrá algún valor definido. Realmente, no es nuestro caso, pues no estamos asignándole valor a la cadena desde ningún punto de la aplicación. Debemos hacerlo cuando, al cargarse el componente, se llame al servicio para ejecutar *selectWord* con la longitud deseada.

```
export class WR {

    disordered? : string

    constructor() {}

    swap(left: number, right: number) {
        let chars = Array.from(this.disordered!)
        let leftChar = chars[left]
        chars[left] = chars[right]
        chars[right] = leftChar
        this.disordered = ""
        for (let i=0; i<chars.length; i++)
            this.disordered = this.disordered + chars[i]
    }

}
```

Figura 200. Primera implementación de WR

3.7. Implementación de *WrComponent (II)*

Como acabamos de comentar, debemos dar valor a *disordered* cuando se cargue el componente del *Word race*. En el código del componente tenemos dos posibles lugares en los que hacerlo: en el constructor y en el método *ngOnInit*, que está de momento vacío.

En cualquiera de los dos casos, lo que debemos hacer es enviar una petición al servidor para que nos dé una palabra de la longitud que deseemos utilizando, claro, el *WRService* que tenemos pendiente de implementar.

El constructor del componente se ejecuta antes de que se cargue el árbol DOM (*Document Object Model:* la estructura de objetos que HTML se carga en la vista), mientras que *ngOnInit* se ejecuta después. Si colocamos la llamada en el constructor, es posible que la respuesta (que ha de mostrarse en la vista) llegue antes de que se haya cargado el DOM, por lo que podría ocurrir un error en tiempo

de ejecución. Por eso, lo colocaremos en *ngOnInit,* al que daremos la siguiente implementación:

```
ngOnInit(): void {
    this.wrService.selectWord(this.match, 5)
}
```

Figura 201. Utilización del servicio

Dijimos hace unas páginas que inyectando el servicio en el constructor se declara un campo. En efecto, en el constructor pasábamos el parámetro *private wrService : WRService,* y hacemos uso de este campo que se nos crea en la Figura 201.

3.8. Implementación del servicio *WRService (I)*

La operación *selectWord* utilizada en la Figura 201 toma dos parámetros: la partida de *Word race* y la longitud de la palabra buscada. Implementemos esta operación en *WRService*.

De momento, el código completo de esta clase será el siguiente:

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { WR } from './wr/WR';

@Injectable({
  providedIn: 'root'
})
export class WRService {

  constructor(private client : HttpClient) { }

  selectWord(match: WR, length: number) {
    this.client.get<any>("http://localhost/words/selectWord?length=" + length)
  }
}
```

Figura 202. Código inicial de WRService

Como vemos, *selectWord* utiliza el campo inyectado *this.client* para enviar una petición de tipo *get* (recordemos que el servicio web *selectWord* en el *WordRaceController* es de este tipo) a la url que se pasa como parámetro.

Si recordamos el desarrollo del *backend*, el resultado que nos devolverá este servicio será un objeto de tipo *Word* en JSON, con los tres campos *id, word* y *disordered*. De estos tres campos, solo nos interesa el tercero, que contiene la cadena desordenada. No tendremos, entonces (al menos de momento), una clase *Word* en el *front-end:* por esto, después del *get*, escribimos *<any>,* para indicar que la respuesta que esperamos puede ser de cualquier tipo: un JSON cualquiera en este caso.

En aplicaciones web es posible que la respuesta tarde mucho en llegar (por complejidad del cálculo solicitado al servidor, por latencias grandes, etcétera); a la vez, se desea que la experiencia del usuario sea buena, en el sentido de que, si en efecto tarda en llegar, al usuario no se le bloquee la interfaz. Por ello, los métodos *get, post,* etcétera del tipo *HttpClient* no devuelven directamente la respuesta, sino un *observable* de la respuesta, que se llena cuando el cliente la recibido completamente.

La siguiente figura explica el funcionamiento de los observables, y también de las aplicaciones Angular, con más detalle:

1) Cuando la aplicación arranca, Angular carga primero los módulos importados en *app.module.ts:* entre ellos, el *HttpClient* (recuérdese Figura 194, página 138).
2) Luego, carga los servicios de nuestra aplicación, inyectando si es necesario algún objeto de los que ha creado previamente. En el mensaje 2, crea el *WRService* y le inyecta el *HttpClient*.
3) En los mensajes 3 y 4 carga la vista y el modelo y los enlaza, inyectando al modelo, si se requiere (en nuestro caso sí, como se muestra en el mensaje 4) algún objeto creado previamente (el *WRService).*
4) El constructor del componente (mensaje 5) crea una partida vacía de *Word race*.
5) En el mensaje 6, Angular llama automáticamente a *ngOnInit,* que solicita al servicio (mensaje 6.1) una palabra de 5 letras.
6) El servicio llama (mensaje 6.1.1) al método *get* del *HttpClient* que, antes de enviar la petición, construye un observable (mensaje 6.1.1.1) en el que colocará la respuesta cuando esta llegue. Luego, envía la petición al servidor (mensaje 6.1.1.2).
7) Supongamos que la respuesta tarda 30 segundos en llegar. Bien, pues el observable está ahí vacío, sin nada durante ese rato; cuando la respuesta llega (mensaje 7), el cliente rellena el observable con la información recibida.

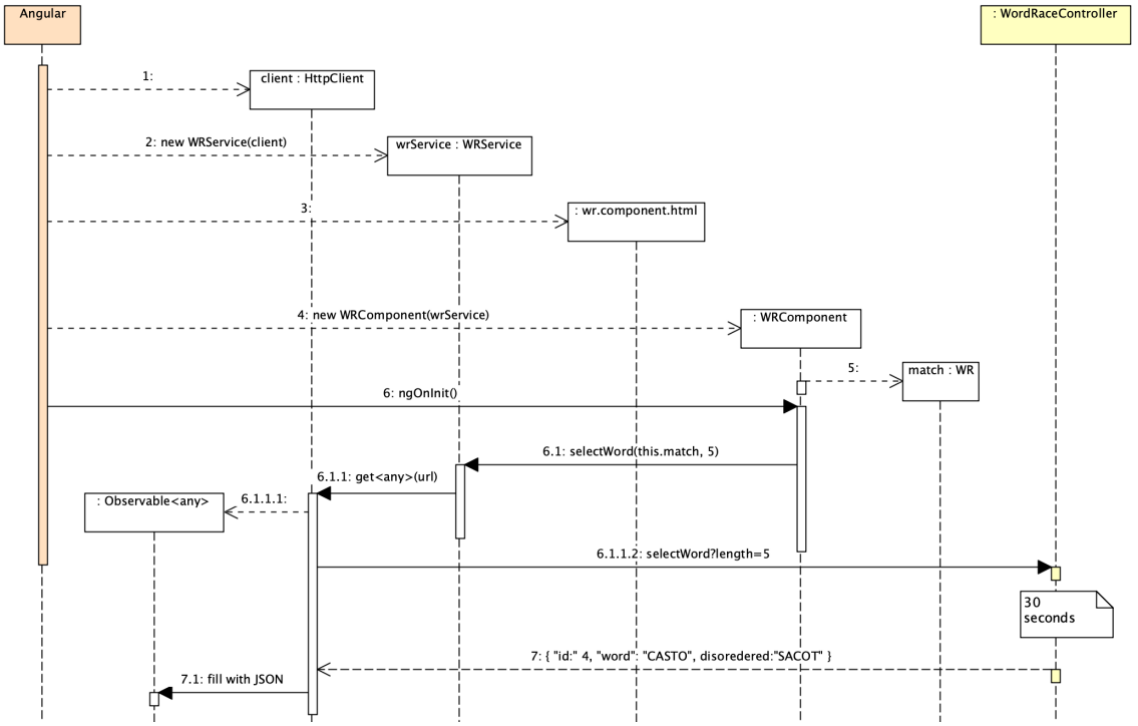


Figura 203. Arranque de la aplicación Angular, envío de una petición y rellenado de un observable

Para poder utilizar la respuesta, debemos suscribirnos al observable, de modo que podamos recoger la información que contiene para poder procesarla.

En el código que mostrábamos en la Figura 202, el *client* envía la petición *get* pero no se hace con la respuesta. El tipo devuelto por esa instrucción es un observable que, cuando se rellene, tendrá el JSON con la palabra devuelta por el servidor.

Para leer la respuesta y poder procesarla, decíamos que tenemos que suscribirnos a ese observable. En la siguiente figura se muestra el nuevo código de *selectWord:*

1) La llamada *this.client.get<any>(url)* permanece igual.
2) Con *subscribe* indicamos que nos queremos suscribir a este observable, para tener conocimiento de la respuesta cuando esta llegue.
3) Cuando llega, cogemos la respuesta en ese objeto *result,* que tendrá los tres campos que ya sabemos (*id, word* y *disordered)*. Lo que hacemos con este resultado es leer el campo *disordered* y asignarlo al campo *disordered* de la *match* que nos ha pasado el componente como parámetro. Y, como estamos desarrollando, también mostrador el objeto por la consola.

```
selectWord(match: WR, length: number) {
  this.client.get<any>("http://localhost/words/selectWord?length=" + length).
    subscribe(result => {
      match.disordered = result.disordered
      console.log(JSON.stringify(result))
    })
}
```

Figura 204. Nuevo código de selectWord

3.9. Enlace de la vista con el modelo (I)

En la parte visual del componente solo tenemos, en este momento, el código por defecto que nos genera Angular: * wr works! *. Obviamente, tenemos que mostrar de algún modo las letras de la palabra, recoger los eventos clic que el usuario realice sobre ellas y llamar a la operación *select* de la clase *WordRace*.

Procedemos de forma parecida a como hicimos con *Order the numbers:* pondremos un botón por cada letra, a cuyos eventos *click* asociaremos la ejecución de *select*.

Por ahora, escribimos en *wr.component.html* el siguiente código:

```
<p>wr works!</p>

<div>
    <button *ngFor="let c of match.disordered; let i=index" (click)="select(i)">
        {{ c }}
    </button>
</div>
```

Figura 205. Código en la vista del juego

3.10. Adición de *Word race* a la pantalla principal

Como ya tenemos creado el componente para el segundo juego, podemos incluir una etiqueta *<app-wr></app-wr>* en la vista del componente de arranque:

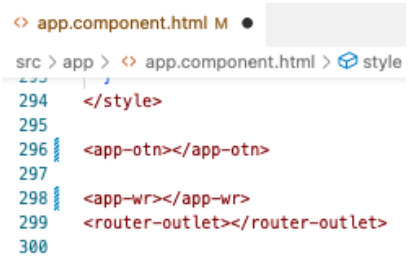


Figura 206. Añadimos la etiqueta para cargar el Word race

4. Ejecución

Nos vamos a encontrar un problema imprevisto en la ejecución del sistema. Vamos a buscarlo.

En primer lugar, arrancamos el contenedor con SQL Server, luego el servidor en Eclipse y, finalmente, escribimos *ng serve* para lanzar el servidor de Angular en el puerto 4200.

Con el último cambio que hemos hecho (Figura 206) esperamos que, al dirigir el navegador a *localhost:4200,* se produzca todo el ciclo de creación de objetos que mostramos en la Figura 203 de modo que, cuando se cree el componente *WrComponent*, se envíe una petición al servidor web de Eclipse (en *localhost:80)*, este busque una palabra aleatoria en la base de datos, se asigne el valor desordenado al campo *disordered* de la *match,* y se muestren las letras en los botones que hemos creado hace un momento.

Para comprobar si la petición llega al recurso *selectWord* del servidor, ponemos un punto de interrupción en su única línea de código:

```
22⊖    @GetMapping("/selectWord")
23     public Word selectWord(@RequestParam int length) {
24         return this.wordRaceService.selectWord(length);
25     }
```

Figura 207. Un punto de interrupción en selectWord

Luego, abrimos un navegador y lo llevamos a *localhost:4200:*

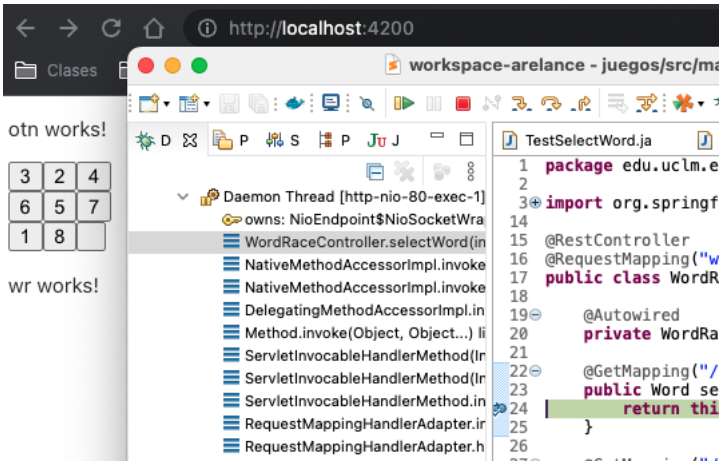


Figura 208. La petición llega al servidor

Cuando soltamos el depurador, sin embargo, el resultado no se muestra en la pantalla. Puede ser que algo hayamos hecho mal en la vista de la Figura 205, o que no estemos recibiendo correctamente la respuesta.

4.1. Depuración del cliente

Como sabemos que la petición llega al servidor, podemos eliminar el punto de interrupción que hemos colocado. Luego, en el navegador, abrimos las opciones de desarrollo, localizamos el *WRService* y le ponemos los puntos de interrupción que se resaltan en la figura:

1) Esperamos que la ejecución se detenga en el primer punto, justo antes de enviar la petición al servidor.
2) Cuando la respuesta llegue, esperamos que la ejecución se detenga de nuevo en la línea 15.

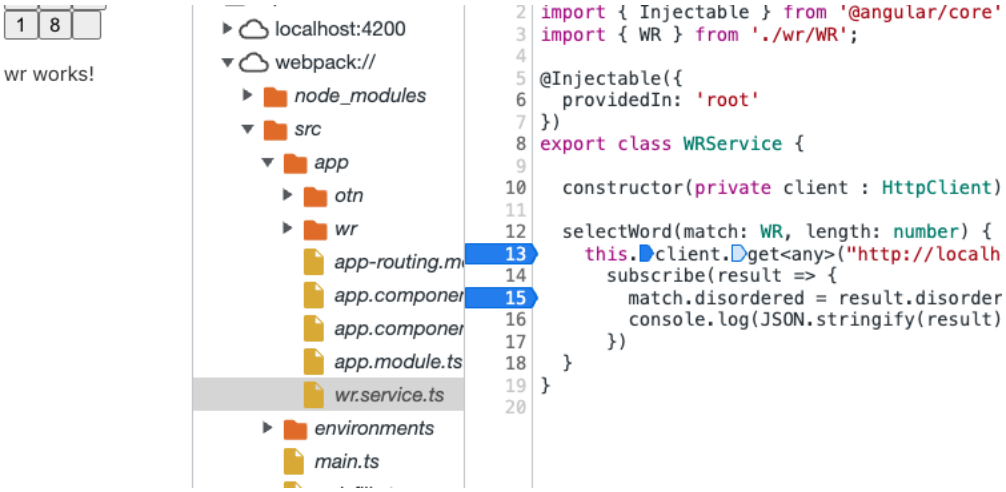


Figura 209. Puntos de interrupción en el lado cliente

Si ahora recargamos la página, el código se detiene en el primer punto, pero pasa del segundo. O sea, parece que la petición se envía pero que la respuesta no se recibe.

Para indagar un poco más, vayamos a la solapa *Network* de las opciones de desarrollo del navegador. Observamos que la petición al recurso *selectWord* está fallando:

Name	Url	Status	Type
localhost	http://localhost:4200/	304	docum
styles.css	http://localhost:4200/styles.css	304	stylesh
runtime.js	http://localhost:4200/runtime.js	304	script
polyfills.js	http://localhost:4200/polyfills.js	304	script
vendor.js	http://localhost:4200/vendor.js	304	script
main.js	http://localhost:4200/main.js	304	script
styles.js	http://localhost:4200/styles.js	304	script
ng-cli-ws	ws://localhost:4200/ng-cli-ws	101	webso
selectWord?le...	http://localhost/words/selectWord?length=5	CORS error	xhr
favicon.ico	http://localhost:4200/favicon.ico	304	vnd.mi

Figura 210. Error en la petición a selectWord

Se nos informa de que es un error por CORS. Una petición CORS (*Cross-Origin Resource Sharing)* es una petición que se hace a un servidor diferente del "origen". El origen es el servidor al que nuestro navegador se conecta inicialmente (en nuestro caso, *localhost:4200)*. Cuando escribimos *localhost:4200* en el navegador, este envía una petición *GET* al servidor, que se convierte en el origen. Típicamente, el servidor entrega al navegador algo de código HTML, que incluye instrucciones de inclusión de scripts, hojas de estilo, imágenes, etcétera. El navegador recibe ese HTML y lo va procesando; según lo va procesando e interpretando, es posible que envíe otras peticiones al mismo u a otros servidores para recibir más recursos.

Cada fila de las que aparecen en la figura anterior es una petición que el navegador ha enviado, como consecuencia de interpretar el código HTML recibido de la primera.

En efecto, el cuerpo de la respuesta a la primera petición es el que se muestra en el lado derecho de la siguiente figura: encontramos una instrucción *link rel* para descargar el fichero *styles.css,* razón por la que el navegador envía la petición

para recuperar este recurso, que aparece en la segunda fila; encontramos también la instrucción de traerse el script *runtime.js*, que es lo que provoca la petición de la tercera fila. El resto de peticiones aparecen en la misma línea de este script, por lo que no se ven.

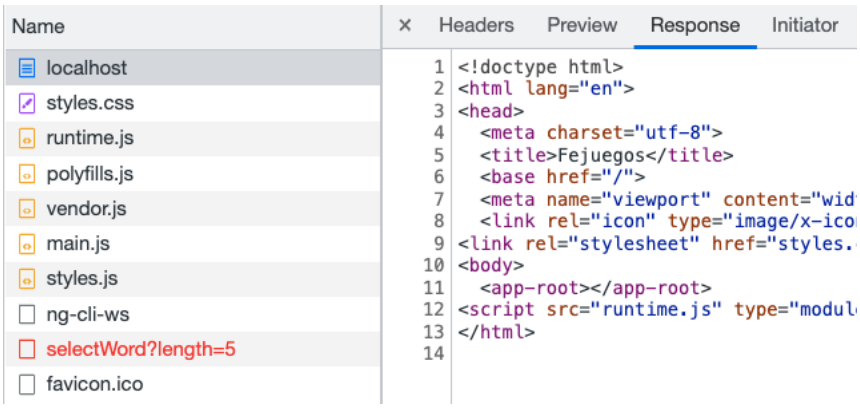


Figura 211. Cuerpo de la respuesta a get localhost:4200

La petición que falla está dirigida a un servidor distinto del origen (ambos servidores están en *localhost,* pero cada uno en un puerto diferente) por lo que el navegador bloquea la respuesta. En la Figura 212 se ilustra además el hecho de que el navegador envía las peticiones según va procesando el código HTML recibido en el mensaje 2, pero las respuestas a estas nuevas peticiones pueden ir llegando en distinto orden: el mensaje 5 es la respuesta al mensaje 4, pero la respuesta al mensaje 3 *(get style.css)* no llega hasta el mensaje 7.

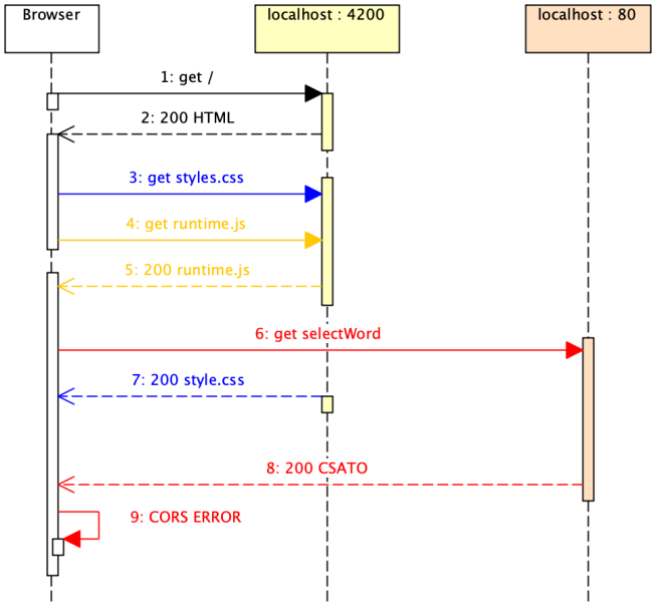


Figura 212. Petición enviada a un servidor distinto del origen

Para que nuestro cliente admita y procese la respuesta, necesitamos al menos que el servidor incluya una cabecera indicando que admite peticiones de cualquier origen. Así que paremos el servidor, añadamos la anotación *@CrossOrigin(origins = "*")* al *WordRaceController* y ejecutémoslo de nuevo.

Ahora sí, el resultado llega al segundo punto de interrupción que habíamos colocado en el cliente (Figura 213, izquierda). Como vemos en el lado derecho, la respuesta incuye una cabecera llamada *Access-Control-Allow-Origin* con valor *.

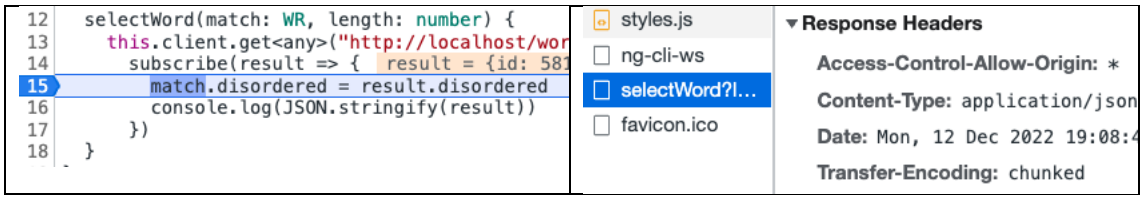


Figura 213. La respuesta ya puede procesarse

4.2. Corrección de la vista

La petición se envía, la palabra se recibe, asignamos la cadena desordenada a *match.disordered* pero, sin embargo, no se muestra en el navegador. El problema viene porque en el **ngFor* pretendemos iterar sobre los caracteres de una cadena, pero una cadena no es iterable.

Bueno, pues nada, iteremos sobre otra cosa: el método *split(token)* de JavaScript, aplicado a una cadena, devuelve un array de cadenas separados por el *token*. Iteremos sobre el resultado de "tokenizar" la cadena (Figura 214, arriba) usando como token la cadena vacía (Figura 214, abajo):

```
<div>
    <button *ngFor="let c of splitDisordered(); let i=index" (click)="select(i)">
        {{ c }}
    </button>
</div>
```

```
  splitDisordered() {
    return this.match.disordered!.split('')
  }
```

Figura 214. Ahora, en la vista llamamos a un método que hemos escrito en el modelo

Ahora, si ejecutamos, se muestra en el navegador la palabra desordenada. ¡Bingo!

Figura 215. En la parte inferior, el Word race *nos muestra la palabra "afinen" desordenada*

5. Compleción del código del cliente

Con el último cambio, el evento *click* se propaga correctamente al modelo, que a su vez lo pasa al objeto de dominio *match* y hace el intercambio de letras. De acuerdo con la especificación del juego, nos falta consultar cada combinación con el servidor y, si la combinación que envía el usuario es correcta, generar una nueva palabra.

Reproducimos otra vez la figura que muestra el escenario en el que el usuario intercambia dos letras, se envía la consulta al servidor acerca de esta palabra y se obtiene un código 404.

Como vemos en el diagrama, es el *WrComponent* el responsable de preguntar al servidor a través del servicio. Vamos entonces al código del método *select* en *WrComponent* y enviemos la petición cada vez que se selecciona la segunda letra.

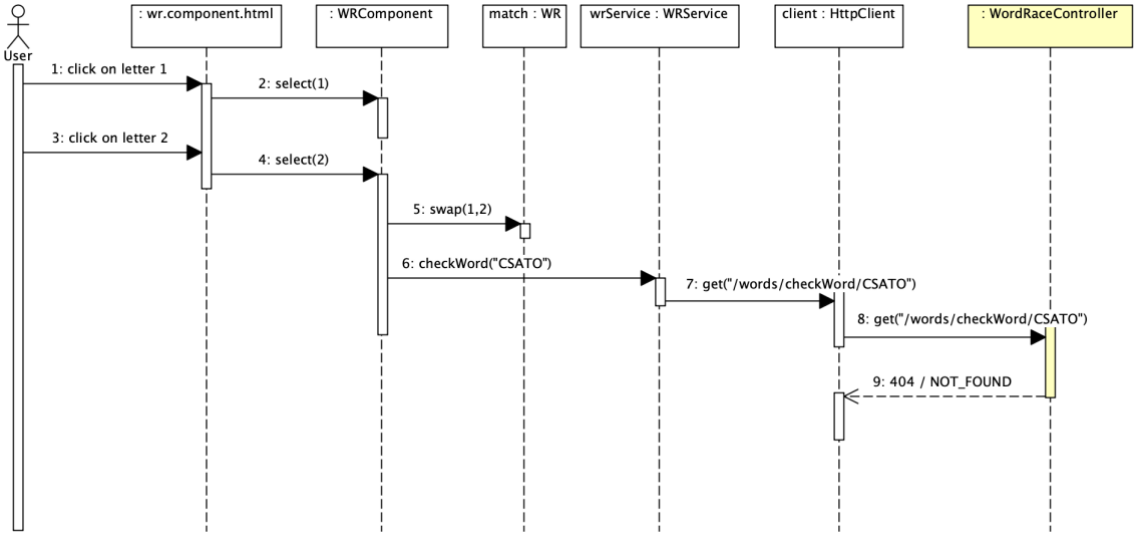


Figura 216. El servidor comprueba que CSATO no es una palabra del diccionario

En principio basta con incluir la sentencia resaltada en el método *select* del componente:

```
  select(pos : number) {
    if (!this.firstPos) {
      this.firstPos = pos
    } else {
      this.match.swap(this.firstPos, pos)
      this.firstPos = undefined
      this.wrService.checkWord(this.match.disordered)
    }
  }
```

Figura 217. Inclusión de una llamada para comprobar la palabra tras el intercambio de letras

El método *checkWord* que debemos incluir en el *WRService* enviará una petición al servicio web *checkWord* del *WordRaceController* que, como recordaremos, no devolvía nada salvo el código de respuesta (200 o 404).

Sobre la llamada a *checkWord* de la figura anterior podemos llamar al asistente para que nos cree el método vacío, por defecto, en *WRService*. Luego, le damos la siguiente implementación, a ver qué muestra cuando se recibe la respuesta:

```
checkWord(disordered: string | undefined) {
    this.client.get<any>("http://localhost/words/checkWord/" + disordered).
      subscribe(result => {
        console.log(JSON.stringify(result))
      })
}
```

Figura 218. Código para ver qué pasa

Pues lo que pasa es que no pasa nada, y esto ocurre porque el primer resultado de *subscribe* solo se inyecta cuando la respuesta recibida desde el servidor es un código 200. Si es 404, la tenemos que coger de otra manera:

```
19    checkWord(disordered: string | undefined) {
20      this.client.get<any>("http://localhost/words/checkWord/" + disordered).
21        subscribe(
22          result => {
23            console.log(JSON.stringify(result))
24          },
25          error => {
26            console.log(error)
27          })
28    }
```

Figura 219. Captura del error 404 en la llamada al servicio web

En la figura anterior, vemos que en el objeto *result* (o como queramos llamarlo) recibimos la respuesta cuando el código es 200, y que en *error* (también puede recibir cualquier otro nombre) recibimos la respuesta cuando el código es de la serie 400 o 500 (errores). Vemos también que *subscribe* aparece tachado, lo que indica que podemos seguir usando ese método, pero que está obsoleto (*deprecated)*. Si usamos un método "deprecado"[7] corremos el riesgo de que en no mucho tiempo deje de ser entendido por el compilador, lo que puede ponernos en problemas en futuras versiones del sistema.

Total, que vamos a intentar hacer un código que no se, permítasenos la expresión, *depreque* pronto. Buscamos en Internet con la cadena "angular get subscribe error deprecated", que nos lleva a StackOverflow. En este caso nos fijamos en la segunda respuesta mejor valorada, dada por el usuario Martin, que nos explica que no es que el método *subscribe* esté obsoleto, sino la forma de uso que le hemos dado en la Figura 219.

La última versión de *subscribe* toma como parámetro un objeto JSON que puede tomar varios campos, al menos *next, error* y *complete*. El valor de cada uno estos elementos es una función que se encarga de manejar la respuesta obtenida. En fin, siguiendo el consejo de Martin, reescribimos *checkWord* de la siguiente forma:

```
checkWord(disordered: string | undefined) {
   this.client.get<any>("http://localhost/words/checkWord/" + disordered).
     subscribe( {
           next : (result) => console.log(result),
           error : (result) => console.log(result)
       }
     )
}
```

Figura 220. "Modernizamos" la llamada a subscribe

En el uso que haremos a partir de ahora de *subscribe* vemos que, en realidad, tanto *next* como *result* son dos manejadores de eventos: ambos pueden traer un resultado, que pasamos al código que ponemos después del operador flecha (=>).

En la Figura 221 se muestra el código de *WrService,* detenido al recibir el error 404, puesto que la palabra *pasna* no se ha encontrado en el diccionario.

[7] En castellano, "deprecar" significa algo completamente distinto: "Rogar, pedir, suplicar con eficacia o instancia".

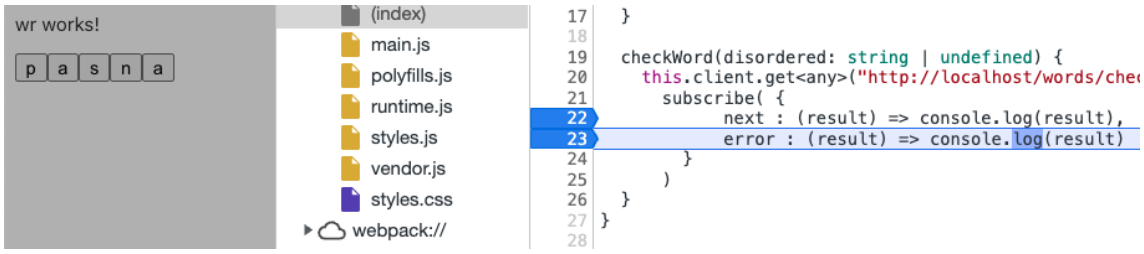


Figura 221. Llegada de una respuesta 404

En la siguiente, se muestra que *pasan* sí que está en ese enorme diccionario de más de seiscientas mil palabras que hemos guardado en la base de datos, pues la ejecución se para en el campo *next*.

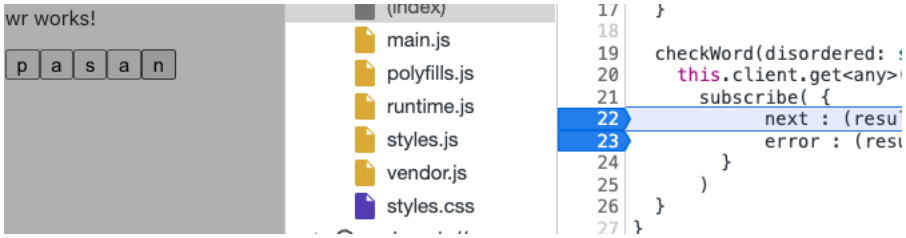


Figura 222. Llegada de una respuesta con código 200

Lo cierto es que, si la palabra no se encuentra, no tenemos que hacer nada; sin embargo, en caso de éxito, sí que debemos solicitar una nueva palabra: pues en este punto es en donde vamos a poner ahora la atención.

De alguna manera, tenemos que devolver al componente el resultado devuelto por el servidor. Probemos a devolver *true* si es 200, y *false* si es 404, como en la Figura 223: con respecto a la última implementación, ya no capturamos entre los paréntesis después de *next* y *error* el resultado obtenido, pues no nos hace falta procesarlo.

```
checkWord(disordered: string | undefined) {
    this.client.get<any>("http://localhost/words/checkWord/" + disordered).
      subscribe( {
            next : () => { return true },
            error : () => {return false }
        }
      )
}
```

Figura 223. Devolvemos true *o* false *según el código de respuesta*

El valor *true* o *false* que estamos devolviendo debemos capturarlo allá donde llamemos a este método *checkWord*. Si no recordamos desde dónde lo hacemos, o si lo hacemos desde muchos sitios y queremos localizar un concreto, podemos pulsar sobre el método en VSC y elegir *Find All References* (lado izquierdo de la Figura 224). En el lado derecho nos aparecen las dos referencias: la declaración en el propio servicio y la llamada desde el componente.

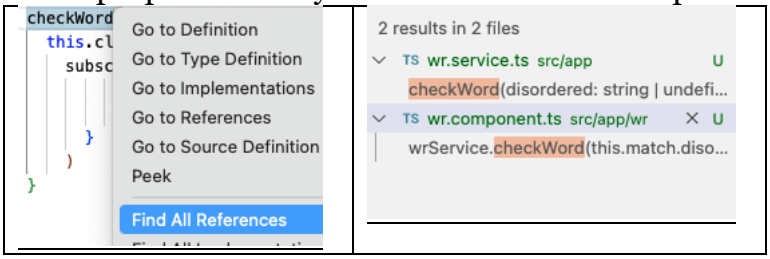


Figura 224. Podemos encontrar las referencias que se hagan a cualquier miembro de una clase

Bien, pues vamos a la referencia válida (la que hace el componente) y en este sitio preguntaremos por el resultado devuelto: si es *true,* generamos una nueva *match* llamando a *ngOnInit;* si no, seguimos jugando como si tal cosa:

```
  select(pos : number) {
    if (!this.firstPos) {
      this.firstPos = pos
    } else {
      this.match.swap(this.firstPos, pos)
      this.firstPos = undefined
      if (this.wrService.checkWord(this.match.disordered))
        this.ngOnInit()
    }
  }
```

Figura 225. Si la palabra existe, generamos una partida nueva

Pero el compilador nos advierte de que no podemos utilizar el resultado de *checkWord* en una instrucción condicional, y lleva razón, pues la declaración de *checkWord* no incluye ningún tipo (es *void)*:

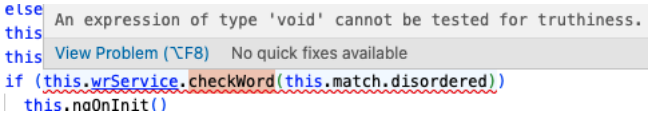


Figura 226. checkWord *no es booleano*

La solución parece sencilla: declaremos que *checkWord* en *WRService* devuelve un boolean y asunto arreglado. O sea:

```
checkWord(disordered: string | undefined) : boolean {
    this.client.get<any>("http://localhost/words/checkWord/" + disordered).
      subscribe( {
            next : () => { return true },
            error : () => {return false }
        }
      )
 }
```

Figura 227. Declaramos como boolean *el resultado de* checkWord

Pero, desgraciadamente, el compilador nos advierte de un error: *A function whose declared type is neither 'void' nor 'any' must return a value*: una función cuyo tipo declarado no es ni *void* ni *any* debe devolver un valor, y es posible que nuestra función, que envía una petición a un servidor y espera en un observable obtener una respuesta, no devuelva nada. Quizás podríamos añadir una última sentencia, *return false* o *return true,* pero este resultado se devolvería al componente antes de recibir la respuesta del servidor, con lo que no nos sirve, porque la petición que enviamos es asíncrona: es decir, que se manda y "ya vendrán los resultados" cuando estén listos.

Esta circunstancia se ilustra en la Figura 228: el usuario dispone de la cadena *"laaic"*, e intercambia las dos primeras letras. En ese momento, se ejecuta *checkWord* en el servicio: como la respuesta se envía al *backend* y tarda un rato en llegar, se llega directamente a la línea 26 y se devuelve *false;* la petición, sin embargo, se ha enviado al servidor y la respuesta llega más tarde y rellena el observable, con lo que se ejecuta el segundo *return false* que, evidentemente, ya no tiene sentido.

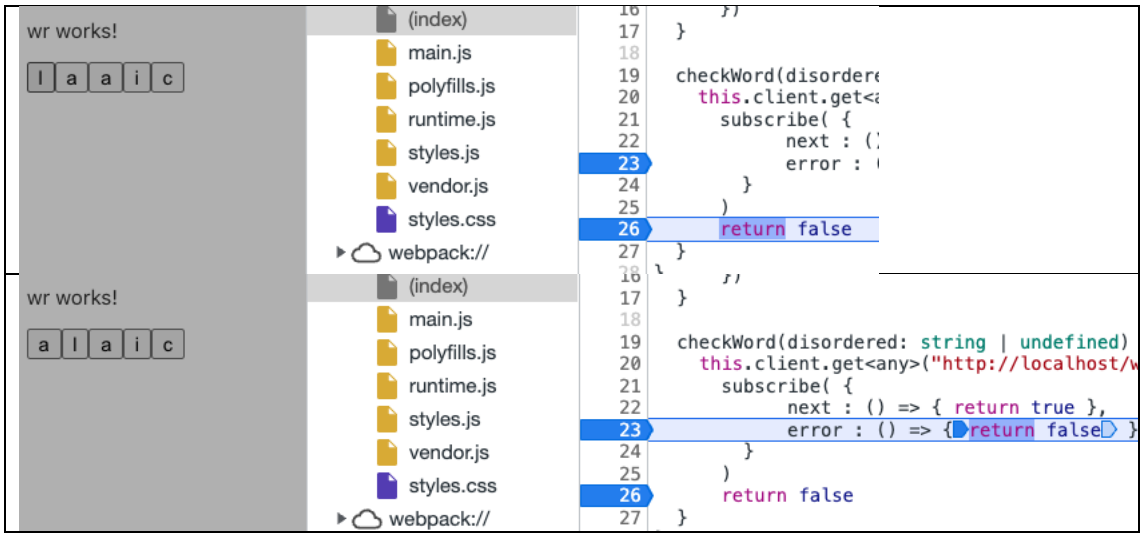


Figura 228. Una petición asíncrona puede ejecutar varios return

5.1. Procesamiento del observable

Tenemos al menos tres formas posibles de procesar la respuesta. Recordemos que, cuando la respuesta llegue, si su código es 200, hay que actualizar la partida solicitando al servidor una nueva palabra.

1) La primera forma de procesar la respuesta es implementar el método *checkWord* de forma parecida a como está implementado *selectWord:* es decir, pasar como parámetro la partida (objeto *match*) que queremos actualizar, y actualizarla si el código es 200. En este caso, el método podría volver a ser *void*.
   El resultado se muestra en la Figura 229: cuando la respuesta es un código 200, le decimos a la partida que solicite una nueva palabra de 5 letras. Sin embargo, por la imposición de estilo arquitectónico y de diseño que ya conocemos, la clase de dominio *WR* no puede acceder al servicio, por lo que descartamos esta solución.

```
checkWord(match : WR, disordered: string | undefined) {
  this.client.get<any>("http://localhost/words/checkWord/" + disordered).
    subscribe( {
          next : () => {
            match.selectNewWord(5)
          }
      }
    )
}
```

Figura 229. Una posible solución que no nos tienen permitida

2) La segunda forma requiere solo una pequeña corrección al código anterior: en lugar de decirle a la partida que solicite una palabra nueva, le decimos al propio *WRService* en el que nos encontramos que solicite una palabra para la partida que nos están pasando como parámetro. Es decir:

```
checkWord(match : WR) {
  this.client.get<any>("http://localhost/words/checkWord/" + match.disordered).
    subscribe( {
          next : () => {
            this.selectWord(match, 5)
          }
      }
    )
}
```

Figura 230. Una posible solución permitida

En el código del componente, eliminamos la instrucción condicional que habíamos puesto en la Figura 225 y dejamos solamente una llamada a *checkWord:*

```
select(pos : number) {
  if (!this.firstPos) {
    this.firstPos = pos
  } else {
    this.match.swap(this.firstPos, pos)
    this.firstPos = undefined
    this.wrService.checkWord(this.match)
  }
}
```

Figura 231. checkWord *ya no devuelve nada*

3) La tercera forma consiste en suscribirnos al observable desde fuera del servicio. En nuestro caso, declaramos el método *checkWord* con el tipo devuelto *Observable<any>*, y capturamos la respuesta en la llamada que hacemos en el componente:
   a. La fila de arriba de la Figura 232 muestra el nuevo código de *checkWord,* en donde hemos explicitado el tipo devuelto por el método.
   b. En la fila de abajo llamamos al servicio y nos suscribimos al observable que devuelve. Cuando llega un 200, solicitamos la nueva palabra llamando al servicio.

```
checkWord(disordered : string) : Observable<any> {
  return this.client.get<any>("http://localhost/words/checkWord/" + disordered)
}
```

```
select(pos : number) {
  if (!this.firstPos) {
    this.firstPos = pos
  } else {
    this.match.swap(this.firstPos, pos)
    this.firstPos = undefined
    this.wrService.checkWord(this.match.disordered!).
      subscribe( {
            next : () => {
              this.wrService.selectWord(this.match, 5)
            }
        }
      )
  }
}
```

Figura 232. Una tercera solución

5.2. Comentarios a la tercera solución

Con la tercera solución tenemos dos métodos en *WRService:* el primero es de tipo *void,* solicita una palabra al servidor y actualiza directamente la partida; el segundo comprueba una palabra y devuelve un observable. En el primero, la

suscripción al observable se hace en el propio método; en el segundo, la suscripción al observable se hace fuera (Figura 233).

```
selectWord(match: WR, length: number) {
  this.client.get<any>("http://localhost/words/selectWord?length=" + length).
    subscribe(result => {
      match.disordered = result.disordered
      console.log(JSON.stringify(result))
    })
}

checkWord(disordered : string) : Observable<any> {
  return this.client.get<any>("http://localhost/words/checkWord/" + disordered)
}
```

Figura 233. Los dos métodos que tenemos en WRService

El sistema funciona y la implementación de ambas operaciones es correcta, pero despista un poco esta diferencia de implementación en una misma clase, cuyo código debería ser homogéneo y tener un mismo estilo; o sea, o bien:

1) Actualizamos directamente la partida en los dos métodos.
2) O bien devolvemos en ambos un observable con la respuesta.

Para la primera opción, dejaríamos el código como en la segunda solución. Para la segunda necesitamos hacer algún pequeño cambio.

En primer lugar, hacemos que *selectWord* devuelva directamente el observable:

```
selectWord(length: number) : Observable<any> {
  return this.client.get<any>("http://localhost/words/selectWord?length=" + length)
}
```

Figura 234. Unificamos el estilo de los dos métodos en el servicio

En segundo lugar, nos suscribimos a su resultado devuelto cuando lo llamamos en *ngOnInit:*

```
ngOnInit(): void {
  this.wrService.selectWord(5).
    subscribe(result => {
      this.match.disordered = result.disordered
      console.log(JSON.stringify(result))
    })
}
```

Figura 235. Nos suscribimos al observable devuelto por el servicio

El método *selectWord* que acabamos de cambiar también lo utilizamos en la implementación del *select* del componente (fila de abajo de la Figura 232). Bien, pues ahora o bien nos suscribimos también a su respuesta, o bien llamamos directamente a *ngOnInit:*

```
select(pos : number) {
  if (!this.firstPos) {
    this.firstPos = pos
  } else {
    this.match.swap(this.firstPos, pos)
    this.firstPos = undefined
    this.wrService.checkWord(this.match.disordered!).
      subscribe( {
            next : () => {
              this.ngOnInit()
            }
        }
      )
  }
}
```

Figura 236. En lugar de suscribirnos, llamamos a otro método

Ahora el sistema ya funciona perfectamente.

Capítulo 9. Introducción a las pruebas funcionales con Selenium

1. Introducción

Selenium es un *framework* que permite realizar pruebas funcionales de aplicaciones web mediante la reproducción de eventos en un navegador: desde un programa Java, podemos levantar un navegador, dirigirlo programáticamente a una URL y ejecutar en la ventana los eventos que haría un usuario. Habitualmente, se utiliza Selenium para escribir casos de prueba en formato JUnit que podemos enriquecer con oráculos, de modo que podemos encontrar errores en el sistema desde el punto de vista del usuario.

Se dispone, además, de *Selenium IDE,* un plugin para diferentes navegadores que permite grabar en un *script* las acciones que realiza un usuario en el navegador. Este *script* puede ser reproducido posteriormente y también ser exportado en algún lenguaje de programación.

2. Instalación

Para utilizar Selenium desde nuestra aplicación Java necesitamos dos elementos:

1) Una dependencia en el fichero *pom.xml*.

```
<dependency>
   <groupId>org.seleniumhq.selenium</groupId>
   <artifactId>selenium-java</artifactId>
   <version>4.0.0-alpha-6</version>
</dependency>
```

Figura 237. Dependencia de Selenium

2) Al menos un *driver* ejecutable adecuado al navegador con el que deseemos realizar las pruebas. Desde luego, podemos disponer de varios *drivers* y, de este modo, ejecutar las mismas pruebas con distintos navegadores.

El *driver* lo podemos encontrar buscando en Internet: por ejemplo, la cadena "chrome driver selenium" nos lleva a una página web de la que podemos descargar diferentes versiones del *driver*. Es importante que el *driver* que descarguemos sea adecuado a la versión del navegador que tengamos instalada. En la Figura 238, descargaríamos el *driver* correspondiente a la versión 108 de Chrome.

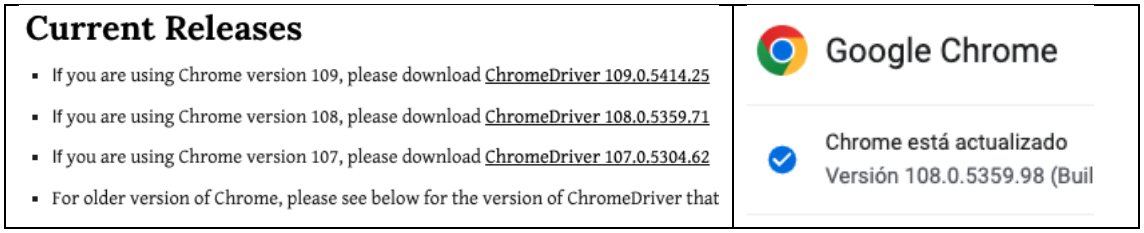


Figura 238. Descarga del chromedriver

Una vez descargado y descomprimido el archivo, dejamos el ejecutable en algún directorio.

Si queremos instalar Selenium IDE, debemos buscar la extensión e instalarla en nuestro navegador. Cuando lo esté, nos aparecerá el icono correspondiente:

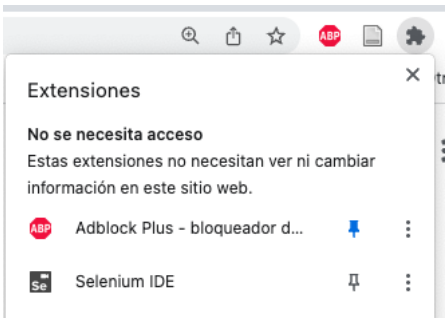


Figura 239. Selenium IDE, ya instalado en el navegador

3. Grabación de un sencillísimo *script*

Si pulsamos en el icono correspondiente a Selenium IDE, se nos abre una ventana en la que elegiremos *Record a new test in a new project*. Damos como nombre, por ejemplo, *TestFunctionalOtn*, pues vamos a grabar un *script* para el primer juego que hemos desarrollado, *Order the numbers*. A continuación, Selenium IDE nos solicita la URL a la que queremos conectarnos: nos aseguramos de que tenemos arrancado el sistema, escribimos *http://localhost:4200* y damos a *Start recording*.

Selenium IDE dirige el navegador a esa URL (Figura 240, izquierda). Además, si realizamos operaciones sobre el navegador, veremos que Selenium IDE va tomando nota de todos los eventos que produzcamos (lado derecho).

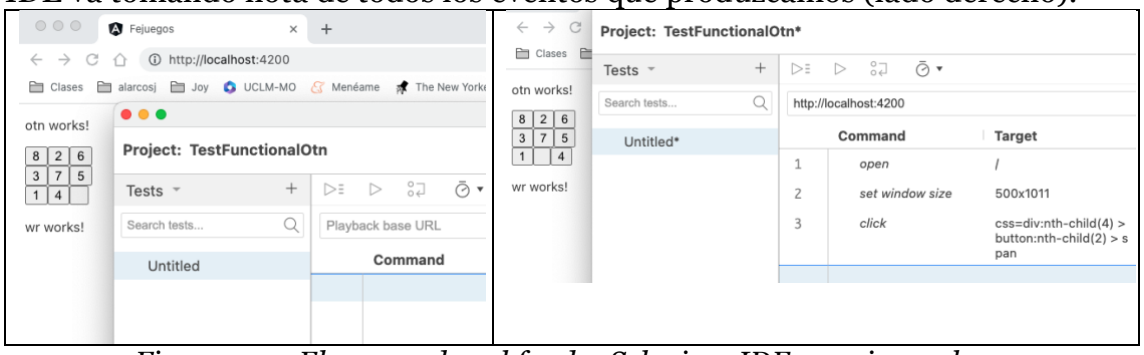


Figura 240. El navegador al fondo; Selenium IDE en primer plano

Nos interesa más la parte programática de los casos de prueba que este plugin, por lo que le damos al *stop* y damos un nombre al caso de prueba: *test1*, por ejemplo. A continuación, exportamos el caso de prueba en Java y lo guardamos en la carpeta del proyecto con el nombre *TestFunctionalOtn:*

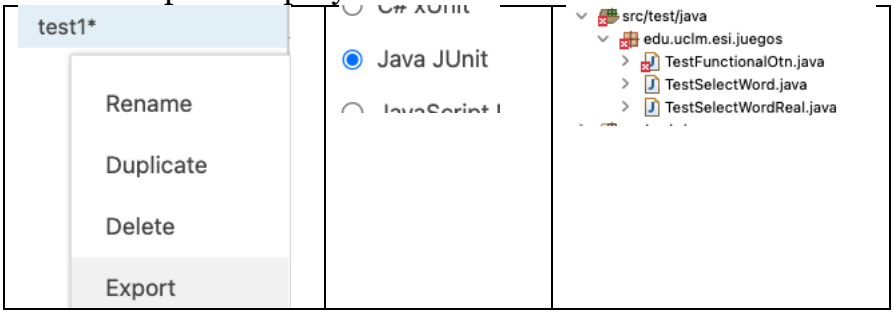


Figura 241. Guardamos el código Java obtenido del script *en nuestro proyecto*

4. Preparación del fichero Java de test

El fichero que se nos ha guardado tiene algún error fácil de corregir: falta el paquete (*edu.uclm.esi.juegos)* y probablemente esté mal el nombre de la clase. Corregidos ambos, vemos que hay muchos *import* que no se están utilizando: los quitamos para tener el código más limpio. La clase completa tendrá el aspecto de la Figura 242:

1) Se declara un campo *driver* de tipo *WebDriver,* que representa el navegador al que se enviarán instrucciones y sobre el que se realizarán comprobaciones.
2) Se declara otro campo *js* de tipo *JavascriptExecutor* y que, como vemos en el método *setUp,* apunta a la misma referencia que el campo *driver:* es decir, también el objeto *js* es el mismo navegador. Podemos usar ambas variables para hacer diferentes tipos de operaciones con el navegador: usaremos *driver,* por ejemplo, para hacer *click* en algún *widget* (como un botón)o buscar algún elemento que necesitemos (el texto de una etiqueta), mientras que usaremos *js* si necesitamos enviar a ejecución algún trozo de Javascript al navegador.
3) El método *test1* es un caso de prueba de JUnit: comienza llevando el navegador a la URL inicial, luego pone la pantalla a cierto tamaño (500x1011 puntos) y, finalmente, envía el evento *click* a cierto elemento de la página, que ha localizado mediante un *cssSelector*.

```
package edu.uclm.esi.juegos;

import java.util.HashMap;
import java.util.Map;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.Dimension;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class TestFunctionalOtn {
  private WebDriver driver;
  private Map<String, Object> vars;
  JavascriptExecutor js;

  @Before
  public void setUp() {
    driver = new ChromeDriver();
    js = (JavascriptExecutor) driver;
    vars = new HashMap<String, Object>();
  }

  @After
  public void tearDown() {
    driver.quit();
  }
```

```
  @Test
  public void test1() {
    driver.get("http://localhost:4200/");
    driver.manage().window().setSize(new Dimension(500, 1011));
    driver.findElement(
      By.cssSelector("div:nth-child(4) > button:nth-child(2) > span")
    ).click();
  }
}
```

Figura 242. Código inicial de TestFunctionalOtn

Si intentamos ejecutar el fichero con JUnit, vemos que no pasa nada. Esto ocurre porque, como vemos en los *import,* las anotaciones *@After, @Before* y *@Test* las está trayendo de *org.junit,* que es una versión de JUnit diferente de la que estamos utilizando (recuérdese que usamos JUnit Júpiter). Quitamos esos *import,* sustituimos las anotaciones *@Before* y *@After* por *@BeforeEach* y *@AfterEach,* e importamos del paquete bueno:

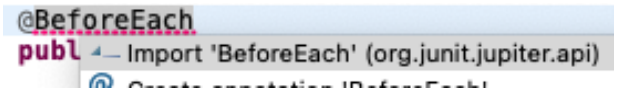


Figura 243. Importamos de org.junit.jupiter.api

Ahora, si ejecutamos , el caso *test1* falla y JUnit nos informa en su consola de un error: *The path to the driver executable must be set by the webdriver.chrome.driver system property*. Debemos indicar dónde se encuentra el ejecutable del *chromedriver* que nos descargamos hace un rato. Lo especificamos, por ejemplo, en la primera instrucción del *setUp* (Figura 244). Si el sistema operativo que usamos es Windows, tendremos que añadir la extensión (*.exe*) al nombre del archivo.

```
@BeforeEach
public void setUp() {
   System.setProperty("webdriver.chrome.driver",
            "/Users/macariopolousaola/chromedriver");
   driver = new ChromeDriver();
   js = (JavascriptExecutor) driver;
   vars = new HashMap<String, Object>();
}
```

Figura 244. Indicamos dónde se encuentra el chromedriver

Ahora sí, JUnit puede ejecutar el caso de prueba: lanza un navegador, ejecuta el *test1* y cierra el navegador, pues así está indicado (*driver.quit()*) en el método *tearDown*.

5. Escritura de un escenario sencillo para *Order the numbers*

En *Order the numbers*, la celda vacía siempre aparece en la última posición. Modificaremos el caso de prueba *test1* para llevarla a la primera.

Para ello, e independientemente de los números que compongan el tablero, seguiremos el proceso mostrado en la Figura 245, en donde hacemos clic en los números 5, 1, 6 y 7.

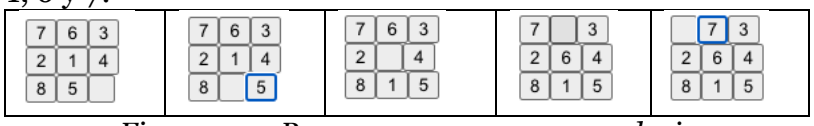


Figura 245. Pasos que queremos reproducir

Para hacer clic sobre el octavo botón (el 5), debemos primero localizarlo de alguna manera. El código que nos generó Selenium IDE busca por *cssSelector*. Nosotros vamos a buscar por *xpath* (*xpath* nos permite localizar elementos en un fichero XML, y HTML es un subconjunto de XML, estableciendo una ruta desde el nodo raíz hasta el elemento o elementos buscados).

La forma más sencilla de conocer la ruta *xpath* del octavo botón es hacer clic con el botón derecho sobre él y elegir la opción *Inspeccionar*. Esto es lo que hacemos en la Figura 246: nótese que, en realidad, el inspector nos ha llevado al *span* que tenemos dentro del botón; como quien ha de recibir el evento *click* es el botón, volvemos a pulsar con el botón secundario del ratón sobre el *button* contenedor y elegimos *Copy full XPath*.

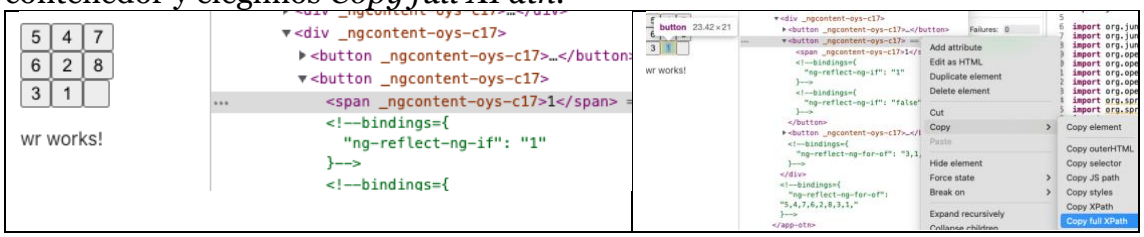


Figura 246. Inspeccionamos el documento para localizar el elemento

De vuelta en el código de *test1,* declararemos una variable *button21* que contenga una referencia al botón cuya ruta *xpath* acabamos de copiar. Lo llamamos *button21* porque, en la matriz de 3x3 botones, ocupa la posición (2,1):

```
@Test
public void test1() {
   driver.get("http://localhost:4200/");
   driver.manage().window().setSize(new Dimension(500, 1011));

   WebElement button21 =
      driver.findElement(By.xpath("/html/body/app-root/app-otn/div[3]/button[2]"));
}
```

Figura 247. Guardamos en button22 *una referencia al octavo botón*

La variable *button21* la hemos declarado de tipo *WebElement:* un *WebElement* es cualquier elemento que podemos encontrar en una página web. Su especificación es la de la Figura 248. Dispone de varias operaciones que nos permiten interactuar con el objeto programáticamente.

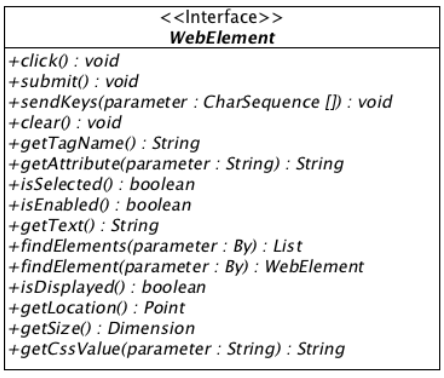


Figura 248. Operaciones de WebElement

Le vamos a enviar un *click:* pues nada, añadimos la instrucción *button22.click():* como vemos, *click()* es el primer método ofrecido por *WebElement*.

Para hacer clic en el botón central, que está en (1,1), procedemos de la misma forma: leemos su *xpath,* lo recuperamos y le enviamos el evento. Lo mismo con el que está en (0,1) y con el que está en (0,0). El código del método queda como:

```
@Test
public void test1() {
   driver.get("http://localhost:4200/");
   driver.manage().window().setSize(new Dimension(500, 1011));

   WebElement button21 =
      driver.findElement(By.xpath("/html/body/app-root/app-otn/div[3]/button[2]"));
   button21.click();

   WebElement button11 =
      driver.findElement(By.xpath("/html/body/app-root/app-otn/div[2]/button[2]"));
   button11.click();

   WebElement button01 =
      driver.findElement(By.xpath("/html/body/app-root/app-otn/div[1]/button[2]"));
   button01.click();

   WebElement button00 =
      driver.findElement(By.xpath("/html/body/app-root/app-otn/div[1]/button[1]"));
   button00.click();
}
```

Figura 249. Enviamos click *a los cuatro botones*

Visualmente comprobamos que la posición vacía se ha desplazado a (0, 0). Pero dos requisitos importantes de los casos de prueba es que sean automatizables (incluyendo sus asertos) y reproducibles: o sea, no queremos tener que estar mirando cuando dentro de un mes hagamos un cambio en el código y reejecutemos las pruebas: preferimos escribir casos de prueba robustos que se puedan lanzar de forma desatendida.

Por tanto, vamos a completar el código para lograr este objetivo. Antes de hacer clic en un botón, comprobaremos que tiene algo escrito; luego, hacemos clic y comprobamos que ya no tiene nada. Para el *button21,* el código puede ser el de la Figura 250:

1) Tras recuperar la instancia al botón, comprobamos que es cierto (*assertTrue)* que tiene algo de texto. Al importar *assertTrue,* téngase cuidado en traerlo del paquete *Assertions.assertTrue* y no de *Assert.assertTrue.*
2) Tras hacer clic, comprobamos en efecto que el texto ha desaparecido.

```
@Test
public void test1() {
   driver.get("http://localhost:4200/");
   driver.manage().window().setSize(new Dimension(500, 1011));

   WebElement button21 =
      driver.findElement(By.xpath("/html/body/app-root/app-otn/div[3]/button[2]"));
   assertTrue(button21.getText().length()>0);

   button21.click();
   assertTrue(button21.getText().trim().length()==0);
   ...
}
```

Figura 250. Metemos dos asertos

Sorprendentemente, JUnit nos da un error al ejecutar esta primera prueba tan sencilla: el primer aserto se supera pero peta el segundo. Si depuramos y colocamos un punto de interrupción en el segundo *assertTrue,* veremos que el

texto del *button21* ha desaparecido, como esperamos. ¿Qué está pasando entonces?

Si sustituimos el segundo *assertTrue* por *System.out.println(button21.getText())* para ver qué texto hay en *button21,* vemos que se imprime el texto que tenía antes. Es decir, parece que el botón se ha intercambiado de posición, no que se haya modificado su texto. Es decir, *button21* parece ser una referencia al botón que, inicialmente, estaba en (2,1), pero la referencia parece apuntar siempre al mismo elemento visual.

Con este diagnóstico, lo que debemos hacer es volver a recuperar el botón después de hacer clic:

```
@Test
public void test1() {
   driver.get("http://localhost:4200/");
   driver.manage().window().setSize(new Dimension(500, 1011));

   WebElement button21 =
      driver.findElement(By.xpath("/html/body/app-root/app-otn/div[3]/button[2]"));
   assertTrue(button21.getText().length()>0);

   button21 =
      driver.findElement(By.xpath("/html/body/app-root/app-otn/div[3]/button[2]"));
   button21.click();
   assertTrue(button21.getText().trim().length()==0);
   ...
}
```

Figura 251. Reasignamos la instancia para que se supere el caso

Debemos hacer el mismo tipo de cambio en todos los botones. Evidentemente, podemos resumir código y no declarar todas las variables *button*.

6. Pruebas de *Word race*

Para probar este juego necesitamos que estén en ejecución el docker, el servidor web en Eclipse en el puerto 80 y el servidor de Angular en el 4200.

Lo que haremos será guardar las letras que hayamos recibido, y hacer clic "a lo loco" hasta que el servidor nos entregue una palabra nueva.

Añadimos el caso de prueba al mismo fichero *TestFuncionalOtn:* el sufijo *Otn* deja de ser significativo, pues vamos a probar desde aquí los dos juegos, así que renombramos el fichero a *TestFunctionalGames,* y también el *test1* a *testOtn*. El nuevo caso de prueba se llamará *testWR*.

Al estar recuperando palabras de 5 letras, disponemos de cinco botones cuyas rutas *xpath* están todas en */html/body/app-root/app-wr/div/*. Cada botón varía desde *button[1]* hasta *button[5]*.

El código de este método puede ser el siguiente:

1) Primero, guardamos en *startingLetters* las letras que componen la palabra recibida.
2) En el bucle, generamos dos números al azar entre 1 y 5, que utilizamos para recuperar los dos botones sobre los que, a continuación, haremos clic.
3) Guardamos en *letters* las letras que hay actualmente sobre los botones.
4) Dejamos de pulsar aleatoriamente cuando las letras que hay en *letters* ya no son las mismas que había al principio.

```
@Test
public void testWR() {
   String xpath = "/html/body/app-root/app-wr/div/";
   WebElement buttonLeft, buttonRight;
   int left, right;

   SecureRandom random = new SecureRandom();
   String startingLetters = this.getLetters(xpath);
   String letters = "";
   do {
      left = random.nextInt(5) + 1;
      right = random.nextInt(5) + 1;
      buttonLeft = driver.findElement(By.xpath(xpath + "button[" + left + "]"));
      buttonRight = driver.findElement(By.xpath(xpath + "button[" + right + "]"));
      buttonLeft.click();
      buttonRight.click();
      letters = this.getLetters(xpath);
   } while (this.areTheSame(letters, startingLetters));
}
private boolean areTheSame(String a, String b) {
   for (int i=0; i<a.length(); i++)
      if (b.indexOf(a.charAt(i))==-1)
         return false;
   return true;
}

private String getLetters(String xpath) {
   String result = "";

   for (int i=1; i<=5; i++)
      result = result + driver.findElement(By.xpath(xpath +
         "button[" + i + "]")).getText();
   return result;
}
```

Figura 252. Caso de prueba para el Word race y dos métodos auxiliares

6.1. Ejecución

Si tenemos bastante suerte, los dos casos de prueba se superarán, pero lo normal será que JUnit emita veredicto de fallo para *testWR,* a pesar de que está bien implementado.

Puede suceder que JUnit esté ejecutando *testWR* antes que *testOtn*: el orden de declaración de los métodos no indica el orden de ejecución. Tal vez que se ejecuten en ese orden, pero tal vez no. Si tenemos la mala suerte de que se ejecute antes *testWR,* el navegador no se habrá lanzado (pues se lanza en *testOtn)*; así que, en primer lugar, añadimos la anotación *@Order(1)* a *testOtn* y *@Order(2)* a *testWR*. Además, anotamos la clase con *@TestMethodOrder(OrderAnnotation.class)*.

Con este cambio, y en un mundo ideal, el flujo de ejecución sería el de la Figura 253. JUnit y Selenium desempeñan ahora el papel de usuario.

1) Se va a ejecutar *testOtn* (que hemos anotado con orden 1).
1) Antes se lanza *setUp*.
2) A continuación se ejecuta *testOtn,* que abre una navegador y solicita al servidor que le entregue la página principal (*index.html,* en donde incrustamos *app-otn* y *app-wr).*
3) Al cargar *app-otn* se invoca al constructor de *OtnComponent,* que crea directamente la instancia de *Otn* con la que jugar.
4) Al cargar *app-wr,* su componente asociado envía una petición al servidor de Eclipse (*localhost:80)* para que nos entregue una palabra de

5 letras. En este escenario ideal, la palabra (por ejemplo, SCATO) viene inmediatamente.

5) Se sigue ejecutando *testOtn*: clic en varios botones. Cuando acaba, JUnit invoca automáticamente a *tearDown*.
6) Al haber más casos, se invoca de nuevo a *setUp*.
7) Se ejecuta *testWR,* que empieza a hacer clics aleatoriamente. En el escenario descrito, compone primero la cadena CSATO, que el servidor no encuentra y responde inmediatamente con un 404; luego, compone CASTO y el servidor nos dice que vale, que 200. El caso de prueba termina y se ejecuta de nuevo el *tearDown*.

Ocurre que el primer caso de prueba (*testOtn)* es muy rápido, pues no tiene interacción con ningún servidor (todo el código reside en el navegador); pero el segundo caso es más lento, porque requiere en envío de peticiones al servidor que escucha en el 80 y obtener sus respuestas.

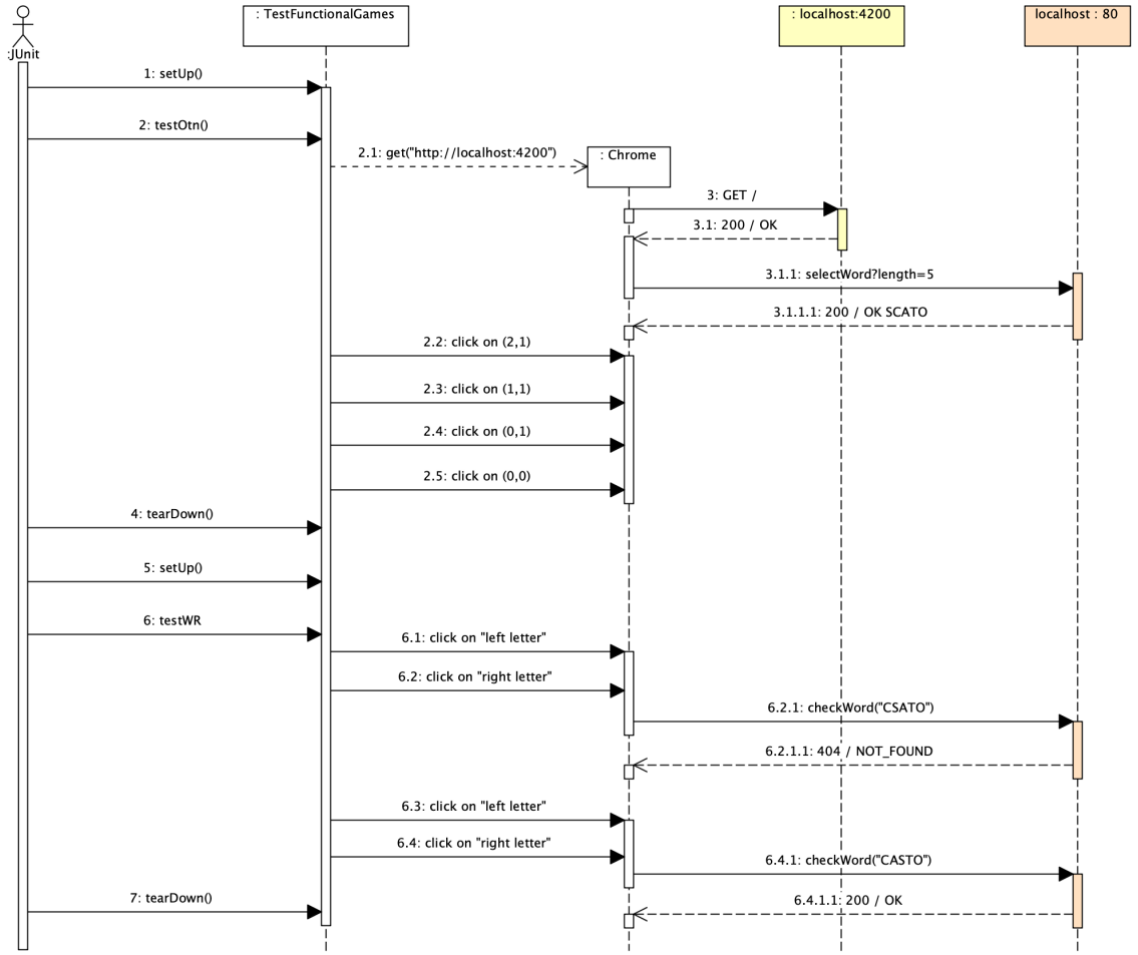


Figura 253. Flujo de ejecución ideal

De hecho, en la implementación que tenemos, incluso la primera respuesta a la petición *selectWord* que enviamos en el mensaje 3.1.1 tarda mucho, y por eso advertimos, cuando programamos el método de la Figura 131 (página 101), que la implementación que le dimos era provisional: se trae de la base de datos todas las palabras (tenemos 646.615), para ir a continuación eliminando las que no tienen el número de letras deseado y seleccionar, de entre todas estas, una cualquiera.

En la práctica, esta mala implementación del método hace que, cuando *testOtn* ha terminado de ejecutarse, aún no haya llegado la respuesta a

selectWord: esto se ilustra en el diagrama de secuencia de la Figura 254, en donde se muestra cómo la cadena SCATO llega al navegador en el mensaje 3.1.1.1, cuando todavía no hay letras sobre las que "cliquear" en el mensaje 6.1. Por eso, JUnit emite el siguiente veredicto de fallo:

no such element: Unable to locate element: {"method":"xpath","selector":"/html/body/app-root/app-wr/div/button[1]"}

En efecto, al no haberse recibido la palabra, Angular no ha podido crear los botones para las letras en el bucle **ngFor,* por lo que en esa ruta *xpath* no hay ningún botón sobre el que Selenium pueda hacer clic.

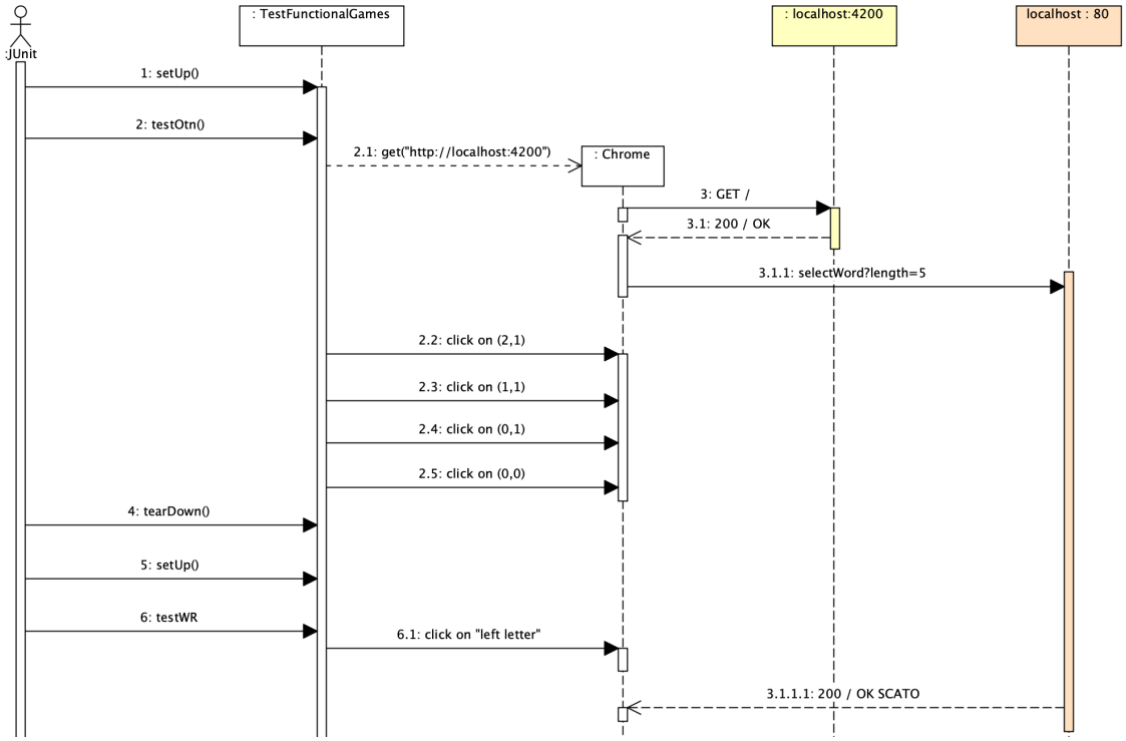


Figura 254. Uno de los flujos de ejecución que pueden ocurrir

6.2. Control del tiempo en Selenium

Acabamos de confirmar que el método que escribimos en la Figura 131 es realmente una chapuza de un nivel bastante considerable. Para confirmarlo de otra manera, en el siguiente capítulo someteremos al sistema a pruebas de rendimiento, por lo que de momento lo vamos a dejar como está. Ya lo mejoraremos más adelante.

Hemos visto que el error salta porque se intenta hacer clic en un botón que todavía no está en el árbol DOM la página web.

Con el trozo de código que se resalta, esperamos un máximo de 15 segundos para ver si aparece el botón que corresponde la primera letra:

```
@Test @Order(2)
public void testWR() {
  String xpath = "/html/body/app-root/app-wr/div/";
  WebElement buttonLeft, buttonRight;
  int left, right;

  new WebDriverWait(driver, Duration.ofSeconds(15)).
    until(ExpectedConditions.elementToBeClickable(By.xpath(xpath + "button[1]")));

   ...
}
```

Figura 255. Adición de un tiempo de espera

Capítulo 10. Pruebas de rendimiento con JMeter (I)

En este primer capítulo dedicado a pruebas de rendimiento someteremos al servidor del puerto 80 a este tipo de pruebas. Encontraremos que responde con tiempos de respuesta muy largos cuando el número de usuarios crece y, además, veremos que empieza a fallar en el servicio de peticiones si crece aún más. Aunque sabemos la causa del error (la mala programación del método *selectWord* del *WordRaceService,* en la Figura 131, página 101), esperaremos al siguiente capítulo para diagnosticarlo con un *profiler*. Luego, corregiremos ese trozo de código y repetiremos las pruebas.

1. Introducción

Las pruebas de rendimiento se utilizan principalmente para probar la velocidad y tiempos de respuesta de una aplicación, para lo que se le envían diferentes cargas de trabajo.

En nuestro caso, utilizaremos la herramienta JMeter para someter al servidor que tenemos en el puerto 80 a pruebas de este tipo. Con JMeter tiene sentido probar el *backend* de una aplicación, pero no el lado cliente: en nuestro sistema, por ejemplo, cada usuario maneja su propio navegador por lo que, si este se ralentiza, será bien por una carga excesiva del computador en donde corre el propio cliente, bien porque el servidor al que el cliente se está conectando (en nuestro caso, el que corre en *localhost:80)* está atendiendo, por ejemplo, un número muy alto de peticiones.

JMeter puede descargarse fácilmente de su web. Se baja como un fichero comprimido que no necesita instalación. Para arrancarlo, hacemos doble clic sobre *ApacheJMeter.jar*.

2. Diseño de una prueba sencilla

Simularemos un escenario en el que 5 usuarios comienzan a jugar al único juego que, de momento, necesita comunicación con el *backend:* el *Word race*. Cuando tengamos este escenario operativo y listo, incrementaremos fácilmente el número de usuarios que jugarán simultáneamente.

Este juego comienza, ya lo sabemos, por la selección de una palabra aleatoria, que el cliente solicita llamando al servicio *selectWord;* luego, el usuario va intercambiando letras, sucediendo que cada intercambio provoca una llamada al servicio *getWord,* que ya sabemos que devuelve o 200 o 404.

2.1. Preparación

Cuando arrancamos JMeter se nos crea un elemento denominado *Plan de pruebas,* al que podemos ir añadiendo elementos. En principio, añadiremos un grupo de hilos (*Thread group),* como en la Figura 256.

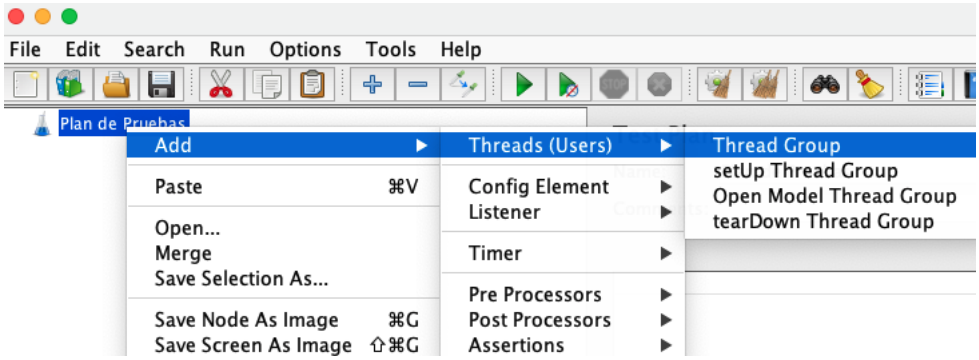


Figura 256. JMeter

Le damos como nombre *Users,* ponemos 5 en el número de hilos y dejamos el resto como está:

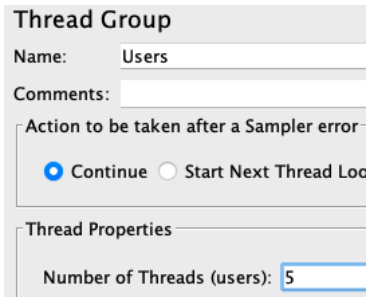


Figura 257. Configuración del grupo de hilos

Al grupo de hilos, que ya se habrá añadido al nodo del *Plan de pruebas*, iremos añadiendo nodos que representarán las acciones que realizará cada uno de los usuarios contenidos en el grupo. Lo primero que hará cada hilo (o usuario) será enviar una petición *get* al recurso *localhost:80/words/selectWord*, pasando 5 como valor del parámetro *length*. Para hacer que del nodo *Users* cuelgue esta petición, elegimos la opción de menú de la Figura 258.

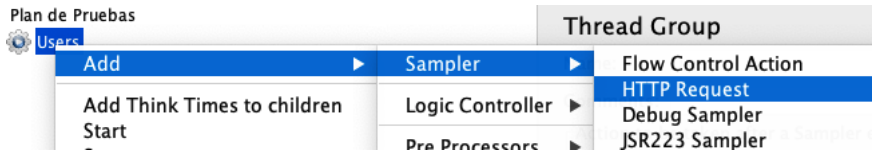


Figura 258. Añadimos una HTTP Request *al nodo* Users

Completamos los datos de la siguiente manera:

1) Como nombre le damos un valor significativo, que nos permita localizar el elemento fácilmente en el plan de pruebas: *selectWord,* por ejemplo.
2) En los campos para el protocolo, la IP y el puerto, los valores *http, localhost* y *80*.
3) Como tipo de método elegimos GET y, como ruta, la del *endpoint* al que dirigimos la petición, añadiendo el valor del parámetro: */words/selectWord?length=5*

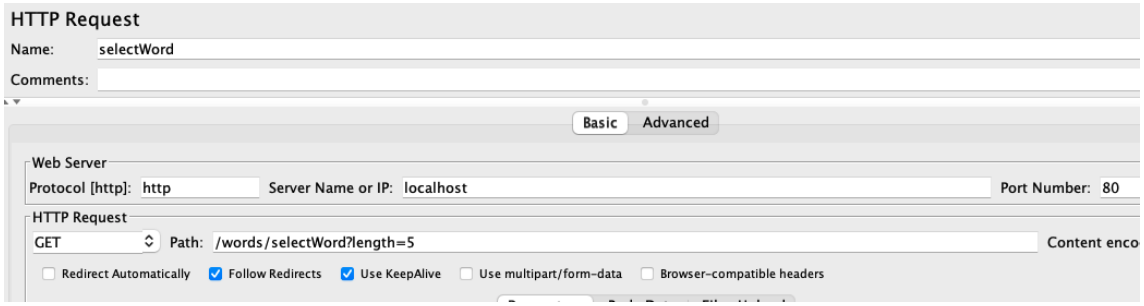


Figura 259. Campos de la petición

Ahora, añadiremos un *Árbol de resultados* al *Plan de pruebas* que, si pinchamos en el nodo *Users,* encontramos en *Add, Listener*. El plan tendrá el siguiente aspecto:

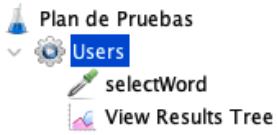


Figura 260. Aspecto del plan de pruebas

2.2. Ejecución con 5 usuarios

Nos aseguramos de que el docker con SQL Server esté escuchando, así como de que el servidor que tenemos Eclipse esté en ejecución. Si pulsamos el botón de ejecutar, JMeter lanza los cinco hilos. Durante la ejecución, el botón *Stop* se activa, y vuelve a desactivarse cuando se han recibido todas las respuestas.

Si nos hemos fijado, habremos notado que, para la poca a la que hemos sometido al servidor, el botón *Stop* ha permanecido habilitado un tiempo significativo.

En el árbol de resultados vemos las 5 peticiones que se han enviado. Todas aparecen en verde, lo que denota que han recibido código 200.

En la solapa *Sampler result* (Figura 261) se nos informa de algunos datos de la petición:

1) El *Load time* (tiempo de carga) es el tiempo transcurrido desde que se envía la petición hasta que se recibe la respuesta completa.
2) La *Latency* (latencia) es el tiempo que pasa desde que se envía la petición hasta que se empieza a recibir la respuesta.
3) El *Connect time* (tiempo de conexión) es el tiempo que se tarda en establecer la conexión incluyendo, si fuera el caso, el *handshake* del protocolo SSL.

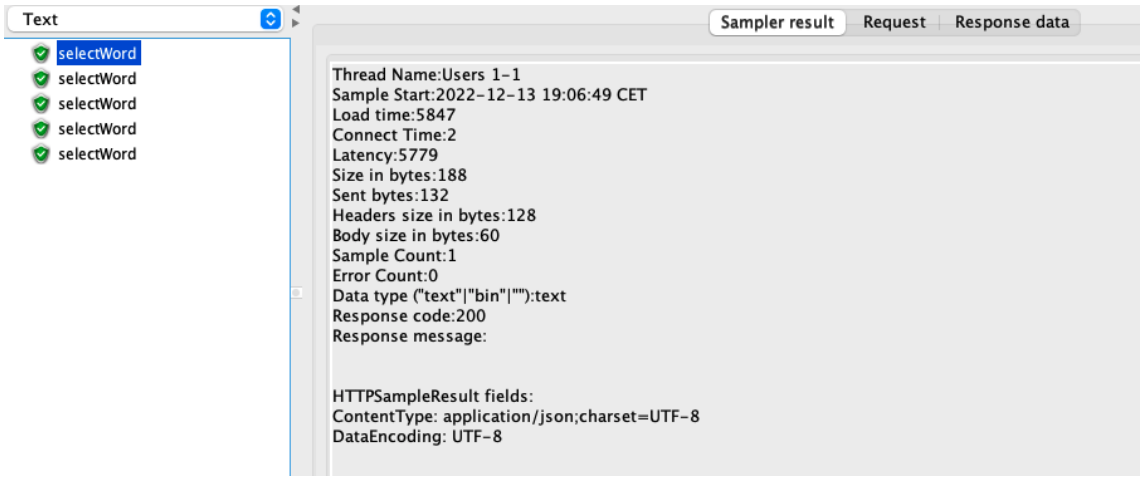


Figura 261. Las cinco peticiones se han servido correctamente

En la solapa *Request* podemos ver la petición que se ha enviado: en los cinco casos es exactamente la misma. En *Response data* aparece la respuesta devuelta por el servidor que, como sabemos, es un JSON con los campos *id, word* y *disordered*.

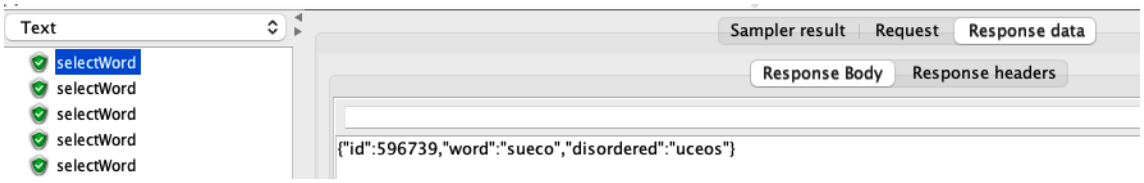


Figura 262. Datos de la respuesta

Podemos añadir al plan un informe resumen (*Summary report)*, que nos da información agrupada. Antes de ejecutar de nuevo, pulsaremos el botón que tiene algo así como dos escobas y una rueda dentada, , para limpiar los resultados anteriores.

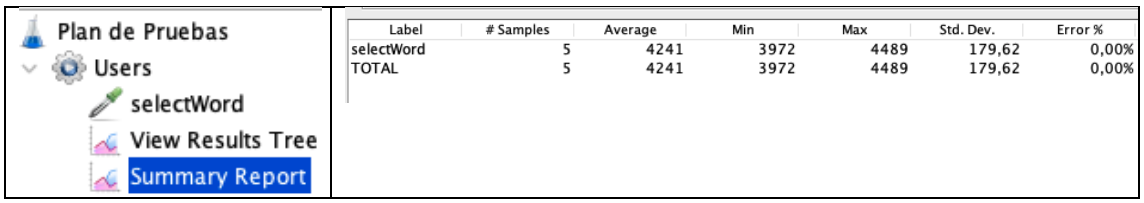


Figura 263. Añadimos un Summary report

2.3. Ejecución con 50 usuarios

La verdad es que los tiempos de respuesta son bastante altos. Aunque sabemos a qué se debe, de momento haremos como que no tenemos ni idea, y trataremos de determinar la causa en el capítulo siguiente.

Veamos cómo se comporta el servidor si subimos a 50 el número de hilos de *Users*.

En la Figura 264 vemos que los tiempos medios de servicio de las peticiones han subido de forma muy significativa, desde los 4,2 segundos del *Summary report* anterior (con 5 usuarios) hasta los 25,48 de este. Además, el número de peticiones que no se sirven ha subido al 38% y, si miramos la consola del servidor en Eclipse, vemos que hay excepciones: en la Figura 265 resaltamos el mensaje de error: la conexión no está disponible, y se dio por perdida después de 30,1 segundos.

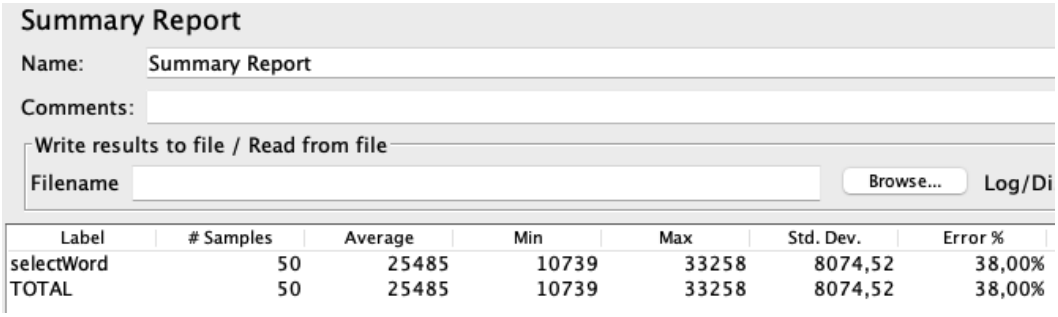


Figura 264. Resultados con 50 usuarios

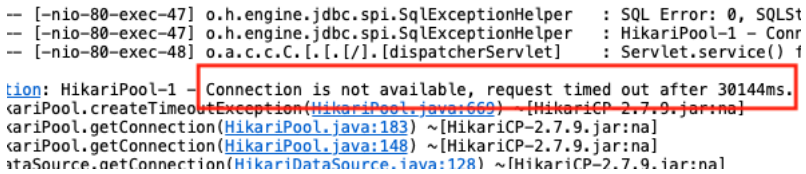


Figura 265. Consola de Eclipse

Por otro lado, en el *Results tree* vemos el resultado individual de cada petición. Las peticiones en rojo de la Figura 266 han obtenido una respuesta distinta de 200: en particular, se ve a la derecha que la que está seleccionada ha obtenido un 500, que es un error genérico del servidor.

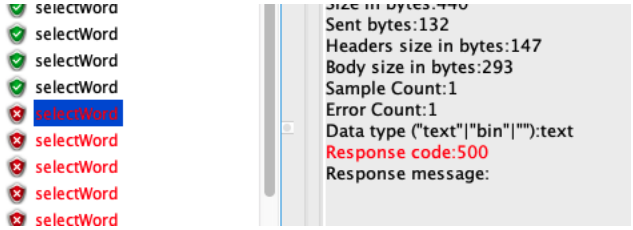


Figura 266. La petición indicada ha obtenido un código 500

Por otro lado, en el *Results tree* observamos una distribución curiosa de las peticiones en verde y en rojo: las primeras peticiones que se envían se sirven lentas, pero bien; luego, hay una serie de larga de peticiones con error y, finalmente, las cosas vuelven a su cauce y el servidor responde correctamente a las últimas.

Figura 267. Distribución de éxitos y errores

Pools de conexiones a bases de datos

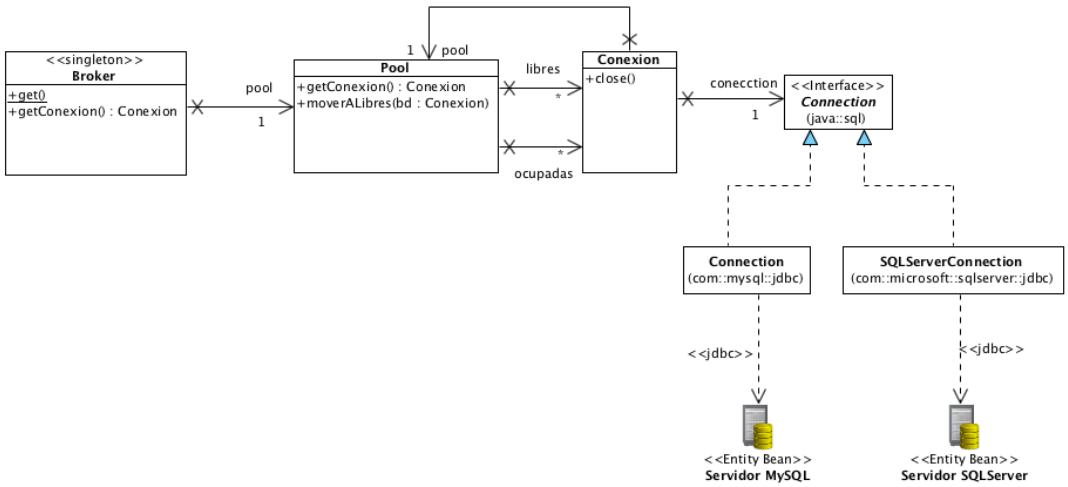
A partir de la figura anterior, podemos sospechar que las peticiones comienzan a fallar cuando se agotan todas las conexiones del *pool*.

Un *pool* de conexiones es un conjunto de conexiones que el servidor de aplicaciones (*localhost:80* en nuestro caso) mantiene permanentemente abiertas con el servidor de base de datos (que tenemos en el docker, escuchando en *localhost:1433*).

El servidor de aplicaciones guarda estas conexiones en dos grupos: las que están en uso y las que están libres. Cuando un cliente desea ejecutar una operación sobre la base de datos, le pide a alguien (un gestor de conexiones) una la conexión, que le devuelve una conexión del pool de libres y la pasa al de ocupadas; cuando el cliente termina de usarla, llama a su operación *close,* que realmente no cierra la conexión, sino que la devuelve al pool de libres.

En la siguiente figura se plantea un gestor de conexiones al que llamamos *Broker*. Este objeto mantiene el *pool* con una colección de objetos de tipo *Conexion* (en castellano) ubicadas en una colección de *libres* y en otra de *ocupadas*. Cada *Conexion* conoce a una *Connection* real del paquete *java.sql*.

Cuando un objeto necesita una conexión, se la pide al *Broker,* que le devuelve una instancia de *Conexion* de las que haya en la lista de *libres* y la pasa a la de *ocupadas*. Cuando la *Conexion* ya no se necesita, el objeto que la mantenía ocupada ejecuta sobre ella el método *close(),* que lo que hace es decirle al *Pool:* «Oye, pool, pásame a mí (que soy una *Conexion*) a la lista de *libres*».



Spring dispone de un modelo de conexiones similar al que se acaba de describir: cuando Spring arranca, crea un *pool* de conexiones que conecta al servidor de bases de datos, y que mantiene abiertas permanentemente. Cuando un repositorio necesita acceder a la base de datos, el gestor de conexiones de Spring le entrega una de las que tiene libres. Si no hay ninguna libre, se produce un error y, si este se propaga hasta el cliente como respuesta a un servicio *http,* se emitirá un código 500.

Cuadro IX. Pools de conexiones a bases de datos

La distribución de errores de la Figura 267 invita a pensar que hay un problema con el *pool:* las primeras peticiones van tomando conexiones libres; como cada petición tarda mucho tiempo en servirse por la mala programación del método *selectWord* del *WordRaceService* (Figura 131, página 101), el *pool* de conexiones libres se va agotando y, cuando se pide una nueva, se produce el error 500 que se explica en el Cuadro IX. Las nuevas peticiones empiezan a fallar, pero

en algún momento las peticiones que enviaron al principio se van sirviendo, con lo que las conexiones que mantenían ocupadas pasan a estar libres.

Capítulo 11. Uso de un *profiler*

1. Introducción

Un *profiler* nos va a permitir ejecutar una aplicación y, entre otras muchas cosas, tener conocimiento de los tiempos de ejecución de cada método o del orden de llamadas.

Ya que el código de nuestro servidor adolece de algún error importante que hace que los tiempos de respuesta sean muy altos, y que provoca un porcentaje de errores también muy alto cuando el número de usuarios crece, utilizaremos la herramienta *YourKit Java Profiler* para encontrar la causa del error.

Podemos descargar esta herramienta de su sitio web y solicitar una licencia de evaluación de 15 días.

Cuando la arrancamos, en la pantalla principal se nos pregunta qué queremos perfilar. Elegimos *Profile from within IDE* y, en el menú que nos muestra, pinchamos en Eclipse.

El asistente nos pedirá que instalemos un plugin en nuestro entorno de desarrollo, y nos conducirá a una página web con las instrucciones de instalación. De acuerdo con estas y a la versión del *profiler* que hayamos descargado, lo instalamos.

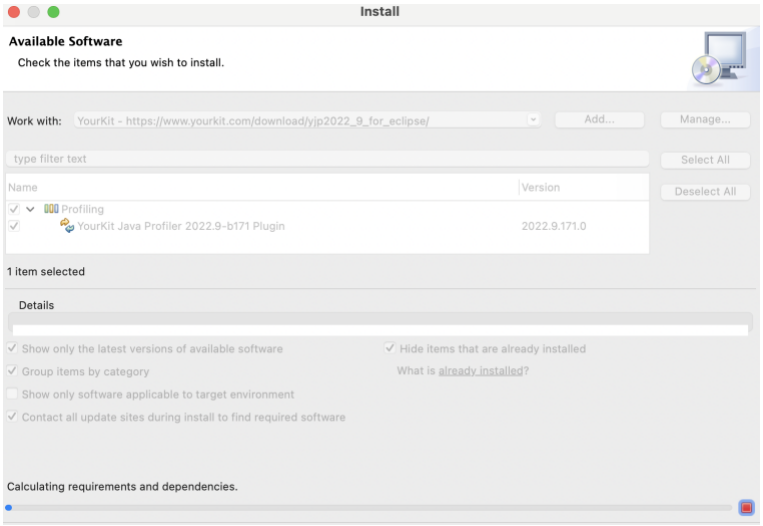


Figura 268. Instalación del plugin para YourKit

2. *Profiling* del servidor

Una vez instalado el plugin, hacemos clic sobre la clase *App* y elegimos *Profile As Java Application* (Figura 269).

Figura 269. Arrancamos el profiling

En este momento, la ventana de la aplicación *YourKit* habrá cambiado, pues empieza a recibir mensajes desde el plugin instalado en Eclipse.

2.1. *Profiling* con un uso manual

Para familiarizarnos con la herramienta, vamos a ver qué pasa si jugamos al *Word race* desde el navegador: lanzamos el proyecto con *ng serve* y nos dirigimos vamos al puerto 4200 de *localhost*.

En cuanto empezamos a jugar, vemos que la gráfica de uso de la CPU muestra algún pico relevante, que vuelve a ser prácticamente plana cuando dejamos.

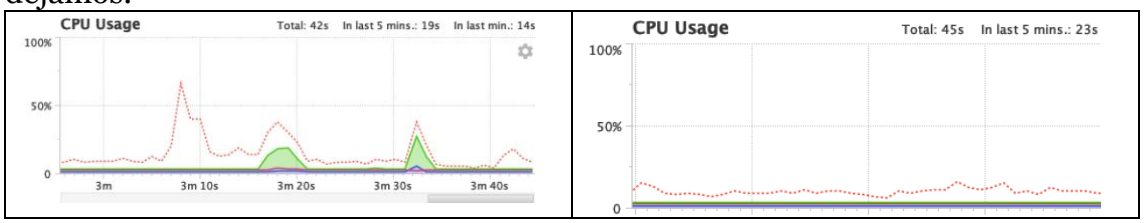


Figura 270. Gráfica de la CPU interactuando con el navegador (izquierda) y sin hacer nada (derecha)

En la solapa *Call tree – All threads merged* de *YourKit* vemos ya información relevante.

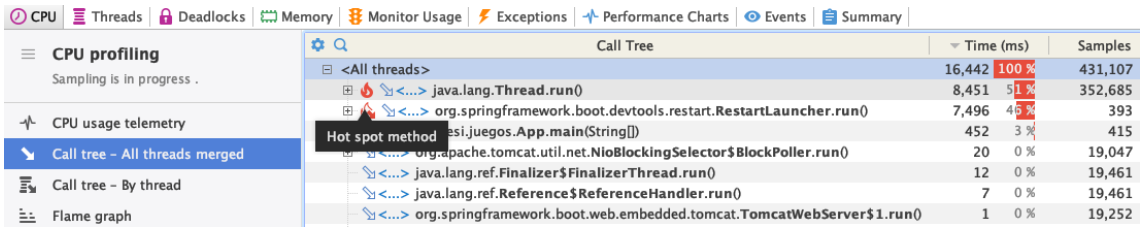


Figura 271. Dos métodos marcados con fuego

Si expandimos el arbolito del primer *hot spot*, vemos que se ha producido una llamada a *selectWord* en el *WordRaceController* que, a su vez, ha llamado al *selectWord* del *WordRaceService* (Figura 272):

1) El tiempo total de ejecución del servicio *selectWord* ha sido de 4.837 milisegundos.
2) De estos, 4.140 han sido empleados por *findAll*.
3) Las líneas 22, 26 y 24 de *WordRaceService* han empleado, respectivamente, 351, 320 y 24 milisegundos en ejecutarse.

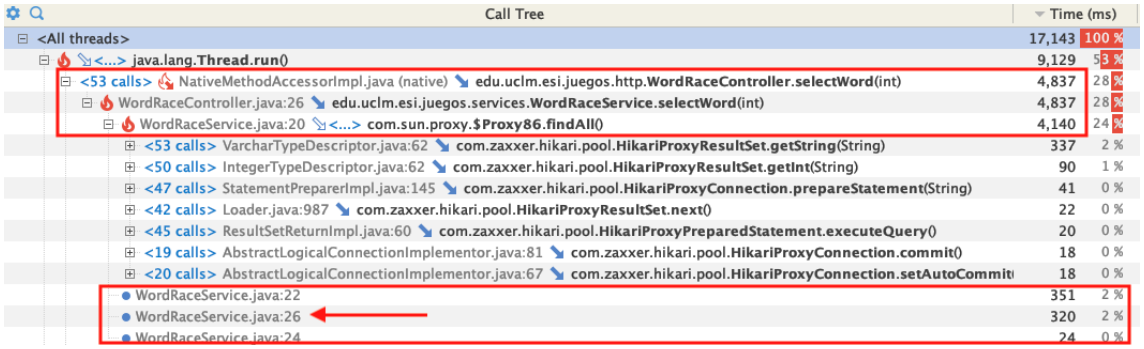


Figura 272. Secuencia de operaciones ejecutada cuando el controlador ha recibido la petición en selectWord

Sabíamos desde hace mucho que el problema se encuentra en este punto (Figura 273): ya habíamos advertido que esa llamada a *findAll*, que se trae todas las palabras del diccionario, no podía ser eficiente; y que tampoco lo era la eliminación de las palabras de una longitud distinta de la deseada.

```
19  public Word selectWord(int length) {
20      List<Word> words = this.wordDAO.findAll();
21
22      for (int i=words.size()-1; i>=0; i--)
23          if (words.get(i).getWord().length()!=length)
24              words.remove(i);
25
26      int pos = new SecureRandom().nextInt(words.size());
27      Word word = words.get(pos);
28      this.disorder(word);
29
30      return word;
31  }
```

Figura 273. Código del método objeto de estudio

2.2. *Profiling* con JMeter

Aunque ya teníamos claro dónde está el problema y acabamos de confirmarlo con un sencillo uso del navegador, vamos a ver qué pasa si ejecutamos los 50 hilos que tenemos en JMeter.

Los resultados son parecidos: en la Figura 274, vemos que *selectWord* ha consumido 256 segundos, de los que 250 los ha empleado *findAll*. Los métodos que cuelgan de *findAll* (*invoke, proceed,* etcétera) son métodos de librería y están fuera de nuestro control. Obviamente, algo hay que hacer para que *selectWord* tarde menos en ejecutarse.

Call Tree	Time (ms)	
<All threads>	1,466,186	100 %
<...> java.lang.Thread.run()	1,457,977	99 %
<53 calls> NativeMethodAccessorImpl.java (native) edu.uclm.esi.juegos.http.WordRaceController.selectWord(int)	256,396	17 %
WordRaceController.java:26 edu.uclm.esi.juegos.services.WordRaceService.selectWord(int)	256,396	17 %
WordRaceService.java:20 com.sun.proxy.$Proxy86.findAll()	250,422	17 %
org.springframework.aop.framework.JdkDynamicAopProxy.invoke(Object, Method, Object[])	250,422	17 %
JdkDynamicAopProxy.java:212 org.springframework.aop.framework.ReflectiveMethodInvocation.proceed()	250,422	17 %
ReflectiveMethodInvocation.java:185 org.springframework.data.repository.core.support.SurroundingTransa	250,422	17 %
SurroundingTransactionDetectorMethodInterceptor.java:61 org.springframework.aop.framework.Reflecti	250,422	17 %
ReflectiveMethodInvocation.java:185 <...> org.springframework.aop.interceptor.ExposeInvocationInt	250,422	17 %
WordRaceService.java:26 java.security.SecureRandom.<init>()	2,652	0 %
WordRaceService.java:22	1,855	0 %
WordRaceService.java:26	645	0 %
WordRaceService.java:26 <...> java.util.Random.nextInt(int)	583	0 %
WordRaceService.java:24	159	0 %
WordRaceService.java:23	77	0 %

Figura 274. Resultados con la ejecución de JMeter

3. Refactorización del código

Una refactorización es un cambio en el código que no altera la funcionalidad, pero sí que le mejora alguna propiedad: legibilidad, modularización o, como vamos a hacer ahora, rendimiento, pues vamos a seguir devolviendo una palabra aleatoria pero en mucho menos tiempo. Modificaremos el método para que quede como en el lado derecho de la Figura 275 y que por supuesto, pues es el objetivo que perseguimos, lo haga en un santiamén.

```
public Word SelectWord(int length) {
   List<Word> words = this.wordDAO.findAll();

   for (int i=words.size()-1; i>=0; i--)
      if (words.get(i).getWord().length()!=length)
         words.remove(i);

   int pos =
      new SecureRandom().nextInt(words.size());

   Word word = words.get(pos);
   this.disorder(word);

   return word;
}
```

```
public Word selectWord(int length) {
   Word word =

   this.wordDAO.findByRandomLength(length);

   this.disorder(word);
   return word;
}
```

Figura 275. Refactorización que queremos hacer

Lamentablemente, las convenciones de nombrado que comentamos en el Cuadro VIII (página 114) no incluyen entre sus términos la palabra *Random*, por lo que debemos dar alguna implementación a *findByRandomLength*. De hecho, esas mismas convenciones de nombrado nos impiden dar ese nombre a ese método, pues Spring generaría una consulta como *select * from word where random_length=5*, con lo que es posible que la aplicación ni siquiera arrancase. Llamaremos al método *getRandomWord*. Al decirle a Eclipse que nos lo cree, el código del repositorio *WordDAO* pasa a ser el que sigue:

```
package edu.uclm.esi.juegos.dao;

import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import edu.uclm.esi.juegos.entities.Word;

@Repository
public interface WordDAO extends JpaRepository <Word, Integer> {
   Optional<Word> findByWord(String word);

   Word getRandomWord(int length);
}
```

Figura 276. Nuevo método en el repositorio

El nuevo método debe lanzar a la base de datos una consulta que nos devuelva una palabra aleatoria de la longitud pasada como parámetro. ¿Cómo hacerlo? Pues… buscando en Internet.

Como estamos usando SQL Server, escribimos "sql server select random row from table" y vemos que el Sr. D. Yaakov Ellis tiene la respuesta no solo para SQL Server, sino para varios más. En el caso que nos ocupa, nos dice:

MySQL	PostgreSQL	SQL Server	IBM DB2	Oracle
`SELECT column FROM table ORDER BY RAND() LIMIT 1`	`SELECT column FROM table ORDER BY RANDOM() LIMIT 1`	`SELECT TOP 1 column FROM table ORDER BY NEWID()`	`SELECT column, RAND() as IDX FROM table ORDER BY IDX FETCH FIRST 1 ROWS ONLY`	`SELECT column FROM (SELECT column FROM table ORDER BY dbms_random.value WHERE rownum = 1`

Figura 277. Recuperación de una fila aleatoria en diversos SGBD[8]

¿Será verdad? Abramos el *Azure Data Studio* (que instalamos en la sección Instalación de un cliente para SQL Server, en la página 119), adaptemos la consulta de SQL Server a nuestra base de datos y ejecutémosla:

[8] https://stackoverflow.com/questions/19412/how-to-request-a-random-row-in-sql

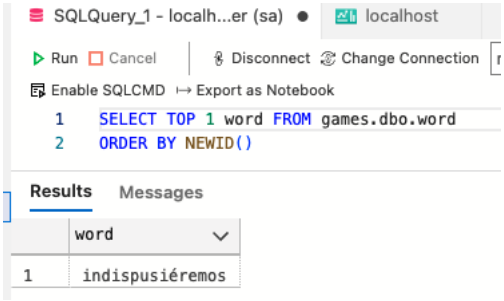


Figura 278. Recuperación de una palabra aleatoria

Pues parece que sí funciona. Lo único que tenemos que hacer ahora es acotar la palabra por longitud:

Figura 279. Recuperación de una palabra aleatoria de cinco letras

Pues, de alguna manera, debemos asociar una consulta como la de Figura 279 al método *getRandomWord* de nuestro repositorio.

Para esto, añadimos la anotación *@Query* a *getRandomWord* y:

1) Asignamos la cadena con la consulta SQL a su campo *value: SELECT TOP 1 word FROM games.dbo.word where len(word)= :length ORDER BY NEWID()*
   Obsérvese que no restringimos la búsqueda a palabras de 5 letras, sino que parametrizamos el valor de la longitud.
2) Asignamos *true* a su campo *nativeQuery*. Como vimos en la respuesta del Sr. Ellis (Figura 277), cada fabricante de bases de datos implementa de manera distinta la búsqueda de una fila aleatoria. Puesto que estamos con el gestor concreto SQL Server, indicamos con *nativeQuery=true* que la consulta está escrita específicamente para el SGBD con el que estamos trabajando.
3) Por último, obsérvese (Figura 280) que anotamos con *@Param("length")* el parámetro que se recibe, que es el mismo que pasamos a la consulta SQL.

```
@Repository
public interface WordDAO extends JpaRepository <Word, Integer> {

   Optional<Word> findByWord(String word);

   @Query(
     value = "SELECT TOP 1 word FROM games.dbo.word where len(word)= :length ORDER BY NEWID()",
     nativeQuery = true)
   Word getRandomWord(@Param("length") int length);

}
```

Figura 280. Así queda implementado el repositorio

3.1. Tests de rendimiento tras la refactorización

Tan contemos estamos que lanzamos el servidor que tenemos en Eclipse y ejecutamos el *Test plan* de JMeter... todas las peticiones fallan. Si miramos la

consola de Eclipse, todas las peticiones fallan porque *The column name id is not valid*.

No pasa nada: modificamos la consulta en la anotación *@Query* e incluimos la columna *id,* dejándola como: *SELECT TOP 1 id, word FROM…*

Ahora las peticiones de los 50 usuarios reciben un código 200. Pero no solo eso, sino que los tiempos de respuesta mejoran ostensiblemente, con una media de 0,1 segundos, frente a los 25 segundos de la Figura 264.

Incluso con 500 usuarios obtenemos respuestas satisfactorias en todos los casos, con medias de 3,7 segundos:

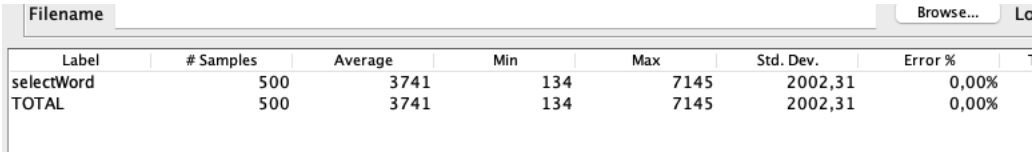


Figura 281. Prueba con 500 usuarios

4. Otra posible solución: utilizar una caché

En las páginas anteriores de este capítulo hemos diagnosticado y solucionado el problema. En esencia, la solución ha consistido en delegar al servidor de base de datos, de forma eficiente, la búsqueda de la palabra.

Otra forma de arreglarlo es la que se resaltamos en la Figura 282: dotamos al *WordRaceService* de una *caché* de palabras indexadas por longitud. La primera vez que se ejecuta *selectWord,* leemos la caché y guardamos en ella todas las palabras en una lista, según su longitud. Cuando llegue la segunda petición, simplemente recuperaremos la lista de palabras correspondiente a la longitud recibida como parámetro y, de dicha lista, devolveremos una palabra al azar.

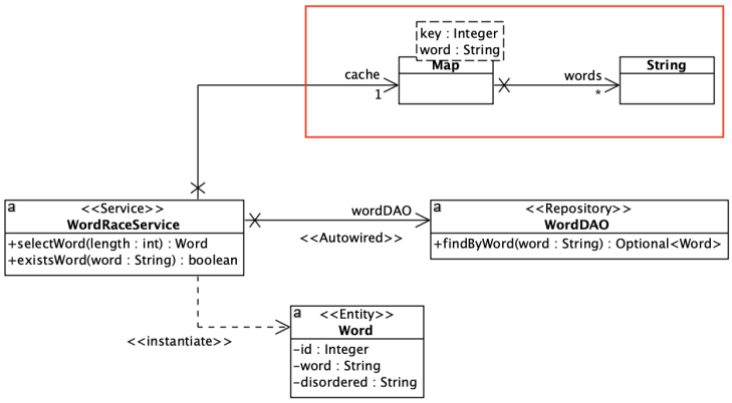


Figura 282. Podemos también usar una caché de palabras

Lo que haremos en la nueva implementación *selectWord* en *WordRaceService* será leer, con la primera petición, todas las palabras de la base de datos, y colocarlas en varias listas contenidas en el campo *caché* según la longitud de cada palabra: todas las palabras de longitud 1 en una lista; las de longitud 2, en otras; …; las de longitud 5, en otra; etcétera.

En la segunda y sucesivas peticiones, la caché estará cargada, con lo que solo tendremos que recuperar la lista correspondiente a la longitud que nos piden y devolver una palabra al azar:

```
@Service
public class WordRaceService {

   @Autowired
   private WordDAO wordDAO;
   private Map<Integer, ArrayList<Word>> cache;

   public WordRaceService() {
      this.cache = new ConcurrentHashMap<>();
   }

   public Word selectWord(int length) {
      Word word;
      ArrayList<Word> selectedWordList;

      if (this.cache.isEmpty()) {
         List<Word> words = this.wordDAO.findAll();

         Integer wordLength;
         ArrayList<Word> wordList;
         for (int i=0; i<words.size(); i++) {
            word = words.get(i);
            wordLength = word.getWord().length();
            wordList = this.cache.get(wordLength);
            if (wordList==null) {
               wordList = new ArrayList<>();
               this.cache.put(wordLength, wordList);
            }
            wordList.add(word);
         }
      }
      selectedWordList = this.cache.get(length);
      int pos = new SecureRandom().nextInt(selectedWordList.size());
      word = selectedWordList.get(pos);
      this.disorder(word);

      return word;
   }
```

Figura 283. Uso de una caché de palabras

Al lanzar el plan de pruebas de JMeter, sin embargo, se producen extrañamente errores. Si inspeccionamos la consola de Eclipse, se nos advierte del mismo error que en la Figura 265 (página 173: no hay conexiones disponibles).

En teoría este error no debería producirse, pues estamos cargando y distribuyendo las 646.616 palabras de la tabla con la primera petición. Pero la primera petición viene seguida de otras varias, que JMeter lanza sin esperar a que la primera haya sido servida. De este modo, volvemos a encontrarnos el mismo problema que con la primitiva implementación.

Pensando en una solución factible (pues estamos seguros de que esto tiene que funcionar), podemos recordar aquel *Manager* que construimos en la sección 7.1 del Capítulo 6 (página 129): en aquel momento, lo utilizamos para cargar la tabla de palabas con las que habíamos bajado de un fichero de texto; ahora, podemos utilizarlo para que, cuando la aplicación arranque, se carguen todas las palabras.

Pues… sí y no: sí, porque la solución funcionaría; pero no, porque el *Manager* es un objeto global, para toda la aplicación. Mejor coloquemos la responsabilidad de cargar e indexar por longitud todas las palabras en un *WordManager*.

Para implementar este *WordManager,* y aunque un buen programador nunca deba copiar y pegar, nos daremos una licencia y copiaremos y pegaremos el nodo *Manager.java* del proyecto en la propia carpeta:

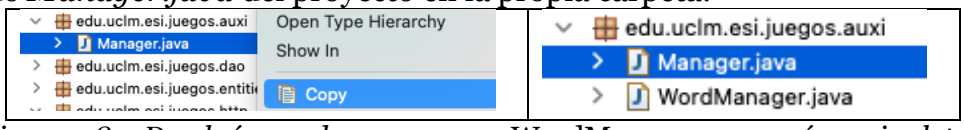


Figura 284. De algún modo creamos un WordManager *que será un singleton*

Al copiar y pegar, Eclipse nos habrá adaptado el nombre y miembros de la clase. Necesitamos modificarle el constructor para que cargue la caché, añadir un método que recupere una palabra aleatoria y otro que la desordene. El código de la Figura 285:

1) En el constructor (que, recordemos, es privado porque esta clase es *singleton* y queremos impedir que se creen inseguramente instancias de esta clase), simplemente instanciamos la caché.
2) En el método *loadCache,* al que llamaremos después de que Spring haya arrancado la aplicación, cargamos la caché de palabras recuperando todas y colocándolas indexadas según su longitud.
3) *getRandomWord* toma una longitud como parámetro, recupera de la caché la lista de palabras de esta longitud y devuelve una al azar.
4) *disorder* tiene exactamente la misma implementación que tenía en *WordRaceService*.

```
@Component
public class WordManager {

   @Autowired
   private WordDAO wordDAO;

   private Map<Integer, ArrayList<Word>> cache;

   private WordManager() {
    this.cache = new ConcurrentHashMap<>();
   }

   public void loadCache() {
    Word word;
    List<Word> words = this.wordDAO.findAll();

    Integer wordLength;
    ArrayList<Word> wordList;
    for (int i=0; i<words.size(); i++) {
     word = words.get(i);
     wordLength = word.getWord().length();
     wordList = this.cache.get(wordLength);
     if (wordList==null) {
      wordList = new ArrayList<>();
      this.cache.put(wordLength, wordList);
     }
     wordList.add(word);
    }
   }

   public Word getRandomWord(int length) {
    ArrayList<Word> selectedWordList = this.cache.get(length);
    int pos = new SecureRandom().nextInt(selectedWordList.size());
    Word word = selectedWordList.get(pos);
    this.disorder(word);
    return word;
   }
```

```
    private void disorder(Word word) {
     char[] disordered = word.getWord().toCharArray();
     int length = disordered.length;
     char aux;
     SecureRandom dice = new SecureRandom();
     for (int i=0; i<length*5; i++) {
      int posA, posB;
      do {
       posA = dice.nextInt(length);
       posB = dice.nextInt(length);
       } while (posA==posB);

       aux = disordered[posA];
       disordered[posA] = disordered[posB];
       disordered[posB] = aux;
     }
     word.setDisordered(new String(disordered));
    }
     ...
}
```

Figura 285. Parte del código del WordManager

Como este nuevo *WordManager* está anotado con *@Component,* Spring lo inspeccionará y llamará a su constructor cuando arranque. Nos quedan dos cosas:

1) Llamar a *WordManager.get().loadCache()* en el método *main* de *App*.
2) Cambiar el código de *selectWord* en el *WordRaceService,* que ahora se reduce a la Figura 286.

```
public Word selectWord(int length) {
    return WordManager.get().getRandomWord(length);
}
```

Figura 286. Sencillísima nueva implementación de selectWord

Ahora, si reejecutamos la prueba de JMeter con 50 usuarios, vemos que el tiempo de respuesta medio se ha reducido a 0,002 segundos (2 milisegundos).

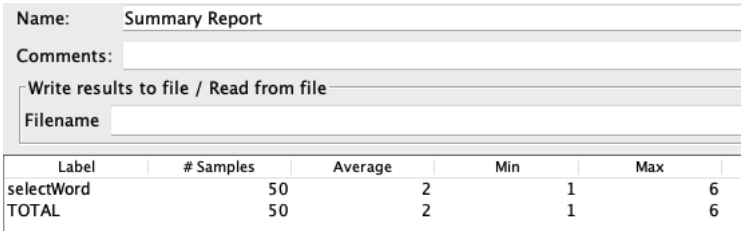


Figura 287. Hemos bajado el tiempo de respuesta medio a solo 2 milisegundos

En la sección 3.1 de este capítulo conseguimos bajar, para 50 usuarios, el tiempo medio de respuesta de 25,845 segundos a una décima de segundo. Con esta tercera solución, dejamos el tiempo de respuesta en 1 milisegundo (cien veces menos que una décima). Incluso con 500 usuarios simultáneos, no obtenemos errores y obtenemos tiempos de respuesta de casi cero.

4.1. Costes y beneficios de la caché

Como casi todo en la vida, usar una caché tiene ventajas e inconvenientes: la ventaja principal es que acelera muchísimo los tiempos de respuesta; entre las desventajas, la carga de la caché supone colocar 646.616 palabras en memoria, la mayoría de las cuales no se utilizarán jamás.

¿Por cuál de las soluciones debemos optar? Pues… depende. Si el servidor de aplicaciones se va a dedicar solamente a servir partidas de *Word race* de la forma en que está implementado, usemos la caché; si va a servir más aplicaciones,

quizás sea mejor la segunda solución. En cualquier caso, enseñemos los números a la dirección de negocio a ver qué opina.

Capítulo 12. Reejecución de pruebas

1. Pruebas de regresión

Las pruebas de regresión son aquellas pruebas que se ejecutan cuando el sistema ha sido modificado. Su objetivo es comprobar que los cambios introducidos no han introducido errores que antes no existían.

En nuestro paquete de *test* tenemos tres ficheros (Figura 288, izquierda):

1) *TestFunctionalGames* incluye casos de prueba de Selenium.
2) *TestSelectWord* incluye los casos de prueba que implementamos al desarrollar el *backend* con TDD. Recordará el lector que utilizaba una base de datos en memoria y que guardaba en esta unas pocas palabras predeterminadas con las que luego hacía las pruebas. Este fichero utilizaba el *application.properties* que teníamos en el subsistema de *tests;* pero luego, en algún momento, lo renombramos a *application.properties.old* para no utilizar la base de datos en memoria y sí usar la base de datos real (Figura 288, derecha).
3) *TestSelectWordReal* utilizaba un *MockMvc* para hacer algún tipo de prueba con la base de datos de SQL Server.

Figura 288. Tenemos tres ficheros con casos de prueba

2. Reejecución de los casos de prueba

Aunque no estamos desarrollando todo con *Test-driven development*, podemos recordar que uno de sus pasos (página 88) exige "Ejecutar todos los casos de prueba de los que disponemos hasta la fecha".

1) Primero ejecutamos los casos de prueba de Selenium: estos se lanzan contra *localhost:4200,* por lo que necesitamos tener lanzados el servidor de Angular, el de Eclipse y el docker con SQL Server. JUnit emite para ellos veredicto de paso, mostrando la barra verde.
2) En segundo lugar ejecutamos *TestSelectWordReal,* que emite veredicto de fallo. Si revisitamos el código del único caso de prueba que contiene esta clase (Figura 187, página 134), vemos que se limita a recuperar una palabra de 15 letras y enviar tres peticiones a *checkWord:* las dos primeras esperan un 404 y la tercera un 200.
   ¿Qué pasará? Pues muy sencillo: Spring está lanzando la aplicación desde la clase de test, pero no estamos cargando la caché con la lista de palabras. Para forzarle a que lo haga, añadimos a *TestSelectWordReal* un método *@BeforeEach* que fuerce la carga de la caché (Figura 289).

```
@SpringBootTest
@ExtendWith(SpringExtension.class)
@AutoConfigureMockMvc
public class TestSelectWordReal {

    @Autowired
    private MockMvc server;

    @BeforeEach
    void setUp() {
      WordManager.get().loadCache();
    }

    @Test
    void test1() throws Exception {
      ...
}
```

Figura 289. Forzamos la carga de la caché

Con este último cambio, el caso de prueba se supera, pero en un futuro podemos encontrarnos un problema importante si añadimos a la clase más métodos *@Test* (es decir, más casos de prueba): la carga de la caché es una tarea lenta, y no es preciso hacerla antes de cada caso, sino que bastaría con ejecutarla una sola vez.

Así que vamos a modificar esa etiqueta *@BeforeEach* por *@BeforeAll*.

```
@BeforeEach
void setUp() {
    WordManager.get().loadCache();
}
```

```
@BeforeAll
void setUp() {
    WordManager.get().loadCache();
}
```

Figura 290. Cambiamos la anotación

Lamentablemente, al intentar reejecutar, JUnit nos informa de un error en su consola, si bien nos apunta alguna posible solución.

@BeforeAll method 'void edu.uclm.esi.juegos.TestSelectWordReal.setUp()' must be static unless the test class is annotated with @TestInstance(Lifecycle.PER_CLASS).

Nos dice que el método ha de ser estático a menos que la clase tenga la anotación *@TestInstance(LifeCycle.PER_CLASS)*. Probamos lo primero y añadimos al *setUp* el modificador *static,* pero sigue fallando, ahora por una *NullPointerException*. Pues probemos lo segundo: le quitamos el *static* y añadimos esa anotación a la clase:

```
@SpringBootTest
@ExtendWith(SpringExtension.class)
@AutoConfigureMockMvc
@TestInstance(Lifecycle.PER_CLASS)
public class TestSelectWordReal {

    @Autowired
    private MockMvc server;

    @BeforeAll
    void setUp() { // Quitamos el static
      WordManager.get().loadCache();
    }
      ...
}
```

Figura 291. Configuración final de la clase

Por fin, se supera también el caso que teníamos aquí.

Capítulo 13. Pruebas de rendimiento con JMeter (II)

1. Continuamos con el escenario de pruebas

Detectado y corregido, gracias a los tests de rendimiento del Capítulo 10, y al *profiling* realizado en el Capítulo 11, el problema de esa especie de cuello de botella que provocaba la ralentización del sistema y el rechazo de peticiones, habremos optado por alguna de las soluciones propuestas.

Sea cual fuere la solución elegida, los jugadores de *Word race* comienzan a intercambiar letras cuando el servidor les ha entregado una palabra desordenada. Cada intercambio de letras envía una petición al recurso */words/checkWord* del servidor, que debemos reproducir, para todos los usuarios de los que dispongamos en el grupo de hilos, en JMeter.

Como se recordará, cuando hicimos la prueba con Selenium de esta funcionalidad, pulsábamos letras "a lo loco" hasta encontrar una combinación que el recurso *checkWords* diese por buena.

Ahora haremos algo parecido: cada hilo (que representa a un usuario) enviará un número aleatorio de peticiones a *checkWord* que esperamos devuelvan un código 404; en algún momento, enviaremos una petición con la palabra ordenada, esperando recibir un 200 del servidor, lo que le indicará que la combinación es correcta.

2. Desarrollo del escenario

Construiremos este nuevo escenario poco a poco. Haremos una "prueba del mono" (*monkey test)* para que, de momento, cada usuario lance una petición a *checkWord* después de haber recibido la respuesta a *selectWord*. Si funciona, seguimos.

2.1. Adición de una petición a *checkWord*

Tras recibir la palabra con la llamada a *selectWord,* añadimos una nueva petición al recurso *checkWord* (Figura 292, izquierda). De momento, en todos los casos comprobaremos la palabra "casto" (en minúsculas ahora, pues así es como tenemos las palabras en la tabla de la base de datos: Figura 292, centro). Esperamos recibir un código 200 para todos los usuarios, que es lo que sucede (Figura 292, derecha).

Podemos ir observando que, mientras que en esta ejecución el tiempo medio de recuperación de una palabra es de 1 milisegundo (se ha dejado la implementación de la caché), la comprobación de la corrección de una cadena requiere una media de 0,24 segundos, que parece demasiado. Volveremos sobre esto más adelante.

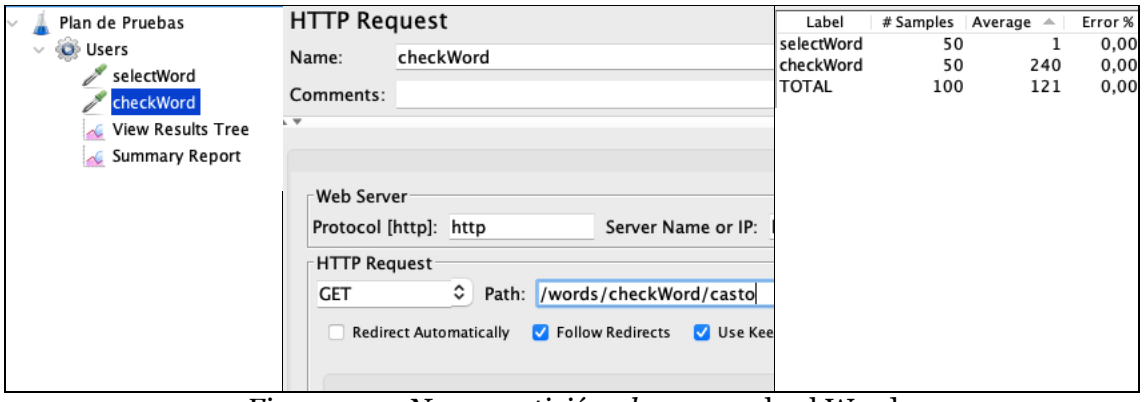


Figura 292. Nueva petición al recurso checkWord

2.2. Envío de la propia palabra recibida

El cuerpo de la respuesta que recibe cada petición a *selectWord* es un JSON con los campos *id, word* (que es la palabra ordenada) y *disordered*. En la Figura 262 (página 172) mostrábamos la información completa de la respuesta: en aquel ejemplo, *word=suecos* y *disordered=uceos*.

Ahora, vamos a modificar la petición *checkWord* para que envíe el valor del campo *word* de la respuesta recibida. Obviamente, esperamos obtener también un código 200 en todos los casos.

Necesitamos leer la respuesta a la petición *selectWord* y enviar el campo *word* del JSON recibido en la petición *checkWord*. Para esto:

1) Añadimos a *selectWord* un *JSON Extractor*, que permitirá leer la respuesta JSON recibida:

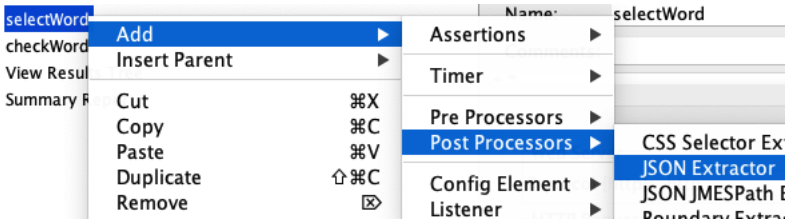


Figura 293. Añadimos un JSON Extractor

2) Configuramos este *JSON Extractor* como en la Figura 294:
   a. Como nombre le damos *word,* lo que nos permitirá identificarlo rápidamente en el árbol del plan de pruebas.
   b. Como nombre de la variable creada damos también *word*. Este nombre de variable será el que utilicemos después, cuando enviemos a comprobar la palabra.
   c. En el campo *JSON Path expressions* escribimos *$.word:* con esto, lo que decimos es que en la variable *word* se guarde el campo *word* de la respuesta (representada por el *$).*

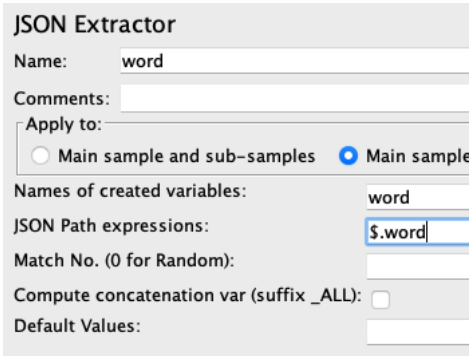


Figura 294. Configuración del JSON Extractor

3) Tenemos que modificar la petición *checkWord* para que ya no mande la palabra "casto", sino el valor de la variable *word* que acabamos de recuperar. Se trata de hacer el pequeño cambio resaltado en la Figura 295: en JMeter, la notación para hacer referencia al valor de una variable es *${NOMBRE_DE_LA_VARIABLE}*.

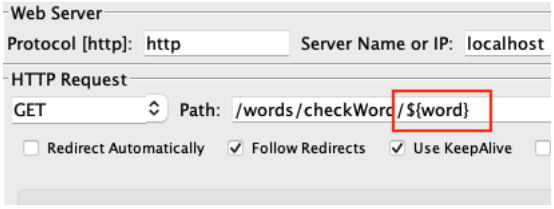


Figura 295. Enviamos el valor de la variable

Todas las peticiones reciben código 200. Además, si inspeccionamos las enviadas a *checkWord,* observaremos que son todas ellas distintas (Figura 296), lo que indica que estamos leyendo correctamente el campo *word* de la respuesta.

Figura 296. Ejemplos de tres peticiones para validar tres diferentes palabras

2.3. Pseudocódigo del plan de pruebas

El intercambio de letras se produce en el lado cliente, en el método *swap* que programamos en TypeScript. Cada intercambio de letras en el cliente envía una petición al servidor: esta segunda parte es lo que vamos a hacer aquí.

En pseudocódigo, el proceso que seguiremos será el siguiente:

```
word = campo word de /words/selectWord?length=5
disordered = campo disordered de la misma
petición
hacer {
    enviar petición a
/words/checkWord/${disordered}
} un número aleatorio de veces
enviar petición a /words/checkWord/${word}
```

Figura 297. Pseudocódigo del proceso

2.4. Diseño del plan de pruebas

La siguiente figura ilustra y explica el diseño del plan de pruebas, que es conforme al pseudocódigo de la figura anterior:

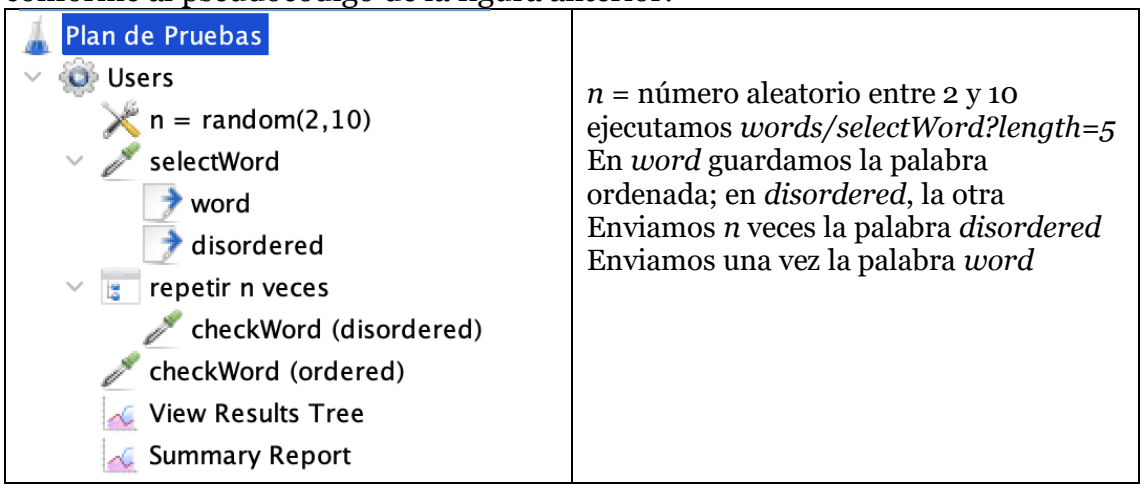


Figura 298. Nuevo plan de pruebas

El elemento etiquetado como *n = random(2,10)* es de tipo Variable aleatoria y tiene la configuración de la Figura 299. Se crea una variable *n* a la que podremos hacer referencia con *${n}*. Nótese que, en el último campo, creamos un valor diferente para cada usuario.

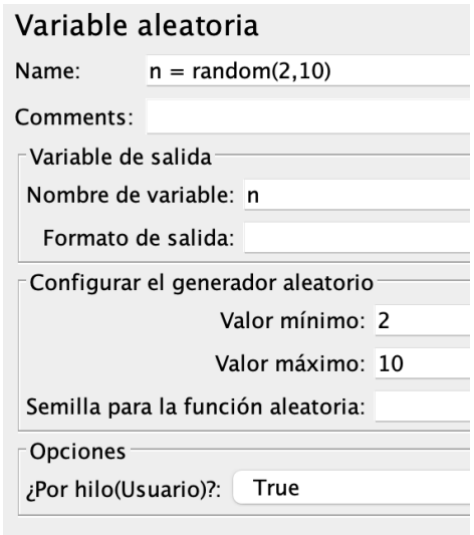


Figura 299. Creación de una variable aleatoria

Los nodos *word* y *disordered* recogen, como ya hemos visto, los valores de los campos *word* y *disordered* de la respuesta a *selectWord*.

El elemento *repetir n veces* es un controlador de bucle (*Loop controller)*. Su configuración es muy sencilla, pues basta con decirle que tiene que ejecutarse *${n}* veces:

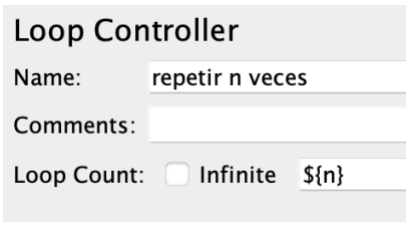


Figura 300. Configuración del Loop controller

Finalmente, *checkWord (disordered)*, que cuelga del bucle, envía al servidor la palabra desordenada. Cuando se termina el bucle, *checWord (ordered)* envía la palabra buena.

3. Ejecución y análisis de los resultados

La ejecución del escenario con 500 usuarios nos da los resultados de la Figura 301: todas las peticiones *selectWord* y *checkWord (ordered)* reciben, como esperábamos, un código 200. Como también esperábamos, la gran mayoría de las peticiones que enviamos desde dentro del bucle, que llevan la palabra desordenada, obtienen un código de error: esto no significa, obviamente, que el sistema esté mal desarrollado pues, en efecto, el servidor devuelve 404 cuando no encuentra la palabra. Hay 6,21% de peticiones 200 del *disordered* que no dan error: se corresponden, seguramente, con palabras desordenadas que, sin embargo, conforman alguna palabra válida. Es el caso de la petición de la Figura 302: probablemente la palabra ordenada fuese "raspa", pero la desordenación que se ha recibido ha sido "pasar", que también está en el diccionario.

Label	# Samples	Average	Error %
selectWord	50	34	0,00%
checkWord (ordered)	50	670	0,00%
checkWord (disordered)	306	800	93,79%
TOTAL	406	690	70,69%

Figura 301. Resultados de la ejecución

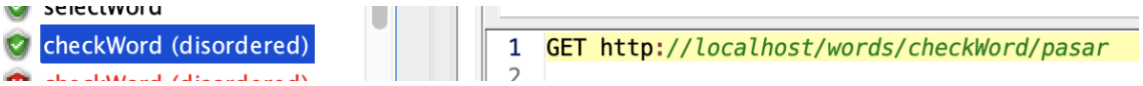


Figura 302. Una petición a checkWord *con una palabra desordenada que, sin embargo, se ha resuelto con código 200*

Con respecto a *selectWord,* el tiempo medio de las peticiones a *checkWord* es muy alto (más medio segundo cada una). Esto se debe a que la comprobación de la existencia de la palabra se realiza preguntando a la base de datos (Figura 303): el controlador pregunta le pasa la palabra al servicio para que la busque; este, a su vez, utiliza el repositorio para buscarla.

```
@GetMapping("/checkWord/{word}")
public void checkWord(@PathVariable String word) {
    if (!this.wordRaceService.existsWord(word))
     throw new ResponseStatusException(HttpStatus.NOT_FOUND);
}
```

```
public boolean existsWord(String word) {
    Optional<Word> optWord = this.wordDAO.findByWord(word);
    return optWord.isPresent();
}
```

Figura 303. La cadena recibida se busca en la base de datos

4. Posibles refactorizaciones para mejorar los tiempos

Ahora, cuando SQL Server recibe la consulta para buscar la palabra, realiza una búsqueda desde el principio al fin de la tabla. Una posible de acelerar la búsqueda es crear un índice en la tabla *word* por la columna… también *word*.

4.1. Opción A: creación de un índice en la tabla

Si inspeccionamos la base de datos con Azure Studio, vemos que en la tabla solo hay un índice único, que corresponde a la clave principal (Figura 304).

Figura 304. Parte de la estructura de la base de datos games

Para reducir el tiempo de búsqueda podemos crear un índice asociado a la columna *word*. Suponiendo que cada palabra exista solamente una vez, dicho índice podría ser también un índice único (también llamado *clave alternativa)*. En la siguiente figura, escribimos la sentencia DDL para crear el índice descrito; sin embargo, falla, y SQL Server nos dice que no ha podido crearse porque hay un valor duplicado. Nos informa además de este valor, “Óscar”.

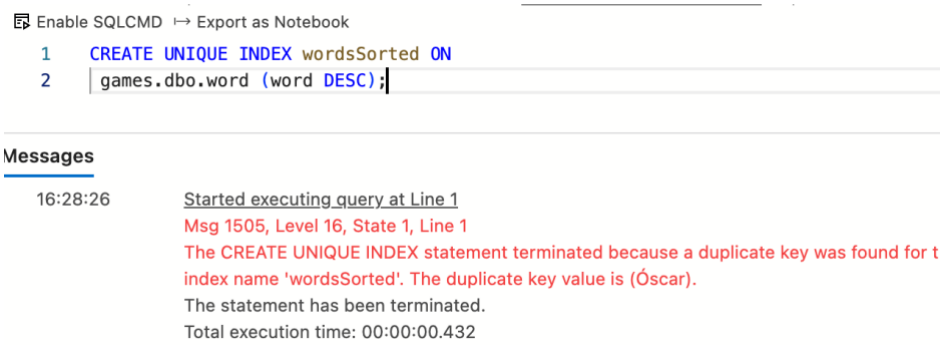


Figura 305. Intento de creación de un índice único en la tabla word

Efectivamente la cadena está "casi" repetida (Figura 306, izquierda): en la fila 478.830 está con minúscula, y en la otra con mayúscula. Eliminemos la segunda y tratemos de crear nuevamente el índice.

Figura 306. Eliminamos un valor repetido

Vuelve a fallar, en este caso por la palabra "Ángela". Bueno, pues vamos a dejar las cosas como están y crearemos un índice que no sea único:

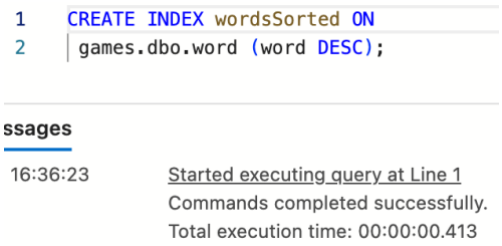


Figura 307. Creación de un índice no único

Tras la creación de este índice se aceleran las búsquedas y se ralentizan ligeramente las inserciones. Vamos a ejecutar otra vez el plan de pruebas de JMeter para comprobar si, en efecto, los tiempos mejoran:

Pues tampoco es que mejoren demasiado… de hecho, son incluso un poco peores que antes:

Label	# Samples	Average	Error %
selectWord	50	1	0,00%
checkWord (ordered)	50	679	0,00%
checkWord (disordered)	285	814	97,19%
TOTAL	385	691	71,95%

Figura 308. No se ha mejorado como esperábamos

4.2. Opción B: buscar en la caché

Ya que tenemos todas las palabras en la caché, podemos reescribir el código del servicio para que se busque la palabra directamente en ella: en la fila superior de la siguiente figura, el método *existsWord* de *WordRaceService* ya no le pide nada al repositorio, sino que le pide al *WordManager*, que tiene la caché de palabras, que la busque. En su nuevo método *existsWord,* el *WordManager* recupera la caché de las palabras de la longitud deseada y busca la palabra.

```
public boolean existsWord(String word) {
    return WordManager.get().existsWord(word);
}
```

```
public boolean existsWord(String word) {
    ArrayList<Word> wordList = this.cache.get(word.length());
    for (int i=0; i<wordList.size(); i++)
     if (wordList.get(i).getWord().equals(word))
         return true;
    return false;
}
```

Figura 309. Buscamos la palabra directamente en caché

Como vemos ahora, los tiempos medios con 50 usuarios bajan a un único y sorprendente milisegundo:

Label	# Samples	Average	Error %
selectWord	50	1	0,00%
checkWord (disordered)	299	1	97,32%
checkWord (ordered)	50	1	0,00%
TOTAL	399	1	72,93%

Figura 310. Tiempos de la nueva versión con 50 usuarios

Animados por los resultados, metamos 1000 usuarios. Los tiempos suben pero siguen siendo muy aceptables (Figura 311). Además, sigue sin haber errores en las peticiones en las que no los esperamos.

Label	# Samples	Average	Error %
checkWord (ordered)	1000	18	0,00%
selectWord	1000	30	0,00%
checkWord (disordered)	6021	53	98,39%
TOTAL	8021	46	73,86%

Figura 311. Tiempos de la nueva versión con 1000 usuarios

4.2.1. Mejorando aún más el tiempo de respuesta

La búsqueda que hace el método *existsWord* en el *WordManager* es una búsqueda lineal, pues recorre la lista de principio a fin. En la mayoría de sus ejecuciones, este método realizará este recorrido de manera completa, pues casi todas las cadenas que se buscan no corresponden a palabras del diccionario.

Por tanto, vamos a hacer dos cambios en *WordManager:*

1) Después de cargar la caché, ordenaremos las palabras alfabéticamente.
2) Cuando busquemos una palabra, haremos una búsqueda binaria.

El nuevo código queda como en la Figura 312. Nótese que:

1) El tipo de los objetos contenidos en la caché ha pasado de ser *String* a *Word.*
2) En el bucle resaltado de *loadCache,* se ordenan todas las listas contenidas en la caché.

3) El método *existsWord* ya no hace una búsqueda secuencial, sino una binaria.

```
@Component
public class WordManager {
    @Autowired
    private WordDAO wordDAO;

    private Map<Integer, ArrayList<String>> cache;

    private WordManager() {
     this.cache = new ConcurrentHashMap<>();
    }

    public void loadCache() {
     Word word;
     List<Word> words = this.wordDAO.findAll();

     Integer wordLength;
     ArrayList<String> wordList;
     for (int i=0; i<words.size(); i++) {
          word = words.get(i);
          wordLength = word.getWord().length();
          wordList = this.cache.get(wordLength);
          if (wordList==null) {
               wordList = new ArrayList<>();
               this.cache.put(wordLength, wordList);
          }
          wordList.add(word.getWord());
     }

     Iterator<Integer> lengths = cache.keySet().iterator();
     while (lengths.hasNext()) {
          wordLength = lengths.next();
          wordList = cache.get(wordLength);
          wordList.sort(new Comparator<String>() {

               @Override
               public int compare(String a, String b) {
                    return a.compareTo(b);
               }
          });
     }
    }

    public Word getRandomWord(int length) {
     ArrayList<String> selectedWordList = this.cache.get(length);
     int pos = new SecureRandom().nextInt(selectedWordList.size());
     String s = selectedWordList.get(pos);
     Word word = new Word();
     word.setWord(s);
     this.disorder(word);
     return word;
    }

    public boolean existsWord(String word) {
     ArrayList<String> wordList = this.cache.get(word.length());
     return Collections.binarySearch(wordList, word)>=0;
    }
    ...
}
```

Figura 312. Nuevo código del WordManager

Ahora, con 50 usuarios, obtenemos tiempos medios de hasta cero milisegundos, y de 1 milisegundo con 1000, obteniendo en todos los casos los códigos de respuesta esperados.

Label	# Samples	Average	Error %
checkWord (disordered)	291	0	100,00%
checkWord (ordered)	50	0	0,00%
selectWord	50	1	0,00%
TOTAL	391	0	74,42%

Label	# Samples	Average	Error %
selectWord	1000	1	0,00%
checkWord (disordered)	5907	1	97,55%
checkWord (ordered)	1000	1	0,00%
TOTAL	7907	1	72,87%

Figura 313. Tiempos con 50 y 1000 usuarios

CUARTA PARTE: *PUT*, *POST*, *DELETE*, *OPTIONS*, PETICIONES CORS

Capítulo 14. Modificación del sistema

1. Nuevos requisitos

Se nos ha pedido que modifiquemos, por un lado, el aspecto visual del sistema para que sea un poco más atractivo y, por otro, que añadamos nuevas funcionalidades para que se permita a los usuarios que estén registrados jugar unos contra otros:

1) Respecto del primer punto, y como conocen el nulo criterio estético del autor de este libro, se han dado por vencidos. No obstante, nos piden que la página principal no muestre directamente los dos juegos, sino que, de algún modo, se dé al usuario la opción de jugar a *Order the numbers* o a *Word race*.
2) Respecto del segundo, se nos pide que añadamos las funcionalidades necesarias para que los usuarios que estén registrados puedan jugar unos contra otros. La forma que nos proponen es que, cuando un usuario registrado se loguea, pueda elegir un juego y o bien jugar una partida como se ha hecho hasta ahora, o bien solicitar el juego contra un oponente. En este caso, entrará a una sala en la que esperará a que llegue un contrincante: si, cuando entra a la sala, ya hay un contrincante esperando, la partida se iniciará automáticamente.
   Cuando dos jugadores juegan a la vez, a uno le aparecerá su propio tablero de juego (el de *Order the numbers* o el de *Wod race),* pero también el del contrincante, que deberá actualizarse en tiempo real. Además, los tableros iniciales de ambos jugadores han de ser el mismo.
   Nos piden además que, si el jugador lleva más de 30 segundos en la sala de espera sin que aparezca el contrincante, se lance un jugador autónomo, falso, pero que le sirva al jugador humano para competir.

2. Análisis y diseño de los cambios

Los cambios que nos han solicitado en primer punto son relativamente sencillos. Los del segundo punto no son triviales, pues requieren que dos clientes registrados se comuniquen en tiempo real entre sí.

La *vista de escenarios* o *vista funcional* de Kruchten (página 17) es la que utiliza diagramas de casos de uso para describir las funcionalidades del sistema que estamos desarrollando y cómo interactúan las funcionalidades de sus posibles subsistemas. Esta es la vista que utilizamos en la Figura 35 (página 51) para describir los casos de uso servidos por cada subsistema (*front-end* y *backend)* y las relaciones entre las funcionalidades de ambos.

Pero realmente, ese sistema compuesto por dos subsistemas era muy sencillo, pues era un típico cliente-servidor basado en peticiones y respuestas. Podía tener alguna dificultad de programación que hemos ido solventando. Pero no había precondiciones entre funcionalidades (para jugar contra un contrincante hay que estar logueado, por ejemplo) ni comunicación entre los dos *front-ends* de dos jugadores. Era un sistema relativamente simple.

De hecho, el autor de este libro, que tiene amplia experiencia en el diseño de *vistas funcionales*, ha empezado directamente con un diagrama de casos de

uso, pero ha preferido pararse y plantear antes un escenario de ejecución con una *vista lógica* (página 16) utilizando un diagrama de secuencia.

Para ello, ha imaginado un escenario en el que se loguean dos usuarios (que, por supuesto, están previamente registrados) llamados Pepe y Ana. Pepe llega primero y elige jugar al *Order the numbers* contra alguien, y Ana elige la misma opción. Cuando Pepe elige jugar a este juego, el servidor crea una sala de espera, pues es lo que nos han indicado en los nuevos requisitos; cuando Ana elige la opción de jugar al mismo juego, el servidor busca partidas pendientes y ve una (la que ha iniciado Pepe): entonces, el servidor añade a Ana y, de algún modo, comunica a los navegadores (el de Pepe y el de Ana) que la partida está lista para ser jugada.

El diagrama de secuencia que se ha construido, y que ilustra el escenario descrito en el párrafo anterior, es el de la Figura 314:

1) En los mensajes 1, 2, 3 y 5, Pepe escribe sus credenciales, pulsa el botón que corresponda y obtiene una respuesta positiva (200) del servidor.
2) En los mensajes 4, 6, 7 y 8, sucede lo mismo con Ana.
   Como curiosidad, obsérvense los números de secuencia: en el concretísimo escenario que describe el diagrama, Ana escribe su nombre (mensaje 4) antes de que a Pepe le llegue la respuesta a su petición (mensaje 5). Es un detalle minúsculo, pero tenga el lector en cuenta esta "asincronía" de los mensajes en sus desarrollos reales, pues no todo sucede secuencialmente.
3) En el mensaje 9, Pepe solicita jugar contra alguien al *Order the numbers,* motivo por el que el servidor lo añade a la sala de espera.
4) Ana expresa el mismo deseo en el mensaje 12: el servidor recibe la petición y añade a Ana a la sala de espera.
5) Como ya estaba Pepe esperando a jugar a ese mismo juego, la sala de espera crea una partida y, de alguna manera, les dice a los dos navegadores que la partida está lista (mensajes 6 y 17).
6) A partir de ese momento, entendemos que Pepe y Ana podrán comenzar a jugar.

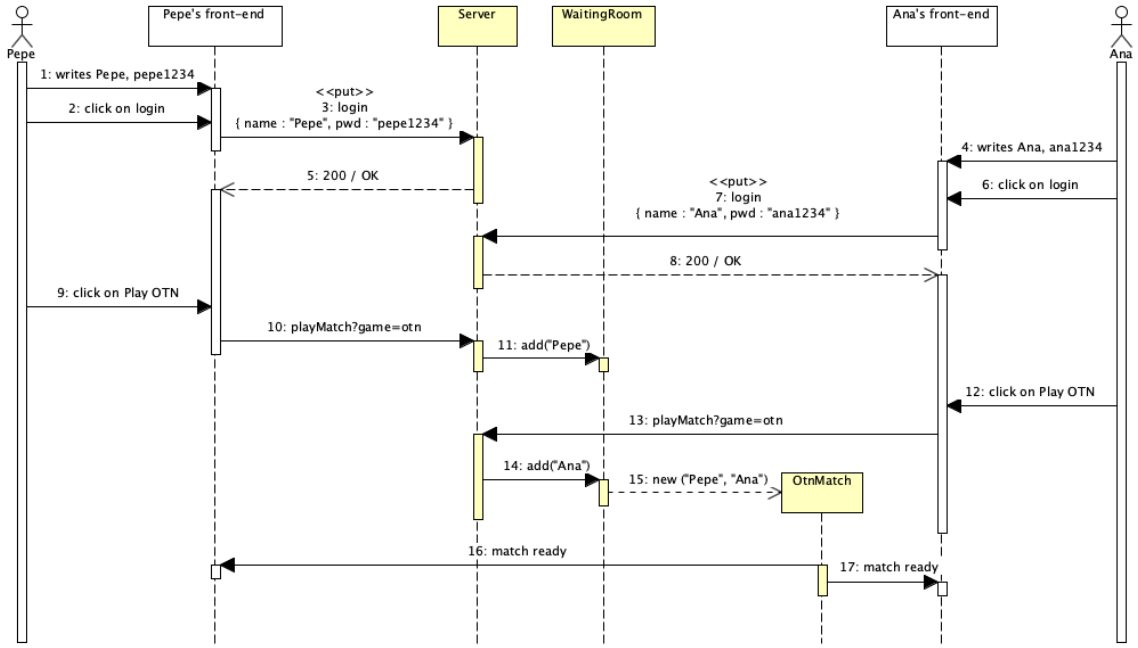


Figura 314. Diagrama de secuencia (una "vista lógica" con la terminología de Kruchten) del comienzo de una partida entre dos jugadores

Este diagrama de secuencia puede parecer muy trivial y sencillo, pero no olvidemos que el diseño arquitectónico, según Pressman, ha de facilitar "la comunicación entre los diferentes *stakeholders"*.

Y efectivamente, el diagrama de secuencia anterior, adecuadamente explicado, es fácilmente entendible casi por cualquiera. Un técnico que lo analice con detalle tal vez dirá: «Oye, pero en el mensaje 15, cuando creas la instancia de *OtnMatch,* la partida no se va a crear pasando como argumentos las cadenas "Pepe" y "Ana", sino que serán probablemente dos objetos que representen a los jugadores Pepe y Ana». Por supuesto que lleva razón, pero estamos trabajando con un documento de muy alto nivel que, en primer lugar, nos sirve a nosotros mismos para plantear el diseño del sistema y, en segundo, para explicar a nuestro cliente la idea de desarrollo que tenemos. Si el cliente nos da el OK, que es lo que esperamos, podremos hacer un diseño arquitectónico más técnico y próximo a la realidad, y lo implementaremos aplicando las mejores prácticas que conozcamos.

Con lo que tenemos de la primera versión del sistema, junto a la reflexión hecha para elaborar el diagrama de secuencia de la Figura 314, podemos ir razonando en términos de funcionalidades que deberá ofrecer cada uno de los dos subsistemas. Pensando solamente en el *Order the numbers,* el siguiente es un posible diagrama de casos de uso de la nueva versión:

1) En ambos lados se ha colocado un paquete que contendrá las funcionalidades relacionadas con la gestión de cuentas de usuario: se muestran los casos de uso *login* y *register,* pero probablemente haya otros para cancelar la cuenta, recuperar la contraseña, etcétera.
2) En el lado cliente, los casos de uso *Start match* y *Move digit* tienen ahora dos puntos de extensión pues, si la partida es multijugador, hay que pedirle al servidor que nos dé un contrincante.
3) Como acabamos de decir, el caso de uso *Start multiplayer* se ejecuta cuando la partida va a ser de dos jugadores, y hay que informar al servidor. Esto supone que, en el *back,* se ejecute la funcionalidad *Add to waiting room*. Volviendo al flujo de mensajes de Pepe y Ana, este caso

de uso añadirá a Pepe a la sala de espera cuando Pepe llegue; más tarde, cuando Ana llega y solicita jugar al mismo juego, se ejecuta también este caso de uso: el servidor detectará que hay dos personas esperando e iniciará *Start match*.

Algún lector dirá: «¿Y por qué no se ha puesto un punto de extensión desde *Start match* hacia *Add to waiting room* con una condición que denote que *Start match* se ejecuta cuando hay dos jugadores, igual que se ha hecho en dos casos de uso del lado cliente?». Pues la respuesta no es clara: podríamos haberlo hecho perfectamente, pero tenemos en mente que según los nuevos requisitos debe crearse un jugador "artificial" cuando el humano lleve 30 segundos esperando. Aunque este extremo no está recogido todavía en el diagrama, asumimos que en algún momento habrá que incorporar algún mecanismo que dispare la creación de ese jugador virtual para que la partida comience. Es por esto por lo que se ha preferido crear esa relación de inclusión en lugar de una de extensión.

4) Cuando el servidor ejecuta su *Start match* debe notificarlo al *front-end* de los dos jugadores, lo que se denota con la relación hacia el *Update match* del lado cliente.
5) Cuando un jugador mueve un dígito en una partida multijugador, hay que enviar el movimiento al servidor: este lo recoge en su *Update match* que, a su vez, lo notifica al *Update match* del *front-end* del contrincante.

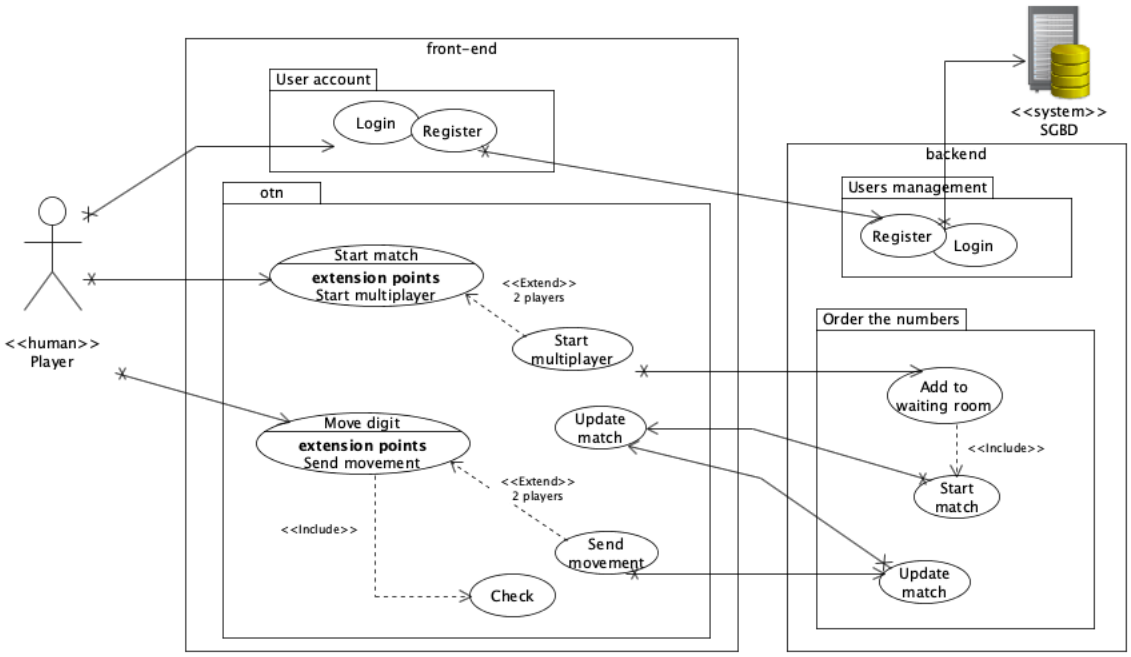


Figura 315. Vista funcional parcial de la versión 2

2.1. Estereotipado de las asociaciones

La introducción del requisito de actualización en tiempo real nos invita a estereotipar algunos de las asociaciones entre casos de uso, para aumentar la capacidad semántica y expresiva del diagrama.

El intercambio de mensajes se realizará mediante principalmente por *http*, pero la actualización de los clientes en tiempo real se hará por *ws*. El siguiente diagrama muestra exactamente los mismos casos de uso y relaciones que el

anterior, pero se incluyen los estereotipos *«http»* y *«ws»* en las asociaciones entre casos de uso de los dos subsistemas.

Se podría haber colocado directamente la Figura 316 en el lugar de la Figura 315, y se podría haber comentado la conveniencia de usar estereotipos en el mismo epígrafe anterior en lugar de en este. Pero se incluyen este nuevo diagrama y este epígrafe titulado «Estereotipado de las asociaciones» para explicitar la utilidad de los estereotipos, que enriquecen mucho la cantidad de información que contienen los diagramas.

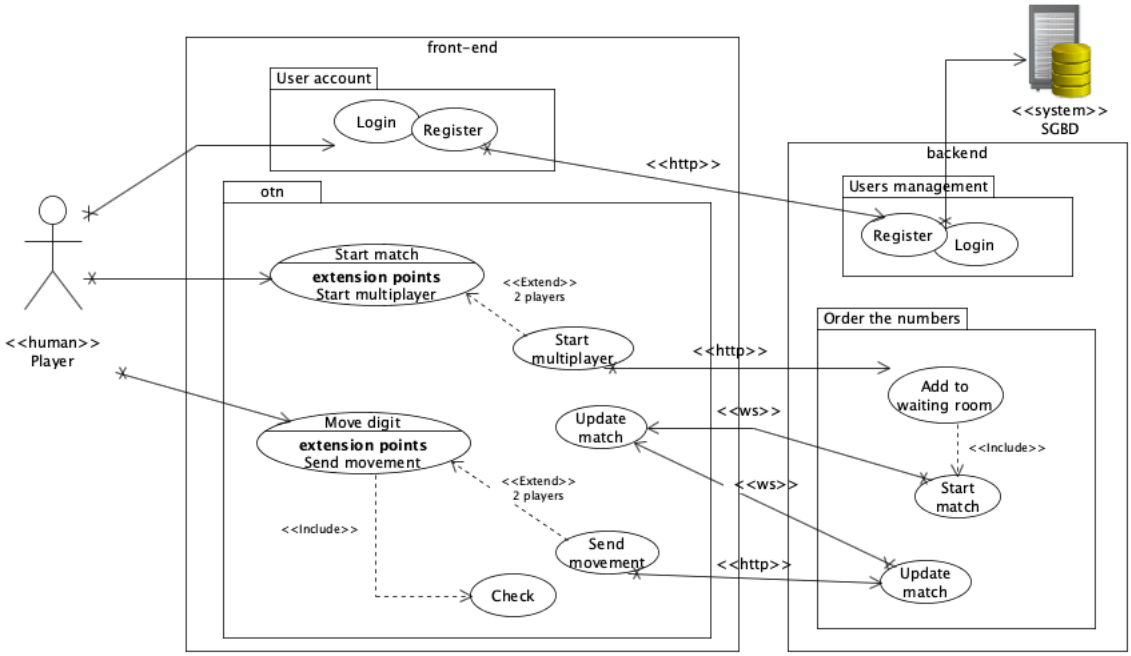


Figura 316. Vista funcional parcial de la versión 2 con estereotipos

2.2. El jugador virtual

Debemos considerar también al jugador automático que se crea a los 30 segundos de estar en la sala de espera sin que llegue un contrincante humano.

En los diagramas de casos de uso, los actores suelen ser de tres tipos: personas (como nuestro *Player)*, otros sistemas (como *el SGBD)* y el tiempo, que dispara ocasionalmente alguna funcionalidad. En este caso, el nuevo actor *Timer* crea un usuario falso cuando pase el tiempo establecido. Colocado en la parte inferior derecha, ese reloj dispara en algún momento la funcionalidad de crear el jugador artificial, que a su vez supone que el jugador se añada a la espera de espera y comience la partida.

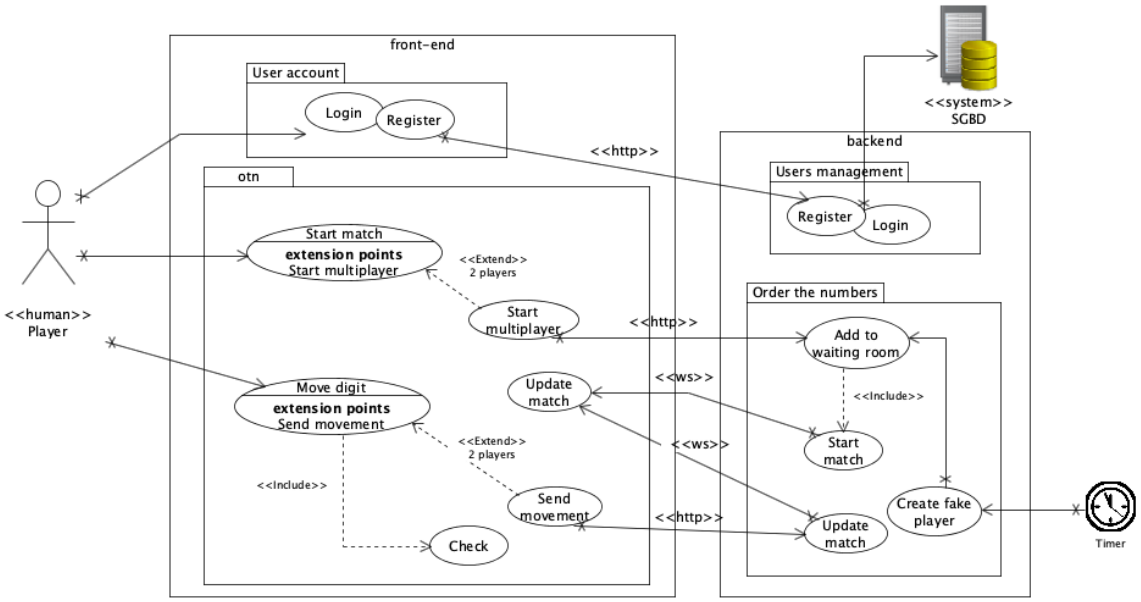


Figura 317. Adición de un reloj como un actor

Puede discutirse la conveniencia de utilizar el reloj como actor, pues en la realidad se implementará en el interior del *backend*. Esto es cierto, pero no hay que olvidar el propósito del diagrama de casos de uso, que es proporcionar una *vista funcional* del sistema, con tanta información como sea posible pero sin recargarlo. Con el reloj, el lector del diagrama entenderá enseguida que de vez en cuando hay algún evento que se dispara de forma automática.

2.3. Inclusión del *Word race*

En la versión inicial de este juego (Figura 35, página 51) teníamos solamente dos casos de uso en el cliente (*Start match* y *Swap letters)* y dos en el servidor (*Select word,* que se ejecutaba al empezar la partida, y *Check word*, que comprueba si la cadena recibida es una palabra del diccionario).

Según los requisitos de la segunda versión, el *Word race* debe tener un comportamiento realmente muy parecido al de *Order the numbers*. De hecho, los casos de uso de la figura anterior que hemos identificado para *Order the numbers* casi que nos sirven para la nueva versión del *Word race:*

1) En la vista funcional de la primera versión, ya había un *Start match* diferente para cada juego. En esta nueva versión mantenemos esta distinción pero, además, debemos contemplar también la posibilidad de iniciar una partida contra un contrincante-
2) Respecto de *Move digit,* este nombre no resulta adecuado para el *Word race*, pues en este no se mueven dígitos, sino que se intercambian letras. *Move digit* incluye el caso de uso *Check,* que comprueba si el tablero de números está ya ordenado; en el *Word race* la comprobación de la jugada se hace en el servidor, que es quien conoce las palabras.
3) En el lado servidor, el caso de uso *Update match* actualiza la partida y actualiza a los clientes. En el caso del *Word race,* además, debe comprobar si la palabra existe.

En la Figura 318 se recoge el nuevo modelo de casos de uso:

1) En el lado cliente se ha creado un caso de uso abstracto (*Start match)* con dos especializaciones, una por cada juego:

a. El *Start* de *Order the numbers* se utiliza para iniciar una partida individual, como en la versión 1, por lo que no necesita conexión al servidor.
b. El *Start* de *Word race* sí necesita conectar al servidor para obtener la palabra con la que jugar.
c. Además, *Start match* dispone de un punto de extensión (*Start multiplayer)* para iniciar una partida en modo multijugador.

2) También en el cliente se ha creado otro caso de uso abstracto, *Move*, que está especializado en ambos juegos. Cuando es multijugador, el movimiento se envía mediante el punto de extensión *Send movement*.
3) En el lado servidor se ha creado un punto de extensión *Check word* para el caso de uso *Update match,* que se ejecuta cuando el juego al que se está jugando es el *Word race*.

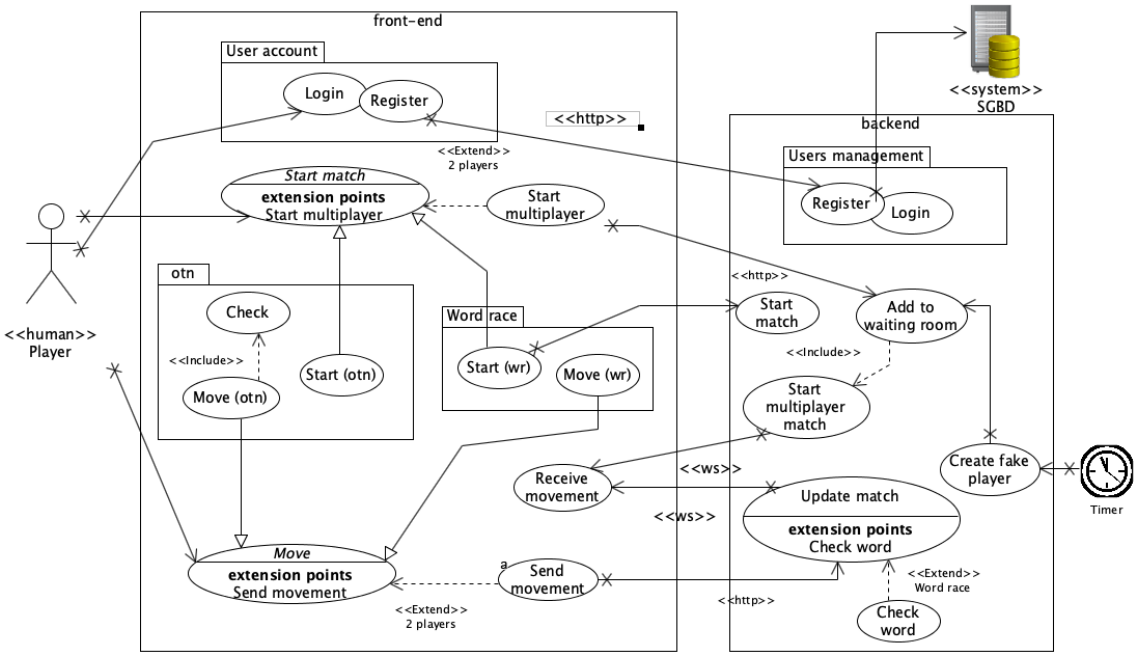


Figura 318. Vista funcional de los dos juegos

3. Plan de desarrollo

Para jugar en modo multijugador es necesario que los jugadores estén registrados y logueados en el sistema, por lo que parece adecuado priorizar el desarrollo de esos casos de uso. En este punto podemos pensar que habrá una pantalla para escribir el nombre y la contraseña y que, si las credenciales son válidas, el navegador del usuario mostrará una nueva pantalla para elegir el juego.

En los nuevos requisitos nos pidieron también que hubiese pantallas distintas para cada juego, y sugerimos en aquel momento (primeras líneas de la Sección 2 de este capítulo) utilizar para esto el *router de Angular*.

Cuando los usuarios puedan registrarse y loguearse, y el navegador muestre diferentes páginas en función de los resultados, pasaremos al desarrollo de las nuevas funcionalidades para los juegos. Habrá que hacer cambios importantes en el diseño y arquitectura de los dos subsistemas, por lo que postergamos la priorización de estos casos de uso para más adelante.

Con estas consideraciones, el plan de desarrollo queda como sigue:

1) Integrar el *router* para elegir el juego.

2) Desarrollar el paquete de gestión de cuentas de usuario en el *backend*.
3) Desarrollar el paquete de gestión de la cuenta en el *front-end* y conectarlo con el *back*.
4) Planificar el resto del proyecto cuando lo anterior esté hecho.

Capítulo 15. Integración del *router* en el lado cliente

1. El *router*

Cuando generamos el proyecto del *front-end* (Figura 37, página 54), Angular nos preguntó que si deseábamos añadir el *Angular routing,* respondiendo que *Yes*. Luego, eliminamos bastante código de *app.component.html,* pero mantuvimos la última línea, *<router-outlet></router-outlet>* (Figura 46, página 58). Este elemento hace referencia al *router*, que nos va a permitir navegar entre las varias páginas que compongan el lado cliente.

La declaración de este elemento aparece también en el fichero *app.module.ts* (Figura 194, página 138) y, además, disponemos en el árbol del proyecto del fichero *app-routing.module.ts*, en el que podemos configurar el *router*.

La Figura 319 muestra la parte más interesante de la estructura actual de nuestro *front*: nótese la multiplicidad 1 que hay en los extremos de las agregaciones que salen de *app.component.html*. Este "1" denota que *app.component.html* siempre tiene una instancia de cada uno de los componentes contenidos. Así, cuando se carga la página, se cargan también los dos y sus contenidos se "incrustan" directamente en la página principal.

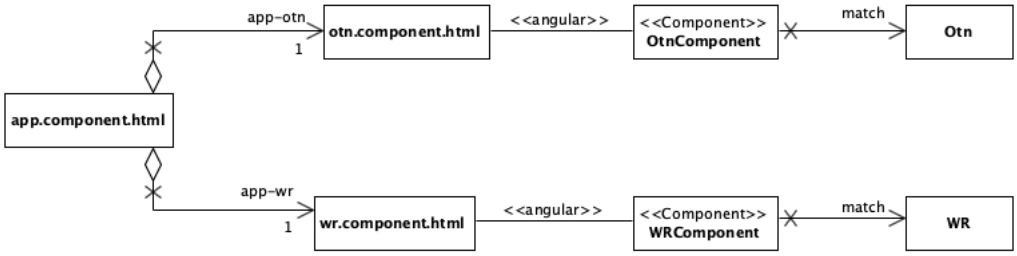


Figura 319. Estructura actual del front-end

El *router* nos va a permitir cargar diferentes componentes según, por ejemplo, los enlaces sobre los que pinchemos o los resultados que obtengamos.

De acuerdo con los requisitos de esta nueva versión, en la página principal debe aparecer una lista con los juegos disponibles, y debe cargarse la página correspondiente al juego elegido.

A nivel de clases, el resultado de aplicar el *router* equivale a sustituir las dos multiplicidades con valor "1" de la figura anterior y sustituirlas por "0..1". El *router* lo tenemos también incrustado en el componente *app* (Figura 194, página 138), y lo vamos a utilizar para que nos coloque el componente que deseemos en la página principal. Con respecto a la figura anterior, la estructura pasará ahora a ser de este modo:

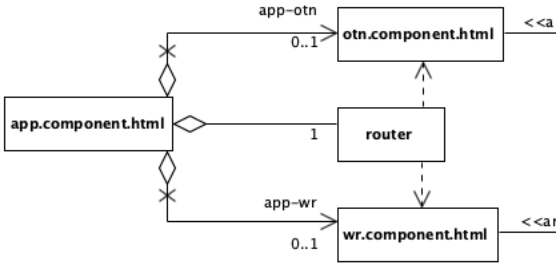


Figura 320. Un posible diagrama para entender el router

Y, en forma de diagrama de secuencia, el comportamiento del *router* será el siguiente (Figura 321): el usuario hace clic en un cierto enlace, lo que envía un mensaje al modelo asociado a la vista; el modelo le pide al *router* que cargue una página nueva; el *router,* que está incrustado en *app.component.html*, la carga.

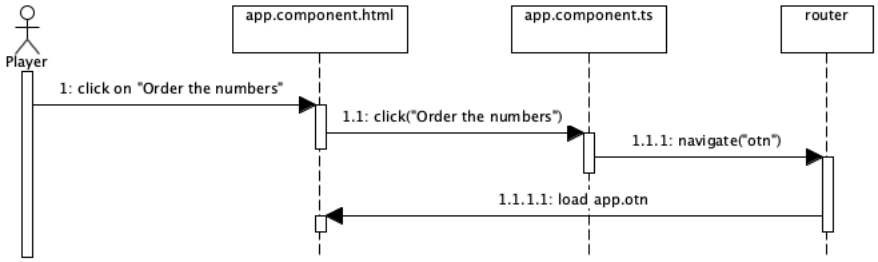


Figura 321. Comportamiento del router

2. Actualización del *front-end*

En las últimas líneas de nuestro *app.component.html* incluimos las etiquetas con los dos componentes de los dos juegos y la del *router*. Quitaremos las dos primeras y las sustituiremos por dos enlaces:

`<app-otn></app-otn>` `<app-wr></app-wr>` `<router-outlet></router-outlet>`	`<a (click)="goTo('otn')">Play Order the numbers</a>` ` ` `<a (click)="goTo('wr')">Play Word race</a>` `<router-outlet></router-outlet>`

Figura 322. Modificación del código del componente principal

El evento clic de ambos enlaces está asociado a un método *goTo* que debemos escribir en el modelo (o sea, en *app.component.ts)*. En la Figura 323:

1) Hemos añadido el método *goTo,* que utiliza *this.router*.
2) Como *this.router* no existe en el modelo, lo hemos inyectado en el constructor.
3) Finalmente, lo hemos importado de *@angular/router*.

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {

  title = 'fejuegos';

  constructor(private router : Router) {}

  goTo(page : string) {
    this.router.navigate([page])
  }
}
```

Figura 323. El método goTo *redirige el* router

Ahora, cuando se haga clic en el primer enlace, esperamos que se ejecute el método *goTo,* que este llame al *navigate* del *router* y que este cargue la página pasada como parámetro. Sin embargo, debemos especificar antes a qué componentes corresponden cada una de las posibles páginas que pasamos. Esto lo hacemos en el fichero *app-routing.module.ts:* su constante *routes* es un array de objetos JSON con dos campos: *path,* que indica el nombre de la página que usamos en el método *navigate* y *component,* que es el componente que debe cargarse (Figura 324): la url *localhost:4200* devuelve el nuevo código del componente *app*; al hacer clic en alguno de los enlaces, se carga el componente que corresponda y, además, se añade el campo *path* a la url que muestra del navegador.

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { OtnComponent } from './otn/otn.component';
import { WrComponent } from './wr/wr.component';

const routes: Routes = [
  { path : "otn", component : OtnComponent },
  { path : "wr", component : WrComponent}
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

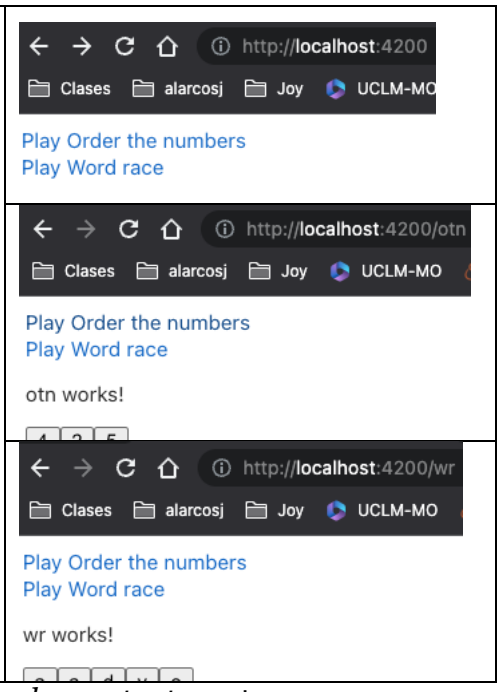


Figura 324. Añadimos las rutas a la constante routes

Capítulo 16. Gestión de usuarios en el servidor

1. Caso de uso *register*

La siguiente figura muestra el posible escenario de registro satisfactorio de un usuario (a este tipo de escenarios, en los que no se espera error, se los llama muchas veces "escenario normal"). Claramente, pertenece al caso de uso *register* del paquete *Users management* del *backend* (Figura 318, página 207). Como se ve, el controlador recibe un *post* con *name* (nombre de usuario), *pwd1* (contraseña), *pwd2* (confirmación de la contraseña) y *email* (dirección de correo electrónico); hace cierta validación de esta información y, si es correcta, le pide al *UserService* que registre a ese usuario (ya solo le pasamos los campos *name, email name, email* y el campo *pwd1,* pues *pwd1* es igual a *pwd2).*

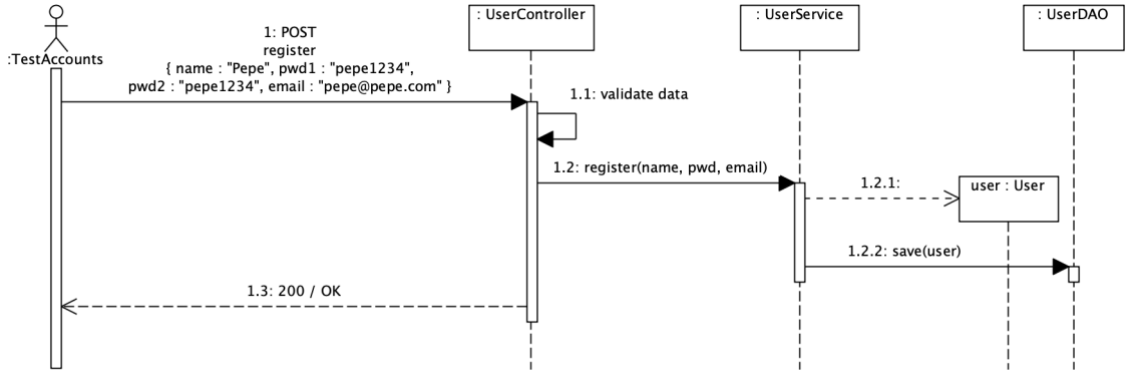


Figura 325. Escenario normal del registro de un usuario

Volveremos a aplicar un enfoque TDD al desarrollo de estas funcionalidades. Por eso, en el diagrama de arriba, hemos llamado *TestAccount* al actor que envía el mensaje al controlador. Además, seguimos teniendo presente la arquitectura que separa de forma bien clara todas las responsabilidades en los diferentes tipos de clases: controladores, servicios, entidades y repositorios.

1.1. Un caso de prueba para el escenario normal

En la Figura 326 creamos un caso de prueba que representa bastante bien la *request* y la *response* que, en el diagrama de secuencia anterior, *TestAccounts* respectivamente envía y recibe del servidor. Nótese que:

1) Hemos puesto el fichero en un paquete *users*, en donde colocaremos las pruebas de temas relacionados con la gestión de usuarios. Por supuesto, este paquete está creado en la zona de tests del proyecto Eclipse (*src/test/java).*
2) La clase se llama *TestAccounts,* como el actor en el diagrama de secuencia.
3) Al caso de prueba le hemos dado un nombre bastante significativo, *testRegister200,* para denotar que se espera obtener un código 200 como respuesta.
4) El caso de prueba crea un objeto *jsoPepe* que contiene la misma información del diagrama de secuencia.
5) Se crea una petición que, ojo, ahora es de tipo *post*, pues esperamos crear un objeto nuevo en el servidor. La url a la que dirigimos la petición es

/users/register (*users* corresponde al controlador, y *register* al nombre del servicio web). Además, le indicamos al servidor que la información que le enviamos en la *payload* es un texto en JSON. Con el último método (*content)* colocamos la *payload* en la petición.

6) Luego se envía la petición (*this.server.perform)* y comprobamos que se recibe un código 200.
7) Finalmente, blindamos un poquito más el caso de prueba añadiendo un aserto que compruebe que el número de usuarios registrados que se llaman "Pepe" es 1.

```
package edu.uclm.esi.juegos.users;

import ...;

@SpringBootTest
@ExtendWith(SpringExtension.class)
@AutoConfigureMockMvc
public class TestAccounts {

   @Autowired
   private MockMvc server;

   @Autowired
   private UserDAO userDAO;

   @Test
   void testRegister200() throws Exception {
      JSONObject jsoPepe= new JSONObject().
            put("name", "Pepe").
            put("pwd1", "pepe1234").
            put("pwd2", "pepe1234").
            put("email", "pepe@pepe.com");

      RequestBuilder request = MockMvcRequestBuilders.
         post("/users/register").
         contentType("application/json").
         content(jsoPepe.toString());

      this.server.perform(request).
         andExpect(status().is(200));

      assertTrue(this.userDAO.countByName("Pepe")==1);
   }
}
```

Figura 326. Escenario de un registro satisfactorio

Aunque los hemos dibujado en el diagrama de secuencia, no hemos desarrollado todavía nada del código de producción: no tenemos controlador, ni servicio, ni entidad ni repositorio: es por esto por lo que Eclipse nos señalará en rojo la declaración de *userDAO* (*UserDAO cannot be resolved to a type*). Pues vamos al tema. Escribiremos solo el código necesario para superar este caso.

1.2. Implementación del controlador

Colocaremos el *UserController* en el paquete *http,* estando al mismo nivel que el ya conocido *WordRaceController*. Aunque no debe hacerse, copiamos este mismo fichero en el árbol del proyecto y lo pegamos con el nuevo nombre, *UserController*. Podemos borrar todos sus métodos y, luego:

1) Mantenemos la anotación *@CrossOrigins("*"),* para que este controlador admita las peticiones que le lleguen desde el origen

(recuérdese el problema que tuvimos con las peticiones a *selectWord,* en torno a la Figura 210, página 146) *localhost:4200*.

2) Cambiamos su *@RequestMapping* a *users* (es la primera parte del nombre del recurso al que el caso de prueba dirige la petición).
3) Creamos un método, que anotamos con *@PostMapping("register")*, pues *register* es la segunda parte del nombre del recurso al que el caso de prueba dirige la petición.
4) Este nuevo método recibe una *payload* con las credenciales del usuario, que viajan desde el cliente en formato JSON. Cuando los interceptores de Spring reciban la petición, buscarán en el controlador *users* un servicio llamado *register* que sea de tipo *post;* si lo encuentran, como va a ser el caso, tratarán de inyectar la *payload* en algún parámetro. Un JSON puede inyectarse en un *Map*, por lo que el método incluye un parámetro de este tipo, que anotamos con *@RequestBody* (lo que denota que en este parámetro debe inyectarse el JSON recibido).

El código del controlador es el de la Figura 327. Nótese que hemos declarado también el objeto de tipo *UserService,* que todavía no existe pero al que, según nuestro estilo de diseño y según el diagrama de secuencia de la Figura 325, debemos reenviar la petición. El método, por otro lado, devuelve un *void*, pero ya sabemos que, de acuerdo con el caso de prueba que hemos escrito arriba, lo que nos interesa conocer en el lado cliente es, en primer lugar, el código de respuesta.

```
package edu.uclm.esi.juegos.http;

...

@RestController
@RequestMapping("users")
@CrossOrigin(origins = "*")
public class UserController {

   @Autowired
   private UserService userService;

   @PostMapping("/register")
   public void register(@RequestBody Map<String, Object> info) {

   }
}
```

Figura 327. Empezamos a escribir el UserController

1.2.1. Método *register*

De acuerdo con el diagrama de secuencia, el método *register* debe hacer una validación de los datos recibidos (mensaje 1.1 de la Figura 325). Tendrá que comprobar, por ejemplo, que la dirección que llega en el campo *email* es en efecto un correo electrónico válido; que los campos *pwd1* y *pwd2* (contraseña y su confirmación) coinciden, y que cumplen ciertos parámetros de seguridad. Se podría comprobar también que no existan ya en la base de datos ni el campo *name* ni *email,* pero esto puede delegarse al propio sistema gestor de bases de datos, que deberá devolver una excepción en caso de que se intente insertar un nombre de usuario que ya existe.

Para hacer las dos validaciones de datos indicadas debemos leer los campos *pwd1, pwd2* y *email* del JSON que se ha recibido y que se ha inyectado en el mapa *info* que hemos puesto como parámetro de este método:

```
@PostMapping("/register")
public void register(@RequestBody Map<String, Object> info) {
   String email = info.get("email").toString();
   String pwd1 = info.get("pwd1").toString();
   String pwd2 = info.get("pwd2").toString();
}
```

Figura 328. Lectura de tres campos del JSON recibido

Para comprobar que el *email* sea en efecto un email, buscamos en Internet con la cadena "check email java" que, en el caso de este escritor, le lleva a *Baeldung,* un lugar muy conocido por programadores Java. En concreto, la página[9] nos da varias formas de validar si una cadena es una dirección de correo electrónico correcta, todas ellas basadas en expresiones regulares (Figura 329):

1) La expresión regular más sencilla solo comprueba que haya una arroba en la dirección.
2) La segunda valida tanto lo que hay a la izquierda de la arroba como a la derecha, pero no admite caracteres no latinos.
3) La tercera admite caracteres *Unicode,* entre los que se encuentran los no latinos.
4) La cuarta es la expresión regular corresponde, al parecer, a la descripción dada en el estándar RFC 5322. Como decía Abraham Lincoln, «No te creas todo lo que lees en Internet solo porque hay una foto con la frase al lado», por lo que el lector interesado puede consultar este estándar en https://www.rfc-editor.org/rfc/pdfrfc/rfc5322.txt.pdf, en cuya página 17 se describe la gramática (no la expresión regular) de las direcciones de correo.

```
^(.+)@(\S+) $.
```

```
^(?=.{1,64}@)[A-Za-z0-9_-]+(\\.[A-Za-z0-9_-]+)*@[^-][A-Za-z0-9-
]+(\\.[A-Za-z0-9-]+)*(\\.[A-Za-z]{2,})$
```

```
^(?=.{1,64}@)[\\p{L}0-9_-]+(\\.[\\p{L}0-9_-]+)*@[^-][\\p{L}0-9-
]+(\\.[\\p{L}0-9-]+)*(\\.[\\p{L}]{2,})$
```

```
^[a-zA-Z0-9_!#$%&'*+/=?`{|}~^.-]+@[a-zA-Z0-9.-]+$
```

Figura 329. Varias expresiones regulares para validar direcciones de correo

En caso de usar una de estas expresiones, usaríamos la cuarta expresión regular, que parece ser la más potente. La propia página de Baeldung nos propone la siguiente función para validar un correo con ella:

```
@Test
public void testUsingRFC5322Regex() {
    emailAddress = "username@domain.com";
    regexPattern = "^[a-zA-Z0-9_!#$%&'*+/=?`{|}~^.-]+@[a-zA-Z0-9.-]+$";
    assertTrue(EmailValidation.patternMatches(emailAddress, regexPattern));
}
```

Figura 330. Validación de una dirección electrónica, tomada de [9]

Si seguimos leyendo un poco más, la página de Baeldung nos comenta que existe también una librería de validadores de Java, implementada por la Apache Foundation. Vamos a probar: incluimos la dependencia en nuestro *pom.xml* (Figura 331, izquierda) y completamos nuestro código (Figura 331, derecha).

[9] https://www.baeldung.com/java-email-validation-regex

```
<dependency>
 <groupId>commons-validator</groupId>
 <artifactId>
   commons-validator
 </artifactId>
 <version>1.7</version>
</dependency>
```

```
@PostMapping("/register")
public void register(@RequestBody Map<String, Object> info) {
    String email = info.get("email").toString();
    String pwd1 = info.get("pwd1").toString();
    String pwd2 = info.get("pwd2").toString();

    if (EmailValidator.getInstance().isValid(email))
        throw new ResponseStatusException(HttpStatus.CONFLICT,
                "El correo suministrado no tiene un formato vá
}
```

Figura 331. Incluimos la dependencia y la usamos

Pero, horror, nos encontramos de nuevo con una advertencia de "deprecación", de obsolescencia: *The method getInstance() from the type EmailValidator is deprecated*. Si ponemos en Internet exactamente la cadena de búsqueda que aparece en *cursiva* en este mismo párrafo, vamos en primer lugar a la API del paquete que hemos descargado (*commons.validator),* que nos describe la clase *EmailValidator*. Inmediatamente leemos:

Deprecated.

Use the new EmailValidator in the routines package. This class will be removed in a future release.

Ya comentamos (página 150) que utilizar elementos deprecados tiene cierto riesgo en futuras actualizaciones del sistema: ahora podemos confiar en que la operación *EmailValidator.getInstance().isValid* que usamos en el lado derecho de la Figura 331 funciona; pero si, en una versión futura, actualizamos la versión 1.7 de esta librería (que es la que incluimos en el lado izquierdo de esa misma figura), es posible que la clase *EmailValidator* ya no exista, con lo que nuestro código ni siquiera compilará.

¿Qué hacemos? En *stackoverflow.com* no encontramos respuestas convincentes: el usuario Subashini[10] dio el 27 de agosto de 2015 una respuesta a un problema parecido pero que no resulta ser el nuestro. Busquemos más: pongamos directamente "apache validator" como cadena de búsqueda, que nos lleva directamente a la página de Apache de este sitio. En su sección de *Downloads* vemos que la última versión publicada es la 1.7, del 3 de agosto de 2020.

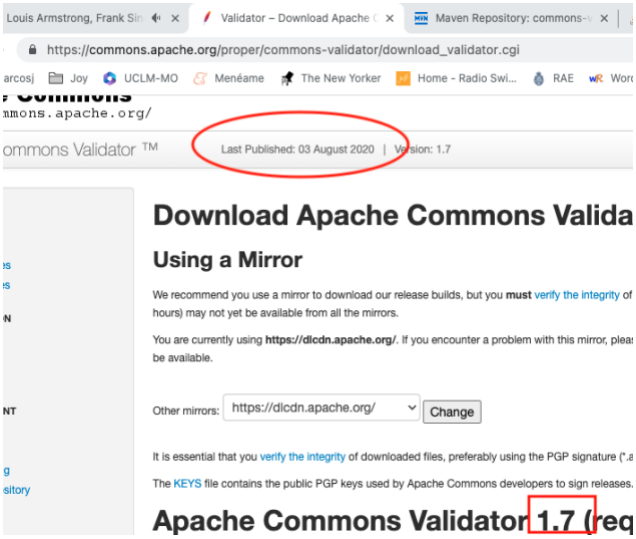


Figura 332. Página con los validators *de Apache*

[10] https://stackoverflow.com/questions/32259179/errors-with-the-apache-email-validator

Bien, pues aquí hay un pequeño dilema: ¿usamos la librería, o pasamos de ella y nos decantamos por la expresión regular? Pues nos decantaremos por usar la expresión regular.

El ejemplo que nos daba Baeldung en la Figura 330 no es demasiado completo, pues usa una clase *EmailValidation* de la que nos disponemos. Pero bueno, las cadenas de la Figura 329 son todas expresiones regulares que pueden ser resueltas con la librería básica *java.util.regex*.

En efecto, si eliminamos los pocos errores de compilación que tenemos comentando unas pocas líneas (campo *userService* en el *UserController,* que todavía no existe, y campo *userDAO* en *TestAccounts)* y ejecutamos el test, este se supera:

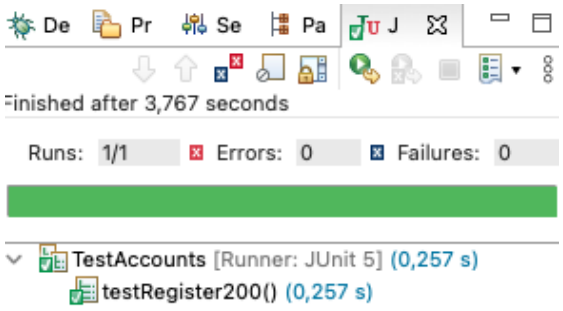


Figura 333. El código que hemos escrito hace que el caso de prueba se supere

El lector atento preguntará por el código que hemos escrito para validar el correo. Aquí va:

1) Aprovechando la librería mencionada (*java.util.regex*), creamos un objeto *pattern* con la expresión regular que hemos elegido.
2) Le decimos al *pattern* que construya un *Matcher* con el correo electrónico que recibe. Si no casa con el patrón, lanzamos un error 409 (conflicto).

```
@PostMapping("/register")
public void register(@RequestBody Map<String, Object> info) {
   String email = info.get("email").toString().trim();
   String pwd1 = info.get("pwd1").toString().trim();
   String pwd2 = info.get("pwd2").toString().trim();

   Pattern pattern = Pattern.compile("^[a-zA-Z0-9_!#$%&'*+/=?`{|}~^.-]+@[a-zA-Z0-9.-]+$");
   Matcher matcher = pattern.matcher(email);
   if (!matcher.find())
      throw new ResponseStatusException(HttpStatus.CONFLICT,
            "El correo suministrado no tiene un formato válido");
}
```

Figura 334. Nuevas líneas en register *para comprobar el correo electrónico*

Nos queda comprobar que las dos contraseñas son iguales y que, por ejemplo, tienen como mínimo 5 caracteres: si no se da algunas de estas condiciones, lanzaremos también una excepción con el mismo código (403) y el mensaje que corresponda.

```
@PostMapping("/register")
public void register(@RequestBody Map<String, Object> info) {
   ...

   if (!pwd1.equals(pwd2))
      throw new ResponseStatusException(HttpStatus.CONFLICT,
            "La contraseña y su confirmación no coinciden");

   if (pwd1.length()<=5)
      throw new ResponseStatusException(HttpStatus.CONFLICT,
            "La contraseña debe tener al menos cinco caracteres");
}
```

Figura 335. Comprobamos las contraseñas

Si no se ha lanzado ninguna excepción en la Figura 335 es porque el correo electrónico es válido, porque las contraseña y su confirmación son iguales y porque la contraseña tiene por lo menos 5 caracteres.

Fetén entonces: vamos a pedirle al servicio, de acuerdo con el mensaje 1.2 de la Figura 325, que registre a esta persona.

Por el diagrama de secuencia que acabamos de citar, y también por la arquitectura que se nos ha impuesto, sabemos que el controlador debe delegar en el servicio la acción real de registrar al usuario. En el mensaje 1.2 le pedimos al servicio *UserService* que ejecute su operación *register* con los tres parámetros (nombre, contraseña y correo electrónico) que ya hemos validado en el método *register* del controlador. Vamos a crear una clase *UserService* en el paquete *services* que contendrá un método *register* que recibirá esos tres parámetros. La llamada en el controlador implica los siguientes cambios, que resaltamos en la Figura 336:

1) "Descomentamos" la declaración del campo *userService,* que comentamos un poquito antes de la Figura 333 para poder mostrarla,
2) leemos el nombre del usuario del mapa *info,* que se inyectó con el JSON que no envió el cliente y
3) le decimos al servicio: «Oye, registra a Pepe, que tiene esta contraseña y este correo elctrónico».

```
package edu.uclm.esi.juegos.http;

...

@RestController
@RequestMapping("users")
@CrossOrigin(origins = "*")
public class UserController {

   @Autowired
   private UserService userService;

   @PostMapping("/register")
   public void register(@RequestBody Map<String, Object> info) {
      String email = info.get("email").toString().trim();
      ...
      String name = info.get("name").toString().trim();
      this.userService.register(name, pwd1, email);
   }
}
```

Figura 336. Delegamos al UserService *el registro del usuario*

1.3. Implementación del servicio

Igual que el *WordRaceService* que ya conocemos, el *UserService* será una clase que contendrá la anotación *@Service* de Spring, y que dispondrá de un método *register(String, String, String)* que, según el diagrama de secuencia de la Figura 325 (se reproduce en la Figura 337 el fragmento que nos interesa), le pedirá al *userDAO* que guarde al usuario.

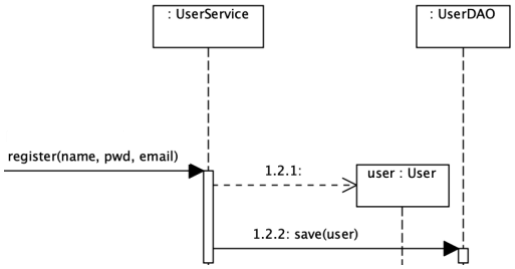


Figura 337. userService *crea una instancia de* User *y la almacena mediante el repositorio*

En lugar de copiar y pegar un fichero (como hemos hecho antes con el controlador), crearemos el servicio desde cero, pero en cuatro cómodos plazos:

1) Dependiendo de nuestro ordenador o sistema operativo, pulsamos *Control+1* o *Cmd+1* para que Eclipse nos proponga soluciones al problema que encuentra, que es que no conoce la clase *UserService*. También podemos pulsar el aspa roja que nos aparece al margen ( ). En cualquier caso, Eclipse nos propone varias soluciones y optamos por la primera: crearla.
2) Nos pregunta que dónde la queremos crear. Dónde vamos a querer: es un servicio, pues en el paquete *services*.
3) Eclipse nos genera un poco de código para esta nueva clase. Escribimos la anotación *@Service* y nos dice que no la conoce: pobre ignorante, impórtala, anda, de *org.springframework.stereotype.Service*.
4) Finalmente, en *UserController* tenemos un error de compilación que nos indica que, en *UserService,* no existe un método con la signatura *register(String, String, String):* es verdad, Eclipse, créanoslo.

Figura 338. Creación desde cero del UserService *y de su método* register

En este punto tenemos el *UserService* con su método *register* creado, que carece de momento de implementación. Según el diagrama de la Figura 337, debemos crear una instancia de *User* y, después, guardarla en la base de datos mediante el repositorio. Aunque no tengamos todavía ni la entidad ni el repositorio, completamos la clase *UserService* dejándola como en la Figura 339.

```java
package edu.uclm.esi.juegos.services;

import org.springframework.stereotype.Service;

@Service
public class UserService {

   @Autowired
   private UserDAO userDAO;

   public void register(String name, String pwd, String email) {
      User user = new User();
      user.setName(name);
      user.setPwd(pwd);
      user.setEmail(email);
      this.userDAO.save(user);
   }

}
```

Figura 339. Primera versión de register *en* UserService

1.4. Implementación de la entidad *User*

User será una *@Entity* que se corresponderá con la tabla *user* de SQL Server. Además de los tres campos *name, pwd* y *email,* le añadiremos un *id* de tipo *String* al que le asignaremos una cadena aleatoria de 36 caracteres que generaremos con *UUID.randomUUID.*

La contraseña, que hasta ahora estamos manejando en un formato legible, la guardaremos encriptada con el algoritmo SHA-512.

Además, indicaremos que los campos *name* y *email*, que se corresponderán con las columnas *name* y *email* de la tabla, serán índices únicos (o claves alternativas: sus valores no podrán repetirse).

El código de la Figura 340 incluye todas estas consideraciones:

1) Además de la anotación *@Entity,* se ha incluido también *@Table,* que es la que utilizamos para especificar los índices.
2) En el constructor asignamos la mencionada cadena aleatoria al campo *id*. Nótese, además, que el campo *id* no solo está anotado con *@Id,* sino también con *@Column:* usamos esta anotación para poder indicar que la columna asociada tendrá exactamente 36 caracteres y que se nos genere como un *varchar(36)*; en otro caso, se crearía como un *varchar(255).*
3) Por último, el método *setPwd* recibe la contraseña en texto plano, pero la encripta con el algoritmo mencionado. Cuando la contraseña se recupere con *getPwd,* la leeremos encriptada.

```java
package edu.uclm.esi.juegos.entities;

import java.util.UUID;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Index;
import jakarta.persistence.Table;

@Entity
@Table(indexes = {
   @Index(columnList = "name", unique = true),
   @Index(columnList = "email", unique = true)
})
public class User {
   @Id @Column(length = 36)
   private String id;
   private String name;
   private String pwd;
   private String email;

   public User() {
      this.id = UUID.randomUUID().toString();
   }

   // get y set para id, name y email


   public String getPwd() {
      return pwd;
   }

   public void setPwd(String pwd) {
      this.pwd = org.apache.commons.codec.digest.DigestUtils.sha512Hex(pwd);
   }
}
```

Figura 340. Código de la entidad

1.5. Implementación del repositorio *UserDAO*

El código del repositorio es muy sencillo: como recordamos de la implementación del *WordDAO* (Figura 130, página 100), el repositorio es una interfaz que extiende *JpaRepository*. En este caso, los dos tipos que pasamos como parámetros son *User* (la entidad que va a gestionar este repositorio) y *String* (el tipo del campo anotado con *@Id*).

```java
package edu.uclm.esi.juegos.dao;

import org.springframework.data.jpa.repository.JpaRepository;

import edu.uclm.esi.juegos.entities.User;

public interface UserDAO extends JpaRepository <User, String> {

}
```

Figura 341. Código del repositorio

1.6. Ejecución del caso de prueba *testRegister200*

Estamos casi listos para ejecutar el caso de prueba que teníamos en la Figura 326 (página 214). Decimos "casi" porque su última sentencia es un aserto que incluye una llamada al método *countByName* del repositorio. De acuerdo con las convenciones de nombrado de Spring (Cuadro V, página 104), esperamos que este método genere una sentencia del tipo *select count(*) from user where name=...*

Lo único que necesitamos es declarar ese método en el repositorio:

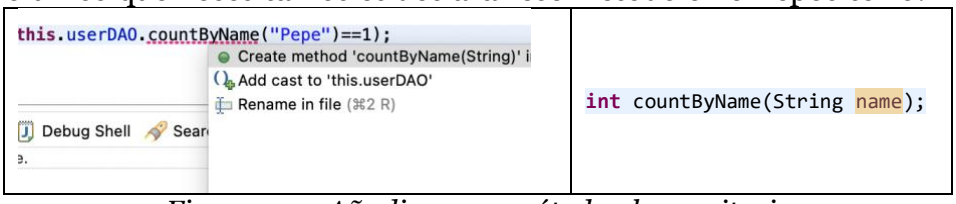


Figura 342. Añadimos un método al repositorio

Antes de lanzar el caso, hacemos un breve repaso de los prerrequisitos de ejecución:

1) Nos aseguramos de que el docker con SQL Server está corriendo.
2) Como la prueba la vamos a hacer con la base de datos de SQL Server, comprobamos que el fichero *application.properties* del área de tests (en *src/test/java)* está desactivado: esto lo hicimos en la Figura 186 (página 133) cuando renombramos el fichero a *application.properties.old*.
3) En la base de datos no disponemos de la tabla *user*, y evidentemente la necesitamos.
   Podemos crearla directamente nosotros en SQL Server, pero también podemos conseguir Spring procese la entidad cuando la aplicación arranque, de manera que genere la instrucción DDL tipo *create table user…* Para esto, en el fichero *application.properties* que estamos utilizando (y que es el de la rama de producción, *src/main/java),* establecemos el valor de la variable *spring.jpa.hibernate.ddl-auto* a *true*.
4) Por último, podemos descomentar las dos líneas de *application.properties* que habíamos comentado en la Figura 184 (página 131), de manera que podamos ver por la consola de Eclipse el código SQL que se vaya generando.

Inesperadamente, Spring interrumpe la ejecución de la aplicación cuando lanzamos la ejecución del caso, y la consola muestra una larguísima excepción. En algún lugar vemos la instrucción DDL que se ha generado para crear la tabla (Figura 343, izquierda), que parece correcta; un poco más abajo aparece un mensaje de error genérico (*Error executing DDL via JDBC Statement*) y, bastantes líneas después, uno que nos da algo más de detalle (*Incorrect syntax near the keyword 'user'*).

Qué pasará, qué misterio habrá, canta Raphael en *Mi gran noche,* una canción que es realmente la versión en castellano de *Tenez-vous bien*, de Salvatore Adamo[11]. Vamos a investigarlo abriendo Azure Data Studio, conectando a la base de datos y pegando el código DDL en una nueva *query*. Aparece el mismo mensaje de error: *Incorrect syntax near the keyword 'user'*. En el lado derecho de la Figura 343 aparece la sentencia y el resultado de la ejecución. El hecho de que, en la parte de arriba, SQL Server nos subraye el nombre de la tabla (*user)* nos da una pista: *user* es una palabra reservada de SQL Server.

[11] https://www.youtube.com/watch?v=CJF9pjdxiHY

```
create table user (
    id varchar(36) not null,
    email varchar(255),
    name varchar(255),
    pwd varchar(255),
    primary key (id)
)
```

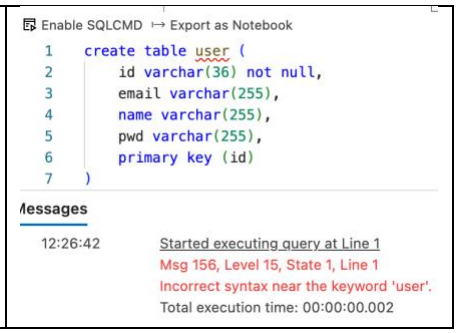


Figura 343. Instrucción DDL generada para crear la tabla user, *y el* error que da SQL Server

Como ya sabemos, Spring se ha basado en el nombre de la entidad para asignar el nombre a la tabla. Una solución burda es cambiar el nombre a la entidad y llamarla, por ejemplo, *Users* (en plural), pero una instancia de esta clase no representa, como se deriva de este nombre, una colección de usuarios. Necesitamos hacer un pequeño cambio en la entidad, pero de otra índole: le diremos a Spring que, aunque la entidad se llame *user*, la tabla ha de llamarse *users* (Figura 344).

```
@Entity
@Table(
    name = "users",
    indexes = {
       @Index(columnList = "name", unique = true),
       @Index(columnList = "email", unique = true)
    }
)
public class User {
    @Id @Column(length = 36)
    private String id;
    ...
```

Figura 344. Asignamos un nombre distinto a la tabla asociada a esta entidad

Ahora sí, JUnit ejecuta el caso de prueba, que emite veredicto de paso. Respecto del método *countByName*, en la consola podemos la instrucción que se ha generado: *select count(usero_.id) as col_0_0_ from users usero_ where usero_.name=?* y, justo en la siguiente línea, el valor que ha asignado al parámetro (la interrogación) de la consulta: *binding parameter [1] as [VARCHAR] - [Pepe].*

1.7. Reejecución de *testRegister200*

Si lo ejecutamos de nuevo, ya no se genera la instrucción DDL y no se crea la tabla, pues al haber especificado el valor *update* en *application.properties,* Spring compara la estructura de la base de datos con las entidades definidas: al encontrar la tabla *users,* omite su creación.

Sin embargo, el caso de prueba falla ahora. La consola de JUnit nos da pistas del error: *ConstraintViolationException: could not execute statement* y, más abajo: *Violation of UNIQUE KEY constraint, insert duplicate key in object 'dbo.users'. The duplicate key value is (Pepe).*

Claro: estamos usando la base de datos de SQL Server, no una en memoria, con lo que Pepe sigue estando presente y no podemos volver a insertarlo.

Podemos hacer dos cosas: o bien borrar todos los datos de la tabla *users* antes de arrancar el caso, o borrarlo después. Lo borraremos antes creando un *setUp* que anotamos con *@BeforeAll* que llame al *deleteAll* del *userDAO.*

Recuérdese (Figura 291, página 188) que debemos anotar la clase con *@TestInstance(Lifecycle.PER_CLASS)*.

1.8. Prueba de escenarios alternativos

Para el caso de uso *register*, algunos escenarios alternativos son, por ejemplo, los intentos de registro de:

1) Un usuario que envía una contraseña demasiado corta.
2) Otro cuyas contraseña y confirmación no coinciden.
3) Un correo electrónico que no es válido.
4) Otro usuario llamado que también se llama Pepe.
5) Otro usuario que introduce también pepe@pepe.com como correo.

Con los casos de prueba describimos el comportamiento del sistema, tanto en condiciones de normalidad como de error. Podemos suponer que, en todas las situaciones enumeradas arriba, el sistema debe devolver un 409 (conflicto) como código de respuesta.

1.8.1. Contraseña corta o no coincidentes

Probaremos estas dos situaciones con el caso de prueba *testRegister409* (Figura 345, izquierda):

1) En la primera parte construimos a la usuaria Ana, que introduce "ana1" como contraseña.
2) En la segunda parte, sustituimos los campos *pwd1* y *pwd2* por dos contraseñas parecidas, pero no coincidentes.

Debemos construir dos peticiones; en ambos casos esperamos que el sistema lance devuelva en la respuesta un código 409, por lo que en el oráculo del caso de prueba describimos exactamente esta situación. Nótese, además, el valor de *@DisplayName* con el que hemos anotado el caso de prueba: JUnit lo utiliza (lado derecho de la figura) para etiquetarlo al mostrar los resultados.

```
@Test @DisplayName("Contraseña corta o no coincidentes")
void testRegister409() throws Exception {
     JSONObject jso = new JSONObject().
           put("name", "Ana").
           put("pwd1", "ana1").
           put("pwd2", "ana1").
           put("email", "ana@ana.com");

     RequestBuilder request = MockMvcRequestBuilders.
       post("/users/register").
       contentType("application/json").
       content(jso.toString());

     this.server.perform(request).
       andExpect(status().isConflict());


     jso.put("pwd1", "ana1234").
       put("pwd2", "Ana1234");

     request = MockMvcRequestBuilders.
           post("/users/register").
           contentType("application/json").
           content(jso.toString());

     this.server.perform(request).
       andExpect(status().isConflict());
}
```

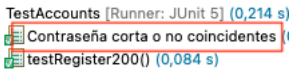


Figura 345. Un caso de prueba y su resultado

1.8.2. Correo electrónico inválido. Parametrización de casos de prueba

La tercera situación debe también devolver un código 409, pues esta es la respuesta que le dijimos al controlador que lanzase cuando la dirección de correo recibida no case con la expresión regular (Figura 334, página 218).

En lugar de escribir un nuevo caso para esta situación, que prácticamente consistiría en copiar y pegar el anterior, podemos crear un caso de prueba parametrizado con los tres objetos que debe enviar al servidor: un usuario con la contraseña corta, otro con las contraseñas no coincidente y otro con un correo inválido.

El siguiente caso resume las tres situaciones de prueba indicadas:

1) El método ya no tiene la anotación *@Test,* sino *@ParameterizedTest*.
2) Se le ha añadido la anotación *@CsvSource*, que toma como parámetro una lista de cadenas. Cada cadena incluye, separados por coma (puede utilizarse otro delimitador, que debe especificarse con el campo *delimiter* de esta anotación), los valores que JUnit inyectará a los parámetros del método. Así, este caso se ejecutará tantas veces como líneas haya.

```
@ParameterizedTest
@CsvSource({
    "Ana, ana1, ana1, ana@ana.com",
    "Ana, ana1234, Ana1234, ana@ana.com",
    "Ana, ana1234, ana1234, ana#ana.com",
})
@DisplayName("Contraseñas mal o email inválido")
void testRegister409(String name, String pwd1, String pwd2, String email) throws Exception {
    JSONObject jso = new JSONObject().
            put("name", name).
            put("pwd1", pwd1).
            put("pwd2", pwd2).
            put("email", email);

    RequestBuilder request = MockMvcRequestBuilders.
        post("/users/register").
        contentType("application/json").
        content(jso.toString());

    this.server.perform(request).
        andExpect(status().isConflict());
}
```

Figura 346. Caso de prueba parametrizado

Cuando los casos se ejecutan, el arbolito de JUnit muestra las tres ejecuciones de este caso de prueba parametrizado, informando de los valores inyectados en cada una:

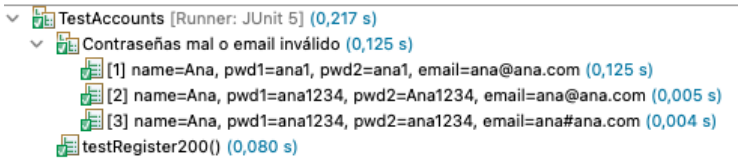


Figura 347. Resultados del caso de prueba parametrizado

1.8.3. Errores de base de datos

Las situaciones 4 y 5 intentan insertar en la tabla de usuarios valores duplicados en alguna clave alternativa. Con la configuración actual, SQL Server lanzará una excepción que, si no capturamos, devolverá al cliente un error 500 o de la serie 5.

Es "feo" que un error 5XX se propague hasta el cliente, porque en muchas ocasiones representa una situación que el programador no ha previsto. Además,

la excepción lleva consigo un mensaje que informa del tipo de error que se ha producido, lo que puede permitir al usuario del cliente conocer la tecnología que estamos utilizando y facilitarle posibles ataques.

Por ello, intentaremos siempre evitar el envío de errores de esta serie, y los enmascararemos en errores de la serie 4.

Como esperamos también obtener un código de conflicto, añadimos las dos siguientes líneas al *@CsvSource* del caso anterior:

1) *"Pepe, pepe12345, pepe12345, jose@jose.com"*: esta línea representa a un segundo usuario Pepe que quiere registrarse.
2) *"Jose, pepe12345, pepe12345, pepe@pepe.com"*: esta línea representa a un usuario que se quiere registrar con la dirección de correo del Pepe original.

Esperamos en ambos casos que al cliente (al caso de prueba, en este caso) le llegue el código 409, lo que exige que antes de ejecutar este caso se haya ejecutado el primero: anotamos con *@Order(1)* el método *testRegister200*, con *@Order(2)* el *testRegister409* y la clase con *@TestMethodOrder(OrderAnnotation.class)*.

Cuando ejecutamos, se superan las tres primeras iteraciones del caso, pero no las dos últimas:

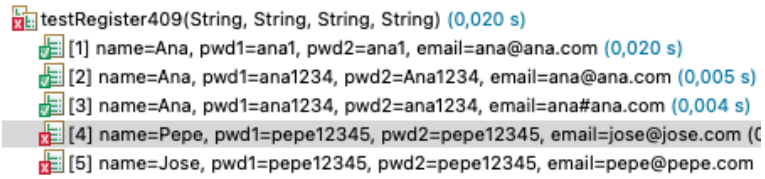


Figura 348. Hay error en dos ejecuciones

Si miramos la traza del error en la consola de JUnit, comprobamos que llega a verse que se ha producido una *SQLServerException*, lo que da indudables pistas sobre el gestor de bases de datos que tenemos.

La excepción de SQL Server se lanza exactamente cuando el método *register* del *userService* llama a *userDAO.save*. Dejaremos que la excepción se propague hasta el controlador, en donde la capturaremos para devolver al cliente la respuesta deseada.

```
@PostMapping("/register")
public void register(@RequestBody Map<String, Object> info) {
   String email = info.get("email").toString().trim();
   ...
   String name = info.get("name").toString().trim();
   try {
      this.userService.register(name, pwd1, email);
   } catch (Exception e) {
      System.out.println(e);
   }
}
```

Figura 349. Pequeño cambio en el recurso register *del* UserController

De momento, simplemente hemos colocado la llamada a *this.userService.register* en un bloque *try catch*. Vamos a ejecutar los casos con el depurador, poniendo un punto de interrupción en el *System.out.print* del bloque *catch*.

Al ejecutar la cuarta iteración de *testRegister409*, el depurador se detiene en ese punto. En la parte de variables de la Figura 350 tenemos mucha información acerca de esta excepción:

1) El objeto *e* (que es la excepción que se ha producido) es de tipo *DataIntegrityViolationException*. Si buscamos este nombre en Internet, vemos enseguida que es una excepción de Spring.
2) *e* tiene una *cause,* que es una *ConstraintViolationException*.
3) A su vez, esta *cause* tiene otra *cause,* que ahora es de tipo *SQLServerException*.
4) Esta segunda *cause* tiene un campo *vendorCode,* que representa el código de error emitido por el sistema gestor de bases de datos que estamos utilizando. Si buscamos en Internet con la cadena "SQL Server error codes", vemos enseguida que se refiere a un intento de inserción de un valor duplicado en un lugar en el que no se puede.

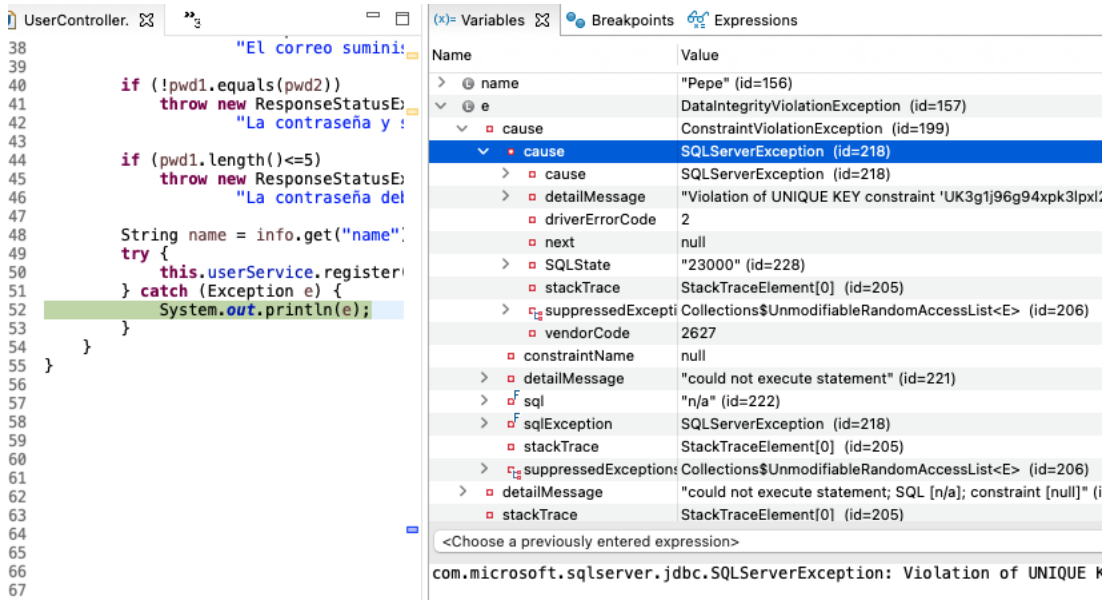


Figura 350. Detenemos la ejecución e inspeccionamos las variables

La excepción que hemos capturado es, como decimos, del tipo *DataIntegrityViolationException* que, en la figura siguiente, vemos que es una especialización de *DataAccessException*. Esta es una *RuntimeException* que, como vimos en el Cuadro VII (página 112), no estamos obligados a capturar.

Las tres excepciones de Spring que se muestran en la figura tienen un constructor en el que se puede pasar un *Throwable,* que es la causa de que se haya producido la excepción. En nuestro caso particular, la causa de la *DataIntegrityViolationException* que capturamos en el punto de interrupción y que se muestra en la zona de variables con el depurador es una *SQLServerException,* que es una especialización del tipo genérico de Java *java.sql.SQLException*. Como vemos, la *SQLException* tiene un método *getErrorCode* que devuelve el código de error que proporciona el vendedor.

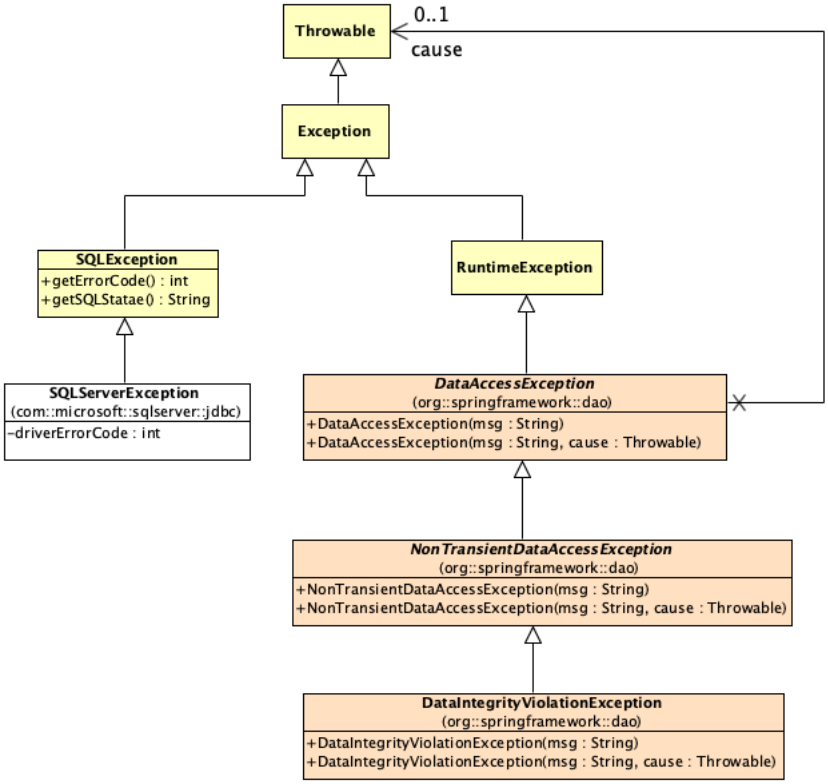


Figura 351. Parte de la jerarquía de excepciones de acceso a datos

Para capturar adecuadamente la excepción y enviar al cliente una respuesta de la serie 4, capturaremos la excepción como una *DataAccessException* que, como vemos en la figura anterior, es la más alta en la jerarquía de excepciones de acceso a datos de Spring.

Para conocer el error que ha lanzado el gestor de la base de datos, leeremos la causa de la causa y, de aquí, el código de error. De momento, entonces, hacemos el siguiente cambio en el código del recurso web:

```
@PostMapping("/register")
public void register(@RequestBody Map<String, Object> info) {
   String email = info.get("email").toString().trim();
   ...
   try {
      this.userService.register(name, pwd1, email);
   } catch (DataAccessException e) {
      SQLException cause = (SQLException) e.getCause().getCause();


      System.out.println(cause.getMessage());
      System.out.println(cause.getErrorCode());



      throw new ResponseStatusException(HttpStatus.CONFLICT,
            cause.getMessage());
   }
}
```

Leemos la causa de la causa

Sacamos por consola el mensaje de error y su código

Devolvemos el 409

Figura 352. Nos vamos acercando al correcto tratamiento de la excepción

Para las dos iteraciones que fallan de *testRegister409,* en la consola obtenemos lo siguiente:

```
Violation of UNIQUE KEY constraint 'UK3g1j96g94xpk3lpxl2qbl985x'. Cannot insert
duplicate key in object 'dbo.users'. The duplicate key value is (Pepe).
2627
Violation of UNIQUE KEY constraint 'UK6dotkott2kjsp8vw4d0m25fb7'. Cannot insert
duplicate key in object 'dbo.users'. The duplicate key value is (pepe@pepe.com).
2627
```

Figura 353. Mensajes y códigos de error

Con esto tenemos bastante más información para enviar al cliente un texto de respuesta errónea más afinado. Lamentablemente, los nombres de las restricciones violadas no son demasiado significativos: *UK3g1…* Vamos a cambiarlos:

1) Vamos a la entidad *User* y, a las dos definiciones de los índices, les asignamos valor al campo *name:*

```
@Index(columnList = "name", unique = true, name = "unique_name"),
@Index(columnList = "email", unique = true, name = "unique_email")
```

Figura 354. Asignamos valores a los índices

2) Reejecutamos el caso de prueba: esperamos que, puesto que tenemos el valor *update* en la propiedad *spring.jpa.hibernate.ddl-auto* de *application.properties,* Spring detecte que los nombres de estas restricciones han cambiado y lance una nueva instrucción DDL para actualizarlos.
3) Como no es así (Spring no detecta el cambio), vamos directamente a Azure Data Studio, eliminamos la tabla con *drop table users* y volvemos a lanzar los casos. Ahora, la tabla se crea de nuevo y se asignan los dos nombres indicados a las dos restricciones de integridad.

En la consola de Eclipse vemos ahora los siguientes resultados:

```
Violation of UNIQUE KEY constraint 'unique_name'. Cannot insert duplicate key in
object 'dbo.users'. The duplicate key value is (Pepe).
2627
Violation of UNIQUE KEY constraint 'unique_email'. Cannot insert duplicate key in
object 'dbo.users'. The duplicate key value is (pepe@pepe.com).
2627
```

Figura 355. Los mensajes de error son ahora más significativos

Si al bloque *catch* le damos la redacción de la Figura 356, podremos conocer tanto el código de error como el nombre de la restricción que se intentaba violar:

```
catch (DataAccessException e) {
   SQLException cause = (SQLException) e.getCause().getCause();
   int errorCode = cause.getErrorCode();
   String message = cause.getMessage();
   String constraint = message.substring(message.indexOf('\'') + 1, message.indexOf('.') - 1);
   System.out.println(constraint);
   throw new ResponseStatusException(HttpStatus.CONFLICT, cause.getMessage());
}
```

Figura 356. Leemos el código de error y el nombre de la restricción

Casi finalmente, lanzamos un mensaje adecuado a la restricción:

```
catch (DataAccessException e) {
   SQLException cause = (SQLException) e.getCause().getCause();
   int errorCode = cause.getErrorCode();
   String message = cause.getMessage();
   String constraint = message.substring(message.indexOf('\'') + 1, message.indexOf('.') -1);
   if (constraint.equals("unique_name"))
      message = "Ese nombre de usuario ya está registrado";
   else if (constraint.equals("unique_email"))
      message = "Ese correo electrónico ya está siendo utilizado";
   else
      message = "Se ha producido un error inesperado";
   throw new ResponseStatusException(HttpStatus.CONFLICT, message);
}
```

Figura 357. Enviamos un mensaje adecuado al cliente

Volvamos ahora al caso de prueba, pues queremos reforzarlo un poco más para que compruebe no solamente el código de respuesta, sino también el texto del mensaje de error. Para las cinco situaciones que estamos probando, hemos programado nuestro código para que devuelva los mensajes de error de la Figura 358.

Combinación de datos	Mensaje esperado
"Ana, ana1, ana1, ana@ana.com"	La contraseña debe tener al menos cinco caracteres
"Ana, ana1234, Ana1234, ana@ana.com"	La contraseña y su confirmación no coinciden
"Ana, ana1234, ana1234, ana#ana.com"	El correo suministrado no tiene un formato válido
"Pepe, pepe12345, pepe12345, jose@jose.com"	Ese nombre de usuario ya está registrado
"Jose, pepe12345, pepe12345, pepe@pepe.com"	Ese correo electrónico ya está siendo utilizado

Figura 358. Mensajes de error esperados para las cinco situaciones alternativas

A diferencia de los casos de prueba que elaboramos para el servicio *selectWord* del *Word race* en el Capítulo 5, en los que el texto que buscábamos venía en el *body* y lo recogíamos con *getContentAsString* (Figura 144, página 109), el mensaje de error no viene en el *body,* sino en la línea de estado de la *response* (véase la estructura de una *response* en el Cuadro I, página 84).

En el siguiente código, el caso de prueba:

1) Incluye, en su *@CsvSource,* el mensaje de error que se espera que devuelva el *backend* con cada combinación de datos.
2) Inyecta ese mensaje de error esperado en el nuevo parámetro *expectedMessage*.
3) Recupera el mensaje obtenido, que es el que devuelve el servidor, de la línea de estado de la respuesta, y comprueba también que el código es de conflicto.
4) Incluye un aserto para comparar el mensaje esperado con el obtenido.

```
@ParameterizedTest @Order(2)
@CsvSource({
    "Ana, ana1, ana1, ana@ana.com, La contraseña debe tener al menos cinco caracteres",
    "Ana, ana1234, Ana1234, ana@ana.com, La contraseña y su confirmación no coinciden",
    "Ana, ana1234, ana1234, ana#ana.com, El correo suministrado no tiene un formato válido",
    "Pepe, pepe12345, pepe12345, jose@jose.com, Ese nombre de usuario ya está registrado",
    "Jose, pepe12345, pepe12345, pepe@pepe.com, Ese correo electrónico ya está siendo utilizado"
})
void testRegister409(String name, String pwd1, String pwd2, String email, String
expectedMessage) throws Exception {
    JSONObject jso = new JSONObject().
          put("name", name).
          put("pwd1", pwd1).
          put("pwd2", pwd2).
          put("email", email);

    RequestBuilder request = MockMvcRequestBuilders.
       post("/users/register").
       contentType("application/json").
       content(jso.toString());

    String obtainedMessage = this.server.perform(request).
       andExpect(status().isConflict()).
       andReturn().getResponse().getErrorMessage();
    assertEquals(expectedMessage, obtainedMessage);
}
```

Figura 359. Código final del caso testRegister409

2. Caso de uso *login*

El siguiente caso de prueba (Figura 360) parametrizado describe tres escenarios que deben devolver, respectivamente, un código 200 para Pepe con su contraseña correcta, y 403 (*forbidden,* prohibido) para Pepe con una contraseña incorrecta y para Ana, que no ha llegado a registrarse:

1) Como Pepe ha de estar registrado para que pueda loguearse, lo anotamos con *@Order(3)*.
2) Colocamos en su *@CsvSource* el código de respuesta esperado, que inyectamos en *expectedCode*.
3) En el *jso,* que representa la *payload* que se va a enviar al servidor, colocamos solo los campos *name* y *pwd,* que serán los que recojamos en el controlador.
4) Enviamos la petición con el método PUT.
5) En el oráculo, usamos el método genérico *is(int)* para comparar con el *expectedCode*.

```
@ParameterizedTest @Order(3)
@CsvSource({
    "Pepe, pepe1234, 200",
    "Pepe, pepe12345, 403",
    "Ana, pepe1234, 403"
})
void testLogin(String name, String pwd, int expectedCode) throws Exception {
    JSONObject jso = new JSONObject().
          put("name", name).
          put("pwd", pwd);

    RequestBuilder request = MockMvcRequestBuilders.
       put("/users/login").
       contentType("application/json").
       content(jso.toString());

    this.server.perform(request).
       andExpect(status().is(expectedCode));
}
```

Figura 360. Probamos tres escenarios de login: *uno normal y dos alternativos*

Habiendo ya desarrollado el caso de uso *register,* el desarrollo de este es casi trivial, si bien va a tener algunas novedades interesantes que veremos en la sección 4 de este capítulo. En principio, el recurso *login* en el controlador puede ser tan sencillo como este:

1) Como hemos dicho, se invoca a este servicio con *put,* por lo que lo anotamos como *@PutMapping*.
2) Igual que *register*, recibe una *payload* en formato JSON que Spring puede inyectarnos en un mapa.
3) Recuperamos los dos campos del JSON que hemos recibido.
4) Invocamos al método del *UserService,* que nos devuelve una instancia de clase *User* si es que lo encuentra. Si no lo encuentra, nos devuelve *null* y devolvemos la respuesta con código 403.

```
@PutMapping("/login")
public void login(@RequestBody Map<String, Object> info) {
   String name = info.get("name").toString().trim();
   String pwd = info.get("pwd").toString().trim();

   User user = this.userService.login(name, pwd);
   if (user==null)
      throw new ResponseStatusException(HttpStatus.FORBIDDEN,
            "Credenciales inválidas");
}
```

Figura 361. Código de login *en el controlador*

El servicio debe solamente pedirle al repositorio que le busque un usuario cuyo *name* es el que pasamos como parámetro, y cuya contraseña es el valor encriptado con SHA-512 (recuérdese la Figura 340, página 222) del parámetro *pwd* que también pasamos:

```
public User login(String name, String pwd) {
   pwd = org.apache.commons.codec.digest.DigestUtils.sha512Hex(pwd);
   return this.userDAO.findByNameAndPwd(name, pwd);
}
```

Figura 362. Código de login, *ahora en el* UserService

Obviamente, el método *findByNameAndPwd* que se usa en la Figura 362 no existe en el repositorio, pero sí que verifica las convenciones de nombrado de los métodos de persistencia de Spring: en efecto, *name* y *pwd* son dos campos de la entidad, con lo que se generará una instrucción SQL del tipo *select * from users where name=... and pwd=...*

Basta entonces con que añadamos la declaración de este método a la interfaz:

```
public interface UserDAO extends JpaRepository <User, String> {

   int countByName(String name);

   User findByNameAndPwd(String name, String pwd);
}
```

Figura 363. Añadimos un método al repositorio UserDAO

3. Caso de uso *removeUser* (I)

Aunque no lo tenemos en nuestros diagramas de casos de uso, es muy posible que queramos ofrecer un servicio que permita a un usuario eliminar su cuenta. Supondremos que esta funcionalidad solo pueden ejecutarla los usuarios que están logueados.

En este caso de prueba, que ejecutamos en cuarto lugar, enviamos dos peticiones: una para loguear a Pepe y otra para eliminar su cuenta. Nótese que la segunda la enviamos con *delete* al servicio *removeUser* del controlador: como se ve, no lleva ningún parámetro, pero esperamos que elimine a Pepe, que se acaba de loguear. En la última línea comprobamos que no queda rastro de Pepe en la base de datos.

```
@Test @Order(4)
void testDelete() throws Exception {
   JSONObject jso = new JSONObject().
         put("name", "pepe").
         put("pwd", "pepe1234");

   RequestBuilder request = MockMvcRequestBuilders.
      put("/users/login").
      contentType("application/json").
      content(jso.toString());
   this.server.perform(request);

   request = MockMvcRequestBuilders.
         delete("/users/removeUser");

   this.server.perform(request).
      andExpect(status().isOk());

   assertTrue(this.userDAO.countByName("Pepe")==0);
}
```

Figura 364. Enviamos dos peticiones: una para loguear y otra para eliminar

En el controlador, el servicio web *removeUser* lo anotaremos con *@DeleteMapping("/removeUser")*. De alguna manera, debe recuperar el usuario que está conectado en esta sesión de navegación para poder eliminarlo. Para esto, tenemos que haberlo colocado en dicha sesión, por ejemplo, cuando se loguea.

4. Modificación del caso de uso *login*

Cada vez que nos conectamos a un sitio web, el servidor crea un objeto de tipo *HttpSession* al que asigna un identificador único. En las respuestas que el servidor envía al cliente, aquel incluye una cabecera con este identificador de sesión.

En la Figura 365, el usuario se ha conectado a la página principal de la Universidad de Castilla-La Mancha. En el recuadro rojo se muestran las cabeceras que el servidor ha enviado al navegador cuando ha recibido la primera petición: como vemos, hay una llamada *Set-Cookie* en la que ha colocado una cookie con nombre *ASP.NET_SessionId* a la que ha asignado un valor que empieza por "rm2".

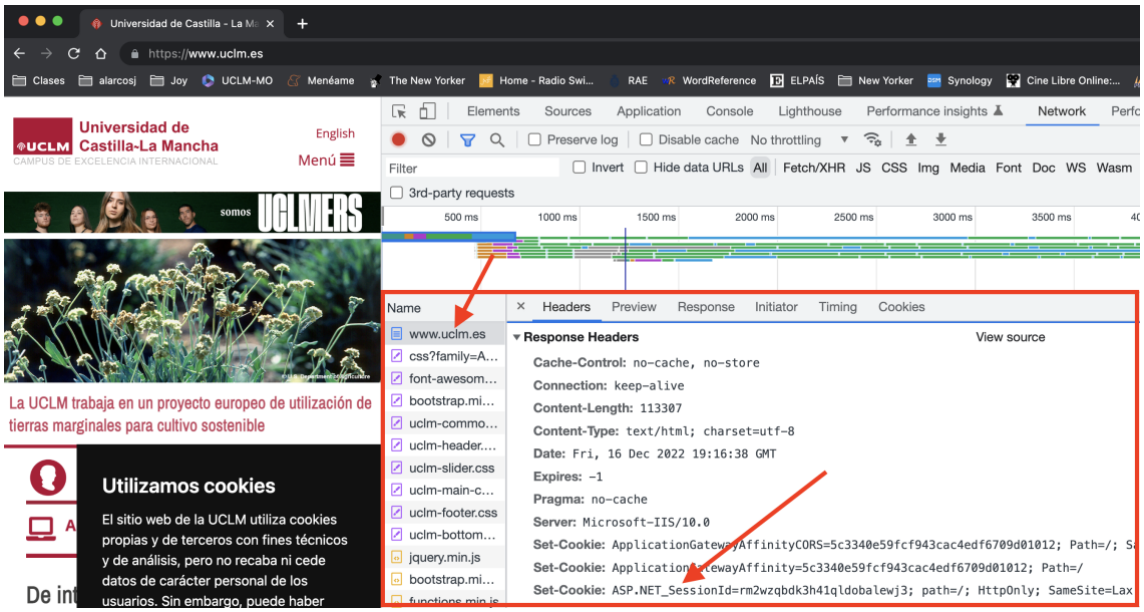


Figura 365. https://www.uclm.es nos devuelve una cookie de sesión

En las subsiguientes peticiones que el navegador envíe a este mismo servidor durante esta sesión de navegación, el navegador incluirá el valor de esta cookie en su sección de cabeceras: es lo que se ilustra en la Figura 366, que muestra no las *Response headers* (como en la anterior), sino las *Request headers*: como se ve, cuando el navegador solicita que se le entregue el recurso *font-awesome.min.css,* que va dirigida al mismo servidor, le incluye el mismo valor de la cookie que el servidor le entregó la primera vez.

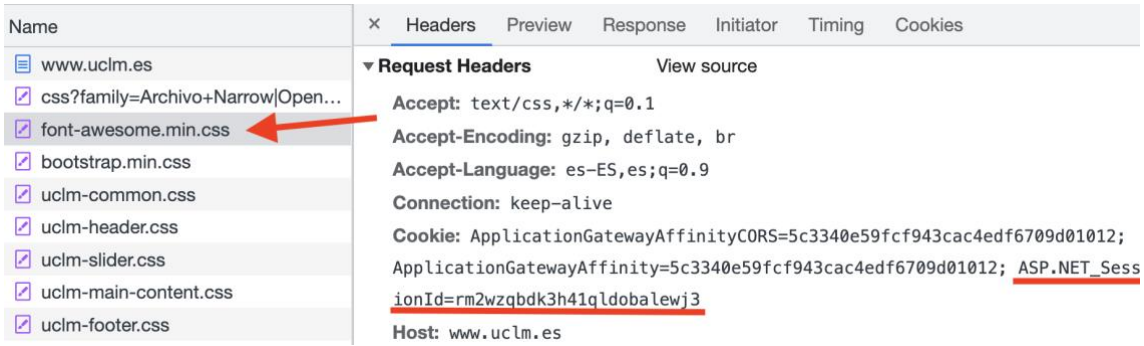


Figura 366. El id de la sesión se reenvía en todas las peticiones

La cookie, sin embargo, no se ha enviado en la segunda petición (*css?family=...),* pues ha ido dirigida a un servidor distinto del origen (se explicó el concepto de "origen" en la sección 4.1 del Capítulo 8, página 145):

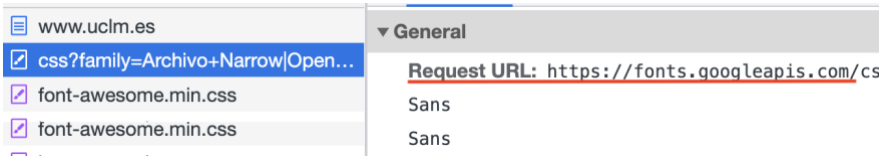


Figura 367. En general, las cookies no se envían a servidores distintos del origen

Como el cliente siempre envía al servidor la cookie de sesión que este le haya dejado previamente, el servidor utiliza este id para conocer que dos peticiones sucesivas proceden del mismo cliente.

El servidor, además, puede colocar información asociada a cada sesión. En nuestro caso, cuando el usuario se loguee, colocaremos en su sesión asociada alguna información que le identifique.

4.1. Inyección de la sesión en los servicios web

Si en un método que implementa un servicio web declaramos un parámetro de tipo *jakarta.servlet.http.HttpSession* (ojo, porque con versiones anteriores de Spring Boot, el tipo comenzaría por *javax* en lugar de por *jakarta*), Spring le inyectará un objeto con la información de la sesión asociada. Como hemos dicho en el párrafo anterior, el servidor puede colocar objetos asociados a esta sesión de navegación.

Esto es exactamente lo que hacemos en esta nueva implementación del *login* que tenemos en el controlador (Figura 368):

1) Declaramos el parámetro *session* de tipo *HtttpSession,* con lo que Spring le inyecta directamente el valor del objeto.
2) En la última línea, a la que llegamos solo cuando no se ha lanzado la excepción de "Credenciales inválidas", colocamos en la sesión el identificador del usuario que se acaba de loguear.

```
@PutMapping("/login")
public void login(HttpSession session, @RequestBody Map<String, Object> info) {
    String name = info.get("name").toString().trim();
    String pwd = info.get("pwd").toString().trim();

    User user = this.userService.login(name, pwd);
    if (user==null)
        throw new ResponseStatusException(HttpStatus.FORBIDDEN,
                "Credenciales inválidas");
    session.setAttribute("userId", user.getId());
}
```

Figura 368. Colocamos el id del usuario en la sesión cuando el logueo es exitoso

4.1.1. La interfaz *HttpSession*

Realmente, en la sesión se puede colocar cualquier cosa. Podríamos haber puesto, en lugar del *id* del *user*, el *user* completo si hubiésemos escrito *session.setAttribute("user", user)*. Ahora el tipo *User* es pequeño (en el sentido de que tiene 3 o 4 campos, todos ellos *String*), pero es posible que su estructura crezca en un futuro (imaginemos, por ejemplo, que de cada *User* se guardasen su lista de partidas con sus movimientos). Como el servidor crea una sesión por cada navegador conectado (técnicamente, crea una *HttpSession* por cada sesión de navegación existente), es relativamente peligroso almacenar en la sesión objetos potencialmente grandes, pues el servidor podría "reventar" en algún momento.

No hay que olvidar, además, que nuestro *backend* corre en un servidor de aplicaciones (tipo Tomcat o JBoss, por ejemplo, o un Internet Information Server en el caso de la UCLM: lo sabemos por el nombre de la cookie de sesión, "*ASP.NET_SessionId*"), cuya forma de gestionar las múltiples sesiones concurrentes que tal vez esté soportando desconocemos en gran medida. Algunos servidores de aplicaciones hacen, cuando lo necesitan, una descarga de las sesiones menos activas desde la memoria principal hacia el disco: cuando una sesión inactiva vuelve a memoria principal, es posible que haya cierta información que se haya perdido. Por eso se aconseja que todos los objetos que se colocan en la sesión implementen la interfaz *java.io.Serializable*. Pero mejor curémonos en salud y coloquemos en cada sesión la menor cantidad posible de información. Por cierto, que el tipo *String* es *Serializable,* así que podemos colocar con confianza el *id* de cada *User*.

La estructura en Java de la *HttpSession* es la de la Figura 369. Cada fabricante de servidores de aplicaciones (Apache de Tomcat, JBoss de Red Hat, etcétera) puede implementar la interfaz como quiera (por ejemplo, con ese mecanismo de llevar y traer objetos a y desde memoria principal a secundaria que se ha descrito), pero debe ofrecer al menos las operaciones de la figura al desarrollador.

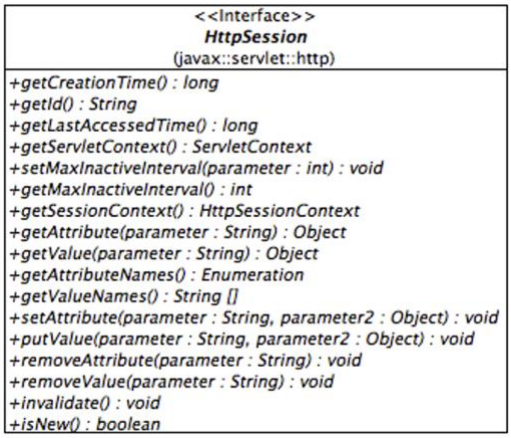


Figura 369. Operaciones de la HttpSession

Los métodos *setAttribute* y *getAttribute* nos invitan a pensar que, en sus implementaciones, la sesión contiene una tabla hash o similar en la que los objetos que se colocan se pueden recuperar por el nombre con el que se han indexado: es decir, si guardamos un dato escribiendo *session.setAttribute("userId", user.getId())*, en otro momento podremos recuperarlo escribiendo *getAttribute("userId")*. En efecto, como veremos en el siguiente epígrafe, exactamente esto es lo que sucede.

5. Caso de uso *removeUser* (II)

La implementación que hemos dado a *login* en la Figura 368 guarda en la sesión el *id* del usuario. Para eliminarlo cuando el usuario llame al servicio *removeUser*, debemos inyectar la sesión en este método y, luego, recuperar el campo *userId,* puesto que lo colocamos exactamente con este nombre cuando el usuario se logueó.

En el controlador tenemos entonces que escribir el código de este servicio web y de su método asociado en *UserService* (Figura 370). Este último solo llama al método *deleteById* del repositorio que no es necesario ni que implementemos ni que declaremos, pues es uno de los que nuestro *UserDAO* hereda de *CrudRepository* (Figura 130, página 100).

```
@DeleteMapping("/removeUser")
public void removeUser(HttpSession session) {
   String userId =
    session.getAttribute("userId").toString();
   this.userService.removeUser(userId);
}
```

```
public void removeUser(String userId) {
   this.userDAO.deleteById(userId);
}
```

Figura 370. removeUser *en el* UserController (arriba) y *en el* UserService

5.1. Ejecución del caso *testDelete*

Ya tenemos todo listo para que el caso *testDelete* de la Figura 364 (página 234) funcione; pero, cuando lo ejecutamos, peta y JUnit nos muestra un veredicto de error: *NullPointerException,* dice.

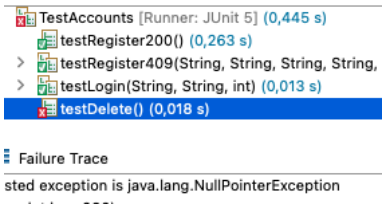


Figura 371. testDelete *falla, y creíamos que funcionaría*

El error está en que desde nuestro cliente, que actualmente es un mero fichero de tests, cada petición está llegando como si la enviase un cliente, o un navegador, distinto. En efecto, sin colocamos en *UserController* un punto de interrupción en la primera línea de *login* y otro en la primera línea de *removeUser,* podremos conocer si las dos peticiones que se están recibiendo proceden del mismo cliente.

Lanzamos los tests de *TestAccounts* con el depurador e inspeccionamos la variable *session* que se está inyectando en cada método al ejecutar *testDelete*. El resultado se muestra en la Figura 372, que puede parecer complicada, pero que el lector entenderá con la sencilla explicación que merece:

1) La primera columna simplemente indica que hemos colocado un punto de interrupción en la primera línea de los dos métodos que nos interesan en este momento.
2) Cuando se lanza *testDelete,* primero se hace un *login* y luego un *removeUser:*
    a. En la primera fila de la segunda columna, el depurador se ha detenido en la primera línea de *login*. En la zona de variables, vemos que existe un objeto *session,* que es el que Spring ha inyectado en el primer parámetro del método. El *id* de esta *session* es 4.
    b. Del mismo modo, el depurador se ha detenido en la primera línea del segundo método llamado por el caso de prueba (*removeUser)*. Spring también ha inyectado la sesión en el parámetro *session*: sin embargo, el *id* de esta *session* es 5, con lo que el servidor de aplicaciones no encuentra ningún objeto *userId* almacenado en ella.

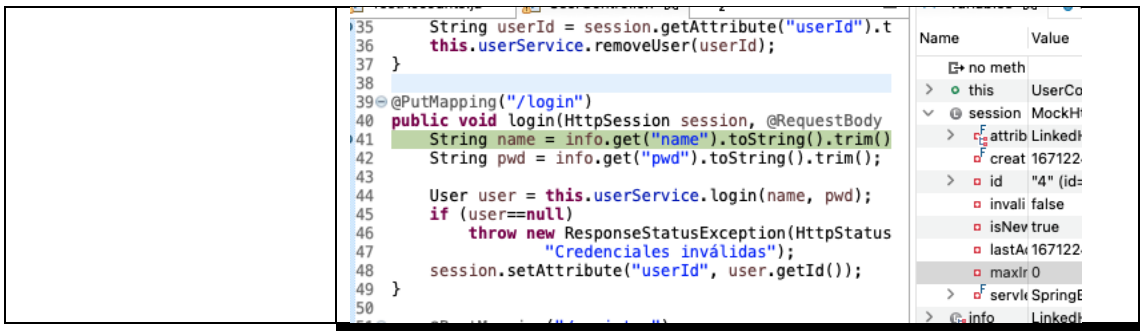


Figura 372. Depuramos el código de producción que ejecuta el caso testDelete

El problema es muy sencillo de corregir: lo único que necesitamos, en el caso de prueba, es decirle a la segunda petición que se envíe con la misma sesión que la primera.

La implementación del nuevo *testDelete* emite ahora veredicto de paso:

1) Primero, enviamos la petición exactamente de la misma forma en que la enviábamos antes, pero ahora cogemos la sesión devuelta por el servidor en el objeto *session*. Leer la cookie de sesión es tan típico que el objeto *MockMvc*, que representa al servidor, ya incorpora directamente esta operación.
2) Cuando enviamos la segunda petición le decimos al caso de prueba: «Oye, envíala, pero dile que somos los de antes: o sea, que somos los que han enviado la petición anterior al recurso *login*».

```
@Test @Order(4)
void testDelete() throws Exception {
   JSONObject jso = new JSONObject().
          put("name", "pepe").
          put("pwd", "pepe1234");
   RequestBuilder request = MockMvcRequestBuilders.
      put("/users/login").
      contentType("application/json").
      content(jso.toString());

   MockHttpSession session = (MockHttpSession) this.server.perform(request).
      andReturn().getRequest().getSession();

   request = MockMvcRequestBuilders.
         delete("/users/removeUser").
         session(session);
   this.server.perform(request).
      andExpect(status().isOk());
   assertTrue(this.userDAO.countByName("Pepe")==0);
}
```

Figura 373. Control de la sesión en el caso de prueba

5.2. Ejecución de *testDelete*

Reproducimos en la Figura 374 exactamente la misma situación que en la Figura 372, de la que dijimos que tal vez fuese "complicada". Los puntos de interrupción están ubicados exactamente en la misma primera línea de los recursos web *removeUser* y *login* del *UserController*.

Seguro que ahora resulta más fácil de entender:

1) En la parte superior de la figura, la ejecución se ha detenido (como esperábamos) en la primera línea de *login*. En la zona de variables observamos que Spring ha inyectado la sesión el parámetro *session* del método. El *id* asignado a la sesión es "4". Además, vemos que este objeto *session,* que corre en el servidor, es el objeto número 194.

2) En la parte inferior se está ejecutando el segundo servicio (*removeUser)* del caso de prueba. Igual que en el punto anterior, Spring también ha inyectado la sesión en el parámetro *session* y, más importante, vemos que el identificador del objeto es el mismo (194): es decir, Spring, o el servidor de aplicaciones, o… alguien, nos está colocando en esta segunda petición exactamente la misma instancia de la sesión que en la petición previa. Observamos, además que el *id* de la sesión es el mismo que antes ("4"): es decir, nos estamos refiriendo exactamente al mismo objeto de tipo *HttpSession*.

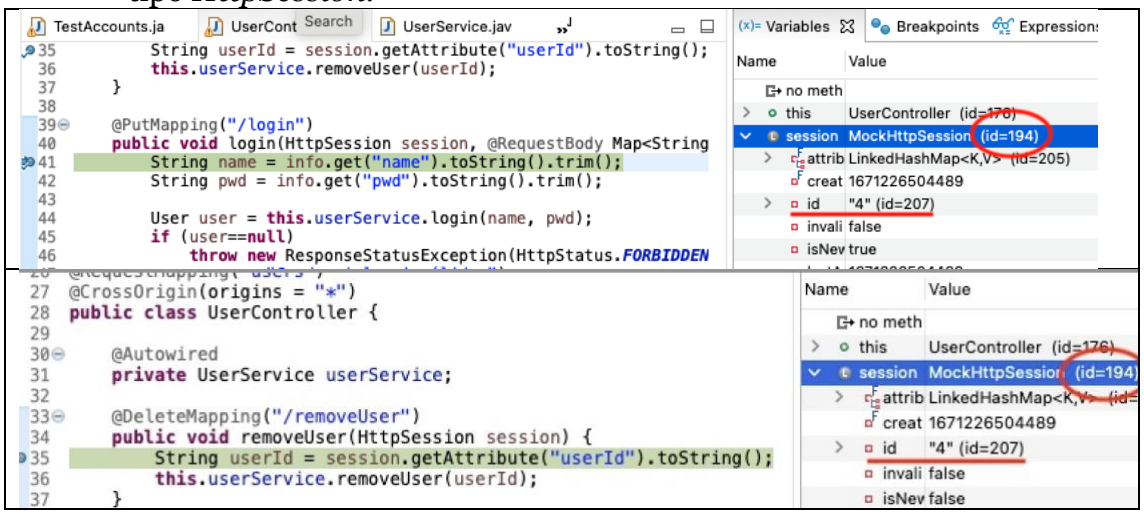


Figura 374. Las dos peticiones utilizan exactamente la misma sesión

No hace falta decir que JUnit muestra veredicto de paso (o sea, la barra verde) para todos los casos de *TestAccounts*.

Capítulo 17. Mejora de los casos de prueba

1. Medida de la cobertura

Cuando hablamos de medir la cobertura, nos referimos al hecho de conocer cuánto código de nuestro sistema están recorriendo los casos de prueba. Como estamos utilizando un desarrollo dirigido por pruebas, el código de producción es, al menos en teoría, el estrictamente necesario para superar los casos que guía su escritura, por lo que podemos suponer que la cobertura será muy muy alta.

Para Eclipse disponemos de un plugin llamado *Eclemma Code Coverage* que nos da informes de las sentencias que se recorren durante una ejecución. Para instalarlo, basta con buscarlo en el *Market Place* de Eclipse.

Vamos a usar este plugin para conocer cuánto código están recorriendo nuestros tests. Tenemos cuatro ficheros con casos de prueba, pero hay uno (*TestSelectWord)* que utiliza la base de datos en memoria, por lo que lo vamos a excluir de la ejecución: pulsamos sobre él con el botón derecho, elegimos *Build Path* y *Exclude*.

Para ejecutar los tres ficheros que nos quedan de una sola vez (y conocer así la cobertura que alcanzan todos juntos), pulsamos con el botón derecho sobre la carpeta de tests (*src/test/java)* y elegimos *Coverage As JUnit test*. La opción *Coverage As* lanzará los casos de prueba con Eclemma, que acabamos de instalar (Figura 375). No olvidemos que:

1) Los casos de *TestFunctionalGames* se ejecutan con Selenium y se dirigen al servidor Angular (en *localhost:4200*), por lo que…
2) … este debe estar arrancado (*ng serve)*. Este servidor redirige las peticiones al puerto 80, así que…
3) …la aplicación debe estar arrancada en Eclipse.
4) Por supuesto, también debe estar lanzado el docker con la base de datos.

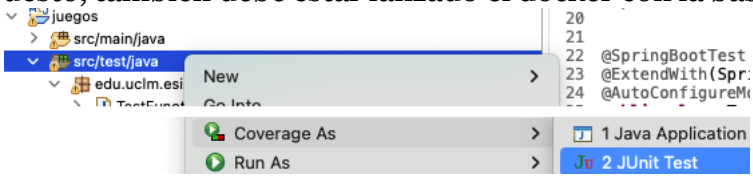


Figura 375. Ejecución con Eclemma

Los resultados se muestran en la vista *Coverage* de Eclipse. La Figura 376 es un gran pantallazo extraído de Eclipse:

1) En la parte inferior figuran los resultados de cobertura del proyecto completo y los porcentajes de instrucciones que se han recorrido (es decir, que se han visitado al ejecutar los casos). En realidad, son "instrucciones" y no "sentencias", pues Eclemma trabaja a nivel de *bytecode* (el ensamblador al que se traduce el código fuente Java).
2) En la parte de arriba se muestra parte del código de *UserController*, sobre la que se ha alcanzado un 98,8% de cobertura. Vemos que la sentencia en color rojo no se ha ejecutado y que la que está en amarillo se ha ejecutado "parcialmente".
   En Eclemma, el tema de las ejecuciones "parciales" aparece en las sentencias condicionales y es algo confuso: en el ejemplo que estamos analizando, la línea 81 tiene una sola condición (comparar una cadena

con otra) y la sentencia 82 se ha recorrido, por lo que no debe preocuparnos el amarillo de la 81.

3) Se ofrece también parte del código de la entidad *User*, de la que se recorre solamente el 63,9%, un valor muy bajo. Esta clase, sin embargo, es un POJO (página 101) y no tiene tampoco que causar mayor problema.

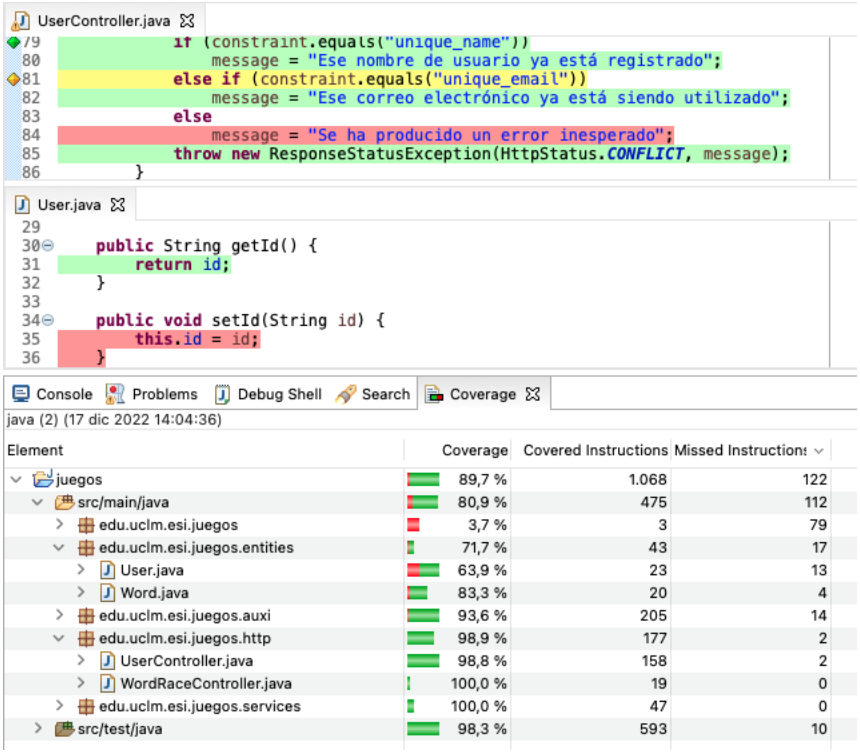


Figura 376. Resultados de la cobertura

El análisis de la cobertura nos ayuda a conocer las zonas del código que no se han explorado y, por tanto, nos guía para escribir nuevos casos de prueba que recorran precisamente esas zonas. Podemos estar bastante satisfechos de los resultados de la figura anterior: en los servicios, que son los que implementan las operaciones de negocio, se han recorrido todas las sentencias. Además, y muy importante, todos los casos de prueba se superan: es decir, la inmensa mayoría de nuestro sistema se está comportando correctamente.

Como no se está pasando por la línea 84 del *UserController*, debemos escribir algún caso de prueba que la recorra.

2. Técnicas para la creación de buenos datos de prueba

Un caso de prueba ejecuta un escenario determinado del sistema. El escenario consiste en ejecutar ciertas operaciones a las que pasamos ciertos valores para sus parámetros.

Para conseguir escribir buenos casos de prueba tenemos primero que conocer técnicas para proponer buenos datos de prueba para pasar a los parámetros de las operaciones.

Una vez que tengamos buenos datos de prueba, deberemos combinaremos sabiamente de distintas maneras para conseguir que los casos alcancen en el sistema tanta cobertura como sea posible. En la sección 3 de este capítulo aprenderemos a combinar adecuadamente los datos obtenidos.

2.1. Clases de equivalencia

Con esta técnica, se divide el dominio de entrada de cada parámetro en conjuntos disjuntos (cada conjunto es una "partición" o "clase de equivalencia"), asumiéndose que el sistema bajo prueba (al que con frecuencia se lo llama *SUT,* acrónimo del inglés *System Under Test)* se comportará igual para cualquier valor de la clase.

Para probar la operación *register* de nuestro sistema, nos fijamos en que toma cuatro parámetros. Podemos crear las clases de equivalencia que se muestran en la Figura 377.

Parámetro	Clases de equivalencia
name	Valor que no existe. Valor que ya existe.
email	Correo válido. Correo inválido.
pwd1	Cadena menor de 5 caracteres. Cadena mayor de 5 caracteres.
pwd2	Cadena igual a *pwd1*. Cadena distinta de *pwd1*.

Figura 377. Clases de equivalencia para los parámetros de register

Con los datos que pasamos a *testRegister200* y los que pasamos a *testRegister409* (Figura 358, página 231), estamos utilizando adecuadamente esta técnica.

2.2. Valores límite

Cuando los valores de las clases de equivalencia son ordenables, la técnica de valores límite la complementa. No es el caso, pero supongamos que en el registro solicitamos la fecha de nacimiento para impedir que accedan al sistema los menores de edad. Tendríamos un parámetro adicional en la operación, *birthDate*, con dos clases de equivalencia: (1) fecha actual menos *birthDate>=18,* y (2) fecha actual menos *birthDate<18*.

Así, y para que los datos que usemos tengan validez, seleccionaríamos un valor de cada clase de equivalencia (Figura 378).

Clase de equivalencia	Dato de prueba
fecha actual menos *birthDate>=18*	5 de diciembre de 1971
fecha actual menos *birthDate<18*	15 de junio de 2022

Figura 378. Clases de equivalencia y valores para una fecha de nacimiento

Al aplicar la técnica de valores límite, debemos añadir, a la lista de datos de prueba, los valores límite de las clases de equivalencia. También se recomienda añadir el primer valor de las clases de equivalencia adyacentes.

De este modo, si hoy es 17 de diciembre de 2022, para la primera clase de equivalencia añadiremos (Figura 378) a los datos de prueba el 17 de diciembre de 2004 (que es el valor límite), también el 16 de diciembre de 2004 (primer valor de la clase de equivalencia adyacente).

	Dato de prueba	
Clase de equivalencia	**De Clase de equiv.**	**De Valor límite**
fecha actual menos *birthDate>=18*	5 de diciembre de 1971	17 de diciembre de 2004 16 de diciembre de 2004
fecha actual menos *birthDate<18*	15 de junio de 2022	

Figura 379. Datos de prueba con clases de equivalencia y valores límite

2.3. Conjetura de errores

Mediante la técnica de la conjetura de errores, el *tester* propone valores para los que, de alguna manera, intuye que el sistema puede mostrar un comportamiento erróneo. Para nuestro recurso *register*, la verdad, no hemos pensado en ver qué pasa si no enviamos nada como nombre de usuario, si

enviamos un trozo de código en JavaScript o si escribimos una contraseña demasiado larga. Como sabemos, la contraseña se encripta con SHA-512, y se guarda en la tabla en un *varchar(255)*: si un usuario mete una contraseña de 255 caracteres, ¿ocupará más el valor encriptado y lanzará el servicio un error 500? La respuesta a esta segunda pregunta es no, pues precisamente 255 es la longitud máxima de los *hash* generados por ese algoritmo.

Pero es verdad que no sabemos cómo se comportará el sistema si no se envía ningún nombre de usuario al registrarse. ¿Esperaremos un 200, un 409, un 403, un temible 500?

2.4. Obtención de todas las salidas

Cuando probamos, debemos intentar que el sistema ofrezca todas sus posibles salidas, lo que incluye la ejecución de los escenarios normales y también la de los alternativos.

En nuestro caso, el método *register* está devolviendo a los casos *testRegister200* y *testRegister409* casi todas las respuestas posibles, pues hay una línea que no se está visitando (la que devuelve "Se ha producido un error inesperado" en la línea de estado).

3. Criterios de cobertura para los datos de prueba

Una vez que disponemos un buen conjunto de datos prueba, debemos combinarlos con alguna estrategia de combinación para obtener los casos de prueba, pues estos han de utilizar, "de alguna manera", los valores que se han propuesto. Podemos, por ejemplo, usar el correo "pepe@pepe.com" en un solo caso o tal vez en dos; podemos usar "pepe" y "pepe@pepe.com" una vez o dos. Para medir el grado en que se utilizan los valores de prueba, se utilizan criterios de cobertura para valores.

Para ilustrar los diferentes criterios de cobertura para datos de prueba, resumimos en la siguiente figura los que hemos recopilado para probar la funcionalidad *register*.

Parámetro	Datos de prueba
name	Pepe, Ana, cadena vacía
email	pepe@pepe.com, ana#ana.com
pwd1	pepe1234, ana1
pwd2	pepe1234, pepe12345

Figura 380. Resumen de los datos de prueba para los parámetros de register

3.1. *Each use* (cada valor)

Cuando se construyen casos de prueba conforme a este criterio, cada valor de cada parámetro debe incluirse, al menos, en un caso de prueba.

Para nuestro ejemplo de *register,* y puesto que el parámetro que más valores tiene es *name*, este criterio de cobertura nos obligará a crear al menos tres casos de prueba. Las combinaciones de la Figura 381 utilizan (también se dice que "visitan") cada valor seleccionado al menos una vez.

Combinación de datos	Resultado esperado
Pepe, pepe@pepe.com, pepe1234, pepe1234	200
Ana, ana#ana.com, ana1, pepe12345	403
— (cadena vacía), pepe@pepe.com, pepe1234, pepe1234	403

Figura 381. Un test suite que verifica Each use

3.2. *Pairwise* (todos los pares)

Con este criterio, debe visitarse al menos una vez cada par de valores de cada par de parámetros. Es decir, hay que crear combinaciones de datos que utilicen todos los valores de *name* con todos los valores de *email,* pero también todos los de *name* con *pwd1* y con *pwd2,* y también todos los de *email* con *pwd1,* de *email* con *pwd2* y, por último, de *pwd1* con *pwd2*.

Muchos errores que cometemos los programadores ocurren por interacciones de dos valores que no hemos previsto al escribir nuestro código. Por eso, crear casos de prueba que usen todos los pares de valores es una forma muy buena de encontrar errores ocultos. Si, además, los valores los hemos creado con las técnicas anteriores (clases de equivalencia, valores límite, conjetura de errores y obtención de todas las salidas) y, claro, los casos de prueba se superan, entonces tendremos una garantía muy alta acerca del buen funcionamiento de nuestro software.

3.2.1. Construcción manual de los casos de prueba

Si construimos estas combinaciones manualmente, debemos crear primero las tablas de pares. Si hay *n* parámetros, habrá *n·(n-1)/2* tablas. En cada tabla colocamos los pares de valores de los parámetros. En nuestro caso, las 6 tablas son las siguientes:

(name, email)
Pepe, pepe@pepe.com
Pepe, ana#ana.com
Ana, pepe@pepe.com
Ana, ana#ana.com
—, pepe@pepe.com
—, ana#ana.com

(name, pwd1)
Pepe, pepe1234
Pepe, ana1
Ana, pepe1234
Ana, ana1
—, pepe1234
—, ana1

(name, pwd2)
Pepe, pepe1234
Pepe, pepe12345
Ana, pepe1234
Ana, pepe12345
—, pepe1234
—, pepe12345

(email, pwd1)
pepe@pepe.com, pepe1234
pepe@pepe.com, ana1
ana#ana.com, pepe1234
ana#ana.com, ana1

(email, pwd2)
pepe@pepe.com, pepe1234
pepe@pepe.com, pepe12345
ana#ana.com, pepe1234
ana#ana.com, pepe12345

(pwd1, pwd2)
pepe1234, pepe1234
pepe1234, pepe12345
ana1, pepe1234
ana1, pepe12345

Figura 382. Tablas de pares para los parámetros de register

Ahora, para construir los casos, vamos seleccionando y tachando pares de valores, que vamos añadiendo a la combinación de valores para el caso de prueba. La idea es utilizar cada par el menor número de veces posible (aunque tampoco este objetivo debe obsesionarnos), pues así obtendremos menos casos.

Procediendo como acabamos de contar, escribimos el primer caso eligiendo, por ejemplo, el primer par de *(name, email)* y, de las restantes tablas, pares no visitados que sean compatibles con lo que llevamos escrito del caso. Luego, marcamos los pares utilizados (Figura 383).

Combinación
Pepe, pepe@pepe.com, pepe1234, pepe1234

(name, email)
Pepe, pepe@pepe.com
Pepe, ana#ana.com
Ana, pepe@pepe.com
Ana, ana#ana.com
—, pepe@pepe.com
—, ana#ana.com

(name, pwd1)
Pepe, pepe1234
Pepe, ana1
Ana, pepe1234
Ana, ana1
—, pepe1234
—, ana1

(name, pwd2)
Pepe, pepe1234
Pepe, pepe12345
Ana, pepe1234
Ana, pepe12345
—, pepe1234
—, pepe12345

(email, pwd1)	(email, pwd2)	(pwd1, pwd2)
pepe@pepe.com, pepe1234 pepe@pepe.com, ana1 ana#ana.com, pepe1234 ana#ana.com, ana1	pepe@pepe.com, pepe1234 pepe@pepe.com, pepe12345 ana#ana.com, pepe1234 ana#ana.com, pepe12345	pepe1234, pepe1234 pepe1234, pepe12345 ana1, pepe1234 ana1, pepe12345

Figura 383. Seleccionamos valores de prueba y tachamos los pares visitados

Para el segundo caso, elegimos un par no visitado de, por ejemplo, la primera tabla, luego añadimos pares compatibles de las otras tablas y marcamos los nuevos pares visitados.

Combinaciones
Pepe, pepe@pepe.com, pepe1234, pepe1234 Pepe, ana#ana.com, ana1, pepe12345

(name, email)	(name, pwd1)	(name, pwd2)
Pepe, pepe@pepe.com Pepe, ana#ana.com Ana, pepe@pepe.com Ana, ana#ana.com —, pepe@pepe.com —, ana#ana.com	Pepe, pepe1234 Pepe, ana1 Ana, pepe1234 Ana, ana1 —, pepe1234 —, ana1	Pepe, pepe1234 Pepe, pepe12345 Ana, pepe1234 Ana, pepe12345 —, pepe1234 —, pepe12345

(email, pwd1)	(email, pwd2)	(pwd1, pwd2)
pepe@pepe.com, pepe1234 pepe@pepe.com, ana1 ana#ana.com, pepe1234 ana#ana.com, ana1	pepe@pepe.com, pepe1234 pepe@pepe.com, pepe12345 ana#ana.com, pepe1234 ana#ana.com, pepe12345	pepe1234, pepe1234 pepe1234, pepe12345 ana1, pepe1234 ana1, pepe12345

Figura 384. Creamos una nueva combinación y marcamos los nuevos pares

Seguiremos procediendo de esta forma hasta haber visitado todos los pares. Obviamente, necesitaremos visitar alguno más de una vez para alcanzar la cobertura *pairwise*.

3.2.2. Herramientas para generar de casos de prueba con *pairwise*

Existen muchas herramientas online para generar casos de prueba con cobertura *pairwise*. Una sencilla búsqueda en Internet con la cadena "pairwise tool" nos da acceso a varias. En el lado izquierdo de la Figura 385 se está utilizando la ofrecida en *pairwise.teremokgames.com*. Con los valores suministrados, esta herramienta genera los 8 casos de la derecha.

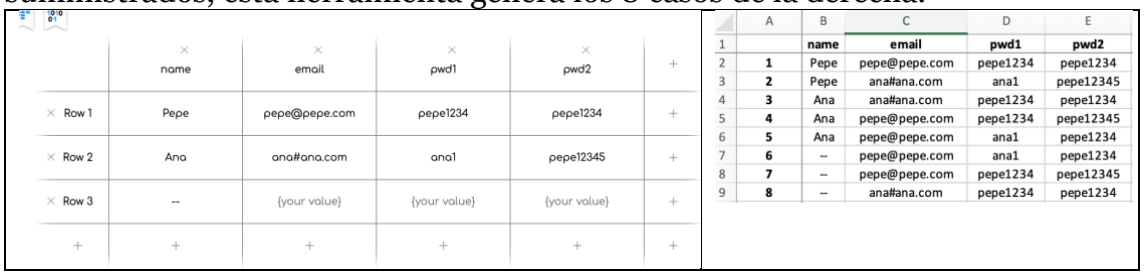


Figura 385. Una herramienta online para pairwise

Una vez que tenemos las combinaciones con los datos, debemos traducirlas a casos de prueba ejecutables. Para ello, primero debemos determinar el resultado que esperamos ofrezca el sistema como salida (Figura 386).

	name	email	pwd1	pwd2	Resultado esperado
1	Pepe	pepe@pepe.com	pepe1234	pepe1234	200
2	Pepe	ana#ana.com	ana1	pepe12345	409. El correo suministrado
3	Ana	ana#ana.com	pepe1234	pepe1234	no tiene un formato válido
4	Ana	pepe@pepe.com	pepe1234	pepe12345	409. La contraseña y su
5	Ana	pepe@pepe.com	ana1	pepe1234	confirmación no coinciden

6	--	pepe@pepe.com	ana1	pepe1234	409. El nombre de usuario debe tener al menos 4 caracteres
7	--	pepe@pepe.com	pepe1234	pepe12345	
8	--	ana#ana.com	pepe1234	pepe1234	

Figura 386. Determinamos el resultado esperado

3.2.3. Control de la longitud del nombre de usuario

Las combinaciones 6 a 8, que representan nombres de usuario demasiado cortos, no van a dar como respuesta el resultado esperado, puesto que no estamos comprobando la longitud del nombre en ningún sitio. Lo podemos hacer de varias maneras. Por ejemplo, de estas dos:

1) Añadiendo una comprobación en el propio controlador (Figura 387).

```
@PostMapping("/register")
public void register(@RequestBody Map<String, Object> info) {
     ...

     String name = info.get("name").toString().trim();
     if (name.length()<4)
       throw new ResponseStatusException(HttpStatus.CONFLICT,
           "El nombre de usuario debe tener al menos 4 caracteres");
     try {
       this.userService.register(name, pwd1, email);
     } catch (DataAccessException e) {
       ...
}
```

Figura 387. Comprobamos la longitud del nombre en UserController

2) Añadiendo una restricción *check* a la base de datos para que la columna *name* tenga 4 caracteres o más. La restricción la podemos añadir directamente a la entidad *User* (Figura 388, arriba). Tras borrar la tabla de la base de datos y lanzar de nuevo la aplicación, podremos ver que se ha creado una restricción con nombre *name_length_4*.

```
public class User {
     @Id @Column(length = 36)
     private String id;
     @Column(columnDefinition =
        "varchar(20) CONSTRAINT [name_length_4] CHECK ((len(ltrim(rtrim([name])))>(3)))")
     private String name;
...
```

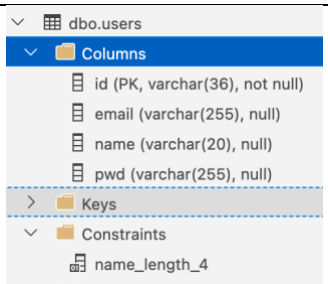


Figura 388. Creamos una restricción check *específica en la base de datos*

Si optamos por esta segunda solución, que resulta sin duda más elegante, podemos quitar las líneas añadidas en la Figura 387 y capturar en el *catch* el intento de violación de la restricción.

Recordará el lector (de la Figura 355, página 230) que SQL Server lanza un error con código 2627 al intentar violar una restricción de índice único. En la nueva situación (restricción *check)*, el código de error que devuelve el gestor de bases de datos es el 547, y el mensaje que podemos encontrar en el texto descriptivo es *The INSERT statement conflicted with the CHECK constraint "name_length_4"*, mientras que antes era *Violation of UNIQUE KEY constraint 'unique_email'*. Hay una pequeña diferencia (comillas simples y dobles) para

localizar el nombre de la restricción y devolver el mensaje de error adecuado (Figura 389).

```
String name = info.get("name").toString().trim();
try {
   this.userService.register(name, pwd1, email);
} catch (DataAccessException e) {
   SQLException cause = (SQLException) e.getCause().getCause();
   int errorCode = cause.getErrorCode();
   String constraint;
   if (errorCode==547)
      constraint = message.substring(message.indexOf('\"') + 1,
         message.indexOf('.') -1);
   else
      constraint = message.substring(message.indexOf('\'') + 1,
         message.indexOf('.') -1);
   ...
   if (constraint.equals("unique_name"))
      ...
   else if (constraint.equals("name_length_4"))
      message = "El nombre de usuario debe tener al menos 4 caracteres";
   else
      message = "Se ha producido un error inesperado";
   throw new ResponseStatusException(HttpStatus.CONFLICT, message);
}
```

Figura 389. Añadimos otra línea para capturar la excepción

3.2.4. Escritura de los casos de prueba

Debemos traducir a código ejecutable las combinaciones de datos que, junto a los resultados esperados, resumimos en la Figura 386. El siguiente caso de prueba se ha añadido a un nuevo fichero *TestPairwise*.

```
@ParameterizedTest
@CsvSource(delimiter = '\t', nullValues = "NULL", value =  {
   "Pepe    pepe@pepe.com      pepe1234  pepe1234  200       NULL",
   "Pepe    ana#ana.com        ana1      pepe12345 409       El correo suministrado no tiene un formato válido",
   "Ana     ana#ana.com        pepe1234  pepe1234  409       El correo suministrado no tiene un formato válido",
   "Ana     pepe@pepe.com      pepe1234  pepe12345 409       La contraseña y su confirmación no coinciden",
   "Ana     pepe@pepe.com      ana1      pepe1234  409       La contraseña y su confirmación no coinciden",
   "-- pepe@pepe.com ana1      pepe1234  409       El nombre de usuario debe tener al menos 4 caracteres",
   "-- pepe@pepe.com pepe1234  pepe12345 409       El nombre de usuario debe tener al menos 4 caracteres",
   "-- ana#ana.com   pepe1234  pepe1234  409       El nombre de usuario debe tener al menos 4 caracteres"
})
void testRegister409(String name, String email, String pwd1, String pwd2, int expectedCode,
String expectedMessage) throws Exception {
     JSONObject jso = new JSONObject().
           put("name", name).
           put("pwd1", pwd1).
           put("pwd2", pwd2).
           put("email", email);

     RequestBuilder request = MockMvcRequestBuilders.
       post("/users/register").
       contentType("application/json").
       content(jso.toString());

     String obtainedMessage = this.server.perform(request).
       andExpect(status().is(expectedCode)).
       andReturn().getResponse().getErrorMessage();
     assertEquals(expectedMessage, obtainedMessage);
}
```

Figura 390. Nuevo caso de prueba con los datos de la Figura 386

Al ejecutar, las tres últimas instancias del caso dan veredicto de error: sucede que, antes de intentar insertar el usuario en la tabla, se hacen las otras comprobaciones, se lanza la excepción y la ejecución se interrumpe.

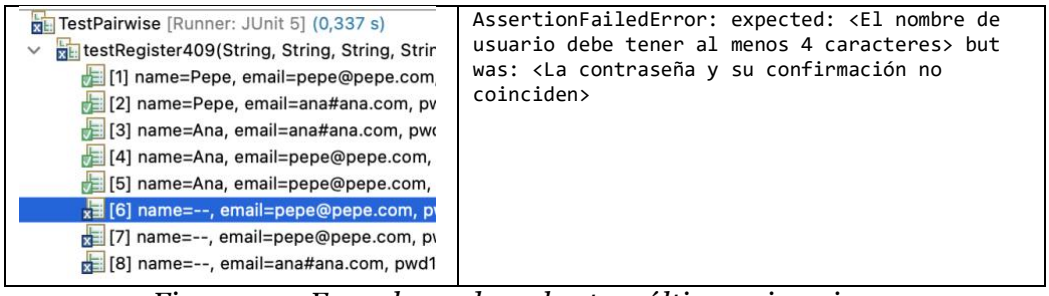


Figura 391. Error lanzado en las tres últimas ejecuciones

Además, la operación *register* que estamos probando tiene las 8 posibles salidas que aparecen en la Figura 392, y no todas se obtienen con las combinaciones de datos anteriores.

Código	Texto	Orden de comprobación
200		
409	El correo suministrado no tiene un formato válido	1
409	La contraseña y su confirmación no coinciden	2
409	La contraseña debe tener al menos cinco caracteres	3
409	Ese nombre de usuario ya está registrado	4
409	Ese correo electrónico ya está siendo utilizado	5
409	El nombre de usuario debe tener al menos 4 caracteres	6
409	Se ha producido un error inesperado	7

Figura 392. Las ocho posibles salidas de register

Por ejemplo, ninguna combinación fuerza a que se intente violar la restricción *unique_name*. O sea, debemos revisar y completar el *test suite* añadiendo nuevos casos de prueba o combinando los datos de otra forma. En cualquier caso, detectaríamos que no se están lanzando todas las salidas al hacer el análisis de la cobertura.

Las siguientes combinaciones consiguen que se lancen todas las salidas:

```
@CsvSource(delimiter = '\t', nullValues = "NULL", value =  {
  "Pepe  pepe@pepe.com   pepe1234  pepe1234  200  NULL",
  "Anita ana#ana.com     pepe1234  pepe1234  409  El correo suministrado no tiene un formato válido",
  "Anita ana@ana.com     ana1234   pepe1234  409  La contraseña y su confirmación no coinciden",
  "Anita ana@ana.com     ana1      ana1      409  La contraseña debe tener al menos cinco caracteres",
  "Pepe  ana@ana.com     pepe1234  pepe1234  409  Ese nombre de usuario ya está registrado",
  "Anita pepe@pepe.com   pepe1234  pepe1234  409  Ese correo electrónico ya está siendo utilizado",
  "Ana   ana@ana.com     pepe1234  pepe1234  409  El nombre de usuario debe tener al menos 4 caracteres",
  "123456789012345678901 ana@ana.com          pepe1234       pepe1234 409      Se ha producido un error
inesperado"
})
```

Figura 393. Siete casos que obtienen todas las salidas

Como se ve, la última salida se fuerza intentando registrar a un usuario con un nombre de más de 20 caracteres, que es la longitud que establecimos en la restricción *check* de la Figura 388 para la columna *name*.

3.3. *All combinations* (todas las combinaciones)

Con esta técnica generamos todas las posibles combinaciones que puedan formarse con los valores de prueba.

Desde luego, si los datos de prueba que tenemos son buenos, la cobertura que obtendremos será la mejor posible, pues no hay ninguna combinación de datos imprevista. Como en nuestro ejemplo tenemos 4 parámetros con respectivamente 3, 2, 2 y 2 valores, el número de combinaciones es 3·2·2·2=24. Utilizar esta esta estrategia de combinación tiene algunos inconvenientes:

1) El número de casos es muy elevado y hay que pensar el oráculo para cada uno.

2) Habrá casos de prueba redundantes, que recorren exactamente las mismas zonas del programa.
3) Cuando el sistema evolucione, quizá sea necesario modificar los casos para que se adapten a la nueva versión del programa.
4) Los casos de prueba tienen un coste de ejecución: tardan en ejecutarse y requieren recursos. Cuantos más tengamos, en más coste incurriremos.

4. Otros criterios de cobertura para código fuente

Además del porcentaje de sentencias, hay otros criterios de cobertura para código fuente que, en general, miden el grado en que se usan las instrucciones condicionales.

Supongamos que, en el *catch* de *register* comprobamos en una sola instrucción si se violan las restricciones *unique_name* o *unique_email*. Es decir, sustituimos los dos *if* por uno solo, cuyas condiciones quedan separadas por *OR*:

```
...
if (errorCode==547)
   constraint = message.substring(message.indexOf('\"') + 1, message.indexOf('.') -1);
else
   constraint = message.substring(message.indexOf('\'') + 1, message.indexOf('.') -1);
if (constraint.equals("unique_name") || constraint.equals("unique_email"))
   message = "El nombre de usuario o el correo electrónico ya están registrados";
else if (constraint.equals("name_length_4"))
   message = "El nombre de usuario debe tener al menos 4 caracteres";
else
   message = "Se ha producido un error inesperado";
...
```

Figura 394. Una decisión con dos condiciones

Si adecuamos el resultado esperado al nuevo mensaje ("El nombre de usuario o el correo electrónico ya están registrados"), podemos pensar que nos basta un solo caso de prueba para recorrer ese *if:* como vemos a continuación, eliminamos el segundo caso y nos quedamos con solo uno:

```
@CsvSource(delimiter = '\t', nullValues = "NULL", value =  {
   ...
   "Pepe ana@ana.com pepe1234 pepe1234 409 El nombre de usuario o el correo electrónico ya están
registrados",
   //"Anita  pepe@pepe.com      pepe1234pepe1234409     El nombre de usuario o el correo
electrónico ya están registrados",
   ...
})
```

Al ejecutar con Eclemma, la sentencia aparece marcada en amarillo, lo que denota que se ha ejecutado "parcialmente": o sea, hemos alcanzado la línea siguiente, pero no sabemos si es porque se ha verificado la primera condición (*unique_name)* o la segunda (*unique_email).*

```
else
    constraint = message.substring(message.indexOf('\'') + 1, message.indexO
if (constraint.equals("unique_name") || constraint.equals("unique_email"))
    message = "El nombre de usuario o el correo electrónico ya están registr
else if (constraint.equals("name_length_4"))
    message = "El nombre de usuario debe tener al menos 4 caracteres";
```

Figura 395. No se han cubierto las dos condiciones

Un criterio de cobertura para condiciones ampliamente aceptado es el MC/DC (*Modified Condition-Decision Coverage*). Con este criterio, debemos recorrer las dos ramas *(true* y *false)* de cada decisión gracias a la contribución individual de cada condición sin que, en la medida de lo posible, influyan las otras condiciones. Así, para conseguir cobertura MC/DC en esa instrucción, deberíamos construir tres casos de prueba: uno para ir a la rama *true* gracias a

que el valor de *constraint* sea *unique_name,* otro para ir también por *true* haciendo que *constraint* sea *unique_email,* y otro que salga por su rama *false*, que será cuando no se verifique ninguna de las dos condiciones.

Supongamos que tenemos el siguiente fragmento de código:

```
1.  if (a==1 or b==2 or c==3) {
2.      x(a, b);
3.  } else {
4.      y(b, a, c);
5.  }
```

Figura 396. Una instrucción condicional

Para recorrer esa decisión con cobertura MC/DC, debemos ir a la rama *true* y a la rama *else* gracias a la contribución individual de cada condición sin que influyan, en la medida de lo posible, ninguna de las otras condiciones. Es decir:

- Debemos ir a *true* haciendo que *a==1* sin que influyan *b==2* ni *c==3*.
- Debemos ir a *true* haciendo que *b==2* sin que influyan *a==1* ni *c==3*.
- Debemos ir a *true* haciendo que *c==3* sin que influyan *a==1* ni *b==2*.
- Y, además, debemos ir a la rama *else* gracias a cada condición individual. No obstante, como es una decisión formada por tres condiciones separadas por *or*, habrá que hacer *false* las tres condiciones para ir a la rama *else*.

Las siguientes cuatro combinaciones logran cobertura MC/DC en la decisión:

Variables			Condiciones			Decisión
a	b	c	a==1	b==2	c==3	
100	100	100	F	F	F	false
1	100	100	T	F	F	true
100	2	100	F	T	F	true
100	100	3	F	F	T	true

Figura 397. Cuatro combinaciones que alcanzan MC/DC en la decisión

Hagamos otro ejemplo para el siguiente fragmento de código, en el que sustituimos un *OR* por un *AND:*

```
1.if (a==1 or (b==2 and c==3)) {
2.    x(a, b);
3.} else {
4.    y(b, a, c);
5.}
```

Figura 398. Cambiamos un OR *por un* AND

Para facilitar el seguimiento del ejemplo, usamos 1, 2 y 3 como valores "buenos" para *a, b, c*, y 100 como valor "malo".

En este caso, debemos ir a la rama *true* gracias, por un lado, a *a==1* y, por otro, gracias a *(b==2 and c==3)*. Del mismo modo, debemos ir a la rama *else* gracias a que, por un lado, sea falsa la primera condición y gracias a que, por otro, sea falsa la segunda.

Vayamos poco a poco. En primer lugar, construimos una tabla como la siguiente (utilizamos colores para indicar el desarrollo de cada fila de la tabla) y rellenamos algunas columnas de las condiciones (nótese que hay dos, una de ellas compuesta) con *true* y *false:*

Variables			Condiciones				Decisión
a	b	c	a==1	b==2	c==3	(b==2 and c==3)	(a==1 or (b==2 and c==3))
			T			F	
			F			T	
			F			F	

Figura 399. MC/DC, paso 1

Como se ve, estamos preparando la tabla para irnos a cada rama teniendo en cuenta cada condición de la decisión. Podemos rellenar ahora la columna de la decisión:

Variables			Condiciones				Decisión
a	b	c	a==1	b==2	c==3	(b==2 and c==3)	(a==1 or (b==2 and c==3))
			T			F	true
			F			T	true
			F			F	false

Figura 400. MC/DC, paso 2

Ahora, desarrollamos si es necesario las filas que correspondan a la condición compuesta. Nótese que de las filas 1 y 3 de la tabla anterior, que necesitan que *(b==2 and c==3)* sea *false*, surgen dos filas, una en la que *b!=2* y otra en la que *c!=3:*

Variables			Condiciones				Decisión
a	b	c	a==1	b==2	c==3	(b==2 and c==3)	(a==1 or (b==2 and c==3))
			T	T	F	F	true
			T	F	T	F	true
			F	T	T	T	true
			F	T	F	F	false
			F	F	T	F	false

Figura 401. MC/DC, paso 3

Ahora solo nos queda dar valores a *a, b* y *c* para que se verifiquen los valores indicados:

Variables			Condiciones				Decisión
a	b	c	a==1	b==2	c==3	(b==2 and c==3)	(a==1 or (b==2 and c==3))
1	2	100	T	T	F	F	true
1	100	3	T	F	T	F	true
100	2	3	F	T	T	T	true
100	2	100	F	T	F	F	false
100	100	3	F	F	T	F	false

Figura 402. MC/DC, paso 4 y último

5. Obtención de casos de prueba a partir de máquinas de estado

Uno de los usos habituales de las máquinas de estado es la descripción de casos de uso del sistema que se está desarrollando.

La siguiente figura puede ilustrar un comportamiento deseado para nuestro sistema:

1) Cuando el usuario se equivoca al loguearse, incrementamos un contador. A la tercera vez, la cuenta se le bloquea.
2) Si se loguea correctamente, le reseteamos el contador y comprobamos si tiene ya una sesión abierta. En caso afirmativo, se la cerramos.

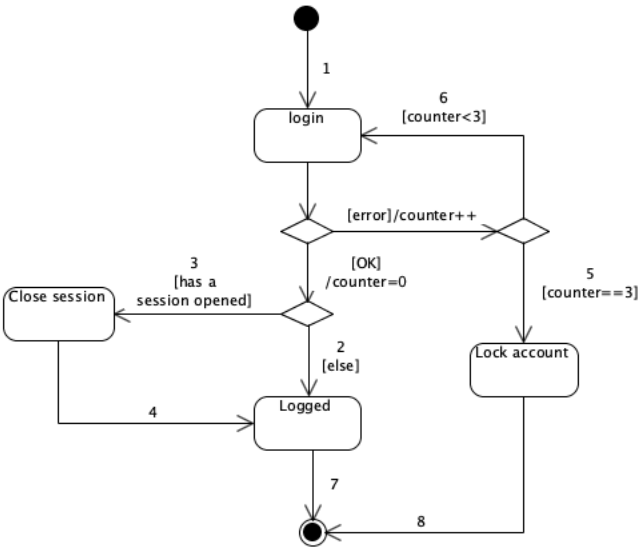


Figura 403. Posible comportamiento del login

A partir de esta máquina de estados, podemos construir casos de prueba conforme a diversos criterios de cobertura:

1) Recorrer todos los estados.
2) Recorrer todas las transiciones.
3) Recorrer todos los pares de entrada y salida transiciones en cada estado.
4) Recorrer todos los caminos que el *tester* considere interesantes.

5.1. Todos los estados y todos los caminos interesantes

Para verificar este criterio, debemos pasar al menos una vez por cada estado. Así, no tendríamos por qué recorrer la transición 2 *[error]* que sale del estado *register,* ni tampoco por qué pasar por la 7, etiquetada con *[else]*. Para pasar, sin embargo, por el estado *Lock account* sí que tendríamos que forzar a que el contador fuese 3, lo que podemos considerar un "camino interesante".

5.2. Todas las transiciones

En este caso, sí es preciso pasar por las transiciones etiquetadas con *[error], [else],* etcétera, pues hay que pasar por todas. Al recorrer todas las transiciones se recorren también todos los estados, por lo que se dice que el criterio de cobertura de todas las transiciones "subsume" al de todos los estados.

5.3. Todos los pares de entrada y salida

Para verificar este criterio, debemos construir las tablas de pares de transiciones de entrada y salida de cada estado y escribir casos de prueba que los recorran todos. Este criterio subsume a todas las transiciones.

login	*Lock account*	Close session	Logged
(1,2) (1,3) (1,5) (1,6)	*(5,8)*	(3,4)	(2,7) (4,7)

Figura 404. Pares de transiciones para la máquina de la Figura 403

El proceso para crear los casos de prueba es parecido al que seguimos para crear manualmente los casos con cobertura *pairwise* (sección 3.2.1, página 245): iteramos eligiendo pares no visitados, que vamos añadiendo al *test suite*. Con las dos secuencias de operaciones de la Figura 405 conseguimos recorrer todos los pares de transiciones.

Caso	Pares	Operaciones y datos
1	(1,2) (2,7) (1,3) (3,4) (4,7)	login Pepe, pepe1234 *Este par se cubre automáticamente el loguearse con credenciales válidas* login Pepe, pepe1234 *Estos pares se cubre automáticamente al loguearse correctamente con una sesión abierta*
2	(1,6) (1,6) (1,5) (5,8)	login Pepe, pepe12345 login Pepe, pepe12345 login Pepe, pepe12345 *Este par se recorre automáticamente al llegar a* Lock account

Figura 405. Casos de prueba con cobertura de pares de estados

5.4. Escritura de oráculos

Una de las ventajas de las máquinas de estado es que los estados resumen, precisamente, diferentes estados del sistema, y el estado de un sistema puede describirse como invariantes sobre las propiedades del sistema. Una invariante es una condición que debe cumplirse en un determinado estado. Por ejemplo, en el estado *Logged,* el usuario ha de tener una sesión abierta; al salir de *Lock account,* la cuenta ha de estar bloqueada y tener un valor 3 en el contador; antes de llegar a *Close session,* el usuario tiene dos sesiones abiertas.

5.5. Escritura de casos de prueba. Refactorización de código

En esta sección traducimos a código los dos casos de prueba procedentes del criterio de cobertura de todos los pares de transiciones. Obviamente, para que se superen cumpliendo los requisitos de la máquina de estados es necesario modificar el código de producción. Crearemos en el paquete *edu.uclm.esi.juegos.users* un nuevo fichero, *TestStateMachine* (Figura 406), con los casos de prueba que visitan todos los pares de transiciones.

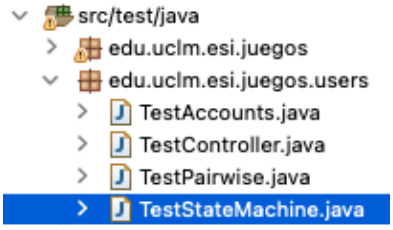


Figura 406. Fichero para los casos de prueba procedentes de la máquina de estados

5.5.1. Primer caso de prueba: pares (1,2), (2,7), (1,3), (3,4), (4,7)

El siguiente caso de prueba es la traducción a nuestro sistema del primer recorrido de pares de la Figura 405:

1) En el primer bloque, registramos a Pepe con sus credenciales habituales.
2) En el segundo, Pepe se loguea correctamente, recorriendo así el par (1,2).
3) En el tercero, enviamos una petición a un nuevo recurso *numberOfSessions* de un nuevo controlador *tests,* que devuelve el número de sesiones abiertas por el usuario, y escribimos un aserto para comprobar que solo tiene una.
4) En el cuarto, Pepe vuelve a loguearse con sus credenciales buenas.
5) Y, en el quinto, volvemos a contar el número de sesiones que tiene abiertas este usuario.

```
@Test
void testLoginTransitionPairs1() throws Exception {
   JSONObject jsoRegister = new JSONObject().
         put("name", "Pepe").
         put("pwd1", "pepe1234").
         put("pwd2", "pepe1234").
         put("email", "pepe@pepe.com");
   RequestBuilder request = MockMvcRequestBuilders.
      post("/users/register").
      contentType("application/json").
      content(jsoRegister.toString());
   this.server.perform(request).andReturn();                                    (1)
   JSONObject jsoLogin = new JSONObject().
         put("name", "Pepe").
         put("pwd", "pepe1234");
   RequestBuilder requestLogin = MockMvcRequestBuilders.
      put("/users/login").
      contentType("application/json").
      content(jsoLogin.toString());
   MockHttpSession session =
      (MockHttpSession) this.server.perform(requestLogin).
         andReturn().getRequest().getSession();                                 (2)
   RequestBuilder requestCount = MockMvcRequestBuilders.
         get("/tests/numberOfSessions").
         contentType("application/json").
         session(session);
   String response = this.server.perform(requestCount).
      andReturn().getResponse().getContentAsString();
   int numberOfSessions = Integer.parseInt(response);
   assertEquals(1, numberOfSessions);                                           (3)
   session = (MockHttpSession) this.server.perform(requestLogin).
         andReturn().getRequest().getSession();                                 (4)
   response = this.server.perform(requestCount).
         andReturn().getResponse().getContentAsString();
   assertEquals(1, numberOfSessions);                                           (5)
}
```

Figura 407. Traducción a código del primer caso de prueba de la Figura 405

Este método puede refactorizarse para hacerlo más simple: ofrecerá la misma funcionalidad, pero de forma más clara y legible, y con menos propensión a cometer errores.

En primer lugar, podemos extraer el bloque 1 a un nuevo método auxiliar *register*. Para ello, seleccionamos las líneas que queramos llevar al nuevo método y elegimos *Extract Method* en el menú *Refactor* (Figura 408, izquierda). El IDE nos preguntará el nombre del nuevo método, y le damos *register*. El entorno crea el nuevo método y sustituye el código seleccionado por una llamada (Figura 408, derecha).

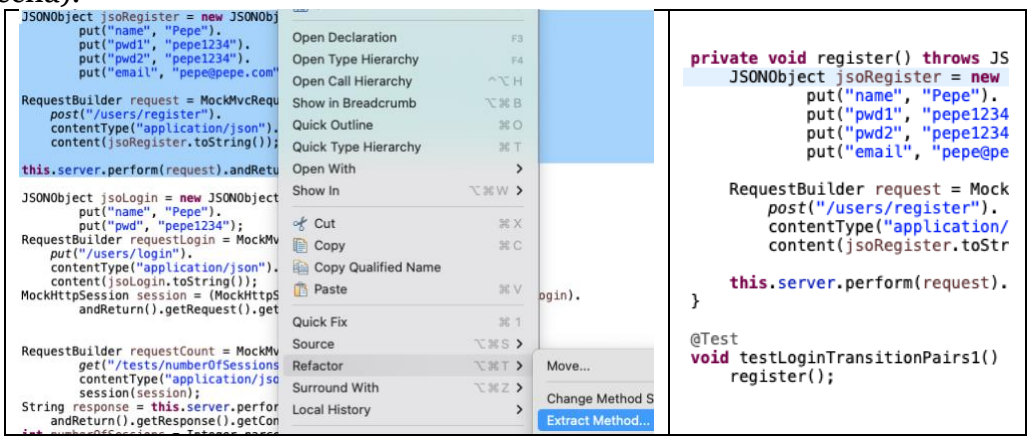


Figura 408. Eclipse (y cualquier IDE moderno) nos crea un nuevo método a partir del código seleccionado mediante una refactorización automática

En los bloques 3 y 5 hacemos también prácticamente lo mismo: contar el número de sesiones del usuario. Si podemos, hacemos la refactorización automática; si no, la hacemos con un pequeño esfuerzo manual (Figura 409).

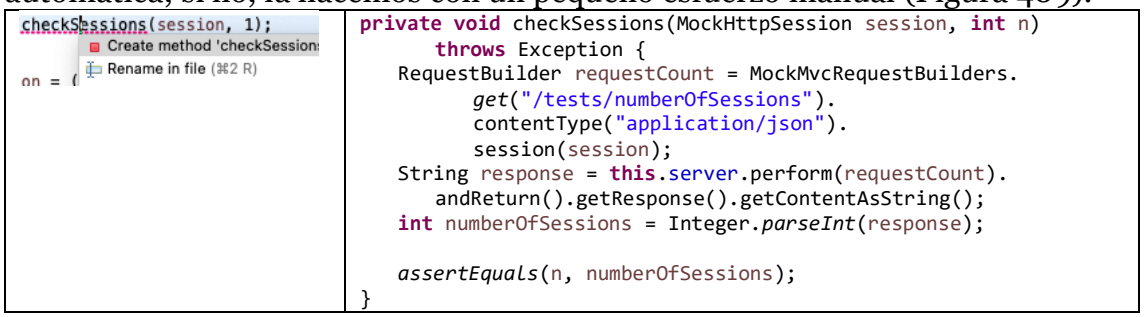


Figura 409. Refactorización manual

Como hemos comentado con respecto a los bloques 3 y 5, y podemos seguir en el método *checkSessions* que acabamos de refactorizar, hemos creado un controlador con nombre público *tests* que ofrece el servicio *numberOfSessions*. Este controlador solo sirve para realizar pruebas y, por eso, lo hemos escrito en el subsistema de tests: se trata del fichero *TestController* que se muestra en la Figura 410.

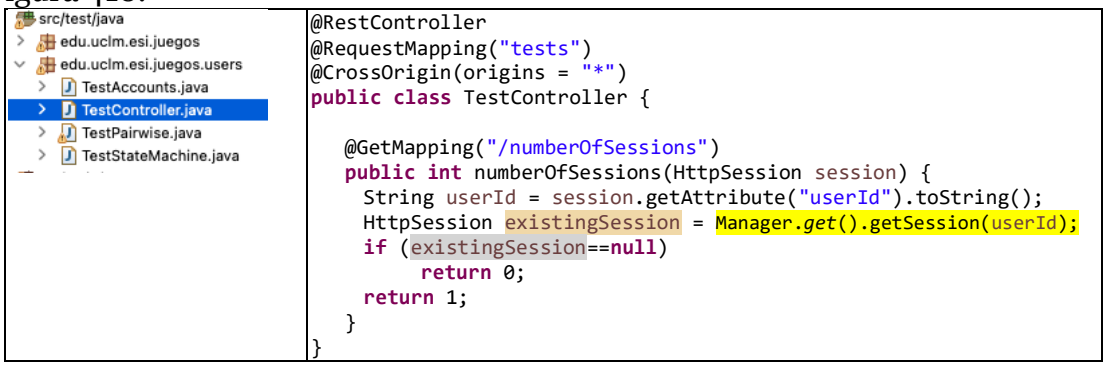


Figura 410. En ocasiones, crearemos controladores, servicios u otras clases para apoyar la fase de pruebas

Para controlar, como se espera de la máquina de estados con la que hemos descrito el nuevo comportamiento del caso de uso *login*, las sesiones que tiene abiertas un usuario, necesitamos llevar esta de alguna manera. En el *TestController* que acabamos de mostrar usamos el *Manager,* que es un singleton al que podemos acceder desde cualquier lugar de la aplicación. Cuando el usuario se logue correctamente, comprobamos si ya tiene sesión abierta y, si la tiene, la invalidamos y añadimos la nueva (Figura 411).

```
@PutMapping("/login")
public void login(HttpSession session, @RequestBody Map<String, Object> info) {
   String name = info.get("name").toString().trim();
   String pwd = info.get("pwd").toString().trim();

   User user = this.userService.login(name, pwd);
   if (user==null)
      throw new ResponseStatusException(HttpStatus.FORBIDDEN,
            "Credenciales inválidas");
   session.setAttribute("userId", user.getId());
   HttpSession existingSession = Manager.get().getSession(user.getId());
   if (existingSession!=null) {
      existingSession.invalidate();
      Manager.get().removeSession(user.getId());
   }
   Manager.get().addSession(user.getId(), session);
}
```

Figura 411. Gestión de las sesiones abiertas en el UserController

Como se ve en la Figura 412, el *Manager* mantiene una tabla *hash* con las sesiones http activas indexadas por el *id* del usuario.

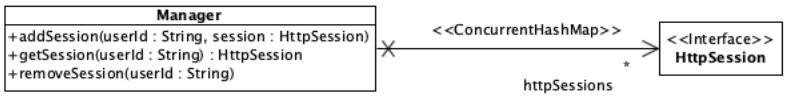


Figura 412. Usamos el *Manager* para controlar las sesiones http abiertas

5.5.2. Segundo caso de prueba: pares (1,6), (1,6), (1,5), (5,8)

Este segundo caso de prueba requiere mantener un contador que cuente los intentos de *login* erróneos que ha hecho el usuario. Podemos delegar esta responsabilidad al *UserService,* que es el objeto finalmente encargado de ejecutar la operación de negocio *login:* en lugar de buscar al usuario por *name* y *pwd* (el cual creamos en la Figura 363, página 233), lo buscaremos por *name* (habrá que declarar el método *User findByName(String name)* en el repositorio *UserDAO).*

El nuevo método *login* de *UserService* puede ser el de la Figura 413:

1) Buscamos al usuario por *name.*
2) Si no existe no hay cuenta a la que bloquear, así que le devolvemos *null* al controlador.
3) Si sí existe y su cuenta está bloqueada (es decir, su campo *counter* es mayor o igual a 3), lanzamos desde aquí la *ResponseStatusException* con el texto "Cuenta bloqueada" y un código 403 (*forbidden).*
4) Si sí existe y la cuenta no está bloqueada, entonces encriptamos la contraseña con SHA-512:
   a. Si la contraseña encriptada no coincide con la que tenemos en la tabla, incrementamos el contador, actualizamos la fila asociada a esta instancia de la entidad *User* en la tabla *users* y devolvemos *null.*
   b. Si la contraseña es la correcta, le ponemos el contador a cero, también actualizamos la fila asociada a esta instancia de la entidad *User* en la tabla *users* y devolvemos el *user* que hemos recuperado de la base de datos.

```
public User login(String name, String pwd) {
   User user = this.userDAO.findByName(name);
   if (user==null)
      return null;
   if (user.getCounter()!=null && user.getCounter()>=3)
      throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Cuenta bloqueada");
   pwd = org.apache.commons.codec.digest.DigestUtils.sha512Hex(pwd);
   if (!user.getPwd().equals(pwd)) {
      user.setCounter(user.getCounter() + 1);
      this.userDAO.save(user);
      user = null;
   } else {
      user.setCounter(0);
      this.userDAO.save(user);
   }
   return user;
}
```

Figura 413. Nueva implementación de login *en* UserService

El código del nuevo caso de prueba puede ser el de la Figura 414: utiliza, como se ve, el método auxiliar *wrongLogin,* que intenta loguear a Pepe con una contraseña incorrecta.

```
@Test @Order(2)
void testLoginTransitionPairs2() throws Exception {
   this.wrongLogin();
   this.wrongLogin();
   this.wrongLogin();
   String response = this.wrongLogin();
   assertEquals("Cuenta bloqueada", response);
}

private String wrongLogin() throws Exception {
   JSONObject jsoLogin = new JSONObject().
         put("name", "Pepe").
         put("pwd", "pepe12345");
   RequestBuilder requestLogin = MockMvcRequestBuilders.
      put("/users/login").
      contentType("application/json").
      content(jsoLogin.toString());
   return this.server.perform(requestLogin).
      andReturn().getResponse().getErrorMessage();
}
```

Figura 414. Traducción a código del segundo caso de prueba de la Figura 405

5.5.3. Reejecución de los casos

Hemos hecho cambios importantes en el método de logueo, por lo que vamos a reejecutar todos o casi todos los casos de prueba que llevamos hasta ahora. No ejecutamos todos porque, como comentamos al hablar de la estrategia *All combinations* (sección 3.3, página 249), ejecutar casos de prueba lleva un coste asociado.

En nuestra situación este es coste es inapreciable, pero, para dejar un poco de impronta, probaremos solamente los casos que afecten a las funcionalidades modificadas: es decir, ejecutaremos solo los que tenemos en los ficheros del paquete *edu.uclm.esi.juegos.users: TestAccounts, TestPairwise* y *TestStateMachine*. No olvidemos lanzar el servidor de Angular, el de Eclipse y el docker.

Ahora obtenemos valores menores de cobertura, pero es un resultado esperado, pues los casos de prueba que hemos ejecutado son solamente los relacionados con la gestión de cuentas del usuario (Figura 415).

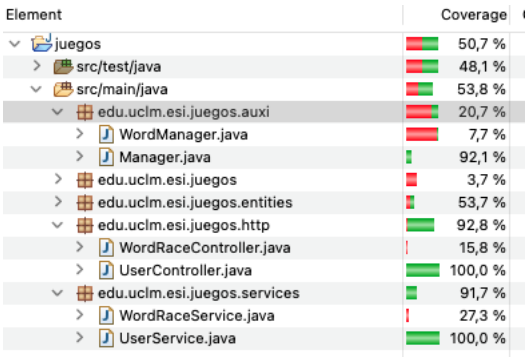


Figura 415. Cobertura de los casos en el paquete edu.uclm.esi.juegos.users

Capítulo 18. *login* y *register* en el *front-end*

1. Justificación

Hay funcionalidades habituales de la gestión de usuarios que no se han desarrollado en el capítulo anterior. Por ejemplo, cuando un usuario se registra, lo habitual es enviarle algún tipo de mensaje (un SMS o un correo electrónico) para que confirme el registro en nuestro sistema. Igualmente, si el usuario olvida su contraseña, le pediremos su correo electrónico y le enviaremos ahí algún enlace para que cree una nueva. En el capítulo anterior, la cuenta del usuario se bloquea si se intenta loguear tres veces con credenciales inválidas, y podemos asumir que el desbloqueo se consigue pinchando en un enlace que llegue por correo. Estas tres situaciones requieren el acceso a un sistema externo (un servidor SMTP) que envíe los correos electrónicos correspondientes: confirmar el registro, recordar la contraseña o desbloquear la cuenta.

En este libro se ha reservado la Sexta parte («Integración de servicios de terceros») para tratar los aspectos de una aplicación web que necesitan servicios que ofrece un servidor web diferente del nuestro.

Entonces, con la postergación plenamente consciente de esos casos de uso que pertenecerían al anterior capítulo, desarrollaremos aquí el *front-end* que permita a los usuarios aprovechar las funcionalidades que se dibujaban en la Figura 318 (página 207) y que, por ahora, van a ser las contrapartes de las implementadas en el capítulo anterior. El caso de uso *removeUser* provocará algún problema especial, por lo que aprovecharemos el Capítulo 19 para desarrollarlo.

2. Interpretación de los resultados de las pruebas del Capítulo 16

El código del *backend* que gestiona las cuentas de usuarios (con tres casos de uso: *register, login* y *removeUser)* se ha desarrollado con una metodología *pseudo TDD* (no hemos sido demasiado estrictos: vaya el lector a la página 88 y repase). Pero bueno, igual que ya dijimos que este no es un libro de gestión de proyectos software, tampoco lo es de metodologías de desarrollo; pretende ser un manual práctico y técnico (con barnices de teoría de arquitectura software, arquitectura web, patrones, *testing* y buenas prácticas, que se pretende calen e impregnen al lector) para el desarrollo de aplicaciones web con Angular y Spring: ambos *frameworks*, así como las directrices de nuestros jefes de proyecto, nos vienen imponiendo ciertos estilos arquitectónicos en ambos lados (cliente y servidor) a los que, estemos o no conformes, debemos adaptarnos. Desde luego, el lector es libre de utilizar un modelo diferente de paso de mensajes o de asignación de responsabilidades: cuando un usuario logueado quiere eliminar su cuenta, claro que podemos implementarlo para que el controlador se lo pida directamente al repositorio en vez de haciéndolo a través del servicio, pero esta solución no se ajustaría al deseo del director técnico del proyecto (recuérdense los comentarios acerca de sus imposiciones que mencionamos en los párrafos previos a la Figura 111, página 91).

Por otro lado, y desde un nivel de abstracción más bajo, tengamos en cuenta que el servidor de aplicaciones que hemos utilizado en el capítulo anterior (el campo *server* de *TestAccounts, TestPairwise* y *TestStateMachine,* que es de tipo *MockMvc)* no es un servidor real, sino "falso", pero cumple fielmente las especificaciones estándares: quiere esto decir que, si un caso de prueba que se envía desde un *@Test* a un *MockMvc,* se superará también si se envía desde un cliente real, como un navegador, a un servidor auténtico, como Tomcat.

De este modo, confiamos en la buena implementación que hemos escrito en nuestros casos de prueba anteriores, y pasamos directamente a la implementación del *front-end*.

3. Preparación

En este lado cliente, todo el código relativo a la gestión de la cuenta del usuario lo colocaremos en una carpeta *account*. En esta, crearemos un componente para cada una de las tres funcionalidades de las que disponemos hasta ahora en el *back*: registrarse, loguearse y eliminar la cuenta (Figura 416). Para el primer componente, escribiremos en la consola *ng generate component account/register*. Para los otros dos, pues lo mismo.

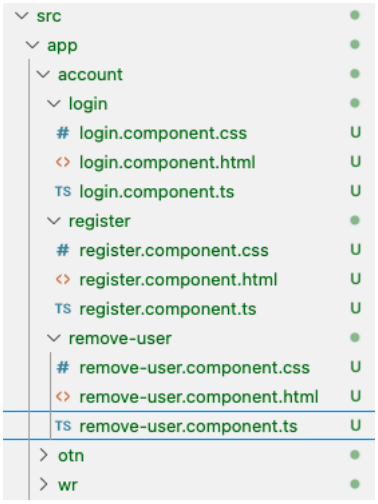


Figura 416. Los componentes para register, login *y* removeUser se agrupan en *app/account*

Por otro lado, al menos en *register* y en *login*, el usuario deberá escribir sus credenciales en cajas de texto (elementos de tipo *input)*. Para poder bindearlas con los respectivos campos que añadamos a los componentes (por ejemplo, la caja en donde al loguearse se escriba la contraseña debe bindearse a un campo *pwd*), necesitamos usar el *FormsModule* de Angular. Para esto, debemos añadir a *app.module.ts* las dos líneas que se destacan en la figura siguiente:

```
import { NgModule } from '@angular/core';
import { FormsModule } from '@angular/forms';
...

@NgModule({
  declarations: [
    ...
  ],
  imports: [
    ...
    FormsModule
  ],
  ...
})
export class AppModule { }
```

Figura 417. Fichero app.module.ts

Estas tres funcionalidades han de enviar mensajes al servidor, por lo que crearemos un servicio *UserService* en el mismo paquete *account*, que se encargará de enviar al *UserController* del *backend* las peticiones que correspondan. Con *ng generate service account/User*, Angular nos genera el código básico de este servicio.

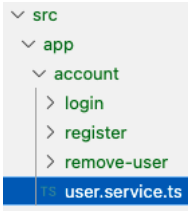


Figura 418. Creamos el UserService

4. Implementación del servicio

De momento, el *UserService* debe ofrecer al *front-end* los tres métodos que sirvan las tres funcionalidades que hemos programado. Además, igual que hicimos con el *WRService* (Figura 196, página 139), debemos inyectar a su constructor un *HttpClient*.

El código de *UserService* es tan sencillo como el de la Figura 419:

1) Inyectamos el cliente http en el constructor.
2) Implementamos las tres operaciones:
    a. *register* y *login* reciben un parámetro de tipo *any*, que contendrá un JSON con el objeto que se va a enviar al servidor.
    b. *removeUser* elimina al usuario que hay en la sesión y, por tanto, no necesita enviar ningún objeto.
    c. Las tres operaciones devuelven un *Observable* (Figura 203, página 143) de cualquier cosa (este "cualquier cosa" es realmente nada, pues los tres servicios web del *UserController* son de tipo *void).*
    d. Cada operación envía su petición con un método diferente: *post, put* o *delete,* igual que hicimos al implementar los casos de prueba del servidor.
3) Obsérvese además el pequeño detalle de la importación de *httpUrl* de *../commons*. En la url de las tres peticiones se utiliza este valor en lugar de "hardcodear" la url, como hacíamos en la Figura 233 (página 155).

commons es un fichero *ts* muy sencillito en el que colocaremos aquellos elementos que podamos necesitar en lugares diferentes. De momento tiene una sola línea: *export const httpUrl : string = "http://localhost:80/".*

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { httpUrl } from '../commons';

@Injectable({
  providedIn: 'root'
})
export class UserService {

  constructor(private client : HttpClient) { }

  register(user : any) : Observable<any> {
    return this.client.post<any>(httpUrl + "users/register", user)
  }

  login(user : any) : Observable<any> {
    return this.client.put<any>(httpUrl + "users/login", user)
  }

  removeUser() : Observable<any> {
    return this.client.delete<any>(httpUrl + "users/removeUser")
  }
}
```

Figura 419. Implementación del UserService

5. Caso de uso *register*

register es un componente de Angular que tendrá en la vista cuatro campos para introducir los cuatro datos necesarios para registrarse, un botón para ejecutar la operación y un *span* en el que mostraremos el posible mensaje de error.

Un componente en Angular, ya lo sabemos, tiene una vista y un modelo. En la parte superior de la Figura 420 aparece el código *html* con el formato dado a la vista: las cuatro cajas, el botón y el *span;* cada *widget* está bindeado a un miembro del modelo: la primera caja, al campo *name;* el *span,* al campo *error;* el evento *click* del botón, al método *register*.

Este método, como vemos, construye un JSON con los valores de los cuatro campos necesarios para crearse una cuenta en el sistema, y luego lo envía a la operación *register* del *userService,* que hemos inyectado en el constructor.

```
<input type="text" [(ngModel)]="name" placeholder="User's name">
<br>
<input type="text" [(ngModel)]="email" placeholder="email">
<br>
<input type="password" [(ngModel)]="pwd1" placeholder="password">
<br>
<input type=" password " [(ngModel)]="pwd2" placeholder="Repeat password">
<br>
<button (click)="register()">Create account</button>
<br>
<span style="color:red">{{ error }}</span>
```

```
import { Component, OnInit } from '@angular/core';
import { UserService } from '../user.service';

@Component({
  selector: 'app-register',
  templateUrl: './register.component.html',
  styleUrls: ['./register.component.css']
})
export class RegisterComponent implements OnInit {
  name? : string
  pwd1? : string
  pwd2? : string
  email? : string
  error? : string

  constructor(private userService : UserService) { }

  ngOnInit(): void {
  }

  register() {
    let user = {
      name : this.name, pwd1 : this.pwd1,
      pwd2 : this.pwd2, email : this.email
    }
    this.userService.register(user).subscribe({
      next : () => alert("OK"),
      error : (result) => this.error = result
    })
  }
}
```

Figura 420. El componente register

5.1. Prueba manual

Para probar manualmente este caso de uso, basta con que añadamos a *app.component.html* una etiqueta *<app-register></app-register>*. Con la tabla *users* vacía, basta con que escribamos las credenciales habituales de Pepe para obtener el *alert("OK")* del manejador del evento *next* en el método *register* del modelo del componente (línea resaltada en la figura anterior).

Pero, si tratamos de registrar a un usuario con datos inválidos, en el navegador aparece un extraño mensaje con el texto *[object Object]* (Figura 421).

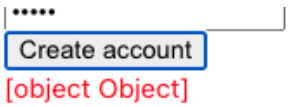


Figura 421. Mensaje de error al registrar con algún dato inválido

Si usamos el depurador de *Chrome,* vemos que el resultado, que es el valor que asignamos al campo *this.error* de *RegisterComponent,* es un objeto JSON que trae el texto del error en el campo *result.message.error*.

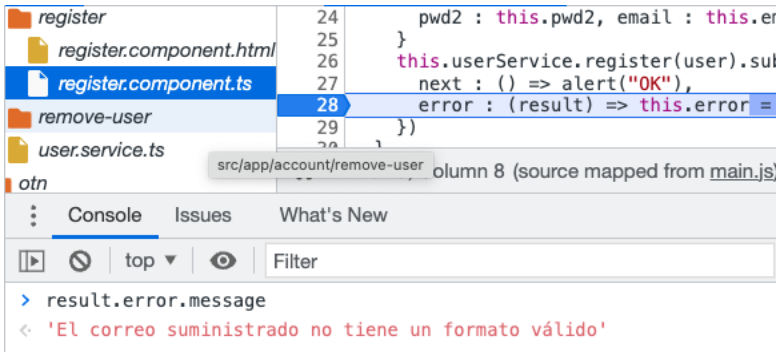


Figura 422. Dónde viene el mensaje de error

Por tanto, en la línea 28 que mostramos en la figura anterior, asignaremos a *this.error* el valor de ese campo del objeto *result*:

```
...
export class RegisterComponent implements OnInit {
  ...

  register() {
    ...
    this.userService.register(user).subscribe({
      next : () => alert("OK"),
      error : (result) => this.error = result.error.message
    })
  }
}
```

Figura 423. Recuperamos correctamente el mensaje de error

5.2. Pruebas con Selenium

De manera parecida a aquel caso de prueba parametrizado cuyos valores mostrábamos en la Figura 393 (página 249), crearemos ahora un fichero llamado *TestFunctionalAccounts* en el que vamos a escribir dos casos de prueba en Selenium para obtener, también, todas las posibles salidas:

1) Igual que allí, necesitamos un caso que nos devuelva un código 200 y que, de acuerdo con la línea resaltada en la Figura 420, debe mostrar un diálogo en el que aparezca la palabra "OK".
   En el *testRegister200* de la Figura 424:
   a. Localizamos las cuatro cajas de texto y el botón.
   b. Borramos el texto que pudieran tener.
   c. Escribimos las credenciales de Pepe, que son todas correctas.
   d. Pulsamos el botón que ejecuta *register*.
   e. Esperamos que se muestre una alerta con el texto "OK", que es lo que hacemos en las últimas líneas. Usamos para esto el objeto *Alert* de *Selenium*. En la última sentencia del caso de prueba, pulsamos el botón del pequeño cuadro de diálogo que ha aparecido.

```
@ParameterizedTest @Order(1)
@CsvSource(delimiter = '\t', nullValues = "NULL", value =  {
   "Pepe pepe@pepe.com       pepe1234             pepe1234             NULL"
})
public void testRegister200(String name, String email, String pwd1, String
pwd2, String expectedMessage) {
   WebElement inputName = driver.findElement(By.xpath("/html/body/app-
root/app-register/input[1]"));
   WebElement inputEmail = driver.findElement(By.xpath("/html/body/app-
root/app-register/input[2]"));
   WebElement inputPwd1 = driver.findElement(By.xpath("/html/body/app-
root/app-register/input[3]"));
   WebElement inputPwd2 = driver.findElement(By.xpath("/html/body/app-
root/app-register/input[4]"));
   WebElement buttonRegister = driver.findElement(By.xpath("/html/body/app-
root/app-register/button"));

   inputName.clear(); inputEmail.clear();
   inputPwd1.clear(); inputPwd2.clear();

   inputName.sendKeys(name); inputEmail.sendKeys(email);
   inputPwd1.sendKeys(pwd1); inputPwd2.sendKeys(pwd2);
   buttonRegister.click();

   Alert alert = driver.switchTo().alert();
   assertEquals(alert.getText(), "OK");
   alert.accept();
}
```

Figura 424. Un caso de prueba de registro satisfactorio

Es posible que, al ejecutar este caso de prueba, el *alert* que esperamos no haya aparecido todavía al hacer clic en el botón. Antes de pasar a buscar el *alert* en la sentencia *driver.switchTo().alert(),* podemos forzar una pausa de, por ejemplo, un segundo (Figura 425):

```
...
buttonRegister.click();
this.pause(1000);
Alert alert = driver.switchTo().alert();
...
```

```
private void pause(int millis) {
    try {
      Thread.sleep(millis);
    } catch (Exception e) {
    }
}
```

Figura 425. Forzamos una pausa de un segundo

2) También necesitamos a los restantes casos de prueba, que han de devolver error y, según también la Figura 420, han de asignar el texto del error al campo *error,* que debe mostrarse en el *span* con el que este campo está bindeado. El código de la Figura 426 resume siete situaciones en las que se esperan distintos mensajes de error. Como en el caso de prueba anterior, recuperamos las referencias a las cajas en las que escribir, al botón y, ahora, al *span* en el que se ha de mostrar el mensaje de error: se escriben las credenciales, se hace clic en el botón y, para cada situación, comparamos el texto esperado (que se pasa como quinto parámetro al método) con el texto obtenido (que recuperamos en la variable *spanText).*

```
@ParameterizedTest @Order(2)
@CsvSource(delimiter = '\t', nullValues = "NULL", value =  {
    "Anita ana#ana.com     pepe1234 pepe1234 El correo suministrado no tiene un formato válido",
    "Anita ana@ana.com     ana1234  pepe1234 La contraseña y su confirmación no coinciden",
    "Anita ana@ana.com     ana1     ana1     La contraseña debe tener al menos cinco caracteres",
    "Pepe  ana@ana.com     pepe1234 pepe1234 Ese nombre de usuario ya está registrado",
    "Anita pepe@pepe.com   pepe1234 pepe1234 Ese correo electrónico ya está siendo utilizado",
    "Ana   ana@ana.com     pepe1234 pepe1234 El nombre de usuario debe tener al menos 4 caracteres",
    "12345678901234567890 1ana@ana.com      pepe1234 pepe1234 Se ha producido un error inesperado",
})
public void testRegister409(String name, String email, String pwd1, String pwd2, String
expectedMessage) {
    WebElement inputName = driver.findElement(By.xpath("/html/body/app-root/app-
register/input[1]"));
    WebElement inputEmail = driver.findElement(By.xpath("/html/body/app-root/app-
register/input[2]"));
    WebElement inputPwd1 = driver.findElement(By.xpath("/html/body/app-root/app-
register/input[3]"));
    WebElement inputPwd2 = driver.findElement(By.xpath("/html/body/app-root/app-
register/input[4]"));
    WebElement buttonRegister = driver.findElement(By.xpath("/html/body/app-root/app-
register/button"));
    WebElement span = driver.findElement(By.xpath("/html/body/app-root/app-register/span"));

    inputName.clear(); inputEmail.clear();
    inputPwd1.clear(); inputPwd2.clear();

    inputName.sendKeys(name); inputEmail.sendKeys(email);
    inputPwd1.sendKeys(pwd1); inputPwd2.sendKeys(pwd2);
    buttonRegister.click();

    String spanText = span.getText();
    assertTrue(expectedMessage.equals(spanText));
}
```

Figura 426. Un caso de prueba con siete instancias en las que se espera error

Como se ve en la Figura 427, las 8 instancias de estos dos casos de prueba se superan.

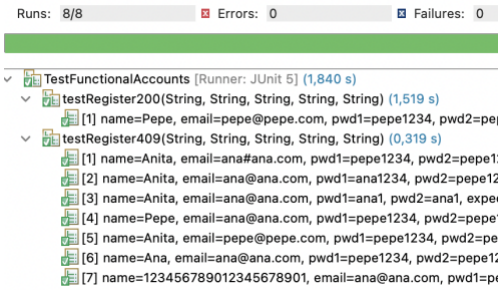


Figura 427. Los ocho casos de prueba de Selenium se superan

6. Caso de uso *login*

El desarrollo de este caso de uso en el lado cliente es muy parecido al anterior:

1) En la vista colocaremos una caja para el nombre de usuario, otra para la contraseña, un botón para ejecutar el método *login* (que crearemos en el modelo), un *span* para el posible error y dejaremos preparado un anchor (una etiqueta *<a>*) para ejecutar la futura funcionalidad de recuperar la contraseña.
2) En el modelo declaramos tres campos *name, pwd* y *error* (todos ellos, igual que en la Figura 420, con la interrogación al final, pues inicialmente no tienen valor, con lo que son *undefined)*, implementamos el método *login* (que llamará al servicio *login*) y, también importante, inyectamos el *UserService* en el constructor.

```html
<input type="text" [(ngModel)]="name"
   placeholder="User's name">
<br>
<input type="password" [(ngModel)]="pwd"
   placeholder="Password">
<br>
<button (click)="login()">Login</button>
<br>
<a>Recover password</a>
<br>
<span style="color:red">{{ error }}</span>
```

```typescript
import { Component, OnInit } from '@angular/core';
import { UserService } from '../user.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent implements OnInit {
  name? : string
  pwd? : string
  error? : string

  constructor(private userService : UserService) { }

  ngOnInit(): void {
  }

  login() {
    let user = {
      name : this.name,
      pwd : this.pwd
    }
    this.userService.login(user).subscribe({
      next : () => alert("OK"),
      error : (result) =>
          this.error = result.error.message
    })
  }
}
```

Figura 428. El componente login

6.1. Prueba manual

Para probar esta funcionalidad, añadimos la etiqueta *<app-login></app-login>* al componente principal el cual, de momento, tiene un aspecto bastante feo (Figura 429).

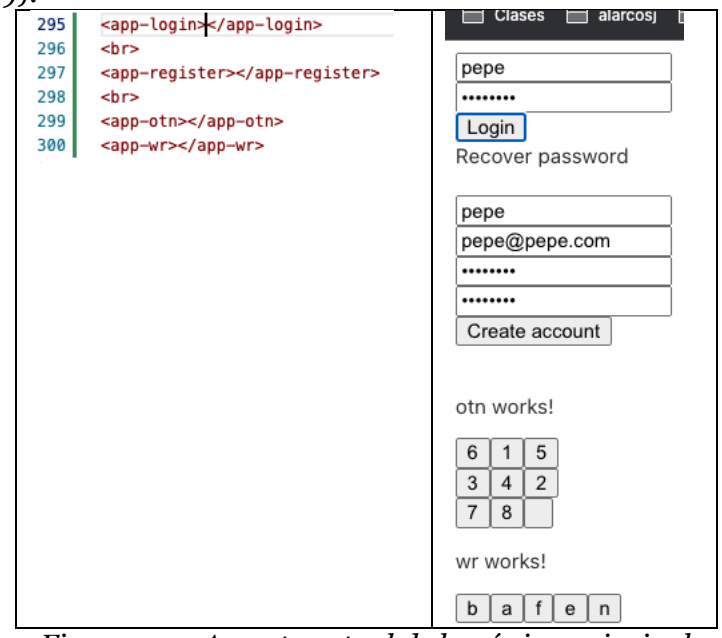


Figura 429. Aspecto actual de la página principal

Podemos registrar a Pepe y loguearlo después. En ambos casos, y asumiendo que la tabla *users* está vacía y no contiene a Pepe, se mostrarán los dos diálogos fruto de las sentencias *alert("OK")* que tenemos en los modelos de ambos componentes.

6.2. Más casos de prueba

Para ser coherentes con todo el proceso de desarrollo y pruebas que venimos haciendo, debemos crear con Selenium los casos procedentes de la máquina de estados del caso de uso *login* (Figura 407 y Figura 414). Desarrollaremos el segundo (*testLoginTransitionPairs2*, Figura 414, página 258), y quede la implementación del primero como tarea para el lector o lectora, que así practicará un poco de pruebas de los pares de estados, un poco de Selenium, un poco de JUnit.

Esa misma lectora o lector tal vez piense que, si ya hemos probado los pares de transiciones en *TestStateMachine* (Figura 405), y si dice el autor del libro en la sección 2 de este mismo capítulo que el falso servidor que hemos utilizado, de tipo *MockMvc,* "cumple fielmente las especificaciones estándares", no hace falta reescribir en formato Selenium el caso *testLoginTransitionPairs1*. Pero, sorprendentemente, ahora encontramos un error en el *backend*.

La Figura 430 muestra el caso *testLoginTransitionPairs2* que ya desarrollamos (usando el *MockMvc)* en formato Selenium. Este caso se ha añadido con *@Order(3)* al fichero *TestFunctionalAccounts* de la Figura 424. En ese fichero, se ejecuta primero *testRegister200* con un registro correcto de Pepe; luego, se ejecutan varias instancias de *testRegister409* para obtener todas las salidas; el caso que acabamos de añadir intenta loguear a Pepe tres veces con una contraseña errónea (y espera el mensaje de error "Credenciales inválidas") y, a la cuarta vez, espera obtener el mensaje "Cuenta bloqueada".

```
@Test @Order(3)
void testLoginTransitionPairs2() throws Exception {
   WebElement inputName = driver.findElement(By.xpath("/html/body/app-root/app-
login/input[1]"));
   WebElement inputPwd = driver.findElement(By.xpath("/html/body/app-root/app-login/input[2]"));
   WebElement buttonLogin = driver.findElement(By.xpath("/html/body/app-root/app-
login/button"));
   WebElement span = driver.findElement(By.xpath("/html/body/app-root/app-login/span"));

   for (int i=1; i<=3; i++) {
      inputName.clear(); inputPwd.clear();
      inputName.sendKeys("Pepe"); inputPwd.sendKeys("pepe12345");
      buttonLogin.click();
      pause(200);
      String spanText = span.getText();
      assertTrue("Credenciales inválidas".equals(spanText));
   }

   inputName.clear(); inputPwd.clear();
   inputName.sendKeys("Pepe"); inputPwd.sendKeys("pepe12345");
   buttonLogin.click();
   pause(200);
   String spanText = span.getText();
   assertTrue("Cuenta bloqueada".equals(spanText));
}
```

Figura 430. "Traducción" a Selenium del caso de la Figura 414 (página 258)

Como sucedía antes, los dos primeros casos de prueba se superan, pero el que acabamos de añadir no (Figura 431). El error se produce en la línea resaltada de la figura anterior.

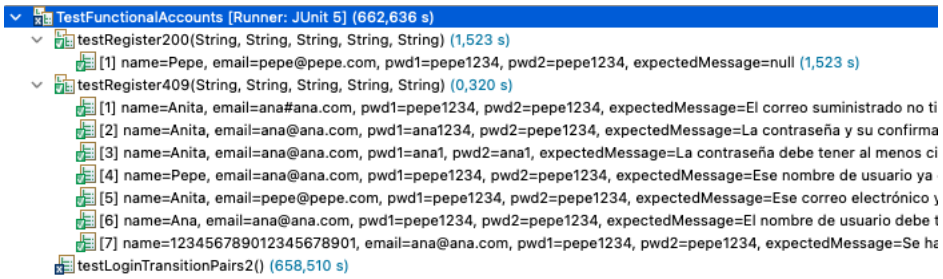


Figura 431. El nuevo caso de prueba da veredicto de error

6.2.1. Depuración de varios procesos

Cualquier entorno de desarrollo actual permite depurar varios procesos de los que el propio IDE está sirviendo. Como tenemos ese problema raro, vamos a depurar tanto el proceso lanzado al ejecutar *App* (el código de producción) como *TestFunctionalAccounts*.

Puesto que los dos primeros casos de prueba (*testRegister200* y *testRegister409)* se superan, colocaremos puntos de interrupción en el caso de prueba que da error. Ya hemos comentado que en la consola de JUnit se informa de que no se verifica el aserto *assertTrue("Credenciales inválidas".equals(spanText)):* esta diferencia del comportamiento esperado respecto del obtenido sucede cuando hacemos clic en *buttonLogin,* por lo que pondremos dos puntos de interrupción: uno en la línea 127 (cuando se va a pulsar el botón de *login)* y otro justo después de que Selenium haga ese clic: es decir, en la línea 128 (lado izquierdo de la Figura 432). Como sabemos que ese evento *click* envía un mensaje a *users/login,* colocamos otro punto de interrupción en la línea 45 de *UserController* (lado derecho de la figura).

```
117 @Test @Order(3)
118 void testLoginTransitionPairs2() throws Exceptio
119     WebElement inputName = driver.findElement(By
120     WebElement inputPwd = driver.findElement(By.
121     WebElement buttonLogin = driver.findElement(
122     WebElement span = driver.findElement(By.xpat
123
124     for (int i=1; i<=3; i++) {
125         inputName.clear(); inputPwd.clear();
126         inputName.sendKeys("Pepe"); inputPwd.sen
127         buttonLogin.click();
128         pause(1000);
129         String spanText = span.getText();
130         assertTrue("Credenciales inválidas".equa
131     }
132
133     inputName.clear(); inputPwd.clear();
134     inputName.sendKeys("Pepe"); inputPwd.sendKey
135     buttonLogin.click();
136     pause(200);
137     String spanText = span.getText();
138     assertTrue("Cuenta bloqueada".equals(spanTex
139 }
```

```
40 @PutMapping("/login")
41 public void login(HttpSession session, @R
42     String name = info.get("name").toStri
43     String pwd = info.get("pwd").toString
44
45     User user = this.userService.login(na
46     if (user==null)
47         throw new ResponseStatusException
48                 "Credenciales inválidas")
49     session.setAttribute("userId", user.g
50     HttpSession existingSession = Manager
51     if (existingSession!=null) {
52         existingSession.invalidate();
53         Manager.get().removeSession(user.
54     }
55     Manager.get().addSession(user.getId()
56 }
```

Figura 432. Colocamos puntos de interrupción para los dos procesos

Ahora estamos en disposición de lanzar los dos procesos (el código de producción y los casos de prueba): en la Figura 433 vemos que con el botón del escarabajo podemos ejecutar cualquier aplicación en modo *debug*. Primero lanzamos *App* y, cuando haya arrancado completamente, *TestFunctionalAccounts*.

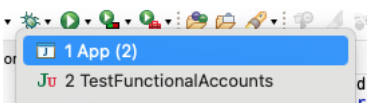


Figura 433. El botón del escarabajo ofrece ejecuciones, pero depurando

Tenemos dos puntos de interrupción: uno en *login* y otro en el caso de pruea. Como los dos primeros casos afectan solo a *register,* la ejecución no se

detiene hasta alcanzar la línea 127: si, en este momento, "soltamos" la ejecución dándole al botón *Resume* ( ), la petición que el evento clic envía llega a la línea 45 del otro proceso.

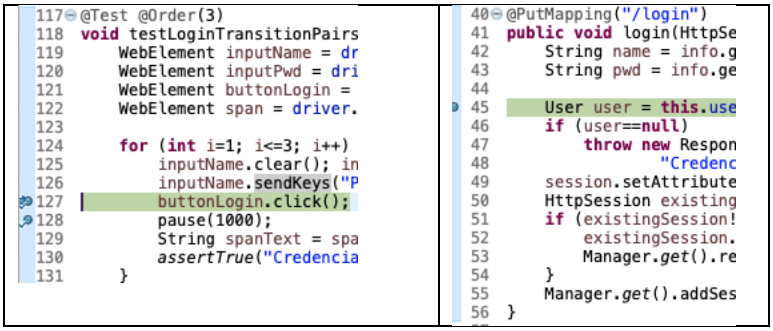


Figura 434. Al pulsar Resume *en el punto de interrupción de la izquierda, el depurador detiene la ejecución en la línea de la derecha*

Además, en la vista de depuración de Eclipse (o del entorno de desarrollo que estemos utilizando) podremos ver tantos hilos como procesos haya detenido el depurador. En la Figura 435 hay dos: uno del proceso que ha lanzado la aplicación (*App)* y otro del proceso que ha lanzado los tests.

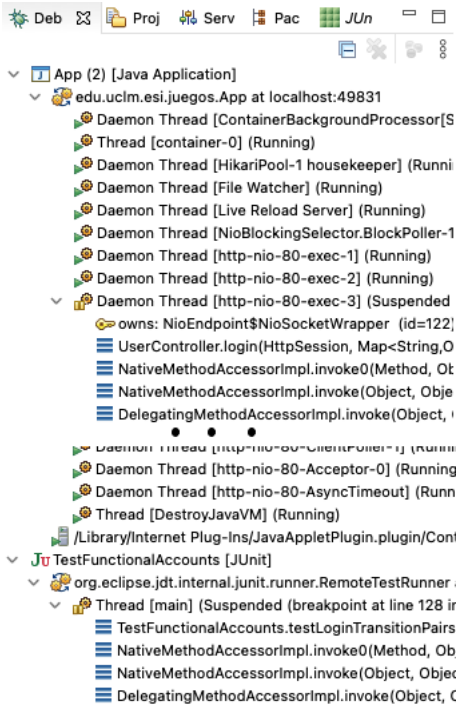


Figura 435. El depurador mantiene dos procesos en pausa

En este momento nos encontramos en el proceso de arriba (*App)*, puesto que hemos "soltado" la ejecución del caso de prueba. O sea, estamos en la línea 45 de *UserController*. En esta línea se va a llamar al método *login* de *UserService*. En Eclipse y en todos los entornos de desarrollo tenemos varios botones para controlar el depurador. Como sospechamos que el error está dentro de la implementación del *login* de *UserService,* pulsamos el primero de los tres botones de control de la ejecución, *Step into* (Figura 436).

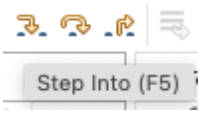


Figura 436. Step into, Step over *y* Step return

El código salta a la primera línea del método al que se está llamando (Figura 437). Ahora podemos avanzar con *Step over* (si avanzásemos con *Step into* entraríamos quizás en la implementación de *findByName,* que no nos interesa)

```
11 @Service
12 public class UserService {
13
14     @Autowired
15     private UserDAO userDAO;
16
17     public void register(String name, String pwd, S
24
25     public User login(String name, String pwd) {
26         User user = this.userDAO.findByName(name);
27         if (user==null)
28             return null;
29         if (user.getCounter()!=null && user.getCoun
30             throw new ResponseStatusException(HttpS
31         pwd = org.apache.commons.codec.digest.Diges
32         if (!user.getPwd().equals(pwd)) {
33             user.setCounter(user.getCounter() + 1);
34             this.userDAO.save(user);
```

Figura 437. Entramos al login *de* UserService

Como vemos en la zona de variables, tras ejecutar la primera sentencia de *login,* tenemos en *user* un objeto recuperado de la base de datos, cuyo *counter* es *null, email* es pepe@pepe.com, su *id* empieza por "f29b", su *name* es Pepe y su contraseña encriptada comienza por "315d803" (Figura 438). Vemos, además, que el valor de la *pwd* que se ha recibido como parámetro es "pepe12345".

```
25     public User login(String name, String pw
26         User user = this.userDAO.findByName(
27         if (user==null)
28             return null;
29         if (user.getCounter()!=null && user.
30             throw new ResponseStatusExceptio
31         pwd = org.apache.commons.codec.diges
32         if (!user.getPwd().equals(pwd)) {
33             user.setCounter(user.getCounter(
34             this.userDAO.save(user);
35             user = null;
36         } else {
37             user.setCounter(0);
```

```
pwd      "pepe12345" (id=138
user     User  (id=207)
  counter   null
  email     "pepe@pepe.com" (i
  id        "f29b6563-e69c-493
  name      "Pepe" (id=211)
  pwd       "315d80320c7cbf5b3
```

Figura 438. Hemos recuperado al user *con nombre Pepe*

En la línea 27 el objeto *user* no es *null,* por lo que se comprobará el valor de su campo *counter*. Llegamos a la línea 31, en donde se encripta la contraseña recibida, que es distinta de la del usuario (Figura 439).

```
pwd      "29f58c
user     User  (id
  counter   null
  email     "pepe@
  id        "f29b65
  name      "Pepe" (
  pwd       "315d80
```

Figura 439. La contraseña recibida, tras encriptarse, es distinta de la contraseña encriptada real

Como no son iguales, se llega a la línea 33, que intenta incrementar en 1 el campo *counter* del usuario (Figura 440): y justo es aquí donde la aplicación "peta", pues el campo *counter*, como acabamos de ver en la figura anterior, es *null,* y no podemos incrementar en 1 un valor *null*.

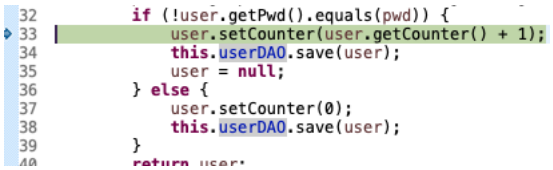


Figura 440. Se intenta incrementar el campo counter *del* user

Encontrado el error, podemos parar tranquilamente los dos procesos y corregirlo. Como siempre, hay varias maneras: podríamos asignar a la columna *counter* de la entidad *User* el valor cero por defecto; o, también, y es la solución por la que optaremos, consultar antes de intentar actualizar el valor

```
if (!user.getPwd().equals(pwd)) {
    if (user.getCounter()==null)
        user.setCounter(1);
    else
        user.setCounter(user.getCounter() + 1);
    this.userDAO.save(user);
    user = null;
} else {
    user.setCounter(0);
    this.userDAO.save(user);
}
```

Figura 441. Corrección de un error detectado

Ahora, como esperábamos, los casos de prueba se superan:

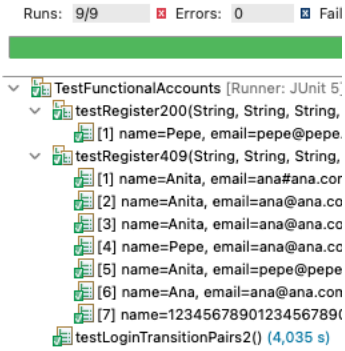


Figura 442. Finalmente, todos los casos de prueba exhiben veredicto de paso

Capítulo 19. El caso de uso *removeUser*. Peticiones CORS

1. Caso de uso *removeUser*

Para finalizar con la gestión de usuarios en el lado cliente que vamos a desarrollar por ahora, nos falta simplemente escribir el código correspondiente al caso de uso *removeUser,* que elimina la información del usuario de la base de datos. Recordará el lector que, para eliminarse, el usuario debe estar logueado previamente.

En este caso, y simplemente por variar, incrustaremos el componente *remove-user* en *login. component.html,* en lugar de en *app.component.html.*

```
7   <a>Recover password</a>
8   <br>
9   <app-remove-user></app-remove-user>
10  <br>
11  <span style="color: red">{{ error }}</span>
```

Figura 443. Colocamos la etiqueta <app-remove-user> *en el código de* login.component.html

El código del nuevo componente es muy sencillo:

```
<a
(click)="askForConfirmation()">
   Remove account</a>
```

```
import { Component, OnInit } from '@angular/core';
import { UserService } from '../user.service';

@Component({
  selector: 'app-remove-user',
  templateUrl: './remove-user.component.html',
  styleUrls: ['./remove-user.component.css']
})
export class RemoveUserComponent implements OnInit {

  constructor(private userService : UserService) { }

  ngOnInit(): void {
  }

  askForConfirmation() {
    if (confirm("Remove account?")) {
      this.removeUser();
    }
  }

  private removeUser() {
    this.userService.removeUser().subscribe({
      next : () => alert("Account removed")
    })
  }
}
```

Figura 444. Vista y modelo de removeUser

Como se ve en la figura anterior, el enlace de la vista ejecuta el método *askForConfirmation* del modelo, que muestra un cuadro de diálogo para pedir confirmación. Si se dice que sí, se envía el mensaje al *backend* mediante el servicio.

1.1. Prueba manual del caso de uso

Para probar la funcionalidad, debemos registrar a un usuario, luego loguearlo para que su *userId* se conoce en la *HttpSession* y, después, pulsar en el

enlace *Remove account*. Cuando la petición llegue al recurso, se recuperará el *userId* de la sesión y le pediremos al *userService* que, a su vez, borre al usuario de la tabla.

Sin embargo, el método *removeUser* del controlador no encuentra ningún objeto *userId* en la *session,* a pesar de que sí que lo hemos colocado al ejecutar *login* (Figura 368). El problema es parecido al que encontramos alrededor de la Figura 372: la segunda petición (a *removeUser)* llega asociada a una *HttpSession* diferente de la primera. Igual que tuvimos que hacer uso de la *MockHttpSession,* ahora tenemos que realizar algún cambio en el código de producción del *front* para que todas las peticiones del mismo navegador lleguen asociadas a la misma sesión.

2. Diagnóstico del error

El escenario que se está produciendo es el que se ilustra en la Figura 445, y lo explicamos en los siguientes párrafos.

1) El usuario se conecta inicialmente mediante un GET al servidor de Angular, que escucha en el puerto 4200. La comunicación entre el navegador de Pepe y el exterior se inicia en mensaje 1, por lo que el origen (*Origin)* de la comunicación es *localhost:4200*.
2) En el mensaje de respuesta 1.1, el origen envía al navegador la respuesta que corresponda (los ficheros *html* y *ts* que estamos escribiendo), entre los que se encuentra el componente *login*.
3) En el mensaje 2, el usuario rellena sus credenciales para loguearse. La petición se envía a *localhost:80* mediante un PUT. En la *payload* de la petición van las credenciales de acceso en forma de JSON; como el navegador detecta que la petición se dirige a un host distinto del origen, añade automáticamente a la sección de cabeceras la cabecera *Origin*, a la que le asigna como valor el nombre del host que inició la comunicación.
4) *localhost:80* recibe la petición. Como el *UserController* tiene la anotación *@CrossOrigins("*")* (sección 1.2 del Capítulo 16, página 214), el controlador acepta la petición: crea una sesión nueva en el mensaje 2.1 (la llamamos "abc", por ejemplo), busca al usuario en la base de datos y coloca en la sesión "abc" el *id* del usuario Pepe.
5) En el mensaje de respuesta 2.5, el servidor devuelve un código 200 y una cabecera con una cookie en la que ha guardado el identificador de la sesión.
6) Acto seguido, el usuario pulsa *removeUser* en el mensaje 3: el navegador envía la petición con el método DELETE, y esperamos que le añada a la sección de cabeceras la cookie con el identificador de la sesión, pero el navegador no la añade.
7) Cuando *localhost:80* recibe la nueva petición sin *id* de sesión, los interceptores no son capaces de inyectar ningún objeto *HttpSession* preexistente en el parámetro *session* de *removeUSer,* por lo por lo que crean una nueva, que es la que le inyectan al argumento.
8) En el mensaje 3.2, el controlador busca el atributo *userId* en la sesión y lo intenta pasar a String. Como no hay nada, salta una *NullPointerException* y se devuelve un error 500.

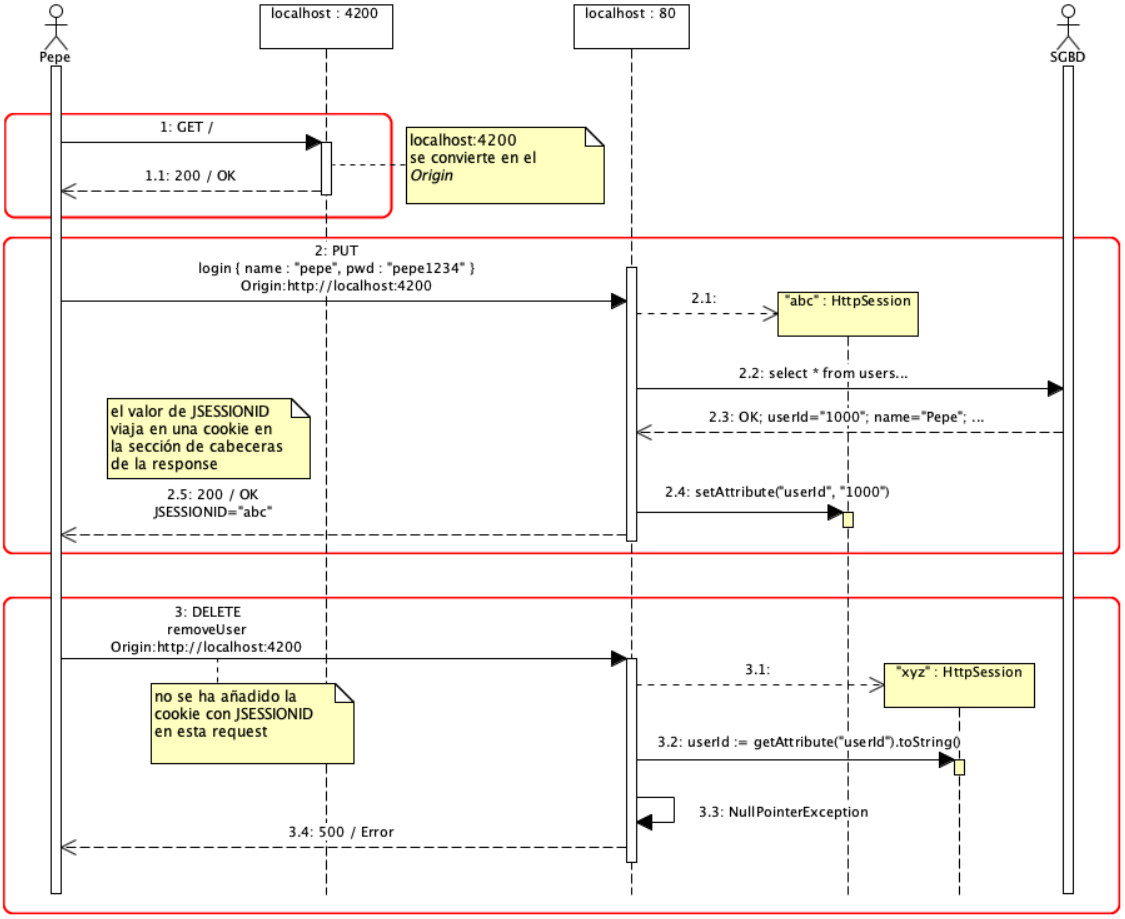


Figura 445. Lo que está ocurriendo

El error ocurre porque la petición DELETE no lleva la cookie de sesión que se le entregó al navegador en la respuesta a la petición PUT. En la primera petición, el navegador añadió correctamente la cabecera *Origin* en la petición (Figura 446, abajo, sección *Request Headers*) y recibió la cookie de sesión (Figura 446, arriba, en las *Response Headers*), pero en la segunda petición no la envía porque va dirigida a un servidor distinto del origen.

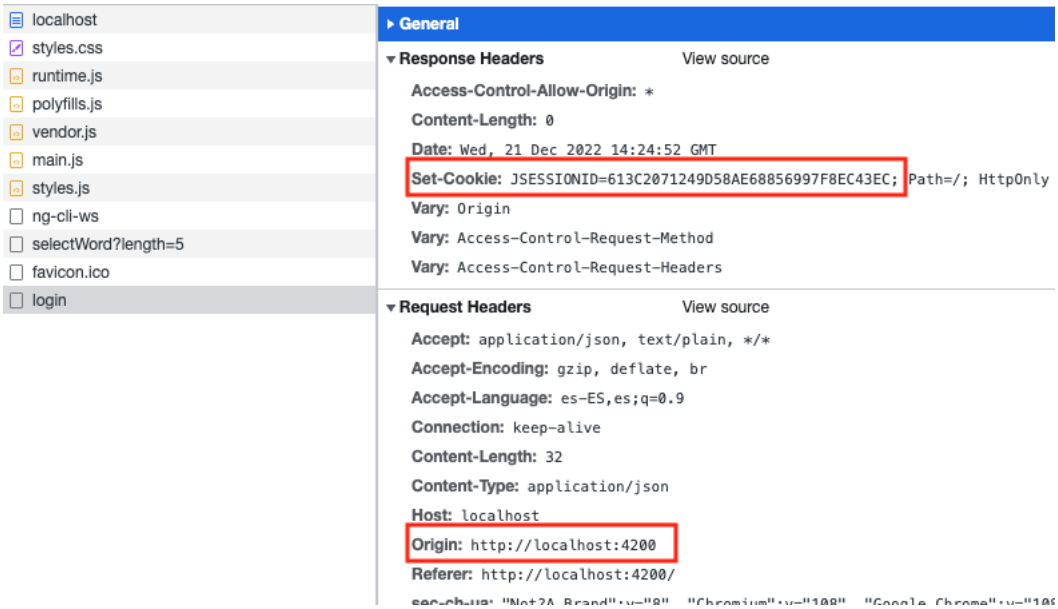


Figura 446. Abajo, el navegador incluye la cabecera Origin *en la petición; arriba, se recibe la cookie de sesión*

3. Peticiones CORS (*Cross-Origin Resource Sharing)*

Como ya vimos en la sección 4.1 del Capítulo 8 (página 145), una petición CORS es aquella que el cliente envía a un servidor distinto del origen. Salvo que dispongamos otra cosa, el navegador rechazará las cookies que reciba de fuera del origen. Por eso, en el escenario anterior, la cookie se recibe pero no se procesa.

Para enviar una petición que reciba y procese cookies a un servidor distinto del origen, necesitamos cumplir varios requisitos tanto el lado cliente como en el servidor:

1) Se debe enviar con un campo llamado *withCredentials* establecido a *true,* entendiendo que esas "credenciales" son las cookies que el servidor pueda enviarnos. Poner ese campo a *true* es como decirle al servidor: «Oye, que admito cookies».
2) El servidor debe admitir también peticiones con credenciales. Para esto, debe añadir a la respuesta una cabecera *Access-Control-Allow-Credentials* con valor *true*.
3) El servidor debe asignar a la cabecera *Access-Control-Allow-Origin* de la respuesta el host origen real, no valiendo (como en la cabecera *Access-Control-Allow-Origin* de la figura anterior, el asterisco (*). En nuestro ejemplo, el host debería enviar: *Access-Control-Allow-Origin : http://localhost:4200,* pues este es el host con el cual el navegador inició la comunicación.

Hacen falta algunos requisitos más que vamos a ver poco a poco.

De momento, hagamos los cambios en los lados cliente y servidor:

- En el cliente, modificamos las dos peticiones que reciben necesitan manipular la cookie de sesión: *login*, porque debe recibirla y procesarla; *removeUser,* porque debe enviarla.

```
login(user : any) : Observable<any> {
  return this.client.put<any>(httpUrl + "users/login", user, { withCredentials : true})
}

removeUser() : Observable<any> {
  return this.client.delete<any>(httpUrl + "users/removeUser", { withCredentials : true})
}
```

Figura 447. Enviamos las peticiones con credenciales

- En el lado del servidor, especificamos que *http://localhost:4200* es el único origen permitido, y que este controlador admite peticiones con credenciales.

```
@RestController
@RequestMapping("users")
@CrossOrigin(origins = "http://localhost:4200", allowCredentials = "true")
public class UserController {

    @Autowired
    private UserService userService;
    ...
}
```

Figura 448. Cambios en la anotación @CrossOrigin

3.1. Envío de la petición *login*

Cuando ahora ejecutamos la aplicación desde el navegador e intentamos loguearnos, podemos ver en la solapa *Network* de las herramientas de desarrollo del navegador que este ha enviado dos peticiones al recurso (Figura 449):

1) La primera es una petición con el método OPTIONS y con ella el navegador le pregunta al servidor por el tipo de peticiones CORS que admite.

Estado	Método	Dominio	Archivo
200	OPTIONS	localhost	login
200	PUT	localhost	login

Figura 449. Petición de preflight *para* login *y petición real*

La respuesta a esta *preflight* es un código 200 y la siguiente serie de cabeceras (Figura 450):

a. En la primera nos dice que nuestro origen está autorizado.
b. En la segunda, que admite credenciales.
c. En la tercera, que admite peticiones que lleven la cabecera *content-type*. Si la petición real (la que se va a enviar a continuación) lleva una cabecera manual distinta (con "manual" queremos nos referimos a aquellas cabeceras que el programador puede añadir manualmente; hay algunas que el navegador añade automáticamente, como *Connection* o *User-Agent),* el servidor rechazará la petición.
d. En la cuarta, el servidor nos dice que el método PUT que queremos enviar en la petición real está autorizado.
e. La cabecera *Allow* indica los métodos que soporta este recurso, pero no necesariamente que todos ellos estén disponibles para peticiones CORS.

f. Las tres cabeceras *Vary* indican que el comportamiento del recurso puede variar en función del origen y del método o cabeceras enviadas.
g. Las dos últimas, que no hay cuerpo de respuesta y la fecha y hora en que se creó la respuesta.

```
HTTP/1.1 200
Access-Control-Allow-Origin: http://localhost:4200
Access-Control-Allow-Credentials: true
Access-Control-Allow-Headers: content-type
Access-Control-Allow-Methods: PUT
Access-Control-Max-Age: 1800
Allow: GET, HEAD, POST, PUT, DELETE, OPTIONS, PATCH
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Content-Length: 0
Date: Wed, 21 Dec 2022 18:12:57 GMT
```

Figura 450. Respuesta a la petición de preflight

2) Como el método está autorizado, el navegador envía la petición real (la segunda que aparecía en la Figura 449, con el método PUT). En la respuesta a este método se recibe la cabecera *Set-Cookie* con el valor del *id* de la *HttpSession* que se ha creado en el servidor.

3.2. Envío de la petición *removeUser*

Como las dos peticiones se envían con credenciales y hemos modificado la anotación *@CrossOrigin,* antes de enviar la petición real a *removeUser,* se envía también una *preflight* con el método OPTIONS en la que el navegador pregunta al servidor que si le admitirá la ejecución de una petición de tipo DELETE sobre el recurso *removeUser*. La petición de *preflight* (Figura 451) lleva, en efecto, el nombre del recurso en la url y el tipo de método que desea ejecutar en la cabecera *Access-Control-Request-Method*. También, claro, al ir dirigida a un servidor distinto del origen, incluye la cabecera *Origin*.

```
OPTIONS /users/removeUser HTTP/1.1
Host: localhost
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:107.0) Gecko/20100101
Firefox/107.0
Accept: */*
Accept-Language: es-ES,es;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate, br
Access-Control-Request-Method: DELETE
Referer: http://localhost:4200/
Origin: http://localhost:4200
...
```

Figura 451. Preflight *que se envía para ejecutar* removeUser

Igual que con la *preflight* anterior, el servidor nos responde con un 200 y, entonces, el navegador envía la petición real (de tipo DELETE) al servidor. Como vemos en la Figura 452, se solicita el mismo recurso que en la *preflight* (*/users/removeUser*), pero con un método distinto (DELETE frente a OPTIONS). Se incluye también la cabecera *origin* y, por fin, el valor de la cookie de sesión que nos devolvió el servidor al hacer el login.

```
DELETE /users/removeUser HTTP/1.1
Host: localhost
...
Origin: http://localhost:4200
Connection: keep-alive
Referer: http://localhost:4200/
Cookie: JSESSIONID=8ED0CE046232DF34270EEF4C9CA2B397
...
```

Figura 452. Envío de la petición real

El problema que planteamos al principio de este capítulo y que ilustramos en la Figura 445 queda así solucionado.

4. Reejecución de las pruebas

Los cambios, aunque pequeños, han sido bastante significativos, pues han cambiado el mecanismo de acceso a varios recursos web. Ejecutaremos de nuevo todos los casos para comprobar que no se han introducido errores que antes no existían. A este tipo de pruebas se las llama "pruebas de regresión".

Bien, pues botón derecho sobre *src/test/java* y ejecutamos *Run As JUnit Test*. Recordemos que debemos tener todo arrancado: el 4200, el 80 y el docker. Podemos comprobar que todos los casos se superan.

5. Filtrado de métodos

Por defecto, la anotación *@CrossOrigin* da permiso para ejecutar cualquier tipo de método (hemos visto que se nos ha dado acceso a métodos POST, como *register,* a PUT, como *login* y a DELETE, como *removeUser*) ofrecido por el controlador. De hecho, en el *javadoc* del atributo *methods* (que vamos a comentar inmediatamente) de esta anotación, se nos dice que, por defecto, los métodos soportados son los mismos mapeados en el controlador (*By default the supported methods are the same as the ones to which a controller method is mapped*): en este caso, *methods* se refiere a los "verbos" (GET, POST, etcétera) utilizados en el controlado. Por tanto, por defecto, añadir la etiqueta *@CrossOrigin* habilita la ejecución de todos los tipos de métodos http declarados en el controlador.

Supongamos que, por algún motivo, queremos impedir que el servicio *register* del *UserController* pueda ser invocado mediante una petición *CORS*.

Lo que haremos será especificar en la anotación *@CrossOrigins* que los métodos permitidos para peticiones de origen cruzado son solo PUT (para *login)* y DELETE (para *removeUser).* Esto lo conseguimos añadiendo estos dos métodos al atributo *methods* de la cabecera (Figura 453).

```
@RestController
@RequestMapping("users")
@CrossOrigin(origins = "http://localhost:4200",
   allowCredentials = "true",
   methods = { RequestMethod.PUT, RequestMethod.DELETE}
)
public class UserController {
   ...
}
```

Figura 453. Listamos los métodos permitidos en la anotación @CrossOrigin

Igual que antes, al enviar la petición *register* se envía primero una *preflight* en la que preguntamos al servidor: «Oye, ¿puedo enviar una petición al recurso *user/register* con el método POST?». El servidor responde que no; aun así, tanto

Chrome (Figura 454, arriba a la izquierda) como *Firefox* (arriba a la derecha) envían la petición real al servidor, que obviamente la rechaza; el navegador Safari (abajo) parece no intentar enviar la petición real al obtener el *forbidden* previo como respuesta a la *preflight*.

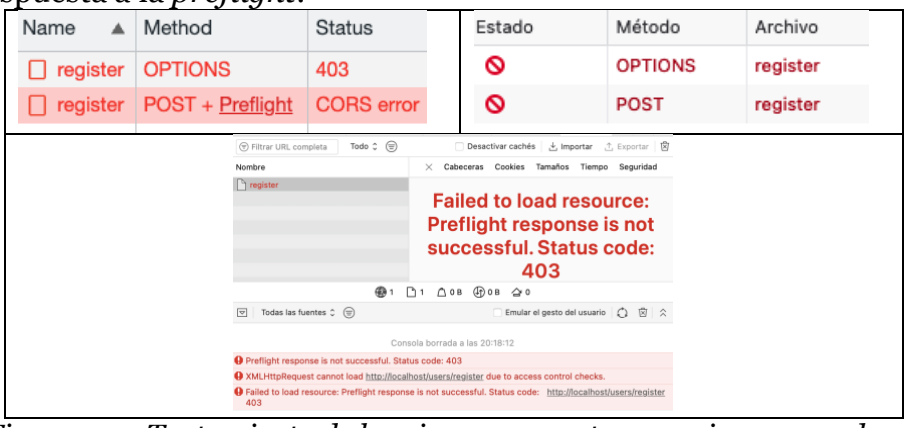


Figura 454. Tratamiento de la misma respuesta en varios navegadores

6. Filtrado de cabeceras

Podríamos suponer ahora que, en las cabeceras con las que respondemos al *login,* añadimos una cabecera *Current* con el *id* de usuario codificado con, digamos, Base64. Base64 es un mecanismo que nos permite encriptar cualquier elemento a una cadena, y desencriptarla a su formato original: por ejemplo, una imagen, un audio, un vídeo o también una cadena pueden "traducirse" a Base64, y descodificarse después para obtener el objeto original. Es diferente del algoritmo SHA-512 que usamos para encriptar la contraseña, pues el hash que obtenemos con este no es desencriptable salvo por algún método de fuerza bruta.

Para ejecutar *removeUser,* esperamos que el cliente nos envíe una cabecera que, por ejemplo, se llame *Current64* que contenta el *id* del cliente en Base64 que le hemos enviado previamente. Si no llegan ambos parámetros o no concuerdan, lanzaremos un mensaje con código 400 con el texto "Se ha producido un error de seguridad".

6.1. Personalización de la *response*

Igual que los interceptores de Spring nos inyectan la sesión en el parámetro *HttpSession,* también pueden inyectarnos la respuesta que vamos a enviar si declaramos algún parámetro de tipo *HttpServletResponse*.

En la Figura 455, Spring nos inyecta este objeto en el argumento *response;* más abajo, tras comprobar que las credenciales son correctas y añadir la sesión a la lista de sesiones que mantiene el *Manager* (recuérdese Figura 411, página 257), codificamos el *userId* con Base64 y lo añadimos a la cabecera *Current* del objeto *response*.

```
@PutMapping("/login")
public void login(HttpSession session, HttpServletResponse response,
        @RequestBody Map<String, Object> info) {
   String name = info.get("name").toString().trim();

   ...

   Manager.get().addSession(user.getId(), session);
   String base64 = java.util.Base64.getEncoder().
               encodeToString(user.getId().getBytes());
   response.setHeader("Current", base64);
}
```

Figura 455. Adición de una cabecera a la respuesta

Como vemos en la Figura 456, la respuesta se recibe en el navegador.

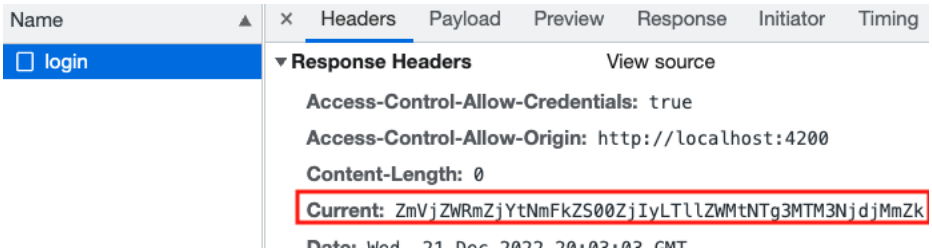


Figura 456. Recibimos la cabecera que hemos enviado

Sin embargo, en el navegador no podremos leerla salvo que lo indiquemos expresamente en el servidor: al tratarse de una petición CORS, debemos añadir una cabecera adicional (*Access-Control-Expose-Headers*) en la que le decimos al navegador a qué cabeceras puede tener acceso.

```
@PutMapping(value = "/login")
@CrossOrigin(exposedHeaders = "Current")
public void login(HttpSession session, ...) {
   ...
}
```

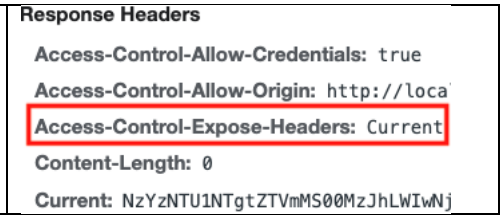


Figura 457. Acceso a la cabecera Current

6.2. Tratamiento de la *response* y personalización de la *request*

Cuando el navegador recibe una respuesta con código 200 a la petición *login,* debe anotar en algún lado el valor de la cabecera *Current* para, luego, enviarla con el nombre *Current64* al ejecutar *removeUser*.

Como recordará el lector, la respuesta a la operación *login* del *UserService* en el *front* es un *Observable<any>,* que se procesaba en el modelo del componente *login*. Hasta ahora, al recibir un código 200, en este método solo hacíamos un triste *alert("OK")* (Figura 428, página 269) que ya debemos empezar a modificar.

Lo que haremos será guardar el valor de la cabecera *Current* (que contiene el *userId* en *base64)* en algún medio de almacenamiento que posea el navegador. Este posee al menos tres:

1) Las cookies, que provienen de un servidor. La característica de las cookies es que, en cada nueva petición a ese mismo servidor, las cookies que él (el servidor) nos dejó se le envían de nuevo.
2) El *sessionStorage*, en donde podemos almacenar todo lo que queramos (cada navegador impone un límite en bytes, si bien suele ser bastante generoso) en forma de pares *(clave, valor),* donde tanto la clave como el

valor son cadenas. Cuando cerramos el navegador, la información que tengamos aquí almacenada desaparece.

3) El *localStorage*, en donde podemos también almacenar todo lo que queramos y de la misma forma que en el *sessionStorage*. La información que almacenemos aquí permanecerá en el ordenador local del usuario hasta el fin de los tiempos (salvo que se borre explícitamente).

La diferencia principal entre estos dos últimos mecanismos de almacenamiento y las cookies es que estas se envían siempre al servidor que nos las dejó, mientras que la información contenida en los *storages* no se manda a ningún lado, salvo que lo programemos explícitamente.

6.2.1. Tratamiento de la *response*

Cuando a la petición enviada al recurso *login* le llega una *response* con código 200 (es decir, las credenciales enviadas han sido correctas), el cliente debe recuperar el valor de la cabecera *Current* para enviarlo posteriormente.

Con la implementación actual que tenemos del método *login* en el *UserService* del cliente, una petición con código 200 nos devuelve *null* en el resultado de la petición (Figura 458).

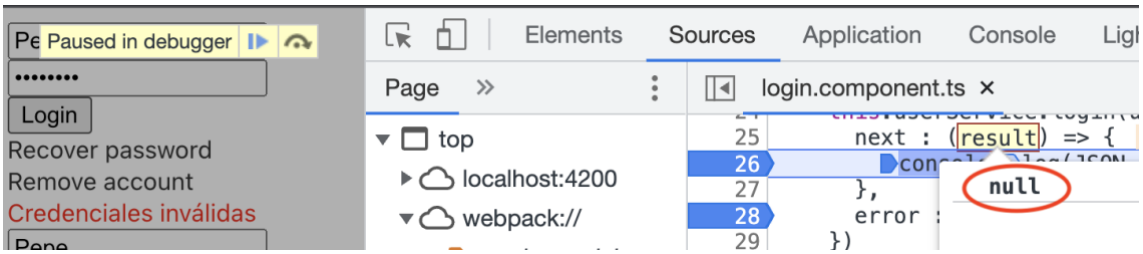


Figura 458. La respuesta que obtenemos es null: *¿y las cabeceras y el código de respuesta?*

Esto sucede porque, por defecto (es decir, si no decimos otra cosa), Angular solo nos entrega el cuerpo de la respuesta, no haciéndonos visible ni el código de respuesta ni las cabeceras. Y, claro, como el servicio *login* en el controlador es *void*, Angular nos entrega un *null*. Sí que sabremos que nos ha llegado un código de la serie del 200 porque, en la figura anterior, se está ejecutando el manejador de eventos asociado a *next*.

Pero podemos modificar los métodos *get, put,* etcétera que Angular nos ofrece para acceder a más contenidos de la *response*. Si al método que corresponda le añadimos el parámetro *observe: "response"*, podemos acceder a la respuesta completa (Figura 459), lo que incluye el código de respuesta, el mensaje de estado, etcétera.

```
login(user : any) : Observable<any> {
  return this.client.put<any>(httpUrl + "users/login", user,
    { withCredentials : true, observe : "response" })
}
```

Figura 459. Le decimos a Angular que queremos acceso a toda la respuesta

La operación *login* de la figura anterior le devuelve al método *login* de *UserComponent* un observable con la respuesta que, ahora, está completa. En el código de la Figura 460 leemos la cabecera *Current* y la almacenamos en el *sessionStorage*.

```
login() {
  let user = {
    name : this.name,
    pwd : this.pwd
  }
  this.userService.login(user).subscribe({
    next : (response) => {
      sessionStorage.setItem("current", response.headers.get("current"))
    },
    error : (result) => this.error = result.error.message
  })
}
```

Figura 460. Leemos la cabecera Current *y la guardamos en el* sessionStorage

6.2.2. Personalización de la *request*

Según la especificación que se ha dado, las peticiones que se envíen a *removeUser* deben llevar una cabecera llamada *Current64* con el *id* del usuario en base 64. Lo único que necesitamos es añadir la cabecera a esta petición, como hacemos en la Figura 461: construimos un objeto de tipo *HttpHeaders* y lo añadimos.

```
removeUser() : Observable<any> {
  let headers = new HttpHeaders().
    set("Current64", sessionStorage.getItem("current")!)

  return this.client.delete<any>(httpUrl + "users/removeUser",
    {
      withCredentials : true,
      headers
    })
}
```

Figura 461. Añadimos una cabecera a la petición

6.3. Tratamiento de la *request* en el servidor

En el método *removeUser* del controlador debemos leer la cabecera *Current64* que nos está enviando el cliente. Las cabeceras vienen en la propia petición, por lo que necesitamos que Spring nos inyecte el objeto con la petición en el método. Para conseguirlo, basta con declararlo como un parámetro de tipo *HttpServletRequest*; luego, accedemos a ese objeto y leemos su valor (Figura 462).

```
@DeleteMapping("/removeUser")
public void removeUser(HttpSession session, HttpServletRequest request) {
   String userId = session.getAttribute("userId").toString();
   String base64 = request.getHeader("current64");
   if (base64==null)
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
         "Se ha producido un error de seguridad");
   base64 = new String(java.util.Base64.getDecoder().decode(base64));
   if (!base64.equals(userId))
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
         "Se ha producido un error de seguridad");

   this.userService.removeUser(userId);
}
```

Figura 462. Inyección de la request *y acceso a la cabecera*

Por cierto, que a la *session* podemos acceder también a partir de la *request*, por lo que el método de arriba lo podríamos reescribir de esta manera:

```
@DeleteMapping("/removeUser")
public void removeUser(HttpServletRequest request) {
   HttpSession session = request.getSession();
   String userId = session.getAttribute("userId").toString();
   ...
}
```

Figura 463. Accedemos a la session *a través de la* request

7. Pruebas del caso de uso

El caso de prueba de la Figura 464 prueba el caso de uso que acabamos de desarrollar y lo hemos añadido con *@Order(4)* a *TestFunctionalAccounts:*

1) Como la cuenta de Pepe estará bloqueada por múltiples intentos de *login* fallido, lo desbloqueamos poniéndole el *counter* a cero, accediendo directamente a la base de datos a través del *userDAO*.
2) Luego, lo logueamos con sus credenciales buenas.
3) Finalmente, pulsamos el enlace para eliminar la cuenta. Aparecerá el diálogo de confirmación y aceptamos. Para comprobar que la cuenta ha sido eliminada, buscamos a Pepe en la base de datos y comprobamos que sea la respuesta que de devuelve el repositorio es *null*.

```
@Test @Order(4)
void testRemoveAccount() throws Exception {
   User user = this.userDAO.findByName("Pepe");
   user.setCounter(0);
   this.userDAO.save(user);

   WebElement inputName =
      driver.findElement(By.xpath("/html/body/app-root/app-login/input[1]"));
   WebElement inputPwd =
      driver.findElement(By.xpath("/html/body/app-root/app-login/input[2]"));
   WebElement buttonLogin =
      driver.findElement(By.xpath("/html/body/app-root/app-login/button"));

   inputName.clear(); inputPwd.clear();
   inputName.sendKeys("Pepe"); inputPwd.sendKeys("pepe1234");
   buttonLogin.click();
   pause(1000);

   WebElement linkRemove = driver.findElement(
      By.xpath("/html/body/app-root/app-login/app-remove-user/a"));
   linkRemove.click();

   Alert alert = driver.switchTo().alert();
   alert.accept();

   assertNull(this.userDAO.findByName("Pepe"));
}
```

Figura 464. Prueba de removeAccount *con Selenium*

QUINTA PARTE: WEBSOCKETS PARA LOS JUEGOS MULTIJUGADOR

Capítulo 20. *Otn* para dos jugadores: *backend* (I)

1. Análisis y diseño de la solución

La Figura 465 ya apareció en el Capítulo 14, pero la reproducimos de nuevo por su interés para este momento del desarrollo.

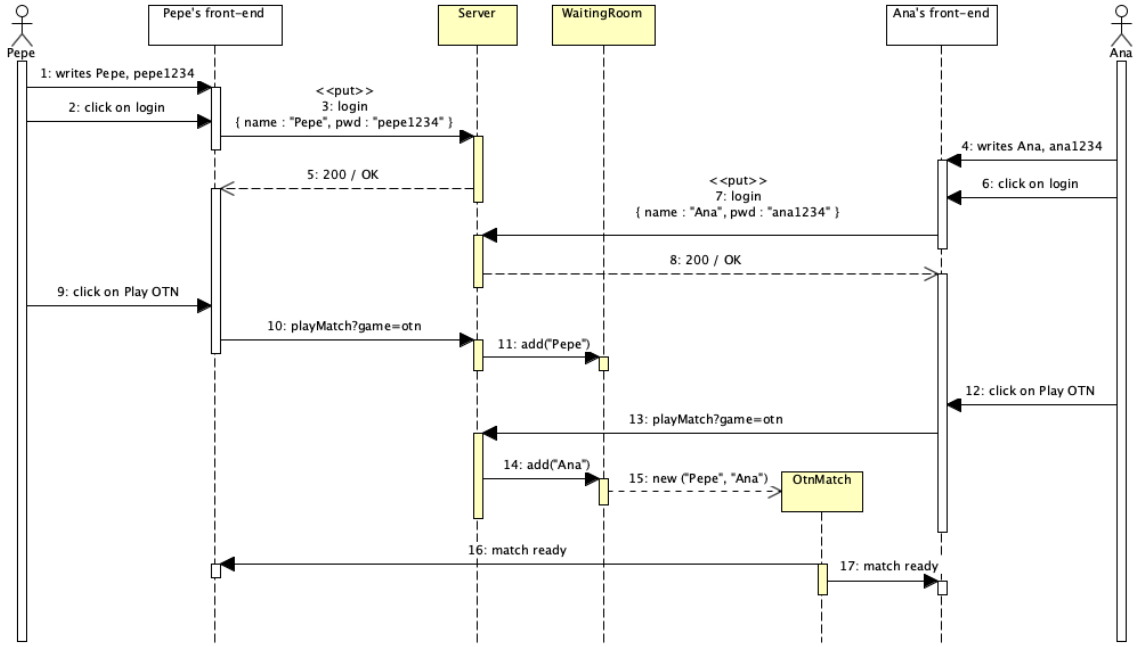


Figura 465. Paso de mensajes para iniciar una partida entre dos jugadores

Nos interesa fijarnos en la secuencia de mensajes que sucede a partir del nº 9, cuando Pepe hace clic en jugar al *Order the numbers* en modo multijugador: a algún controlador del servidor le llega un mensaje que indica que este jugador, que tiene una sesión creada bien conocida y que está bien controlada gracias a todo lo que ya sabemos de las peticiones CORS, desea iniciar una partida. Implementaremos, para esta responsabilidad, un *GamesController* que, por ahora, se encargará de gestionar las peticiones de inicio de partida.

2. Implementación del *GamesController*

En esta sección analizaremos dos alternativas de implementación para el servidor. En ambas habrá una sala de espera implementada en una clase *WaitingRoom:*

1) En la primera alternativa, la clase *WaitingRoom* guardará los jugadores que están esperando a la llegada de un oponente.
2) En la segunda, la sala de espera guardará una lista de las partidas que están pendientes (es decir, aquellas en las que un jugador que está esperando).

2.1. Alternativa 1: sala de espera de jugadores

El *GamesController* ofrece un servicio *playMatch* que recibe un parámetro *name* con el nombre del juego. El siguiente diagrama de secuencia es un

fragmento del mostrado en la Figura 465, que ilustra a alto nivel lo que debe suceder en el *back* cuando llegan dos jugadores:

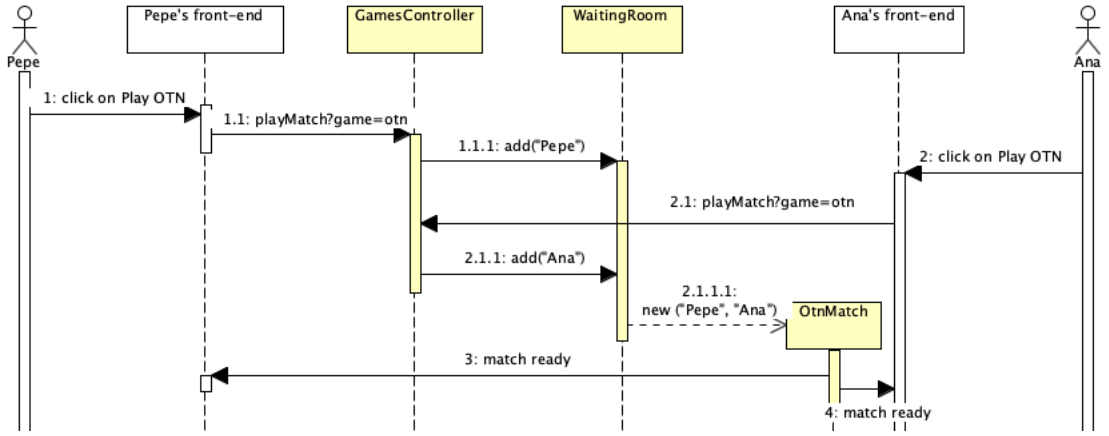


Figura 466. Paso de mensajes con la alternativa 1

Según este paso de mensajes, el *GamesController* debe conocer a la *WaitingRoom,* sobre la que ejecutará la operación para añadir al usuario a la sala de espera. En lugar de añadir el *name* del usuario (que es lo que figura en el diagrama para evitar enturbiarlo en exceso y que pierda la legibilidad que se le espera), en la implementación añadiremos el objeto *user* completo: en la sesión tenemos su *id* y, mediante un *userDAO,* podemos recuperarlo de la base de datos. En la figura, se ilustra esa relación entre el controlador y la sala de espera. Esta almacena los usuarios que están esperando, en una tabla *hash* indexada por el nombre del juego.

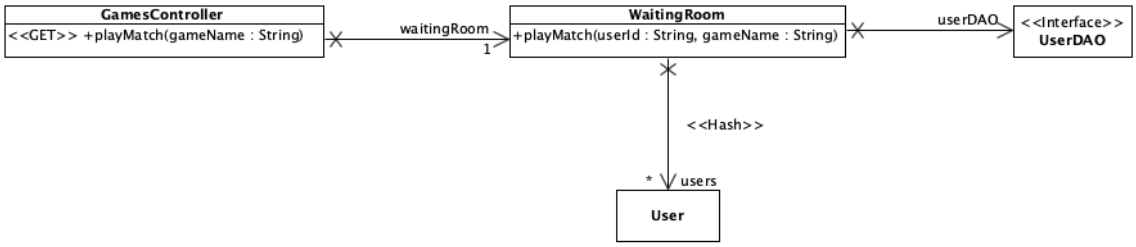


Figura 467. Alternativa 1 para la sala de espera

Este controlador lo colocaremos en el mismo paquete en el que tenemos el *UserController* y el *WordRaceController*. Su implementación, de momento, es muy sencilla. Obsérvese, en la Figura 468, que:

1) Anotamos el campo *WaitingRoom* con *@Autowired,* lo que indica que será un campo que Spring instanciará automáticamente al arrancar. Como va a realizar operaciones de negocio importantes, la clase *WaitingRoom* la anotaremos con *@Service*.
2) La operación *play* devuelve al cliente un resultado de tipo *Match,* que representa una partida de cualquier cosa. Realmente, los dos juegos de tablero que ofrecemos en modo competición van a ser muy similares, por lo que la situación invita a crear una clase abstracta *Match*, de la que luego probablemente hereden *OtnMatch* y *WordRaceMatch*.

```java
@RestController
@RequestMapping("games")
@CrossOrigin(origins = "http://localhost:4200",
   allowCredentials = "true"
)
public class GamesController {

   @Autowired
   private WaitingRoom waitingRoom;

   @GetMapping("/playMatch")
   public Match playMatch(HttpSession session, @RequestParam String gameName) {
      String userId = session.getAttribute("userId").toString();
      return this.waitingRoom.playMatch(userId, gameName);
   }

}
```

Figura 468. Primeras líneas del GamesController

La *WaitingRoom*, como decíamos, es un *@Service*. Debe conocer a un *userDAO* para recuperar el usuario a partir de su *id,* y ha de tener una tabla *hash* en la que guardar los jugadores que están esperando para jugar a cada juego. Puede pensarse que, puesto que solo hay dos juegos, bastaría con tener dos campos para representar a los dos posibles jugadores que haya en espera: pero, igual que hemos propuesto crear la clase abstracta *Match,* también aquí podemos anticiparnos un poco al futuro y pensar que tal vez se ofrezcan más adelante otros juegos. Por eso, en lugar de un campo por tipo de juego, los guardamos todos indexados por el nombre del juego para el que están esperando.

Por otro lado, ya debemos considerar también la gestión de los jugadores que están esperando: cuando el *@Service* ejecute su *playMatch,* buscará en la lista de jugadores en espera (campo *users)* un jugador que quiera jugar al mismo juego: si lo hay, lo retira, crea una partida para los dos jugadores y le devuelve al controlador la partida recién creada; si no, coloca al recién llegado en *users* y devuelve *null*.

```java
@Service
public class WaitingRoom {
   @Autowired
   private UserDAO userDAO;
   private ConcurrentHashMap<String, User> users;

   public WaitingRoom() {
      this.users = new ConcurrentHashMap<>();
   }

   public Match playMatch(String userId, String gameName) {
      Match match = null;
      User user = this.userDAO.findById(userId).get();
      User opponent = this.users.remove(gameName);
      if (opponent==null) {
         this.users.put(gameName, user);
      } else {
         match = Match.newMatch(gameName, user, opponent);
      }
      return match;
   }

}
```

Figura 469. La sala de espera con la alternativa 1

Por lo poco que vamos sabiendo con el código que vamos escribiendo, la clase *Match* es abstracta y dispone de un método estático *newMatch* que crea una instancia de una partida del juego pasado en su primer parámetro, *gameName*.

2.2. Alternativa 2: sala de espera de partidas

El diseño de clases para esta segunda alternativa se ofrece en la Figura 470: cuando llega el primer jugador, la sala de espera construye la instancia de clase *User* correspondiente al *userId* que ha llegado en la sesión; luego, construye una partida, le asigna a ese jugador y la coloca en una lista *pendingMatches* de partidas pendientes. Cuando llegue un segundo jugador para jugar a ese mismo juego, sacará la partida de la lista de pendientes, le asignará la instancia correspondiente al segundo jugador y realizará alguna acción adicional para que la partida pueda comenzar.

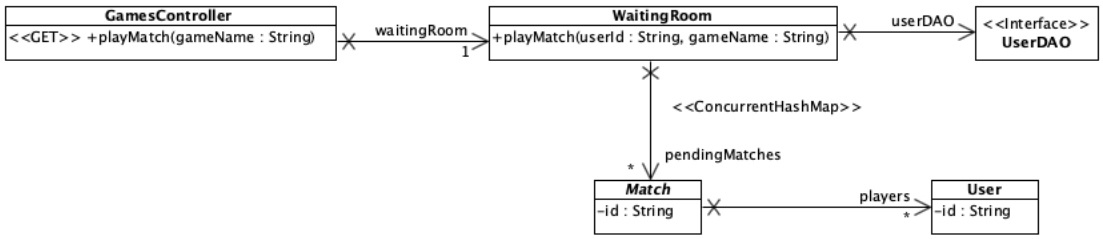


Figura 470. Diseño de clases para la segunda alternativa

Con esta estructura de clases, el escenario de la Figura 466 queda como sigue, en el que podemos resaltar un pequeño detalle de notación UML: como sabemos, la colección *pendingMatches* almacena una lista de objetos genéricos de la clase abstracta *Match;* sin embargo, en el escenario que describe la figura se crea un objeto concreto de clase *OtnMatch* que, obviamente, puede ser añadido a *pendingMatches* en el mensaje 6. Observamos, por último, que el controlador devuelve a los clientes la partida que ha creado en formato JSON.

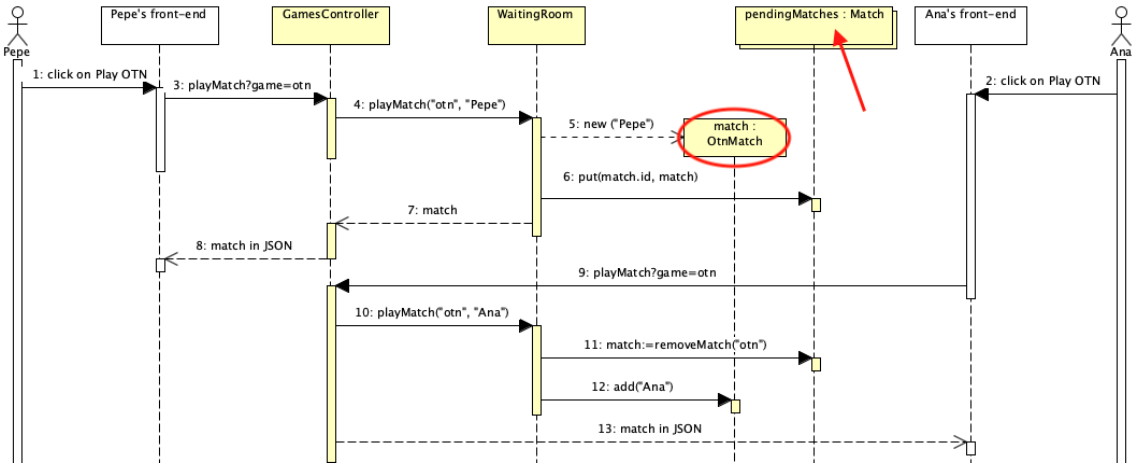


Figura 471. Paso de mensajes con la alternativa 2

Con este escenario, mientras que el código del controlador (que teníamos en la Figura 468) nos sigue sirviendo, cambia el de la *WaitingRoom*:

1) En lugar de tener una lista de usuarios, mantiene la lista *pendingMatches* con las partidas pendientes.
2) Cuando se ejecuta *playMatch,* buscamos si hay una partida pendiente para este juego:

a. Si no la hay, la creamos, y la añadimos a la lista de partidas pendientes para el juego. Podríamos haber dejado, en lugar de una lista de partidas, una sola partida, como hemos hecho en la alternativa 1 con un solo *User* en espera; pero tanto allí como aquí resulta conveniente pensar que, al ser el *backend* una aplicación que puede tener conectados muchos clientes, tal vez haya algún tipo de acceso concurrente y, en algún momento, haya más de una partida pendiente por cada juego.
b. Si sí la hay, sacamos la primera que haya en la lista de pendientes, añadimos al segundo jugador y, por ahora, la devolvemos al controlador.

```
@Service
public class WaitingRoom {
   @Autowired
   private UserDAO userDAO;
   private ConcurrentHashMap<String, List<Match>> pendingMatches;

   public WaitingRoom() {
      this.pendingMatches = new ConcurrentHashMap<>();
   }

   public Match playMatch(String userId, String gameName) {
      Match match = null;

      User user = this.userDAO.findById(userId).get();

      List<Match> matches = this.pendingMatches.get(gameName);
      if (matches==null || matches.isEmpty()) {
         matches = new ArrayList<>();
         match = Match.newMatch(gameName, user);
         matches.add(match);
         this.pendingMatches.put(gameName, matches);
      } else {
         match = matches.remove(0);
         match.add(user);
      }
      return match;
   }
}
```

Figura 472. La sala de espera con la alternativa 2

3. Implementación de *Match*. Programación reflexiva

Como no nos han dicho nada acerca de si hay que guardar en la base de datos información de las partidas, no anotaremos *Match* como *@Entity*. Al no serlo, la colocaremos en un paquete diferente, en donde residan objetos importantes que formen parte del negocio pero que no tengan por qué tener persistencia. Este paquete será *edu.uclm.esi.juegos.model*.

Por lo que podemos ir imaginando, habrá dos especializaciones de *Match*, que colocaremos también en el paquete *model*.

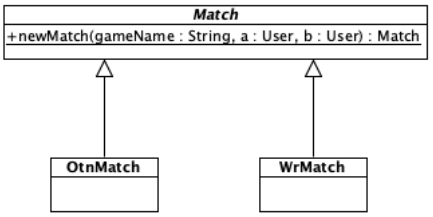


Figura 473. Especializaciones de Match

En el *@Service* creamos la partida llamando al método estático *newMatch*. Lo habitual habría sido preguntar por el valor del parámetro *gameName* y, si es "otn", crear una instancia de *OtnMatch* y, si es "wr", crearla de tipo *WrMatch,* pero nosotros hemos decidido, con las dos alternativas de implementación de la *WaitingRoom,* utilizar dicho método estático.

Java y muchos otros lenguajes permiten utilizar reflexión, que permite a los programadores interrogar a objetos sin conocerlos. Es muy probable que en el entorno de desarrollo que estemos utilizando nos hayamos fijado que, al escribir un punto después de un objeto, se nos ofrece la operación *getClass(),* que devuelve un objeto de clase *Class*. El tipo *Class* representa cualquier clase. Todas las clases tienen campos, constructores, métodos, pueden heredar… A los campos se les puede asignar valor, los constructores y métodos se pueden invocar…

En el siguiente programita de ejemplo utilizamos reflexión: en las primeras líneas creamos a nuestro conocido usuario Pepe de la forma tradicional; a continuación, con *Class<?> clazz = pepe.getClass()* guardamos en *clazz* la clase del objeto *pepe*: es decir, en *clazz* tenemos la clase *User*. En la siguiente instrucción invocamos al constructor sin parámetros de la clase que acabamos de recuperar: es decir, la sentencia *User ana = (User) clazz.newInstance()* consigue exactamente el mismo efecto que si hubiéramos escrito *User ana = new User()*.

Luego, guardamos en *fName* el campo *name* de esta clase y lo hacemos accesible (porque es *private)*. *fName* es de tipo *Field* (un campo de una clase), y le damos valor en *fName.set(ana, "Ana"),* que lo que está diciendo es que al campo *fName* se le asigne el valor "Ana" en el objeto *ana* que hemos creado unas líneas más arriba.

A continuación, cogemos reflexivamente el método *setEmail(String)* en el objeto *mSetEmail* (la "m" de "método", y "SetEmail" de que nos referimos al método *setEmail),* y lo ejecutamos sobre el objeto *ana* pasándole "ana@ana.com" como parámetro.

```
import java.lang.reflect.Field;
import java.lang.reflect.Method;

import edu.uclm.esi.juegos.entities.User;

public class Reflection {

   public static void main(String[] args) throws Exception {
      User pepe = new User();
      pepe.setName("Pepe");
      pepe.setEmail("pepe@pepe.com");
      pepe.setPwd("pepe1234");

      Class<?> clazz = pepe.getClass();

      User ana = (User) clazz.newInstance();

      Field fName = clazz.getDeclaredField("name");
      fName.setAccessible(true);
      fName.set(ana, "Ana");

      Method mSetEmail = clazz.getDeclaredMethod("setEmail", String.class);
      mSetEmail.invoke(ana, "ana@ana.com");

      System.out.println(ana.getName());
      System.out.println(ana.getEmail());
   }
}
```

Figura 474. Ejemplo de programación reflexiva

Bien, pues la operación *newMatch* creará reflexivamente una partida del juego que se le pase como primer parámetro que, de momento, es "otn" o "wr".

Con respecto a los valores de los parámetros, los nombres de las clases que hemos creado para las partidas (diagrama de clases de la Figura 473) coinciden con "otn" y con "wr", pero estos tienen su primera letra en minúscula y a aquellos les sigue el sufijo "Match". Para crear una instancia de la partida que corresponda haremos una sencilla conversión para obtener el nombre adecuado de la clase que queremos instanciar y, como hemos hecho en el ejemplo anterior, invocaremos a su constructor sin parámetros con *newInstance*.

3.1. Creación reflexiva de *Match* para la alternativa 1

Previendo que quizás más adelante ofrezcamos juegos para más de dos jugadores, el método recibe como parámetro no dos usuarios, sino una lista, que se van añadiendo a la lista de *players* de la partida cuando ha sido creada.

```
public abstract class Match {

    protected List<User> players;

    public Match() {
        this.players = new ArrayList<>();
    }

    public static Match newMatch(String gameName, User... users) {
       Match match = null;
       String firstLetter = ("" + gameName.charAt(0)).toUpperCase();
       String className = firstLetter + gameName.substring(1) + "Match";

       try {
          Class<Match> clazz =
                (Class<Match>) Class.forName("edu.uclm.esi.juegos.model." + className);
          match = clazz.newInstance();
          for (User user : users)
                match.players.add(user);
       } catch (Exception e) {
          // TODO Auto-generated catch block
          e.printStackTrace();
       }

       return match;
    }

    public List<User> getPlayers() {
        return players;
    }
}
```

Figura 475. Creación reflexiva de una partida para la alternativa 1

Gráficamente, lo que tenemos ahora, expresado en UML, es lo siguiente:

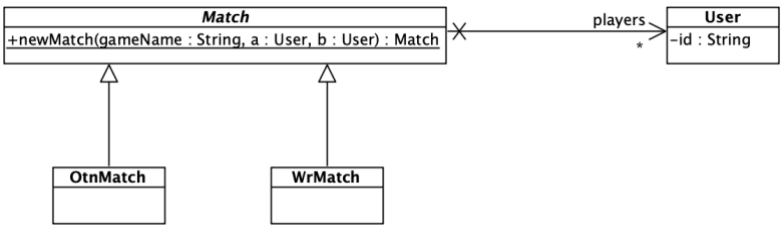


Figura 476. La partida conoce a muchos players, *que son instancias de* User

3.2. Creación reflexiva de *Match* para la alternativa 2

En este caso, el método *newMatch* seguirá recibiendo como parámetro el nombre corto del juego, pero ahora un solo usuario: creará la instancia, añadirá al usuario y la devolverá al servicio, que ya verá qué hacer con ella.

```
public static Match newMatch(String gameName, User user) {
   Match match = null;
   String firstLetter = ("" + gameName.charAt(0)).toUpperCase();
   String className = firstLetter + gameName.substring(1) + "Match";

   try {
      Class<Match> clazz =
         (Class<Match>) Class.forName("edu.uclm.esi.juegos.model." + className);
      match = clazz.newInstance();
      match.players.add(user);
   } catch (Exception e) {
      // TODO Auto-generated catch block
      e.printStackTrace();
   }

   return match;
}
```

Figura 477. Creación reflexiva de una partida para la alternativa 2

¿Y qué hará el servicio *WaitingRoom* con una partida si ya dispone del número requerido de jugadores? Pues puede, por ejemplo:

1) Añadirla a otra colección de partidas en juego que posea ella misma: en este caso, el nombre de la clase, *WaitingRoom,* despistaría, pues denota una sala de espera. Si optásemos por esto, sería recomendable cambiarle el nombre a este servicio.
2) Podría residir en otro servicio que nos inventemos: por ejemplo, *InPlayMatches*.
3) O también podemos guardarlas en el *Manager*. Esta clase podría mantener una colección de las partidas en juego indexadas por su *id* (Figura 478).

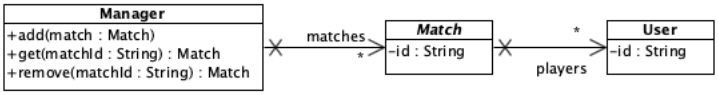


Figura 478. El Manager *dispone de la colección de partidas en juego*

4. Elección de alternativa

Supondremos que, tras varias conversaciones y discusiones entre el equipo de desarrollo, se ha pensado que la alternativa 1 no está suficientemente preparada para accesos concurrentes de múltiples usuarios que quieran jugar al mismo juego (aunque realmente el cambio sería sencillo), por lo que se opta por la alternativa 2: sala de espera de partidas pendientes y mantenimiento de las partidas en el juego en el *Manager*.

Así, cuando una partida está lista para ser jugada (porque tenga, por ejemplo, el número de jugadores requerido), la pasamos a la lista de partidas en juego del *Manager:* esto nos obliga (Figura 479) a modificar ligeramente el método *playMatch* del servicio, y a crear un método abstracto *isReady* en la clase *Match,* cuya implementación dependerá del tipo de juego al que estemos jugando. En *OtnMatch,* que es la que nos viene ocupando por el momento, devolverá *true* cuando el número de *players* sea 2.

```
public Match playMatch(String userId, String gameName) {
   ...

   if (matches==null || matches.isEmpty()) {
      ...
   } else {
      match = matches.get(0);
      match.add(user);
      if (match.isReady()) {
         matches.remove(0);
         Manager.get().add(match);
      }
   }
   return match;
}
```

Figura 479. Cambiamos la partida de colección cuando está preparada

Ahora, se añade el mensaje resaltado al diagrama de secuencia de la Figura 471:

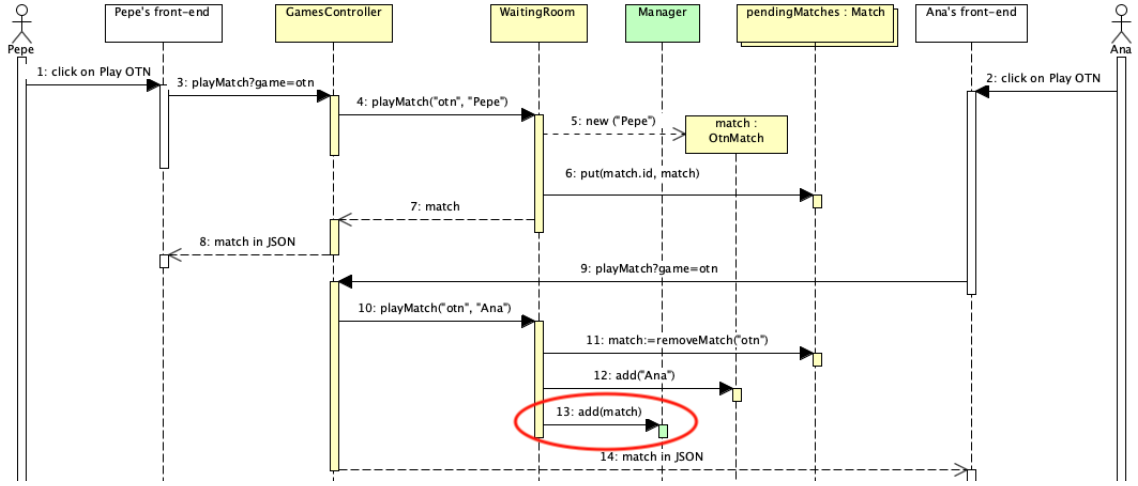


Figura 480. La partida se añade a la lista que posee el Manager

5. Prueba del recurso web *play*

Si, elegida la segunda opción, lo que llevamos implementado hasta ahora está bien, debemos ser capaces de escribir un caso de prueba para que Pepe y Ana puedan iniciar una partida de *Order the numbers*. Escribiremos un caso para que Pepe y Ana se registren, se logueen y elijan comenzar una partida de este juego.

La clase *testOtn2* la hemos colocado en el paquete *edu.uclm.esi.juegos.multi* y tiene el código de la Figura 481, que puede parecer farragoso pero que es muy sencillo y que, respecto a los casos de prueba de otros capítulos, solo tiene alguna pequeña modificación para bregar con las peticiones CORS:

1) Primero se ejecuta el *@BeforeAll,* en el que se registran y loguean los dos usuarios.
2) Luego, en *testPlayOtn,* ambos jugadores eligen jugar al *Otn*. En su oráculo, comprobamos que el *id* de la partida es el mismo en ambos casos.

```
@SpringBootTest
@ExtendWith(SpringExtension.class)
@AutoConfigureMockMvc
@TestInstance(Lifecycle.PER_CLASS)
@TestMethodOrder(OrderAnnotation.class)
public class TestOtn2 {
```

```
@Autowired
private MockMvc server;

@Autowired
private UserDAO userDAO;

private MockHttpSession pepeSession;

private MockHttpSession anaSession;

private static String origin = "http://localhost:4200";

@BeforeAll
void setUp() throws Exception {
   this.userDAO.deleteAll();
   this.pepeSession = this.register("Pepe");
   this.anaSession = this.register("Ana1");

   this.login("Pepe", this.pepeSession);
   this.login("Ana1", this.anaSession);
}

@Test @Order(1)
void testPlayOtn() throws Exception {
   JSONObject jsoPepe = this.playOtn(this.pepeSession);
   JSONObject jsoAna = this.playOtn(this.anaSession);

   assertEquals(jsoPepe.getString("id"), jsoAna.getString("id"));
}

private JSONObject playOtn(MockHttpSession session) throws Exception {
   RequestBuilder request = MockMvcRequestBuilders.
         get("/games/play?gameName=otn").
         header("origin", origin).
         header("withCredentials", "true").
         session(session);
   String response = this.server.perform(request).
      andExpect(status().isOk()).
      andReturn().getResponse().getContentAsString();
   return new JSONObject(response);
}

private MockHttpSession register(String name) throws Exception {
   JSONObject jsoUser= new JSONObject().
         put("name", name).
         put("pwd1", "12345678").
         put("pwd2", "12345678").
         put("email", name + "@" + name + ".com");
   RequestBuilder request = MockMvcRequestBuilders.
         post("/users/register").
         header("origin", origin).
         header("withCredentials", "true").
         contentType("application/json").
         content(jsoUser.toString());
   return (MockHttpSession) this.server.perform(request).
         andReturn().getRequest().getSession();
}

private void login(String name, MockHttpSession session) throws Exception {
   JSONObject jsoUser= new JSONObject().
         put("name", name).put("pwd", "12345678");
   RequestBuilder request = MockMvcRequestBuilders.
         put("/users/login").
         session(session).
         header("origin", origin).
```

```
            header("withCredentials", "true").
            contentType("application/json").
            content(jsoUser.toString());
        this.server.perform(request).
           andExpect(status().isOk());
    }
}
```

Figura 481. Caso para probar que Pepe y Ana pueden jugar la misma partida

Como anunciábamos antes de la figura anterior, lo único que debemos tener en cuenta es que nuestros controladores reciben peticiones de origen cruzado, por lo que debemos añadir las cabeceras adecuadas: *origin* y *withCredentials*. Además, cada petición ya debe enviar información de su propia sesión, por lo que hemos creado dos campos *MockHttpSession* para Pepe y para Ana, que asignamos cuando se registran y que enviamos tanto al hacer *login* como al hacer *play*.

Capítulo 21. WebSockets

1. Comunicación mediante protocolo *ws*

Para establecer una comunicación mediante protocolo *ws,* tanto el cliente como el servidor deben entender los mensajes que circulan mediante este protocolo.

Para utilizar web sockets en un navegador, nos basta con utilizar el objeto WebSocket que incluye cualquier navegador moderno. Para usarlos en el servidor, podemos usar alguna implementación de este protocolo que nos dé alguna librería.

2. WebSockets en el lado cliente

WebSocket es una clase estandarizada que implementa cualquier navegador moderno. La interfaz oficial de esta clase se muestra en la Figura 482:

1) El campo *url* representa el *endpoint* de la conexión (es decir: dónde está escuchando el WebSocket del servidor).
2) Los cuatro campos constantes indican los posibles estados del objeto: conectando, conexión abierta, cerrando o cerrado.
3) El atributo de solo lectura *readyState* cambia según vaya cambiando el estado del WebSocket: cuando está abierto, vale *OPEN;* cuando está cerrado, su valor es *CLOSED*.
4) *bufferedAmount* indica la cantidad de información, en bytes, que el WebSocket mantiene a la espera de enviar.
5) Las funciones asociadas a los manejadores de eventos *onopen, onerror, onclose* y *onmessage* se ejecutan cuando el objeto se abre, se produce un error, se cierra o recibe un mensaje del servidor.
6) Finalmente, las diferentes versiones de *send* se utilizan para enviar información al servidor.

```
enum BinaryType { "blob", "arraybuffer" };
[Constructor(DOMString url, optional (DOMString or DOMString[]) protocols)]
interface WebSocket : EventTarget {
  readonly attribute DOMString url;

  // ready state
  const unsigned short CONNECTING = 0;
  const unsigned short OPEN = 1;
  const unsigned short CLOSING = 2;
  const unsigned short CLOSED = 3;

  readonly attribute unsigned short readyState;
  readonly attribute unsigned long bufferedAmount;

  // networking
  attribute EventHandler onopen;
  attribute EventHandler onerror;
  attribute EventHandler onclose;
  readonly attribute DOMString extensions;
  readonly attribute DOMString protocol;
  void close([Clamp] optional unsigned short code, optional DOMString reason);

  // messaging
  attribute EventHandler onmessage;
  attribute BinaryType binaryType;
```

```
  void send(DOMString data);
  void send(Blob data);
  void send(ArrayBuffer data);
  void send(ArrayBufferView data);
};
```

Figura 482. Interfaz del tipo WebSocket en el lado cliente

3. WebSockets en el lado servidor

Para recibir y enviar mensajes al cliente, en el servidor debe haber también algún objeto escuchando que entienda el protocolo *ws*. Debe ser capaz de aceptar una conexión, de cerrarla, de recibir mensajes, de enviarlos y de recuperarse ante algún error.

Existen varias librerías en Java que implementan WebSockets. Por ejemplo, en el paquete *javax.websocket* se definen las anotaciones *@ServerEndPoint* (que se utiliza para indicar que la clase es un WebSocket), *@OnOpen, @OnClose, @OnMessage* y *@OnError*. En ese paquete se declara también el tipo *Session,* que representa una sesión de WebSocket en el servidor. Podemos entender *Session* como el elemento que utiliza el servidor para enviar y recibir información al y del cliente: en un WebSocket que ofrezca funcionalidades de chat entre usuarios habrá tantas instancias de clase *Session* como usuarios estén conectados al chat.

En nuestro caso utilizaremos los WebSockets de Spring que, si bien se programan de manera distinta, responden por supuesto al mismo protocolo y funcionan de manera similar: por ejemplo, a la anotación *@OnOpen* le corresponde el método *afterConnectionEstablished;* a *@OnMessage,* los métodos *handleTextMessage* o *handleBinaryMessage;* el objeto *Session* pasa a ser *WebSocketSession*.

4. Establecimiento de la conexión

Aunque la comunicación se realiza mediante protocolo *ws*, el establecimiento de la conexión entre el cliente y el servidor (el *handshake* o apretón de manos) se inicia con una *request* de tipo GET por *http* del estilo de la siguiente (obsérvese el nombre del recurso solicitado, *servidorDeChat*, que es el *endpoint*):

```
GET /servidorDeChat HTTP/1.1
```

En esta *request* se envían además dos cabeceras fijas (*Upgrade* y *Connection*) y una cabecera *Sec-WebSocket-Key* con un token aleatorio codificado en base64:

```
GET /servidorDeChat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
 ...
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
```

Al recibir la *request*, el servidor:

1) Descodifica el token.
2) Le concatena la llamada "cadena mágica": `258EAFA5-E914-47DA-95CA-C5AB0DC85B11`. En este ejemplo, la cadena total será: `dGhlIHNhbXBsZSBub25jZQ==258EAFA5-E914-47DA-95CA-C5AB0DC85B11`
3) Recodifica la cadena resultante calculando el hash de la cadena mediante SHA-1. Para este ejemplo, el resultado es: *0xb3 0x7a 0x4f 0x2c 0xc0*

0x62 0x4f 0x16 0x90 0xf6 0x46 0x06 0xcf 0x38 0x59 0x45 0xb2 0xbe 0xc4 0xea

4) El código hash obtenido en el paso anterior se codifica con base64: `s3pPLMBiTxaQ9kYGzzhZRbK+xOo=`
5) El servidor le devuelve al cliente una *response* con código 101 (*Switching protocols*) junto a una cabecera *Sec-WebSocket-Accept* con este último token:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

6) De acuerdo con la especificación, el cliente debe rechazar la conexión si el token recibido no es el que él ha calculado previamente o si el código de estado no es 101.

La Figura 483 ilustra el posible paso de mensajes que sucede desde que Pepe y Ana se conectan a un servidor de chat, y Pepe le escribe *Hello* a Ana:

1) El *front* de cada usuario (mensajes 1 y 2) crea un WebSocket de cliente (según la especificación de la Figura 482). Este envía la petición *http* para el establecimiento de la conexión. La petición, realmente, no llega directamente al WebSocket del servidor (*WSChat)*, sino que es capturada y tratada previamente por los interceptores, que inyectan en *WSChat* un objeto de tipo *WebSocketSession,* que se añade a una lista sesiones.
2) Cuando los websockets de cliente reciben la respuesta 101 ejecutan su manejador de eventos *onopen*. Después, Pepe escribe usando su interfaz de usuario, que envía el mensaje 5.1.1 al servidor mediante el WebSocket que creó previamente.
3) El servidor recibe el mensaje en su *handleTextMessage*. Podemos suponer que lo difunde a todos los usuarios conectados, por lo que ejecuta *send* sobre cada *WebSocketSession* de la colección. Cuando el cliente lo recibe, ejecuta su manejador *onmessage*.

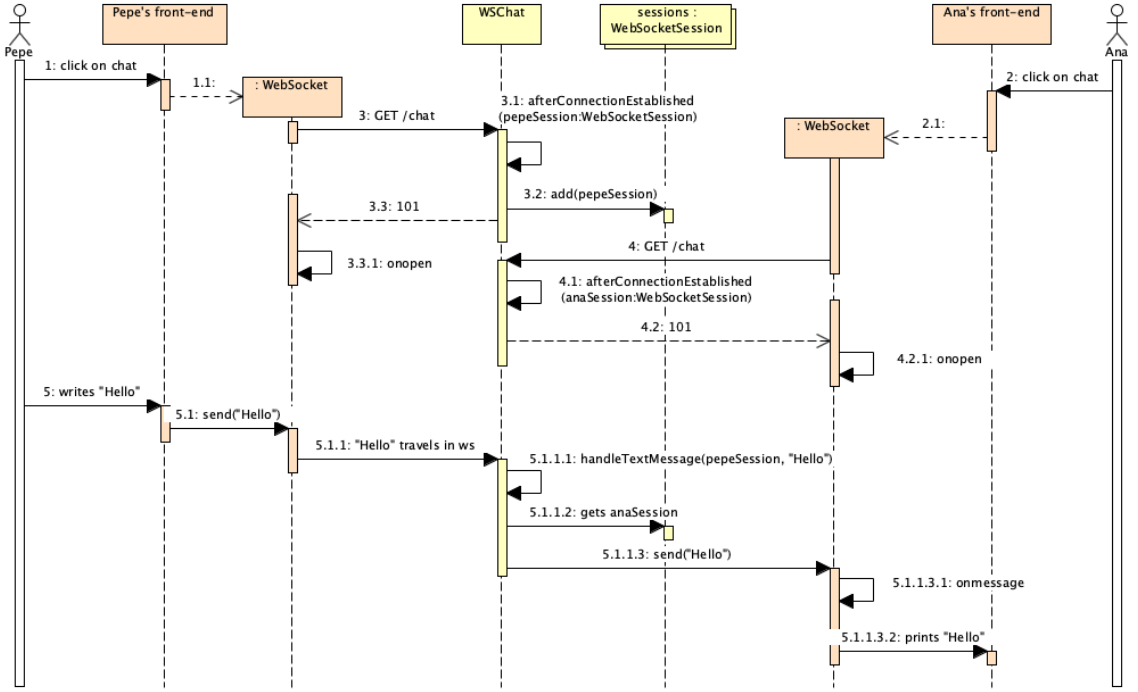


Figura 483. Paso de mensajes con ws

5. Implementación del WebSocket en el servidor

Para que Spring reconozca adecuadamente el WebSocket necesitamos crear dos clases: una con el WebSocket propiamente dicho (Figura 484), y otra en la que registraremos el nombre del recurso con el que el WebSocket estará accesible mediante una url (Figura 485). Colocaremos ambas clases en el paquete *edu.uclm.esi.juegos.ws*.

```java
package edu.uclm.esi.juegos.ws;

import org.springframework.stereotype.Component;
import org.springframework.web.socket.BinaryMessage;
import org.springframework.web.socket.TextMessage;
import org.springframework.web.socket.WebSocketSession;
import org.springframework.web.socket.handler.TextWebSocketHandler;

@Component
public class WSGames extends TextWebSocketHandler {

   @Override
   public void afterConnectionEstablished(WebSocketSession session) throws Exception {
      System.out.println();
   }

   @Override
   protected void handleTextMessage(WebSocketSession session, TextMessage message)
      throws Exception {
      System.out.println();
      session.sendMessage(new TextMessage("Mensaje recibido"));
   }

   @Override
   protected void handleBinaryMessage(WebSocketSession session, BinaryMessage message) {

   }

   @Override
   public void handleTransportError(WebSocketSession session, Throwable exception)
      throws Exception {

   }
}
```

Figura 484. El WebSocket propiamente dicho

```java
package edu.uclm.esi.juegos.ws;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.socket.config.annotation.EnableWebSocket;
import org.springframework.web.socket.config.annotation.WebSocketConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketHandlerRegistry;
import org.springframework.web.socket.server.support.HttpSessionHandshakeInterceptor;

@Configuration
@EnableWebSocket
public class WSConfigurer implements WebSocketConfigurer {
   @Override
   public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
      registry.
         addHandler(new WSGames(), "/wsGames").
         setAllowedOrigins("*").
         addInterceptors(new HttpSessionHandshakeInterceptor());
   }
}
```

Figura 485. La clase de configuración

5.1. ¿Funciona el WebSocket?

En *afterConnectionEstablished* y en *handleTextMessage* hemos colocado algunas instrucciones sobre las que colocaremos puntos de interrupción. Para comprobar si el WebSocket escucha, instalamos en nuestro navegador una extensión llamada *Simple Web Socket Client*. Lanzamos el docker y la aplicación en modo depuración, abrimos la extensión recién instalada y conectamos escribiendo la url del servidor de WebSocket y pulsando *Open*.

La figura siguiente evidencia que la respuesta 101 (*Switching protocols)* llega al cliente (en este caso, la extensión del navegador) antes de que el

WebSocket de servidor ejecute *afterConnectionEstablished:* en efecto, el cliente ya muestra OPENED, pero el servidor no ha terminado de procesar la entrada de la conexión. Esto sucede porque los interceptores, que han capturado la petición, han encontrado el recurso solicitado en el servidor (*wsGames)* y ya le han devuelto la respuesta al cliente.

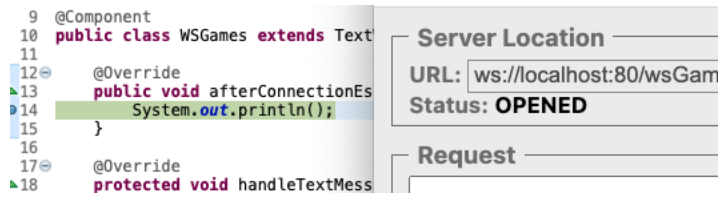


Figura 486. La respuesta 101 llega antes de que ejecutar onopen *en el cliente*

El caso es que, en el punto en el que hemos detenido la ejecución del servidor, Spring ha inyectado en el parámetro *session* de *afterConnectionEstablished* el objeto de tipo *WebSocketSession* que aparece en el lado derecho de la figura siguiente:

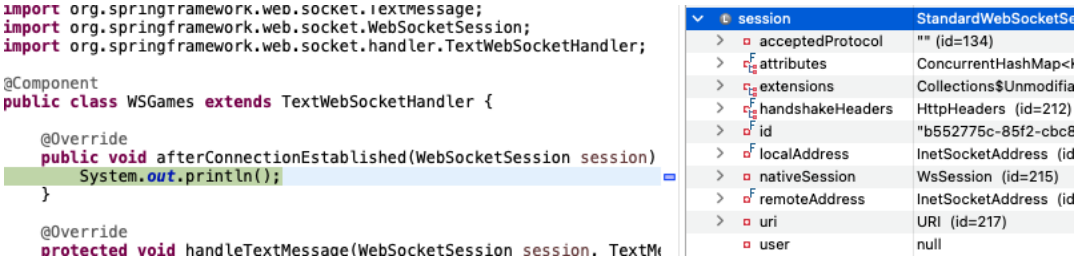


Figura 487. Se inyecta el objeto session

Es de destacar que esta sesión no tiene relación con la sesión http que ha iniciado la conexión, por lo que tendremos que utilizar algún truco si queremos poder manejar ambas conjuntamente.

En la Figura 488 se ha enviado un mensaje mediante la extensión, que ha sido capturado por *handleTextMessage*. Este método toma dos argumentos: la *session* que lo ha enviado (es decir, el WebSocket cliente, que es el mismo que se creó al establecer la conexión) y un *TextMessage*, cuya *payload* es el mensaje enviado.

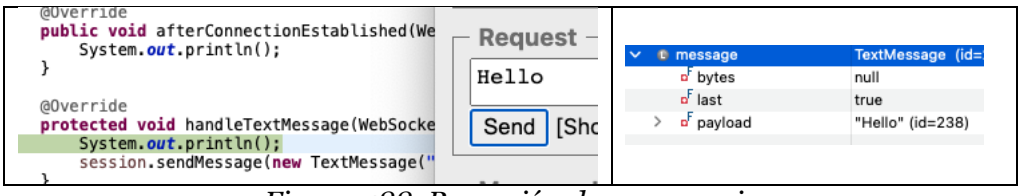


Figura 488. Recepción de un mensaje

En la última línea de *handleTextMessage,* enviamos un mensaje al mismo WebSocket que nos ha escrito, que aparecerá en el *log* de la extensión del navegador.

Capítulo 22. *Otn* para dos jugadores: *backend* (II)

1. Comportamiento deseado

Una vez que los dos jugadores reciben un *id* de la partida a la que están jugando, pueden establecer con el servidor una conexión *ws* para enviar, recibir o enviar y recibir las actualizaciones que se produzcan sobre el tablero propio o el del oponente.

Lamentablemente, a fecha de hoy no disponemos (que sepa el autor) de un *MockWebSocket* que, igual que el *MockMvc,* nos sirva para manejar la conexión *ws* desde los casos de prueba. Pero afortunadamente sí que disponemos de un conjunto de elementos en el paquete *javax.websocket* que nos van a permitir probar la conexión de ambos jugadores.

1.1. Establecimiento de la conexión *ws*

Volviendo a la Figura 480, vemos que el *GamesController* les devuelve a los clientes, como respuesta a la petición *playMatch*, un objeto de clase *Match* (la partida que van a jugar).

Bien, pues los clientes, cuando reciban esa respuesta, establecerán la conexión por protocolo *ws* con un WebSocket que hemos de crear en el servidor, y al que llamaremos *WSGames*. En la Figura 489 hemos añadido estos dos mensajes de establecimiento de la conexión, que se envían como consecuencia de recibir la respuesta http con la partida.

Como sabemos por el capítulo anterior, cuando la conexión *ws* se establece, en el cliente se ejecuta el manejador de eventos *onopen,* que en el diagrama está representado por los mensajes 9 y 18.

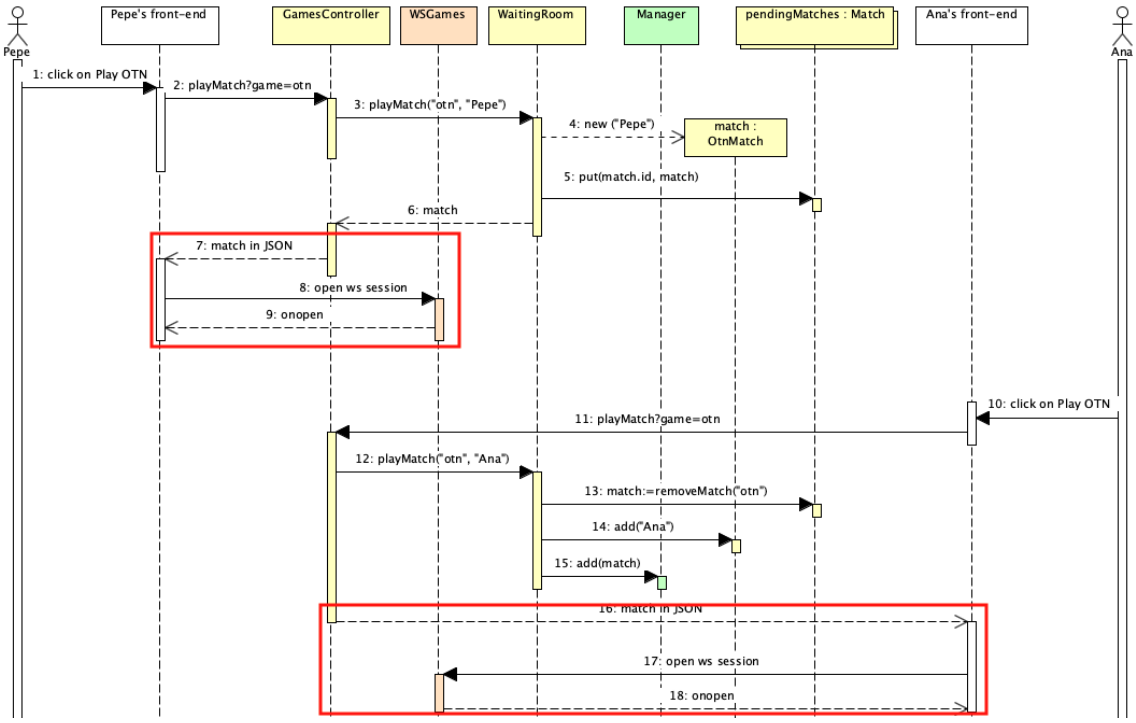


Figura 489. Los clientes se conectan al WebSocket del servidor

Como vimos en la Figura 487 (página 306) y comentamos en su párrafo posterior, la sesión *ws* no tiene relación con la sesión http. Sin embargo, nos interesa que el servidor pueda enlazarlas de alguna manera, es decir, que sepa que la sesión http de Pepe se corresponde con su sesión *ws*.

Para esto, en lugar de conectar, como mostrábamos en el capítulo anterior, con una url del tipo *ws://localhost:80/wsGames,* ahora conectaremos añadiéndole el *id* de la *HttpSession,* es decir: *ws://localhost:80/wsGames?httpSessionId=abcd*.

Para esto tenemos que conocer el *id* de la sesión en el cliente. Lo tenemos disponible en la cookie JSESSIONID que nos devuelve el servidor (que consiste en una cadena de 32 caracteres), y en JavaScript tenemos la posibilidad de acceder a esta cabecera y de leer su valor. Una sencilla búsqueda en Internet con la cadena "javascript read jsessionid cookie" nos lleva a StackOverflow, que nos muestra una función muy sencilla. Otra posibilidad, que es la que vamos a utilizar, es devolver el *id* de la *HttpSession* como resultado del recurso web *login* en el *UserController:* simplemente declaramos el método como *String* y le colocamos como última línea *return session.getId()*.

Cuando se reciba la petición de conexión en el servidor *WSGames* (es decir, cuando se ejecute *afterConnectionEstablished)*, leeremos el parámetro *httpSessionId* que viene en la url, leeremos el *id* de la *WebSocketSession* que nos haya inyectado Spring y, de alguna manera, conectaremos una con otra.

2. Diseño de un Web Socket cliente para los casos de prueba

Como hemos dicho, disponemos de varias clases en *javax.websocket* para crear un cliente *ws* en nuestro *backend*. En la figura siguiente se muestra la estructura que daremos a nuestra clase *WSClient,* que ubicaremos en el paquete *edu.uclm.esi.juegos.multi* del subsistema de pruebas (*src/test/java)*. Dispone de:

1) Un campo *session* de tipo *Session,* que representa la sesión con la que permanecerá conectado al WebSocket del servidor. Esta clase dispone de un método *send* que permite enviar mensajes al servidor.
2) Un método *open,* que usaremos para establecer la conexión con *WSGAmes*. Como vemos, toma dos parámetros: la url del servidor y el *id* de la sesión http.
3) Tres métodos *onOpen, onError* y *onMessage,* que se corresponden con los manejadores *onopen, onerror* y *onmessage* de la especificación estándar para WebSockets en el lado cliente que ya vimos en la Figura 482 (página 302).

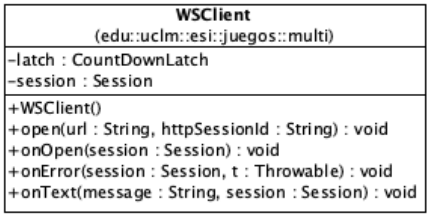


Figura 490. Estructura de nuestro WSClient

Cuando este cliente reciba mensajes del servidor tiene que enviárselos a un receptor (*receiver)* que esté interesado en ellos. Puesto que vamos a usar ese cliente desde la clase *TestOtn2* que empezamos a construir en la Figura 481 (página 299), tal receptor puede ser la propia clase *TestOtn2*. Gráficamente:

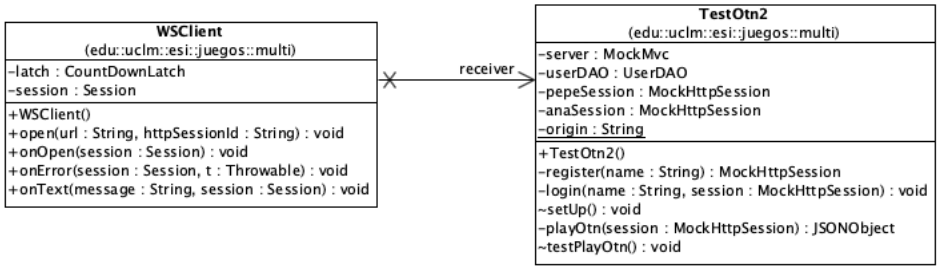


Figura 491. Posible conexión del WSClient *con* TestOtn2

Si implementásemos la solución descrita en la figura nos encontraríamos un problema importante en tiempo de ejecución: el *server* http de *TestOtn2* es un servidor "falso" (un *MockMvc)*, mientras que el *WSClient* que hemos diseñado necesita un servidor real. Las sesiones http (campos *pepeSession* y *anaSession)* existen en el *MockMvc*, pero no se corresponderían con el servidor real que necesitan los clientes de WebSocket.

Por eso, copiaremos el fichero *TestOtn2*, lo pegaremos como *TestOtn2Real* y realizaremos algunas modificaciones para que las peticiones http y *ws* se encaminen al mismo servidor, que ha de ser necesariamente real. Así, sustituimos la clase *TestOtn2* por la nueva en nuestro diagrama de clases. En la Figura 492, obsérvese que:

1) El *receiver* del *WSClient* pasa a ser la nueva clase.
2) El *server* al que se conectará la clase de prueba ya no es un *MockMvc,* sino que es un *WebTestClient,* que es una clase Spring que permite conectarnos a servidores reales.
3) Los campos *pepeSession* y *anaSession* pasan a ser cadenas. Su valor será el *id* de la *HttpSession* de cada uno.

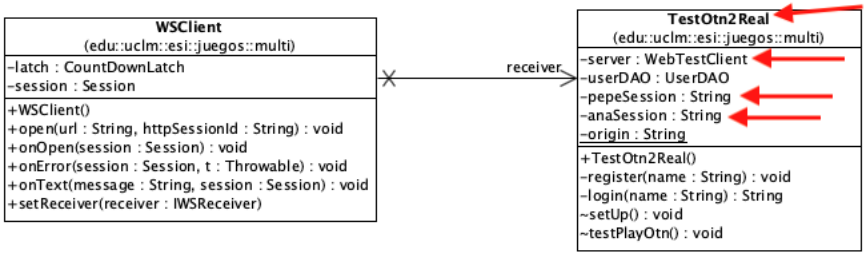


Figura 492. Creamos una clase de prueba nueva

Pero pensando que tal vez este *WSClient* nos pueda servir en otras partes del desarrollo de este proyecto, o tal vez incluso en otros proyectos, podemos hacer que el campo *receiver* sea un objeto genérico que responda a una operación *receive*. Esto parece buena idea porque desacopla *WSClient* del receptor de los mensajes, por lo que implementaremos una interfaz *IWSReceiver* (Figura 493), cuyo método *receive* toma el propio cliente que ha recibido el mensaje, el evento que se ha producido y el propio texto del mensaje.

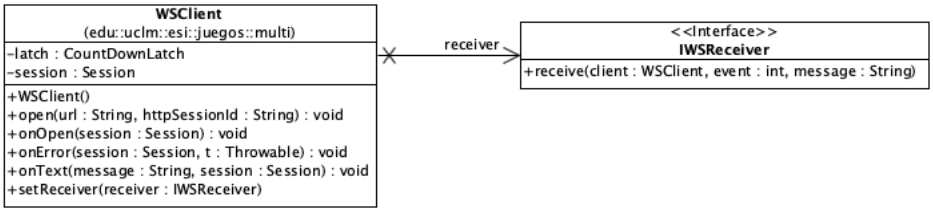


Figura 493. Optamos por conectar el WSClient *a una interfaz*

En nuestro caso concreto, *TestOtn2Real* ha de estar dispuesto a recibir mensajes del *WSClient,* por lo que debemos hacer que implemente esa interfaz:

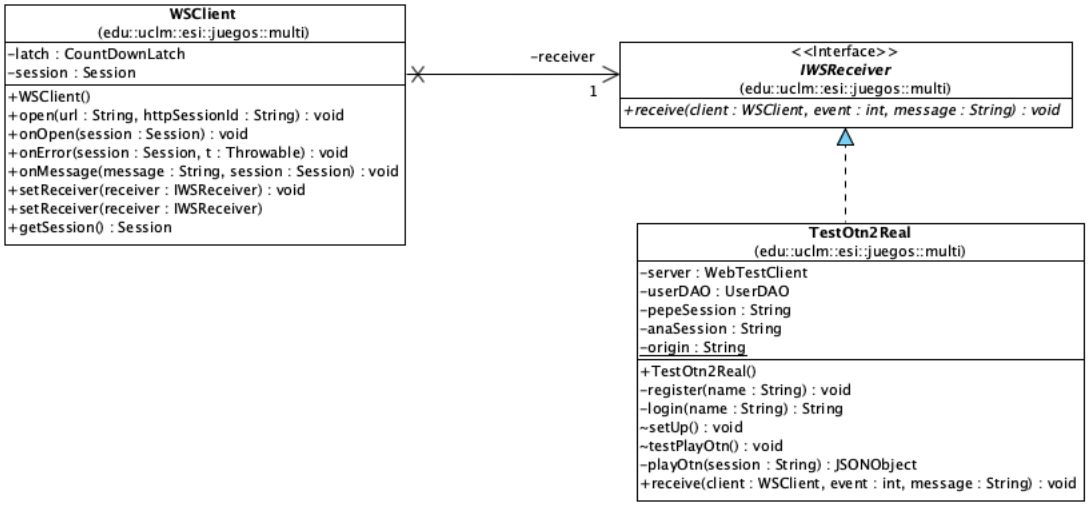


Figura 494. TestOtn2Real *es un receptor de eventos de* WSClient

Además, *TestOtn2Real* debe tener, igual que mantiene en los campos *pepeSession* y *anaSession* el *id* de las sesiones http de Pepe y Ana, dos clientes de WebSocket para estos usuarios. Igualmente, le añadimos un método *connectWebsocket* que conectará un *WSClient* con la sesión http que pasemos como parámetro.

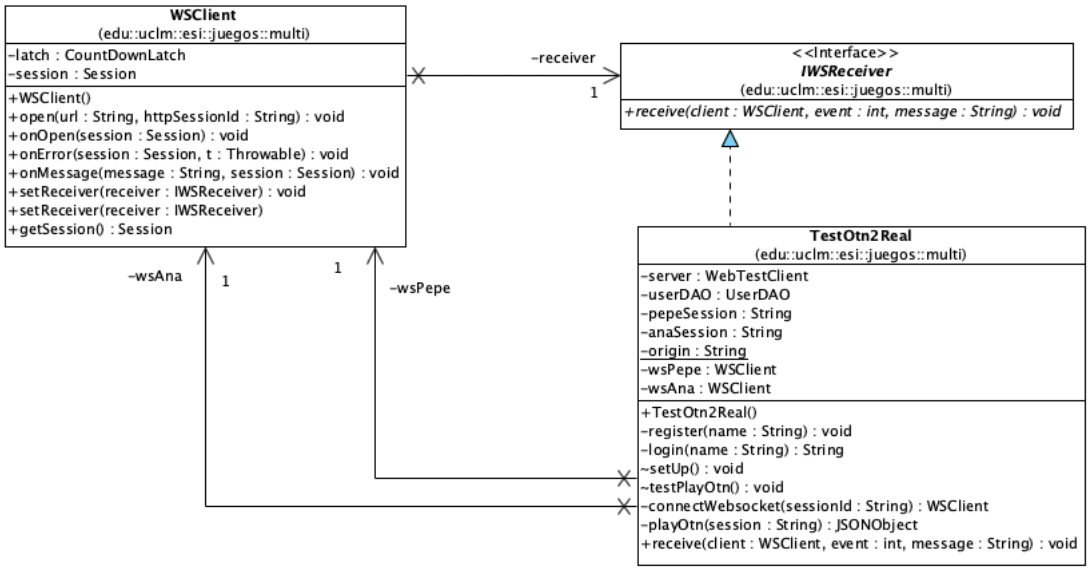


Figura 495. Añadimos a TestOtn2Real *un cliente de WebSocket para Pepe y otro para Ana*

3. Implementación del WebSocket cliente

Para crear la estructura de clases de la figura anterior, creamos en primer lugar la interfaz:

```
package edu.uclm.esi.juegos.multi;

public interface IWSReceiver {
   void receive(WSClient client, int event, String message);
}
```

Figura 496. Código de IWSReceiver

Luego podemos seguir por el *WSClient*, con el código de la Figura 497. Nótese que las anotaciones *OnOpen, OnError* y *OnMessage* anotan a métodos que se llaman *onOpen, onError* y *onMessage*: en realidad, lo importante aquí es la anotación, por lo que los métodos se podrían haber llamado de cualquier otra forma.

Nótese que en este caso sí utilizamos el paquete *javax* para importar diferentes elementos de los WebSockets. Por ello hemos incluido la dependencia *javax.websocket* en el fichero *pom.xml*.

```
<dependency>
    <groupId>javax.websocket</groupId>
    <artifactId>javax.websocket-api</artifactId>
    <version>1.1</version>
</dependency>
```

```
package edu.uclm.esi.juegos.multi;

import java.net.URI;
import java.util.concurrent.CountDownLatch;

import javax.websocket.ClientEndpoint;
import javax.websocket.ContainerProvider;
import javax.websocket.OnError;
import javax.websocket.OnMessage;
import javax.websocket.OnOpen;
import javax.websocket.Session;
import javax.websocket.WebSocketContainer;

@ClientEndpoint
public class WSClient {
   private CountDownLatch latch = new CountDownLatch(1);
   private Session session;
   private IWSReceiver receiver;
   public static final int ON_OPEN = 0, ON_ERROR = 1, ON_MESSAGE = 2,
         ON_CLOSE = 3;

    public void open(String url, String httpSessionId) throws Exception {
      WebSocketContainer container = ContainerProvider.getWebSocketContainer();
       container.connectToServer(this, new URI(url + httpSessionId));
       this.latch.await();
    }

   @OnOpen
    public void onOpen(Session session) {
       this.session = session;
       latch.countDown();
       this.receiver.receive(this, ON_OPEN, null);
    }

   @OnError
   public void onError(Session session, Throwable t) {
      this.session = session;
      latch.countDown();
      this.receiver.receive(this, ON_ERROR, t.getMessage());
   }
```

```
    @OnMessage
    public void onMessage(String message, Session session) {
      this.receiver.receive(this, ON_MESSAGE, message);
    }

    public void setReceiver(IWSReceiver receiver) {
      this.receiver = receiver;
   }

    public Session getSession() {
      return session;
   }
}
```

Figura 497. Código del WSClient

4. Implementación de la nueva clase de prueba

Para terminar de conformar la estructura de clases de la Figura 495 debemos modificar el código copiado y pegado en *TestOtn2Real*. Los primeros cambios se muestran en la Figura 498.

```
...
public class TestOtn2Real implements IWSReceiver {
   @Autowired  // Ya no hay @Autowired
   private WebTestClient server;
   @Autowired
   private UserDAO userDAO;

   private String pepeSession;
   private String anaSession;

   private WSClient wsPepe;
   private WSClient wsAna;
      ...
}
```

Figura 498. Conformando la estructura de clases

En el *TestOtn2* inicial, el campo *server* era *@Autowired* porque tenía la capacidad de enlazarse directamente al *backend* sin necesidad de arrancar un servidor real. Ahora necesitamos indicarle al nuevo *server* el endpoint al que tiene que conectarse. Para esto, modificamos el método *setUp* de *TestOtn2Real,* indicándole en las primeras líneas a dónde ha de conectarse y qué cabeceras debe enviar siempre.

```
@BeforeAll
void setUp() throws Exception {
   this.server = WebTestClient.bindToServer()
         .baseUrl("http://localhost")
         .defaultHeader("origin", origin)
         .defaultHeader("withCredentials", "true")
         .build();

   this.userDAO.deleteAll();
   this.register("Pepe");
   this.pepeSession = this.login("Pepe");

   this.register("Ana1");
   this.anaSession = this.login("Ana1");
}
```

Figura 499. Modificación del setUp

El método *connectWebsocket* simplemente crea y devuelve un *WSClient,* que se conecta al *endpoint* en el que escucha *WSGames*. En la url de conexión, le pasamos el *id* de la sesión http:

```
private WSClient connectWebsocket(String sessionId, IWSReceiver receiver) throws Exception {
   WSClient client = new WSClient();
   client.setReceiver(receiver);
   client.open("ws://localhost:80/wsGames?httpSessionId=", sessionId);
   return client;
}
```

Figura 500. connectWebSocket *en* TestOtn2Real

Los tres métodos que envían peticiones http han cambiado un poco, porque ahora deben ceñirse al objeto *WebTestClient* en lugar de a *MockMvc*. En la Figura 501 mostramos el nuevo código de estas operaciones, que no deja de ser muy parecida al anterior.

1) El método *login*, por ejemplo, envía con *put* el *jsoUser*. La operación *request.exchange()* envía la petición, cuya respuesta se recoge en el objeto *response*. Luego, se recupera y devuelve la *payload* de la respuesta, que es el *id* de la sesión.
2) *playOtn* debe enviarse al servidor indicándole que la sesión http es la correspondiente al usuario que se acaba de loguear. Antes conseguíamos esto con la *MockHttpSession*, de la que carecemos ahora. Para conseguir el mismo efecto, añadimos la cabecera *Cookie* con la JSESSIONID que corresponda al usuario.

```
private String login(String name) throws Exception {
   JSONObject jsoUser= new JSONObject().
         put("name", name).put("pwd", "12345678");

   RequestHeadersSpec<?> request = this.server
      .put()
      .uri("/users/login")
      .header("Content-Type", "application/json")
      .body(BodyInserters.fromObject(jsoUser.toString()));

   ResponseSpec response = request.exchange();

   EntityExchangeResult<String> result = response.expectBody(String.class).returnResult();
   return result.getResponseBody();
}

private void register(String name) throws Exception {
   JSONObject jsoUser= new JSONObject().
         put("name", name).
         put("pwd1", "12345678").
         put("pwd2", "12345678").
         put("email", name + "@" + name + ".com");

   RequestHeadersSpec<?> request = this.server.post()
      .uri("/users/register")
      .header("Content-Type", "application/json")
      .body(BodyInserters.fromObject(jsoUser.toString()));

   request.exchange();
}

private JSONObject playOtn(String session) throws Exception {
   RequestHeadersSpec<?> request = this.server.get()
      .uri("/games/play?gameName=otn")
      .header("Content-Type", "application/json")
      .header("Cookie", "JSESSIONID=" + session);

   ResponseSpec response = request.exchange();

   EntityExchangeResult<String> result = response.expectBody(String.class).returnResult();
   String body = result.getResponseBody();
   return new JSONObject(body);
}
```

Figura 501. Adaptación de los métodos que envían peticiones http

Además, *TestOtn2Real* implementa la interfaz *IWSReceiver,* por lo que tiene que incluir el método *receive* que hereda de ella. De momento, lo único que esperamos es que solo se produzca el evento *onopen:* si se produce otro, le decimos a JUnit que emita veredicto de fallo con *fail*:

```
@Override
public void receive(WSClient client, int event, String message) {
   if (event != WSClient.ON_OPEN)
      fail("Unexpected message in the WebSocket");
}
```

Figura 502. Implementación provisional de receive

Por último, el único caso de prueba que tenemos por ahora, *testPlayOtn*, debe iniciar la conexión *ws* de cada jugador cuando se reciba la respuesta a la petición *http* (Figura 503), que se guarda en los objetos *jsoPepe* y *jsoAna:* en estos viene el *id* de la partida, que almacenamos en el campo *matchId*. Lo utilizamos en *connectWebsocket* para indicarle al servidor que estamos listos para jugar esa partida.

```
@Test @Order(1)
void testPlayOtn() throws Exception {
   JSONObject jsoPepe = this.playOtn(this.pepeSession);
   JSONObject jsoAna = this.playOtn(this.anaSession);

   assertEquals(jsoPepe.getString("id"), jsoAna.getString("id"));
   this.matchId = jsoPepe.getString("id");

   this.wsPepe = this.connectWebsocket(pepeSession, this);
   this.wsAna = this.connectWebsocket(anaSession, this);
}
```

Figura 503. Modificación de testPlayOtn

5. Recepción de la conexión en *WSGames*

En la Figura 484 (página 305) se escribió un código de ejemplo para *WSGames,* que probamos con la extensión *Simple Web Socket* del navegador. Recordemos que *afterConnectionEstablished* se ejecuta cuando el cliente abre la conexión *ws,* y que se inyecta en su parámetro *session* un objeto de tipo *WebSocketSession*. Recordemos también que las sesiones *http* y *ws* son independientes, pero que nos interesa mantenerlas enlazadas, porque así podremos saber que los mensajes recibidos por tal conexión *ws* corresponden, por ejemplo, al usuario cuyo *userId* está almacenado en tal sesión *http*.

5.1. Enlace de las dos sesiones (I). Patrón *Wrapper*

La última versión que tenemos del *Manager* es la de la Figura 478 (página 296). En su estado actual, el *Manager* dispone de una colección con las sesiones http de los usuarios que se han logueado y de otra con las partidas que se han creado. Este *Manager* puede ser un buen lugar para mantener asociada cada *HttpSession* con su *WebSocketSession*.

Como nos interesa mantener la conexión entre cada par de sesiones, crearemos una clase que haga de contenedor de ambas. La clase *HWSession* ("H" de http, "W" de *ws*) de la siguiente figura es una clase que nos inventamos nosotros y que contiene a los dos tipos de sesión. Esta clase es algo así como un envoltorio que nos facilita el acceso a los objetos que contiene, y corresponde a un uso del patrón de diseño *Wrapper* que, precisamente, significa "envoltorio".

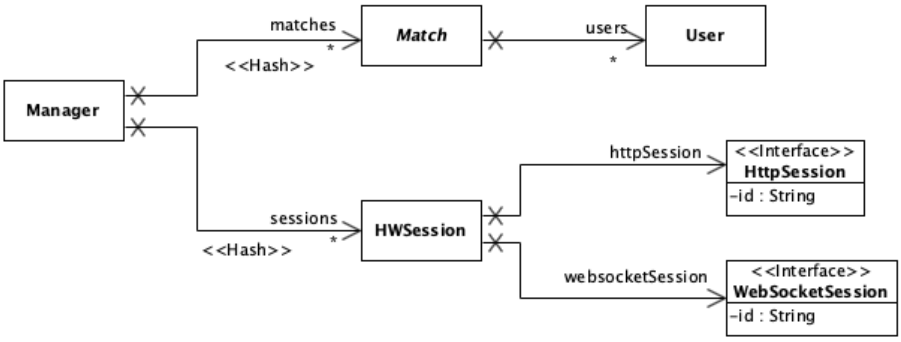


Figura 504. Creación de un wrapper *para almacenar los dos tipos de sesión*

Además, el *Manager* podría conocer a las sesiones *HWSession* de varias formas distintas: en la Figura 505, mantiene las sesiones en tres colecciones, todas ellas tablas hash (se ha quitado el estereotipo para no enturbiar demasiado el diagrama): en la primera, se encuentran indexadas por el *id* del usuario; en la segunda, por el *id* de la sesión http; en la tercera, por el *id* de la WebSocketSession. Obviamente, los objetos que van a contener esas tres colecciones serán los mismos: es decir, no se ocupa el triple de memoria, puesto que las diferentes *sessionsBy…* son referencias a las mismas instancias.

Mantener las tres colecciones puede sernos de ayuda para manejar las sesiones por sus tres posibles identificadores.

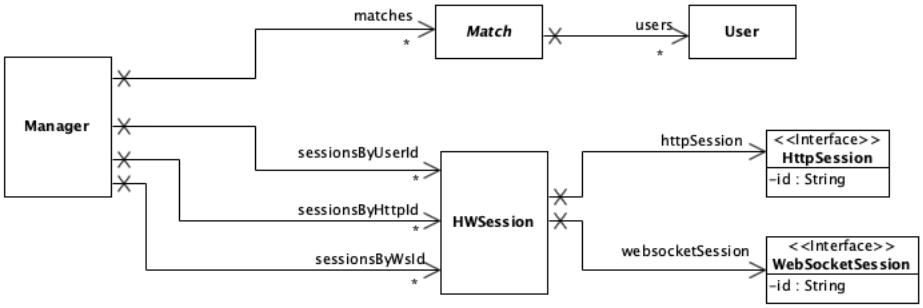


Figura 505. Podemos acceder a las sesiones de diferentes formas

La traducción a código del *Manager* es la de la Figura 506: se han incluido las tres colecciones y diferentes métodos para añadir y eliminar sesiones, que han de actuar con cuidado sobre las tres colecciones, para que el contenido de todas ellas esté siempre sincronizado.

```
@Component
public class Manager {

   @Autowired
   private WordDAO wordDAO;

   private ConcurrentHashMap<String, HWSession> sessionsByUserId;
   private ConcurrentHashMap<String, HWSession> sessionsByHttpId;
   private ConcurrentHashMap<String, HWSession> sessionsByWsId;

   private ConcurrentHashMap<String, Match> matches;

   private Manager() {
      this.sessionsByUserId = new ConcurrentHashMap<>();
      this.sessionsByHttpId = new ConcurrentHashMap<>();
      this.sessionsByWsId = new ConcurrentHashMap<>();
      this.matches = new ConcurrentHashMap<>();
   }

   private static class ManagerHolder {
```

```
        static Manager singleton=new Manager();
    }

    @Bean
    public static Manager get() {
        return ManagerHolder.singleton;
    }

    public WordDAO getWordDAO() {
        return wordDAO;
    }

    public void addSessionByUserId(String userId, HttpSession httpSession) {
        HWSession hwSession = new HWSession(httpSession);
        this.sessionsByUserId.put(userId, hwSession);
        this.sessionsByHttpId.put(httpSession.getId(), hwSession);
    }

    public void setWebsocketSession(String httpSessionId,
                                    WebSocketSession websocketSession) {
        HWSession hwSession = this.sessionsByHttpId.get(httpSessionId);
        hwSession.setWebsocketSession(websocketSession);
        this.sessionsByWsId.put(websocketSession.getId(), hwSession);
    }

    public HWSession getSessionByUserId(String userId) {
        return this.sessionsByUserId.get(userId);
    }

    public HWSession removeSessionByUserId(String userId) {
        HWSession hwSession = this.sessionsByUserId.remove(userId);
        this.sessionsByHttpId.remove(hwSession.getHttpSession().getId());
        if (hwSession.getWebsocketSession()!=null)
            this.sessionsByWsId.remove(hwSession.getWebsocketSession().getId());
        return hwSession;
    }

    public HWSession getSessionByHttpId(String httpId) {
        return this.sessionsByHttpId.get(httpId);
    }

    public HWSession getSessionByWebSocketId(String wsId) {
        return this.sessionsByWsId.get(wsId);
    }

    public HWSession removeSessionByHttpId(String httpId) {
        HWSession hwSession = this.sessionsByHttpId.remove(httpId);
        this.sessionsByUserId.remove(hwSession.getUserId());
        if (hwSession.getWebsocketSession()!=null)
            this.sessionsByWsId.remove(hwSession.getWebsocketSession().getId());
        return hwSession;
    }

    public void invalidate(HWSession existingSession) {
        existingSession.getHttpSession().invalidate();
        try {
            existingSession.getWebsocketSession().close();
        } catch (IOException e) { }
        this.removeSessionByHttpId(existingSession.getHttpSession().getId());
    }

    public void add(Match match) {
        this.matches.put(match.getId(), match);
    }

    public Match getMatch(String matchId) {
        return this.matches.get(matchId);
    }

    public Match removeMatch(String matchId) {
        return this.matches.remove(matchId);
    }
}
```

Figura 506. Vista parcial del código del Manager

El método *setWebSocketSession,* se utilizará cuando se cree *WSGames* disponga de ya de la sesión *ws*. El método *invalidate* invalida la sesión http (recuérdese la Figura 411, página 257), cerrando también la sesión de WebSocket y eliminando la *HWSession* de las tres colecciones.

Como comentábamos, *HWSession* actúa de *wrapper* para controlar el acceso a las dos sesiones que contiene:

```
public class HWSession {
   private HttpSession httpSession;
   private WebSocketSession websocketSession;

   public HWSession(HttpSession httpSession) {
      this.httpSession = httpSession;
   }

   public HWSession(HttpSession httpSession, WebSocketSession websocketSession) {
      this.httpSession = httpSession;
      this.websocketSession = websocketSession;
   }

   public void setHttpSession(HttpSession httpSession) {
      this.httpSession = httpSession;
   }

   public HttpSession getHttpSession() {
      return httpSession;
   }

   public void setWebsocketSession(WebSocketSession websocketSession) {
      this.websocketSession = websocketSession;
   }

   public WebSocketSession getWebsocketSession() {
      return websocketSession;
   }

   public String getUserId() {
      return this.httpSession.getAttribute("userId").toString();
   }
}
```

Figura 507. Implementación del wrapper *HWSession*

6. Enlace de las dos sesiones (II). Cambios en *WSGames*

Cuando el cliente se conecta al WebSocket del servidor, envía en la url el *id* de su sesión http en el parámetro *httpSessionId*. Para enlazar las dos sesiones, en *afterConnectionEstablished* (Figura 508):

1) Leemos la consulta (*query)* de la url de conexión.
2) Tomamos el valor del parámetro *httpSessionId*.
3) Buscamos en el Manager la *HWSession* correspondiente a ese *httpSessionId*.
4) Asignamos a esa *HWSession* la WebSocketSession que los interceptores nos han inyectado en el parámetro *session*.

```
@Override
public void afterConnectionEstablished(WebSocketSession session) throws Exception {
   String query = session.getUri().getQuery();
   String httpSessionId = query.substring("httpSessionId=".length());
   Manager.get().setWebsocketSession(httpSessionId, session);
}
```

Figura 508. Enlazamos las dos sesiones

Cuando se ejecuta *afterConnectionEstablished* en el servidor es cuando se ejecuta *onopen* en el cliente (recuérdese la Figura 489, página 307). En ese momento, el cliente *ws* debe enviarle de vuelta al servidor un mensaje diciéndole

«Oye, estoy preparado para jugar a la partida que me has dicho antes, y cuyo *id* es tal». Esto lo haremos reescribiendo el método *receive* en *TestOtn2Real:* construimos el mensaje como un JSON con los campos *type* y *matchId,* y lo enviamos mediante al *WSGames* el *WSClient*.

```
@Override
public void receive(WSClient client, int event, String message) {
   try {
      if (event == WSClient.ON_OPEN) {
         JSONObject jso = new JSONObject()
               .put("type", "PLAYER READY")
               .put("matchId", this.matchId);
         client.getSession().getAsyncRemote().sendText(jso.toString());
      } else {
         fail("Unexpected message in the WebSocket");
      }
   } catch (Exception e) {
      fail("Unexpected exception: " + e.getMessage());
   }
}
```

Figura 509. El jugador anuncia al servidor que está preparado

Y, claro, *WSGames* tiene que ser capaz de bregar con este mensaje. Cuando lo reciba, ejecutará su método *handleTextMessage,* que lleva como parámetros la *WebSocketSession* que envía el mensaje y el mensaje propiamente dicho. En la Figura 510:

1) Convertimos a JSON el cuerpo del mensaje y preguntamos por el valor de su campo *type*.
2) Si es PLAYER READY, buscamos la partida en la colección de partidas del *Manager* que, recuérdese, tiene solamente las partidas listas para jugar (Figura 479, página 297).
3) Y si, en efecto, la partida está preparada, la marcamos como empezada y le decimos que notifique a sus jugadores que ya comienza.
4) Si la partida no estuviera preparada no sucedería nada, y el jugador que ha dicho que está listo tendría que esperar a que llegase el oponente.

```
@Override
protected void handleTextMessage(WebSocketSession session,
      TextMessage message) throws Exception {
   JSONObject jso = new JSONObject(message.getPayload());
   String type = jso.getString("type");

   if (type.equals("PLAYER READY")) {
      String matchId = jso.getString("matchId");
      Match match = Manager.get().getMatch(matchId);
      if (match!=null) {
         match.setStarted(true);
         match.notifyStart();
      }
   }
}
```

Figura 510. Procesamos el mensaje de que el jugador está listo

El nuevo método, *notifyStart,* en la clase *Match,* recorre la lista *players*. Como cada *player* tiene su *id,* recuperamos a través del *Manager* la *WebSocketSession* asociada a este *player* y, en la Figura 511, intenta enviar a cada uno el mensaje deseado. Pero hay una posible excepción que capturar al enviar el mensaje que puede producirse, por ejemplo, si el navegador del usuario que ha de recibir el mensaje se cierra y ya no hay *WebSocketSession* a la que enviarlo.

```
46    public void notifyStart() {
47        for (User player : players) {
48            HWSession hwSession = Manager.get().getSessionByUserId(player.getId());
49            WebSocketSession wsSession = hwSession.getWebsocketSession();
50
51            JSONObject jso = new JSONObject().
52                    put("type", "MATCH STARTED").
53                    put("matchId", this.id);
54            TextMessage message = new TextMessage(jso.toString());
55            wsSession.sendMessage(message);
56        }
57    }
```

Figura 511. Recorremos la lista de players *e intentamos enviar un mensaje*

Aquí podríamos tomar varias decisiones: lanzar la excepción al método (*playMatch*) que ha llamado a este, o capturarla y tratarla en este mismo lugar. Y, si es una partida de dos jugadores y uno se desconecta cuando está empezando, ¿notificamos al otro de que ha ganado? ¿Y si los dos se desconectan simultáneamente? Para no complicarnos, capturaremos la excepción aquí directamente, en un *try catch*, y ya nos dirá el responsable de negocio qué quiere que hagamos.

```
public void notifyStart() {
   for (User player : players) {
      HWSession hwSession = Manager.get().getSessionByUserId(player.getId());
      WebSocketSession wsSession = hwSession.getWebsocketSession();

      JSONObject jso = new JSONObject().
            put("type", "MATCH STARTED").
            put("matchId", this.id);
      TextMessage message = new TextMessage(jso.toString());
      try {
         wsSession.sendMessage(message);
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

Figura 512. Capturamos la posible excepción al enviar el mensaje

7. Envío de movimientos desde el cliente

Cuando el cliente recibe el mensaje con el *type* MATCH STARTED puede empezar a jugar. Como sabemos, este mensaje se recibe en el método anotado con *@OnMessage* en el *WSClient,* que se lo pasa al *receiver*, que es del tipo *TestOtn2Real,* en el que tendremos que adaptar el código de *receive*.

De momento, simplemente imprimiremos por consola el texto mensaje recibido (Figura 513). Veremos que se producen algunos errores extraños.

```
@Override
public void receive(WSClient client, int event, String message) {
   try {
      if (event == WSClient.ON_OPEN) {
         JSONObject jso = new JSONObject()
               .put("type", "PLAYER READY")
               .put("matchId", this.matchId);
         client.getSession().getAsyncRemote().sendText(jso.toString());
      } else if (event == WSClient.ON_MESSAGE) {
         System.out.println(message);
      } else {
         fail("Unexpected message in the WebSocket");
      }
   } catch (Exception e) {
      fail("Unexpected exception: " + e.getMessage());
   }
}
```

Figura 513. Respondemos al evento onmessage

7.1. Ejecución de *TestOtn2Real*

Como esta clase se conecta al servidor real, debemos lanzar primero el servidor de Eclipse (ejecutando la clase *App)* y asegurarnos de que el docker está escuchando. Entonces podemos lanzar los casos.

El comportamiento que más o menos esperamos, de forma resumida (se han quitado instancias que en este momento no aportan demasiado), es el siguiente:

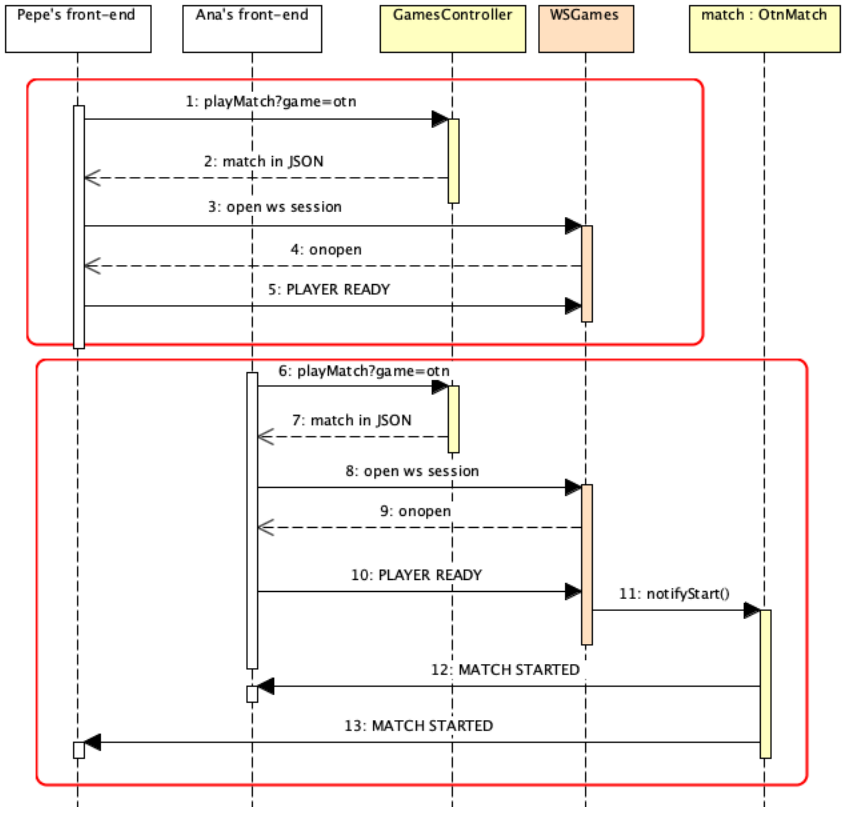


Figura 514. Comportamiento esperado del escenario descrito en TestOtn2Real

Cuando lanzamos los casos tendremos dos consolas correspondientes a los procesos de la aplicación (*App)* y de la ejecución de los casos de prueba (*TestOtn2Real)*, que podemos alternar con el botoncito que aparece en la vista de consolas de Eclipse.

Como, en la Figura 513, hemos indicado que se muestre por consola (en la de los tests) el texto del mensaje recibido, esperaríamos que este se imprimiera dos veces: una porque el mensaje le llega al cliente *wsPepe* y otro porque le llega a *wsAna*.

Sin embargo, la mayoría de las veces aparece solo una vez. En la consola del proceso *App* podemos ver un mensaje de error que dice: *The WebSocket session [c] has been closed and no method (apart from close()) may be called on a closed session*, lo que significa que se ha intentado enviar un mensaje cuando el cliente de WebSocket ya se había cerrado. Esto ocurre porque JUnit, que es quien controla la ejecución del único método *@Test* que tenemos, finaliza la ejecución de su proceso y cierra los dos clientes; el *WSGames* no ha terminado de procesar la información recibida y, cuando intenta enviar, por ejemplo, el mensaje MATCH READY, el receptor ya se ha cerrado.

Para impedir que JUnit salga antes de que se hayan recibido los dos mensajes crearemos en *TestOtn2Real* un objeto *latch* de tipo *CountDownLatch* (algo así como "cerrojo de cuenta atrás"). Esta clase permite que un proceso espere a que otros hayan terminado para seguir. En nuestro caso, tenemos dos procesos asíncronos (los dos clientes), por lo que inicializaremos el campo *latch* con valor 2, y lo decrementaremos (de momento) cuando cada cliente haya recibido el mensaje MATCH READY. Además, añadiremos una sentencia al final de *testPlayOtn* para que el proceso de JUnit espere a que el contador del *latch* sea cero.

```
public class TestOtn2Real implements IWSReceiver {
   ...
   private CountDownLatch latch = new CountDownLatch(2);
   ...

   @Test @Order(1)
   void testPlayOtn() throws Exception {
      ...

      this.latch.await();
   }

   @Override
   public void receive(WSClient client, int event, String message) {
      try {
         ...
         else if (event == WSClient.ON_MESSAGE) {
            System.out.println(message);
            this.latch.countDown();
         }
         ...
   }
}
```

Figura 515. Uso de un CountDownLatch

Si ejecutamos de nuevo, el mensaje se muestra en la consola de tests dos veces, como esperábamos, pero en la de *App* se nos dice que *The remote endpoint was in state [TEXT_PARTIAL_WRITING] which is an invalid state for called method*. Esto significa que se ha intentado enviar un mensaje a un cliente cuando todavía no se había terminado de enviarle el anterior.

En efecto, tenemos un pequeño error en *handleTextMessage:* la ejecución del caso de prueba es tan rápida que los dos mensajes PLAYER READY procedentes de *wsPepe* y de *wsAna* llegan a la vez. En la línea que se resalta en la Figura 516 coinciden los dos hilos enviados por los dos clientes y, al entrar a

notifyStart (teníamos su implementación en la Figura 512) porque *wsPepe* ha dicho que está listo, se le notifica al mismo *wsPepe* que la partida ha comenzado; pero, antes de que el mensaje termine de enviarse, *wsAna* también le pide a este objeto que le notifique a *wsPepe* que la partida ha comenzado, y es ahí en donde se produce el error.

```
@Override
protected void handleTextMessage(WebSocketSession session, TextMessage message) ... {
   JSONObject jso = new JSONObject(message.getPayload());
   String type = jso.getString("type");

   if (type.equals("PLAYER READY")) {
      String matchId = jso.getString("matchId");
      Match match = Manager.get().getMatch(matchId);
      if (match!=null) {
         match.setStarted(true);
         match.notifyStart();
      }
   }
}
```

Figura 516. Código actual de handleTextMessage

Este problema lo podemos detectar con el depurador. Ya una vez (Figura 435, página 272) depuramos dos procesos. Si ahora colocamos un punto de interrupción en la setencia *match.notifyStart*, veremos que coinciden los dos hilos: el desencadenado por *wsPepe* y el de *wsAna:*

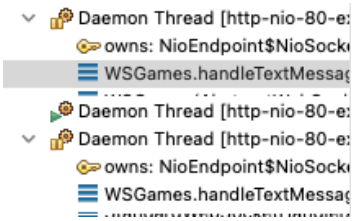


Figura 517. Los dos procesos coinciden al intentar notificar al mismo cliente

Para corregirlo, podemos modificar la instrucción *if (match!null)* para que solo se ejecute *notifyStart* cuando la partida no haya comenzado (Figura 518).

```
@Override
protected void handleTextMessage(WebSocketSession session, TextMessage message) ... {
   JSONObject jso = new JSONObject(message.getPayload());
   String type = jso.getString("type");

   if (type.equals("PLAYER READY")) {
      String matchId = jso.getString("matchId");
      Match match = Manager.get().getMatch(matchId);
      if (match!=null && !match.isStarted()) {
         match.setStarted(true);
         match.notifyStart();
      }
   }
}
```

Figura 518. Limitamos a partidas no comenzadas

La gran mayoría de las veces, esta solución funcionará, pero habrá algunas en que los dos hilos lleguen tan a la vez, que aunque el primero modifique el estado con *setStarted(true),* el segundo consultará *isStarted()* antes de que el estado se haya cambiado. Para evitar este error de concurrencia, sincronizamos la región crítica del método:

```
@Override
protected void handleTextMessage(WebSocketSession session, TextMessage message) ... {
    JSONObject jso = new JSONObject(message.getPayload());
    String type = jso.getString("type");

    if (type.equals("PLAYER READY")) {
        String matchId = jso.getString("matchId");
        Match match = Manager.get().getMatch(matchId);
        if (match!=null) {
            synchronized (match) {
                if (!match.isStarted()) {
                    match.setStarted(true);
                    match.notifyStart();
                }
            }
        }
    }
}
```

Figura 519. Limitamos el acceso a la región crítica

De este modo (también podríamos verlo con el depurador), no habrá dos procesos que entren simultáneamente al bloque *synchronized:* el mensaje llegará una sola vez a cada cliente sin que haya más conflicto.

7.2. Tratamiento del cierre de sesiones *ws*

Con los últimos cambios, el sistema parece comportarse correctamente, pero siguen apareciendo dos feas excepciones en la consola de Eclipse:

```
java.io.EOFException
        at org.apache.tomcat.util.net.NioEndpoint$NioSocketWrapper.fillReadBuffer(NioEndpoint.java:1289)
        at org.apache.tomcat.util.net.NioEndpoint$NioSocketWrapper.read(NioEndpoint.java:1223)
        at org.apache.tomcat.websocket.server.WsFrameServer.onDataAvailable(WsFrameServer.java:72)
        at org.apache.tomcat.websocket.server.WsFrameServer.doOnDataAvailable(WsFrameServer.java:171)
        at org.apache.tomcat.websocket.server.WsFrameServer.notifyDataAvailable(WsFrameServer.java:151)
        at org.apache.tomcat.websocket.server.WsHttpUpgradeHandler.upgradeDispatch(WsHttpUpgradeHandler.java:
        at org.apache.coyote.http11.upgrade.UpgradeProcessorInternal.dispatch(UpgradeProcessorInternal.java:5
        at org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:53)
        at org.apache.coyote.AbstractProtocol$ConnectionHandler.process(AbstractProtocol.java:806)
        at org.apache.tomcat.util.net.NioEndpoint$SocketProcessor.doRun(NioEndpoint.java:1498)
        at org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:49)
        at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
        at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
        at org.apache.tomcat.util.threads.TaskThread$WrappingRunnable.run(TaskThread.java:61)
        at java.lang.Thread.run(Thread.java:750)
java.io.EOFException
        at org.apache.tomcat.util.net.NioEndpoint$NioSocketWrapper.fillReadBuffer(NioEndpoint.java:1289)
        at org.apache.tomcat.util.net.NioEndpoint$NioSocketWrapper.read(NioEndpoint.java:1223)
        at org.apache.tomcat.websocket.server.WsFrameServer.onDataAvailable(WsFrameServer.java:72)
```

Figura 520. Una fea excepción por cada cliente

Esto ocurre porque los clientes, digamos, se marchan "sin despedirse" del servidor. Para que el fin de las conexiones *ws* se trate adecuadamente, insertaremos dos líneas en *testPlayOtn* para que los dos clientes se despidan educadamente:

```
@Test @Order(1)
void testPlayOtn() throws Exception {
    JSONObject jsoPepe = this.playOtn(this.pepeSession);
    JSONObject jsoAna = this.playOtn(this.anaSession);

    assertEquals(jsoPepe.getString("id"), jsoAna.getString("id"));
    this.matchId = jsoPepe.getString("id");

    this.wsPepe = this.connectWebsocket(pepeSession, this);
    this.wsAna = this.connectWebsocket(anaSession, this);

    this.wsPepe.getSession().close();
    this.wsAna.getSession().close();
    this.latch.await();
}
```

Figura 521. Le decimos al servidor que nos marchamos

El método *close* produce en el servidor una llamada a *afterConnectionClosed*. Desde el momento en que se ejecuta este método, la *WebSocketSession* deja de ser válida, por lo que sería conveniente tratarla de forma adecuada. Lo que haremos será escribir código para dicho método en *WSGames*. En la siguiente figura, localizamos el *wrapper* correspondiente a la sesión que se ha cerrado y le ponemos su *WebSocketSession* a *null*.

```
@Override
public void afterConnectionClosed(WebSocketSession session, CloseStatus status) throws Exception
{
   HWSession hwSession = Manager.get().getSessionByWebSocketId(session.getId());
   hwSession.setWebsocketSession(null);
}
```

Figura 522. Tratamiento que damos al cierre de la sesión ws

7.3. Tratamiento del cierre de sesiones *http*

Tal vez la lectora o lector se hayan percatado, navegando por Internet, de que algún sitio web le advierte de que su sesión está a punto de caducar y que si quiere extenderla. En nuestro caso, las sesiones abiertas las mantenemos en varias colecciones del *Manager* en forma de objetos *HWSession*. Es posible que un jugador se loguee, comience una partida multijugador, se marche y vuelva a la pantalla mucho tiempo después, cuando tal vez la sesión *http* ha expirado. Cuando esto suceda, nuestro *backend* debería detectar que esa sesión ha terminado y darle, por ejemplo, la partida por perdida.

Para esto existe la interfaz *HttpSessionListener*, que está pendiente de cuándo las sesiones se crean y de cuándo se destruyen. Así, la destrucción de una sesión por inactividad podrá detectarse y actuarse en consecuencia.

Para ver cómo funciona, crearemos la siguiente clase *SessionListener* en la rama de producción del paquete *edu.uclm.esi.juegos.http*. La anotación *@WebListener* se utiliza para que Spring esté pendiente de diferentes tipos de eventos *http,* como es nuestro caso.

```
package edu.uclm.esi.juegos.http;

import javax.servlet.annotation.WebListener;
import javax.servlet.http.HttpSession;
import javax.servlet.http.HttpSessionEvent;
import javax.servlet.http.HttpSessionListener;

import edu.uclm.esi.juegos.auxi.HWSession;
import edu.uclm.esi.juegos.auxi.Manager;

@WebListener
public class SessionListener implements HttpSessionListener {

   @Override
   public void sessionCreated(HttpSessionEvent se) {
   }

   @Override
   public void sessionDestroyed(HttpSessionEvent se) {
      HttpSession session = se.getSession();
      HWSession hwSession = Manager.get().getSessionByHttpId(session.getId());
      Manager.get().invalidate(hwSession);
   }
}
```

Figura 523. Un SessionListener *que está pendiente de la destrucción de sesiones*

Cuando una sesión se destruya, se ejecutará el evento *sessionDestroyed*. En la Figura 523, lo que hace es recuperar el *wrapper* asociado a la *HttpSession* e invalidarlo.

Para comprobar cómo funciona haremos dos pequeños cambios:

1) Primero, establecemos una duración máxima de un segundo en el recurso web *login* de *UserController:*

```
@PutMapping(value = "/login")
@CrossOrigin(exposedHeaders = "Current")
public String login(HttpSession session, HttpServletResponse response, ...) {
   String name = info.get("name").toString().trim();
   ...
   session.setMaxInactiveInterval(1);
   return session.getId();
}
```

Figura 524. La duración máxima de la sesión es de un segundo

2) Luego, en el método *testPlayOtn* de *TestOtn2Real,* hacemos una pausa de tres segundos antes de solicitar jugar a *Order the numbers:*

```
@Test @Order(1)
void testPlayOtn() throws Exception {
   Thread.sleep(3000);
   JSONObject jsoPepe = this.playOtn(this.pepeSession);
   ...
}
```

Figura 525. Pausa de tres segundos para que la sesión expire

Si colocamos sendos puntos de interrupción en las líneas resaltadas de la Figura 523 y de la Figura 525, veremos que el programa se detiene antes en la primera, en la que la sesión *http* ya ha sido invalidada; cuando ejecutamos *playOtn,* el servidor recibe el *id* de una sesión que no sirve, por lo que se produce un error y el caso de prueba emite veredicto de fallo.

Antes de continuar, eliminemos o comentemos el *setMaxInactiveInterval* y el *Thread.sleep* que acabamos de poner.

8. Desarrollo de una partida de *Order the numbers*

Una partida aleatoria de este juego genera un tablero de 3 x 3 en donde la última casilla está vacía, y las 8 primeras contienen los números del 1 al 8 desordenados aleatoriamente.

8.1. Planteamiento de un posible escenario

Para simular una partida multijugador en la que sepamos que habrá un fin en algún momento, generaremos un tablero un poco especial y sobre el que los dos jugadores, Pepe y Ana, harán determinados movimientos preestablecidos.

Ambos partirán del mismo tablero que, para nuestro propósito, les llegará ordenado, pero cada uno hará diversos intercambios de posiciones. Suponiendo que, en la Figura 526, Pepe es el jugador de arriba y Ana la de abajo, ella ganará la partida en su cuarto movimiento.

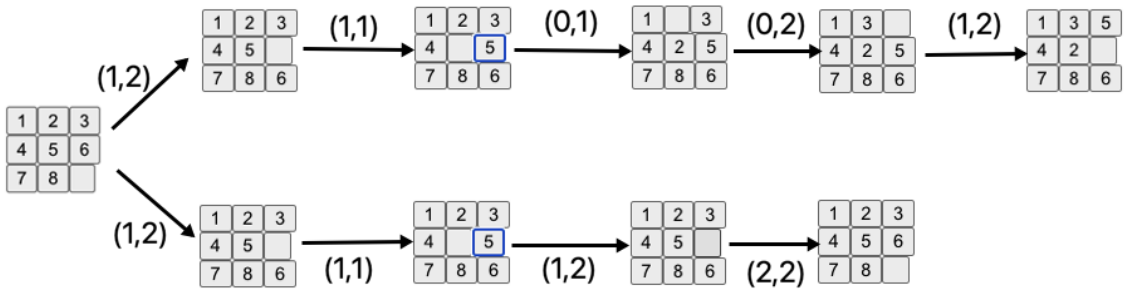


Figura 526. Movimientos que harán Pepe y Ana

El siguiente diagrama resume el paso de mensajes que sucede con los dos primeros movimientos de Pepe:

1) El cliente de WebSocket de Pepe envía al servidor un JSON con el tipo de mensaje que está enviando, las coordenadas y el *id* de la partida.
2) El servidor procesa el movimiento recibido y notifica a los clientes de los dos jugadores que el tablero de Pepe se ha actualizado.

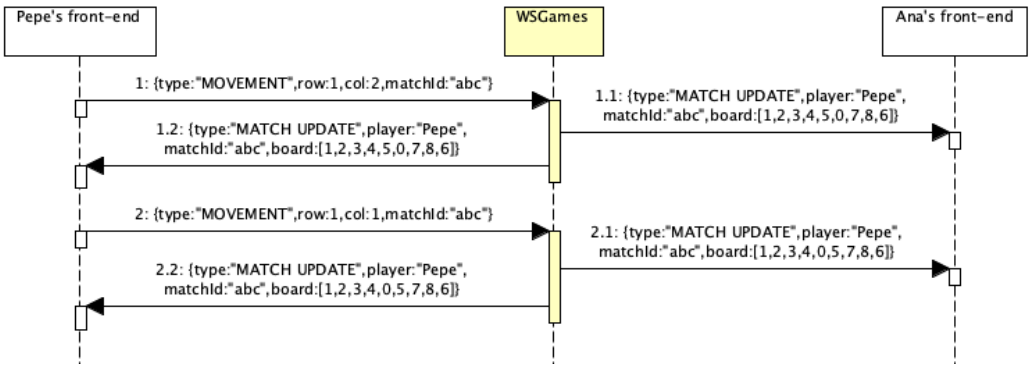


Figura 527. Posible paso de mensajes

Claro, que realmente pasan muchas más cosas, pues intervienen más objetos y hay más mensajes que circulan entre ellos. En la Figura 528 se incluye una aproximación más realista al escenario descrito, y es probable que, al implementarlo, nos demos cuenta de que faltan aún más operaciones u objetos.

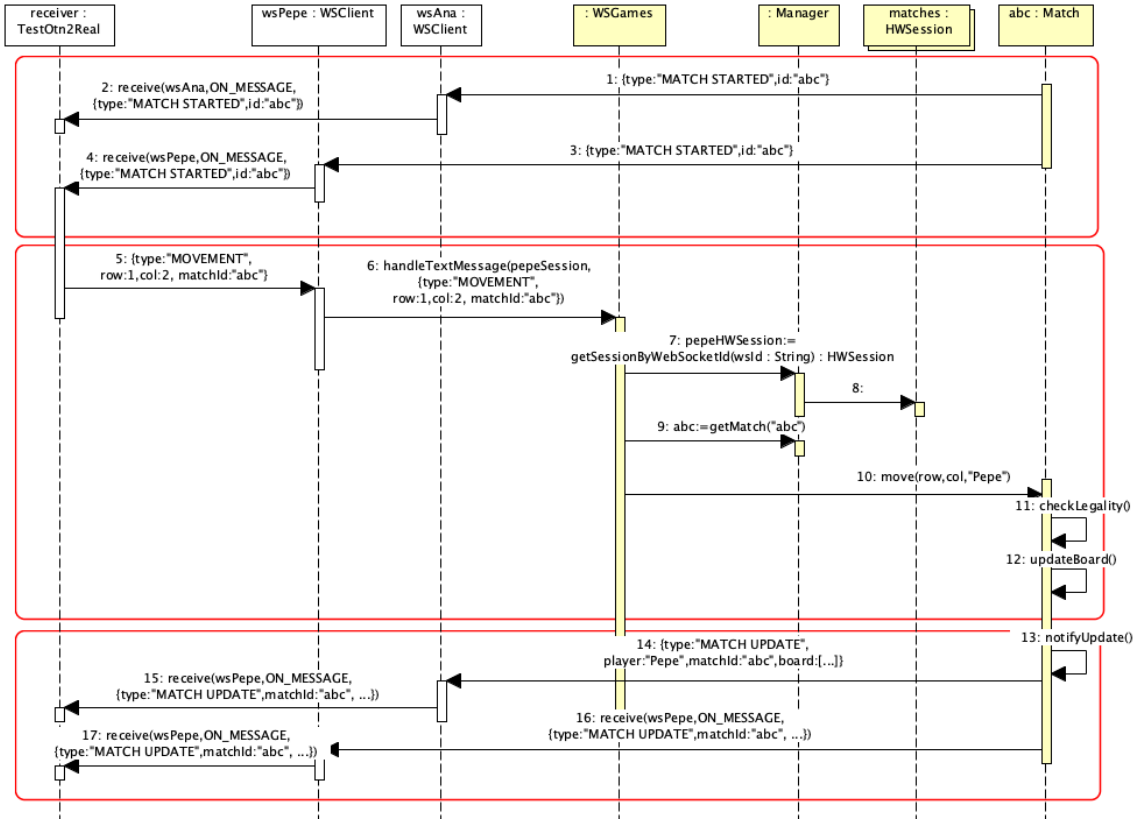


Figura 528. El escenario anterior, más detallado

En cualquier caso, el escenario comienza cuando los dos clientes reciben el mensaje de que la partida con *id="abc"* comienza. Los dos clientes se lo pasan a

su receptor (en nuestro caso es la misma instancia de *TestOtn2Real)*, que programaremos para que comience jugando Pepe, que hará clic sobre la posición (1,2). Este mensaje lo recoge *WSGames*, que, mediante el *Manager*: (1) busca, gracias al *id* de la *WebSocketSession,* al usuario que ha enviado el movimiento y (2) localiza la partida "abc" en la lista de partidas. Cuando ya la tiene (la instancia de más a la derecha en el diagrama, a la que hemos llamado *abc*, de tipo *OtnMatch)*, comprueba que el intercambio de posiciones que ha enviado el jugador es legal, actualiza el tablero y lo comunica a los dos clientes.

8.2. Procesamiento del movimiento en el servidor. Patrón *Template-method*

Conforme a la especificación anterior, el *WSGames* debe procesar mensajes cuyo *type* sea MOVEMENT y realizar las tres acciones indicadas en el diagrama. Podemos añadir, a su método *handleTextMessage*, el fragmento de la Figura 529: se recupera la *HWSession* del usuario, se lee el *matchId* que se ha recibido, se busca la partida, leemos la fila y columna del movimiento y entonces, ya sí, le decimos a la partida que ejecute ese movimiento.

```
else if (type.equals("MOVEMENT")) {
   HWSession hwSession = Manager.get().getSessionByWebSocketId(session.getId());
   String matchId = jso.getString("matchId");
   Match match = Manager.get().getMatch(matchId);
   int row = jso.getInt("row");
   int col = jso.getInt("col");
   match.move(row, col, hwSession.getUserId());
}
```

Figura 529. Traducción a código de los mensajes 7, 8 y 9 de la Figura 528

Pero, pensando ya en que debemos añadir el *Word race* y, quizás, otros juegos, observamos que tal vez no siempre nos sirvan los campos *row* y *col* que se nos han enviado. Si fuese un juego de ajedrez, por ejemplo, quizás nos enviarían las coordenadas de inicio y fin; en la oca, bastaría con la puntuación del dado…

Modificaremos, ahora que estamos a tiempo, la forma en que los clientes envían los mensajes: seguirá habiendo un campo *type:"MOVEMENT"* y otro *matchId,* pero el propio movimiento lo encapsularemos en otro objeto *movement,* cuyo contenido variará en función del juego. Obsérvese en la Figura 530 el pequeño pero importante cambio en el formato de los mensajes.

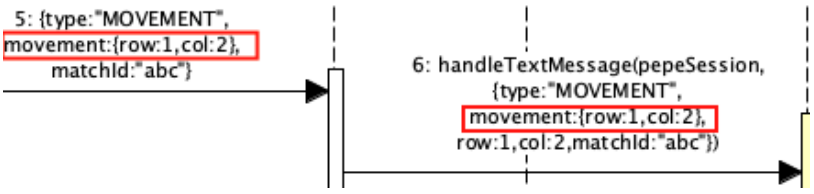


Figura 530. Modificación del mensaje con el movimiento

Ahora, la forma de procesar el movimiento en *WSGames* cambia también ligera pero potentemente:

```
else if (type.equals("MOVEMENT")) {
   HWSession hwSession = Manager.get().getSessionByWebSocketId(session.getId());
   String matchId = jso.getString("matchId");
   Match match = Manager.get().getMatch(matchId);
   JSONObject jsoMovement = jso.getJSONObject("movement");
   match.move(jsoMovement, hwSession.getUserId());
}
```

Figura 531. Recogemos el movimiento como un JSON

De acuerdo con este último fragmento de código, la clase *Match* debe responder a la operación *move(JSONObject, String):* en función del subtipo al que el objeto *match* esté instanciado, el contenido del JSON ha de procesarse de una u otra manera. Por eso, una primera alternativa pasa por hacer *move* abstracta en la clase *Match:*

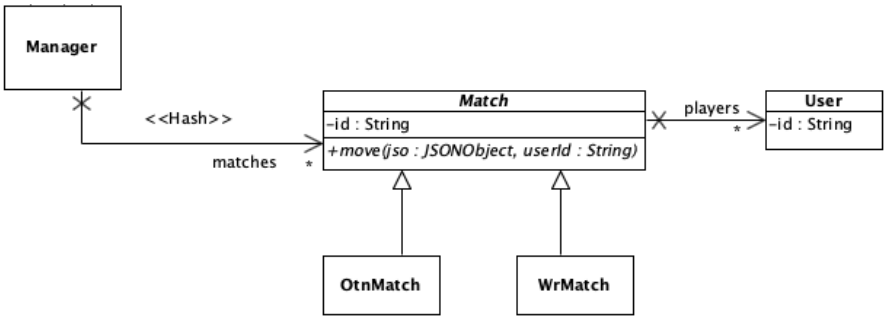


Figura 532. Hacemos *move* abstracta en *Match*

No obstante, si hacemos un poco de zoom en los mensajes 10, 11 y 12 de la Figura 528, podemos entender que el comportamiento del sistema al recibir un movimiento va a ser probablemente el mismo con cualquier tipo de juego (Figura 533): se comprueba la legalidad, se actualiza el tablero y se notifica a los clientes.

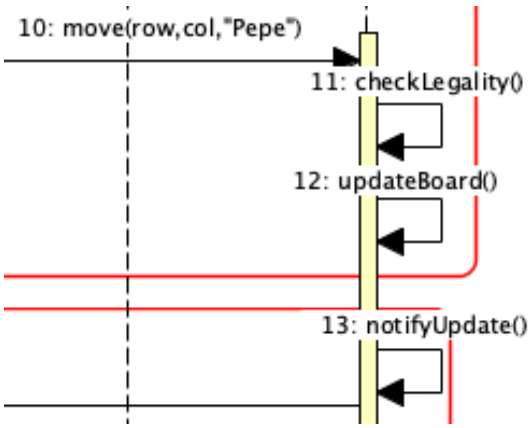


Figura 533. Siempre se hace lo mismo al recibir un movimiento

Comprobar la legalidad y actualizar el tablero dependen del tipo de juego, pero la actualización a los clientes será probablemente siempre igual (pues consiste en enviar un mensaje mediante sus sesiones de WebSocket). Por eso, podemos utilizar el patrón *Template-Method* para describir un comportamiento que es siempre igual, pero que tiene variaciones puntuales que de dependen del subtipo. Ahora (Figura 534), *move* pasa a ser concreta, pero llamará a dos operaciones abstractas (*checkLegality* y *updateBoard)* y a una concreta (*notifyUpdate).*

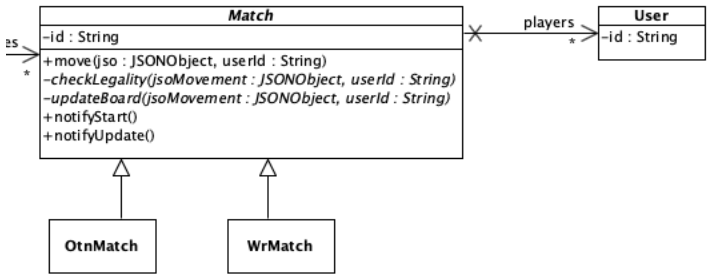


Figura 534. Ya no hacemos move *abstracta*

En *Match*, la traducción a código de este *Template-method* es la siguiente:

```
public void move(JSONObject jsoMovement, String userId) {
   this.checkLegality(jsoMovement, userId);
   this.updateBoard(jsoMovement, userId);
   this.notifyUpdate();
}

protected abstract void checkLegality(JSONObject jsoMovement, String userId);

protected abstract void updateBoard(JSONObject jsoMovement, String userId);

private void notifyUpdate() {
   // TODO Auto-generated method stub
}
```

Figura 535. Nuevas operaciones en Match

8.3. Diseño de los subtipos de *Match*

En cada subclase de *Match* (*OtnMatch, WrMatch* y en otros futuros juegos*)* tenemos que implementar las dos operaciones abstractas indicadas en la figura anterior: ambas reciben como parámetro el movimiento en JSON y el *id* del jugador que ha enviado el movimiento.

Al pensar la implementación de estas operaciones en *OtnMatch* nos encontramos con un primer problema, y es que este subtipo debe tener dos tableros, uno para Pepe y otro para Ana, independientemente de que al principio los dos tableros tengan los dígitos igualmente desordenados.

El *Order the numbers* es muy simple, y tal vez nos baste con tener dos arrays de números, uno por jugador. Pensemos en un juego algo más complicado, como el ajedrez: solo hay dos jugadores pero, en este caso, comparten el mismo tablero; además, la comprobación de legalidad de los movimientos es bastante más complicada en el ajedrez que en el *otn*.

Esta característica de los diferentes tipos de juegos nos hace plantearnos diferentes alternativas de diseño e implementación.

Una posibilidad es la de la Figura 536, en la que cada partida tiene uno o varios tableros abstractos. Cada tablero (nótese la cardinalidad 0..1) puede conocer a un jugador distinto (caso del *Order the numbers)* o puede que no (en el ajedrez, por ejemplo, sobre el tablero juegan las dos mismas personas). El tablero tiene casillas y en cada casilla puede haber piezas (en el *Order the numbers* o en el ajedrez solo puede haber una, pero en el parchís, por ejemplo, pueden coincidir dos fichas sola misma casilla).

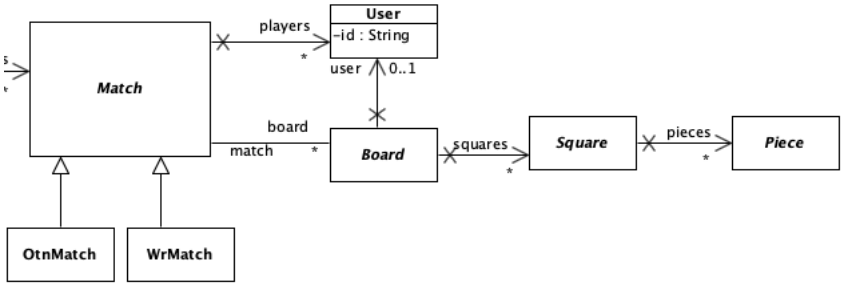


Figura 536. Un posible diseño de Match

El diseño de la figura anterior, con la explicación que hemos dado en el párrafo que la precede, describe bastante bien casi la estructura de cualquier juego de tablero multijugador. Por ejemplo, ese diseño nos sirve para el ajedrez, en donde creamos tantas especializaciones de *Piece* como tipos de piezas hay.

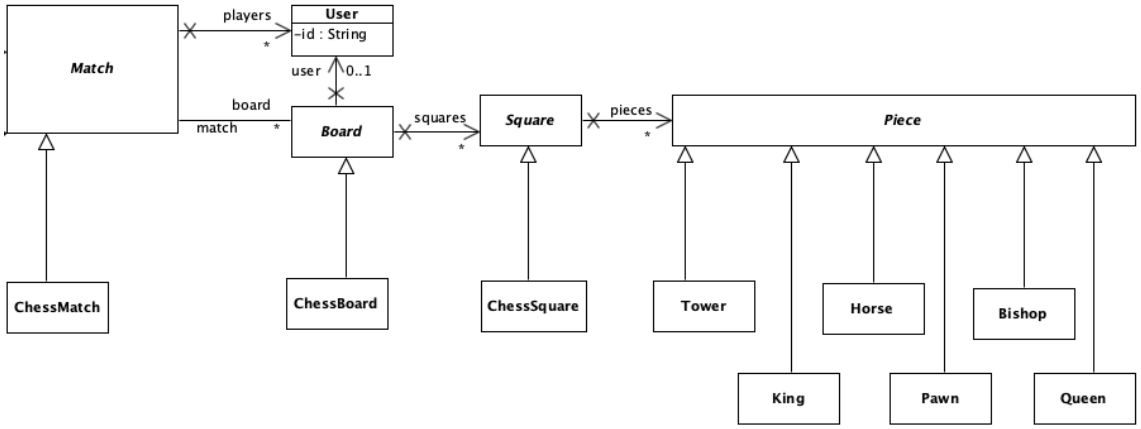


Figura 537. El diseño de la Figura 536 aplicado al ajedrez (I)

Incluso podríamos crear una especialización intermedia abstracta de *Piece* que represente a cualquier ficha de ajedrez y que incorporaría su color (Figura 538, izquierda); o, tal vez, esa especialización intermedia abstracta puede apuntar al jugador que posee esa pieza (Figura 538, derecha). También podríamos incluir, en la *ChessPiece* de la derecha, el campo *color* que tenemos a la izquierda. En fin.

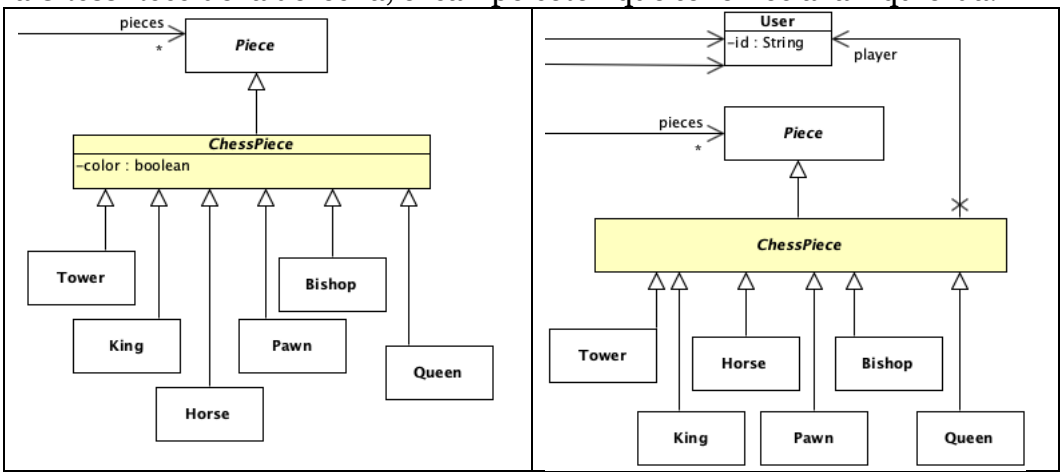


Figura 538. El diseño de la Figura 536 aplicado al ajedrez (II)

Pero ese diseño tan genérico puede imponer restricciones para juegos que puedan ser relativamente simples, como el que nos ocupa. En efecto, para hacer

un *OtnMatch* coherente, deberíamos construir varias clases que tal vez no tengan mucho sentido:

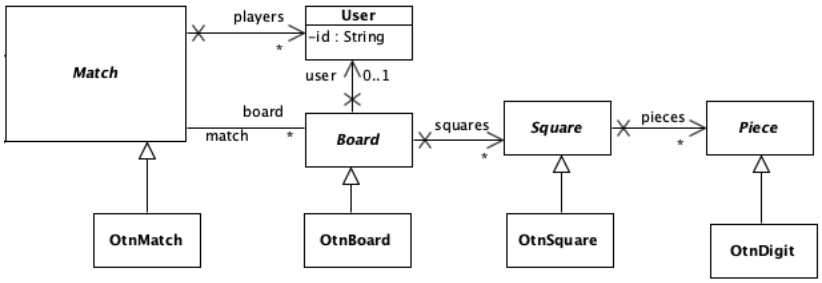


Figura 539. El diseño de la Figura 536 aplicado al Order the numbers

Este sería un buen momento para preguntar al director técnico del proyecto, que quizás se dirigiría al responsable de negocio. Si estuviésemos usando Scrum como metodología de gestión del proyecto, sería tal vez el *Scrum Master* el que debería preguntar al *Product Owner* tras una *Daily Meeting* o la *Spring Review*.

En cualquier caso, supongamos que quien manda sobre nosotros nos dice que sí, que usemos como base la estructura de clases de la Figura 536, pues se prevé que el sistema de juegos online pueda tener cierto éxito, y se deseará extenderlo con otros.

8.4. Clases abstractas

Pues nada, acatamos respetuosamente las órdenes del *boss* y procedemos a la implementación de todas las clases que se derivan de la Figura 539.

Comenzamos por la modificación de la clase *Match* y por la creación de las clases abstractas *Board, Square* y *Piece*. De momento, el código de estas clases es extremadamente sencillo (Figura 540). Por destacar algo, podemos citar que:

1) Todos los campos son protegidos para que sean accesibles directamente en las subclases. Hay programadores que prefieren mantener todos los campos como privados y permitir el acceso a ellos mediante métodos *get:* nosotros no.
2) Puesto que la partida debe llegar al cliente para que este puede pintar los tableros, hemos añadido métodos *get* en cada clase.

```
public abstract class Match {

   protected String id;
   protected List<User> players;
   protected boolean started;
   protected List<Board> boards;

   public Match() {
      this.id = UUID.randomUUID().toString();
      this.players = new ArrayList<>();
      this.boards = new ArrayList<>();
   }

   public List<Board> getBoards() {
      return boards;
   }
```

```
public abstract class Board {
   protected Match match;
   protected List<Square> squares;

   public Board() {
      this.squares = new ArrayList<>();
```

```
    }

    public List<Square> getSquares() {
        return squares;
    }
}
```

```
public abstract class Square {
    protected List<Piece> pieces;

    public Square() {
        this.pieces = new ArrayList<>();
    }

    public List<Piece> getPieces() {
        return pieces;
    }
}
```

```
public abstract class Piece {
    public abstract Object getValue();
}
```

Figura 540. Código inicial de las clases abstractas

Con total seguridad, el código de estas clases que acabamos de escribir cambiará conforme vayamos desarrollando las subclases.

8.5. Implementación de las clases concretas

Nos centraremos ahora en la implementación de las especializaciones para el juego *Order the numbers* que se indicaban en la Figura 539.

8.5.1. *OtnMatch* (I)

Al constructor de esta clase lo llama el método estático *newMatch* que, reflexivamente, llama al constructor del juego que se le indique. Cuando se termina de ejecutar *newMatch,* a la partida ya se le han añadido los jugadores correspondientes. Lo que debe hacer el constructor de *OtnMatch* es crear dos tableros exactamente iguales, que han de añadirse a la colección de tableros de esta partida. De acuerdo con el diseño de la Figura 536, cada tablero conoce a su partida (la asociación es bidireccional), por lo que la llamada al constructor de *OtnBoard* recibe como parámetro la instancia de la propia partida que lo está creando. El método *copy*, al que llamamos para copiar el primer tablero en el segundo, no existe en el tipo de la clase de *board1* (o sea, no existe en *OtnBoard*) y habrá que implementarlo.

```java
public class OtnMatch extends Match {

    public OtnMatch() {
        super();
        OtnBoard board1 = new OtnBoard(this);
        this.boards.add(board1);

        OtnBoard board2 = board1.copy();
        this.boards.add(board2);
    }

    @Override
    public boolean isReady() {
        return this.getPlayers().size()==2;
    }

    @Override
    protected void checkLegality(JSONObject jsoMovement, String userId) {
        // TODO Auto-generated method stub
    }

    @Override
    protected void updateBoard(JSONObject jsoMovement, String userId) {
        // TODO Auto-generated method stub
    }
}
```

Figura 541. Implementación de OtnMatch

Obsérvese también que se ha añadido la implementación de las dos operaciones abstractas (*checkLegality* y *updateBoard*) a las que nos obliga *Match* desde que decidimos aplicar el patrón *Template-method* en la Figura 535.

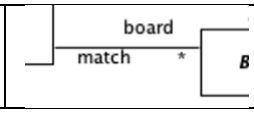
8.5.2. *OtnBoard* (I)

En la segunda sentencia del constructor de *OtnMatch,* Eclipse nos indica (Figura 542) que no hay un constructor con esa signatura en la clase *OtnBoard*.

Figura 542. No existe ese constructor

Bien, pues pulsamos en la segunda solución que nos propone, la cual nos genera en *OtnBoard* el código que tenemos a la izquierda de la Figura 543. Sin embargo, como todos los tableros han de conocer a su partida (centro de la figura), modificamos el código para que se llame al constructor de la superclase (lado derecho).

```java
public OtnBoard(OtnMatch otnMatch) {
    // TODO Auto-generated constructor stub
}
```




```java
public OtnBoard(OtnMatch match) {
    super(match);
}
```

Figura 543. Modificación del constructor de OtnBoard

En el lado derecho de la figura anterior, Eclipse nos dice que no hay constructor en *Board* que responda a ese código, por lo que lo modificamos y lo dejamos como en la Figura 544, de modo que acepte cualquier subtipo de *Match*.

```java
public Board(Match match) {
    this.match = match;
    this.squares = new ArrayList<>();
}
```

Figura 544. Nuevo código en el constructor de Board

El tablero de *Order the numbers* tiene exactamente 9 casillas, en cada una de las cuales hay un dígito del 1 al 8, ambos incluidos. En la última habrá un valor especial, por ejemplo el cero, que denotará que esa casilla está vacía.

Así pues, completamos el código del constructor de *OtnBoard* añadiendo estas casillas, inicialmente ordenadas del 1 al 8, y luego el 0, y llamamos a un método *disorder* que desordenará los números, pero que ya implementaremos. Además, y como nos lo exige el constructor de *OtnMatch,* añadimos también el método *copy* que implementaremos también más tarde.

```
public class OtnBoard extends Board {

   public OtnBoard(OtnMatch match) {
      super(match);
      for (int i=1; i<=8; i++) {
         this.squares.add(new OtnSquare(i));
      }
      this.squares.add(new OtnSquare(0));

      this.disorder();
   }

   private void disorder() {
      // TODO Auto-generated method stub
   }

   public OtnBoard copy() {
   // TODO Auto-generated method stub
      return null;
   }
}
```

Figura 545. Código de OtnBoard

8.5.3. *OtnSquare* (I)

Esta es una clase muy simplona en este juego, que según se planteó tiene únicamente una pieza que será de tipo *OtnDigit* (Figura 546).

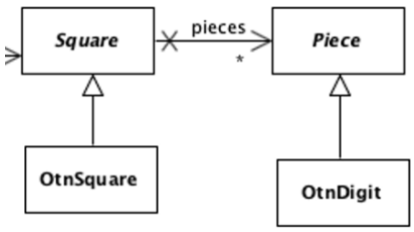


Figura 546. Tableros y piezas en Order the numbers

Por el código del constructor de *OtnBoard* (Figura 545), debe haber un constructor en *OtnSquare* que reciba como parámetro un número. Lo cierto es que, tan simplona es *OtnSquare,* que podría contener directamente directamente el dígito y no usar la colección *pieces:*

```
public class OtnSquare extends Square {

   private int digit;

   public OtnSquare(int digit) {
      this.digit = digit;
   }
}
```

Figura 547. Posible implementación de OtnSquare

Lo comentamos con el jefe: «Mira, es tan trivial este juego, que creo que podemos colocar el dígito perfectamente en *OtnSquare* y pasar de crear una especialización para la *Piece*». Pero el jefe nos dice que no, que nos ciñamos estrictamente al diseño que se propuso, aunque introduzca algo de complejidad.

Pues acataremos de nuevo el deseo de quien manda sobre nosotros y creamos esa clase. Primero, modificamos el código de *OtnSquare:*

```
public OtnSquare(int digit) {
    super();
    OtnDigit digit = new OtnDigit(digit);
    this.pieces.add(digit);
}
```

Figura 548. Implementación que creemos que nos piden

Y, en segundo lugar, pasamos a implementar *OtnDigit*.

8.5.4. *OtnDigit* (I)

«A ver», nos dice el jefe, «mantén la coherencia y no llames a esta clase *OtnDigit,* que puede despistar al ser una especialización de *Piece*. Mejor llámala *OtnPiece*». Y lleva razón, por lo que sustituimos el último trozo de código que hemos mostrado y lo dejamos como en la Figura 549. La lectora o el lector pensará que se podría haber denominado directamente *OtnPiece* a *OtnDigit* desde el principio, pero entonces habría pasado desapercibido el hecho que el autor quiere enfatizar, que es la importancia de mantener ciertas convenciones de nombrado cuando se trabaja en equipo, pues probablemente en un futuro venga alguien a mantener nuestro código, y le despistará leer *OtnDigit* en vez de *OtnPiece*.

```
public OtnSquare(int digit) {
    super();
    OtnPiece piece = new OtnPiece(digit);
    this.pieces.add(piece);
}
```

Figura 549. Implementación que sí que nos piden

También es importante (Figura 550) mantener la coherencia en la documentación del proyecto.

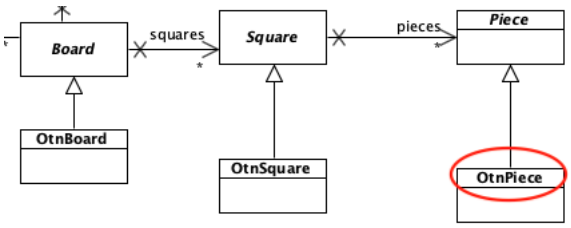


Figura 550. Sin comentarios

Pues de momento, *OtnPiece* es tan sencilla como:

```java
package edu.uclm.esi.juegos.model;

public class OtnPiece extends Piece {

   private int digit;

   public OtnPiece(int digit) {
      this.digit = digit;
   }

   @Override
   public Object getValue() {
      return this.getDigit();
   }
}
```

Figura 551. Código de OtnPiece

8.5.5. *OtnMatch* (II)

OtnMatch debe responder a las operaciones abstractas *checkLegality* y *updateBoard* que hereda de la generalización *Match* (se dice que una superclase es una "generalización" de sus subclases, a las que también se las llama "especializaciones").

Vayamos con la primera operación, *checkLegality,* que toma como parámetros un JSON con los campos *row* y *col* sobre los que el usuario ha hecho clic, y el *id* del usuario que ha enviado el movimiento.

Como se nos ha impuesto seguir ese diseño de *Match, Board,* etcétera, en el *checkLegality* de *OtnMatch* simplemente (Figura 552):

1) Leemos los campos *row* y *col* que nos vienen en el JSON que contiene el movimiento.
2) Si el jugador que juega, cuyo *id* viene en el segundo parámetro, es el primer jugador de la lista de jugadores (en este juego solo hay 2), le pedimos al tablero del primer jugador que compruebe si el movimiento es legal; si no, se lo pedimos al segundo.

```java
@Override
protected void checkLegality(JSONObject jsoMovement, String userId) {
   int row = jsoMovement.getInt("row");
   int col = jsoMovement.getInt("col");
   if (this.players.get(0).getId().equals(userId)) {
      this.boards.get(0).checkLegality(row, col);
   } else {
      this.boards.get(1).checkLegality(row, col);
   }
}
```

Figura 552. checkLegality *en* OtnMatch

Pero *checkLegality* no es un método que exista ni en *Board* ni en *OtnBoard,* por lo que el entorno de desarrollo nos lo subraya diciendo que «Oye, ese método que dices no existe, ¿qué quieres que haga con él?» (Figura 553).

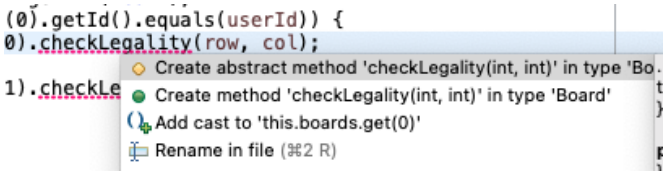


Figura 553. ¿checkLegality?

Nuestro IDE (Eclipse en este caso) nos propone dos soluciones: o bien crear un método abstracto *checkLegality* en *Board,* o bien crearlo también en *Board* como método concreto:

1) Si es abstracto, obligaremos a que todas las subclases de *Board* implementen esa operación y, ya vimos (Figura 530, página 327), a no todos los juegos les viene bien que los movimientos lleguen con solo dos números indicativos de la acción que el usuario ha realizado.
2) Si es concreto, mucho peor, pues tendremos ahí una operación implementada que solo servirá para el *Order the numbers,* y que tendrá que implementarse como vacía, lazando una excepción o de alguna otra manera extraña en cada subclase.

Como soluciones, tenemos:

1) Delegar la ejecución completa de la operación a la clase *Board* (Figura 554). Esta solución es interesante si creamos el método como abstracto, pues cada subclase de *Board* leerá los campos que le interesen del *jsoMovement*.

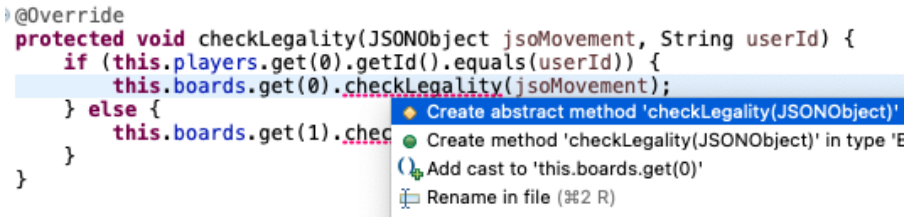


Figura 554. Delegamos la responsabilidad al board *asociado*

2) Como estamos en la subclase *OtnMatch* y tenemos la seguridad de que el tablero sobre el que se ejecuta el movimiento es de tipo *OtnBoard,* podemos seleccionar el tablero según el *userId* y "castearlo" al subtipo *OtnBoard* (Figura 555).

```
@Override
protected void checkLegality(JSONObject jsoMovement, String userId) {
   int row = jsoMovement.getInt("row");
   int col = jsoMovement.getInt("col");
   OtnBoard board;
   if (this.players.get(0).getId().equals(userId)) {
      board = (OtnBoard) this.boards.get(0);
   } else {
      board = (OtnBoard) this.boards.get(1);
   }
   board.checkLegality(row, col);
}
```

Figura 555. "Casteamos" a OtnBoard

Las dos soluciones son buenas, pero los *cast* resultan "poco elegantes" para un profesional de la programación, pues pueden dar a entender que no se maneja correctamente la complicación que pueda tener la herencia. Ojo, porque a veces es imprescindible hacer *casting* de objetos, y de hecho haremos bastantes dentro de pocas líneas. Pero por ahora elegiremos la primera opción, creando el método como abstracto en *Board* (Figura 556).

```
public abstract class Board {
   protected Match match;
   protected List<Square> squares;

   public Board(Match match) {
      this.match = match;
      this.squares = new ArrayList<>();
   }

   public abstract void checkLegality(JSONObject jsoMovement);

   public List<Square> getSquares() {
      return squares;
   }
}
```

Figura 556. checkLegality, *abstracto en* Board

8.5.6. *OtnBoard* (II)

Ahora, en esta clase debemos implementar el método heredado *checkLegality* que acabamos de crear como abstracto en *Board*. En esta especialización sí podemos recoger con seguridad los campos *row* y *col* que nos han llegado desde el cliente. El movimiento será legal cuando la casilla sobre la que se ha hecho clic sea adyacente a la casilla vacía. Si el movimiento no es legal… pues lanzaremos una excepción personalizada a la que llamaremos *GameRuleViolationException*:

```
@Override
public void checkLegality(JSONObject jsoMovement) {
   int row = jsoMovement.getInt("row");
   int col = jsoMovement.getInt("col");
   if (!this.isAdjacent(row, col, this.emptyPosition))
      throw new GameRuleViolationException("Illegal movement");
}
```

Figura 557. Comprobación de que el movimiento es legal

En el trozo de código anterior tenemos hasta tres errores de compilación, pues no se conoce ni la operación *isAdjacent*, ni el campo *emptyPosition* ni la excepción *GameRuleViolationException*.

Empezando por el final:

1) La excepción se lanzará en cualquier juego cuando el movimiento que intenta hacerse sea contrario a las reglas.
2) El campo *emptyPosition* evidentemente no existe: ¿lo creamos específicamente en esta clase, *OtnBoard,* o en la superclase *Board?* Bueno, no a todos los juegos les interesa saber cuál es la posición vacía: de hecho, hay muchos en los que hay más de una posición vacía. En general, solo llevaremos a las superclases lo que interese completamente a las subclases: por esto, si a *OtnChessBoard* (el supuesto tablero de ajedrez) no le interesa el campo *emptyPosition,* no lo llevaremos a la generalización y lo dejaremos en la especialización.
3) Con la operación *isAdjacent* pasa ¾ de lo mismo, que no les interesa a todos los juegos: quizás le interesa al rey del ajedrez; pero no a la ficha del juego de la oca, ni a la reina o al caballo. Por tanto, este método no lo llevaremos tampoco a *Board* y lo dejaremos en *OtnBoard*.

Todos estos errores de compilación nos obligan a hacer varios cambios que, sin embargo, a todos nos parecerán coherentes:

1) Respecto de la excepción, crearemos la clase *GameRuleViolationException* en un paquete

edu.uclm.esi.juegos.exceptions como especialización de *Exception*. Tendrá un constructor que recibirá como parámetro el texto del error (Figura 558).

```
package edu.uclm.esi.juegos.exceptions;

public class GameRuleViolationException extends Exception {

   public GameRuleViolationException(String message) {
      super(message);
   }
}
```

Figura 558. La GameRuleViolationException

2) Lanzar la excepción desde *checkLegality* nos obliga a que este método incluya la cláusula *throws GameRuleViolationException* en su cabecera, por lo que cambian tanto el *checkLegality* de *OtnBoard* (Figura 559, abajo), como el de *Board* (Figura 559, arriba).

```
19      public abstract void checkLegality(JSONObject jsoMovement) throws GameRuleViolationException;
20  }
```

OtnBoard.java

```
21
22      @Override
23      public void checkLegality(JSONObject jsoMovement) throws GameRuleViolationException {
24          int row = jsoMovement.getInt("row");
25          int col = jsoMovement.getInt("col");
26          if (!this.isAdjacent(row, col, this.emptyPosition))
27              throw new GameRuleViolationException("Illegal movement");
28      }
```

Figura 559. checkLegality *en* Board *(arriba) y en* OtnBoard

3) El campo *emptyPosition* hemos decidido no crearlo en la superclase (carece de sentido) y dejarlo en esta subclase. Se trata de la única casilla vacía que hay en el tablero de este juego, por lo que tenemos la seguridad de que se trata de una instancia de tipo *OtnSquare*. Además, al crearse el juego, estará siempre en la última posición del array de casillas: tan seguros estamos de este extremo que declaramos la *emptyPosition* del subtipo *OtnSquare,* y por eso le hacemos un *cast* en el constructor (Figura 560).

```
private OtnSquare emptyPosition;

public OtnBoard(OtnMatch match) {
   super(match);
   for (int i=1; i<=8; i++) {
      this.squares.add(new OtnSquare(i));
   }
   this.squares.add(new OtnSquare(0));

   this.disorder();
   this.emptyPosition = (OtnSquare) this.squares.get(8);
}
```

Figura 560. La emptyPosition *es de tipo* OtnSquare

4) Por último, *isAdjacent* debe devolver *true* si la casilla está junto a la casilla vacía, excluyendo diagonales. Como vemos en la Figura 561, el IDE nos indica que en la clase del parámetro *emptyPosition* no existen ni la operación *getRow* ni *getCol,* por lo que habrá que implementarlas para que cada casilla sepa en qué lugar del tablero se encuentra.

```
private boolean isAdjacent(int row, int col, OtnSquare emptyPosition) {
   return this.emptyPosition.getRow()-1==row && this.emptyPosition. getCol()==col
      || this.emptyPosition.getRow()+1==row && this.emptyPosition.getCol()==col
      || this.emptyPosition.getRow()==row && this.emptyPosition.getCol()-1==col
      || this.emptyPosition. getRow()==row && this.emptyPosition.getCol()+1==col;
}
```

Figura 561. Código de isAdjacent, *muy-muy similar al de la Figura 84 (página 72)*

Hemos comenzado el punto anterior escribiendo "Por último", pero no es cierto, pues hay que implementar *getRow* y *getCol* en *OtnSquare* y, para que estos dos métodos devuelvan correctamente sus coordenadas, estas tienen que estar almacenadas en algún campo que tengan. Y, para que se almacenen correctamente, alguien tiene que decírselas.

Quien mejor lo sabe es el constructor de *OtnBoard* (Figura 562) que, ahora:

1) Crea el array de casillas no directamente del 1 al 8, sino recorriendo filas y columnas (puede ser más sencillo de depurar) y asignando como dígito el valor de la variable *cont,* que se va incrementando.
2) Luego, a la casilla que ocupa la novena posición (la 8 del array) le asigna el valor especial cero.
3) Para terminar, asigna el campo *emptyPosition* a la última casilla.

```
public OtnBoard(OtnMatch match) {
   super(match);
   int cont = 1;
   for (int i=1; i<=3; i++) {
      for (int j=1; j<=3; j++) {
         this.squares.add(new OtnSquare(cont++, i-1, j-1));
      }
   }
   OtnSquare last = (OtnSquare) this.squares.get(8);
   last.setDigit(0);

   this.disorder();
   this.emptyPosition = (OtnSquare) this.squares.get(8);
}
```

Figura 562. Reescritura del constructor de OtnBoard

8.5.7. *OtnSquare* (II)

Este último cambio nos obliga a redactar de nuevo la clase *OtnSquare,* pues ahora su constructor toma tres parámetros: *digit, row* y *col*, que se procesan en la Figura 563.

```
public class OtnSquare extends Square {
   private int row, col;

   public OtnSquare(int digit, int row, int col) {
      super();
      this.row = row;
      this.col = col;
      OtnPiece piece = new OtnPiece(digit);
      this.pieces.add(piece);
   }

   public int getRow() {
      return this.row;
   }

   public int getCol() {
      return this.col;
   }

   public void setDigit(int value) {
      OtnPiece last = (OtnPiece) this.pieces.get(0);
      last.setDigit(value);
   }
}
```

Figura 563. Nuevo código de OtnSquare

8.5.8. *OtnMatch* (III)

Tan adentro hemos ido de clase en clase, que casi no nos hemos dado cuenta de que tenemos errores de compilación en *OtnMatch* (Figura 564). En efecto, cuando dijimos que *checkLegality* podía lanzar una *GameRuleViolationException* (Figura 559), se nos olvidó controlarla en *OtnMatch*.

```
@Override
protected void checkLegality(JSONObject jsoMovement, String userId) {
   if (this.players.get(0).getId().equals(userId)) {
      this.boards.get(0).checkLegality(jsoMovement);
   } else {
      this.boards.get(1).checkLegality(jsoMovement);
   }
}
```

Figura 564. OtnMach *no compila*

El compilador nos obliga o bien a capturar la posible excepción en *OtnMatch,* o bien a cambiar la signatura de la operación en *OtnMatch* y en *Match*. Optamos por supuesto por la segunda opción, pues en cada juego se tratará la *GameRuleViolationException* de forma distinta: en la fila superior de la Figura 565 se ha añadido la cláusula *throws* en *OtnMatch;* en la inferior, a la firma de la operación abstracta en *Match*.

```
@Override
protected void checkLegality(JSONObject jsoMovement, String userId)
   throws GameRuleViolationException {
   ...
}
```

```
protected abstract void checkLegality(JSONObject jsoMovement, String userId)
   throws GameRuleViolationException;
```

Figura 565. checkLegality *ha de lanzar la excepción en todas las especializaciones de* Match

8.6. *OtnBoard* (III)

Nos falta implementar los métodos *disorder* y *copy* en *OtnBoard*. El primero lo dejamos sin implementar, para poder ejecutar la prueba con el tablero

sin desordenar; el segundo nos hace falta para que Pepe y Ana partan del mismo tablero.

```
public OtnBoard copy() {
   OtnBoard result = new OtnBoard((OtnMatch) this.match);
   OtnSquare originalSquare, copySquare;
   for (int i=0; i<9; i++) {
      originalSquare = (OtnSquare) this.squares.get(i);
      copySquare = (OtnSquare) result.squares.get(i);
      copySquare.setDigit(originalSquare.getDigit());
   }
   return result;
}
```

Figura 566. Copia del tablero

8.7. *Match*

Nos falta un pequeño detalle en *Match:* implementar su método *notifyUpdate* para enviar a los clientes los mensajes con las actualizaciones de los tableros.

Tenemos ya en funcionamiento el *notifyStart* (Figura 512, página 319), y este nuevo ha de ser muy parecido. Según el escenario detallado de la Figura 528 (página 326), el mensaje llevará MATCH UPDATE como valor de *type*, incluirá también el *matchId,* añadirá un campo *player* que identifique al jugador que ha movido, y un campo adicional *board* con el contenido del tablero que se actualiza.

Podemos copiar el código de *notifyStart,* pegarlo y modificar el nombre y cuerpo del método, pero en lugar de eso construiremos un único método que nos sirva para enviar cualquier cosa.

Como siempre enviamos (al menos de momento) objetos JSON bastante sencillos, y estos están formados por pares *(clave, valor),* escribiremos un método *notify* que tome un número variable de argumentos en función del mensaje que se quiera enviar.

Véase, en la Figura 567, la nueva implementación de *notifyStart,* que simplemente llama al nuevo *notify* pasando como parámetro el valor que quiere asignar al campo *type.* El segundo parámetro de *notify* es un array que colocamos en el objeto JSON que se va a enviar.

```
public void notifyStart() {
   this.notify("MATCH STARTED");
}

private void notify(String type, Object... keyValue) {
   JSONObject jso = new JSONObject().
         put("type", type).put("matchId", this.id);
   for (int i=0; i<keyValue.length; i++) {
      jso.put(keyValue[i].toString(), keyValue[i+1]);
      i++;
   }

   for (User player : players) {
      HWSession hwSession = Manager.get().getSessionByUserId(player.getId());
      WebSocketSession wsSession = hwSession.getWebsocketSession();

      TextMessage message = new TextMessage(jso.toString());
      try {
         wsSession.sendMessage(message);
      } catch (IOException e) {
         System.err.println("In notify: " +
               wsSession.getId() + "-> " + e.getMessage());
      }
   }
}

private void notifyUpdate() {
   this.notify("MATCH UPDATE", "userId", userId, "board", board.toJSON());
}
```

Figura 567. Refactorización en Match *para notificar a los clientes*

El tercer método de la figura anterior, *notifyUpdate*, pasa también el campo *type* y los campos *userId* y *board* que, sin embargo, no estamos pasando como parámetros. Además, estamos llamando a un método *toJSON* que no existe en el tipo *Board*. De alguna manera tenemos que arreglar la situación.

A *notifyUpdate* lo llamamos desde *move* en ese patrón *Template-method* que aplicamos en su momento:

```
public void move(JSONObject jsoMovement, String userId)
      throws GameRuleViolationException {
   this.checkLegality(jsoMovement, userId);
   this.updateBoard(jsoMovement, userId);
   this.notifyUpdate();
}
```

Figura 568. A notifyUpdate *le faltan parámetros*

El primer cambio es sencillo, pues basta con pasarle los dos parámetros a *notifyUpdate* (Figura 569): el *userId* lo tenemos, pero lo que no tenemos es el *board* sobre el que se ha jugado.

```
public void move(JSONObject jsoMovement, String userId)
      throws GameRuleViolationException {
   this.checkLegality(jsoMovement, userId);
   this.updateBoard(jsoMovement, userId);
   this.notifyUpdate(userId, board);
}
```

Figura 569. Cambios en Match

Para recuperar ese *board*, modificamos el código de *updateBoard* para que lo devuelva y lo asignamos a una nueva variable *board:*

```
protected abstract Board updateBoard(JSONObject jsoMovement, String userId);

public void move(JSONObject jsoMovement, String userId)
        throws GameRuleViolationException {
   this.checkLegality(jsoMovement, userId);
   Board board = this.updateBoard(jsoMovement, userId);
   this.notifyUpdate(userId, board);
}
```

Figura 570. Recuperamos el tablero sobre el que se ha jugado

Añadir el tipo *Board* al método abstracto *updateBoard* nos recuerda que, en *OtnMatch,* el método concreto *updateBoard* estaba sin implementar. Le podemos dar el siguiente código, en el que recuperamos el tablero que corresponda al jugador y, sobre él, ejecutamos un método *update* que aún no tenemos implementado, pero que crearemos como abstracto en *Board* y al que daremos implementación en *OtnBoard*.

```
@Override
protected Board updateBoard(JSONObject jsoMovement, String userId) {
   Board board = this.players.get(0).getId().equals(userId) ?
           this.boards.get(0) : this.boards.get(1);
   board.update(jsoMovement);
   return board;
}
```

Figura 571. updateBoard *en* OtnMatch

Y, por fin, el método *update* en *OtnBoard* puede ser el de la Figura 572, que se apoya en el método auxiliar *getSquare* que también se muestra:

```
@Override
protected boolean update(JSONObject jsoMovement) {
   int row = jsoMovement.getInt("row");
   int col = jsoMovement.getInt("col");
   OtnSquare clickedSquare = this.getSquare(row, col);
   int digit = clickedSquare.getDigit();

   int emptyRow = this.emptyPosition.getRow();
   int emptyCol = this.emptyPosition.getCol();
   this.emptyPosition = clickedSquare;
   this.emptyPosition.setDigit(0);

   OtnSquare changedSquared = this.getSquare(emptyRow, emptyCol);
   changedSquared.setDigit(digit);;

   return this.matchFinished();
}
private OtnSquare getSquare(int row, int col) {
   OtnSquare square;
   for (int i=0; i<this.squares.size(); i++) {
      square = (OtnSquare) this.squares.get(i);
      if (square.getRow()==row && square.getCol()==col)
         return square;
   }
   return null;
}
```

Figura 572. Update *en* OtnBoard

Nos queda por corregir un pequeño error de compilación en *Match*, y es que llamamos a un método *toJSON* sobre *board* que no tenemos implementado (Figura 567). Haremos lo mismo que con *update:* lo creamos como abstracto en *Board* (que devuelva un JSONArray) y lo implementamos en *OtnBoard:*

```
@Override
protected JSONArray toJSON() {
   JSONArray jsa = new JSONArray();
   OtnSquare square;
   for (int i=0; i<this.squares.size(); i++) {
      square = (OtnSquare) this.squares.get(i);
      jsa.put(square.getDigit());
   }
   return jsa;
}
```

Figura 573. toJSON *en* OtnBoard

9. Ejecución del escenario

Ahora que parece que todo compila, falta que funcione, lo cual es harto improbable habida cuenta de la cantidad no despreciable de código de cierta complejidad que hemos escrito.

Vamos a intentar reproducir el escenario de la Figura 526, en el que Pepe y Ana comenzaban una partida con un tablero ordenado, hacían unos cuantos movimientos y ganaba Ana.

Para esto, tenemos que tocar en el método *receive* de *TestOtn2Real,* de modo que los movimientos empiecen a enviarse cuando se reciba un evento ON_MESSAGE con *type* MATCH STARTED. En la Figura 574 se destaca el lugar en el que debe procesarse el mensaje.

```
@Override
public void receive(WSClient client, int event, String message) {
   try {
      if (event == WSClient.ON_OPEN) {
         JSONObject jso = new JSONObject()
               .put("type", "PLAYER READY")
               .put("matchId", this.matchId);
         client.getSession().getAsyncRemote().sendText(jso.toString());
      } else if (event == WSClient.ON_MESSAGE) {
         JSONObject jsoMessage = new JSONObject(message);
         String type = jsoMessage.getString("type");
         if (type.equals("MATCH STARTED")) {

         }
         this.latch.countDown();
      } else if (event == WSClient.ON_CLOSE) {
         System.out.println("Session closed");
      } else {
         fail("Unexpected message in the WebSocket: " + message);
      }
   } catch (Exception e) {
      fail("Unexpected exception: " + e.getMessage());
   }
}
```

Figura 574. Zona de receive *en donde debemos desarrollar la partida*

Como los dos jugadores (reproducimos de nuevo el escenario) comienzan con el mismo movimiento haciendo clic en (1,2), cuando se reciba MATCH STARTED, el cliente debe enviar (según la especificación de la Figura 530, página 327) al servidor un objeto JSON con el campo *type=MOVEMENT,* el *matchId* que corresponda y un campo adicional *movement* que encapsule la fila y columna.

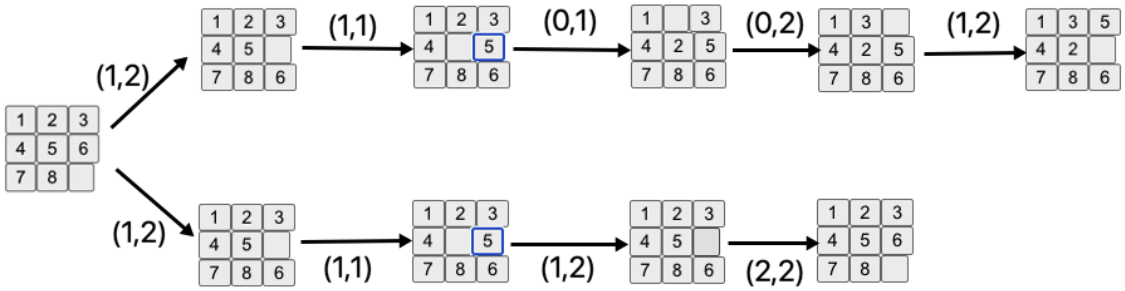


Figura 575. Reproducimos de nuevo la partida para facilitar el seguimiento

```
@Override
public void receive(WSClient client, int event, String message) {
   try {
      ...
      } else if (event == WSClient.ON_MESSAGE) {
         ...
         if (type.equals("MATCH STARTED")) {
            sendMovement(1, 2, client);
         }
         this.latch.countDown();
      }
      ...
}

private void sendMovement(int row, int col, WSClient client) throws JSONException {
   JSONObject coordinates = new JSONObject();
   coordinates.put("row", row).put("col", col);

   JSONObject jsoMovement = new JSONObject();
   jsoMovement.put("type", "MOVEMENT");
   jsoMovement.put("matchId", this.matchId);
   jsoMovement.put("movement", coordinates);
   client.getSession().getAsyncRemote().sendText(jsoMovement.toString());
}
```

Figura 576. Envío del primer movimiento

9.1. Primer intento

Vamos a preparar nuestra pequeña infraestructura. Primero, y para que JUnit no termine antes de que el escenario termine de ejecutarse, vamos a comentar la línea *this.latch.countDown()* en *receive,* y vamos también a comentar el cierre de los dos clientes en el método *testPlayOtn*.

Ahora sí, lanzamos el *docker* y lanzamos en modo *debug* el proceso *App*. Pondremos también un punto de interrupción en la línea *JSONObject jsoMessage = new JSONObject(message)* del *receive* de *TestOtn2Real:* esperamos que los usuarios se registren, se logueen, envíen ambos la petición de jugar al *Order the numbers* y que como hasta ahora, cuando llegue la respuesta, los dos clientes WebSocket de Pepe y Ana se conecten a *WSGames* indicando que están listos. En este momento, *WSGames* envía el MATCH STARTED a los dos clientes y entonces, cuando se reciba en los clientes, se construye el JSON con el mensaje: es en esta línea en donde hemos colocado el punto de interrupción. Esperamos, además, que en la vista del depurador veamos dos hilos detenidos (como en la Figura 517, página 322), pues el mensaje ha llegado a los dos hilos de los dos clientes.

La siguiente figura es bastante interesante: se ha construido el JSON con el mensaje recibido del servidor. Este método toma como primer parámetro (*client)* el *WSClient* que lo ha recibido. Como vemos en la parte de la derecha, vemos que

el identificador del objeto *client* es 115, que coincide con el identificador del objeto *wsPepe* de *this:* es decir, el mensaje que se está procesando es el que ha recibido el cliente de Pepe.

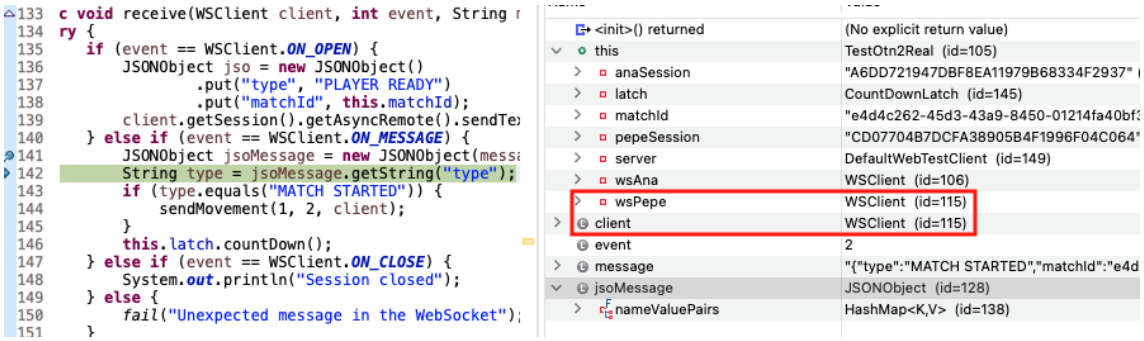


Figura 577. Una instantánea del estado del programa

Si avanzamoscon (*Step over*) un par de líneas llegamos a *sendMovement*. Ahí le podemos dar a *Step into* ( ) para entrar al código de este método. Una vez dentro, volvemos a dar a *Step over* hasta llegar a la última sentencia, en donde vemos (Figura 578) que se ha elaborado un mensaje conforme al formato que espera *WSGames,* y que lo va a enviar el cliente con identificador 115, que es el de Pepe.

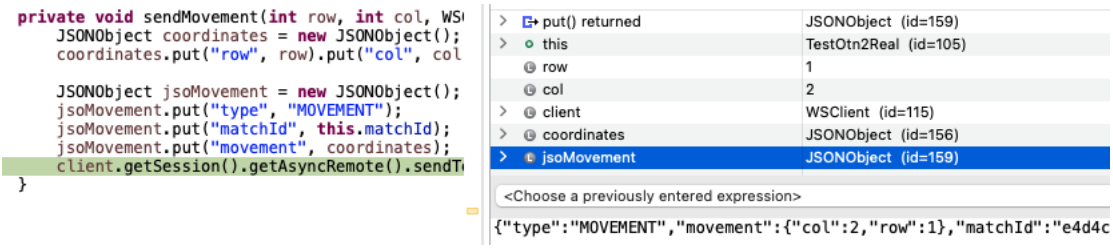


Figura 578. Listos para enviar nuestro primer movimiento

Podemos desactivar todos los puntos de interrupción y soltar la ejecución con *Resume* ( ) y ver qué sucede. Y lo que sucede es que *App* sigue en funcionamiento, pues está escuchando para servir partidas, y que el proceso de test tampoco, pues hemos comentado la línea que decrementa el *CountDownLatch*. Ninguna de las consolas muestra ningún error, con lo que parece que todo va funcionando como esperábamos.

Es conveniente ejecutar varias veces las pruebas con WebSockets, sobre todo en situaciones como la nuestra, en la que estamos haciendo una prueba inicial para ver que todo funciona. Si lanzamos de nuevo el proceso *TestOtn2Real,* es probable que veamos el mismo error *The remote endpoint was in state [TEXT_PARTIAL_WRITING]...* que ya comentamos hace varias páginas: en algún lugar del servidor se está intentando enviar al mismo cliente un segundo mensaje antes de que se le haya terminado en mandar el primero.

¿Dónde enviamos ese segundo mensaje tan rápidamente? Si investigamos en la traza de la excepción, descubrimos que es en el *notify* de *Match:*

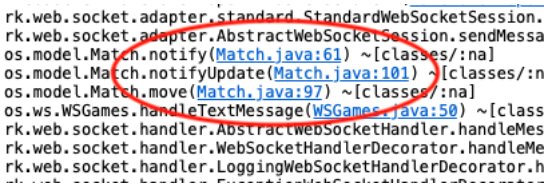


Figura 579. Localizamos el punto en que se produce el error

notify es el método que creamos en la Figura 567 para unificar la forma en la que enviamos mensajes. Lo que haremos será modificar el método para impedir que dos hilos diferentes puedan ejecutar simultáneamente el método *send* de la *WebSocketSession:*

```
private void notify(String type, Object... keyValue) {
   JSONObject jso = new JSONObject().
         put("type", type).
         put("matchId", this.id);

   for (int i=0; i<keyValue.length; i++) {
      jso.put(keyValue[i].toString(), keyValue[i+1]);
      i++;
   }

   for (User player : players) {
      HWSession hwSession = Manager.get().getSessionByUserId(player.getId());
      WebSocketSession wsSession = hwSession.getWebsocketSession();
      synchronized (wsSession) {
         TextMessage message = new TextMessage(jso.toString());
         try {
            wsSession.sendMessage(message);
         } catch (IOException e) {
            System.err.println("In notify: " +
               wsSession.getId() + "-> " + e.getMessage());
         }
      }
   }
}
```

Figura 580. Sincronizamos el acceso al objeto wsSession

Ahora, al ejecutar, no se produce error. Por cierto, que nos vinos bien crear ese método *notify* y no tener código prácticamente duplicado en *notifyStart* y *notifyUpdate,* porque entonces tendríamos que haber sincronizado en los dos lugares.

9.2. Continuación de la partida

Para continuar adecuadamente con el escenario, el método *receive* debe saber cuál es el cliente (*wsPepe* o *wsAna)* que recibe el mensaje para que pueda enviar el siguiente movimiento que corresponda. Esto puede complicar demasiado el método *receive* que, por cierto, ahora debe responder al mensaje con *type=MATCH UPDATE.*

Para mantener el código claro y legible, vamos a crear dos campos de tipo *ArrayList* en *TestOtn2Real* en los que guardaremos los movimientos que deben realizar Pepe y Ana y que inicializaremos en *testPlayOtn.* Para representar cada movimiento del juego usamos la clase *java.awt.Point,* que almacena dos enteros *x* e *y*.

```
private ArrayList<Point> pepeMovements, anaMovements;

@Test @Order(1)
void testPlayOtn() throws Exception {
   pepeMovements = new ArrayList<>();
   pepeMovements.add(new Point(1, 2)); pepeMovements.add(new Point(1, 1));
   pepeMovements.add(new Point(0, 1)); pepeMovements.add(new Point(0, 2));
   pepeMovements.add(new Point(1,2));

   anaMovements = new ArrayList<>();
   anaMovements.add(new Point(1, 2)); anaMovements.add(new Point(1, 1));
   anaMovements.add(new Point(1, 2)); anaMovements.add(new Point(2, 2));

   JSONObject jsoPepe = this.playOtn(this.pepeSession);
   JSONObject jsoAna = this.playOtn(this.anaSession);

   assertEquals(jsoPepe.getString("id"), jsoAna.getString("id"));
   this.matchId = jsoPepe.getString("id");

   this.wsPepe = this.connectWebsocket(pepeSession, this);
   this.wsAna = this.connectWebsocket(anaSession, this);

   //this.wsPepe.getSession().close();
   //this.wsAna.getSession().close();
   this.latch.await();
}
```

Figura 581. Guardamos los movimientos de Pepe y Ana

Ahora, cuando llegue un mensaje de tipo MATCH STARTED o MATCH UPDATE, el cliente que lo reciba debe enviar (con *sendMovement*) el movimiento que le toque, de los que tenemos almacenados en *pepeMovements* y *anaMovements*. Es decir, ya no vamos a pasar las coordenadas en las que el usuario hace clic, sino solamente el *client* que ha recibido el mensaje

```
@Override
public void receive(WSClient client, int event, String message) {
   try {
      ...
      } else if (event == WSClient.ON_MESSAGE) {
         JSONObject jsoMessage = new JSONObject(message);
         String type = jsoMessage.getString("type");
         if (type.equals("MATCH STARTED") || type.equals("MATCH UPDATE")) {
            sendMovement(client);
         }
         //this.latch.countDown();
      } ...
}
```

Figura 582. Cambio en receive

Ahora, para enviar un movimiento, *sendMovement* preguntará por el cliente que ha recibido el mensaje. Si es el de Pepe, retira el primer *point* que haya en *pepeMovements* (si es que hay alguno), y hace lo mismo si es el de Ana.

A continuación, envía el movimiento de la misma forma que antes.

```
private void sendMovement(WSClient client) throws JSONException {
   Point point = null;
   if (client == this.wsPepe && !pepeMovements.isEmpty())
      point = pepeMovements.remove(0);
   else if (!anaMovements.isEmpty())
      point = anaMovements.remove(0);

   if (point==null)
      return;

   JSONObject coordinates = new JSONObject();
   coordinates.put("row", point.getX()).put("col", point.getY());

   JSONObject jsoMovement = new JSONObject();
   jsoMovement.put("type", "MOVEMENT");
   jsoMovement.put("matchId", this.matchId);
   jsoMovement.put("movement", coordinates);
   client.getSession().getAsyncRemote().sendText(jsoMovement.toString());
}
```

Figura 583. Nueva implementación de sendMovement

Si ejecutamos, vemos que no se produce error en ningún y que todo el paso de mensajes fluye tranquila y regularmente sin excepciones.

9.3. Fin de la partida

Cuando Ana envía el último movimiento de *anaMovements,* el servidor debería contestar a los clientes diciendo que la partida ha terminado, y se ha ganado o perdido.

Esto nos supone añadir nueva lógica al servidor, que vamos a añadir en el el método *move* (nuestro *Template-Method)* de *Match:* si, tras actualizar el tablero, no hay ganador, notificamos como hasta ahora; si sí lo hay, notificamos que la partida ha terminado y el jugador que ha ganado.

```
public void move(JSONObject jsoMovement, String userId)
      throws GameRuleViolationException {
   this.checkLegality(jsoMovement, userId);
   Board board = this.updateBoard(jsoMovement, userId);
   if (this.winner==null)
      this.notifyUpdate(userId, board);
   else
      this.notify("MATCH FINISHED", "winner", this.winner.getName());
}
```

Figura 584. Modificación de move *en Match*

No tenemos campo *winner* en la partida, por lo que lo declaramos de tipo *User* y *protected,* para poder tener acceso en las subclases. Crearemos además un método *setWinner(winner : User)* para que el *board,* después de actualizarse, le pueda decir a su *match* quién es el ganador. En este punto, nos viene bien que la relación entre *Match* y *Board* sea bidireccional (Figura 536, 330).

Vamos a modificar el *update* concreto que tenemos en *OtnBoard* (y también el abstracto de *Board*) para que devuelva *true* si la ha partida ha terminado:

```
@Override
protected boolean update(JSONObject jsoMovement) {
   int row = jsoMovement.getInt("row");
   int col = jsoMovement.getInt("col");
   OtnSquare square = this.getSquare(row, col);
   int digit = square.getDigit();
   this.emptyPosition.setDigit(digit);
   this.emptyPosition = square;

   return this.matchFinished();
}
```

Figura 585. Debemos comprobar si la partida termina

El método *matchFinished* simplemente compara los dígitos del tablero actual con la lista ordenada de los números del 1 al 8:

```
private boolean matchFinished() {
   int[] ordered = { 1, 2, 3, 4, 5, 6, 7, 8 };
   ArrayList<Integer> solution = new ArrayList<>();
   OtnSquare square;
   for (int i=0; i<this.squares.size(); i++) {
      square = (OtnSquare) this.squares.get(i);
      if (square.getDigit()!=0)
         solution.add(square.getDigit());
   }
   for (int i=0; i< ordered.length; i++)
      if (ordered[i]!=solution.get(i))
         return false;
   return true;
}
```

Figura 586. matchFinished *en* OtnBoard

Ahora volvemos a *OtnMatch,* cuyo *updateBoard* debe asignar valor al campo *winner* si la partida ha terminado:

```
@Override
protected Board updateBoard(JSONObject jsoMovement, String userId) {
   User player;
   Board board;
   if (this.players.get(0).getId().equals(userId)) {
      player = this.players.get(0);
      board = this.boards.get(0);
   } else {
      player = this.players.get(1);
      board = this.boards.get(1);
   }

   boolean matchFinished = board.update(jsoMovement);
   if (matchFinished)
      this.winner = player;
   return board;
}
```

Figura 587. Reescribimos el updateBoard *en* OtnMatch

Si lanzamos nuevamente los casos de prueba y tenemos la precaución de poner *System.out.println(message)* al principio de *receive* para ver los mensajes que llegan a los clientes, veremos que en la consola de *TestOtn2Real* aparecen todos los que han ido llegando, desde los *null* del *ON_OPEN* hasta los del fin de partida:

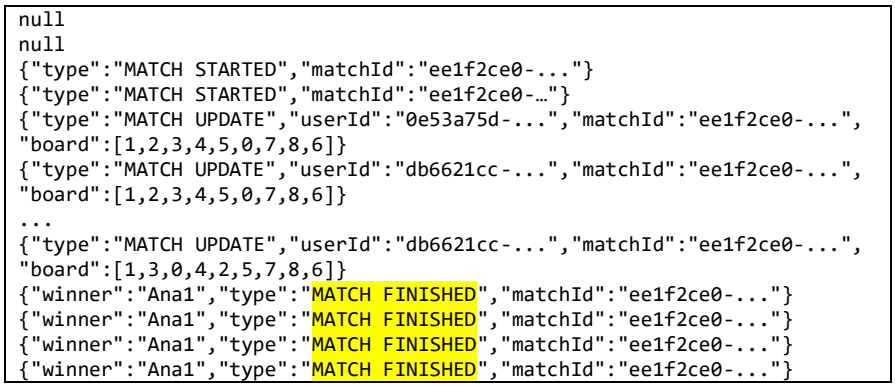


Figura 588. Mensajes recibidos en los clientes

El proceso de pruebas, sin embargo, no para, porque eliminamos esa instrucción *countDown* que teníamos por ahí. Debemos salir del proceso cuando se hayan recibido los 22 mensajes: inicializamos a 22 el campo *latch* y ejecutamos *countDown* con cada mensaje recibido. En *testPlayOtn* cerramos los dos clientes después de que se ejecuta el *latch.await()* que, como sabemos, sucederá cuando el contador del *latch* llegue a cero.

9.4. Control de excepciones: probando un movimiento ilegal

Veamos qué sucede si Pepe envía como primer movimiento un clic en (0, 0), que es una casilla no adyacente con la vacía: esperamos que *checkLegality* lance la *GameRuleViolationException,* que se propagará a su método llamante (*move)* que, a su vez, la lanzará el *handleTextMessage* de *WSGames*.

Cuando esa excepción llega a *handleTextMessage,* puesto que no la estamos tratando, la conexión con el cliente se cierra y se producirá un error a continuación. En la Figura 589 se ve la traza de la excepción que se ha producido, que ha seguido los pasos de *handleTextMessage, move* y los *checkLegality* de *OtnMath* y *OtnBoard*.

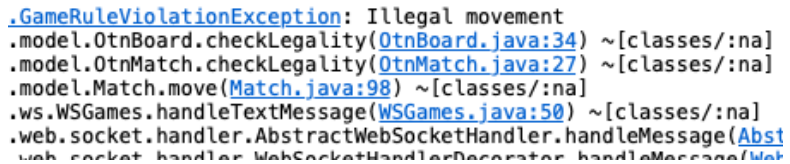


Figura 589. Excepción por movimiento ilegal

En efecto, la excepción va volviendo por el camino inverso y llega a *handleTextMessage,* que no tiene instrucciones sobre cómo tratarla, por lo que cierra la sesión y, a partir de ese momento, se impide toda comunicación con ese cliente.

Para evitarlo, capturamos la excepción en *handleTextMessage* cuando vayamos a enviar el movimiento:

```
@Override
protected void handleTextMessage(WebSocketSession session, TextMessage message)
      throws Exception {
   ...
   } else if (type.equals("MOVEMENT")) {
      ...
      try {
         match.move(jsoMovement, hwSession.getUserId());
      } catch (GameRuleViolationException e) {
         // TODO: handle exception
      }
   }
}
```

Figura 590. Captura de la GameRuleViolationException *en* WSGames

Aquí no podemos devolver un código 403, pues no estamos en *http* sino en *ws*. Lo que sí podemos hacer, por si debe el cliente debe hacer algo con la respuesta, es enviar un mensaje de error:

```
@Override
protected void handleTextMessage(WebSocketSession session, TextMessage message)
      throws Exception {
   ...
   } else if (type.equals("MOVEMENT")) {
      ...
      try {
         match.move(jsoMovement, hwSession.getUserId());
      } catch (GameRuleViolationException e) {
         JSONObject jsoError = new JSONObject();
         jsoError.put("type", "ERROR")
            .put("reason", e.getMessage())
            .put("matchId", matchId)
            .put("player", hwSession.getUserId());
         TextMessage response = new TextMessage(jsoError.toString());
         session.sendMessage(response);
      }
   }
}
```

Figura 591. Enviamos un mensaje con el valor ERROR en el campo type

Al ejecutar, en algún momento llegará a la consola de tests dos mensajes (se envían a Pepe y a Ana) como el siguiente:

```
{"reason":"Illegal
movement","matchId":"c651c5bf...","type":"ERROR","player":"db6621cc-a060-4c50-acfc-
e7f0e8a47363"}
```

La partida ya no se para y sigue, y se llegan a recibir los mensajes MATCH FINISHED, pero el proceso de test no se detiene, y JUnit parece querer estar esperando eternamente. Como sabemos que los casos de prueba han de terminar cuando se recibe un mensaje con este *type,* modificamos el método *receive* para que, cuando llegue un mensaje cuyo *type* sea MATCH FINISHED, decrementaremos el contador del *latch* hasta que este sea 0:

```
@Override
public void receive(WSClient client, int event, String message) {
   try {
      ...
         else if (type.equals("MATCH FINISHED")) {
            while (this.latch.getCount()>=0)
               this.latch.countDown();
         }
      }
   ...
}
```

Figura 592. Forzamos la terminación del proceso

Capítulo 23. *Otn* para dos jugadores: lado *front-end*

1. Reorganización del *front*

Se ha reorganizado el lado cliente no para que nos den un premio al mejor diseño web, sino para que sea, al menos, un poquito más usable. Además, esta reestructuración nos permitirá introducir los *EventEmitter,* con los que podemos crear nuestros propios eventos a los que (igual, por ejemplo, que al evento *click)* escucharán los componentes.

1.1. *app.component.ts*

La Figura 593 muestra el modelo del componente principal: además del *title,* se han definido cinco campos para almacenar el nombre del usuario, si está logueado, y otros tres para determinar si debe mostrar el componente de *login,* de *register* y del modo de juego multijugador.

```
export class AppComponent {
  title = 'fejuegos';
  userName? : string
  userLogged : boolean = false
  showingLogin : boolean = false
  showingRegister : boolean = false
  showingMultiplayer : boolean = false

  constructor(private router : Router) {}

  goTo(page : string) {
    this.router.navigate([page])
  }

  showLogin() {
    this.showingRegister = false
    this.showingLogin = true
    this.showingMultiplayer = false
  }

  showMultiplayer(userName : string) {
    this.userName = userName
    this.userLogged = true
    this.showingRegister = false
    this.showingLogin = false
    this.showingMultiplayer = true
  }
}
```

Figura 593. Código del modelo del componente principal

En función de los eventos que el usuario haga sobre la vista, se ejecutarán los métodos *showLogin* y *showMultiplayer,* como veremos a continuación.

1.2. *app.component.html*

La vista se ha tenido que adaptar forma coherente con el modelo (Figura 594). Como vemos:

1) Si el usuario no está logueado y no se están mostrando ni el componente de loguin ni el de registro, se le ofrecen dos *anchors* para que, si hace *click* en ellos, se muestre el componente que seleccione.

2) Si no está logueado y se debe mostrar el componente de *login,* se le muestra (instrucción <*app-login (onUserLogged)*=...>). El *(onUserLogged)* es un evento del que el componente en el que estamos (*app)* debe estar pendiente: si se produce, se ejecutará el método *showMultiplayer* sobre el modelo (que tenemos en la figura anterior).
3) Si se le debe mostrar el componente de registro porque *showingRegister* sea *true,* se le muestra, y estaremos atentos al evento *onUserRegistered* para mostrarle el componente de login (es elegante que, si un usuario se registra correctamente, lo conduzcamos a un lugar desde el que pueda loguearse).
4) Si hay que mostrar el componente de jugar contra un oponente porque *showingMultiplayer* es *true,* mostraremos el nombre del usuario y el componente <*app-multiplayer*>.
5) Y luego, en cualquier caso, dejaremos al usuario la opción de jugar una partida individual en el último <*div*>.

```
<div>
  <strong>Play against others</strong>
  <br>
  <div *ngIf="!userLogged">
    <a *ngIf="!showingLogin && !showingRegister" (click)="showingLogin=true">
      Click to login
    </a>

    <a *ngIf="!showingLogin && !showingRegister" (click)="showingRegister=true">
      Create an account
    </a>
  </div>

  <div *ngIf="!userLogged && showingLogin">
    <app-login (onUserLogged)="showMultiplayer($event)"></app-login>
  </div>

  <div *ngIf="showingRegister">
    <app-register (onUserRegistered)="showLogin()"></app-register>
  </div>

  <div *ngIf="showingMultiplayer">
    Logged as {{ userName }}
    <br>
    <app-multiplayer></app-multiplayer>
  </div>
</div>
<div>
  <strong>Play with yourself</strong>
  <br>
  <a (click)="goTo('otn')">Play Order the numbers</a>
     
  <a (click)="goTo('wr')">Play Word race</a>
</div>

<router-outlet></router-outlet>
```

Figura 594. Código de la vista del componente principal

1.3. Estructura de clases

La estructura de clases en UML que tenemos en este momento es la de la Figura 595. Hay que entender que UML es un lenguaje para modelar y visualizar sistemas orientados a objetos, y la estructura de los proyectos de Angular (y de otros *frameworks*, incluyendo el proyecto más básico en el que se mezclan *html* y JavaScript) combina parte de objetos (el modelo) con la vista, por lo que habrá

quien pueda ser crítico con los diagramas que venimos dibujando para la parte del *front-end*. Sea como sea:

1) El componente de arranque (*app.component)* puede o no estar mostrando los componentes *login, register* y *multiPlayer,* según los valores de *showingLogin, showingRegister* y *showingMultiplayer*.
2) Al cargar el componente de arranque, este se suscribe a los eventos *onUserLogged* y *onUserRegistered,* que son los *EventEmitter* de *app-login* y *app-register* y que, al producirse, ejecutarán en *app* los métodos *showLogin* y *showMultiplayer*.
3) Cuando, por ejemplo, un usuario se loguea correctamente, el método *login* del *LoginComponent* emite el evento *onUserLogged,* al que está suscrito el componente de arranque. Este (*app.component)* ejecuta su método asociado al evento que se ha disparado (*showMultiplayer)*.

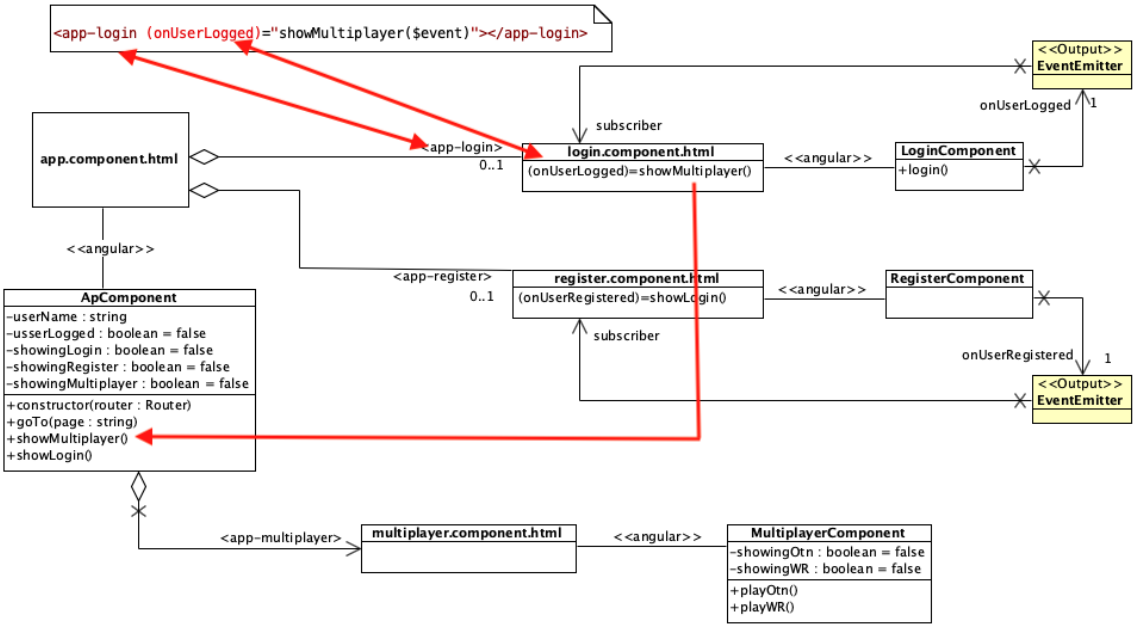


Figura 595. Introducimos emisores de eventos

El paso de mensajes que sucede desde que se carga la aplicación hasta que Pepe se loguea correctamente puede ser el siguiente:

1) En los mensajes 1 y 2, se carga el componente principal que, a su vez, instancia el componente *login*. Este, en el mensaje 3, crea el emisor de del evento *onUserLogged*. Realmente pasan muchas más cosas, que controla Angular, pero indicamos solamente las que nos resultan más interesantes.
2) En el mensaje 4, y puesto que *app.component.html* tiene incrustado el componente *app.login.html* con el atributo *(onUserLogged)=showLogin(),* se suscribe al emisor de eventos.
3) Luego, el usuario hace clic en el enlace *Click to login,* introduce sus credenciales, se validan en el servidor y, como han sido correctas, se ejecuta el código asociado al *next* de la petición. En este *next* se llama al método *emit* de *onUserLogged:* como a este *EventEmitter* se suscribió el componente principal, este ejecuta el método asociado al evento, *showMultiplayer*.

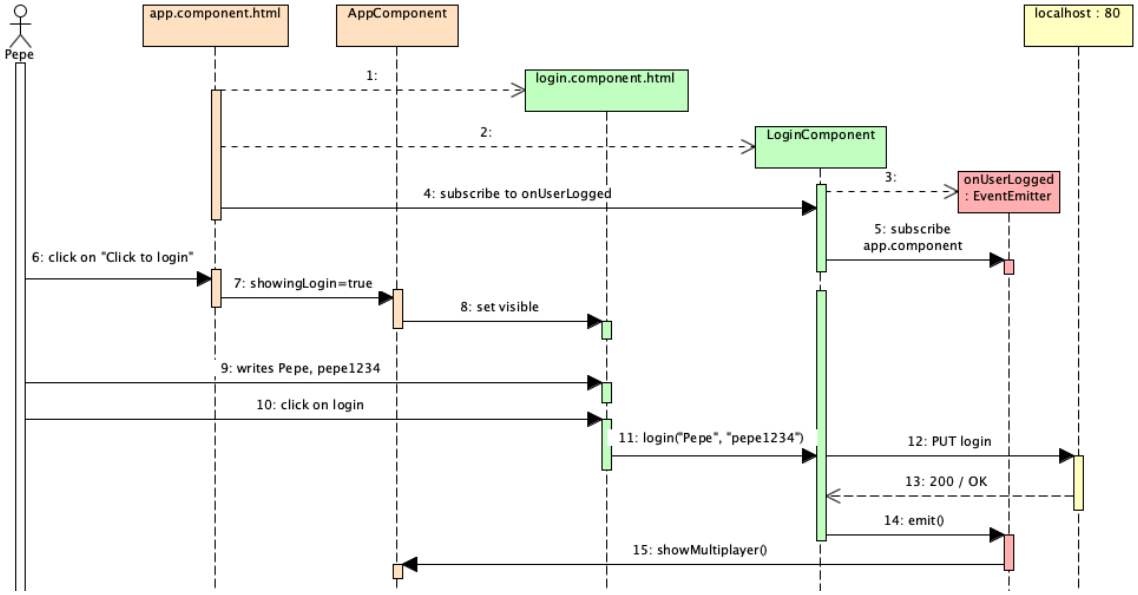


Figura 596. Suscripción y difusión del evento onUserLogged

2. Componentes de los juegos multijugador

En la siguiente figura se han incluidos los dos componentes para jugar al *Order the numbers* o al *Word race,* cuya visualización se controla mediante los campos *showingOtn* y *showingWR* del modelo implementado en la clase *MultiplayerComponent*.

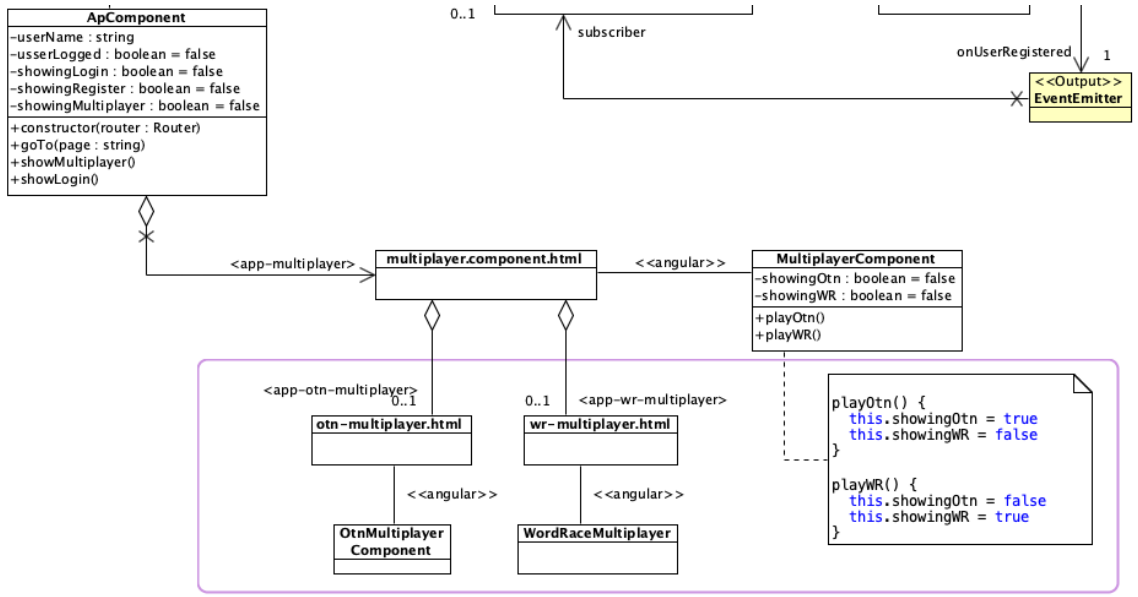


Figura 597. Inclusión de los componentes multijugador

El código de la vista (*multiplayer.component.html)* es muy sencillo: si ninguno de los campos asociados en el modelo (*MultiplayerComponent* en la Figura 597; *multiplayer.component.ts* en el fichero del proyecto en Visual Studio Code) es *true,* entonces se muestran sendos botones con los textos *Play Order the numbers against an opponent* y *Play Word race against an opponent*. Si el usuario hace clic en el aguno de ellos, se ejecutará el método asociado al evento (respectivamente *playOtn* y *playWR)* que, como se ve en el comentario de la figura anterior, ponen a *true* uno de los dos campos. Como Angular está

pendiente de mantener la sincronización entre modelo y vista (de ahí que estas asociaciones las estemos estereotipando como *«angular»)*, se mostrará el componente <*app-otn-multiplayer*> o el <*app-wrmultiplayer*>.

```
<button *ngIf="!showingOtn && !showingWR" (click)="playOtn()">
    Play Order the numbers against an opponent
</button>
  
<button *ngIf="!showingOtn && !showingWR" (click)="playWR()">
    Play Word race against an opponent
</button>
<br>
<i>{{ message }}</i>

<app-otn-multiplayer *ngIf="showingOtn"></app-otn-multiplayer>

<app-wrmultiplayer *ngIf="showingWR"></app-wrmultiplayer>
```

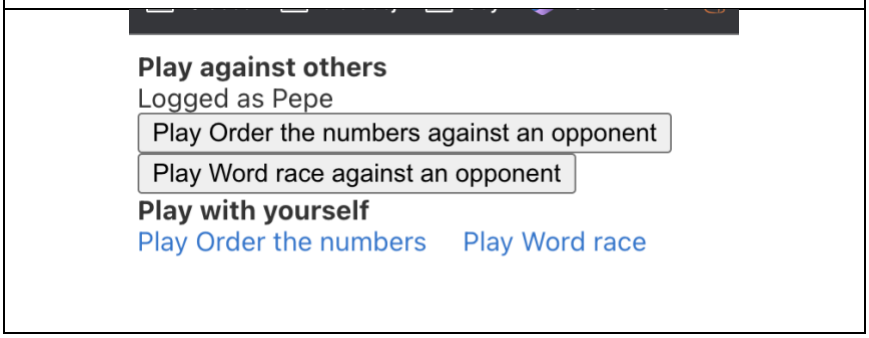


Figura 598. Código de multiplayer.component.html. Abajo, aspecto cuando Pepe se ha logueado correctamente

3. Cambios en *login*

Recordará el lector que, para establecer la conexión *ws* con el servidor *WSGames* y que este identificara que la *WebSocketSession* que los interceptores le inyectan procede de la *HttpSession* del usuario que se logueó (que el *UserController* envía como respuesta al método *login*), enviábamos en la *url* de conexión a *WSGames* un parámetro *httpSessionId* que se procesaba en *afterConnectionEstablished*.

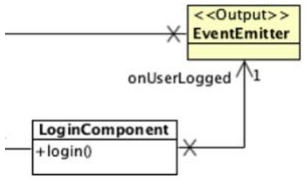
Ocurre que debemos leer ese *id* de la sesión *http* que nos devuelve *login,* por lo que debemos modificar el código que teníamos en la Figura 460, página 285, para recogerlo y, luego, poder agregarlo a la url de conexión al WebSocket del servidor. Antes solo recogíamos la cabecera *current* que nos llegaba del servidor, que almacenábamos en el *sessionStorage*; ahora, además:

1) Debemos declarar el *EventEmitter* que anunciábamos en la Figura 595.
2) Si el *login* es satisfactorio (respuesta 200, manejador *next*), entonces:
    a. Recogemos el cuerpo de la respuesta (el *id* de la sesión *http)* y lo guardamos también en el *sessionStorage*.
    b. Emitimos el evento *onUserLogged*.

```
export class LoginComponent implements OnInit {
  name? : string
  pwd? : string
  error? : string

  @Output() onUserLogged : EventEmitter<any> =
      new EventEmitter()

  constructor(private userService : UserService, private router : Router) {
    this.name = "Pepe"
```



```
    this.pwd = "12345678"
  }

  ngOnInit(): void {
  }

  login() {
    let user = {
      name : this.name,
      pwd : this.pwd
    }
    this.userService.login(user).subscribe({
      next : (response : any) => {
        sessionStorage.setItem("current", response.headers.get("current"))
        sessionStorage.setItem("httpSessionId", response.body)
        this.error = undefined
        this.onUserLogged.emit(this.name)
      },
      error : (result : any) => {
        this.error = result.error.message
      }
    })
  }
}
```

Figura 599. Modificación de LoginComponent

Además, esto nos exige un pequeño cambio en *UserService:*

1) Además de mantener la petición con la cabcera *withCredentials : true* y la indicación de que queremos observar la respuesta completa (*observe:"response"*), como ya explicamos alrededor de la Figura 459 (página 284), indicamos que la respuesta que esperamos es de tipo "text" (correspondiente al tipo MIME *text/plain).*
2) Por último, cambiamos el tipo contenido en el observable a *Object,* y eliminamos el tipo <*any*> que teníamos a continuación del método *put*. El lector o lectora dirá «¿Y por qué no poner directamente <*string*> en ambos lugares, puesto que se espera una cadena?». Pues porque no compila, tal vez porque se está enviando la respuesta completa (cabecera y cuerpo). Una búsqueda en Internet nos ha dado la solución de la Figura 600.

```
login(user : any) : Observable<Object> {
  return this.client.put<any>(httpUrl + "users/login", user,
    {
      withCredentials : true,
      observe : "response",
      responseType : "text"
    })
}
```

Figura 600. Cambios en el login *del* UserService

4. Inicio del juego multijugador: mensajes *http* y *ws*

En el lado cliente, debemos modificar las operaciones *playOtn* y *playWR* (de momento, solo la primera) del *MultiplayerComponent* para que se pueda reproducir el escenario del inicio de partida que se describió en la Figura 514 (página 320) y, además, se realicen en el *front* las operaciones de mostrar y ocultar componentes que correspondan.

Lo primero que necesitamos es enviar una petición *htpp* al *GamesController* diciéndole que queremos jugar al *otn*. Para esto implementamos en el cliente un servicio de acceso a ese controlador:

```
export class GamesService {

  constructor(private client : HttpClient) { }

  playMatch(gameName : String) : Observable<any> {
    return this.client.get<any>(httpUrl + "games/play?gameName=" + gameName,
      {
        withCredentials : true,
        observe : "response"
      }
    )
  }
}
```

Figura 601. Código de GamesService

Una vez implementado el servicio, y como lo usaremos en el *MultiplayerComponent,* debemos inyectarlo en el constructor de este. Usando el *MultiplayerComponent,* el usuario pulsará el botón *Play order the numbers against an opponent* de la Figura 598, lo que lanza la ejecución del método *playOtn* en el modelo. Este método, mediante el *GameService,* envía la petición al *GamesController* y, cuando se recibe la respuesta, y volviendo de nuevo al escenario de la Figura 514, el cliente debe establecer la conexión *ws* con *WSGames*.

El nuevo código de *playOtn* puede ser el de la Figura 602: cuando llega la respuesta, se muestra el componente the *Order the numbers,* se oculta el otro, se muestra un mensaje y se establece la conexión *ws* con el servidor, agregando a la url el parámetro *httpSessionId* que tenemos almacenado en el *sessionStorage*. Se espera que *WSGames* cree la conexión y se ejecute la función asociada al manejador *onopen*, que envía el mensaje *PLAYER READY* al servidor.

```
export class MultiplayerComponent implements OnInit {
  showingOtn : boolean = false
  showingWR : boolean = false
  message? : String
  ws? : WebSocket

  constructor(private gamesService : GamesService) { }

  ngOnInit(): void {
  }

  playOtn() {
    this.gamesService.playMatch("otn").subscribe({
      next : (response) => {
        this.showingOtn = true
        this.showingWR = false
        this.message = "Waiting for an opponent..."
        this.ws = new WebSocket(wsUrl + "wsGames?httpSessionId=" +
                sessionStorage.getItem("httpSessionId"))
        this.ws.onopen = function() {
          let message = {
            type : "PLAYER READY",
            matchId : response.body.id
          }
```

```
          this.send(JSON.stringify(message))
        }
        this.ws.onmessage = function(event) {

        }
      }
    })
  }

  playWR() {
    this.showingOtn = false
    this.showingWR = true
  }
}
```

Figura 602. playOtn *en* MultiplayerComponent

Hasta ahora, tenemos clara la estructura de clases del siguiente diagrama, en la que el *MultiplayerComponent* utiliza el *GamesService* para acceder por *http* al controlador, y un WebSocket nativo estandarizado (ya vimos en el Capítulo 21 que todos los navegadores dan una implementación a la clase *WebSocket)* para conectarse al *WSGames*. En la figura se resaltan los campos *ws* del *MultiplayerComponent* (declarado como *ws?*, porque no se instanciará hasta que se reciba respuesta del servidor) y del *OtnMultiplayerComponent,* que también necesita el mismo *WebSocket* para enviar y recibir movimientos del servidor.

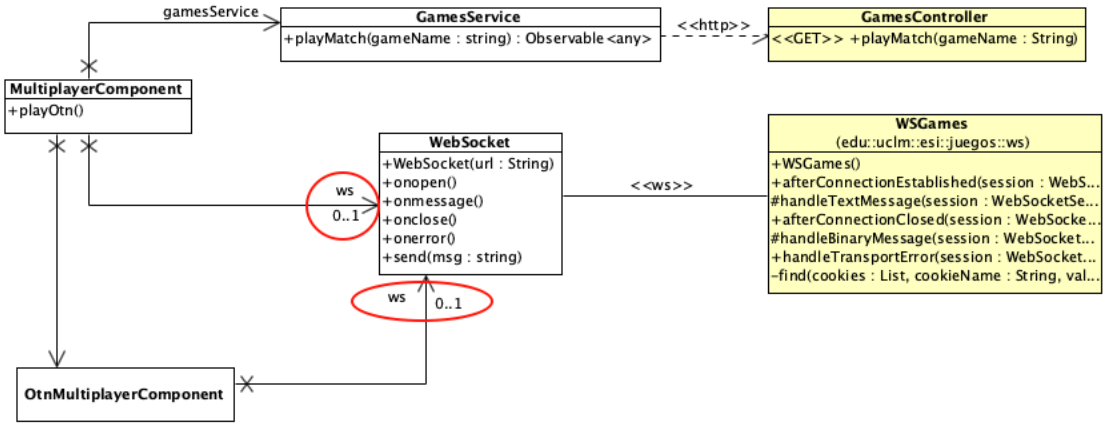


Figura 603. Parte de la estructura del front-end *y su conexión con el* back

5. Alternativas de diseño del componente del juego

De cuando desarrollamos el juego para un usuario tenemos el *OtnComponent,* que pinta el tablero del *Order the numbers* y permite que una persona juegue. Quizá podríamos aprovechar parte de la lógica y de la presentación de ese componente para implementar el *OtnMultiplayerComponent*. Así, la siguiente es una posible propuesta de diseño del *front,* en la que el *OtnMultiplayerComponent* hereda de *OtnComponent:* cuando la partida es individual, el campo *match* se instancia a *Otn;* cuando sea multijugador, se instancia a *OtnMultiplayer*.

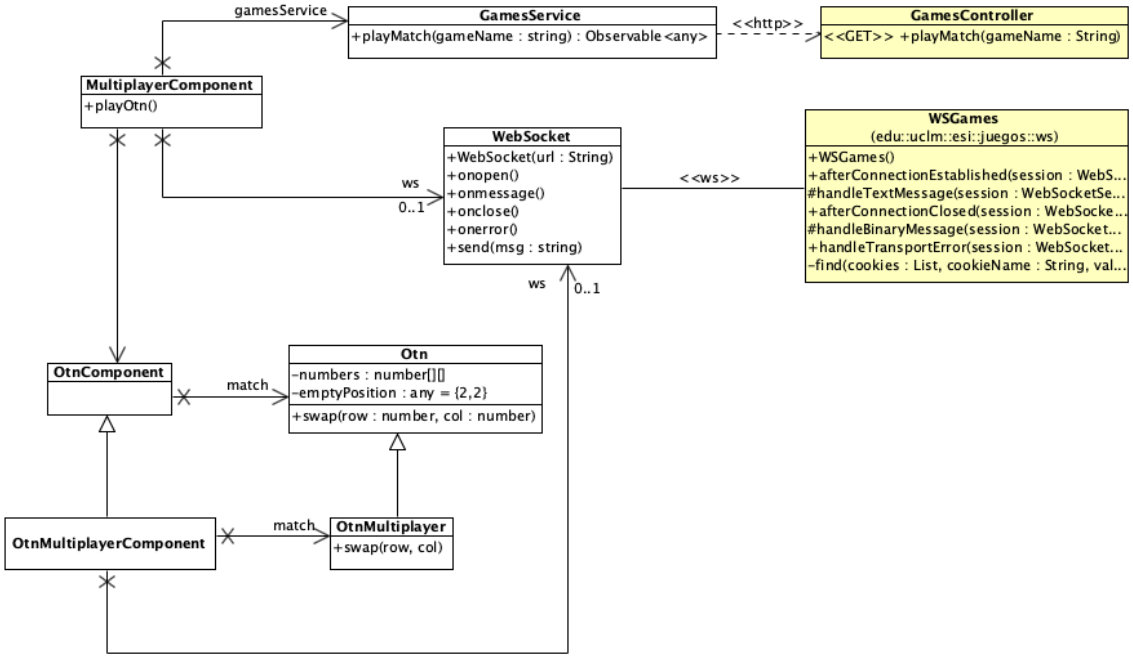


Figura 604. Alternativa 1 de diseño del front

Esta alternativa no es buena porque, cuando apliquemos herencia, debe verificarse que todo lo que tengamos en la superclase debe ser utilizable en las subclases (regla del 100%) y, además, que todas las subclases sean realmente instancias de la superclase (regla del *is a*, "es un"). Con la solución planteada, *OtnMultiplayerComponent* hereda el campo *match* de *OtnComponent* que, sin embargo, no necesita para nada. Podríamos empeñarnos y forzar esa herencia, pero seguro que, a la larga, traería muchos más problemas que beneficios.

La siguiente es otra posibilidad, en la que hay dos clases abstractas *OtnComponent* y *OtnMatch*. Al ejecutar *playOtn* en el *MultiplayerComponent*, el componente se instanciará al subtipo *OtnMultiplayerComponent* que, a su vez, trabaja con la clase *OtnMulti*.

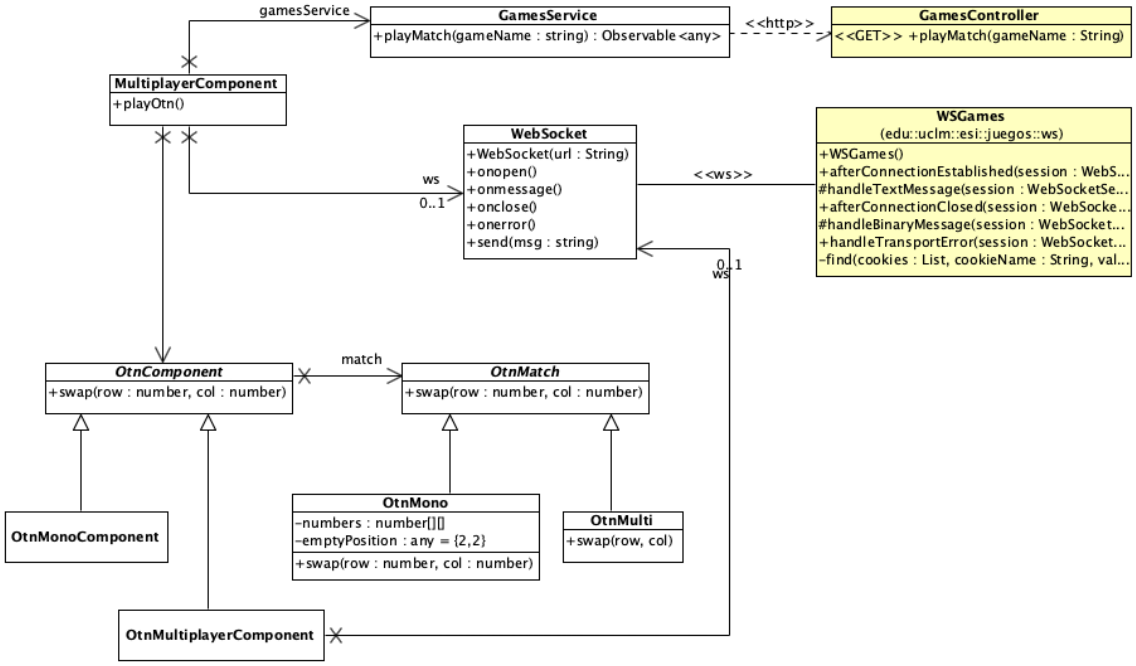


Figura 605. Alternativa 2 de diseño del front

Si el jefe nos dejara, podríamos llevarnos el acceso al *WebSocket* a la propia clase *OtnMulti* en lugar de hacerlo desde el componente:

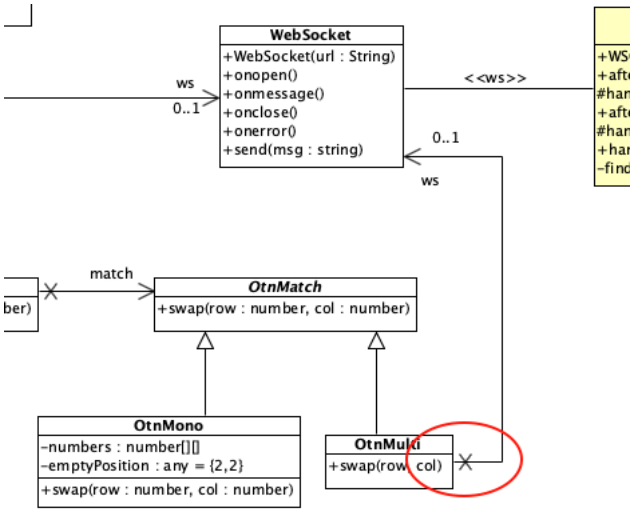


Figura 606. Alternativa 3 de diseño del front

También podríamos crear una clase *Board* para representar el array de números de cada jugador: la clase *OtnMono* tendría un solo *board,* mientras que *OtnMulti* tendría uno para el tablero del jugador local, y otro para el del oponente:

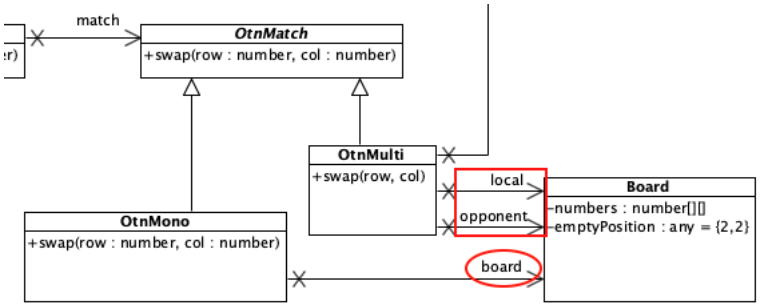


Figura 607. Alternativa 4 de diseño del front

Incluso otra posibilidad es la de la Figura 608, en la que *OtnMatch* vuelve a ser concreta, y delega la ejecución de *swap* a *OtnState,* una clase abstracta asociada a la que llevamos las operaciones cuyo comportamiento dependa del estado de la clase principal. Este es un diseño basado en el patrón *State* (Estado): cuando *OtnComponent* instancie su objeto *match,* le dirá cuál es su estado. Dependiendo de que sea para uno o dos jugadores, *state* se instanciará a *OtnMono* o a *OtnMulti.*

Este será el diseño en que nos basaremos para terminar implementar *Order the numbers* en el lado cliente. Asumiremos, además, que el director técnico nos permite acceder al *WebSocket* directamente desde *OtnMulti.*

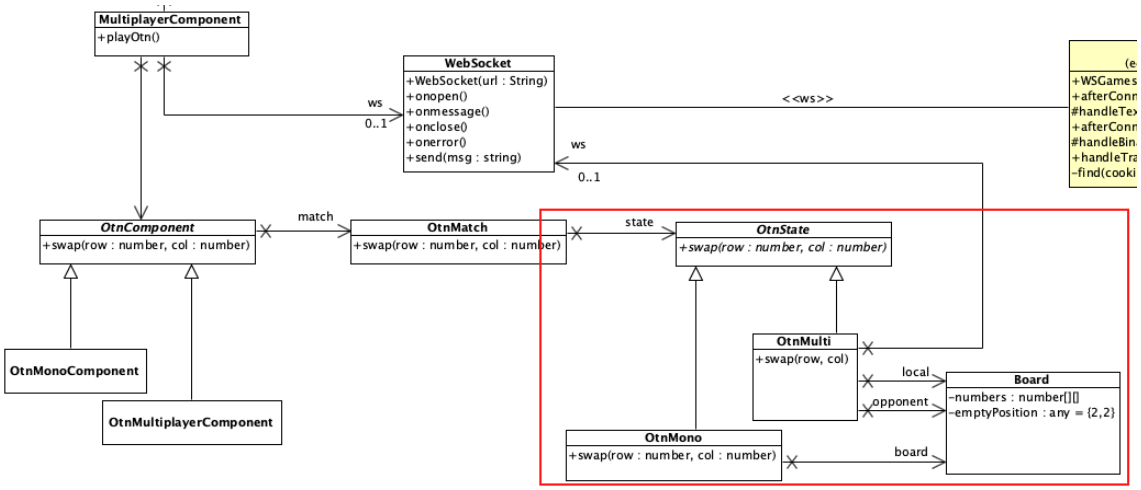


Figura 608. Alternativa 5 con el patrón State

De la figura anterior, podríamos quitar el componente abstracto *OtnComponent*, y dejar que *OtnMonoComponent* y *OtnMultiplayerComponent* conozcan a una *OtnMatch:*

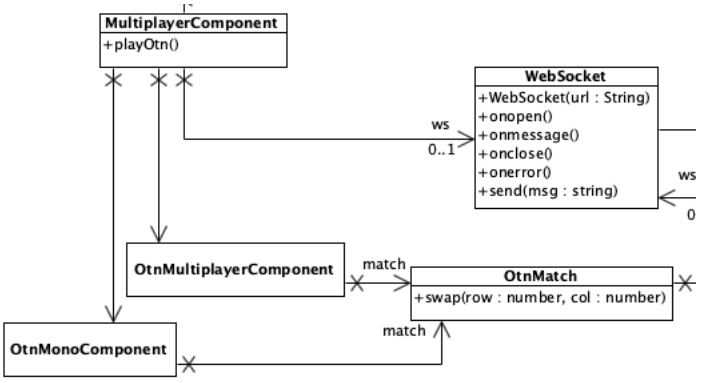


Figura 609 Alternativa 6 (se mantienen el patrón State*, oculto a la derecha)*

Con este diseño, parte de los primeros mensajes que se ejecutarán cuando *Pepe* decida jugar al *Order the numbers* en modo multijugador pueden ser los descritos en la Figura 610:

1) Primero, *AppComponent* se suscribe al evento *onMatchReceived,* que *MultiplayerComponent* emitirá tras procesar la respuesta recibida del *playMatch* del servidor.
2) Cuando dicha respuesta llega, en el mensaje 7 se crea un *OtnMulti,* el cual se conecta al servidor de *WebSocket*.
3) Luego, en el mensaje 8 se crea la partida, pasando como parámetro al constructor el estado que se acaba de crear.
4) Es entonces cuando el *MultiplayerComponent* emite el evento a *AppComponent,* el cual le pasa la pasa partida al *OtnMultiplayerComponent* en el mensaje 13. Esto lo haremos anotando un campo con *@Input* en este componente, y se explicará en la sección 6.3 (página 368).

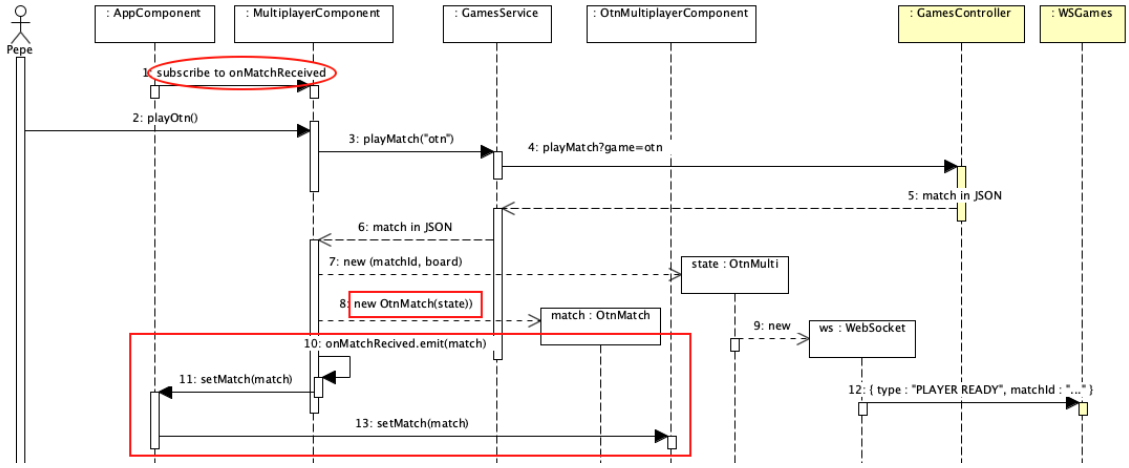


Figura 610. Solicitud de inicio de partida con la alternativa 6

6. Implementación de la alternativa 6

Los cambios de estructura y comportamiento que se han descrito suponen modificar algunas clases.

6.1. Cambios en *MultiplayerComponent*

El siguiente fragmento de código muestra la nueva implementación de esta clase. Con respecto a la Figura 602, vemos que se ha eliminado la declaración del campo *ws*, así como todo el código relacionado con el *WebSocket*, que ahora está en el estado asociado a la partida. Además, como se ve en las últimas líneas resaltadas, en este mismo método se instancian el estado al subtipo *OtnMulti OtnMulti* (mensaje 7 del diagrama de secuencia anterior) y la partida (mensaje 8), y también se comunica que se ha producido el evento *onMatchReceived*.

```
export class MultiplayerComponent implements OnInit {
  showingOtn : boolean = false
  showingWR : boolean = false
  message? : string

  @Output() onMatchReceived : EventEmitter<OtnMatch> = new EventEmitter()

  constructor(private gamesService : GamesService) { }

  ngOnInit(): void {
  }

  playOtn() {
    this.gamesService.playMatch("otn").subscribe({
      next : (response) => {
        this.showingOtn = true
        this.showingWR = false
        this.message = "Waiting for an opponent..."
        let state = new OtnMulti(response.body.id, response.body.boards)
        let match = new OtnMatch(state)
        this.onMatchReceived.emit(match)
      }
    })
  }
   ...
}
```

Figura 611. Nuevo código del MultiplayerComponent

Cuando se instancia el estado, este se construye pasando el *id* y los dos tableros. En la Figura 612 se muestra la estructura de la respuesta recibida a la petición *http:* como, en el servidor, la clase *Match* () tiene tres métodos *get* (*getId, getPlayers* y *getBoards)* y dos *is* (*isStarted* e *isReady),* Spring devuelve al cliente un solo objeto JSON con esos cuatro campos:

1) *id* es una cadena simple.
2) *players* es un array con todos los jugadores de la partida (en la figura, de momento, solo hay uno). Cada jugador es un objeto JSON que, a su vez, contiene el resultado de aplicarle a cada uno sus métodos *get* e *is*.
3) *started* y *ready* contienen en este ejemplo el valor *false* porque la partida no ha comenzado y no está lista.
4) En *boards* viene el resultado de ejecutar *getBoards* que devuelve una estructura bastante compleja (de *getBoards* a *getSquares,* de aquí a *getPieces* y de aquí a *getValue).*

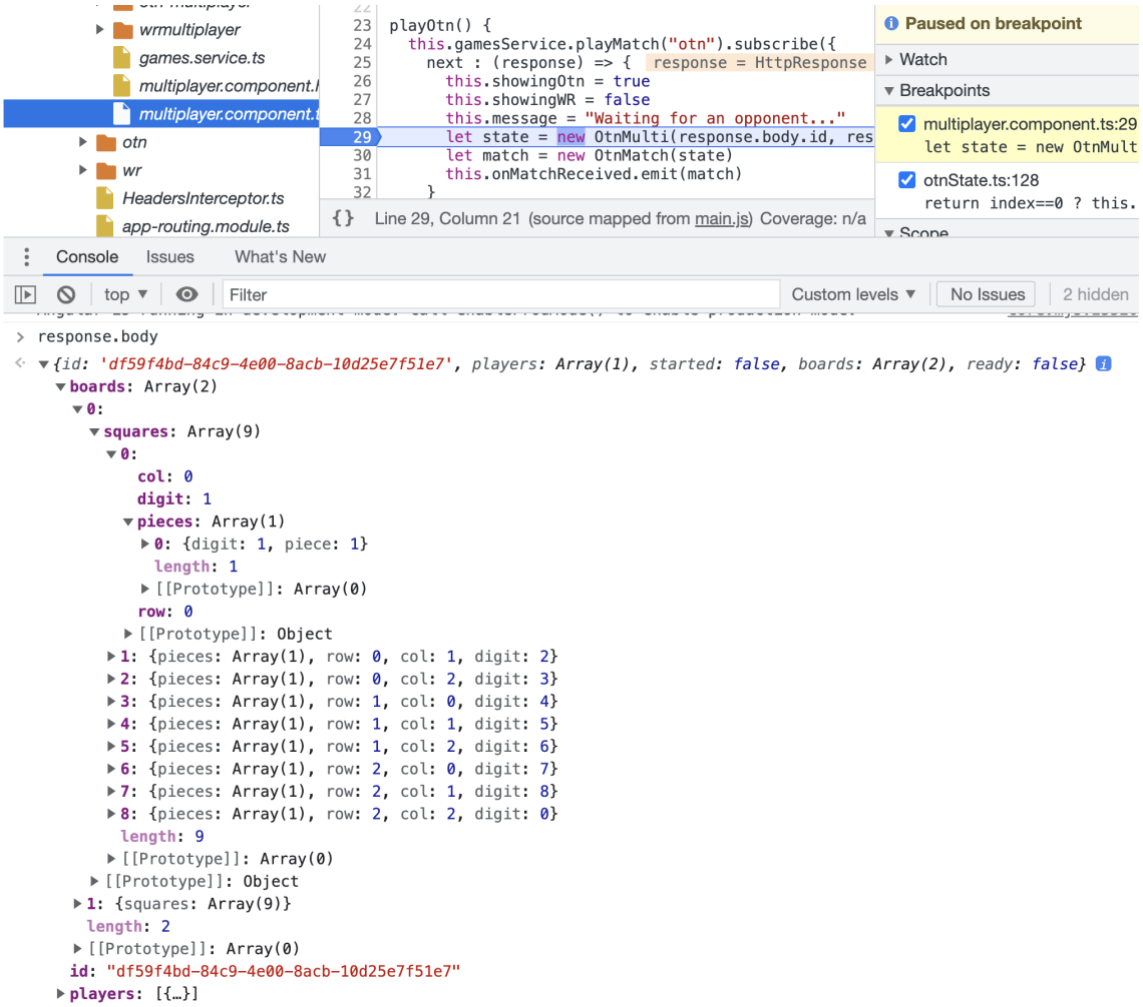


Figura 612. Contenidos de la respuesta recibida en playOtn

6.2. Recepción del evento en *App*

El elemento *app.component.html* incluía una etiqueta de tipo <*app-multiplayer*>. Ahora, *app* debe estar atento al evento *onMatchReceived* que emite el componente incrustado, por lo que lo declaramos y asignamos la operación de *AppComponent* que deba ejecutarse al recibirlo:

```
<div *ngIf="showingMultiplayer">
  Logged as {{ userName }}
  <br>
  <app-multiplayer (onMatchReceived)="setMatch($event)"></app-multiplayer>
</div>
```

Figura 613. App *está pendiente de* onMatchReceived

Ahora, en *AppComponent* debemos implementar el manejador de ese evento: como vemos en la figura anterior, se trata del método *setMatch*, que debemos añadir a la clase:

```
setMatch(match : OtnMatch) {
  this.match = match
}
```

Figura 614. Manejador de onMatchReceived *en* AppComponent

6.3. Emisión del evento al *OtnMultiplayerComponent*

En la figura anterior, se asigna a un campo *match* la partida que se nos pasa como parámetro. Igual que <*app-multiplayer*> recibe la emisión del evento *onMatchReceived,* debe también emitir la partida que se ha cargado al <*app-otn-multiplayer*>.

La Figura 615 muestra el código de *multiplayer.component.html*. En el texto resaltado, este componente envía los cambios en su campo *match* al *match* del modelo asociado a <*app-otn-multiplayer*>. Ese atributo tan sencillo (*[match]=”match”)* es en realidad el mensaje número 13 de la Figura 610.

```
<button *ngIf="!showingOtn && !showingWR" (click)="playOtn()">
    Play Order the numbers against an opponent
</button>
  
<button *ngIf="!showingOtn && !showingWR" (click)="playWR()">
    Play Word race against an opponent
</button>
<br>
<i>{{ message }}</i>

<app-otn-multiplayer *ngIf="showingOtn" [match]="match"></app-otn-multiplayer>

<app-wrmultiplayer *ngIf="showingWR"></app-wrmultiplayer>
```

Figura 615. OtnMultiplayer *está pendiente de los cambios en el campo* match

Para que *OtnMultiplayer* reciba el *match* que se le pasa, solo necesitamos declarar un parámetro *match* que hay que anotar con *@Input,* como se muestra en la Figura 616. También se muestra el método *swap,* cuya ejecución se delega a *match.swap*.

```
export class OtnMultiplayerComponent implements OnInit {

  @Input() match? : OtnMatch

  constructor() { }

  ngOnInit(): void {
  }

  swap(row: number, col: number): void {
    this.match!.swap(row, col)
  }

}
```

Figura 616. OtnMultiplayerComponent

6.4. *OtnMatch:* qué clase tan sencilla

OtnMatch recibe la solicitud de ejecutar *swap* del componente anterior y, como delega el comportamiento a su estado asociado, su implementación es prácticamente trivial:

```
import { OtnState } from "./otnState"

export class OtnMatch {
    state : OtnState

    constructor(state : OtnState) {
        this.state = state
    }

    swap(row: number, col: number) {
        this.state.swap(row, col)
    }
}
```

Figura 617. OtnMatch *delega* swap *a su* state

6.5. Implementación de los estados

Como se veía en la Figura 608, *OtnState* es una clase abstracta que incluye una sola operación, también abstracta *swap(row, col)*. Si trivial es *OtnMatch,* no digamos esta:

```
export abstract class OtnState {
     abstract swap(row: number, col: number) : void
}
```

Figura 618. La clase que implementa el estado

6.5.1. *OtnMono*

OtnMono (para un jugador) debe intercambiar las posiciones en el *board* asociado, como se hacía en la primera versión de la aplicación. Intentaremos aprovechar aquel código, que tenemos desde la Figura 90 (página 75) para la vista del componente *OtnMonoComponent,* y que reproducimos de nuevo en la Figura 619. Vemos que, ahora, en la nueva implementación que hemos dado a *OtnMatch* no hay ningún campo llamado *numbers,* con lo que tenemos un error de compilación que debemos corregir.

```
<p>otn works!</p>

<div *ngFor="let row of match.numbers; let i = index">
    <button (click)="swap(i, j)" *ngFor="let col of row; let j = index">
        <span *ngIf="col">{{ col }}</span>
        <span *ngIf="!col"> </span>
    </button>
</div>
```

Figura 619. El campo numbers *no existe en* match

Efectivamente, vemos en la Figura 620 que, para llegar tanto desde *OtnMonoComponent* como desde *OtnMultiplayerComponent* a los tableros (clase *Board),* debemos llegar a la partida, de la partida al estado (que es abstracto) y, de este, al tablero, que puede ser uno en el caso del subtipo *OtnMono* o dos en el caso de *OtnMulti.*

Por otro lado, en la vista de un jugador (*otn-mono.component.html)* debe mostrarse un único tablero, pero han de mostrarse dos en *otn-multiplayer.component.html.* Tenemos varias opciones para implementar en *OtnMatch*:

1) Crear una operación *getBoards()* que devuelva al componente todos los tableros que tenga la partida (que serán uno o dos, según). En este caso, *OtnMonoComponent* sabe que solo le viene un tablero, y lo procesará de una manera distinta a como *OtnMultiplayerComponent* procesa los dos que le llegan.
2) Crear una operación *getBoard(index : number)* que devuelva el tablero que le pasemos como parámetro: será 0 en el mono y 0 o 1 en el multi.
3) Crear dos o incluso tres operaciones *getBoard(), getLocal()* y *getOpponent()* que devuelvan los tableros apuntados por las propiedades *local, opponent* y *board* de las dos especializaciones de los estados. Obviamente, esta opción puede funcionar pero no es buena desde el punto de vista del diseño del sistema, porque *OtnMatch* estaría ofreciendo operaciones que no se utilizan en ciertas situaciones: la primera sería inútil para el modo multijugador, y las dos últimas lo serían para *OtnMonoComponent*. Sería algo parecido a las reglas del 100% y "es un" que presentábamos en la Sección 5 de este capítulo (página 362).

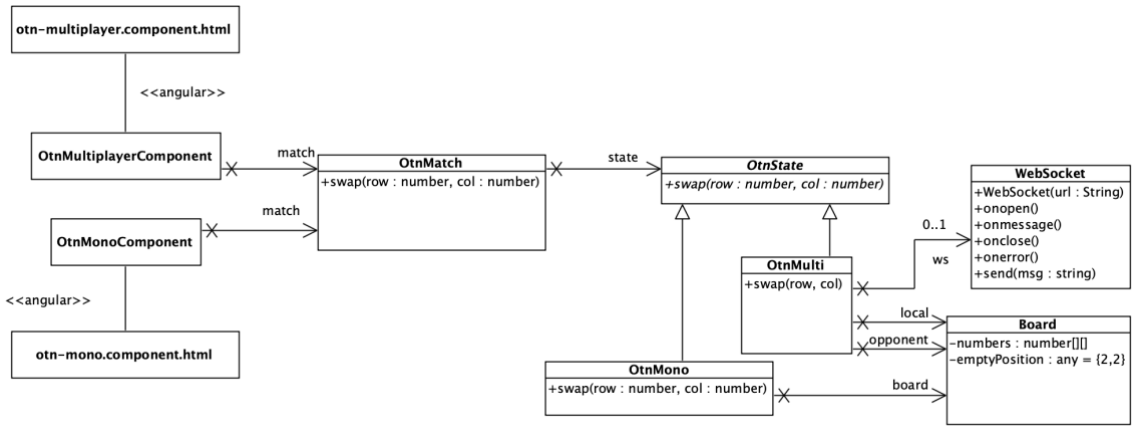


Figura 620. Relaciones de los componentes del otn *con los objetos de dominio*

Elegiremos la segunda alternativa por el simple hecho de que *getBoards(),* al estar en plural, da a entender que se devuelven varios tableros, lo cual no es cierto para el modo *mono*. El lector o lectora dirá «Bueno, pero *getBoard(index)*

da a entender que se puede recuperar uno de entre varios tableros». Pues sí, lleva usted razón.

Los cambios que hemos tenido que hacer en las diferentes clases se muestran en la siguiente figura, en la que también se ilustra el recorrido que se desencadena desde que se carga la vista *otn-mono.component.html*.

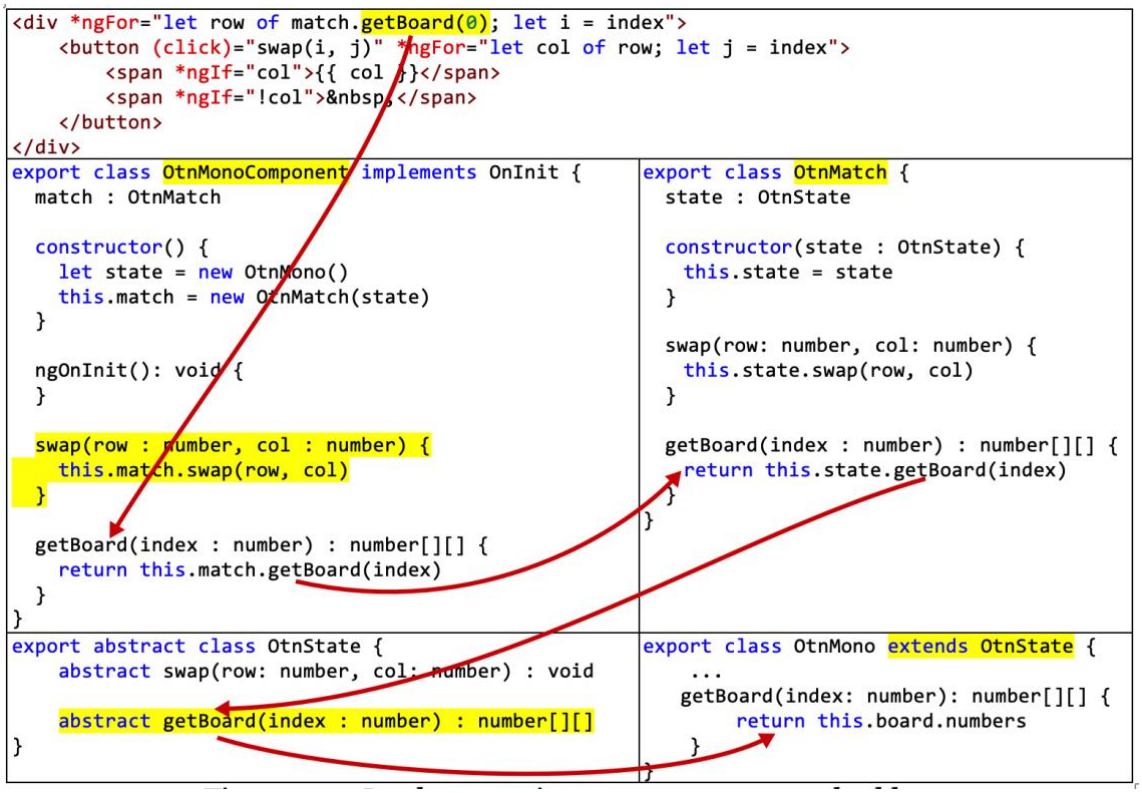


Figura 621. Implementaciones para recuperar el tablero

6.5.2. *OtnMulti* (I)

Una *OtnMatch* está en estado *OtnMulti* cuando hay dos jugadores. En este caso, la operación *swap* debe enviar el movimiento al servidor, que lo validará y, en caso de que sea correcto, actualizará los clientes de los dos jugadores. Se podría optar por hacer una validación previa del movimiento en el cliente, y evitar así el envío del mensaje al servidor cuando los dígitos que se desean intercambiar no sean adyacentes, pero supondremos que hemos recibido instrucciones de implementarlo de la primera manera.

Como sabemos, el *MultiplayerComponent* es quien instancia al *OtnMulti*. Este (Figura 622):

1) Tiene el campo *ws* de tipo WebSocket, que utilizará para enviar y recibir mensajes.
2) Guarda el *id* de la partida.
3) Tiene (como dibujábamos en la Figura 608) dos campos *local* y *opponent*, que representan los tableros del jugador local y del oponente, que se inicializan con los números que el *MultiplayerComponent* le pasó al recibir la respuesta *http* del *GamesController* (Figura 611).
4) Además, instancia el WebSocket. Cuando la conexión se abre, su manejador *onopen* envía un mensaje al servidor con el tipo PLAYER READY.
5) Para *onmessage* debemos todavía escribir el código para manejar este evento.

```
export class OtnMulti extends OtnState {
    ws : WebSocket
    matchId : string
    local : Board = new Board()
    opponent : Board = new Board()

    constructor(matchId : string, numbers : any) {
        super()
        this.matchId = matchId
        this.local.numbers = numbers
        this.opponent.numbers = numbers
        this.ws = this.buildWebSocket(matchId)
    }

    private buildWebSocket(matchId : string) : WebSocket {
        let ws = new WebSocket(wsUrl + "wsGames?httpSessionId=" +
                                    sessionStorage.getItem("httpSessionId"))
        ws.onopen = function() {
          let message = {
            type : "PLAYER READY",
            matchId : matchId
          }
          this.send(JSON.stringify(message))
        }
        ws.onmessage = function(event) {
          let info = event.data
          info = JSON.parse(info)
          let type = info.type
          if (type=="MATCH STARTED") {

          }
        }
        return ws
    }

    swap(row: number, col: number): void {
        let movement = {
            type : "MOVEMENT",
            matchId : this.matchId,
            row : row,
            col : col
        }
        this.ws!.send(JSON.stringify(movement))
    }
}
```

Figura 622. Implementación de OtnMulti *(I)*

Respecto del procesamiento de mensajes, *OtnMulti* debe permitir que el sistema se comporte como el *WSClient* que construimos en el capítulo anterior; respecto de la vista, debe conseguir que se actualicen los tableros con los cambios que se vayan produciendo.

6.5.3. *OtnMulti* (II): mensaje MATCH STARTED

Al recibir este mensaje, en la vista deben pintarse los dos tableros. Ya conocemos de la Figura 612 la estructura de la respuesta recibida del servidor, que ahora debemos procesar para que los tableros se dibujen adecuadamente.

7. Prueba manual del escenario de inicio de partida

Antes de crear casos de prueba de Selenium o de otro tipo, haremos una prueba manual del escenario: se logueará Pepe con Chrome, se logueará Ana con Firefox y ambos pulsarán en jugar al *Order the numbers en modo multijugador*.

Al tratarse de una prueba manual la hacemos con dos navegadores distintos para evitar que dos ventanas del mismo navegador puedan "pisarse" o compartir la sesión.

7.1. Un error típico

Con el docker y los servidores en 4200 y en 80 lanzados, abrimos dos navegadores y los apuntamos a *localhost:4200*. En cada uno se loguea uno de nuestros usuarios. En el lado izquierdo de la Figura 623, Pepe ya ha hecho clic en jugar a *Order the numbers* contra un oponente, y por eso se le muestra el texto *Waiting for an opponent…* A la derecha, Ana todavía no le ha dado al botón.

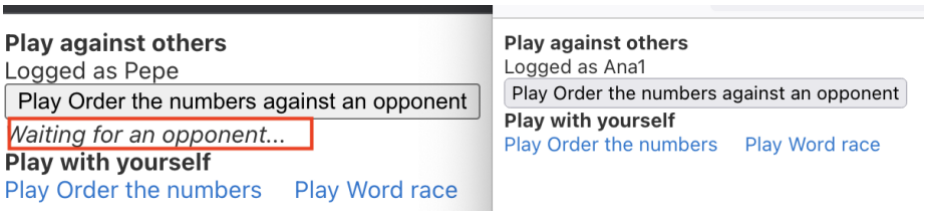


Figura 623. Inicio del escenario

Cuando Ana pulse, enviará al *GamesController* una petición para que ejecute *playMatch*, igual que ha hecho Pepe, para jugar al *Order the numbers* en modo multijugador: el controlador creará la partida y se notificará a los clientes que esta ha comenzado mediante sus sesiones de *WebSocket*; sin embargo, la respuesta aún no ha llegado al navegador de Ana, por lo que este no ha establecido todavía la conexión *ws* con el servidor: así, el envío del mensaje falla y entramos en el *catch* de la Figura 512.

El problema se ubica en la zona resaltada de la Figura 624: la partida, a la que hemos dado la responsabilidad de notificar su propio inicio, lo hace antes de que el segundo cliente haya establecido la conexión.

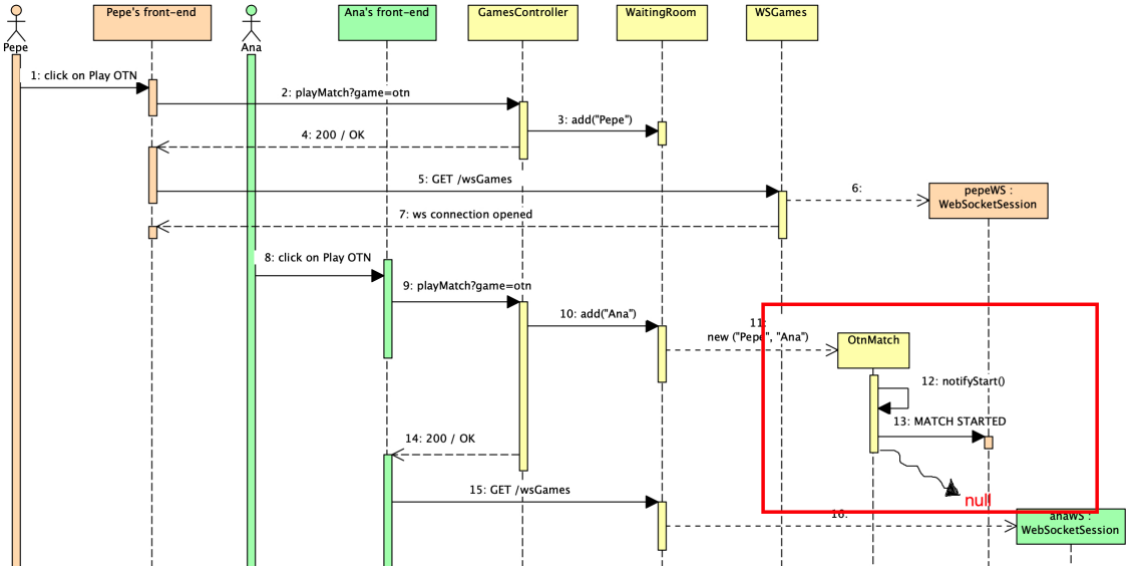


Figura 624. El mensaje MATCH STARTED *sale antes de que se haya creado la segunda conexión*

Con todo esto, resulta que la solución que dimos, y que consistía en establecer la conexión *ws* nada más recibir respuesta del *UserController,* no nos sirve, pues este envía antes el mensaje de MATCH STARTED por *ws* que el *UserController* en enviar la respuesta 200 a la petición *playMatch*.

7.2. Una solución

Una solución fácil pasa por establecer la conexión *ws* en el mismo momento en que se carga el componente *LoginComponent:* es una solución sencilla pero claramente "poco elegante", pues obligaría a nuestro *WebSocket* del servidor a mantener quizás muchas sesiones abiertas de múltiples usuarios que no se han logueado. La lectora o lector pensará que bueno, que no habrá tantos intentos incorrectos de logueo, pero no es imaginable la cantidad de ciberataques que se producen en el mismo momento en que un recurso se hace público. De hecho, con el mismo JMeter que ya conocemos podríamos echar abajo el servidor local.

La segunda solución exige un poco más de esfuerzo. En la Figura 625 se muestra cómo modificamos el primer intercambio de mensajes:

1) Pepe envía la petición http al *GamesController,* que le devuelve una respuesta 200. Al recibirla, e igual que hemos hecho antes, establecemos la comunicación con *WSGames*.
2) Aquí es en donde se introduce el cambio decisivo: cuando la conexión *ws* se establece, en el lado cliente se ejecuta el manejador *onopen*, y es ahora cuando le decimos al WebSocket del servidor que estamos listos para escucharlo, mediante ese mensaje 8 en el que le enviamos un JSON con el *type* PLAYER READY.

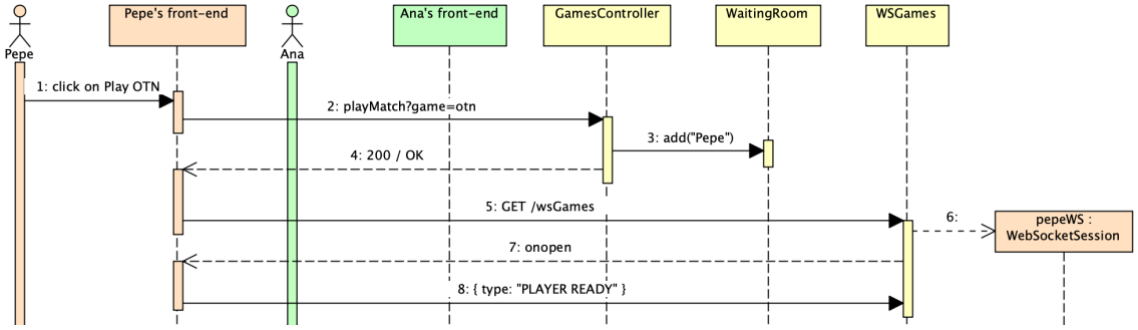


Figura 625. Modificación del comportamiento del sistema

El código correspondiente a este cambio, en el lado cliente, es el de la siguiente figura: cuando hemos recibido la respuesta afirmativa al establecimiento de la conexión *ws* es cuando le decimos al servidor: «Oye, estoy listo para me digas cosas».

```
playOtn() {
  this.gamesService.playMatch("otn").subscribe({
    next : (response) => {
      if (response.body) {
        this.showingOtn = true
        this.showingWR = false
      } else {
        this.message = "Waiting for an opponent..."
      }
      this.ws = new WebSocket(wsUrl + "wsGames")
      this.ws.onopen = function() {
        let message = { type : "PLAYER READY"}
        this.send(JSON.stringify(message))
      }
      this.ws.onmessage = function(event) {
        let info = event.data
        info = JSON.parse(info)
        alert(info.type)
      }
    }
  })
}
```

Figura 626. Cambio en el código del cliente

En el servidor debemos hacer, de momento, dos cambios: uno para que no se notifique de forma inmediata a los clientes que la partida está lista, y otro en *WSGames* para procesar los mensajes que nos lleguen con el tipo PLAYER READY.

1) Para el primero, basta con eliminar la llamada a *notifyStart*.
2) Para el segundo, tenemos que capturar y procesar los mensajes que lleguen a *handleTextMessage*. En el siguiente fragmento de código, recuperamos la *payload* que se ha recibido del cliente. Se trata de una cadena en formato JSON que se ha elaborado en la figura anterior: allí la serializábamos (es decir: convertíamos el objeto a cadena) y, ahora, la deserializamos (recibimos la cadena y la convertimos a JSON). Luego, recuperamos la *hwSession* (nuestro *wrapper)* asociada a la sesión de WebSocket que se nos ha inyectado y, bueno, debemos completar el código para procesarla.

```
@Override
protected void handleTextMessage(WebSocketSession session, TextMessage message) throws Exception {
     JSONObject jso = new JSONObject(message.getPayload());
     String type = jso.getString("type");
     HWSession hwSession = Manager.get().getSessionByWebSocketId(session.getId());

     if (type.equals("PLAYER READY")) {
       System.out.println("Player " + hwSession.getUserId() + " ready");
     }
}
```

Figura 627. Uno de los cambios en el lado servidor

Con el código de la Figura 627 podemos saber el *id* del usuario que ya está preparado para que le enviemos mensajes. De algún modo, tenemos que localizar su partida y, si esta ya dispone del número de jugadores necesarios para iniciarse, entonces podremos enviar el mensaje de que la partida está lista.

Aunque los requisitos no hablan nada de que un jugador pueda jugar varias partidas simultáneamente, nos curamos en salud y, en la Figura 628, añadimos a nuestra *wrapper* una colección con todas las posibles partidas que esté jugando cada usuario, indexada por el *id* de cada una.

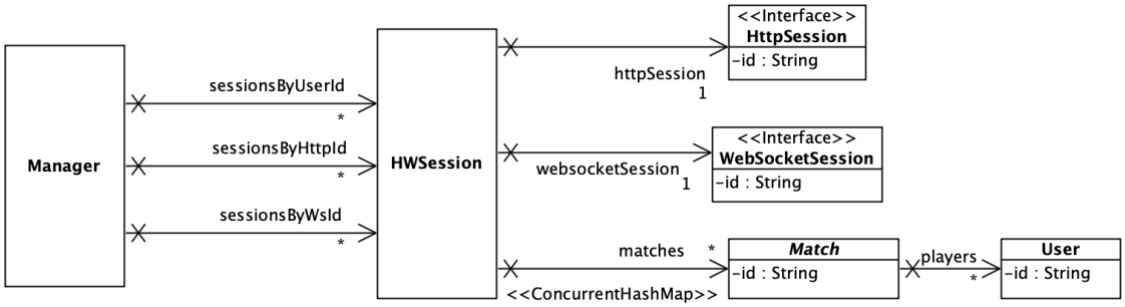


Figura 628. El Manager *mantiene una colección* matches *con las partidas*

La clase *Match* no tenía *id*, pero le asignaremos un valor aleatorio en su constructor, igual que hicimos con la entidad *User* (Figura 340, página 222). Ahora, cuando la instancia de *Match* se crea, la asignamos a la *HwSession* de cada jugador.

```
public static Match newMatch(String gameName, User... users) {
   Match match = null;
   ...

   try {
      ...
      match = clazz.newInstance();
      for (User user : users) {
         match.players.add(user);
         HWSession hwSession = Manager.get().getSessionByUserId(user.getId());
         hwSession.addMatch(match);
      }
   }
   ...
   return match;
}
```

Figura 629. La partida se añade a la lista de partidas de cada jugador

El flujo de mensajes que está sucediendo ahora es (Figura 630) que Pepe elige el *otn* y que Ana también. Cuando llega esta segunda petición (mensajes 7 y 8), la sala de espera crea la partida para estos jugadores y asigna, a la *HWSession* de cada uno, la instancia de *Match*.

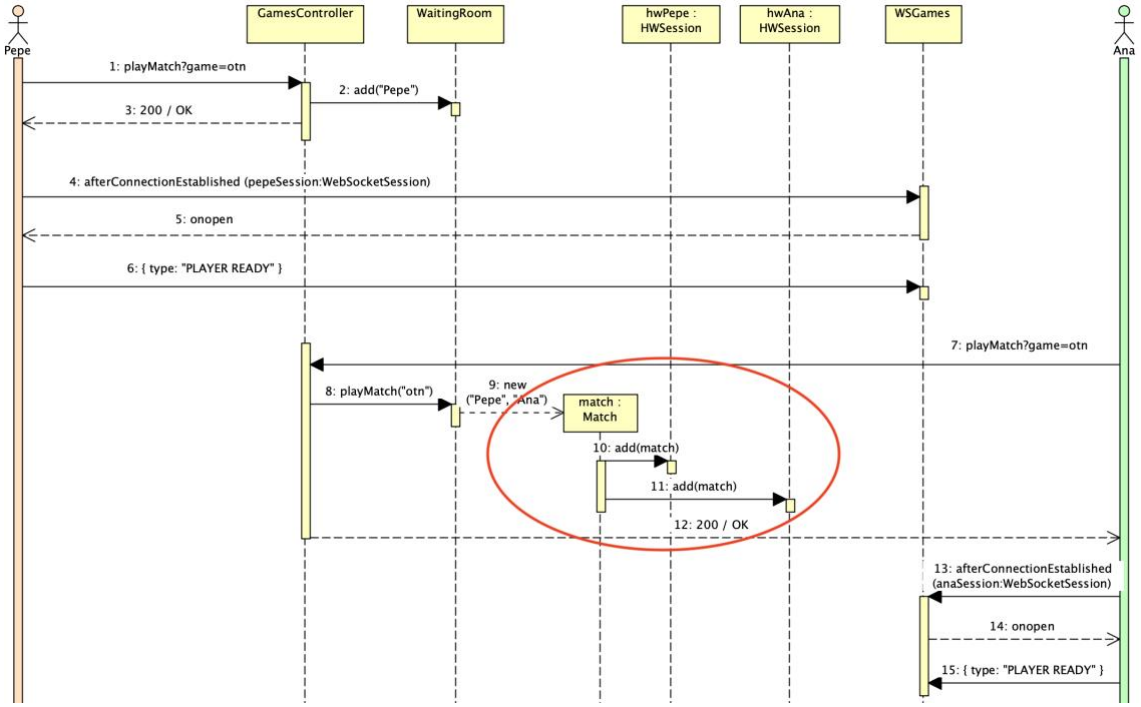


Figura 630. Nuevo flujo de mensajes

Fijémonos ahora, en la figura anterior, en los mensajes 6 y 15, que se refieren al anuncio que los clientes le hacen a *WSGames* indicándoles que ya están listos. Cuando llega el mensaje de Pepe (el 6) no hay todavía partida creada; pero, cuando llega el de Ana (mensaje 15), esta sí que esta instanciada y tiene a ambos jugadores asignados.

Bien, pues para que podamos anunciar a Pepe y a Ana que la partida está lista y que pueden empezar a jugar, procederemos como sigue:

1) Cuando llegue un mensaje PLAYER READY, recuperaremos la *HWSession* correspondiente al *id* de la WebSocketSession.
2) Si esta *HWSession* no tiene partida asignada (sería el caso del mensaje 6), no hacemos nada.
3) Pero si sí tiene instancia de *Match*, recuperamos las *WebSocketSession* asociadas a los jugadores y enviamos el mensaje de tipo MATCH READY. La Figura 631 ilustra de manera muy resumida el envío de este mensaje a los dos clientes.

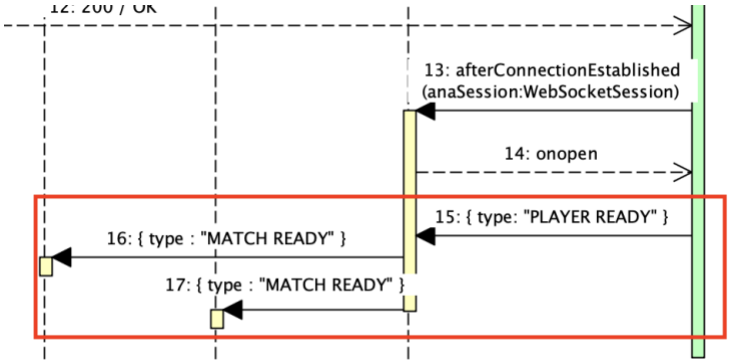


Figura 631. Se anuncia que la partida está preparada

En *WSGames* debemos completar el método *handleTextMessage* para que exhiba ese comportamiento: en la siguiente figura, buscamos una partida no

comenzada (con *getPendingMatch)* asociada al jugador que ha enviado este mensaje.

```
@Override
protected void handleTextMessage(WebSocketSession session, TextMessage message)
          throws Exception {
     JSONObject jso = new JSONObject(message.getPayload());
     String type = jso.getString("type");
     HWSession hwSession = Manager.get().getSessionByWebSocketId(session.getId());

     if (type.equals("PLAYER READY")) {
       Match match = hwSession.getPendingMatch();
       if (match!=null) {
            match.setStarted(true);
            match.notifyStart();
       }
     }
}
```

Figura 632. Modificamos handleTextMessage

Si, en el método anterior, se encuentra tal partida, se marca como iniciada y se ejecuta su operación *notifyStart,* que ya teníamos implementada, y que modificamos ligeramente para que el cliente reciba el *id* de la partida.

```
public void notifyStart() {
     for (User player : players) {
       ...

       JSONObject jso = new JSONObject().
            put("type", "MATCH STARTED").
            put("matchId", this.id);
       ...
     }
}
```

Figura 633. Enviamos al cliente el id *de la partida*

7.3. Reejecución del escenario

Si ejecutamos de nuevo el escenario con el que empezamos esta sección, veremos que en ambos clientes se muestra el *alert* con el texto MATCH STARTED, con lo que parece que vamos bien encaminados.

SEXTA PARTE: INTEGRACIÓN DE SERVICIOS DE TERCEROS

1. Nuevos casos de uso

En la Figura 634 se muestran nuevos casos de uso relacionados con la gestión de cuentas de usuario, para aproximar un poco más la vista funcional del sistema a la realidad deseada.

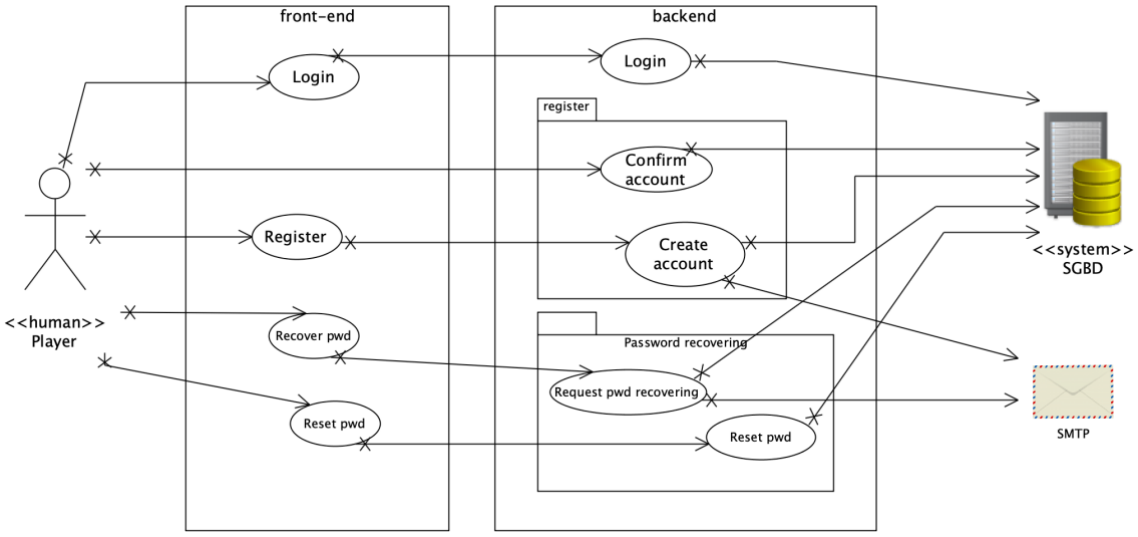


Figura 634. Casos de uso sobre la gestión de cuentas de usuario

Por ejemplo, mientras que, hasta ahora, hemos aceptado que el caso de uso *register* cree directamente la cuenta del usuario en el *backend*, en la realidad se hará en los dos pasos que se ilustran en la Figura 635:

1) En el *front*, el usuario ejecuta su funcionalidad *register* (mensaje 1), que envía el mensaje 2 a *Create account* en el servidor. Este caso de uso crea la cuenta en el mensaje 3 y envía en el mensaje 4 un correo electrónico al usuario. La cuenta queda pendiente de activación.
2) En el mensaje 5, el usuario hace clic en un enlace que le ha debido de llegar en el correo electrónico anterior. Esto supone que la cuenta se active mediante la ejecución directa del caso de uso *Confirm account* del lado servidor, que actúa sobre la base de datos con el mensaje 6.

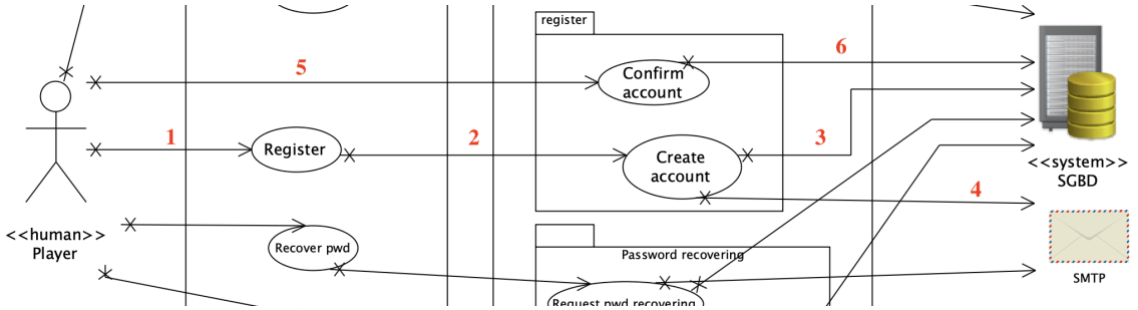


Figura 635. Zoom sobre las funcionalidades de registro del usuario

Respecto de todo lo que hemos desarrollado hasta ahora, la principal diferencia está en el uso del actor que hemos llamado *smtp* (iniciales de *Simple Mail Transfer Protocol*).

2. Suscripción a un servidor de correo electrónico

Para enviar correos electrónicos desde nuestro sistema utilizaremos la API que ofrece el sitio *sendinblue.com*. Necesitamos disponer de una cuenta de correo electrónico real que será la que aparezca como remitente (en nuestro caso hemos creado *libroadpweb@gmail.com*) y, con ella, crear una cuenta en *sendinblue*.

Se nos proporcionan claves para acceder desde nuestro sistema a los servicios ofrecidos, tanto por *smtp* como por *http* (Figura 636).

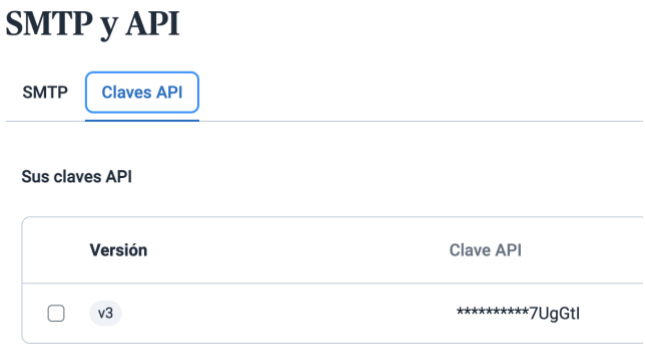


Figura 636. Claves para SMTP y para la API http

Utilizaremos la API *http*. Aunque para nosotros *sendinblue* es un actor externo, podemos suponer que, como se ilustra en la Figura 637, este sistema transformará las peticiones que le enviemos por *http* a través de su API a mensajes *smtp*, que serán los que envíe a los servidores de correo electrónico de los destinatarios.

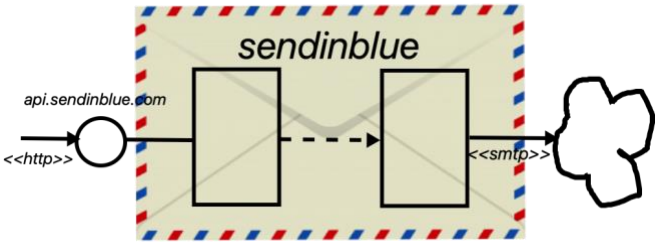


Figura 637. Enviamos peticiones http *y salen mensajes con protocolo* smtp

En la Figura 636 vemos que se nos ha generado una "Clave API", y también se nos da un ejemplo de comando CURL para probar el servicio, que podemos adaptar y dejarlo como en la Figura 638. De acuerdo con las instrucciones que se dan, es importante añadir esas tres cabeceras (una de ellas es la "Clave API") y escribir en el campo *email* del *sender* la dirección que utilizamos para crear la cuenta en *sendinblue*.

```
curl --request POST \
  --url https://api.sendinblue.com/v3/smtp/email \
  --header 'accept: application/json' \
  --header 'api-key:xkeysib-...7UgGtI' \
  --header 'content-type: application/json' \
  --data '{
   "sender":{
      "name":"Libro",
      "email":"libroadpweb@gmail.com"
   },
   "to":[
      {
         "email":"macario.polo@uclm.es",
         "name":"Macario Polo Usaola"
      }
   ],
   "subject":"Prueba de envío",
   "htmlContent":"<html><head></head><body><p>Hola.</p></body></html>"
}'
```

Figura 638. Petición http *para enviar un correo*

Si ejecutamos el comando desde una terminal, *sendinblue* nos devuelve un JSON con el *id* del mensaje que se ha enviado. Bien, pues ya estamos listos para integrar este servicio en nuestro sistema.

3. Integración del servicio en nuestro *backend*

El registro efectivo de un usuario se realiza, según se ha explicado, en dos fases (Figura 639):

1) En la primera, se crea la cuenta y se termina enviando un email al usuario con un enlace. Como se ve, además de guardar al usuario en la base de datos, se generará un token aleatorio, que será el que se envíe en el cuerpo del correo.
2) En la segunda, el usuario hace clic sobre el correo recibido, lo que lleva a que se confirme la creación de la cuenta y esta pase a estar activa.

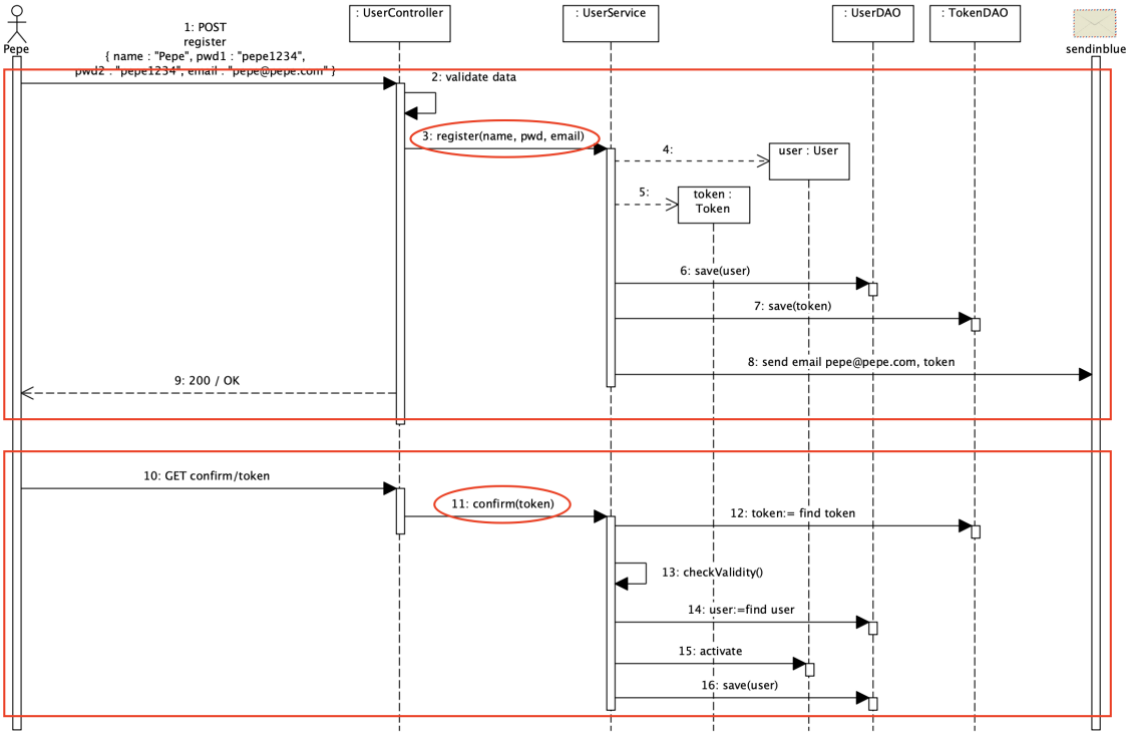


Figura 639. Las dos fases en la creación de una cuenta

Como se ve en el diagrama de secuencia anterior, se requieren dos recursos web en el *UserController: register*, que ya existe (aunque se deberá cambiar su comportamiento) y *confirm,* que recibe el token como parámetro y activa la cuenta.

3.1. Fase 1: creación de la cuenta, generación del token y envío del correo

Para implementar la nueva forma de crear una cuenta haremos varios cambios (Figura 640):

1) Añadiremos una columna *validationDate* a la tabla *User* para guardar la fecha en que el usuario ha confirmado la cuenta. Podría ser de tipo booleano, pero almacenar la fecha nos da un poquito más de información: será *null* si no la validado, y otro valor en caso de que sí.
2) Crearemos una tabla *Token* para almacenar los tokens que se generan al crear las cuentas del usuario. Cada token tendrá un *id* aleatorio y guardaremos su fecha de creación.
3) Además de guardar el usuario en la base de datos, el método *register* del *UserService* almacenará también el token y enviará el correo al usuario.
4) Nótese también que anotamos el método *register* con *@Transactional* (ojo, porque al ir a importar la anotación, Eclipse nos da dos opciones, y debemos elegir la de *org.springframework)*. Con esto conseguimos que las operaciones de persistencia se ejecuten en una transacción: es decir que, si por algún motivo, fallase la sentencia de guardar el token, tampoco se guardaría el usuario. *@Transactional* puede anotar a un método o a una clase: en este caso, todos los métodos de la clase se ejecutarían transaccionalmente.

```java
public class User {
    @Id @Column(length = 36)
    private String id;
    ...
    private Long validationDate;
    ...
```

```java
@Entity
public class Token {
    @Id
    private String id;
    private long time;
    @ManyToOne
    private User user;
    public Token() {
        this.id = UUID.randomUUID().toString();
        this.time = System.currentTimeMillis();
    }
    ...
```

```java
@Transactional
public void register(String name, String pwd, String email) {
    User user = new User();
    user.setName(name);
    user.setPwd(pwd);
    user.setEmail(email);

    Token token = new Token();
    token.setUser(user);
    this.userDAO.save(user);
    this.tokenDAO.save(token);

    this.sendEmail(user, token);
}
```

Figura 640. Cambios para la creación de una cuenta

Por otro lado, necesitamos una clase que envíe la petición POST al servidor de *sendinblue* con la estructura de la Figura 638. Para esto, e igual que ya nos creamos un cliente de WebSocket que trabaja con protocolo *ws,* ahora necesitamos un cliente que trabaje con *http*.

La clase *HttpClient* que se muestra en la Figura 641 incluye un método *sendPost* que envía, a la *url* que se pasa como primer parámetro, el objeto JSON que se pasa en tercer lugar, con las cabeceras del segundo. Utiliza el paquete *org.apache.http,* por lo que debemos añadirlo al *pom.xml*.

```xml
<dependency>
    <groupId>org.wso2.apache.httpcomponents</groupId>
    <artifactId>httpclient</artifactId>
    <version>4.3.1.wso2v1</version>
</dependency>
```

```java
package edu.uclm.esi.juegos.auxi;

import java.io.IOException;

import org.apache.http.HttpEntity;
import org.apache.http.client.methods.CloseableHttpResponse;
import org.apache.http.client.methods.HttpPost;
import org.apache.http.entity.StringEntity;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClientBuilder;
import org.apache.http.util.EntityUtils;
import org.json.JSONArray;
import org.json.JSONObject;
import org.springframework.http.HttpStatus;
import org.springframework.web.server.ResponseStatusException;

public class HttpClient {

   public String sendPost(String url, JSONArray headers, JSONObject payload) {
      try(CloseableHttpClient client = HttpClientBuilder.create().build()) {
         HttpPost post = new HttpPost(url);
         try {
            HttpEntity entity = new StringEntity(payload.toString());
            post.setEntity(entity);
            for (int i=0; i<headers.length(); i++) {
               String header = headers.getString(i);
               String headerName = header.substring(0, header.indexOf(':')).trim();
               String headerValue = header.substring(header.indexOf(':')+1).trim();
               post.setHeader(headerName, headerValue);
            }

            CloseableHttpResponse response = client.execute(post);
            entity = response.getEntity();
            String responseText = EntityUtils.toString(entity);
```

```
            return responseText;
         } catch (Exception e) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e.getMessage());
         }
      } catch (IOException e1) {
         throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e1.getMessage());
      }
   }
}
```

Figura 641. Un cliente http

Para que el método anterior funcione con nuestro proveedor del servicio de correo electrónico, los valores de los tres parámetros han de ser los adecuados, conforme al ejemplo de petición de la Figura 638. Una solución pasa por "hardcodear" (es decir, escribir directamente en el código fuente) los valores de los parámetros (la clave de la API, la dirección del remitente o el texto del mensaje), pero esto es una mala praxis evidente. Por ello, crearemos un par de ficheros de texto que guardaremos en *src/main/resources:* en uno tendremos el texto del correo (Figura 642, arriba); el otro almacena un JSON con los parámetros de acceso al servicio (Figura 642, abajo).

```
<html>
<body>
<p>
Gracias por registrarte en nuestro sistema de juegos.
</p>
<p>
Por, favor, haz clic <a
href="http://localhost/users/confirm/#TOKEN#">aquí</a> para
confirmar la creación de tu cuenta y poder acceder a todos nuestros
servicios.
</p>
</body>
</html>
```

```
{
   "endpoint" : "https://api.sendinblue.com/v3/smtp/email",
   "headers" : [
      "accept : application/json",
      "api-key : xkeysib-...7UgGtI",
      "content-type : application/json"
   ],
   "sender" : {
      "name" : "Libro",
      "email" : "libroadpweb@gmail.com"
   },
   "subject" : "Confirmación de cuenta"
}
```

Figura 642. Ficheros welcome.html.txt *y* sendinblue.parameters.txt, *ambos en* src/main/resources

Cuando se han guardado en la base de datos el usuario y el token se invoca (Figura 640) a un método *sendEmail(user, token)* que procesa los dos ficheros anteriores y envía el correo a través del *HttpClient:*

1) Se lee el contenido de *welcome.html.txt* y se reemplaza #TOKEN# por el *id* del token.
2) Se lee el contenido de *sendinblue.parameters.txt* en forma de JSON.
3) Con la información de ambos ficheros, se construye una petición como la que espera el servidor, que se envía en la última línea.

```
private void sendEmail(User user, Token token) throws IOException {
   String body = Manager.get().readFile("welcome.html.txt");
   body = body.replace("#TOKEN#", token.getId());

   JSONObject emailParameters =
      new JSONObject(Manager.get().readFile("sendinblue.parameters.txt"));
   String endPoint = emailParameters.getString("endpoint");
   JSONArray headers = emailParameters.getJSONArray("headers");

   JSONObject payload = new JSONObject();
   payload.put("sender", emailParameters.getJSONObject("sender"));
   JSONArray to = new JSONArray();
   to.put(new JSONObject().put("email", user.getEmail()).put("name", user.getName()));
   payload.put("to", to);
   payload.put("subject", emailParameters.getString("subject"));
   payload.put("htmlContent", body);

   HttpClient client = new HttpClient();
   client.sendPost(endPoint, headers, payload);
}
```

Figura 643. Procesamiento de los ficheros para el envío del correo

3.2. Fase 2: confirmación de la cuenta

La url sobre la que el usuario, al recibir el correo de creación de cuenta, debe hacer clic, es *http://localhost/users/confirm/#TOKEN#,* donde el #TOKEN# es el *id* del token que se le ha creado.

Se trata entonces de una petición de tipo GET que se recibe en el *UserController* (Figura 644, arriba) y cuyo comportamiento se delega al *UserService* (Figura 644, abajo). Si el token es válido (existe y se generó hace menos de 24 horas), entonces actualizamos la *valdiationDate* del usuario asociado con la fecha actual y eliminamos el token de la base de datos.

```
@GetMapping("/confirm/{tokenId}")
public void confirm(@PathVariable String tokenId) {
   try {
      this.userService.confirm(tokenId);
   } catch (Exception e) {
      throw new ResponseStatusException(HttpStatus.CONFLICT, e.getMessage());
   }
}
```

```
public void confirm(String tokenId) {
   Optional<Token> optToken = this.tokenDAO.findById(tokenId);
   if (!optToken.isPresent())
      throw new ResponseStatusException(HttpStatus.NOT_FOUND,
            "No se encuentra el token o ha caducado");
   Token token = optToken.get();
   long time = System.currentTimeMillis();
   if (time-token.getTime()>24*60*60*1000)
      throw new ResponseStatusException(HttpStatus.NOT_FOUND,
            "No se encuentra el token o ha caducado");
   User user = token.getUser();
   user.setValidationDate(time);
   this.userDAO.save(user);
   this.tokenDAO.delete(token);
}
```

Figura 644. confirm *en* UserController *y en* UserService

Claramente, debemos impedir el login si la *validationDate* es *null:*

```
public String login(HttpSession session, HttpServletResponse response,
      @RequestBody Map<String, Object> info) {
   ...

   User user = this.userService.login(name, pwd);
   if (user==null || user.getValidationDate()==null)
      throw new ResponseStatusException(HttpStatus.FORBIDDEN,
            "Credenciales inválidas");
   ...
}
```

Figura 645. Una nueva comprobación en el login *del* UserController

4. Envío de un redireccionamiento

Cuando el usuario recién registrado hace clic en el enlace del correo electrónico que ha recibido, se le abre un navegador que envía la petición al recurso *confirm* del servidor. Este confirma la creación de la cuenta, pero no el navegador se le queda en blanco (Figura 646, izquierda). Si el token estuviera caducado o no fuera válido, se le mostraría una fea página de error (Figura 646, derecha).

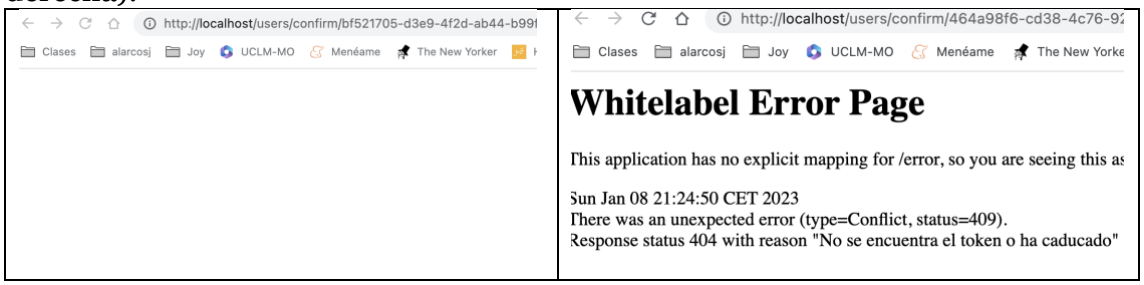


Figura 646. Posibles respuestas al confirmar un token

Lo ideal es que, en el primer caso, se redirija el navegador a una página en la que el usuario pueda loguearse, y a una en la que se le explique el error en el segundo caso.

Para la confirmación de cuenta satisfactoria, enviaremos al cliente una respuesta de tipo 304, que redirige el navegador a la url que le proporcionemos. En la Figura 647, declaramos la *response* como un parámetro del servicio para que Spring nos la inyecte; luego, ejecutamos su método *sendRedirect* para que el navegador sepa que debe dirigirse a *localhost:4200*.

```
@GetMapping("/confirm/{tokenId}")
public void confirm(HttpServletResponse response, @PathVariable String tokenId) {
   try {
      this.userService.confirm(tokenId);
      response.sendRedirect("http://localhost:4200/");
   } catch (Exception e) {
      throw new ResponseStatusException(HttpStatus.CONFLICT, e.getMessage());
   }
}
```

Figura 647. Envío de una respuesta con redireccionamiento

Evidentemente, en lugar de "hardcodear" la url, lo suyo sería tenerla en algún fichero de configuración, como hicimos con *sendinblue.parameters.txt* (Figura 642, página 386).

Capítulo 25. Integración de servicios de pago

1. Descripción del escenario

En el capítulo anterior, *sendinblue* nos proporcionaba una "Clave API" que, junto a la dirección del remitente, era suficiente para identificar a nuestro *backend* y poder utilizar el servicio ofrecido.

Cuando se utiliza un servicio de pago externo es necesario disponer de dos claves que identifican de forma única al sistema que utiliza el servicio:

1) Antes de ejecutar el pago propiamente dicho, nuestro servidor solicita mediante la primera clave (se la denomina "clave secreta") una preautorización del gasto que se va a hacer, a la que el proveedor de pagos responde con un token que identifica la transacción.
2) Cuando nuestro servidor recibe ese token, se lo envía de vuelta al *front-end*. Este lo envía junto a la segunda clave ("clave publicable") directamente al servicio de pagos. Con la clave publicable, el proveedor identifica al *backend* que inició la transacción, y con el token se identifica la propia transacción.

El flujo de mensajes se ilustra en la Figura 648 y se describe con detalle en los siguientes párrafos:

1) Primero, Pepe llega en algún momento a una página (*payment.html,* por ejemplo) dentro de su cliente en la que se envía a un controlador de pagos nuestro (*PaymentsController)* una solicitud para realizar un cargo de 50 €.
2) Nuestro *PaymentsController* se identifica ante el servidor externo (usaremos *Stripe* en este caso) mediante nuestra clave secreta ("12345") y le solicita un código de preautorización para ese importe.
3) *Stripe* nos devuelve un token ("asdfg") que nosotros le entregamos al *front-end* como respuesta a su solicitud *prepay*.
4) En una segunda fase, el *front* del usuario carga el formulario para escribir los datos del medio de pago (por ejemplo, la tarjeta de débito o crédito).
5) Cuando el usuario pulsa el botón de pagar (mensaje 3), el cliente envía al proveedor de pagos la clave pública ("xyzt"), el token que se nos devolvió ("asdfg"), el importe y los datos del medio de pago.

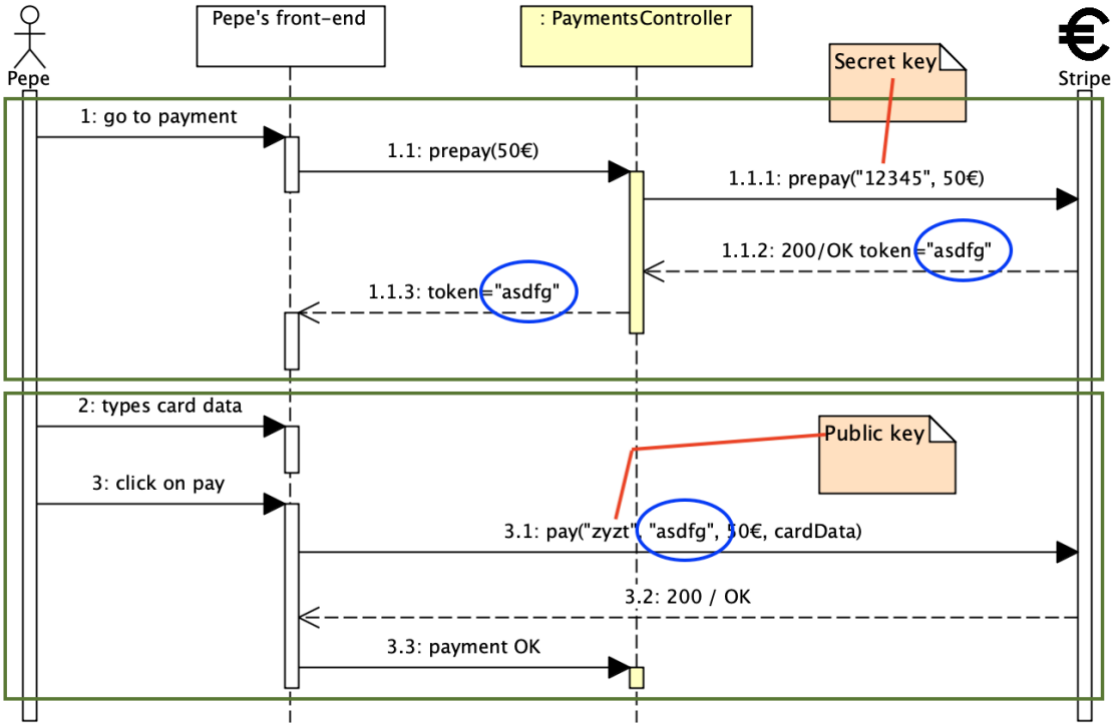


Figura 648. Flujo que sucede al realizar un pago

2. Aplicación a las partidas multijugador

Supondremos que, para las partidas multijugador, es necesario comprar un bono: hay bonos de 10 partidas por 10 euros, y de 20 partidas por 15 euros. Como hemos dicho, usaremos *Stripe* (https://stripe.com) como proveedor de pagos, que ofrece una interfaz bastante sencilla para empezar a funcionar.

2.1. Creación de una cuenta en Stripe

En primer lugar, vamos a Stripe y creamos una cuenta:

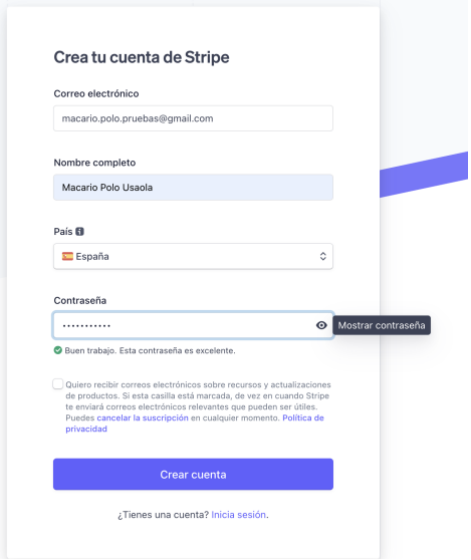


Figura 649. Creación de una cuenta en Stripe

Tendremos que confirmar la creación con un link que, al igual que hemos hecho nosotros en el capítulo anterior, nos llegará al correo electrónico que hayamos elegido.

Una vez creada y confirmada la cuenta, iremos al apartado *Desarrolladores* y, dentro de este, a *Claves de API:*

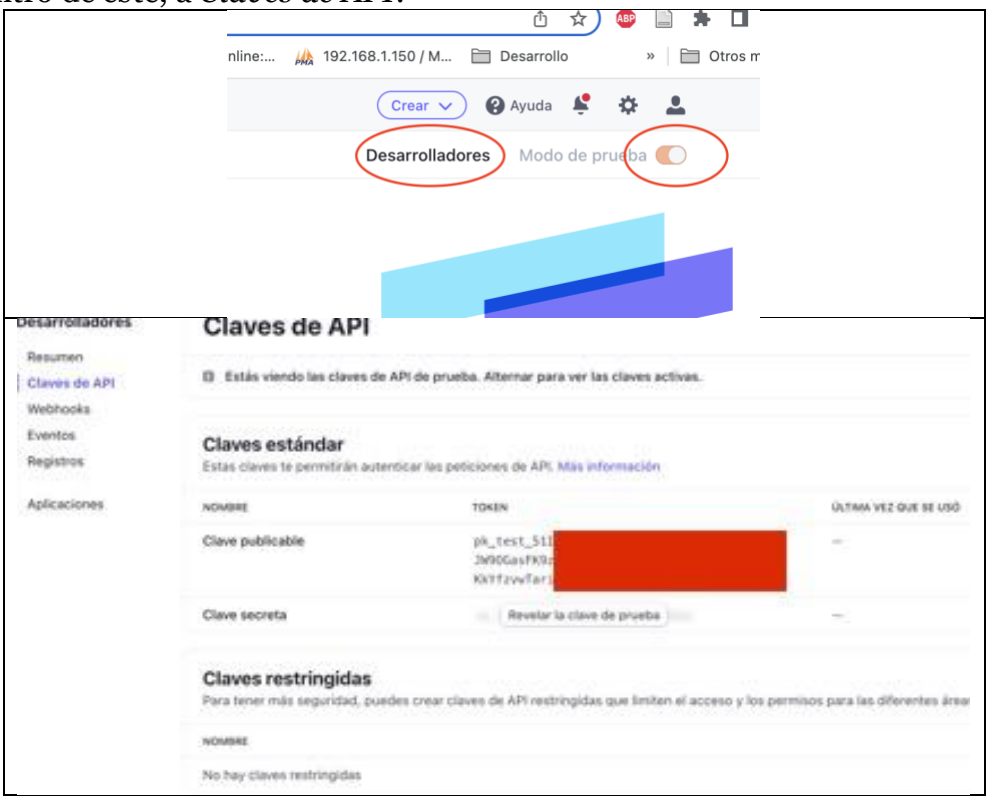


Figura 650. Claves publicable y secreta

Como vemos, tenemos la clave publicable y la clave secreta. La clave publicable podremos ponerla con tranquilidad en la página web; la secreta debe permanecer bajo nuestra custodia. Ambas sirven para identificar a nuestro sistema dentro de Stripe.

2.2. Necesidades para procesar el flujo de mensajes

Como se ha comentado, la transacción se inicia por una petición que recibimos del cliente y que nuestro servidor envía a Stripe. En la Figura 648, al controlador de nuestro sistema que recibe la petición lo hemos llamado *PaymentsController,* y debe ofrecer dos servicios: (1) *prepay*, que solicita la preautorización a Stripe y obtiene de este un token que identifica la transacción, y (2) otro, representado en esa figura por el mensaje 3.3, que se ejecuta cuando el cliente ha recibido un OK por parte de Stripe.

Además, en el lado cliente tenemos que habilitar las operaciones necesarias para que el usuario pueda enviarnos desde su *front-end* la petición al servicio *prepay* (mensaje 1.1), recibir el token que nos ha enviado Stripe (mensaje 1.1.3), mostrar el formulario para rellenar los datos del medio de pago, enviar el pago directamente a Stripe (mensaje 3.1) y enviarnos la respuesta de que todo ha ido bien (mensaje 3.3).

2.3. Implementación del controlador de pagos (I)

El controlador *PaymentsController* debe ofrecer, en primer lugar, el servicio que en la Figura 648 hemos llamado *prepay*.

Como ofrecemos dos tipos de bono (10 partidas por 10 euros o 20 partidas por 15 euros), supondremos que la elección del usuario se envía desde el cliente como un parámetro *matches* en la *url* de la petición: si desea el bono de 10 partidas, la petición llegará como *payments/prepay?matches=10*.

El código del controlador que necesitamos por el momento se muestra completamente en la siguiente figura, en la que destacamos:

1) El controlador está ubicado en el paquete *http,* junto al resto de controladores. Su nombre público es *payments,* y dispone de la anotación para admitir peticiones desde nuestro origen.
2) Asignamos al campo estático *apiKey* del objeto Stripe (que está importado) la clave secreta. No obstante, esas líneas de código aparecen tachadas porque no es conveniente tenerla ahí "hardcodeada", sino que es preferible mantenerla en un fichero de configuración.
3) En el servicio web *prepay* se comprueba que el usuario esté logueado y que el número de partidas que se quieren comprar sea 10 o 20.
4) En función del número de partidas, a la variable *total* se le asigna 10 o 15 euros. Desde luego, esto estaría mucho mejor si residiera en la base de datos en lugar de estar aquí "hardcodeado", pero pase por esta vez.
5) Como Stripe requiere que el importe se le pase en céntimos de euro, multiplicamos el importe de la compra por 100.
6) Luego, y siguiendo las instrucciones que nos ofrece Stripe en su sitio web para usar su API, creamos un objeto *params* de tipo *PaymentIntentCreateParams* que construimos pasando la divisa y el importe.
7) Usando la librería de Stripe (que hemos debido incluir en nuestro fichero *pom.xml,* según las instrucciones del sitio web de Stripe), se envía una petición al servidor de Stripe.
8) Como respuesta a esta petición, Stripe nos devuelve un *intent* en el que encontramos un campo *client_secret,* que es el token "asdfg" que resaltábamos con un óvalo en varios mensajes de la Figura 648. Este token *client_secret* es el que enviamos de vuelta al *front-end*.
9) Por último, declaramos el servicio *confirm,* que tenemos pendiente de implementar.

```
<dependency>
    <groupId>com.stripe</groupId>
    <artifactId>stripe-java</artifactId>
    <version>22.5.1</version>
</dependency>
```

```
 package edu.uclm.esi.juegos.http;

import javax.servlet.http.HttpSession;

import org.json.JSONObject;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

import com.stripe.Stripe;
import com.stripe.model.PaymentIntent;
import com.stripe.param.PaymentIntentCreateParams;


@RestController
@RequestMapping("payments")
@CrossOrigin(origins = "http://localhost:4200",
   allowCredentials = "true"
)
public class PaymentsController {
```

```
    static {
        Stripe.apiKey = "sk_test_51IdX...NT";
    }

    @GetMapping("/prepay")
    public String prepay(HttpSession session, @RequestParam int matches) {
        try {
            if (session.getAttribute("userId")==null)
                throw new ResponseStatusException(HttpStatus.FORBIDDEN,
                    "You must be logged in to buy matches");
            if (matches!=10 && matches!=20)
                throw new ResponseStatusException(HttpStatus.CONFLICT,
                    "You can only buy either 10 or 20 matches");

            long total = matches==10 ? 10 : 15;
            total = total * 100;

            PaymentIntentCreateParams params = new PaymentIntentCreateParams.Builder()
                    .setCurrency("eur")
                    .setAmount(total)
                    .build();

            PaymentIntent intent = PaymentIntent.create(params);
            JSONObject jso = new JSONObject(intent.toJson());
            String clientSecret = jso.getString("client_secret");
            session.setAttribute("client_secret", clientSecret);
            session.setAttribute("matches", matches);
            return clientSecret;
        } catch (Exception e) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e.getMessage());
        }
    }

    @GetMapping("/confirm")
    public void confirm(HttpSession session) {

    }
}
```

Figura 651. Código provisional del PaymentsController. En el recuadro, la dependencia que necesitamos añadir al *pom.xml*

2.4. Comprobación de las partidas pagadas

Cuando el usuario logueado le da al botón *Play Order the numbers against an opponent*, el *back* debe comprobar si el usuario que hay en la sesión tiene crédito suficiente para jugar la partida multijugador que desea.

El lector o lectora recordará que el mensaje se recibe en el *GamesController* que, a su vez, delega la ejecución al servicio *WaitingRoom:* bien, pues es aquí el lugar en el que comprobaremos si el usuario puede o no jugar una partida multijugador. El código del controlador permanece por ahora exactamente igual que estaba (Figura 468, página 291), pero cambia ligeramente la implementación de *WaitingRoom* (Figura 652, arriba), pues ahora comprueba que el campo *paidMatches* (algo así como "partidas pagadas") sea mayor que cero. Si no lo es, le enviamos una respuesta NOT_ACCEPTABLE, cuyo código es 406.

```java
public Match playMatch(String userId, String gameName) {
   Match match = null;

   User user = this.userDAO.findById(userId).get();
   Integer numberOfMatches = user.getPaidMatches();
   if (numberOfMatches==null || numberOfMatches==0)
      throw new ResponseStatusException(HttpStatus.NOT_ACCEPTABLE,
         "Not enough credit");
   ...
}
```

```java
public class User {
   @Id @Column(length = 36)
   private String id;
   @Column...
   private String name;
   ...
   private Integer paidMatches;
   ...
```

Figura 652. Cambios para comprobar si se puede jugar multijugador

Antes de probar esta nueva funcionalidad, necesitamos añadir la columna correspondiente al campo *paidMatches* en la tabla *Users*. Para esto, basta simplemente con modificar el valor del campo *spring.jpa.hibernate.ddl-auto* a *update* en *application.properties*. Antes de ejecutar, tiene las columnas de la izquierda; luego, las de la derecha:

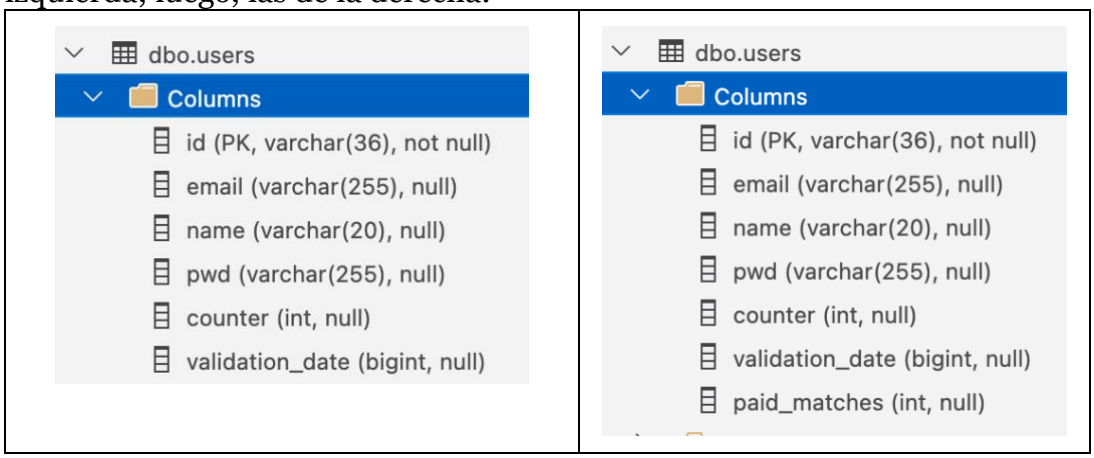


Figura 653. Spring ha añadido la columna paid_matches *a la tabla* Users

Lo importante aquí es que, si el usuario no ha pagado, no se le debe dejar jugar en modo multijugador. Podemos hacer dos cosas: (1) redirigirlo desde el servidor (enviándole una respuesta 304) a una página en la que pueda comprar partidas o (2) procesar en el cliente el código de respuesta (406, NOT_ACCEPTABLE) y, desde el propio cliente, mostrarle la página de pagos. Optaremos por esta segunda opción.

2.5. Procesamiento en el cliente de la respuesta 406

La lectora o lector recordará que la petición al servicio *play* se envía desde el método *playOtn* de la clase *MultiplayerComponent* del *front-end*. De momento, en este método solo procesamos respuestas con código de la familia del 200; ahora debemos estar preparados para recibir códigos 406.

Ya en la Figura 220 (página 150) controlábamos las dos posibles respuestas del servicio *checkWord:* ahora haremos lo mismo, solo que con un redireccionamiento que hará el propio cliente cuando el código de la respuesta

sea 406: en las líneas 34 a 36 de la Figura 654 se captura el error emitido por el *GamesController* y el *WaitingRoom* que hemos visto hace unos párrafos.

```
24   playOtn() {
25     this.gamesService.playMatch("otn").subscribe(
26       next : (response) => {
27         this.showingOtn = true
28         this.showingWR = false
29         this.message = "Waiting for an opponent..
30         let state = new OtnMulti(response.body.id
31         this.match = new OtnMatch(state)
32         this.onMatchReceived.emit(this.match)
33       },
34       error : (response) => {  response = HttpEr
35         console.log(response)
36       }
37     })
38   }
```

Figura 654. Captura del error 406

En la línea 32 de la figura anterior vemos que este componente emite un evento a su componente contenedor (*AppComponent)* cuando la respuesta es satisfactoria. Ahora, para conseguir el mismo efecto que ese redireccionamiento que podríamos haber hecho desde el servidor, emitiremos un evento indicando al contenedor que el usuario no tiene crédito suficiente para jugar una partida multijugador: el componente principal recibirá la ocurrencia de este evento y lo manejará mostrándole una página para que efectúe el pago.

```
export class MultiplayerComponent implements OnInit {
  ...

  @Output() onMatchReceived : EventEmitter<OtnMatch> = new EventEmitter()
  @Output() onNotEnoughCredit : EventEmitter<any> = new EventEmitter()

  constructor(private gamesService : GamesService) { }

  ...

  playOtn() {
    this.gamesService.playMatch("otn").subscribe({
      next : (response) => {
        ...
      },
      error : (response) => {
        this.onNotEnoughCredit.emit()
      }
    })
  }
```

Figura 655. Emitimos un evento cuando el usuario no tiene crédito

Obviamente, el evento *onNotEnoughCredit* se recibe en el componente principal (Figura 656, arriba), que le solicita la ejecución de *showPayment* en *AppComponent*.

```
...
<div *ngIf="showingMultiplayer">
  Logged as {{ userName }}
  <br>
  <app-multiplayer (onMatchReceived)="setMatch($event)"
          (onNotEnoughCredit)="showPayment()"></app-multiplayer>
</div>
...
```

Figura 656. Nos suscribimos al evento onNotEnoughCredit *en app.component.html*

Siguiendo nuestro estilo, el método *showPayment* de *AppComponent* simplemente pone un campo *showingPayment* a *true,* del mismo modo en que poníamos también a *true* las variables *showingRegister, showingLogin* o *showingMultiplayer*.

2.6. Envío de la petición de prepago

Cuando *showingPayment* sea *true,* mostraremos un componente *payments* (que debemos construir con *ng generate component payments*) en el que daremos al jugador la posibilidad de comprar un bono de 10 o de 20 partidas. En la Figura 657, Pepe ha pulsado el botón para jugar contra alguien, pero recibe la respuesta 406, con lo que se le muestra el componente *payments*.

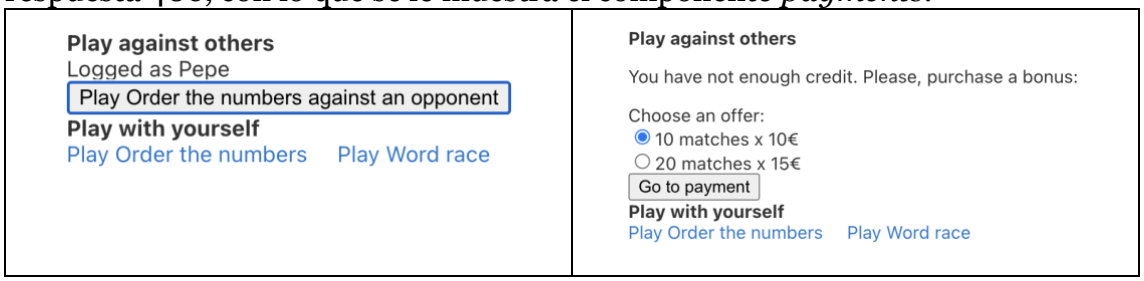


Figura 657. Pepe quiere jugar contra alguien, pero debe comprar un bono

La vista de este componente es muy sencilla, así como su modelo (Figura 658):

1) La vista ofrece las dos opciones de compra y un botón para ir al pago.
2) Al pulsarlo, se ejecuta el método *requestPrepayment* del modelo, que utiliza un *PaymentsService* que envía la petición de prepago a nuestro *PaymentsController*.
3) Como sabemos, el recurso web *prepay* devuelve al *front-end* el *client_secret* generado por Stripe para identificar la transacción. De momento, solo lo mostramos en un *alert*.

```
<p>You have not enough credit. Please, purchase a bonus:</p>

Choose an offer:<br>

<input (change)="amount=10" type="radio" name="option" value="10" checked>
10 matches x 10€<br>
<input (change)="amount=20" type="radio" name="option" value="20">
20 matches x 15€<br>
<button (click)="requestPrepayment()">Go to payment</button>
```

```
import { Component, OnInit } from '@angular/core';
import { PaymentsService } from './payments.service';

@Component({
  selector: 'app-payments',
  templateUrl: './payments.component.html',
  styleUrls: ['./payments.component.css']
})
export class PaymentsComponent implements OnInit {
  amount : number = 10

  constructor(private paymentsService : PaymentsService) { }

  ngOnInit(): void { }

  requestPrepayment() {
    this.paymentsService.prepay(this.amount).subscribe({
      next : (response : any) => {
```

```
        alert(response.body)
      },
      error : (response : any) => {
        alert(response)
      }
    })
  }
}
```

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { httpUrl } from '../commons';

@Injectable({
  providedIn: 'root'
})
export class PaymentsService {

  constructor(private client : HttpClient) { }

  prepay(matches: number) : Observable<Object> {
    return this.client.get(httpUrl + "payments/prepay?matches=" + matches, {
      withCredentials : true,
      observe : "response",
      responseType : "text"
    })
  }
}
```

Figura 658. Código del componente de pagos y de su servicio

2.7. Carga del formulario de pago

Cuando se recibe el *client_secret* que identifica la transacción debemos cargar el formulario para que el usuario rellene los datos de la tarjeta. Esto lo podemos hacer de dos formas: instalar el módulo de Stripe para Angular y seguir las instrucciones que se nos indican (https://www.npmjs.com/package/stripe-angular) o hacerlo manualmente siguiendo las instrucciones de Stripe para cualquier sitio web.

Utilizaremos la segunda alternativa, porque con ella veremos algún aspecto algo interesante de programación con Angular.

Como sabemos, cuando el usuario reciba el *client_secret*, rellene los datos de la tarjeta y pulse el botón de pagar, se debe enviar directamente a Stripe la clave publicable, el *client_secret* y la información de la tarjeta.

Igual que en el *backend* hemos importado la librería de Stripe, también en el *front-end* debemos importar un script del sitio de Stripe. Para ello, en *index.html* añadimos la línea resaltada antes del fin del *body:*

```
...
<body>
  <app-root></app-root>

  <script src="https://js.stripe.com/v3/"></script>
</body>
...
```

Figura 659. Descarga del script de Stripe

Con la instrucción anterior podemos acceder desde cualquier punto de nuestra aplicación a los objetos y operaciones definidos en ese script. En particular, instanciaremos en *PaymentsComponent* un objeto de clase *Stripe* pasándole como parámetro la clave publicable. Además, declararemos un campo *transactionId* de tipo string en el que guardaremos el *client_secret:*

```
export class PaymentsComponent implements OnInit {
  amount : number = 10
  transactionId? : string
  stripe = Stripe("pk_test_51Id...BHC")
   ...
```

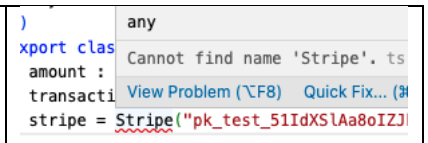


Figura 660. Añadimos dos nuevos campos al componente de pagos

A pesar de que el tipo *Stripe* que estamos incluyendo en el componente es público, el compilador de Angular nos informa del error que aparece en el lado derecho de la figura anterior. Para poder utilizar el objeto, necesitamos añadir la sentencia *declare let Stripe : any* después de los *import:*

```
import { Component, OnInit } from '@angular/core';
import { PaymentsService } from './payments.service';

declare let Stripe : any;
...
```

Figura 661. Habilitamos el acceso al tipo Stripe

Cuando llega la respuesta del *backend* con el *client_secret*, lo guardaremos en el campo *transactionId* y cargaremos el formulario para introducir los datos de la tarjeta:

```
requestPrepayment() {
  this.paymentsService.prepay(this.amount).subscribe({
    next : (response : any) => {
      this.transactionId = response.body
      this.showForm()
    },
    error : (response : any) => {
      alert(response)
    }
  })
}
```

Figura 662. Nueva implementación de requestPrepayment *en* PaymentsComponent

La creación del formulario no la hacemos nosotros directamente, sino que se encarga de ello el campo *stripe* que hemos instanciado en la Figura 660 (véase, en la siguiente figura, la primera línea resaltada). Siguiendo uno de los varios ejemplos de código que podemos encontrar en el sitio web de Stripe (https://stripe.com/docs/payments/elements), redactamos el método *showForm* como en la Figura 663. Obsérvese, por ejemplo, que se crea un objeto *card* a partir de *elements:* este *card* no es tampoco nuestro, sino que es un objeto de alguno de los tipos definidos en el script de Stripe que descargamos en *index.html*. Se usa también la clase CSS *#card-element,* que estará también definida ahí. Como también se ve, los campos para rellenar los datos de la tarjeta se colocan en algún lugar de la página llamado *payment-form*, un formulario al que añadimos (según las instrucciones del proveedor) una función para manejar el evento *submit:* cuando el botón de pago se pulse, se ejecutará el método *payWithCard,* que enviará la información directamente a Stripe.

```
showForm() {
  let elements = this.stripe.elements()
  let style = {
    base: {
      color: "#32325d", fontFamily: 'Arial, sans-serif',
      fontSmoothing: "antialiased", fontSize: "16px",
      "::placeholder": {
        color: "#32325d"
      }
    },
```

```
    invalid: {
      fontFamily: 'Arial, sans-serif', color: "#fa755a",
      iconColor: "#fa755a"
    }
  }

  let card = elements.create("card", { style : style })
  card.mount("#card-element")
  card.on("change", function(event : any) {
    document.querySelector("button")!.disabled = event.empty;
    document.querySelector("#card-error")!.textContent =
            event.error ? event.error.message : "";
  });

  let self = this
  let form = document.getElementById("payment-form");
  form!.addEventListener("submit", function(event) {
    event.preventDefault();
    self.payWithCard(card);
  });
  form!.style.display = "block"
}
```

Figura 663. Creación del formulario para rellenar los datos de la tarjeta

En la vista (*payments.component.html)* debemos incluir ese formulario con *id=payment-form,* para que este elemento sea encontrable desde el modelo. Utilizaremos la siguiente propuesta de Stripe:

```
...
<button (click)="requestPrepayment()">Go to payment</button>

<form id="payment-form" style="display:none">
    <div id="card-element"></div>
    <button id="submit">
        <div class="spinner hidden" id="spinner"></div>
        <span id="button-text">Pay now</span>
    </button>
    <p id="card-error" role="alert"></p>
    <p class="result-message hidden">
        Payment succeeded, see the result in your
        <a href="" target="_blank">Stripe dashboard.</a> Refresh the page to pay again.
    </p>
</form>
```

Figura 664. Formulario para los datos de tarjeta en payments.component.html

2.8. Ejecución del pago

Para enviar el pago al proveedor debemos implementar el método *payWithCard* (la fila superior de la Figura 665 muestra un ejemplo basado en los proporcionados por Stripe). Si recibimos respuesta satisfactoria, lo comunicaremos a nuestro *backend* mediante una petición (Figura 665, abajo) al servicio web *confirm* que tenemos pendiente de implementar en el *PaymentsController*.

```
payWithCard(card : any) {
  let self = this
  this.stripe.confirmCardPayment(this.transactionId, {
    payment_method: {
      card: card
    }
  }).then(function(response : any) {
    if (response.error) {
      alert(response.error.message);
    } else {
      if (response.paymentIntent.status === 'succeeded') {
        alert("Pago exitoso");
        self.paymentsService.confirm().subscribe({
            next : (response : any) => {
              alert(response)
            },
            error : (response : any) => {
              alert(response)
            }
          })
      }
    }
  });
}
```

```
confirm() : Observable<any> {
  return this.client.get<any>(httpUrl + "payments/confirm", {
    withCredentials : true,
    observe : "response"
  })
}
```

Figura 665. Ejecución final del pago

En la figura anterior, al recibir la respuesta satisfactoria de Stripe mostramos un *alert:* lo que debe hacer el lector o lectora es emitir un evento para que se cargue el componente multijugador.

Nuestro *PaymentsController* debe recibir el mensaje enviado en la figura anterior, hacer ciertas comprobaciones con él y, si todo va bien, pedirle al *UserService* que le incremente al usuario el número de partidas pagadas:

```
@Autowired
private UserService userService;

...

@GetMapping("/confirm")
public void confirm(HttpSession session) {
   if (session.getAttribute("client_secret")==null
         || session.getAttribute("matches")==null
         || session.getAttribute("userId")==null)
      throw new ResponseStatusException(HttpStatus.FORBIDDEN);

   String userId = session.getAttribute("userId").toString();
   this.userService.addMatches(userId, (Integer) session.getAttribute("matches"));
}
```

```
public void addMatches(String userId, Integer matches) {
   User user = this.userDAO.findById(userId).get();
   Integer paidMatches = user.getPaidMatches();
   if (paidMatches==null)
      paidMatches = 0;
   user.setPaidMatches(paidMatches + matches);
   this.userDAO.save(user);
}
```

Figura 666. Nuevo código en PaymentsController *y en UserService*

Una vez que el componente de pagos está completamente implementado, podemos probarlo con algunas de las tarjetas de prueba que encontramos en https://stripe.com/docs/testing#cards. Una muy cómoda es la Visa 4242..., pues todos los campos del formulario se rellenan fácil y rápidamente.

Casi para terminar, deberíamos implementar alguna función para que nuestro sistema decremente al usuario conectado el número de partidas pagadas cada vez que inicia una partida multijugador. Por último, sí que deberíamos intentar modificar la apariencia del formulario de pagos aplicando algún estilo que lo haga más atractivo.

SÉPTIMA PARTE: INTRODUCCIÓN A LOS MICROSERVICIOS Y A LA SEGURIDAD

Capítulo 26. Filtros *http* en Spring

1. Introducción

Para que el servidor que tenemos corriendo en el puerto 80 admita adecuadamente las peticiones que le llegan desde el 4200, en varios controladores (en *UserController, GamesController* y PaymentsController) tuvimos que escribir la anotación de la Figura 667 para que estos controladores admitiesen peticiones con credenciales (realmente cookies*)* desde el origen *http://localhost:4200*. El *WordRaceController,* que no necesita cookies, simplemente tenía la anotación `@CrossOrigin(origins = "*")`.

```
@CrossOrigin(origins = "http://localhost:4200",
   allowCredentials = "true"
)
```

Figura 667. Anotación que tenemos en varios controladores

Si queremos admitir peticiones vengan del origen que vengan, en lugar de añadir la anotación anterior en todos los controladores, podríamos escribir una clase que, a modo de interceptor, se ejecute antes de que la petición llegue a cualquier controlador. Una clase de este tipo es lo que se llama un "filtro".

2. Un filtro sencillo

Del Capítulo 19 (especialmente a partir de la página 278), se recordará que las peticiones CORS se envían "en dos veces":

1) Primero va una petición de *preflight* (que se envía con el método OPTIONS), que el navegador utiliza algo así como para preguntar al servidor si puede enviarle una petición real con ciertas características («quiero ejecutar *users/login* con el método PUT», por ejemplo). El servidor responde con una serie de cabeceras, de las que poníamos un ejemplo en la Figura 450 (página 280). En esta figura se mostraban las cabeceras a la petición *preflight* del servicio *users/login,* de las que podíamos deducir que este servicio admitía:
    a. El origen *http://localhost:4200* (porque así lo habíamos "hardcodeado" en la anotación *@Crossorigin* del controlador).
    b. Que la petición real lleve cookies (*withCredentials = "true").*
    c. Que la petición real traiga la cabecera *content-type*.
    d. Que la petición real se envíe con el método PUT.
2) Después viaja la petición real, si es que el navegador considera, conforme a la respuesta recibida en la *preflight*, que será admitida por el servidor.

Supongamos, como hemos dicho en el último párrafo de la sección anterior, que queremos admitir todas las peticiones en todos los controladores vengan de donde vengan y traigan el método que traigan. En lugar de anotar todos los controladores con una anotación *@CrossOrigin* similar, podemos escribir un filtro que capture y procese las peticiones antes de que lleguen a los controladores.

La Figura 668 muestra el código de un filtro, especialización del tipo *OncePerRequestFilter*. Cuando Spring Boot arranca la aplicación, lo detecta

gracias a su anotación *@Component* y lo aplica a todas las peticiones que le vayan llegando al *backend,* independientemente del controlador del que se trate.

El código que se muestra comprueba si existe la cabecera *origin* en la *request:* si no está presente, no entra en el *if* y reenvía la petición al controlador al que vaya encaminada; si sí está presente, preprocesa la *response*, añadiéndole el valor del *origin* a la cabecera *Access-Control-Allow-Origin,* y el resto de valores que se muestran. Con esto, todas las peticiones que lleguen desde cualquier origen y que soliciten alguno de los métodos indicados en la cabecera *Access-Control-Allow-Methods* que añadimos a la respuesta podrán ser servidas. Si es necesario, reléase lo que contamos acerca de las peticiones CORS en el Capítulo 19.

```
package edu.uclm.esi.juegos.http;

import java.io.IOException;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class ResponseFilter extends OncePerRequestFilter {

   @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
          FilterChain filterChain) throws ServletException, IOException {

      if (request.getHeader("origin")!=null) {
         response.setHeader("Access-Control-Allow-Origin", request.getHeader("origin"));
         response.setHeader("Access-Control-Allow-Methods",
            "GET,PUT,POST,DELETE,UPDATE,OPTIONS");
         response.setHeader("Access-Control-Allow-Credentials", "true");
         response.setHeader("Access-Control-Allow-Headers", "content-type");
      }
      filterChain.doFilter(request, response);
    }
}
```

Figura 668. Un filtro que se aplica a cada petición recibida

3. Jerarquía de los filtros

Los filtros son especializaciones de la interfaz *Filter* que, como se ve en la Figura 669, obliga a implementar las operaciones *init, doFilter* y *destroy*. El *ResponseFilter* que aparece abajo a la izquierda es el filtro cuyo código acabamos de mostrar, en el que solamente nos vemos obligados a implementar *doFilterInternal,* pues las operaciones abstractas heredadas de *Filter* se implementan en las superclases de las que hereda nuestro filtro.

La operación *doFilter* toma como parámetros la *request* (que representa la petición recibida), la *response* (la respuesta que se va a enviar) y una cadena de filtros (*FilterChain),* que es una clase que implementa la interfaz *FilterChain* que también se incluye en la figura.

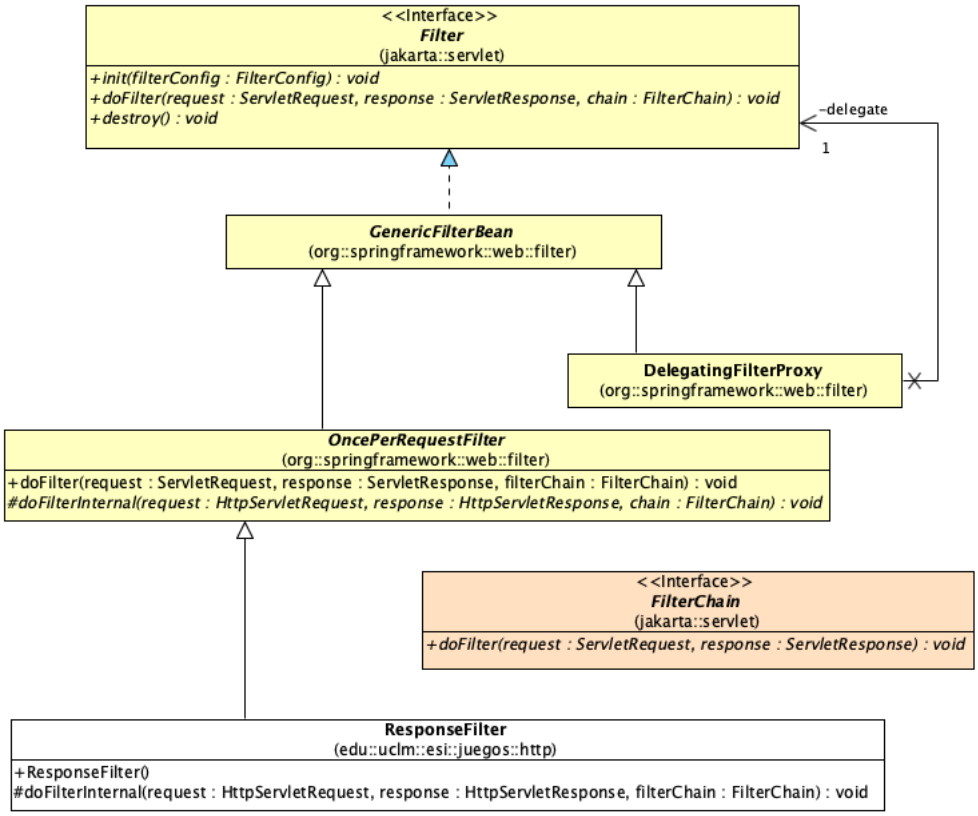


Figura 669. Parte de la jerarquía de filtros

La siguiente figura (Figura 670) es un poco difícil de ver debido a su anchura, pero es interesante para entender bien cómo funciona la cadena de filtros. La ejecución se ha detenido en una petición que se ha enviado desde el cliente al servicio *users/login:* como sabemos, llegarán dos peticiones (la de *preflight* y la real); en este caso, el punto de interrupción está situado en la primera línea ejecutable de nuestro *ResponseFilter,* y corresponde a la petición de *preflight*.

1) En la zona de variables se ve que existe un objeto *filterChain* de tipo *ApplicationFilterChain* que tiene cinco objetos, que se corresponden con los cinco filtros que aparecen en la zona de abajo a la derecha: hay un *OrderedCharacterEncodingFilter,* un *OrderedFormContentFilter,* un *OrderedRequestContextFilter,* luego está nuestro *ResponseFilter* y, finalmente, un *WSFilter*.
2) En la parte de la izquierda, en la que se muestra la pila de llamadas, vemos (cierto que con algo de esfuerzo) que la ejecución está detenida en el método en el que hemos puesto el punto de interrupción (*doFilterInternal).* La operación que se ha ejecutado inmediatamente antes ha sido *doFilter* sobre el mismo objeto: esta operación, que nosotros no hemos implementado, sí que está implementada (ya lo indicamos en el primer párrafo de esta sección) en alguna de las superclases.
3) Justo antes de este *doFilter* se ha ejecutado *internalDoFilter* sobre el objeto *ApplicationFilterChain,* que es (véase a la derecha, en la zona de variables) la propia cadena de filtros.
4) La cadena de filtros aparece varias veces en la pila de llamadas, señal de que la cadena llama al primer filtro que contenga, este devuelve la

respuesta a la cadena, que llama al segundo, que devuelve la respuesta, que llama al tercero, etcétera.

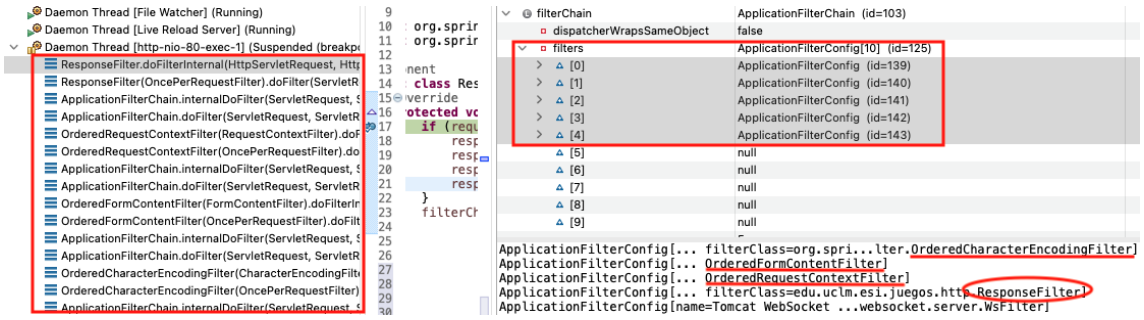


Figura 670. Pila de filtros en la cadena

Como decíamos, la figura anterior se ha detenido con la petición de *preflight* al recurso *users/login*. Si recordamos, esta petición admite el método PUT, y la petición de *preflight* lleva el método OPTIONS, por lo que la *preflight* no llega al *UserController*. Se ejecutan los interceptores y la pila de filtros, y la respuesta se devuelve al cliente. Este proceso se ilustra en la figura siguiente: lo más destacable es que la petición, al tratarse de la *preflight* (enviada autónomamente por el navegador al ir dirigida a un host distinto del origen), no llega al controlador, sino que se para en el último filtro de la cadena.

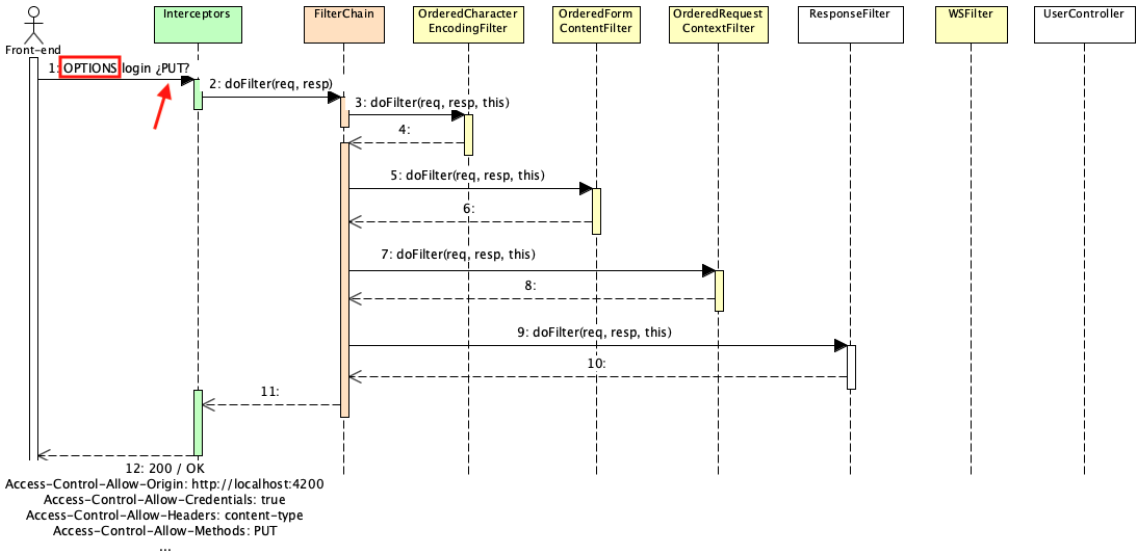


Figura 671. Procesamiento de una preflight *(OPTIONS) en el servidor*

Una vez que el *front* recibe el código 200, sabe que puede enviar la petición PUT por la que había preguntado en el mensaje 1. Cuando la envía, el código ya sí que se detiene en el punto de interrupción que, sabiamente 😎, hemos colocado en la primera línea del método *login* del *UserController* (Figura 672). Si nos fijamos en la pila de operaciones, vemos que la petición ha pasado por exactamente los mismos filtros si bien, cuando estos la han dejado pasar, ha terminado por llegar al *DispatcherServlet* (indicado con una flecha en la figura) y, de ahí, al controlador.

Si comparamos llamadas a los diferentes filtros incluidos en la pila de llamadas de la Figura 672, vemos que sobre el *WSFilter* (destacado con un recuadro) solo se ejecuta *doFilter,* mientras que en los demás se ejecutan tanto *doFilter* como *doFilterInternal:* sin duda, esto se debe a que el *WSFilter* que

Spring Boot nos incluye en la cadena de filtros no procesa la petición recibida por venir con protocolo *http*, cuando el *WSFilter* seguramente dedique a peticiones *ws*.

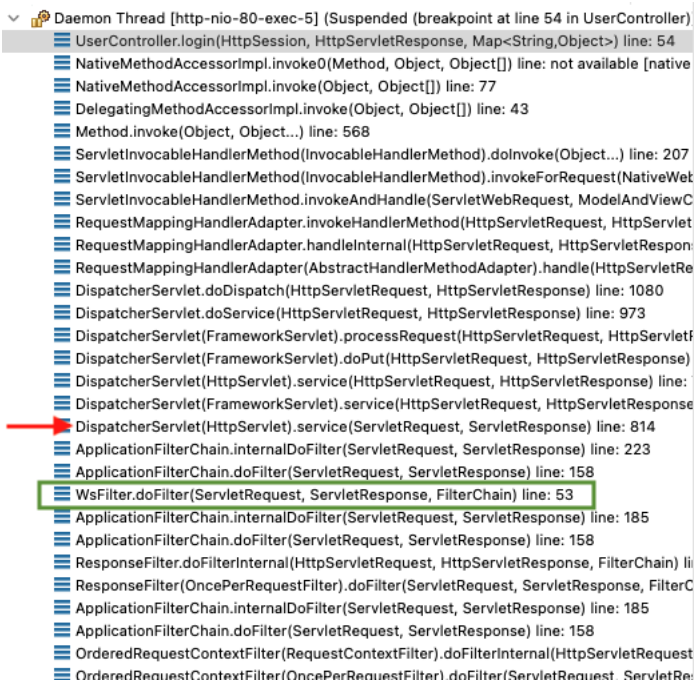


Figura 672. Pila de llamadas al llegar al recurso users/login

El cauce que ha seguido esta segunda petición es, entonces, muy parecido al anterior, pero ahora sí que se llega al controlador (Figura 673).

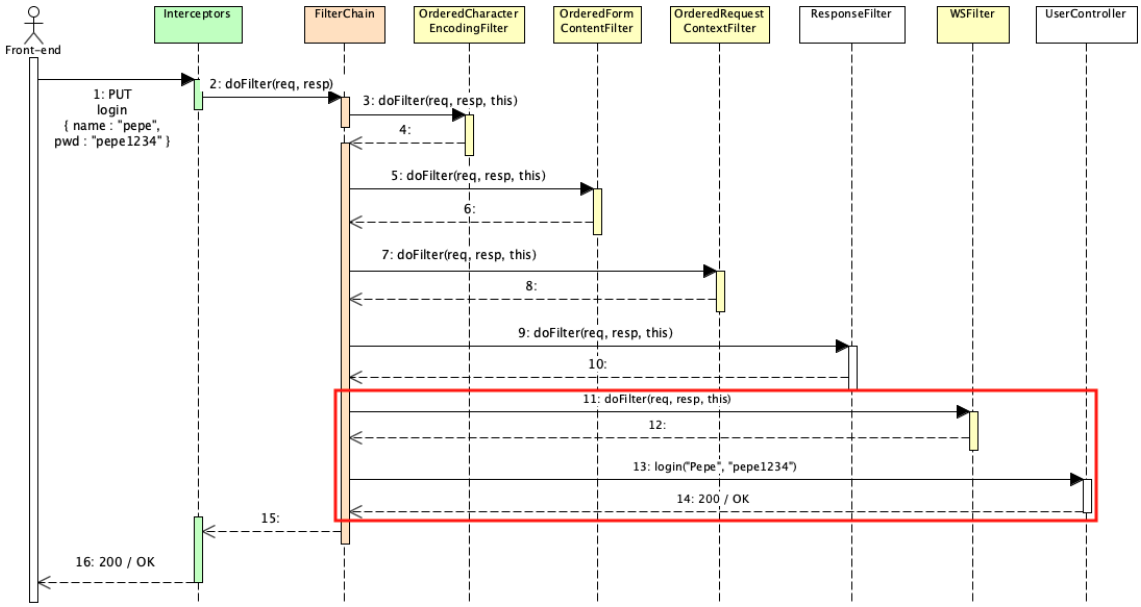


Figura 673. Procesamiento en el servidor de la petición real solicitada (PUT)

4. Manipulación de la cadena de filtros

En nuestro *ResponseFilter* de la Figura 668 añadíamos, en un solo filtro, cuatro cabeceras (*Access-Control-Allow-Origin*, *Access-Control-Allow-Methods*, *Access-Control-Allow-Credentials* y *Access-Control-Allow-Headers*) a las respuestas que devolvemos a todas las peticiones.

Aunque no tenga mucho sentido, o solo tenga un sentido didáctico, podríamos hacer exactamente lo mismo con cuatro filtros, uno por cabecera. Es decir, en lugar de tener una sola clase *ResponseFilter* con la implementación de la Figura 668, podríamos tener las cuatro que se muestran en la Figura 674, en la que cada filtro añade una cabecera diferente (las líneas del *package* y los *import* los mostramos solo en la primera).

```
package edu.uclm.esi.juegos.http;

import java.io.IOException;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class AccessControlAllowOriginFilter extends OncePerRequestFilter {
   @Override
   protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
           FilterChain filterChain) throws ServletException, IOException {
      if (request.getHeader("origin")!=null) {
         response.setHeader("Access-Control-Allow-Origin", request.getHeader("origin"));
      }
      filterChain.doFilter(request, response);
    }
}
```

```
@Component
public class AccessControlAllowMethodsFilter extends OncePerRequestFilter {
   @Override
   protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
           FilterChain filterChain) throws ServletException, IOException {
      response.setHeader("Access-Control-Allow-Methods",
                         "GET,PUT,POST,DELETE,UPDATE,OPTIONS");
      filterChain.doFilter(request, response);
    }
}
```

```
@Component
public class AccessControlAllowHeadersFilter extends OncePerRequestFilter {
   @Override
   protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
           FilterChain filterChain) throws ServletException, IOException {
      response.setHeader("Access-Control-Allow-Headers", "content-type");
      filterChain.doFilter(request, response);
    }
}
```

```
@Component
public class AccessControlAllowCredentialsFilter extends OncePerRequestFilter {
   @Override
   protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
           FilterChain filterChain) throws ServletException, IOException {
      response.setHeader("Access-Control-Allow-Credentials", "true");
      filterChain.doFilter(request, response);
    }
}
```

Figura 674. Cuatro filtros en vez de uno

Si ahora hacemos *login* y detenemos la ejecución en algún punto de interrupción que coloquemos por ahí, vemos que la cadena de filtros contiene los cuatro que hemos creado, pero en un orden arbitrario.

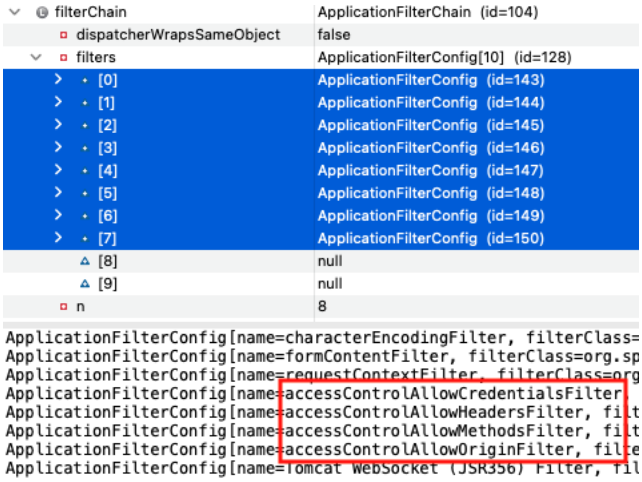


Figura 675. Nuestros cuatro filtros se colocan aleatoriamente

Supongamos que, por alguna razón (por ejemplo: en lugar de comprobar los parámetros de una petición de registro en el controlado, preferimos controlarlos en filtros que se ejecuten antes *users/register)*, queremos establecer un orden para los filtros.

Lo único que necesitamos es anotar las clases que implementan los filtros con *@Order* (anotación definida en la interfaz *org.springframework.core.annotation.Order* y que no tiene nada que ver con el *@Order* de los casos de prueba de JUnit). Si, por ejemplo, que los filtros se procesen en el mismo orden que en la Figura 668, anotaremos cada filtro como sigue:

```
package edu.uclm.esi.juegos.http;
...
import org.springframework.core.annotation.Order;
...
@Component
@Order(0)
public class AccessControlAllowOriginFilter extends OncePerRequestFilter {
...
```

```
@Component
@Order(1)
public class AccessControlAllowMethodsFilter extends OncePerRequestFilter {
...
```

```
@Component
@Order(2)
public class AccessControlAllowCredentialsFilter extends OncePerRequestFilter {
...
```

```
@Component
@Order(3)
public class AccessControlAllowHeadersFilter extends OncePerRequestFilter {
...
```

Figura 676. Anotación @Order *para especificar el orden de procesamiento de los filtros*

Y, ahora sí, la *filterChain* contiene los filtros en el orden deseado:

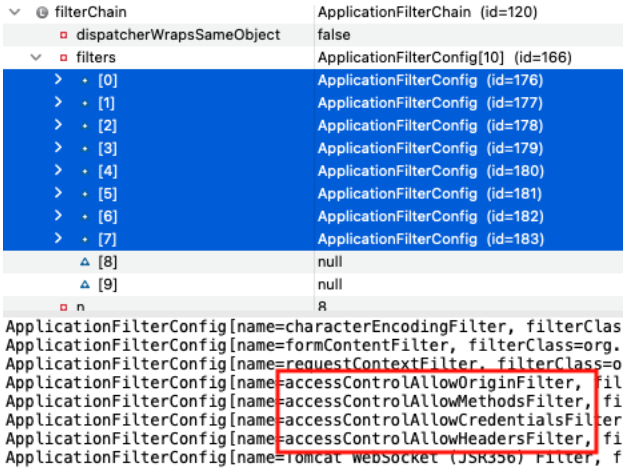


Figura 677. Los filtros se colocan en la cadena en el orden que deseamos

Obviamente, podemos eliminar las cuatro clases creadas en la Figura 676 y quedarnos únicamente con la de la Figura 668.

Capítulo 27. Introducción a los microservicios

1. Migración a una arquitectura de microservicios

La arquitectura de microservicios suele describirse comparándola con la arquitectura monolítica. Con esta, todos los servicios ofrecidos por el *backend* se encuentran en un solo sistema ejecutable; con aquella, los servicios se ofrecen en aplicaciones separadas, a menudo accesibles mediante *endpoints* distintos.

La arquitectura que tenemos hasta ahora de nuestro sistema de juegos es completamente monolítica, pues sus dos grupos de funcionalidades principales (gestión de usuarios y gestión de juegos) se hacen en la misma aplicación.

En efecto, tanto en sus primeros diseños arquitectónicos (Figura 32, página 50) como en los posteriores (Figura 316, 205), hemos considerado que todos nuestros servicios eran ofrecidos por un único servidor situado en *localhost:80*.

En lugar de considerar un diseño arquitectónico de ese estilo, podríamos separar el *backend* en dos o incluso en tres sistemas: uno para la gestión de usuarios, otro para el *Order the numbers* y otro para el *Word race*.

Supondremos, para este capítulo y los siguientes, que nosotros ofrecemos servicios de juegos de tablero, pero también otros, por lo que nos interesa mantener separadas las funcionalidades relacionadas con la gestión de usuarios. Cada servicio que ofrecemos tiene un precio, por lo que nos interesa almacenar el crédito que cada usuario tiene disponible. Además, y por razones que nos vienen impuestas, supondremos que la gestión de usuarios viene de antiguo y se hace con una base de datos MySQL.

Así, en lugar de tener dos sistemas o dos subproyectos tendremos tres (cinco si consideramos los servidores de base de datos; seis o siete si ya incluimos, aunque no se muestren, a los servidores de correo electrónico y de pago) que, además, podemos plantear arquitectónicamente de varias maneras. Describimos solamente tres:

1) La Figura 678 plantea que la parte relacionada con la gestión de cuentas de usuario del *front-end* sea el único que interactúe con el *backend* de gestión de usuarios. En algún momento hay cierta interacción, en el *front,* entre los paquetes *Account* y *Games:* por ejemplo, cuando un usuario quiere jugar en modo multijugador debe haber pagado el bono que creamos en el Capítulo 25, por lo que en algún momento habrá intercambio de información entre *Accounts* y *Games*.

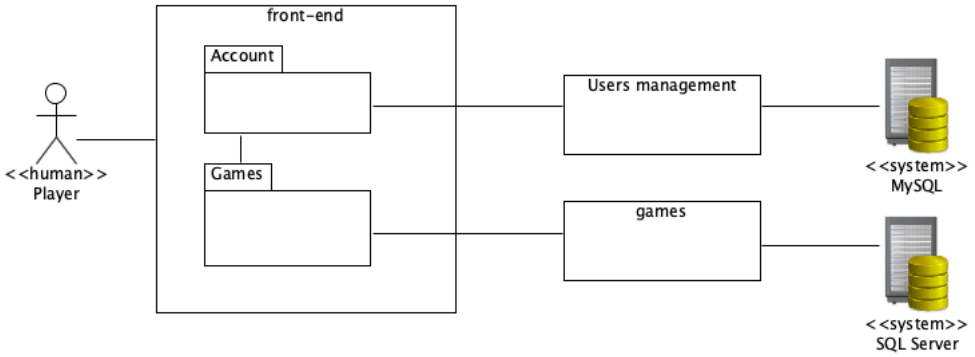


Figura 678. Alternativa 1 para la arquitectura de microservicios

2) En la versión de la Figura 679, el *front-end* se dirige solamente al *backend* de *Games*. De este modo, podemos suponer que, cuando un usuario se quiere registrar, loguear, etcétera, se lo dice a *Games,* que podemos asumir que recibe las peticiones de logueo y las reenvía al servidor *Users management*.

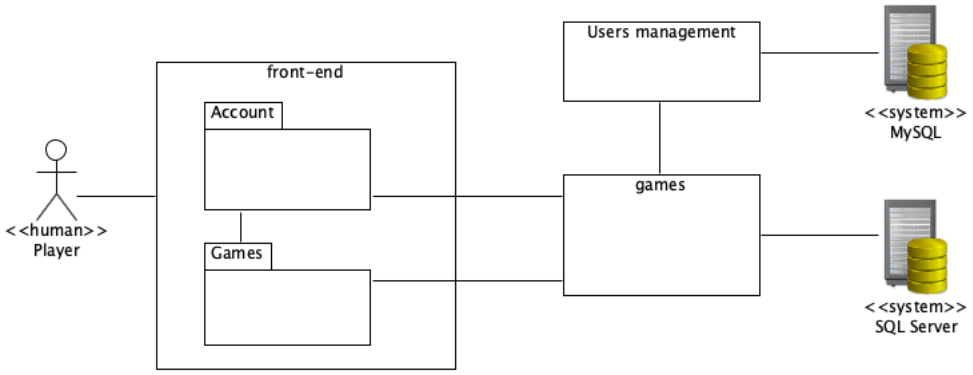


Figura 679. Alternativa 2 para la arquitectura de microservicios

3) En esta tercera y última versión de las que presentamos a tan alto nivel, hay comunicación: (a) entre el subsistema *Account* del *front-end* y el *backend Users management,* (b) entre el subsistema *Games* y el *backend games,* y (c) también entre los dos *backends*.
Supondremos que, con esta figura, el usuario utiliza el *back* de *Users management* para gestionar todo lo relacionado con su cuenta: crearla, loguearse, recuperar la contraseña, eliminar su cuenta, etcétera, y que utiliza el *back* servido por *games* para jugar. Este utiliza el sistema *Users management* para consultar o actualizar el crédito que pueda tener el usuario.

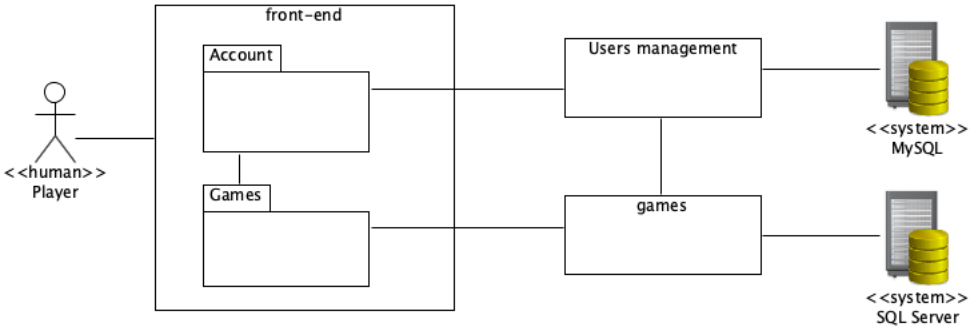


Figura 680. Alternativa 3 para la arquitectura de microservicios

2. Elección de alternativa

Ya que estamos suponiendo tanto, supondremos que, tras una reunión en la que se analizan los pros y los contras de cada alternativa, se opta por la tercera.

3. Creación del servidor de gestión de cuentas

Para gestionar las cuentas de usuario, crearemos un nuevo proyecto en Eclipse llamado *auth*, siguiendo prácticamente los mismos pasos de la sección 4 del Capítulo 5 (página 77 y siguientes). Las únicas diferencias son:

1) Como se nos impone el uso de MySQL para gestionar los usuarios, su fichero *pom.xml* debe importar alguna dependencia para acceder a este gestor de bases de datos.
2) El fichero *application.properties* de este nuevo proyecto debe definir el nuevo *endpoint* en el que escucha el servidor de MySQL.

De este modo, estos dos ficheros pueden quedar como en la Figura 681:

1) En *pom.xml,* sustituimos la dependencia de SQL Server por la de MySQL.
2) En *application.properties* indicamos el puerto en el que escuchará este sistema (no puede haber dos procesos escuchando en el mismo puerto, por lo que marcamos el 8000), sustituimos la url por otra que admita MySQL, y cambiamos el valor de las variables que definen el "dialecto" de acceso a la base de datos y el driver.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="...">
    <modelVersion>4.0.0</modelVersion>

    <groupId>edu.uclm.esi</groupId>
    <artifactId>auth</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>jar</packaging>

    <name>auth</name>
    <url>http://maven.apache.org</url>
    ...
    <dependencies>
        ...
        <dependency>
            <groupId>com.microsoft.sqlserver</groupId>
            <artifactId>mssql-jdbc</artifactId>
            <scope>runtime</scope>
        </dependency>

        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
        </dependency>
        ...
    </dependencies>
</project>
```

```
server.port = 8000

spring.datasource.url = jdbc:sqlserver://localhost;databaseName=games;encrypt=false;
spring.datasource.url = jdbc:mysql://localhost:3306/authlibro?serverTimezone=UTC
spring.datasource.username = sa
spring.datasource.password = DiSoft2022

spring.datasource.driverClassName=com.microsoft.sqlserver.jdbc.SQLServerDriver
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect

spring.jpa.hibernate.dialect=org.hibernate.dialect.SQLServer2009Dialect
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto = update
```

Figura 681. Contenido de pom.xml *y de* application.properties

Pero en la figura anterior hemos mantenido el nombre (*sa*) y la contraseña (*Disoft2022*) de la base de datos de SQL Server. Evidentemente, cuando Spring arranque la aplicación e intente conectarse a una base de datos servida en el puerto 3306 de *localhost* con el nombre *sa* y la contraseña *Disoft2022*, el servidor le dirá que nanay: si pudiera, le diría que ni existe esa base de datos, ni ese usuario ni esa contraseña.

3.1. Instalación de MySQL

Para crear la base de datos *authlibro* en *localhost,* tenemos que haber descargado e instalado un servidor de bases de datos de MySQL, para lo que podemos buscar "MySQL download" en Internet. Se nos ofrecen varias opciones, y de ellas elegiremos "MySQL Community Server".

Cuando descargamos en instalamos el servidor se habrá creado un usuario *root* con alguna contraseña que habremos dado. Salvo que se cambien los permisos, *root* será el completo administrador de esta instancia del gestor de bases de datos: es decir, podrá acceder a toda la información contenida en cualquier esquema que el servidor contenga, borrar, consultar o actualizar información, etcétera.

En nuestro caso, y de acuerdo con la línea *spring.datasource.username* de la Figura 681, necesitamos un usuario que se llame *sa* y cuya contraseña sea *Disoft2022* para que acceda a una base de datos que se llama *authlibro*. Para hacer todo esto, realmente podríamos haber colocado las credenciales *root* y la contraseña que hayamos dado al instalar MySQL Community Server, pero entonces estaríamos exponiendo estas credenciales (como sí que hacíamos, muy incorrectamente, en el Capítulo 6) a todos los programadores que tengan acceso a *application.properties*.

Para esto, lo más sencillo es volver a buscar "MySQL download" y descargar el MySQL Workbench: igual que un navegador es un cliente válido para trabajar con cualquier servidor web gracias a que ambos entienden el protocolo *http,* el MySQL Workbench es un cliente adecuado para trabajar con un servidor de MySQL, pues ambos entienden el mismo no ya protocolo, sino "subprotocolo": en la cadena de conexión *spring.datasource.url* de *application.properties,* cuyo valor ahora es *jdbc:mysql://localhost:3306/authlibro?serverTimezone=UTC*, el token *jdbc* es el protocolo y *mysql* el subprotocolo; en el caso de la conexión a SQL Server, cuya *connection string* es *jdbc:sqlserver://localhost;databaseName=games;encrypt=false, jdbc* es también el protocolo y *sqlserver* el subprotocolo.

O sea que, en esencia, un servidor de base de datos no es muy diferente de un servidor web, puesto que todos ellos:

1) Escuchan en un puerto: por defecto, un servidor web escucha en el 80 si va con *http* o en el 443 si es *https;* un servidor de SQL Sever escucha en el 1433; uno de MySQL, en el 3306.
   En cuanto a la implementación, podemos suponer que un servidor de bases de datos (un servidor de cualquier tipo, en realidad) es un *ServerSocket* (clase definida en el paquete *java.net*) o similar (pues depende del lenguaje en que se implemente) que se pone a escuchar en un puerto recibiendo peticiones y devolviendo respuestas. La parte de interfaz (es decir, la recepción de mensajes de los clientes) no parece entonces muy complicada: se reciben y, si se adaptan al protocolo establecido, se sirven.
2) Reciben mensajes codificados de cierta forma:
   a. Los servidores web, con el formato que ya conocemos (recurso, verbo, versión del protocolo, etcétera), pues *http* es un protocolo completamente estandarizado.
   b. Un servidor de base de datos, con un formato que especifica el propio fabricante (Microsoft u Oracle, en los casos respectivos de SQL Server y de MySQL).

Además del formato de las peticiones que se envían al servidor de base de datos (porque no dejan de ser peticiones del mismo tipo de las que un cliente envía a un servidor web), cada fabricante puede implementar un dialecto diferente de SQL, como vimos en la Figura 277 de la página 180: en aquel ejemplo veíamos que cada servidor de bases de datos necesita una instrucción SQL distinta para recuperar una fila aleatoria de la base de datos. Aunque todos los fabricantes deberían teóricamente adaptarse al estándar de SQL (el último es el ISO/IEC 9075:2016), en la práctica cada uno (Microsoft, Oracle, IBM o PostgreSQL Database Management System) hace ligeras variantes del lenguaje: por este motivo, es relativamente "peligroso" escribir instrucciones SQL con el atributo *nativeQuery=true* en los repositorios de Spring (como hacíamos por ejemplo en la Figura 280, página 181): si cambiamos de gestor de bases de datos, tal vez la sentencia deje de ser válida. Igualmente, los códigos de error que devuelve cada fabricante son diferentes, por lo que todo aquel *catch* con el que estuvimos bregando en la Figura 389 (página 248) nos dejará de servir.

3.1.1. Creación de la base de datos y del usuario *sa*

sa es el usuario administrador por defecto en SQL Server. En MySQL, el usuario administrador por defecto es *root*, pero vamos a crear un usuario en el servidor de MySQL que se llame igual y que tenga acceso solamente a la base de datos *authlibro*. Para ello, abrimos MySQL Workbench y creamos una conexión nueva (con el superusuario *root)* a la IP en la que tengamos el servidor (Figura 682, izquierda) y una base de datos que se llame *authlibro* (Figura 682, derecha).

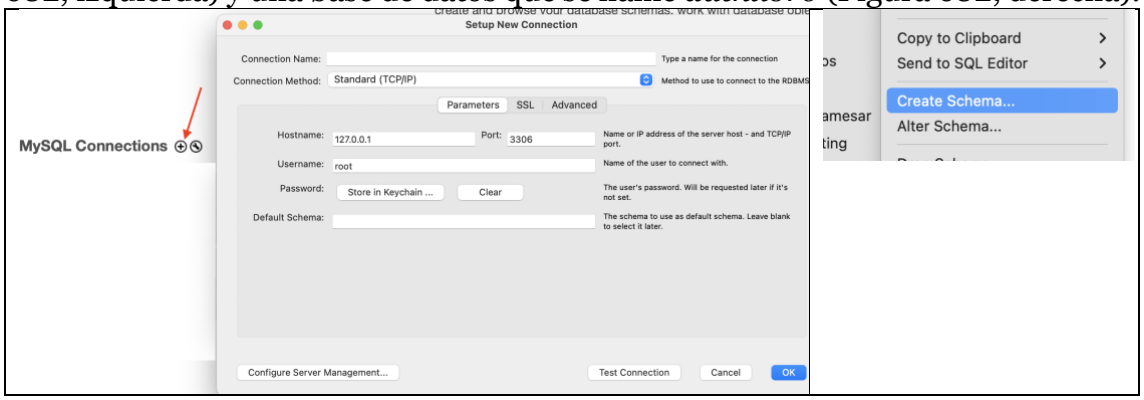


Figura 682. Creación de una conexión en MySQL Workbench (izquierda) y de una base de datos

Una vez creada, aparecerá listada en la solapa *Schemas,* junto a la que aparece también la solapa *Administration,* ambas en la Figura 683.

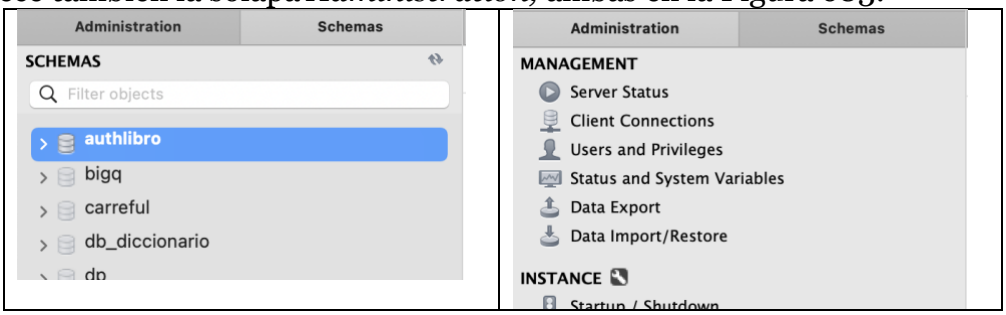


Figura 683. Solapas de esquemas y administración

En la solapa de administración pulsamos en *Users and Privileges* y luego en *Add Account*. Rellenamos los datos del usuario (*sa* como *Login name, Disoft2022* como contraseña) y pulsamos *Apply*. Acto seguido, pulsamos sobre la solapa *Schema Privileges* (resaltada en la Figura 684).

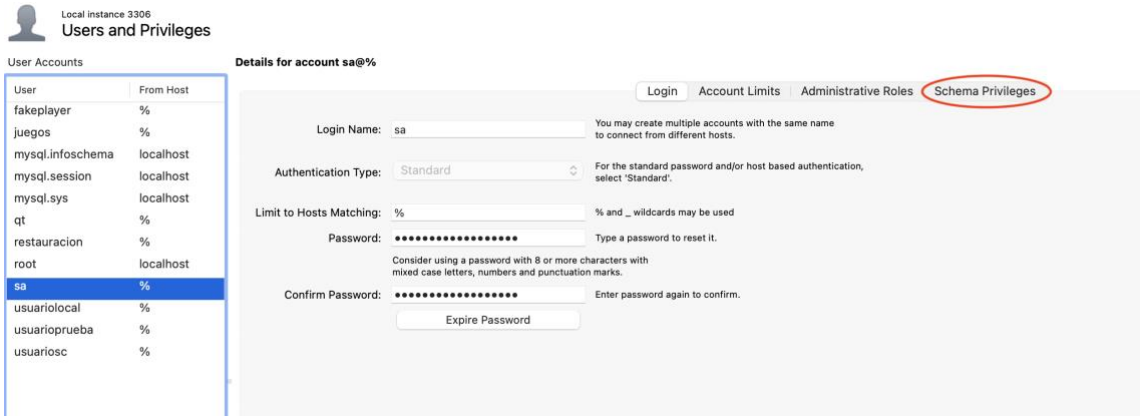


Figura 684. Creación del usuario

Ahora debemos darle al usuario *sa* permisos sobre *authlibro,* eligiendo esta base de datos con el botón *Add Entry*... Finalmente, seleccionamos los permisos que queramos otorgarle y pulsamos *Apply*.

Respecto de los permisos que le daremos, tendremos en cuenta que estamos en modo de desarrollo, y que *sa* es el usuario que utilizará el *backend* para conectarse a la base de datos. Querremos que Spring procese las entidades que creemos y que cree o modifique tablas; querremos también que *sa* pueda insertar usuarios en la tabla *users* que debe crearse cuando un jugador se registre, o eliminarlos cuando se quiera dar de baja... Por ello, podemos seleccionar todos los permisos. Luego, cuando se realice el despliegue de la aplicación, los revisaremos y los reduciremos al mínimo necesario.

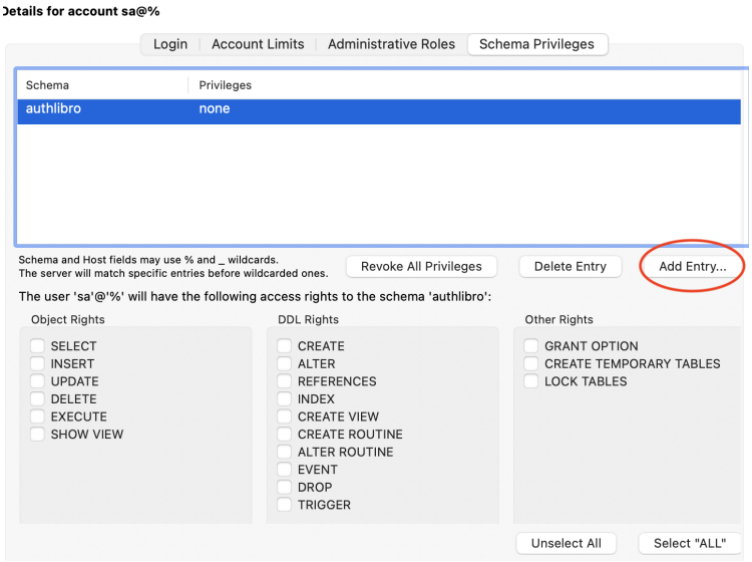


Figura 685. Asignación de permisos al usuario

3.2. Diseño del servidor

A muy alto nivel, y de acuerdo con la alternativa 3 que hemos elegido, la estructura de este servidor será la de la siguiente figura, en la que se ofrecen únicamente servicios relacionados con la gestión de cuentas para que el usuario

pueda registrarse, loguearse, etcétera (se han puesto tres a modo de ejemplo) y un par de funcionalidades adicionales para que otros *backends* (el *Games* que hemos desarrollado, pero tal vez otros) puedan acceder al crédito del usuario en función de lo que haya pagado. Obsérvese que nos traemos a este proyecto el envío de correos, y que la base de datos que usamos es la que acabamos de crear en MySQL.

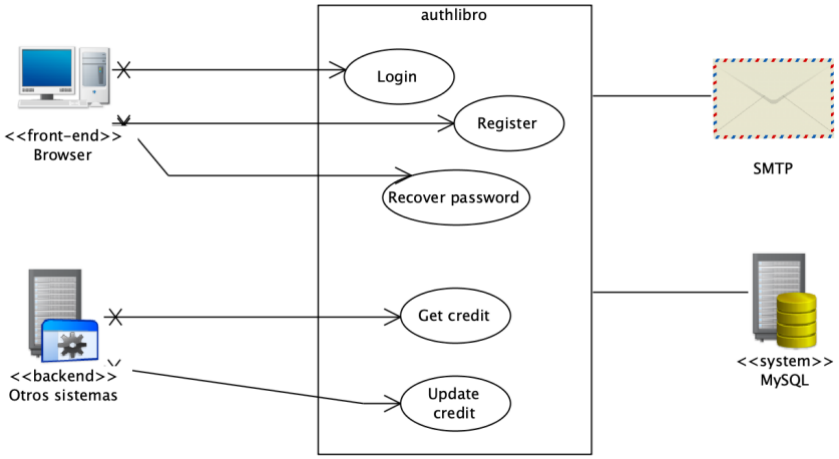


Figura 686. Vista funcional del sistema auth

Por otro lado, y a más bajo nivel, la arquitectura de este sistema será similar a la que hemos venido utilizando: un controlador recibe mensajes y los pasa a un servicio que interactúa con los repositorios y las entidades.

El contenido de este proyecto se muestra en la Figura 687: nos hemos traído todo lo relacionado con la gestión de cuentas. Simplemente por facilitar la lectura y la distinción entre los elementos de un proyecto y otro, hemos antepuesto el prefijo "A" (de Autorización) a todos los ficheros de este nuevo proyecto.

Además, los servicios del nuevo *AUserController* comenzarán por "a" (*alogin, aregister,* etcétera), como también el nombre público del propio controlador (*ausers).*

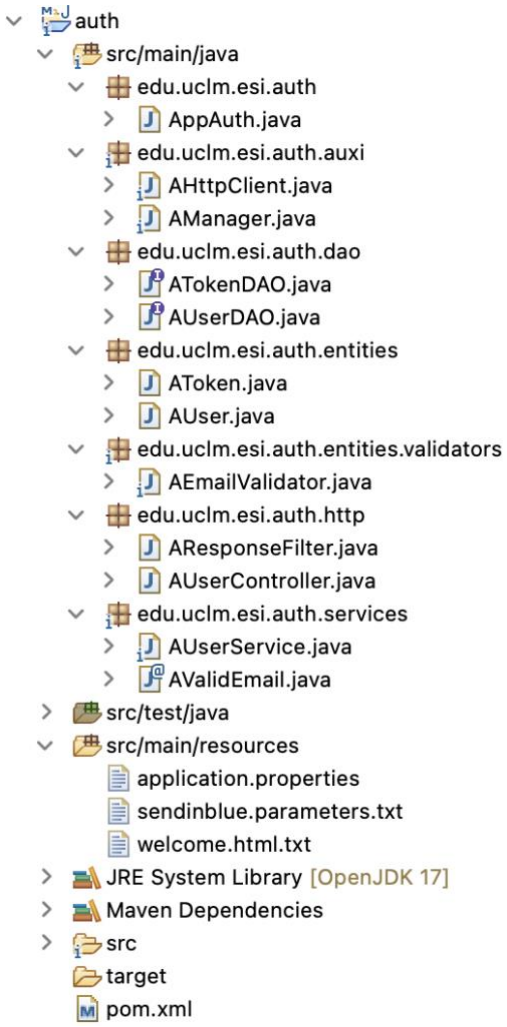


Figura 687. Ficheros del proyecto auth

3.3. Implementación de *auth*

auth se ha construido casi copiando y pegando de *games*, pero hay algunos cambios.

3.3.1. Cambios en *login*

En *AUserController* mantendremos casi la misma implementación que en *games*, pero un logueo exitoso devolverá un JSON con el *userId* y el id de la sesión *http*. Prescindiremos ya de la cabecera *current* que nos inventamos en el Capítulo 19 (página 282).

```
@PutMapping("/alogin")
public HashMap<String, Object> login(HttpSession session,
        @RequestBody Map<String, Object> info) {
   String name = info.get("name").toString().trim();
   String pwd = info.get("pwd").toString().trim();

   String userId = this.userService.login(name, pwd);
   if (userId==null)
      throw new ResponseStatusException(HttpStatus.FORBIDDEN,
            "Credenciales inválidas");
   HashMap<String, Object> result = new HashMap<>();
   result.put("userId", userId);
   result.put("httpSessionId", session.getId());
   return result;
}
```

Figura 688. Implementación del método login *en* AUserController *(servicio* ausers/alogin)

3.3.2. Cambios en *register*

Adelantamos hace unas líneas que, al cambiar de proveedor de bases de datos, el *catch* del *register* deja de ser válido, pues los códigos de error son diferentes en función del fabricante de base de datos: por ejemplo, el error 547 de SQL Server equivale al 1062 de MySQL.

Se invita al lector o lectora a que adapte este bloque *catch*.

3.3.3. Ficheros de recursos

Por otro lado, y como nos traemos aquí la comunicación con el servidor de correo, veíamos en la Figura 687 que nos hemos traído los ficheros *sendinblue.parameters.txt* y *welcome.html.txt*. El primero lo dejamos como está, pero el contenido del segundo lo debemos adaptar al puerto 8000 y a los nuevos nombres, es decir:

```
<html>
<body>
<p>
Gracias por registrarte en nuestro sistema de juegos.
</p>
<p>
Por, favor, haz clic <a href="http://localhost:8000/ausers/aconfirm/#TOKEN#">aquí</a> para
confirmar la creación de tu cuenta y poder acceder a todos nuestros servicios.
</p>
</body>
</html>
```

Figura 689. Nuevo contenido de welcome.html.txt

El contenido de los dos ficheros mencionados lo hacíamos a través del *Manager*, que también nos hemos traído a este proyecto como *AManager*, pero con muchas menos líneas de código, pues no necesitamos gestionar sesiones:

```
package edu.uclm.esi.auth.auxi;

import java.io.IOException;
import java.io.InputStream;

import org.springframework.context.annotation.Bean;
import org.springframework.stereotype.Component;

@Component
public class AManager {

    private AManager() {}

    private static class AManagerHolder {
        static AManager singleton=new AManager();
    }

    @Bean
    public static AManager get() {
        return AManagerHolder.singleton;
    }

    public String readFile(String fileName) throws IOException {
        ClassLoader classLoader = getClass().getClassLoader();
        try (InputStream fis = classLoader.getResourceAsStream(fileName)) {
          byte[] b = new byte[fis.available()];
          fis.read(b);
          return new String(b);
        }
    }
}
```

Figura 690. Código del AManager

3.3.4. Entidad *AUser*

Como se supone que este sistema de autenticación da servicio a muchos otros *backends*, no nos interesa almacenar el número de partidas pagadas que tiene el usuario sino, como apuntamos antes, el crédito que tiene disponible. Por ello, eliminamos el campo *paidMatches* y añadimos uno *credit* de tipo *Double* que iniciamos a cero. El *credit* representará el importe que el usuario tiene disponible para consumir servicios bien de juegos, bien de otro tipo de los que se puedan ofrecer desde otros *backends*.

3.3.5. Validadores por defecto

Hasta ahora, cuando un usuario se quiere registrar, se reciben sus credenciales en el recurso *register* del *UserController* y es ahí donde se comprueba, por ejemplo, que la dirección de correo electrónico tenga un formato válido (Figura 334, página 218), que las contraseñas coincidan, etcétera. También escribimos una restricción en la entidad *User* para forzar a que la columna *name* tuviese al menos 4 caracteres (Figura 388, página 247). En el propio *register* que acabamos de citar escribimos un *catch* algo enrevesado para enviar al usuario un mensaje de error adecuado al tipo de restricción que se estuviera violando en la base de datos.

Es relativamente fácil olvidar alguna comprobación que necesitemos y que se nos cuele algún dato que no deseamos. En este sentido, Spring nos ofrece al menos las anotaciones *@NotNull* y *@NotEmpty* para anotar los campos de una entidad:

1) *@NotNull* obliga a que la columna asociada al campo anotado tenga algún valor (por ejemplo, asignamos al campo *name* del *User* cinco espacios en blanco).
2) *@NotEmpty* subsume a la primera y obliga a que el campo tenga un valor distinto de vacío, lo que impedirá introducir cinco espacios en blanco como valor del nombre de usuario.

En la Figura 691 se ha comentado la restricción del campo *name* y hemos añadimos las dos anotaciones a algunos campos:

1) Para el campo *id*, que ya tiene la anotación *@Id,* ambas anotaciones son redundantes, pues *@Id* ya obliga a que la clave principal tenga valor.
2) Para el resto de campos anotados (*name, pwd* y *email)*, la primera anotación (*@NotNull)* no es necesaria, pues ya decimos que está subsumida por *@NotEmpty*.
3) Ambas anotaciones admiten un campo *message*, que es el texto que se emitirá en caso de que se intenta violar la condición. Si intentamos escribir cinco espacios en blanco para el *name*, se producirá una excepción con el texto "El nombre de usuario no puede estar vacío".
4) Obsérvese, además, que asignamos valores por defecto al campo *counter:* esto implica que, cada vez que se inserte un *User* en la tabla, esta columna se pondrá a 0, que es el valor indicado por defecto.

```java
public class AUser {
   @Id @Column(length = 36)
   @NotNull @NotEmpty
   private String id;
   //@Column(columnDefinition = "varchar(20) ...")
   @NotNull @NotEmpty(message = "El nombre de usuario no puede estar vacío")
   private String name;
   @NotNull @NotEmpty
   private String pwd;
   @NotNull @NotEmpty
   private String email;
   private Integer counter = 0;
   private Long validationDate;
   ...
```

Figura 691. Anotación de los campos en la entidad AUser

En caso de violarse, las anotaciones *@NotNull* y *@NotEmpty* lanzan una *TransactionSystemException*, la cual es una especialización de *TransactionException* que, como se ve en la Figura 692, van por un lugar diferente de las *DataAccessException* que capturábamos en el *register* de la página 247.

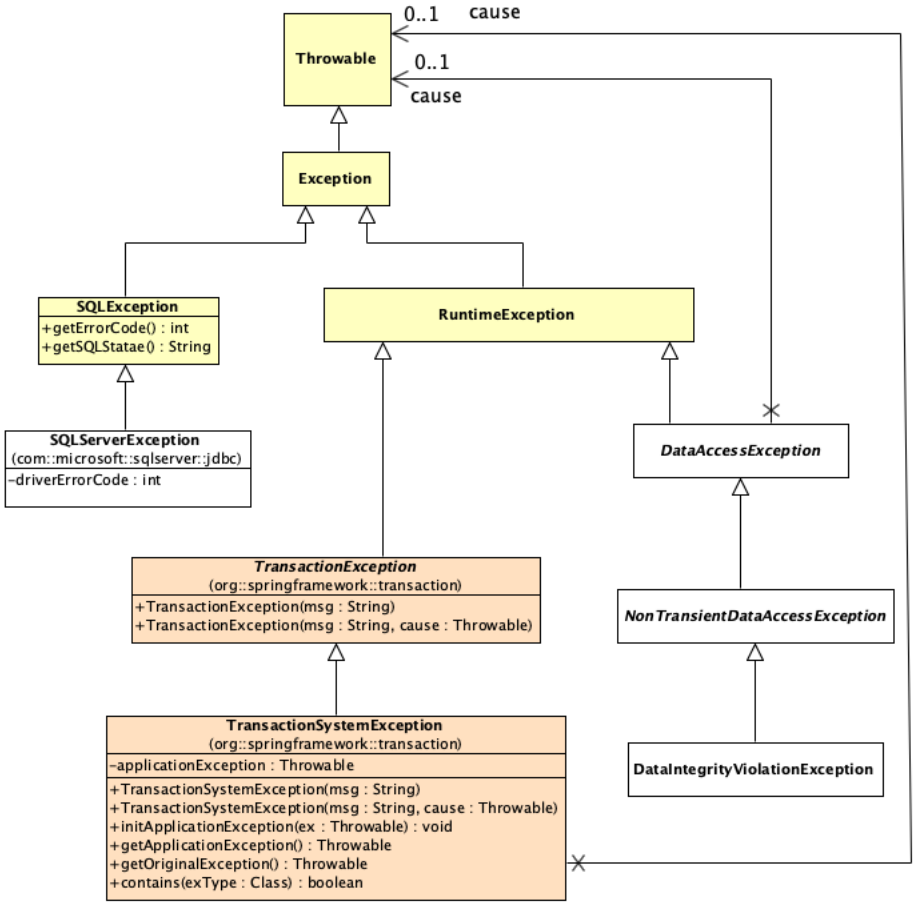


Figura 692. Excepciones por violación de las anotaciones

Necesitaríamos capturar este tipo de excepción en el *catch* del *register* para mostrar al usuario un mensaje de error adecuado.

3.3.6. Validadores personalizados

Podemos crear validadores personalizados para los campos de las entidades. Por ejemplo, en lugar de comprobar en el *AUserController* que el correo electrónico tiene un formato válido, tal vez sería preferible anotar el campo

email con *@ValidEmail*. Esta anotación (*@ValidEmail)* no existe en Spring, pero podemos crearla nosotros mismos.

Para ello, anotamos directamente el campo *email* de la entidad con *@AValidEmail*. Spring nos dirá que eso no existe y que qué queremos hacer: pues queremos crear una anotación que se llame de esa forma:

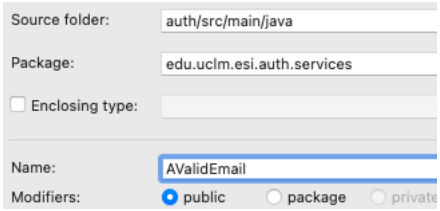


Figura 693. Vamos a crear una anotación ValidEmail

Completaremos el código de la anotación como en la Figura 694, en la que decimos:

1) Que esta anotación es válida para anotar campos (anotación *@Target).*
2) Que es válida para el tiempo de ejecución.
3) Que tiene una clase asociada que se encarga de ejecutar la validación (*AEmailValidator*), aunque de momento no está implementada.
4) Que solo implementamos el método *message,* que devuelve el mensaje de error en caso de que se viole la condición que impone esta anotación.

```
package edu.uclm.esi.juegos.entities;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

import javax.validation.Constraint;
import javax.validation.Payload;

@Target( { ElementType.FIELD })
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = AEmailValidator.class)
public @interface AValidEmail {
    String message() default "El correo electrónico no tiene un formato válido";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

Figura 694. Anotación ValidEmail

Ahora debemos implementar la clase que usará Spring para validar que los correos sean, efectivamente, correos válidos, y que es la *AEmailValidator* que se indica en la línea resaltada de la figura anterior. Como hemos delegado la validación del correo a la propia entidad, desplazamos el código que teníamos en *register* a la nueva clase, que crearemos en un subpaquete *validators* dentro de *edu.uclm.esi.auth.entities*.

Como se ve en la Figura 695, la clase implementa la interfaz *ConstraintValidator,* lo que nos obliga a parametrizar la clase que implementa la anotación (*AValidEmail)*, el tipo del objeto que hay que validar (*String)* y los métodos *initialize* e *isValid*.

```
package edu.uclm.esi.auth.entities.validators;

import java.util.regex.Matcher;
import java.util.regex.Pattern;
```

```
import javax.validation.ConstraintValidator;
import javax.validation.ConstraintValidatorContext;

import edu.uclm.esi.juegos.entities.ValidEmail;

public class AEmailValidator implements ConstraintValidator<AValidEmail, String> {
   private Pattern pattern;
   private Matcher matcher;

   private static final String EMAIL_PATTERN =
      "^[a-zA-Z0-9_!#$%&'*+/=?`{|}~^.-]+@[a-zA-Z0-9.-]+$";

   @Override
   public void initialize(AValidEmail constraintAnnotation) {
   }

   @Override
   public boolean isValid(String email, ConstraintValidatorContext context){
      return (validateEmail(email));
   }

   private boolean validateEmail(String email) {
      pattern = Pattern.compile(EMAIL_PATTERN);
      matcher = pattern.matcher(email);
      return matcher.matches();
   }
}
```

Figura 695. Implementación de la clase que valida las direcciones de correo electrónico

Si vamos ahora al código de *register* en el *AUserController*, comentamos la comprobación que hacíamos del formato de la dirección de correo e intentamos registrar a un usuario con una dirección inválida, veremos que salta una *TransactionException* de las que dibujamos en la Figura 692. Ojo, porque aquí deberíamos intervenir para enviar al usuario un mensaje adecuado al error que se ha producido.

```
@PostMapping("/register")
public void register(@RequestBody Map<String, Object> info) {
   String email = info.get("email").toString().trim();
   String pwd1 = info.get("pwd1").toString().trim();
   String pwd2 = info.get("pwd2").toString().trim();

   /*Pattern pattern = Pattern.compile("^[a-zA-Z0-9...+$");
   Matcher matcher = pattern.matcher(email);
   if (!matcher.find())
      throw new ResponseStatusException(HttpStatus.CONFLICT,
            "El correo suministrado no tiene un formato válido");*/
   ...
   String name = info.get("name").toString().trim();
   try {
      this.userService.register(name, pwd1, email);
   } catch (DataAccessException e) {
      ...
   } catch (TransactionException e) {
      throw new ResponseStatusException(HttpStatus.CONFLICT, e.getMessage());
   } catch (Exception e) {
      System.out.println(e);
   }
}
```

Figura 696. Se lanza una TransactionException *en la línea resaltada*

4. Cambios en el cliente para el control de acceso

Afortunadamente tenemos el *front-end* bien estructurado, por lo que los cambios van a ser muy pequeños:

1) En el *UserService* (Figura 697) debemos:
   a. Actualizar las urls del *endpoint* (declaramos una nueva constante *httpUrlAuth* en el fichero *commons.ts*, como ya explicamos antes de la Figura 419, página 264).
   b. Eliminar el envío de la cabera *Current,* que nos inventamos hace unas páginas para mostrar la creación y manipulación de cabeceras: en breve tendremos tiempo para aprender bien.

```
export class UserService {
  constructor(private client : HttpClient) { }

  register(user : any) : Observable<any> {
    return this.client.post<any>(httpUrlAuth + "ausers/aregister", user,
      {
        headers : new HttpHeaders({
          "content-type" : "application/json"
        }),
        observe : "body",
        withCredentials : true
      })
  }

  login(user : any) : Observable<any> {
    return this.client.put<any>(httpUrlAuth + "ausers/alogin", user,
      {
        observe : "body",
        withCredentials : true
      })
    }

  removeUser() : Observable<any> {
    return this.client.delete<any>(httpUrlAuth + "ausers/aremoveUser",
      {
        withCredentials : true
      })
  }
}
```

Figura 697. Adaptación del UserService

2) Por último, el método *login* del *LoginComponent* debe adaptarse a la respuesta recibida del servidor, que ahora devuelve dos datos (el *userId* y la *httpSessionId*) en un JSON, y que vamos a guardar en el *sessionStorage*.

```
login() {
  let user = {
    name : this.name,
    pwd : this.pwd
  }

  this.userService.login(user).subscribe({
    next : (response : any) => {
      sessionStorage.setItem("userId", response.userId)
      sessionStorage.setItem("httpSessionId", response.httpSessionId)
      this.error = undefined
      this.onUserLogged.emit(this.name)
    },
    error : (result : any) => {
      console.log(result.error.error)
      this.error = result.error.error
    }
  })
}
```

Figura 698. Cambios en el login *del* LoginComponent

5. Cambios en el sistema *games*

El antiguo sistema de juegos ya no ofrece ninguna funcionalidad de gestión de usuarios, por lo que:

1) De momento (lo recuperaremos, aunque con menos funcionalidades) podemos eliminar con tranquilidad el *UserController*.
2) Tampoco nos interesan los tokens, con lo que podemos borrar las clases *Token* y *TokenDAO*.
3) En el *UserService* basta con que dejemos el campo *userDAO* y el método *addMatches*.
4) De la carpeta de recursos podemos eliminar los ficheros de *sendinblue* y de *welcome*.
5) Respecto de los datos de los usuarios, ahora solo necesitamos saber cuántas partidas pagadas le quedan a cada jugador. Por ello, la entidad *User* puede reducirse al código de la Figura 699, si bien será importante que el *id* del *User* coincida con el *id* del *AUser* del sistema *auth*.

```
@Entity
@Table(name = "users")
public class User {
   @Id @Column(length = 36)
   private String id;
   private Integer paidMatches = 0;

   public User() {
      this.id = UUID.randomUUID().toString();
      this.paidMatches = 0;
   }

   public String getId() {
      return id;
   }

   public void setId(String id) {
      this.id = id;
   }
```

```
    public Integer getPaidMatches() {
        return paidMatches;
    }

    public void setPaidMatches(Integer paidMatches) {
        this.paidMatches = paidMatches;
    }
}
```

Figura 699. En games, *la entidad* User *necesita menos código*

6. Cambios para partidas multijugador

Tras la introducción del servidor de pagos, cuando un usuario quería jugar una partida multijugador se comprobaba si disponía de crédito suficiente (Figura 655, página 395). Si lo tenía, se establecía una conexión *ws* con el servidor de WebSockets en la que se enviaba el id de la sesión *http;* si no, se le mostraba el formulario de pagos y, cuando pagaba, se le actualizaba el crédito y el número de partidas pagadas.

Como ahora tenemos las responsabilidades distribuidas, el flujo de mensajes debe cambiar necesariamente. En la Figura 700, Pepe se loguea correctamente contra el servidor *auth,* del que obtiene, entre otros, su *userId*. En el mensaje 4 solicita jugar una partida multijugador de *Order the numbers*: como no tiene crédito, opta por comprar 50 euros: se envía el mensaje de prepago al *back* de *games,* que se encarga, a partir del mensaje 17, de gestionar la transacción: cuando *games* recibe en este mensaje la confirmación del pago debe actualizar el número de partidas pagadas por este jugador en su base de datos de SQL Server, y actualizar el crédito en la de MySQL: para esto, le enviará un mensaje *http* al servidor de *games*.

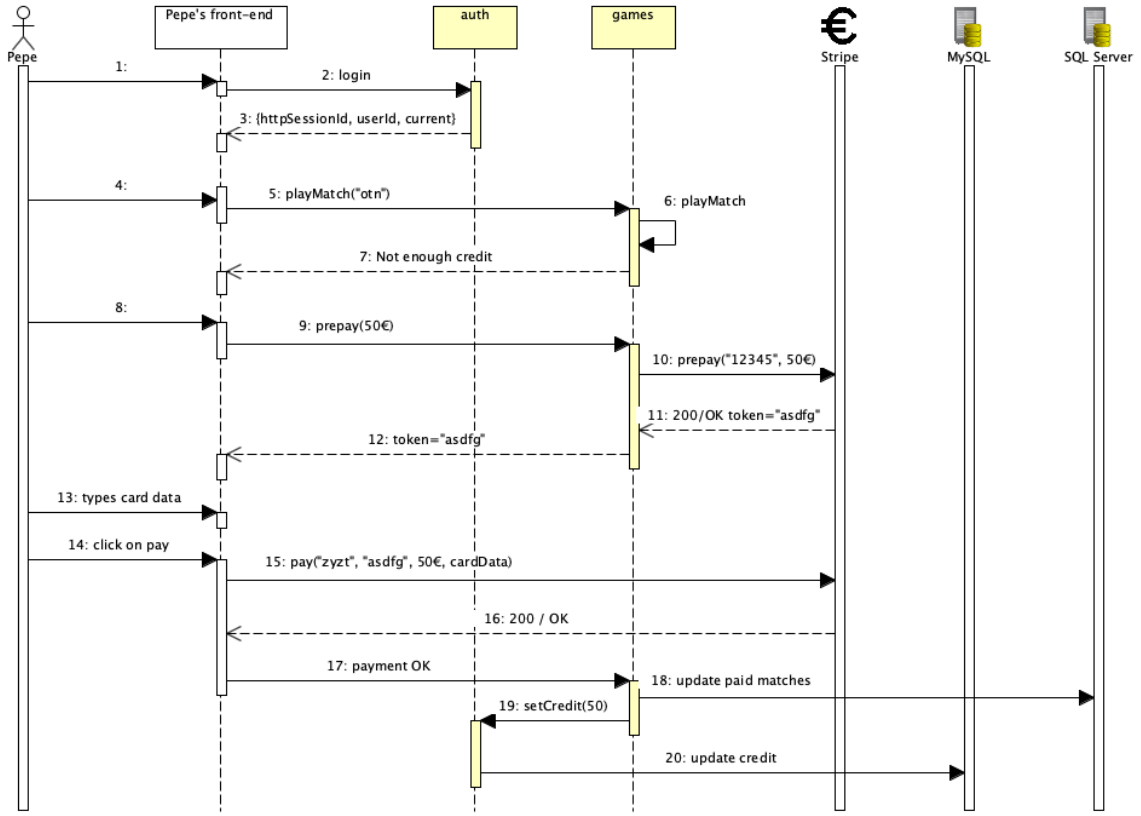


Figura 700. Flujo de mensajes para iniciar una partida multijugador sin crédito previo

En el flujo anterior, sucede que en el mensaje 5, el *GamesController* ejecuta su método *play* (cuyo código, por su interés en este punto, reproducimos nuevamente en la Figura 701).

```
public class GamesController {

   @Autowired
   private WaitingRoom waitingRoom;

   @GetMapping("/play")
   public Match playMatch(HttpSession session, @RequestParam String gameName) {
      String userId = session.getAttribute("userId").toString();
      return this.waitingRoom.playMatch(userId, gameName);
   }
}
```

Figura 701. El GamesController

6.1. Coexistencia de dos sesiones *http*

Y el flujo anterior muestra bien el comportamiento a alto nivel, pero a bajo nivel pasan cosas que no estamos considerando y que nos van a suponer nuevos cambios: en particular, la primera petición que le llegue al servidor de *games* creará una *HttpSession* con, por ejemplo, *id="1234"*. Igualmente, cuando a *auth* le llegue la petición de *login,* creará otra *HttpSession* completamente diferente, pues ambos son dos servidores distintos. Cuando llegue al *GamesController* la petición de jugar, el método *playMatch* buscará el *userId* en su *HttpSession,* que es distinta de la del servidor *auth,* por lo que encontrará *null,* petará y se interrumpirá la ejecución.

El problema se ilustra gráficamente en la Figura 702: la primera petición llega realmente a *games,* que crea una sesión *http;* cuando el usuario se loguea,

auth crea otro objeto diferente que además reside en una máquina distinta. Cuando llega el mensaje 10, no hay *userId* en la sesión en la que se busca por lo que, en lugar de devolver *Not enough credit* en el mensaje 12, se produce un error en el servidor y se devuelve una respuesta de tipo 500.

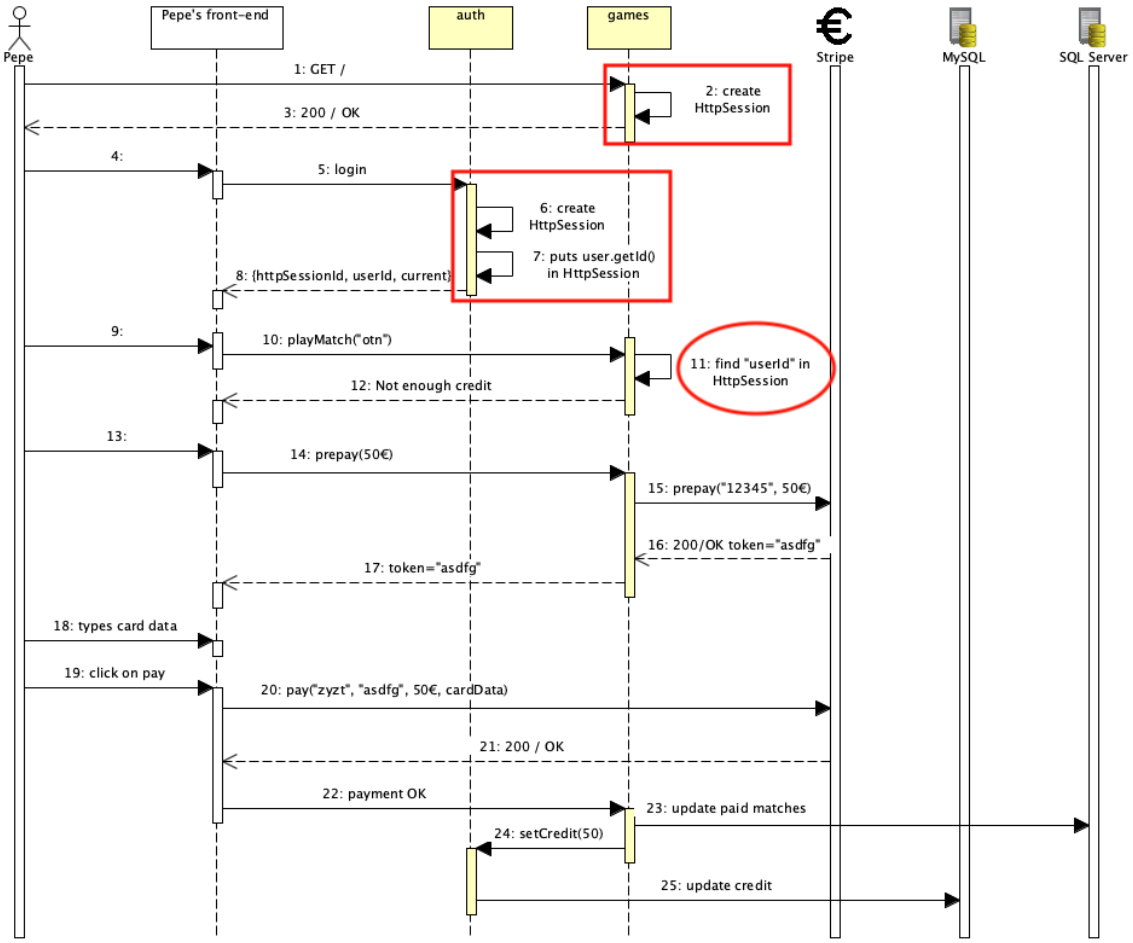


Figura 702. Flujo de mensajes que no llega a suceder

Una posible solución pasa por que el mensaje 10 lleve un parámetro adicional con el *userId*, que el *front-end* tiene almacenado en el *sessionStorage* después de que el usuario se ha logueado, que es lo que hace el *GamesService* del cliente en la parte de arriba de la Figura 703. En la de abajo, el controlador lee ese *userId,* coloca el *userId* en la sesión, crea la partida si puede (si tiene crédito) y la devuelve. Además, es importante que añadamos la sesión a la lista de sesiones que mantiene el *Manager* para que podamos ligar cada conexión *http* con su *ws* correspondiente: antes, esto lo hacíamos en el método *login*; ahora lo debemos hacer en este momento.

```
export class GamesService {

  constructor(private client : HttpClient) { }

  playMatch(gameName : String) : Observable<any> {
    return this.client.get<any>(httpUrl +
      "games/play?gameName=" + gameName +
      "&userId=" + sessionStorage.getItem("userId"),
      {
        withCredentials : true,
        observe : "response"
      }
```

```
    )
  }
}
```

```
public class GamesController {

   @Autowired
   private WaitingRoom waitingRoom;

   @GetMapping("/play")
   public Match playMatch(HttpSession session,
           @RequestParam String gameName, @RequestParam String userId) {

      session.setAttribute("userId", userId);
      Match match = this.waitingRoom.playMatch(userId, gameName);
      Manager.get().addSessionByUserId(userId, session);
      return match;
   }
}
```

Figura 703. Envío y recepción del userId *como un parámetro*

La solución anterior será válida siempre y cuando el usuario con ese *userId* sea reconocido por el sistema *games:* ahora, el registro del usuario se hace en *auth,* pero *games* no sabe nada de él. Es necesario por tanto que, cuando un usuario se registre mediante *auth,* este comunique a *games* que el usuario se ha registrado. Es decir, que el proceso de registro podría ser realmente como en la Figura 704, en la que *auth,* tras almacenar los datos del usuario en su base de datos, le dice a *games* que lo inserte en la suya. Se han omitido mensajes que no vienen al caso en este escenario, como el envío del correo electrónico para confirmar la creación de la cuenta, o los que circulan entre controladores, servicios, etcétera. Alguna lectora o lector sagaz puede pensar que ese mensaje 5 podría enviarse solo cuando el usuario haya confirmado la creación de la cuenta, y lleva razón, pues es otra posibilidad, pero lo dejamos expresamente así para aprender cosas nuevas.

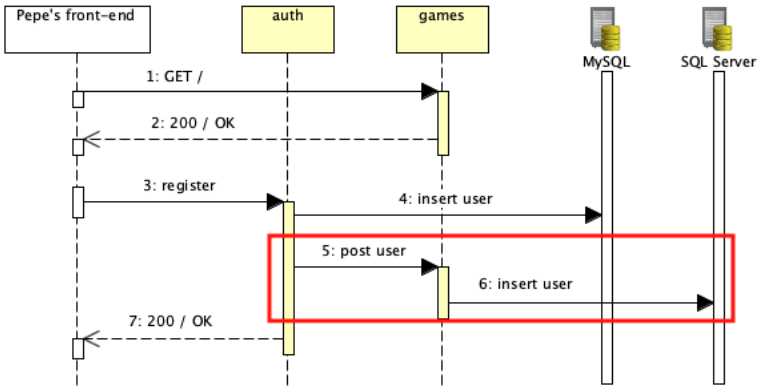


Figura 704. Cambio en el proceso de registro

Bueno, pues el hecho de que *games* pueda ahora recibir esa petición del mensaje 5 que hemos etiquetado como *post user* implica que *games* vuelva a tener un *UserController* para recibir la petición desde *auth*. Por una vez, y supondremos que con el permiso de nuestro jefe de proyecto, accederemos directamente al *UserDAO* desde el controlador sin pasar por un servicio (Figura 705): una novedad de este recurso web es que el método *insertUser* recibe como *payload* un objeto completo de clase *User* (hasta ahora en los *@RequestBody* solo recibíamos mapas inyectados desde objetos JSON).

```
@RestController
@RequestMapping("users")
public class UserController {

   @Autowired
   private UserDAO userDAO;

   @PostMapping("/insertUser")
   public void insertUser(@RequestBody User user) {
      this.userDAO.save(user);
   }
}
```

Figura 705. Recuperamos el UserController *en* games

Para que la nueva inyección del *User* al método *insertUser* funcione, el servidor *auth* debe enviar un JSON que se adapte a la especificación de *User*.

```
@Service
public class AUserService {

   ...
   @Transactional
   public void register(String name, String pwd, String email) throws IOException {
      AUser user = new AUser();
      user.setName(name);
      user.setPwd(pwd);
      user.setEmail(email);

      AToken token = new AToken();
      token.setUser(user);
      this.userDAO.save(user);
      this.tokenDAO.save(token);
      this.sendEmail(email, name, token.getId());

      AHttpClient client = new AHttpClient();
      JSONObject jsoUser = new JSONObject().
         put("id", user.getId());
      client.sendPost("http://localhost/users/insertUser",
         "content-type : application/json", jsoUser);
   }
   ...
```

```
@RestController
@RequestMapping("users")
public class UserController {

   @Autowired
   private UserDAO userDAO;

   @PostMapping("/insertUser")
   public void insertUser(@RequestBody User user) {
      this.userDAO.save(user);
   }
}
```

Figura 706. Modificación del AUserService *y creación del nuevo* UserController

6.2. Adaptación del flujo de pago

Ya que no tenemos el problema de las sesiones, el flujo de mensajes que mostrábamos en la Figura 700 es ahora válido. Simplemente, y para ser un poquito más precisos y describir mejor los cambios necesarios, reproducimos a continuación la misma figura, pero resaltamos los tres lugares en los que se da al *userId* un tratamiento algo distinguido:

1) En la respuesta a *login* (realmente, al recurso *alogin),* el servidor devuelve un JSON con el *userId* de Pepe ("1234", por ejemplo).

2) Cuando Pepe solicita la partida, envía como parámetros el nombre del juego y el *userId*, que el cliente mantiene almacenado en el *sessionStorage*.
3) El flujo de pago continúa de forma normal, pero cuando a *games* le llega la confirmación del pago en el mensaje 16, *games* debe actualizar el número de partidas pagadas en su base de datos, y decirle a *auth* que incremente el crédito del usuario "1234" en el importe que corresponda.

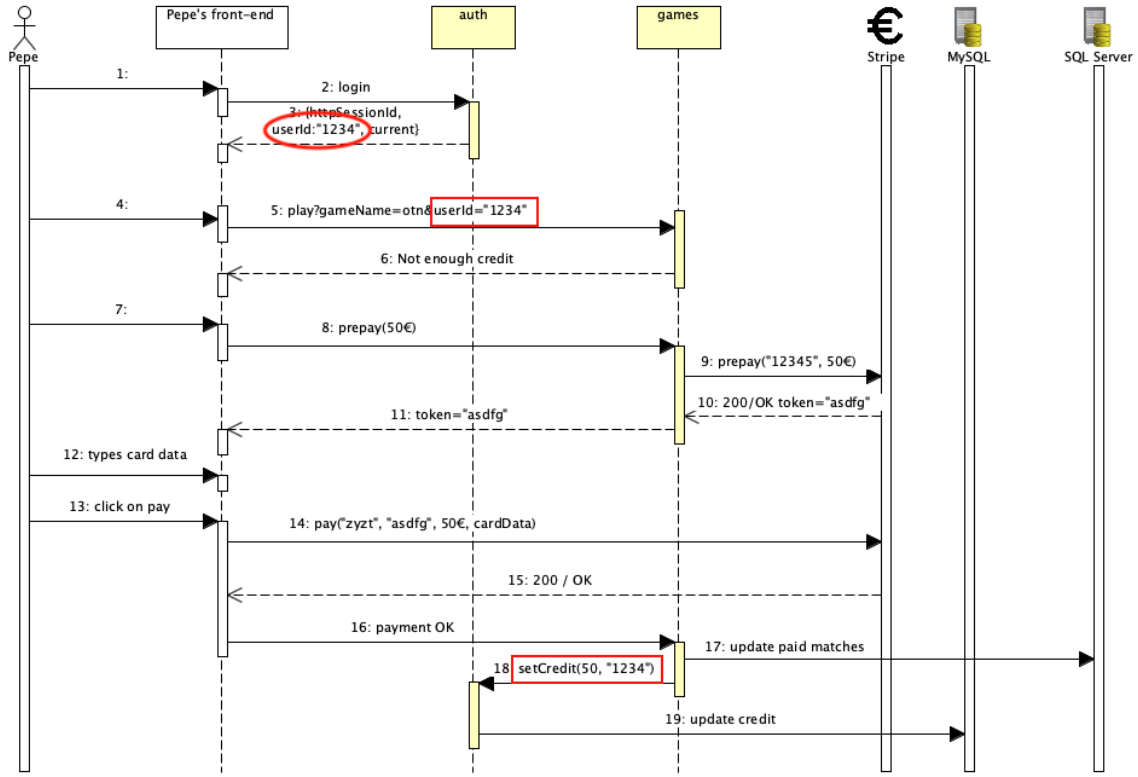


Figura 707. Flujo de mensajes modificado para iniciar una partida multijugador sin crédito previo

Para que *auth* dé respuesta al mensaje 18, debe incluir un servicio *setCredit* o similar que reciba al menos el importe y el *id* del usuario. Añadiremos este servicio como un PUT al *AUserController* (Figura 708, arriba), que delega su ejecución real al *AUserService* (Figura 708, abajo).

```
@PutMapping("/setCredit")
public void setCredit(@RequestBody Map<String, Object> info) {
   String userId = info.get("userId").toString();
   double amount = Double.parseDouble(info.get("amount").toString());
   this.userService.setCredit(userId, amount);
}
```

```
public void setCredit(String userId, double amount) {
   AUser user = this.userDAO.findById(userId).get();
   user.setCredit(user.getCredit() + amount);
   this.userDAO.save(user);
}
```

Figura 708. Código para incrementar el crédito del usuario en el AUserController *y en el* AUserService

Y, para que *games* pueda enviar esa petición, debe enviarla desde algún lugar. Lo haremos desde el *PaymentsController*, al que le añadimos las líneas resaltadas de la siguiente figura:

```
public String prepay(HttpSession session, @RequestParam int matches) {
   try {
      ...

      PaymentIntent intent = PaymentIntent.create(params);
      ...

      HttpClient client = new HttpClient();
      JSONObject jsoCredit = new JSONObject()
            .put("userId", session.getAttribute("userId"))
            .put("amount", total/100);
      client.sendPut("http://localhost:8000/ausers/setCredit",
            "content-type : application/json", jsoCredit);

      return clientSecret;
   } catch (Exception e) {
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e.getMessage());
   }
}
```

Figura 709. Modificación del PaymentsController

Pero el *HttpClient* que tenemos (Figura 641, página 386) no dispone de método *sendPut*, por lo que vamos a crearlo, pero utilizando programación reflexiva (recuérdese la sección 3 del Capítulo 20, en la página 293). Como enviar un POST o un PUT es prácticamente, en lugar de copiar *sendPost,* pegarlo como *sendPut* y hacerle algún cambio, escribiremos un único método *send* que pueda enviar ambos tipos de peticiones.

6.2.1. Creación de un método *send* genérico usando reflexión

Sin utilizar reflexión, y a primera vista, tal método *send* podría tener la siguiente cabecera y empezar con las líneas que mostramos en la Figura 710, en la que pasamos como parámetro el nombre del método y, en función de su valor, creamos la *request* de uno u otro tipo.

Tal objeto debe ser declarado en la línea con las tres interrogaciones.

```
public String send(String method, String url, JSONArray headers, JSONObject payload) {
   try(CloseableHttpClient client = HttpClientBuilder.create().build()) {
      ???
      if (method.equalsIgnoreCase("post"))
         request = new HttpPost(url);
      else
         request = new HttpPut(url);

   }
}
```

Figura 710. Intento de implementación de un send *genérico*

Pero ¿de qué tipo declararíamos ese objeto *request*? Para saberlo podemos buscar la API de los objetos *HttpPost* o *HttpPut,* o bien hacer clic sobre uno de los dos tipos con las teclas *Control* o *Cmd* (en Mac) pulsadas. En la Figura 711 adivinamos que la superclase de ambos subtipos es *HttpEntityEnclosingRequestBase,* por lo que las tres interrogaciones se podrían sustituir por una declaración: *HttpEntityEnclosingRequestBase request*.

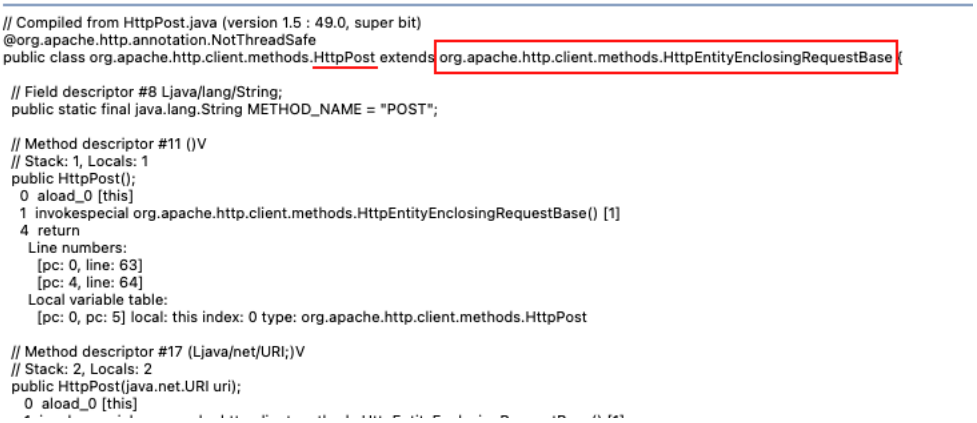


Figura 711. Adivinamos el supertipo de HttpPost

Pero queremos usar programación reflexiva, por lo que no vamos a pasar el nombre del método como parámetro, sino el tipo del propio método.

A bote pronto, la cabecera del nuevo método *send* podría ser la siguiente, en la que pasamos como primer parámetro la clase (*HttpPost.class* o *HttpPut.class*).

```
public String send(Class clazz, String url, JSONArray headers, JSONObject payload)
```

Sin embargo, como sabemos que el tipo ha de ser una especialización de *HttpEntityEnclosingRequestBase,* pasaremos *Class<? extends HttpEntityEnclosingRequestBase> clazz* como primer parámetro.

Cuando creamos de forma normal (es decir, sin reflexión) un objeto de tipo *HttpPost,* le pasamos la *url* como parámetro. Es decir, invocamos a su constructor con una sentencia como *new HttpPost(url)*. Sucede que *HttpPost* tiene varios constructores, pero solo uno que tome una *String* como parámetro: este es, precisamente, el constructor que nos interesa, por lo que lo localizaremos e invocaremos pasándole al argumento *url* como parámetro. En el fragmento resaltado de la Figura 712 localizamos, de la clase que nos pasan como parámetro, aquel constructor que reciba solamente una *String* como argumento. En el resto de la línea (*newInstance(url)*), lo invocamos con el valor de *url* que hayamos recibido.

```
public String send(Class<? extends HttpEntityEnclosingRequestBase> clazz,
        String url, JSONArray headers, JSONObject payload) {

   try(CloseableHttpClient client = HttpClientBuilder.create().build()) {
      try {
         HttpEntityEnclosingRequestBase request =
            clazz.getConstructor(String.class).newInstance(url);
         ...
}
```

Figura 712. Primeras líneas de send

Ahora, la llamada a *sendPut* que hacíamos en el *PaymentsController* la podemos sustituir por:

```
public String prepay(HttpSession session, @RequestParam int matches) {
   try {
      ...
      client.send(HttpPut.class, "http://localhost:8000/ausers/setCredit",
            "content-type : application/json", jsoCredit);

      return clientSecret;
   } catch (Exception e) {
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e.getMessage());
   }
}
```

Figura 713. Pequeño cambio en el PaymentsController

Pero el método *send* que acabamos crear no cuadra con esa llamada que estamos haciendo en la Figura 712, pues el tercer parámetro que estamos pasando es una cabecera de tipo *String,* con valor *content-type : application/json*, pero el parámetro correspondiente en el método es un JSONArray con cadenas que contengan cabeceras.

Realmente, el *payload* que enviamos (al menos hasta ahora) es siempre un JSON, por lo que va de suyo que se añada siempre esa cabecera. Otras cabeceras que queramos añadir pueden ir en un JSONArray o en una lista de cadenas, por lo que vamos a crear una versión nueva del método.

En la Figura 714 hemos colocado el *payload* como tercer parámetro. El cuarto es una lista cadenas que se coloca en un array *headers*. Este método termina invocando a la otra implementación de *send*.

```
public String send(Class<? extends HttpEntityEnclosingRequestBase> clazz,
         String url, JSONObject payload, String... headers) {
   JSONArray jsaHeaders = new JSONArray();
   for (String header : headers)
      jsaHeaders.put(header);
   return this.send(clazz, url, payload, jsaHeaders);
}
```

Figura 714. Una de las implementaciones de send

El código completo del *send* reflexivo se muestra en la Figura 715: es muy parecido al *sendPost* que teníamos en la Figura 641 de la página 386, pero ahora construye reflexivamente la petición y añade siempre la cabecera *content-type* con el valor fijo *application/json*.

```
public String send(Class<? extends HttpEntityEnclosingRequestBase> clazz, String url,
JSONObject payload, JSONArray headers) {
   try(CloseableHttpClient client = HttpClientBuilder.create().build()) {
      try {
         HttpEntityEnclosingRequestBase request =
                        clazz.getConstructor(String.class).newInstance(url);
         HttpEntity entity = new StringEntity(payload.toString());
         request.setEntity(entity);
         request.setHeader("content-type", "application/json");
         if (headers!=null)
            for (int i=0; i<headers.length(); i++) {
               String header = headers.getString(i);
               String headerName = header.substring(0, header.indexOf(':')).trim();
               String headerValue = header.substring(header.indexOf(':')+1).trim();
               request.setHeader(headerName, headerValue);
            }

         CloseableHttpResponse response = client.execute(request);
         int code = response.getStatusLine().getStatusCode();
         if (response.getStatusLine().getStatusCode()!=200) {
            HttpStatus status = HttpStatus.resolve(code);
            String errorMessage = response.getStatusLine().getReasonPhrase();
            throw new ResponseStatusException(status, errorMessage);
         }
```

```
            entity = response.getEntity();
            String responseText = EntityUtils.toString(entity);
            return responseText;
         } catch (Exception e) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e.getMessage());
         }
      } catch (IOException e1) {
         throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e1.getMessage());
      }
   }
```

Figura 715. El método set *recibe un parámetro de tipo* Class

Ahora, las dos posibles llamadas a *send* que hacemos desde el *prepay* del *PaymentsController* (Figura 716) compilan y funcionan:

1) En la de arriba, enviamos explícitamente la cabecera *content-type*.
2) En la de abajo pasamos de ponerla, pues sabemos que la nueva implementación de *send* la va a añadir.

```
public String prepay(HttpSession session, @RequestParam int matches) {
   try {
      ...
      client.send(HttpPut.class, "http://localhost:8000/ausers/setCredit",
            jsoCredit, "content-type : application/json");

      return clientSecret;
   } catch (Exception e) {
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e.getMessage());
   }
}
```

```
public String prepay(HttpSession session, @RequestParam int matches) {
   try {
      ...
      client.send(HttpPut.class, "http://localhost:8000/ausers/setCredit",
            jsoCredit);

      return clientSecret;
   } catch (Exception e) {
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e.getMessage());
   }
}
```

Figura 716. Pequeño cambio en el PaymentsController

6.3. Ejercicio

En la implementación que hemos dado al flujo de pagos anterior hay un problema importantísimo, pues estamos incrementando el crédito sin que la transacción económica haya llegado a validarse.

¿Detecta el lector o lectora el problema?

1) Si es que sí, corríjalo.
2) Si es que no, dele la vuelta a la página, entiéndalo y corríjalo.

El problema se encuentra en que estamos enviando la petición a *setCredit* en el *prepay*, no en el *confirm* (que es el que se ejecuta cuando al cliente le ha llegado desde Stripe la confirmación del pago). Así que estamos incrementando el crédito del jugador antes de que el pago se realice de manera efectiva.

Figura 717. Solución al error encontrado

6.4. Nuevos problemas para iniciar partida

Al haber cambiado al nuevo proveedor de autorizaciones, la llamada que hacemos desde el cliente para establecer la conexión con el WebSocket envía el

httpSessionId que nos devolvió *auth*; pero, como ya vimos en la Figura 702, la sesión de *auth* no tiene nada que ver con la de *games,* por lo que el WebSocket de *games* falla al ejecutar *afterConnectionEstablished* (Figura 718).

```
private buildWebSocket(matchId : string) : WebSocket {
    let ws = new WebSocket(wsUrl + "wsGames?httpSessionId=" +
         sessionStorage.getItem("httpSessionId"))
    ws.onopen = function() {
    ...
```

```
@Override
public void afterConnectionEstablished(WebSocketSession session) throws Exception {
   String query = session.getUri().getQuery();
   String httpSessionId = query.substring("httpSessionId=".length());
   Manager.get().setWebsocketSession(httpSessionId, session);
   ...
```

Figura 718. El cliente envía al servidor un httpSessionId *equivocado*

Para solucionarlo, viene bien que repasamos cómo se establece una conexión *ws* (sección 4 del Capítulo 21, página 302): recordemos que se inicia con una petición *http* que, claro, lleva las cabeceras habituales, entre ellas el *id* de la *HttpSession*.

Bien, pues el objeto de tipo *WebSocketSession* que se nos inyecta en los métodos del *WebSocket* del servidor posee una operación *getHandshakeHeaders,* que devuelve la lista de las cabeceras incluidas en la petición *http* que inició la conexión. Entre estas cabeceras están las cookies, y entre estas cookies está la JSESSIONID. De este modo, en lugar de enviar como un parámetro el *id* de la sesión *http* en la url de conexión al WebSocket, será este quien busque el JSESSIONID entre sus cabeceras.

Así, buscaremos el *id* de la sesión *http* directamente en las cabeceras en el mismo momento en que se crea la conexión *ws*. Esto es lo que hacemos en la Figura 719: leemos las cabeceras, seleccionamos aquella cuyo nombre es COOKIE y llamamos a un método *find* que busca en la llamada JSESSIONID y cuyo valor tiene 32 caracteres de longitud.

```
@Override
public void afterConnectionEstablished(WebSocketSession session) throws Exception {
   HttpHeaders headers = session.getHandshakeHeaders();
   List<String> headerValues = headers.get("COOKIE");

   String httpSessionId = this.find(headerValues, "JSESSIONID", 32);
   Manager.get().setWebsocketSession(httpSessionId, session);
}
```

Figura 719. Recuperamos el JSESSIONID de las cabeceras de la petición http

Si colocamos un punto de interrupción en la línea resaltada de la figura anterior, veremos todas las cabeceras. Cada valor contiene una lista de cadenas: el que ocupa la décima posición de esa colección contiene la cabecera COOKIE Si el cliente enviase más de una cookie, la lista contendría una entrada por cada una.

Figura 720. Inspección de las cabeceras

El método *find* itera sobre los *headerValues,* buscando una cookie que se llame *cookieName* (JSESSIONID en nuestro caso) y cuyo valor (ya lo sabemos) tiene siempre 32 caracteres de longitud, que pasamos a *valueLength*.

```
private String find(List<String> headerValues, String cookieName, int valueLength) {
   String cookieValue = null;
   int posCookieValue;
   for (String headerValue : headerValues) {
      if (headerValue.startsWith(cookieName)) {
         posCookieValue = headerValue.indexOf(cookieName) + cookieName.length() + 1;
         cookieValue = headerValue.substring(posCookieValue,
            posCookieValue + valueLength);
         break;
      }
   }
   return cookieValue;
}
```

Figura 721. Búsqueda del valor de una cookie

Capítulo 28. Introducción de una capa de seguridad

1. Un usuario desleal

Nuestro querido usuario Pepe (que nos viene acompañando durante estas 441 páginas) nos ha defraudado, pues ha descubierto, no sabemos cómo, que nuestro servidor *games* envía (Figura 722) el mensaje *setCredit* a *auth* para incrementar el crédito disponible mediante el servicio PUT cuyo código mostrábamos en la Figura 708. Como vemos en la siguiente figura, Pepe envía la petición con 50 euros y su *id* de usuario ("1234" en el ejemplo).

Figura 722. Un fragmento de la Figura 707

Y, claro, de vez en cuando envía directamente peticiones mediante un comando CURL para incrementar su crédito (Figura 723).

id	counter	credit	email	name	pwd
f513982a-c496-4bb0-9f6c-043273e494d8	0	0	pepe@pepe.com	Pepe	315d8

```
curl -d '{ "userId" : "f513982a-c496-4bb0-9f6c-043273e494d8", "amount" : 50 }'
-H "Content-type: application/json" -X PUT http://localhost:8000/ausers/setCredit
```

id	counter	credit	email	name	pwd
f513982a-c496-4bb0-9f6c-043273e494d8	0	50	macario.polo@gmail.com	Pepe	315d8

Figura 723. Pepe se incrementa el crédito directamente

Podríamos proteger un poco más el servicio *setCredit,* obligando por ejemplo a que todas las peticiones llevasen una cabecera *origin* con uno de nuestros orígenes autorizados, que podríamos tener en alguna tabla de la base de datos. Pero Pepe podría fácilmente *añadir -H "origin:http://localhost"* a su CURL.

Un posible mecanismo de protección pasa por modificar el flujo final de mensajes y dejarlo, por ejemplo, como en la Figura 724:

1) Al recibir el mensaje de que el pago ha sido correcto, se actualiza la base de datos de *games* y se genera un token de seguridad aleatorio ("5678" en este escenario) que se envía al servidor *auth*.
2) Este, antes de actualizar el crédito en su base de datos (la MySQL), pregunta a *games* que si el token recibido es válido.
    a. Si *games* le dice que sí, entonces actualiza el crédito.
    b. Si no, devolverá algún tipo de error y Pepe se quedará como estaba antes.

    Obviamente, el *securityToken* puede tener mecanismos de seguridad adicionales, como una validez máxima, o generar el token aleatorio de acuerdo a alguna fórmula matemática que deban cumplir, por ejemplo, las posiciones en el alfabeto de las letras que contenga.

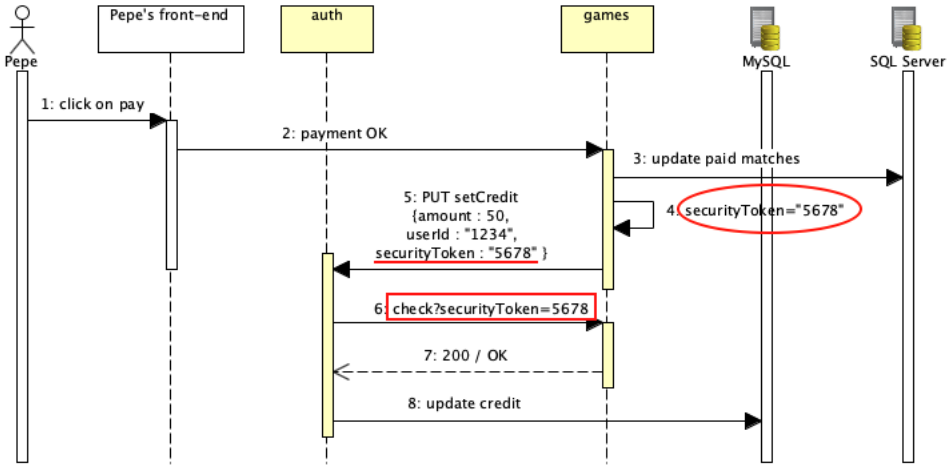


Figura 724. Adición de un mecanismo adicional de autenticación de transacciones

Además de la inclusión de ese *securityToken*, podríamos preguntar si la *ip* que envía la petición (accesible mediante el método *getRemoteAddr()* del objeto *HttpServletRequet,* que podemos incluir como parámetro) es una de nuestras direcciones autorizadas. La diferencia con respecto a la cabera *origin* es importante, pues esta solo dice «Oye, vengo de parte de *localhost:80* y quiero que me actualices el crédito», mientras que *getRemoteAddr* comprueba de forma efectiva que la petición viene de quien dice venir.

De acuerdo con mi amigo y compañero Luis Enrique Sánchez Crespo[12], experto en ciberseguridad, cuanto más raros y personalizados sea los mecanismos de seguridad que incluyamos, cuanto más alejados estén del uso de *frameworks* estándares, más difícil será que nos hackeen. Por esto, insistimos, es importante ocultar al cliente la tecnología que se está utilizando en el servidor: si el atacante sabe que usamos Spring, puede buscar vulnerabilidades conocidas en el CVE (*Common Vulnerabilities and Exposures*[13]) y tratar de explotarlas.

2. Protección de la base de datos

No obstante lo dicho, también es fácil que, por querer huir de estándares, dejemos brechas abiertas en nuestro sistema. Por ejemplo, si exhibimos al *front-end* un error de tipo 500, tal vez un usuario sepa que estamos utilizando MySQL para autenticar, y es posible que conozca o busque en el CVE vulnerabilidades de esta base de datos y que acceda a ella sin autorización.

Además de restringir los permisos (ya lo comentamos en la página 417), podemos limitar las máquinas con acceso a nuestro servidor de MySQL.

Por ejemplo, en el momento de escribir estas líneas, el router le está dando al equipo del autor la dirección 192.168.0.19, y todos los servidores que se están utilizando (el de Angular en el 4200, el de *games* en el 80, el de *auth* en el 8000, el de *SQL Server* en el docker y el de *MySQL* en el 3306) escuchan en dicha dirección.

Supongamos que el servidor *auth* esté corriendo en una máquina diferente de la misma red local cuya *ip* es 192.168.0.20. En MySQL (y en SQL Server, y en cualquier gestor de bases de datos) podemos especificar las direcciones *ip* autorizadas desde las que se realizan las conexiones (Figura 725). Como el

[12] https://dblp.org/pid/78/6763.html
[13] https://cve.mitre.org

usuario que accede a la base de datos es *sa*, y este solo puede correr en la máquina que hay en 192.168.0.20, podríamos limitar sus permisos para que a *sa* solo se le admitan conexiones desde esa *ip:*

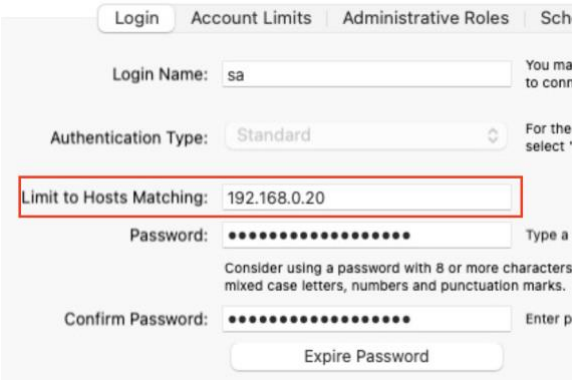


Figura 725. Limitación de los hosts que pueden acceder a la base de datos (hay que reiniciar el servidor tras aplicar el cambio).

Otras formas muy básicas, pero muy efectivas, de proteger las bases de datos, consisten en ponerlas a escuchar en puertos distintos de aquellos por defecto (1433 en SQL Server, 3306 en MySQL), y en sustituir los nombres de los superusuarios que vienen por defecto (*sa* en SQL Server, *root* en MySQL).

3. Introducción a Spring Security

Pero Luis Enrique también aconseja el uso de *frameworks* específicos de seguridad, aparte de los mecanismos personalizados que podamos aplicar.

La versión de Spring Boot que venimos utilizando puede incluir, si así la añadimos al *pom.xml*, la versión 6.0.1 de Spring Security (recuérdese el Cuadro II, página 87, en el que explicábamos cómo Spring Boot se trae las dependencias al repositorio local).

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

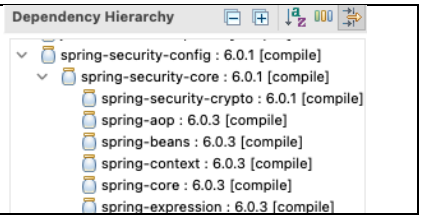


Figura 726. Spring Boot 3.0.1 incluye Spring Security 6.0.1

Si añadimos al proyecto *auth* la dependencia citada en la figura anterior y lo arrancamos, veremos que por la consola de Eclipse aparece el mensaje de la Figura 727. Es decir, simplemente por haber añadido la dependencia, Spring protege todas las peticiones que reciba el servidor mediante un nombre de usuario (por defecto, "user") y esa contraseña aleatoria.

```
Using generated security password: 99572063-6350-42e2-8daa-0d833af22b1f

This generated password is for development use only. Your security configu

2023-01-17T21:41:21.311+01:00  INFO 2956 --- [  restartedMain] o.s.s.web.D
```

Figura 727. Una contraseña aleatoria para el usuario user

Sin haber tomado ninguna otra medida como las citadas al principio de este capítulo (token de seguridad o limitación del acceso a la base de datos), las peticiones que envíe Pepe mediante CURL le dejarán ahora de funcionar, pues Spring ha introducido una *DefaultSecurityFilterChain,* una cadena con una serie de filtros de seguridad, entre los que hay uno que espera que todas las peticiones vengan autenticadas.

3.1. Cabecera *Authorization : BASIC*

Si no decimos otra cosa, la autenticación que se espera es la básica. Ahora, todas las peticiones que se reciban en *auth* deben llevar una cabecera *Authorization* cuyo valor necesariamente ha de ser la cadena formada por la pabra "BASIC" seguida del nombre del usuario ("user"), dos puntos (":") y la *security password* anterior, como se muestra en la fila de arriba de la Figura 728.

Realmente, lo que se acaba de decir es mentira, porque la cadena *user: 99572063-6350-42e2-8daa-0d833af22b1f* debe ir codificada en base64, como en la fila de abajo. Ya sabemos que *base64* es un formato desencriptable, por lo que el servidor, cuando reciba el valor *dXNl...,* podrá fácilmente obtener que el *user* es "user" y que la contraseña es "99752...".

```
BASIC user:99572063-6350-42e2-8daa-0d833af22b1f
```

```
BASIC dXNlcjo5OTU3MjA2My02MzUwLTQyZTItOGRhYS0wZDgzM2FmMjJiMWY=
```

Figura 728. Valor de la cabecera Authorization

La simple inclusión de la dependencia nos crea un recurso GET llamado *login*, al que podemos acceder con nuestro navegador mediante, en el caso del proyecto *auth,* la url *http://localhost:8000/login* (Figura 729).

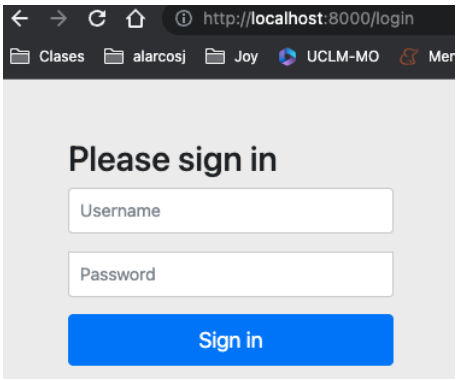


Figura 729. Spring Security crea este recurso html automáticamente

En este punto, el formulario mostrado puede superarse si se escriben *user* y la contraseña aleatoria. Si Pepe intenta intentar con sus credenciales habituales (Pepe y pepe1234), Spring le muestra un mensaje de error (Figura 730, izquierda); si introduce las buenas (*user* en el campo *Username* y la contraseña aleatoria), se lo reconduce a una página de error diferente pero que, en este caso, indica que las credenciales se han validado correctamente (Figura 730, derecha).

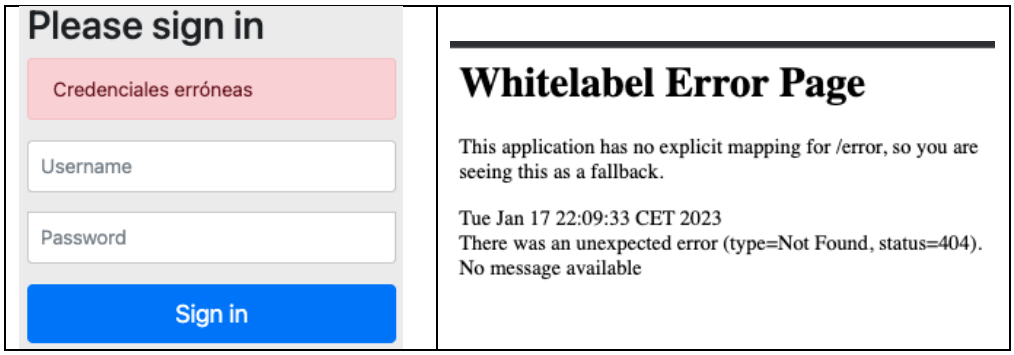


Figura 730. Validación de las credenciales por defecto

A modo de prueba, modifiquemos el nombre y la contraseña que Spring Security usa por defecto: para ello, añadimos el par de líneas resaltado al fichero *application.properties* de *auth* (Figura 731). Ahora, Pepe solo alcanzará el lado derecho de la figura anterior si sabe que el nombre de usuario es *admin* y que la contraseña es *Administrador1234*.

```
server.port = 8000
spring.datasource.url = jdbc:mysql://localhost:3306/authlibro?serverTimezone=UTC
...
spring.security.user.name = admin
spring.security.user.password = Administrador1234
```

Figura 731. Modificación del usuario y de la contraseña por defecto

3.2. Envío de peticiones con *Postman*

Postman es una herramienta que, entre otras muchas cosas, nos permite enviar cómodamente peticiones y recuperar los resultados. Vamos a usar Postman para enviar una petición muy sencillita (podríamos crearla fácilmente con CURL) al recurso *alogin* que, recordemos, admite peticiones PUT. En la siguiente figura se muestra el primer paso para crear la petición: método, url y algunas cabeceras.

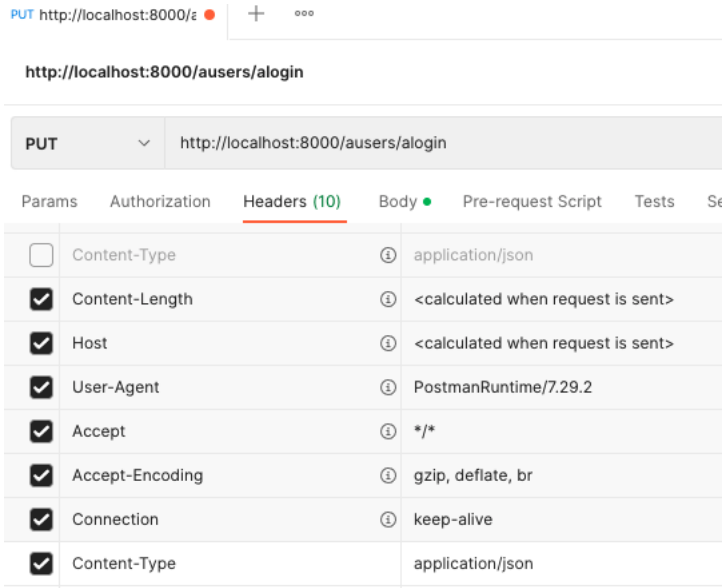


Figura 732. Creación de una petición en Postman (I)

En la solapa *Body* colocamos el *payload* de la petición:

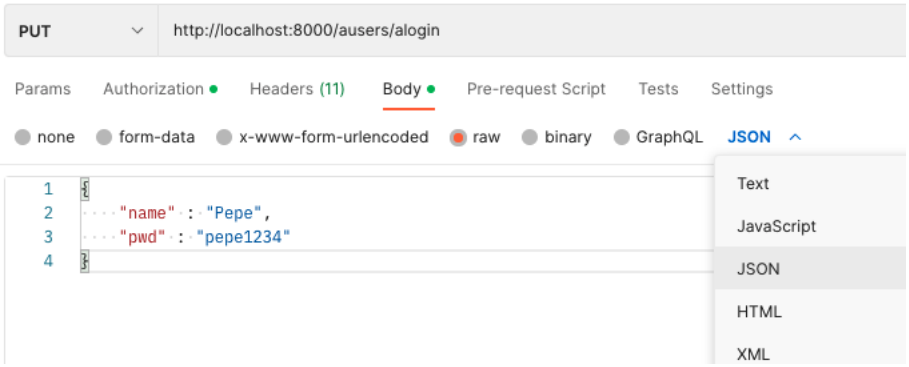


Figura 733. Creación de una petición en Postman (II)

Con el botón *Send* la petición viaja y, cuando llega la respuesta, se muestra en la parte inferior. Como vemos, se obtiene el error 401 que nos lo están devolviendo los interceptores o filtros que ha añadido Spring Security.

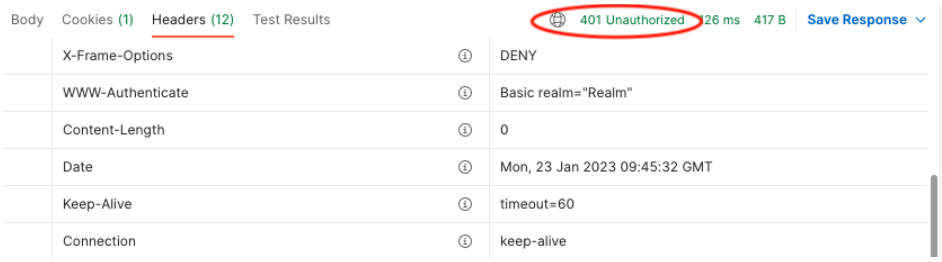


Figura 734. Obtenemos un 401

En realidad, lo que sucede (Figura 735) es que los filtros incluidos en la *DefaultSecurityFilterChain* que ha añadido Spring Security interceptan y rechazan la petición antes de que llegue a su destino.

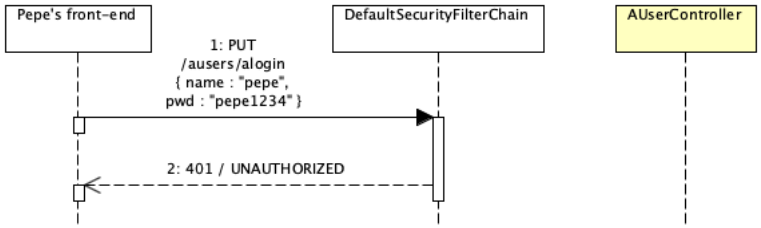


Figura 735. Los filtros de seguridad evitan que la petición llegue a destino

3.2.1. Adición de la cabecera

En principio, para que la petición supere los filtros se le debe añadir la cabera *Authorization BASIC admin:Administrador1234* (ya sabemos que las credenciales van en base64). Con CURL lo haríamos añadiendo un parámetro *-h*, y en Postman lo podemos hacer de dos formas:

1) Añadiendo manualmente la cabecera pasando el valor en base64 (Figura 736), que podemos calcular de muchas formas.

Figura 736. Adición manual de la cabecera

Dos de estas formas son, por ejemplo:

a. Buscando directamente en Internet con la cadena “encode base64”.

b. Escribiendo el comando *btoa("admin:Administrador1234")* en una consola del navegador.

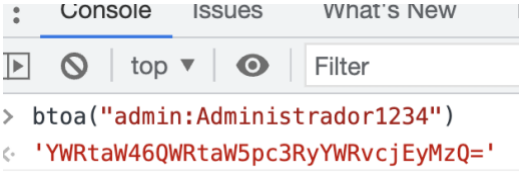


Figura 737. Codificación a base64 *de una cadena en el navegador*

2) Más cómodamente, escribiendo las credenciales en la solapa *Auth*:

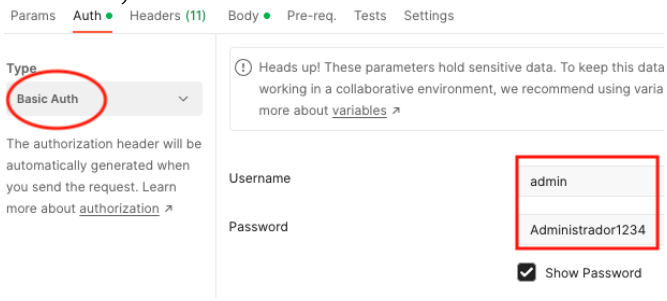


Figura 738. Adición de la cabecera en la solapa Auth

3.2.2. Envío de la petición PUT a *alogin* (I)

Ahora que, de una u otra forma, la cabecera está preparada, esperamos que el *DefaultSecurityFilterChain* cuele la petición hacia el recurso solicitado. Sin embargo, la respuesta es exactamente la misma que antes, exhibiendo el sistema un comportamiento similar al de la Figura 735.

3.2.3. Envío de una petición GET

Una petición GET con la misma cabecera *Authorization*, sin embargo, sí funcionaría. Para comprobarlo podemos añadir al *AUserController* un sencillo servicio *users* de tipo GET que nos devuelva la lista de usuarios que hay almacenados en la base de datos:

```
@GetMapping("/users")
public List<AUser> users() {
   return this.userDAO.findAll();
}
```

Figura 739. Un servicio de tipo GET

La petición pasa por los filtros y devuelve un 200:

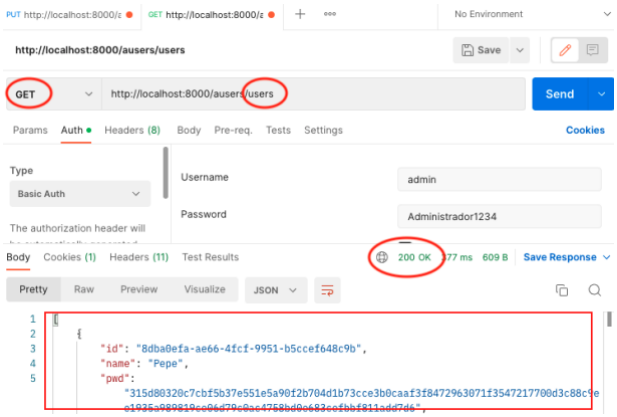


Figura 740. La petición GET si se ejecuta

3.2.4. GET frente a PUT

La petición GET pasa porque los filtros de seguridad consideran que no es peligrosa. En efecto, una petición de este tipo no está pensada para alterar la información: de hecho, en la especificación RFC 7231[14] del IETF (*Internet Engineering Task Force*), los métodos GET, HEAD, OPTIONS y TRACE se consideran seguros y, de hecho, el programador debe garantizar que así lo sean. En este caso, además, la petición se considera segura por dos motivos: (1) porque trae un valor válido para la cabecera *Authorizaton* y (2) porque es de tipo GET.

PUT (como POST o DELETE), sin embargo, está pensado para alterar la información del sistema, por lo que debemos alterar la cadena de filtros para que, además de exigir que traigan una cabecera válida, autorizar explícitamente este tipo de peticiones.

[14] https://datatracker.ietf.org/doc/html/rfc7231

Capítulo 29. Filtros de seguridad (I): autorización de todas las peticiones

1. Autorización de cualquier petición

Vamos a modificar la configuración de seguridad por defecto que implementa Spring Security para que admita cualquier petición, sin necesidad de que lleve la cabecera *Authorization*.

De momento, vamos a crear un paquete *edu.uclm.esi.auth.security* y, dentro de esta, una clase *ASecurityConfiguration* (la "A" inicial, para no confundirnos y saber que estamos en el sistema *auth)*.

La implementación que le damos, por ahora, no restringirá ninguna petición: es decir, capturará las peticiones, pero dejará que todas ellas se ejecuten, como si no estuviésemos usando Spring Security. En la Figura 741 se muestra el posible código de esta clase:

1) Con la anotación *@Configuration* conseguimos que Spring procese esta clase durante su arranque (de hecho, si ponemos un punto de interrupción en la primera línea del método, la ejecución se detendrá en él cuando la aplicación se lance) y ejecute los métodos anotados con *@Bean*.
2) Como se ejecuta durante el arranque de la aplicación, Spring le inyecta un objeto de tipo *HttpSecurity,* que utilizamos en el cuerpo del método para filtrar las peticiones como deseemos.
3) A partir de esa configuración del filtrado de peticiones, devolvemos un *DefaultSecurityFilterChain,* que contiene una colección de filtros de seguridad.

```
package edu.uclm.esi.auth.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.DefaultSecurityFilterChain;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class ASecurityConfiguration {

   @Bean
   SecurityFilterChain securityFilter(HttpSecurity http) throws Exception {
      http
         .csrf().disable()
         .authorizeHttpRequests()
         .requestMatchers("/**")
         .permitAll();

      DefaultSecurityFilterChain filterChain = http.build();
      return filterChain;
   }
}
```

Figura 741. Una configuración de seguridad que acepta cualquier petición

Las cuatro primeras líneas del cuerpo del método (que se podrían poner en una sola) vienen a decir lo que se expresa en la Figura 742:

http	**.authorizeHttpRequests()**	**.requestMatchers("/**")**	**.permitAll();**
Sobre la configuración de seguridad…	…selecciona aquellas peticiones…	…que vayan dirigidas a cualquier url…	…y las autorizas todas

Figura 742. Traducción a castellano de la sentencia de autorización

Nótese, en la figura anterior, que:

1) El nombre del método *authorizeHttpRequests* puede dar lugar a confusión, pues parece que es el encargado de "autorizar", pero de lo que realmente se encarga es de "seleccionar" peticiones.
2) La versión de *requestMatchers* que utilizamos toma uno o más parámetros de tipo cadena que representan los patrones de las urls a las que se aplicarán las restricciones de seguridad. El patrón "/**" selecciona cualquier petición. En lugar de *requestMatchers("/**")* podríamos haber utilizado el método *anyRequest()*.
3) Por último, *permitAll* le dice a Spring Security que autorice todas las peticiones que se han seleccionado en los dos puntos anteriores.

El método devuelve un *DefaultSecurityFilterChain,* que contiene los once filtros mostrados en la Figura 743.

Figura 743. Filtros incluidos en el DefaultSecurityFilterChain

2. Preparación de pruebas

En los siguientes epígrafes vamos a probar tanto desde Postman o CURL, como desde el navegador, los siguientes escenarios:

1) Registro (*/ausers/aregister)*.
2) Confirmación de cuenta (*/ausers/aconfirm*).
3) login (*/ausers/alogin)*.

Para que todo funcione debemos tener lanzados el servidor de Angular (en 4200), el de *games* (80), el de *auth* (8000), el docker con SQL Server (puerto 1433) y el servidor de MySQL (en el 3306).

Antes de ejecutar, recordemos que en *auth* tenemos el *AResponseFilter* que copiamos y pegamos desde la antigua versión de *games* (Figura 668, página 406). Esto quiere decir que las peticiones serán capturadas, en primer lugar, por la lista de filtros del *DefaultSecurityFilterChain* y, en segundo, por el *AResponseFilter*.

Bueno, pues con toda la infraestructura arrancada, y con las dos bases de datos limpias (hemos hecho *delete from…* en todas las tablas), nos ponemos manos a la obra.

3. Prueba del registro

Para el usuario, el escenario que esperamos que se ejecute, y en el que intervienen todos los sistemas que hemos mencionado, es el de la Figura 744:

1) El navegador solicita el *front-end* al servidor de 4200, que se lo devuelve.
2) Usando el navegador, Pepe envía sus credenciales al recurso */ausers/aregister* de *auth*. Este inserta el usuario en la tabla, genera e inserta un token, envía el email y, finalmente, le dice a *games* que inserte el *id* del usuario en su base de datos.

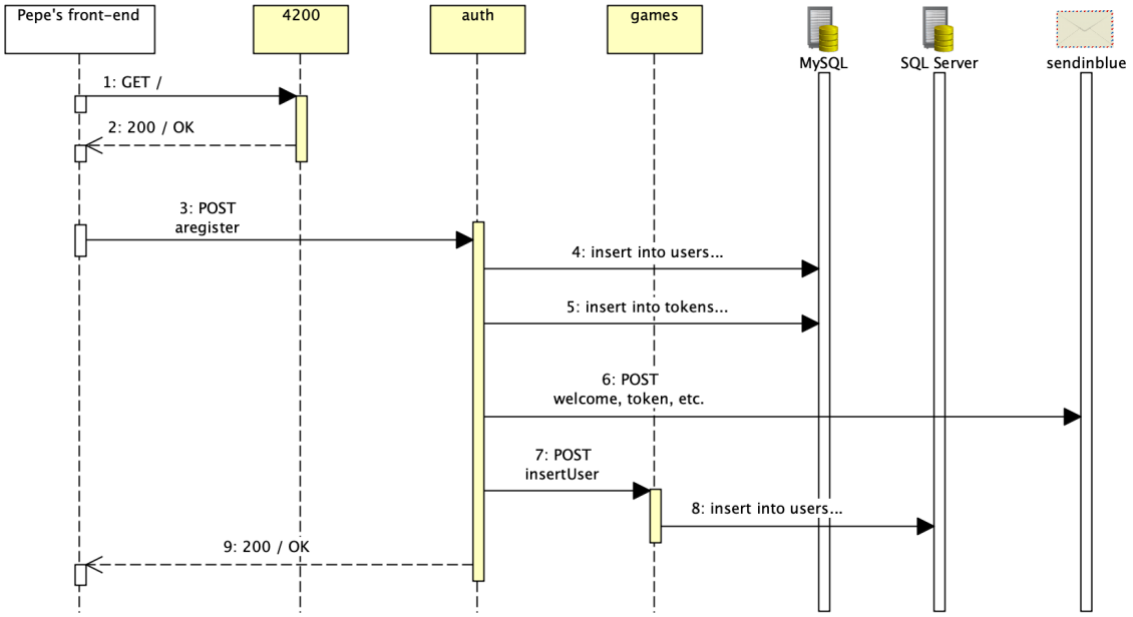


Figura 744. Escenario de registro con todos los sistemas en funcionamiento

3.1. Registro con Postman

Vamos a registrar a Pepe con sus credenciales habituales (pepe@pepe.com, pepe1234, pepe1234) utilizando Postman. En este caso, la petición viaja directamente al servidor *auth*. La petición es de tipo POST y, como estamos permitiendo todas las peticiones, no es necesario que lleve la cabecera *Authorization*. Como vemos en la Figura 745, la petición llega y se obtiene un 200.

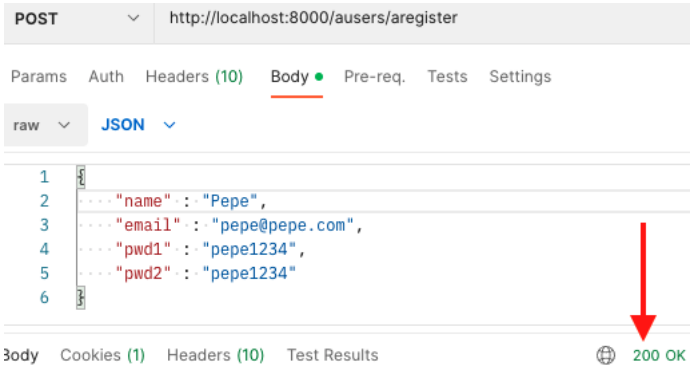


Figura 745. Petición a /aregister

Además, en las dos bases de datos (MySQL para *auth,* SQL Server para *games*), el usuario se ha insertado correctamente (Figura 746). Véase además

que, para facilitar el *login* sin tener que confirmar con el correo electrónico, en MySQL le quitamos a la columna *validation_date* el valor *null*.

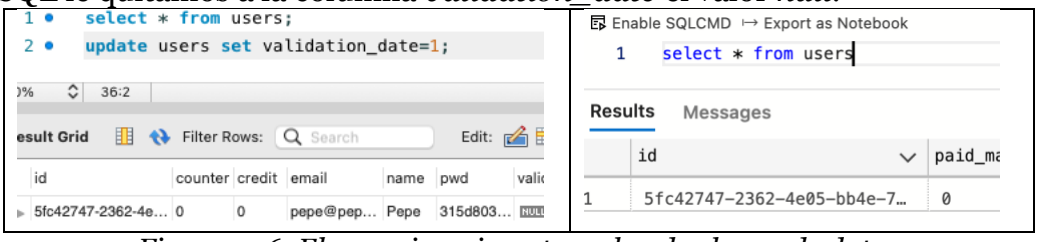


Figura 746. El usuario se inserta en las dos bases de datos

3.2. Registro con el navegador

Para probar exactamente el mismo escenario con el navegador debemos eliminar al usuario de las dos bases de datos.

Hecho esto, comprobamos que el registro también se lleva a cabo. En este caso, le dejamos la *validation_date* a *null* para probar la confirmación de la cuenta.

4. Prueba de la confirmación de cuenta: una curiosidad

Pepe ha recibido el correo con el enlace para que confirme su cuenta, que incluye (es mentira) un enlace directo al recurso */ausers/aconfirm/TOKEN* del sistema *auth*. Pepe revisa el correo, hace clic y, en efecto, se ejecuta el recurso y la cuenta de Pepe queda confirmada.

Decíamos que es arriba que es mentira porque, si uno se fija con detalle, el link incluido en el correo no apunta directamente a nuestro servidor en *localhost:8000*, sino a un extraño destino alojado en *ffdiefh.r.af.d.sendibt2.com* (Figura 747).

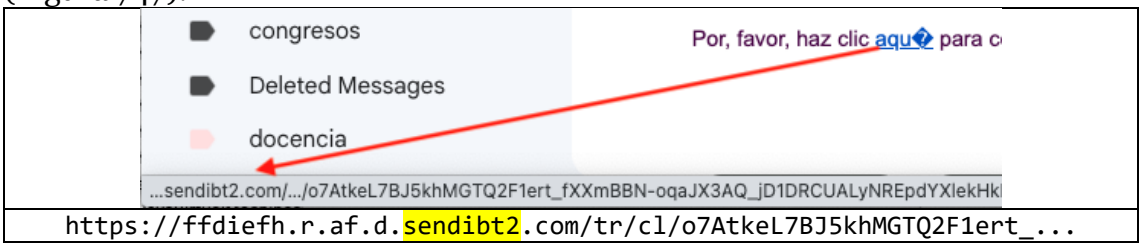


Figura 747. URL a la que apunta el enlace del mensaje recibido

O sea, que parece que *sendiblue* examina y modifica los mensajes que enviamos a través de él. Y en efecto, si registramos a otro usuario y colocamos un punto de interrupción en el lugar adecuado, vemos que el texto que se compone para enviar contiene la url de nuestro sistema (Figura 748); sin embargo, llega otra cosa, con lo que nuestra sospecha es cierta. De hecho, si nos vamos a *sendibt2.com* con el navegador, se nos redirige a la página de *sendinblue*, lo que indica que este proveedor sustituye los enlaces por otros dirigidos a sus propios servidores, que luego, sí, terminan por redirigir la petición al *endpoint* deseado.

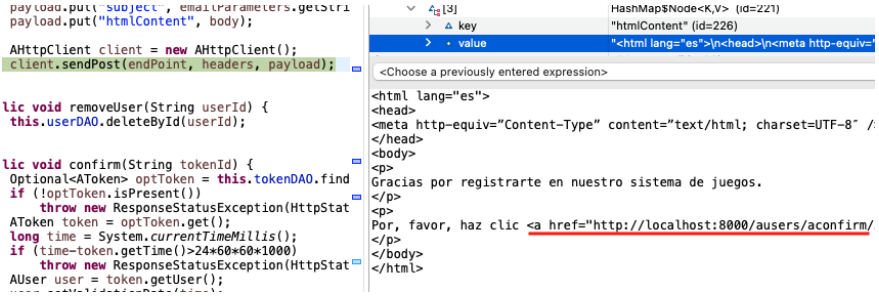


Figura 748. Incluimos un enlace a nuestra url

5. Prueba del login. De Postman a CURL

Una vez que Pepe ha confirmado la creación de su cuenta, el login funciona perfectamente tanto desde Postman como desde el navegador.

Las peticiones de Postman pueden mostrarse como un comando CURL (Figura 749, izquierda), con lo que podremos ejecutarlo desde una terminal (Figura 749, derecha) y observar la respuesta. Para ello no hay más que darle al botoncillo </> que encontramos arriba a la derecha.

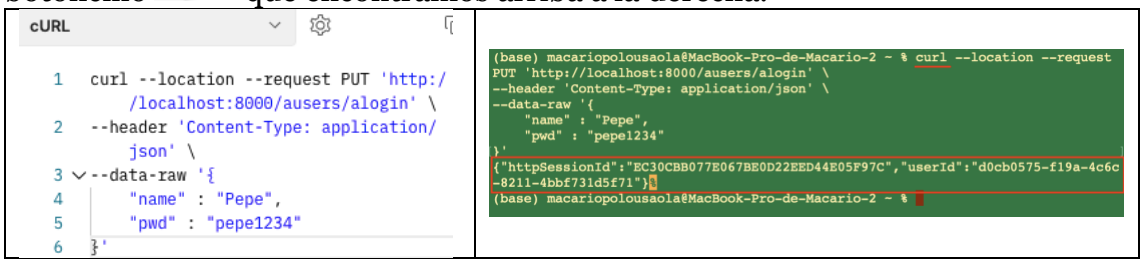


Figura 749. Exportación de la petición en formato CURL

Capítulo 30. Creación manual de un token de seguridad

1. Compra de partidas

En este capítulo implementaremos el mecanismo del *securityToken* que comentábamos en la sección «Un usuario desleal» del Capítulo 28, que se describía en la Figura 724 y que reproducimos de nuevo, con algo más de detalle, en la Figura 750:

1) En el mensaje 4, y teniendo en cuenta que *jsoCredit* es el objeto que se enviará a *auth*, *games* le añade el token aleatorio ("5678") y lo envía a *auth* en el mensaje 5.
2) En el mensaje 6, con el token ya enviado, le actualiza la fecha de creación y lo añade a una colección de *secutiryTokens* que debemos añadir al *Manager*.
3) En el mensaje 8, *auth* le pregunta a *games* que si el token "5678" es válido: *games* lo busca en la lista. En el escenario que se describe, el token existe y la fecha es válida, por lo que le devuelve un 200 a *auth* con el *id* del usuario asociado.
4) En la respuesta que recibe *auth* en el mensaje 10, comprueba que el *id* del usuario que se le devuelve es el mismo que recibió en el mensaje 5. Como coinciden, actualiza su base de datos.

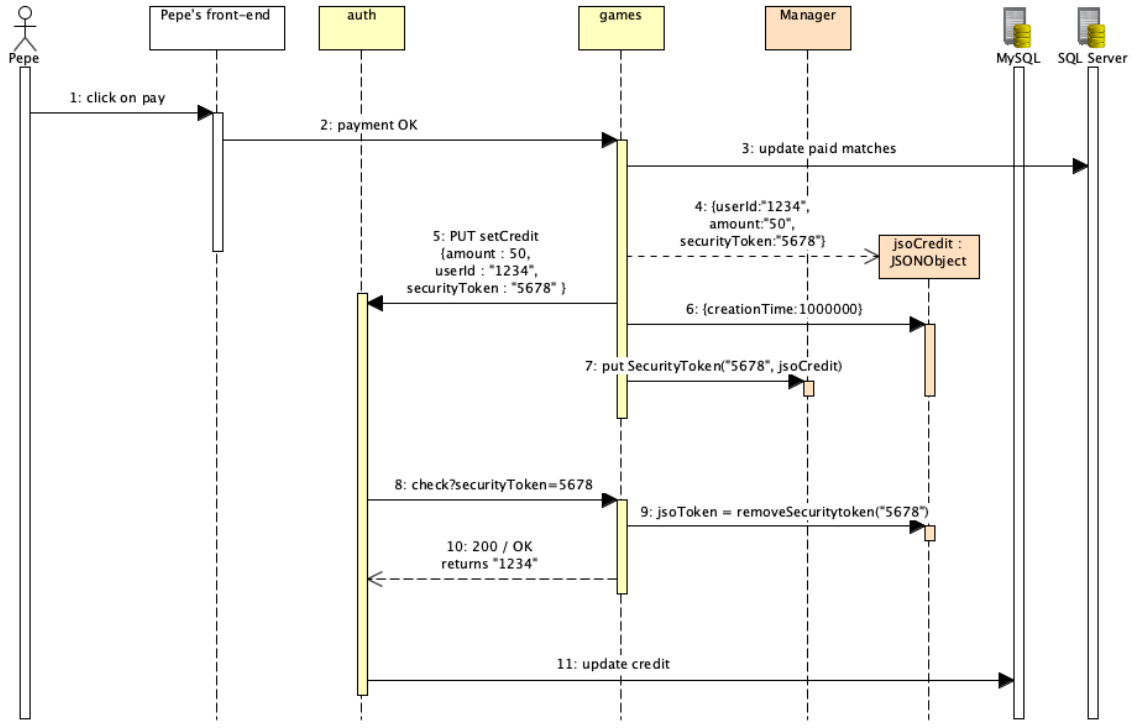


Figura 750. Implementaremos este mecanismo del securityToken

Lamentablemente, el escenario descrito no funciona, porque el mensaje 5 se está enviando de manera síncrona: es decir, el mensaje 6 no se ejecutará mientras no llegue la respuesta al mensaje 5, con lo que, al comprobar si el token

está en el *Manager,* se devolverá un *null* en el mensaje 9 y a *auth* le llegará un error 404.

Para solucionar este pequeño problema debemos enviar la petición de forma asíncrona. En los dos próximos epígrafes se muestran dos maneras diferentes.

2. Creación de un hilo

Lo que haremos será crear un *Thread* que envíe la petición asíncronamente en un hilo separado: en la Figura 751 se crea un objeto *Runnable* que puede lanzarse en un hilo de ejecución diferente. En su método *run* escribimos el código que debe ejecutarse cuando el hilo (realmente el *Thread* que creamos a continuación) se lance mediante *start*.

```
@GetMapping("/confirm")
public void confirm(HttpServletRequest request, HttpSession session) {
   if (session.getAttribute("client_secret")==null
         || session.getAttribute("matches")==null
         || session.getAttribute("userId")==null)
      throw new ResponseStatusException(HttpStatus.FORBIDDEN);

   long total = (long) session.getAttribute("total");
   String securityToken = UUID.randomUUID().toString();
   JSONObject jsoCredit = new JSONObject()
         .put("userId", session.getAttribute("userId"))
         .put("amount", total/100)
         .put("securityToken", securityToken);

   Runnable asyncRequest = new Runnable() {
      @Override
      public void run() {
         HttpClient client = new HttpClient();
         client.send(HttpPut.class,
            "http://localhost:8000/ausers/setCredit", jsoCredit);
      }
   };
   new Thread(asyncRequest).start();

   jsoCredit.put("creationTime", System.currentTimeMillis());
   Manager.get().putSecurityToken(securityToken, jsoCredit);

   String userId = session.getAttribute("userId").toString();
   this.userService.addMatches(userId, (Integer) session.getAttribute("matches"));
}
```

Figura 751. Envío asíncrono de la petición en un hilo

Con esto, el efecto que conseguimos, modelado en UML, es el de la Figura 752, en donde el *PaymentsController* pasa del mensaje 6 al mensaje 8 sin esperar a que llegue la respuesta al mensaje 7.

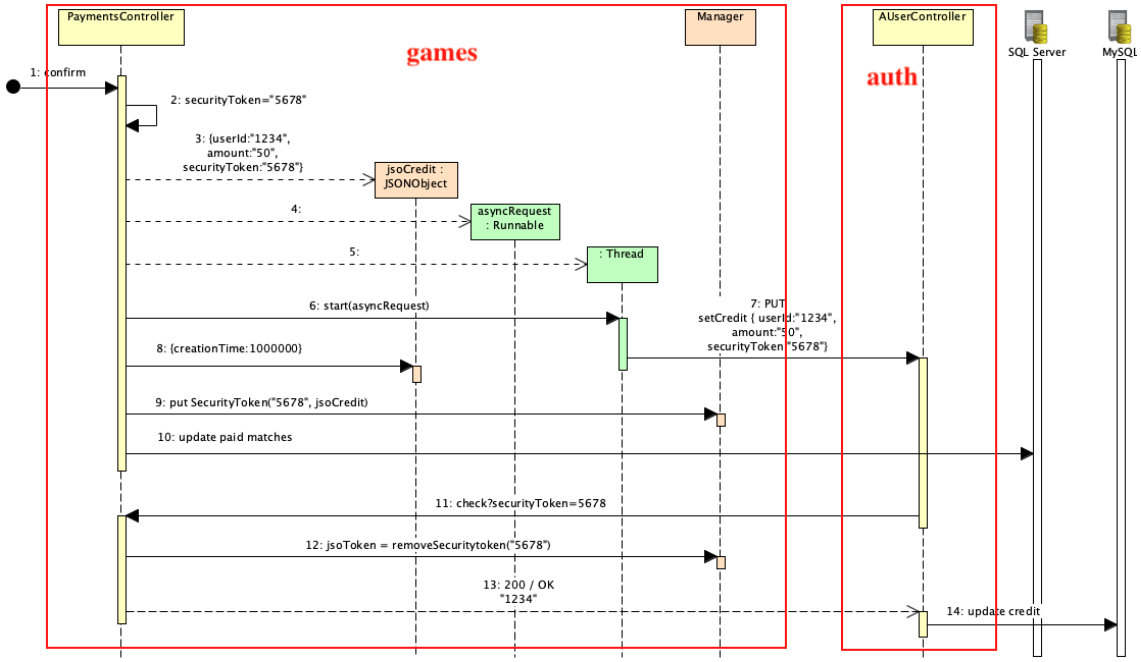


Figura 752. Intercambio de mensajes con el cambio de la Figura 751

Si ponemos dos puntos de interrupción en el recurso *confirm* del *PaymentsController* (uno dentro de *run* y otro en la sentencia en la que añadimos la fecha de creación del token), veremos que el código se detiene en los dos lugares (Figura 753, izquierda), cada uno en un hilo distinto (Figura 753, derecha).

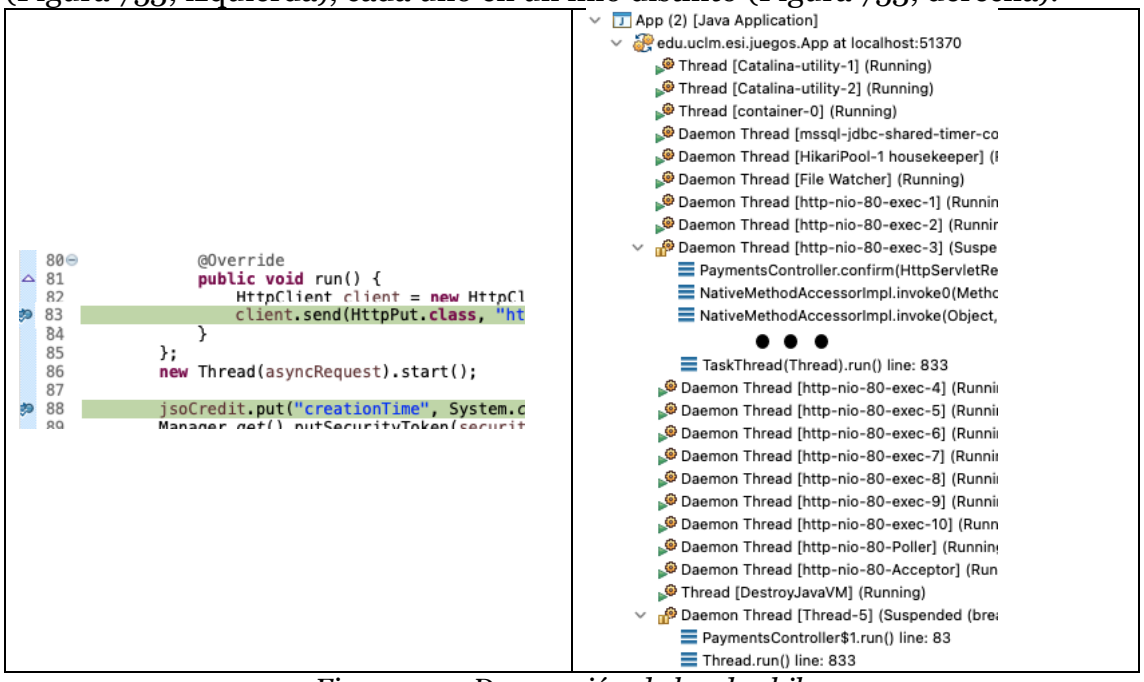


Figura 753. Depuración de los dos hilos

3. Construcción de un cliente asíncrono

Otra posibilidad pasa por crear directamente un cliente que envíe las peticiones asíncronamente. En este caso, vamos a dejar en *games* la implementación con el hilo que hemos hecho en la sección anterior, y vamos a crear un cliente asíncrono en el proyecto *auth*. De este modo trabajaremos simultáneamente con dos posibilidades.

Crearemos una nueva clase ***Async****HttpClient* en el mismo paquete que *HttpClient,* es decir, en *edu.uclm.esi.****auth****.auxi*.

En *HttpClient,* los métodos de envío utilizan un *CloseableHttpClient,* que se encuentra en la librería *org.wso2.apache.httpcomponents* (Figura 641, página 386). En lugar de utilizar una librería para clientes asíncronos del mismo proveedor, y como se trata de aprender, cambiaremos a *OkHttpClient*.

En esta nueva clase *AsyncHttpClient* crearemos un único método *send(String method, String url, JSONObject payload, String…headers)* que podrá enviar peticiones GET, POST y PUT. Para su implementación no usaremos reflexión, para así no añadir contenido accesorio al buen entendimiento de la también buena explicación que se pretende dar.

Tras añadir la dependencia al *pom.xlm*, escribimos una primera versión de nuestro cliente asíncrono como en la Figura 754:

1) El método recibe el tipo de la petición, la *url*, el JSON a enviar y las posibles cabeceras que queremos añadir.
2) Antes de construir la petición creamos un *RequestBuilder,* al que llamaremos pocas líneas después.
3) En función del tipo de petición, construimos o no el *body* de lo que vamos a enviar (indicando, si procede, que es el objeto *payload* y que se trata de un *application/json)* y, después con el *builder* se construye la petición propiamente dicha.
4) A continuación construimos el cliente que enviará la petición, especificando los tiempos máximos de espera para conectar y manejar las respuestas: al estar en modo desarrollo, ponemos tiempos exagerados para evitar excepciones de *timeout*.
5) Como la petición viaja de manera asíncrona, digamos que la respuesta llegará cuando quiera (puede tardar un segundo, pero igual tarda treinta), por lo que la enviamos con *newCall* pasando como parámetro un manejador de eventos (ese *Callback* que se está construyendo), que maneja dos posibles eventos: *onResponse* y *onFailure*.
6) Si todo va bien, la respuesta se recogerá en *onResponse*. De momento, nos conformamos con que ambos manejadores muestren lo que reciben por la consola.
7) Nótese también que el método es *void*, porque precisamente por ser asíncrono no es preciso que devuelva nada, al menos en este caso.
8) Añadimos una segunda versión del método por si queremos pasar las baceras en un JSONArray.

```
<dependency>
    <groupId>com.squareup.okhttp3</groupId>
    <artifactId>okhttp</artifactId>
    <version>4.9.1</version>
</dependency>
```

```
package edu.uclm.esi.auth.auxi;

import java.io.IOException;
import java.util.concurrent.TimeUnit;

import org.json.JSONObject;

import okhttp3.Call;
import okhttp3.Callback;
import okhttp3.MediaType;
import okhttp3.OkHttpClient;
import okhttp3.Request;
import okhttp3.Request.Builder;
import okhttp3.RequestBody;
import okhttp3.Response;

public class AsyncHttpClient {
```

```
public void send(String method, String url, JSONObject payload, String... headers) {
     Builder builder = new Request.Builder().url(url);
     Request request = null;

     if (method.equalsIgnoreCase("get")) {
        request = builder.get().build();
     } else if (method.equalsIgnoreCase("post") || method.equalsIgnoreCase("put")) {
        RequestBody body = RequestBody.create(payload.toString(),
              MediaType.get("application/json"));
        if (method.equalsIgnoreCase("post"))
           request = builder.post(body).build();
        else
           request = builder.put(body).build();
     }

     OkHttpClient client = new OkHttpClient.Builder()
        .connectTimeout(6000, TimeUnit.SECONDS).readTimeout(6000, TimeUnit.SECONDS)
        .writeTimeout(6000, TimeUnit.SECONDS).build();

     Call call = client.newCall(request);
     call.enqueue(new Callback() {

        @Override
        public void onResponse(Call call, Response response) throws IOException {
           System.out.println(response);
        }

        @Override
        public void onFailure(Call call, IOException error) {
           System.out.println(error);
        }
     });
}

public void send(String method, String url, JSONObject payload,
        JSONArray jsaHeaders) {
     String[] headers = new String[jsaHeaders.length()];
     for (int i=0; i<jsaHeaders.length(); i++)
        headers[i] = jsaHeaders.getString(i);
     this.send(method, url, payload, headers);
  }
}
```

Figura 754. Un cliente asíncrono de prueba creado con OkHttpClient

En *auth* estamos usando el cliente síncrono desde tres lugares (en Eclipse podemos verlo buscando las referencias al elemento dentro del proyecto, como se muestra en la Figura 755), que vamos a sustituir por llamadas al recién creado método *send* de *AsyncHttpClient*.

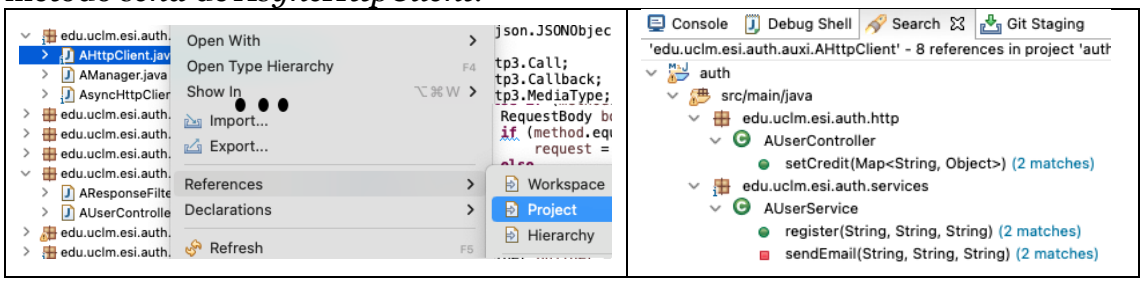


Figura 755. Búsqueda de referencias a un elemento en Eclipse

Primero eliminamos o, por si acaso, excluimos del *classpath* del proyecto el fichero *AHttpClient*. Evidentemente, saltan errores de compilación (Figura 756) en los tres lugares indicados en la figura anterior.

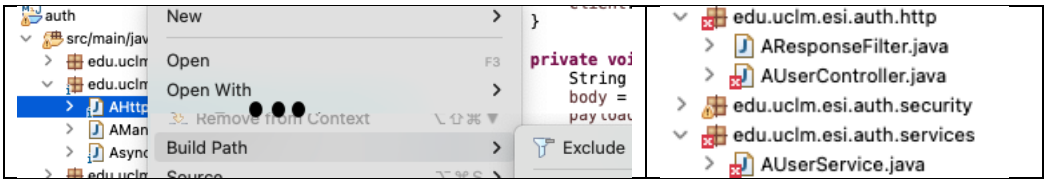


Figura 756. Saltan tres errores de compilación

Sustituyamos las tres llamadas que ya no compilan por otras tres que sí lo hagan. En la Figura 757:

1) Modificamos la llamada a *sendPost* que envía la petición a *sendinblue*.
2) Sustituimos la llamada al *insertUser* que hicimos en el nuevo *UserController* de *games* (allá por la Figura 706, página 432).
3) Finalmente hacemos lo mismo en *setCredit*, que es un poco más problemático: como vemos, la petición se envía con el cliente asíncrono, pero ya no esperamos aquí obtener respuesta alguna, motivo por el cual el compilador nos advierte de un error de compilación en una de las últimas líneas del método: en efecto, *response* aquí no tiene sentido, pues aquí no se recibe la respuesta a la petición, sino que se recibe en el manejador *onResponse* de la Figura 754.

```
private void sendEmail(String email, String name, String tokenId) throws IOException {
   ...
 //AHttpClient client = new AHttpClient();
 //client.sendPost(endPoint, headers, payload);

 AsyncHttpClient client = new AsyncHttpClient();
 client.send("post", endPoint, payload, headers);
}
```

```
@Transactional
public void register(String name, String pwd, String email) throws IOException {
   ...
 //AHttpClient client = new AHttpClient();
 JSONObject jsoUser = new JSONObject().put("id", user.getId());
 //client.sendPost("http://localhost/users/insertUser", "content-type : application/json",
jsoUser);

 AsyncHttpClient client = new AsyncHttpClient();
 client.send("post", "http://localhost/users/insertUser", jsoUser);
}
```

```
@PutMapping("/setCredit")
public void setCredit(@RequestBody Map<String, Object> info) {
 String securityToken = info.get("securityToken").toString();
 //AHttpClient client = new AHttpClient();
 String url = "http://localhost/payments/checkSecurityToken?securityToken=" +
    securityToken;
 //String response = client.sendGet(url, "content-type : application/json");

 AsyncHttpClient client = new AsyncHttpClient();
 client.send("get", url, null);

 String userId = info.get("userId").toString();
 if (response.equals(userId)) {
   double amount = Double.parseDouble(info.get("amount").toString());
   this.userService.setCredit(userId, amount);
 }
}
```

Figura 757. Sustitución del AHttpClient *por el* AsyncHttpClient

Como no tenemos acceso a la *response*, tenemos que buscar alguna manera de recibirla en el manejador *onResponse* de nuestro *AsyncHttpClient* para que, como hacemos en la figura anterior, se compare el texto de la respuesta con el

userId y, si coinciden, que se le incremente el crédito a Pepe o al usuario de quien se trate.

Una **primera forma** que implica poco código, pero que es muy poco reutilizable, pasa por modificar la firma de *send*, de manera que la llamada quede como en la línea resaltada de la fila superior de la Figura 758, en la que decimos que queremos ejecutar un GET a esa *url*, que no hay *payload* y que la respuesta que llegue se compare con la que le pasamos en último lugar. En la declaración de los dos *send* que tenemos, añadimos *info* como cuarto parámetro, pues las listas de tipo "..." (*headers* en nuestro caso) deben ir en último lugar.

```
@PutMapping("/setCredit")
public void setCredit(@RequestBody Map<String, Object> info) {
   String securityToken = info.get("securityToken").toString();
   String url = "http://localhost/payments/checkSecurityToken?securityToken=" +
         securityToken;

   AsyncHttpClient client = new AsyncHttpClient();
   client.send("get", url, null, new JSONObject(info));
}
```

```
send(String method, String url, JSONObject payload, JSONObject info, String... headers)

send(String method, String url, JSONObject payload, JSONObject info, JSONArray jsaHeaders)
```

Figura 758. Una solución que funcionaría, pero que sería poco reutilizable

Con la implementación de la figura anterior, el manejador *onResponse* debería comparar el texto de la respuesta con el valor del parámetro *info* que recibe. Sería una solución muy *ad hoc* para esta situación tan determinada.

Para la **segunda posibilidad** podemos fijarnos en la sentencia *call.enque(new Callback() { ... })* de la Figura 750: *call.enque* envía la petición y espera la respuesta en un *Callback* que, como ya sabemos, la maneja con *onResponse* o con *onFailure*.

Bien, pues vamos a explorar esta segunda alternativa: cuando llamemos a *client.send* en *setCredit,* vamos a pasar directamente el objeto *callback* que tiene que utilizar.

Como los arrays de tipo "..." deben ir en último lugar, modificamos las firmas de los dos *send* por las versiones de la Figura 759

```
send(String method, String url, JSONObject payload, Callback callback, String... headers)

send(String method, String url, JSONObject payload, Callback callback, JSONArray jsaHeaders)
```

Figura 759. Añadimos el callback como parámetro

register y *sendMail* no esperan procesar ninguna respuesta, por lo que podemos pasar directamente el valor *null* a los parámetros:

```
private void sendEmail(String email, String name, String tokenId) throws IOException {
   ...
 client.send("post", endPoint, payload, null, headers);
}
```

```
@Transactional
public void register(String name, String pwd, String email) throws IOException {
   ...
 client.send("post", "http://localhost/users/insertUser", jsoUser, null);
}
```

Figura 760. sendEmail *y* register *no necesitan procesar la respuesta*

setCredit, sin embargo, sí necesita procesar la respuesta, por lo que envía la petición pasando un *callback* que construye exprofeso el tratamiento que debe darse a la repuesta que se obtenga de esta petición:

```
@PutMapping("/setCredit")
public void setCredit(@RequestBody Map<String, Object> info) {
   String securityToken = info.get("securityToken").toString();
   String url = "http://localhost/payments/checkSecurityToken?securityToken=" +
      securityToken;

   AsyncHttpClient client = new AsyncHttpClient();
   client.send("get", url, null, new Callback() {

      @Override
      public void onResponse(Call call, Response response) throws IOException {
         String userId = info.get("userId").toString();
         String responseText = response.body().string();
         if (response.equals(info).get ("userId")) {
            double amount = Double.parseDouble(info.get("amount").toString());
            AUserController.this.userService.setCredit(userId, amount);
         }
      }

      @Override
      public void onFailure(Call arg0, IOException arg1) {
         // TODO Auto-generated method stub
      }
   });
}
```

Figura 761. Llamamos a send *con un determinado* callback

Finalmente, el *send* del *AsyncHttppClient* ejecuta el *callback* cuando llega la respuesta, si es que hay *callback* que ejecutar:

```
public void send(String method, String url, JSONObject payload, String... headers) {
   Builder builder = new Request.Builder().url(url);
   ...

   Call call = client.newCall(request);
   if (callback==null)
      try {
         call.execute();
      } catch (IOException e) {
         e.printStackTrace();
      }
   else
      call.enqueue(callback);
}
```

Figura 762. Procesamiento del posible callback

Capítulo 31. Filtros de seguridad (II): filtrado de peticiones

1. Filtrado de peticiones: protección de *setCredit* en *auth*

Aunque la petición que se envía a *setCredit* en *AUserController* está bastante bien protegida por el *securityToken,* vamos a blindarla aún más exigiendo que llegue autenticada. Lo haremos de dos maneras.

2. Con *Authorization : BASIC + credenciales*

Recordaremos que habíamos creado un usuario "admin" con contraseña "Administrador1234" en *application.properties* (Figura 731, página 445). Vamos a modificar la configuración de seguridad para exigir que las peticiones que se reciban en el *setCredit* del *AUserController* traigan la cabecera *Authorization* con las credenciales de tipo BASIC correspondientes a ese usuario.

En la Figura 763 añadimos, como primera regla, que las peticiones que lleguen a la url */ausers/setCredit* tienen que venir autenticadas, y para el resto no hace falta.

El método *authenticated* comprueba que en la petición venga la cabecera *Authorization;* además, el método *httpBasic* que aparece al final es el que comprueba que la cabecera lleva en su valor el token BASIC. El orden de las reglas es importante, pues Spring aplica a la petición la primera regla que case con el argumento del *requestMatchers*.

```
@Configuration
public class ASecurityConfiguration {
   @Bean
   SecurityFilterChain securityFilter(HttpSecurity http) throws Exception {
      http
         .authorizeHttpRequests()
            .requestMatchers("/ausers/setCredit")
               .authenticated()
            .anyRequest()
               .permitAll()
         .and()
            .httpBasic();

      return http.build();
   }
}
```

Figura 763. Incrementamos la protección de auth/setCredit

Claro que, para que la petición supere los filtros y llegue al servicio *setCredit* del *AUserController*, el sistema *games* debe enviar la petición con dicha cabecera (mensaje 7 de la Figura 764).

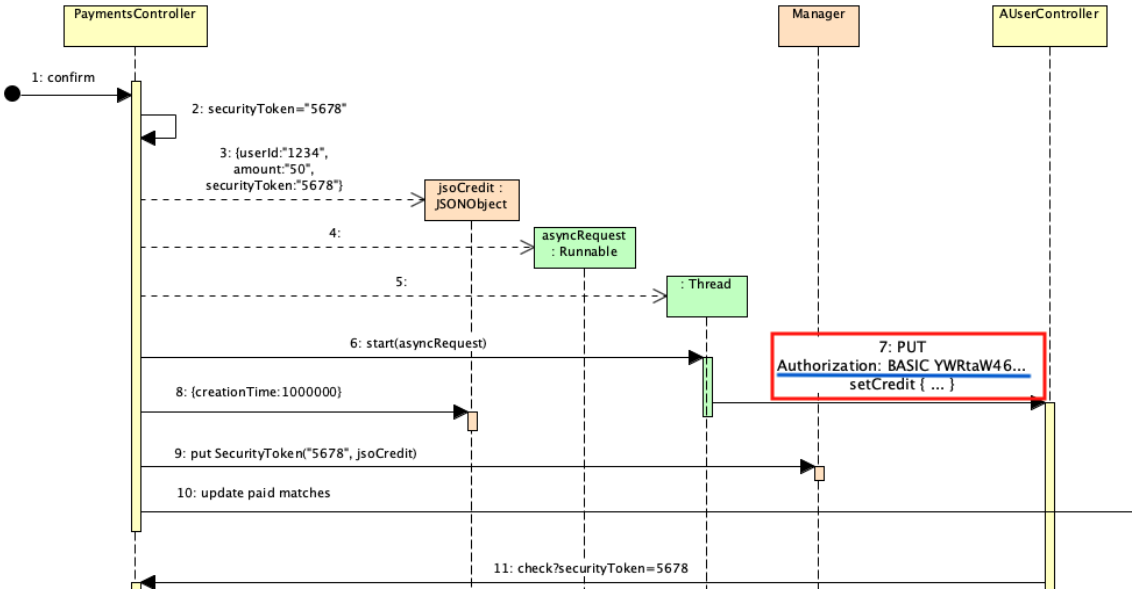


Figura 764. Adición de la cabecera Authorization

El cambio que debe hacerse es bastante simple: en el *PaymentsController*, cuando enviemos la petición, añadimos la cabecera con valor "BASIC admin:Administrador1234" (si bien ya sabemos qué parte irá en base64).

```
@GetMapping("/confirm")
public void confirm(HttpServletRequest request, HttpSession session) {
   ...
   Runnable asyncRequest = new Runnable() {
      @Override
      public void run() {
         HttpClient client = new HttpClient();
         String header = "Authorization : BASIC YWRtaW46QWRtaW5pc3RyYWRvcjEyMzQ=";
         client.send(HttpPut.class, "http://localhost:8000/ausers/setCredit",
            jsoCredit, header);
      }
   };
   ...
}
```

Figura 765. Envío de la petición en un hilo (como en la Figura 751), pero añadiendo la cabecera Authorization

3. Con *Authorization : BEARER + token*

En el epígrafe anterior se está enviando la cabecera *Authorization* con las credenciales almacenadas en los campos *spring.security.user.name* y *spring.security.user.password* del *application.properties*. Si cambian los valores de estos campos, ha de cambiar necesariamente el valor de la cabecera. Si suprimimos o comentamos esas dos líneas, el valor de la cabecera sería "BASIC user:PWD", donde PWD es la contraseña que genera Spring Security (Figura 727, página 443) y la cadena "user:PWD" iría codificada en base64.

La solución de usar BASIC en *Authorization* puede entonces no ser demasiado buena. Vamos a cambiar y a usar una autorización basada en un token usando BEARER en lugar de BASIC en la cabecera.

Lo que haremos será exigir que las peticiones que viajen desde *games* al recurso *setCredit* de *auth* vengan autorizadas con un token que, obviamente, *auth* debe haberle proporcionado previamente a *games*.

3.1. Obtención del token

En la Figura 766 se ilustra un primer escenario muy sencillo y con el que Pepe podría volver a colarse. No obstante, lo presentamos para entender el proceso de solicitud y posterior envío del token.

Antes de enviar la petición a *setCredit* en el mensaje 4, *games* le pide a *auth* un token, que le devuelve "zxcv" en este escenario. La llamada a *setCredit* solo se producirá cuando se haya recibido el *bearerToken,* por lo que se enviará de forma síncrona. Luego, el *PaymentsController* añade la cabecera *Authorization BEARER* con el token en el mensaje 4.

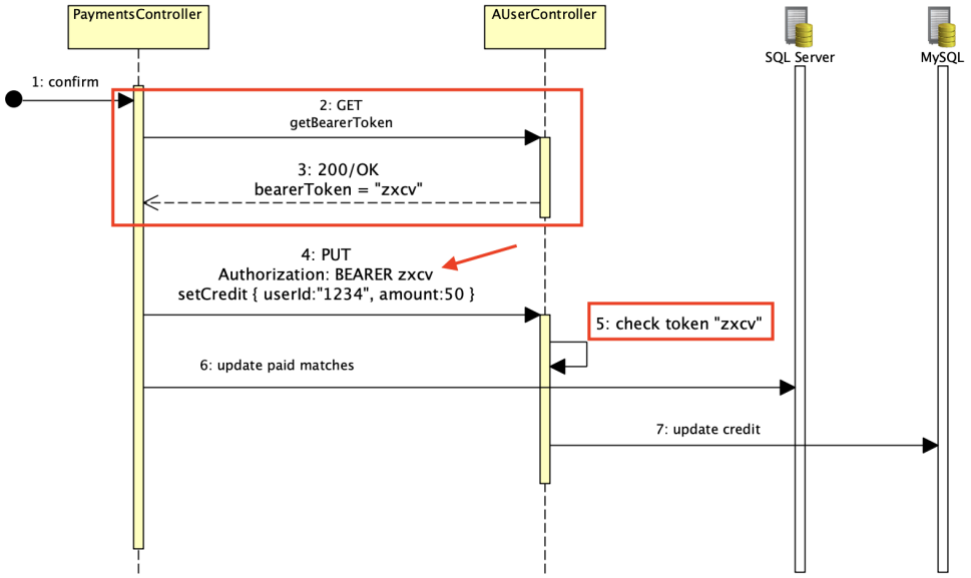


Figura 766. Obtención de un token para Authorization : BEARER

Lo primero que necesitamos es crear un servicio *getBearerToken* en *AUserController,* cuya implementación es muy sencilla:

```
@GetMapping("/getBearerToken")
public String getBearerToken() {
   String token = UUID.randomUUID().toString();
   this.tokens.put(token, token);
   return token;
}
```

Figura 767. Construcción y devolución de un token

En segundo lugar, debemos solicitar ese token antes de enviar la petición a *setCredit* para poder incluirlo en la cabecera. En la Figura 768 nos hemos llevado a un nuevo método *sendSetCredit* la petición al recurso *setCredit*. Antes, hemos modificado *confirm* para que, como hemos comentado hace unos párrafos, solicite de forma síncrona el token. Cuando haya llegado la respuesta podemos enviar la petición a *setCredit*.

```
@GetMapping("/confirm")
public void confirm(HttpServletRequest request, HttpSession session) {
  ...
  final JSONObject jsoCredit = new JSONObject()
        ...;

  HttpClient client = new HttpClient();
  String token = client.sendGet("http://localhost:8000/ausers/getBearerToken", null);
  sendSetCredit(jsoCredit, token);
  ...
}

protected void sendSetCredit(JSONObject jsoCredit, String token) {
  Runnable requestSetCredit = new Runnable() {

     @Override
     public void run() {
        HttpClient client = new HttpClient();
        String header = "Authorization : BEARER " + token;
        client.send(HttpPut.class, "http://localhost:8000/ausers/setCredit",
            jsoCredit, header);
     }
  };
  new Thread(requestSetCredit).start();

}
```

Figura 768. Solicitud y envío del token en BEARER

Si ejecutamos un intento de compra de partidas, el servicio *setCredit* le devuelve un 401 al *PaymensController* porque, aunque el token se está enviando, la configuración de seguridad no ha sido preparada para admitir el modo BEARER.

3.2. Modificación de la configuración de seguridad

Para que se admita la autenticación con el token en BEARER, debemos crear un filtro de tipo *OncePerRequestFilter* (del estilo del que creamos en el Capítulo 26) que procese las peticiones que se dirijan a *setCredit*. El filtro puede ser el de la figura siguiente: si la url solicitada en la *request* termina */ausers/setCredit*, entonces le preguntamos al controlador (que véase que lo declaramos como un *@Autowired*), que tiene los tokens, que si el que se ha recibido es válido; si no lo es, se devuelve un 401 y, muy importante, añadimos el *return* para que la petición no siga su proceso.

```
package edu.uclm.esi.auth.http;
...
@Component
public class ABearerTokenFilter extends OncePerRequestFilter {

   @Autowired
   private AUserController userController;

   @Override
    protected void doFilterInternal(HttpServletRequest request,
         HttpServletResponse response, FilterChain filterChain) throws ... {

      String url = request.getRequestURL().toString();
      if (url.endsWith("/ausers/setCredit")) {
         if (!isAuthenticated(request)) {
              response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
              return;
         }
      }
```

```
        filterChain.doFilter(request, response);
     }

   private boolean isAuthenticated(HttpServletRequest request) {
      String token = request.getHeader("Authorization").substring(7);
      return this.userController.authenticate(token);
   }
}
```

Figura 769. Un filtro para autenticar el BEARER token

Este filtro se ejecuta después de todos los filtros de seguridad añadidos al *DefaultSecurityFilterChain,* por lo que la configuración de seguridad podríamos volver a dejarla como en la Figura 741.

3.3. Obtención segura del token

El protocolo de seguridad que hemos implementado está ya doble o triplemente asegurado ya que (Figura 770): (1) el *PaymentsController* (en el sistema *games)* genera un *securityToken* (mensaje 2) y le pide a *auth* un *bearer token* (mensajes 3 y 4); (3) la petición a *setCredit* lleva este *bearer token* en la cabecera *Authorization,* además de llevar en la *payload* el *securityToken;* (4) antes de actualizar su base de datos local, *auth* le pide a *games* que le confirme la validez del *securityToken* (mensaje 8), a lo que *games* responde en el mensaje 10 con el *userId* asociado al *securityToken;* (5) en una tercera y última comprobación, *auth* compara el *userId* que le ha devuelto *games* en el mensaje 10 con el *userId* que recibió en la *payload* del mensaje 5.

Ahora es muy difícil que Pepe nos la cuele, pero sigue pudiendo enviar libremente la petición a *getBearerToken.* Es difícil que nos la cuele porque, aunque puede recuperar el *bearer token* y luego enviar el mensaje 5 con la cabecera, en el cuerpo del mensaje no podrá enviar un *securityToken* válido, con lo que el escenario de la figura no llegará a ejecutarse.

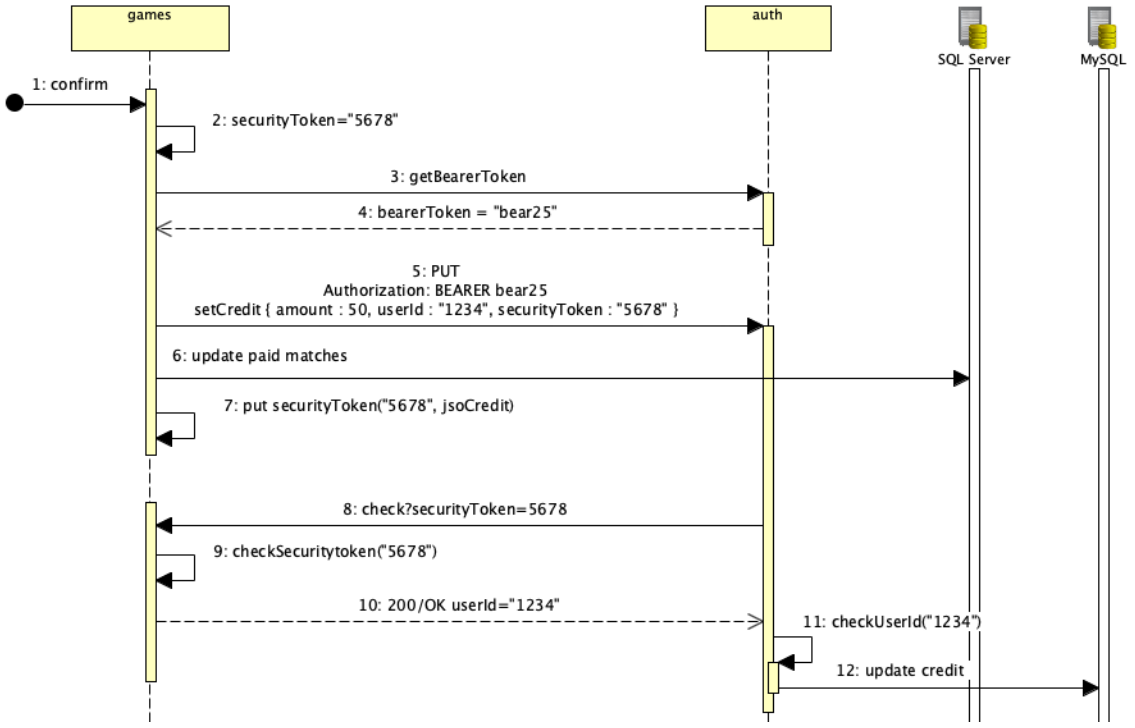


Figura 770. Flujo de mensajes doble o incluso triplemente seguro

Una forma de asegurar las peticiones a *getBearerToken* es imitar el sistema de *sendinblue,* el servidor de correo electrónico que elegimos en el Capítulo 24: dar a cada cliente (y, en este caso, *games* es un cliente *auth*) una clave privada que deba enviar en cada petición a *getBearerToken*. La clave privada podría viajar como parámetro (tipo *getBearerToken?key=345678* o *getBearerToken/345678)*, y *auth* devolvería un código de la serie 4 si tal clave privada no existiera en su base de datos.

Se sugiere al lector o lectora que implemente este mecanismo de seguridad. No es difícil: aunque sea manualmente, cree una tabla *security_tokens* en la base de datos de *auth* (es decir, en la de MySQL) y añada un token manualmente.

OCTAVA PARTE: OTROS ASPECTOS

Capítulo 32. Jugadores virtuales

1. Justificación

En este capítulo presentamos algunas alternativas para este jugador virtual, apuntando varios posibles diseños y varias formas diferentes de implementación, pero no se hace el desarrollo completo, que queda para el curioso o la curiosa lectora.

2. Planteamiento del problema

Según se explicó en la sección «El jugador virtual» (página 205), si un jugador lleva 30 segundos en la sala de espera sin que llegue otro jugador, se creará uno autónomo que simule ser un contrincante de carne y hueso. De algún modo debe conseguirse que, al crear una partida (método *playMatch* del *@Service WaitingRoom),* se ponga en marcha algún tipo de temporizador que cree ese usuario pasado ese tiempo.

Una posible estructura de clases necesaria es la de la siguiente figura, en la que cada partida posee un *Timer* que, pasados los 30 segundos, crea un *VirtualUser* como especialización de *User*.

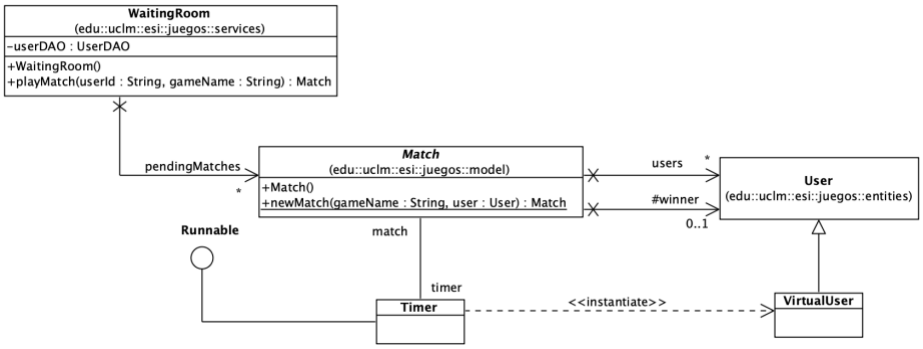


Figura 771. Posible estructura para el jugador virtual

Como hemos hecho otras veces, vamos a ir construyéndolo poco a poco, aunque en esta ocasión no usaremos Test Driven Development.

3. Un *Timer* trivial

En la figura anterior vemos que el *Timer* es un *Runnable* (correrá en un hilo separado, un *Thread)* que conoce a la partida a la que debe unirse pasado el tiempo establecido. Crearemos el *Time* en el constructor de *Match* (al que, se recordará, llamamos reflexivamente en el método *newMatch*). A pesar de que Eclipse nos propone varios paquetes para importar la clase *Timer*, optamos por crear una nueva en *model.virtual* (Figura 772). Como estamos en modo desarrollo, lo inicializamos con un tiempo de espera de 3 segundos en lugar de con 30, y así no tenemos que andar esperando, mano sobre mano, a que la partida arranque.

Figura 772. Creamos el Timer

En la Figura 773 se muestra el nuevo constructor de *Match* y el código inicial del *Timer*. En el diagrama de clases anterior, la *Match* y el *Timer* se conocen mutuamente (véase la asociación bidireccional); en el código, esto lo conseguimos declarando los dos campos (*timer* en *Match,* y *match* en *Timer)* y asignando los valores correspondientes. Además, lanzamos el *Timer* (que, como se ve, es un *Runnable*) desde un *Thread* en la última línea del constructor: al ejecutar el método *start,* se crea un hilo que llama al método *run* del objeto contenido en la *Thread,* el cual crea a su vez la instancia del jugador si la partida no ha comenzado.

```
...
private Timer timer;

public Match() {
    this.id =
          UUID.randomUUID().toString();
    this.players = new ArrayList<>();
    this.boards = new ArrayList<>();
    this.timer = new Timer(this, 3000);
    new Thread(this.timer).start();
}
```

```
package edu.uclm.esi.juegos.model.virtual;

public class Timer implements Runnable {
    private Match match;
    private long time;

    public Timer(Match match, long time) {
        this.match = match;
        this.time = time;
    }

    @Override
    public void run() {
        try {
            Thread.sleep(this.time);
        } catch (InterruptedException e) {
            return;
        }
        if (this.match.isStarted())
            return;
        VirtualUser virtualUser =
            new VirtualUser(this.match);
        new Thread(virtualUser).start();
    }
}
```

Figura 773. El constructor de Match *y una primera versión del* Timer

4. Alternativas de diseño para el jugador virtual

En el diagrama de clases de la Figura 771 hemos dibujado una relación de dependencia desde *Timer* hasta la nueva clase *VirtualUser* estereotipada con *«instantiate»,* lo que da a entender que, en algún momento, el *Timer* crea un objeto de tipo *VirtualUser*.

De alguna manera, el *VirtualUser* debe comportarse, con respecto a la partida, como si se tratase de un jugador humano.

4.1. Alternativa 1: asignación de clientes *http* y *ws*

En la Figura 774 (que también aparecía en la página 307) volvemos a mostrar el flujo de mensajes que sucede cuando Pepe y Ana llegan a jugar una partida del mismo juego y se establece la conexión entre ambos a través del *WSGames*. No obstante, en esta ocasión resaltamos elementos distintos:

1) El mensaje 11 (y su respuesta en el 16) circulan por *http*.
2) Los mensajes 17 y 18 viajan por *ws*.

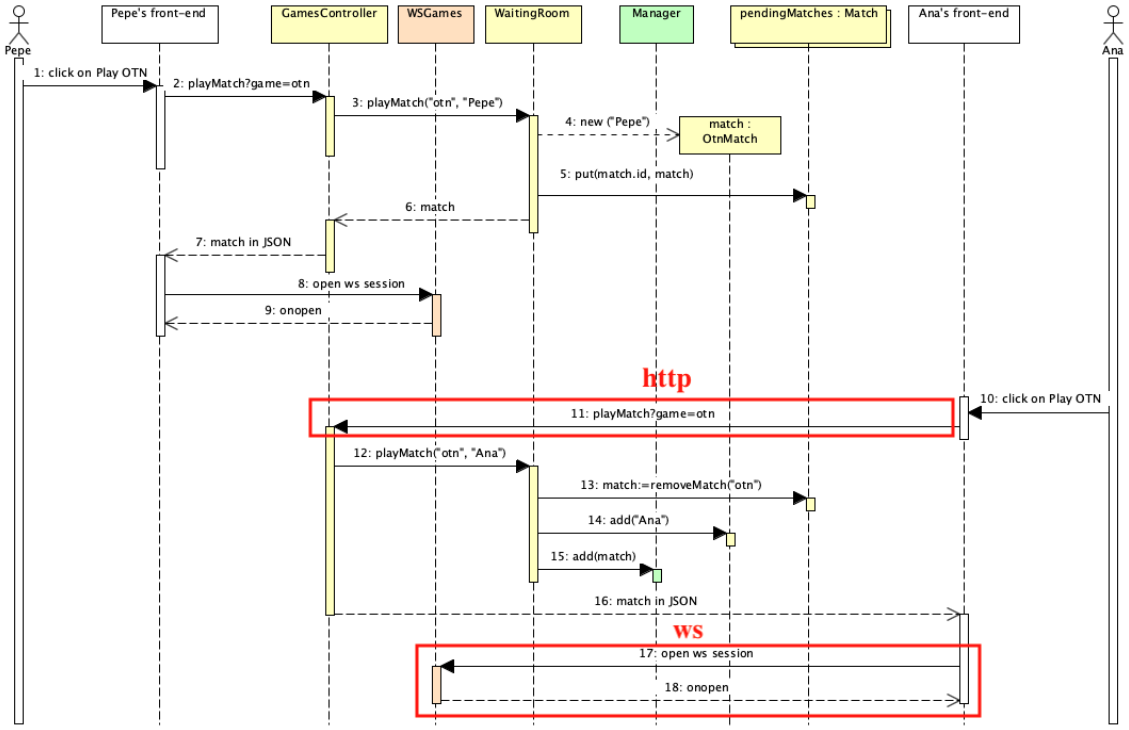


Figura 774. Pepe y Ana, humanos ambos, quieren competir en Order the numbers

Para imitar la forma de actuar de Ana (la jugadora que se incorpora más tarde) se ha elaborado el siguiente nuevo diagrama de secuencia (Figura 775), en el que hemos sustituido tanto a Ana como a su *front-end* por una usuaria a la que hemos llamado Lola. Lola es una instancia de *VirtualUser* que el *Timer* crea cuando ha pasado el tiempo establecido (denotado por el óvalo sobre la línea de vida de la instancia *timer)*.

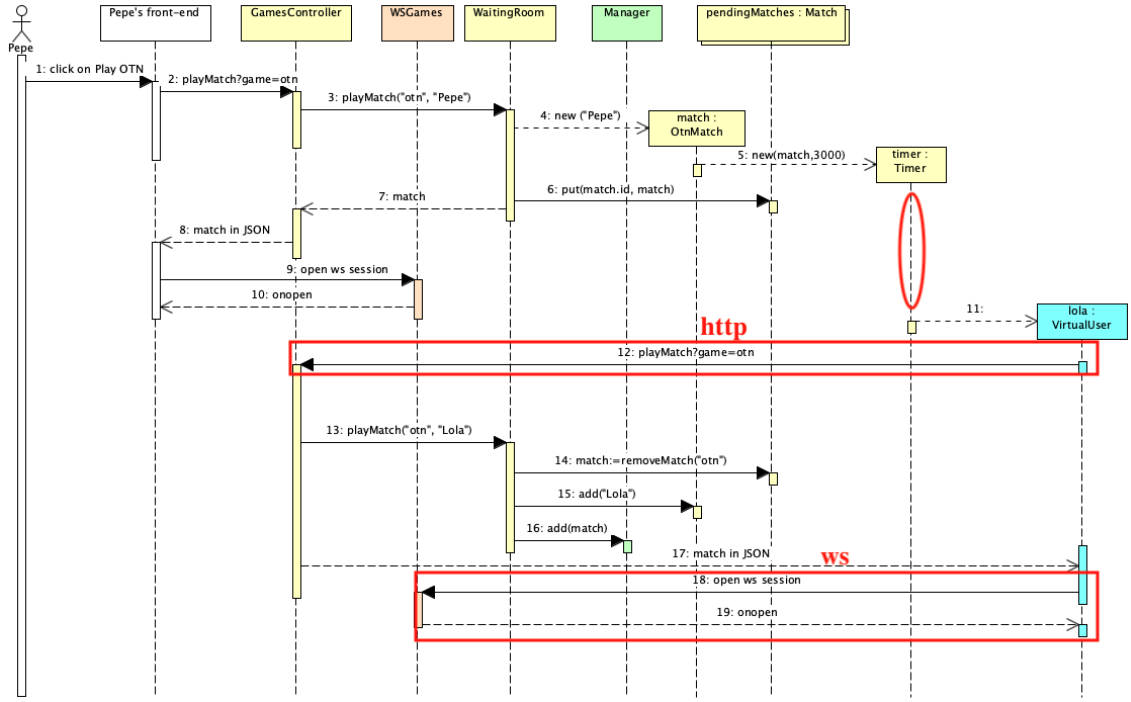


Figura 775. Lo que debe hacer el VirtualUser

El *VirtualUser* ha de ser también *Runnable* para que envíe movimientos de vez en cuando. Además, para funcionar conforme al diagrama de secuencia anterior, debe contar con un cliente *http* y un cliente *ws*. Aunque tenemos estos dos elementos (el *HttpClient* en el paquete *auxi*, y un *WSClient* en el subsistema de tests), en la Figura 776 los representamos como dos nuevas clases específicas para el *VirtualUser,* por lo que les anteponemos una "V" de "Virtual". Ya veremos si podemos reutilizar por completo alguna de las dos clases.

Con esta alternativa, el nuevo diseño de clases podría ser el de la Figura 776, en la que se ilustra cómo el jugador virtual utiliza su cliente *http* para acceder a los controladores (también al *UserController*, pues quizás sea necesario que se loguee) y el *ws* para el WebSocket.

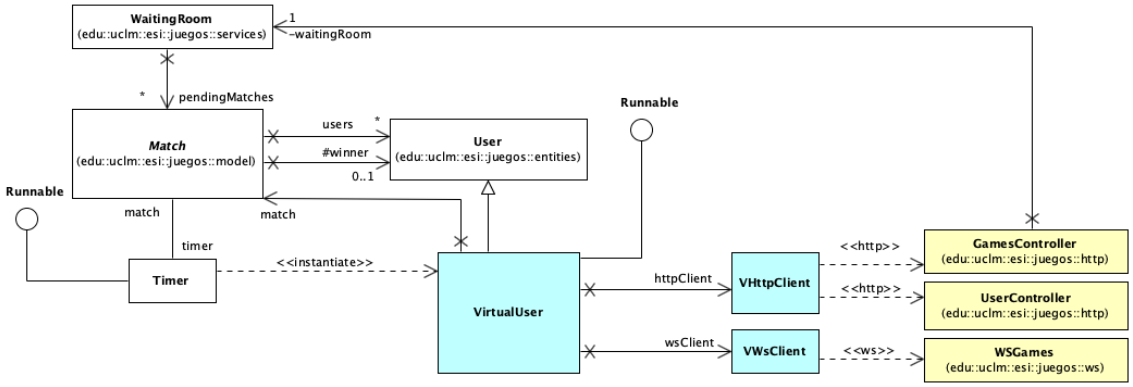


Figura 776. Posible diseño de clases para la alternativa 1

4.2. Alternativa 2: acceso directo a la partida

En la figura anterior vemos que tanto el *VirtualUser* y sus clientes, como los controladores y el *WSGames,* y el resto de elementos (*WaitingRoom, Match,* etcétera) residen en el mismo sistema, por lo que quizás podríamos simplificar el diseño eliminando los clientes y accediendo directamente desde el *VirtualUser* a la *WaitingRoom,* es decir:

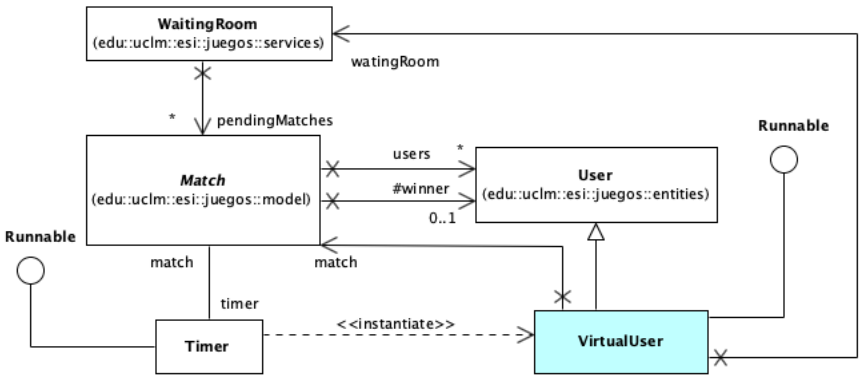


Figura 777. Posible diseño de clases para la alternativa 2

4.3. Elección de alternativa

Aunque la segunda alternativa resultaría sin duda mucho más sencilla, optaremos por elegir la primera, para volver a utilizar clientes *http* y *ws*. En cualquier, nótese que el jugador virtual sabe en qué partida está jugando (campo *match)* y que la propia partida lo estará conteniendo en su lista *users*.

5. Comportamiento del *VirtualUser*

Para que pueda unirse a la partida, el *VHttpClient* debe enviar al servidor los mismos mensajes que un usuario normal, de tal modo que pueda llegar a reproducirse el escenario de la Figura 775:

1) Loguearse, para lo que deberá estar previamente registrado. Su registro lo haremos a mano con un navegador. Crearemos una usuaria "Lola" con contraseña "lola123".
2) Enviar el mensaje al recurso */games/play* de *GamesController*. Para esto, deberá tener suficientes partidas pagadas, por lo que pondremos un valor muy grande directamente actualmente sobre la base de datos.
3) Enviar la solicitud de conexión con el WebSocket.
4) Posteriormente, debe poder enviar movimientos a la partida.

6. Posibles implementaciones del *VHttpClient*

Igual que sucede con un navegador (salvo que se diga explícitamente otra cosa), nuestro nuevo cliente *http* enviará todas las peticiones de forma asíncrona, para lo que usaremos, igual que en *auth,* la librería *OkHttp* (Figura 754, página 459).

Aunque esté feo hacerlo, copiaremos y pegaremos el método *send* que tenemos en el *AsyncHttpClient* (página 459 y completado en la 462), ya que nos sirve para enviar y recibir prácticamente cualquier cosa.

6.1. Alternativa 1: colocando pausas

Considerando que Lola debe loguearse y, luego, solicitar la partida del mismo juego que Pepe (el humano), una primera implementación del *VirtualUser* puede ser la siguiente, en la que:

1) Se declaran los dos campos *client*, aunque por ahora solo inicializamos el primero.
2) En *run* (que se invoca el *Timer* al crear y lanzar la *Thread)* le pedimos al *httpClient* que ejecute su método *login,* que espere 2 segundos y, luego, que ejecute *playMatch*. El parámetro que contiene el nombre del juego será *otn* o *wr,* y se obtiene reflexivamente preguntando por el nombre de la clase del campo *match*.

```
package edu.uclm.esi.juegos.model.virtual;

import edu.uclm.esi.juegos.entities.User;
import edu.uclm.esi.juegos.model.Match;

public class VirtualUser extends User implements Runnable {
   private Match match;
   private VHttpClient httpClient = new VHttpClient();
   private VWsClient wsClient;

   public VirtualUser(Match match) {
      this.match = match;
   }

   @Override
   public void run() {
      this.httpClient.login();
      this.pause(2000);
      this.httpClient.playMatch(this.getGameName());
   }
```

```
   private String getGameName() {
      String gameName = this.match.getClass().getSimpleName();
      int startMatch = gameName.indexOf("Match");
      gameName = gameName.substring(0, startMatch);
      return gameName.toLowerCase();
   }

   ... // método pause, como el de la Figura 425, página 267
}
```

Figura 778. Alternativa 1 de implementación

En este caso, el código de *VHttpClient* podría ser el de la Figura 779:

1) El método *login* envía la petición *put* de autenticación al recurso que escucha en el puerto 8000.
2) Tras la pausa a la que le fuerza el usuario, enviará la petición *get* a *playMatch*. Según la última implementación de este recurso (Figura 703, página 431), debemos enviar dos parámetros en la url: el nombre del juego y el *userId*. Es por esto por lo que declaramos el campo *lolaId*, que hemos obtenido de la base de datos.

```
package edu.uclm.esi.juegos.model.virtual;
...
public class VHttpClient {
   private String authUrl = "http://localhost:8000/";
   private String gamesUrl = "http://localhost/";
   private String lolaId = "29890de3-a84d-4498-a092-6fb46c7018d9";

   public void login() {
       JSONObject jso = new JSONObject()
           .put("name", "Lola")
           .put("pwd", "lola123");

       this.send("put", this.authUrl + "ausers/alogin", jso, null);
   }

   public void playMatch(String gameName) {
       String url = this.gamesUrl + "games/play?gameName=" + gameName +
              "&userId=" + this.lolaId;

       this.send("get", url, null, null);
   }

   public void send(String method, String url, JSONObject payload, Callback callback, String... headers) {
       ...
   }
}
```

Figura 779. login *y* playMatch *en* VHttpClient *para la alternativa 1*

6.2. Alternativa 2: creando *callbacks*

La alternativa anterior tiene la pequeña pega de esa pausa artificial de dos segundos: la petición se envía asíncronamente y suponemos que en ese tiempo habrá llegado la respuesta, por lo que podremos invocar con bastante seguridad a *playMatch*.

En una implementación mucho más elegante, podemos enviar la petición a *login* con una *callback* que envíe la petición a *playMatch* solo cuando se haya recibido la respuesta (Figura 780): al recoger esta en el *callback*, leemos los dos campos que nos envía el recurso *login* y los guardamos en los campos *userId* y *httpSessionId* de este cliente. A continuación, invocamos a *playMatch*, método cuya implementación sigue, de momento, como en la figura anterior.

```
public void login(String gameName) {
   JSONObject jso = new JSONObject()
      .put("name", "Lola")
      .put("pwd", "lola123");

   this.send("put", this.authUrl + "ausers/alogin", jso, new Callback() {
```

```
        @Override
        public void onResponse(Call arg0, Response response) throws IOException {
            JSONObject userHttpIds = new JSONObject(response.body().string());
            VHttpClient.this.lolaId = userHttpIds.getString("userId");
            VHttpClient.this.httpSessionId = userHttpIds.getString("httpSessionId");

            playMatch(gameName);
        }

        @Override
        public void onFailure(Call arg0, IOException arg1) {
            System.err.println(arg1.toString());
        }
    });
}

public void playMatch(String gameName) {
    String url = this.gamesUrl + "games/play?gameName=" + gameName + "&userId=" + this.lolaId;
    this.send("get", url, null, null);
}
```

Figura 780. login *y* playMatch *en* VHttpClient *para la alternativa 2*

Si regresamos por un momento a la Figura 610 (página 366) recordaremos que, cuando el *front-end* que implementamos en Angular recibe la respuesta al recurso *playMatch,* establece la conexión con el *WebSocket* del servidor. Como queremos imitar este comportamiento desde el *VirtualPlayer,* cuando ejecutemos recibamos la respuesta de la llamada al recurso *play* del *GamesController,* también debemos establecer la conexión con *WSGames*. Siguiendo con los *callbacks,* esto significa que el método *playMatch* de la figura anterior debe recibir también como parámetro una *callback* que cree la conexión *ws* cuando reciba la respuesta.

Es decir que, en lugar de tener simplemente *this.send("get", url, null, null)*, debemos hacer *this.send("get", url, null,* ***new Callback() ...****)*.

6.3. Elección de alternativa

La alternativa 2 es, en efecto, mucho más elegante que la primera: esas pausas artificiales pueden entrar en la categoría de los *bad smells* (literalmente, "malos olores"), que vienen a ser esas construcciones de código que, aunque funcionen, tienen un aspecto poco profesional y pueden estar escondiendo algún problema.

7. Implementación con *callbacks*

Si ejecutamos con el depurador y colocamos puntos de interrupción en los lugares adecuados, veremos que lo que llevamos programado del *VirtualPlayer* y de su cliente *http* se comportan perfectamente: Lola se loguea, solicita unirse al mismo juego que Pepe, y la partida se elimina de la sala de espera. Es en este momento cuando la instancia de *VirtualUser* debe conectar su cliente *ws* al servidor de WebSocket.

Dicho gráficamente, lo que esperamos que pase es lo de la Figura 781: cuando el cliente *http* recibe la respuesta, crea el cliente *ws*. Este escenario, sin embargo, no puede suceder con el diseño que hemos dado en el diagrama de clases de la Figura 776, porque el *VHttpClient* no conoce al *VWSClient* y, por tanto, no puede instanciarlo en el mensaje 7.

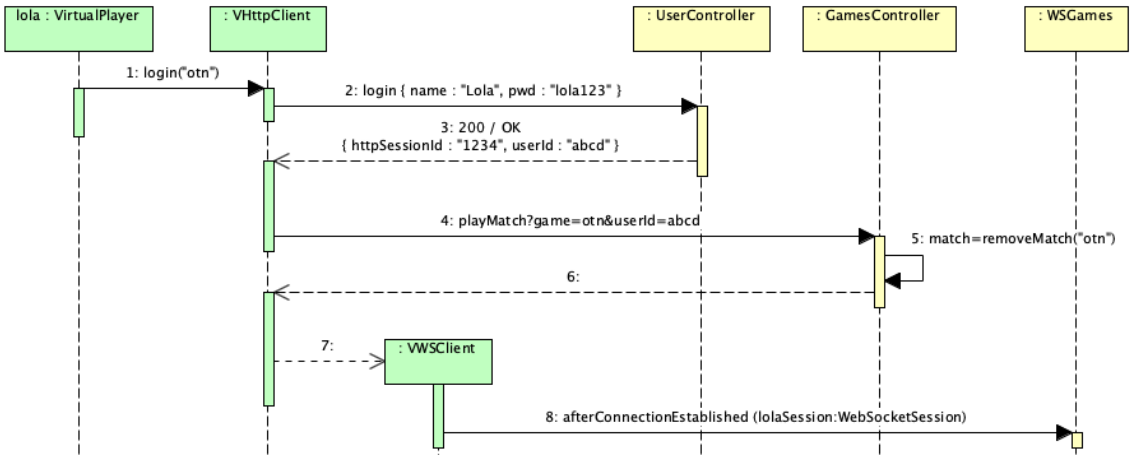


Figura 781. Posible escenario (incoherente con la Figura 776)

Para resolver este pequeño contratiempo podemos hacer dos cosas: (1) que *VHttpClient* conozca a *WSClient,* o (2) que la respuesta al mensaje 6 sea una llamada a la instancia *lola* para que esta cree el cliente *ws*. Es decir:

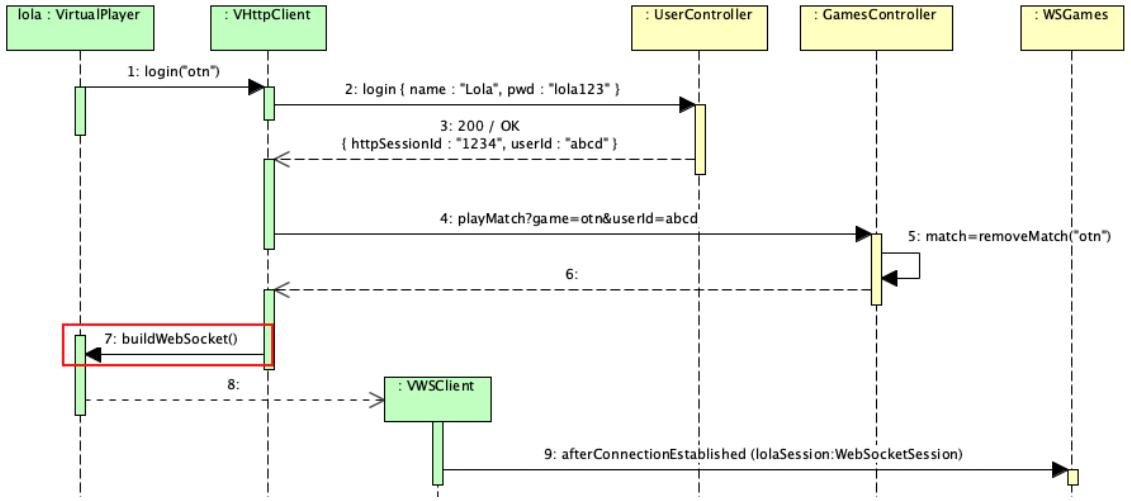


Figura 782. Posible escenario, algo más coherente con la Figura 776

Probablemente sea interesante que *lola* conozca a sus dos *proxies* de comunicación, por lo que optaremos por el segundo escenario. En el pie de la figura anterior hemos escrito "algo más coherente con la Figura 776", porque el escenario no llega a ser completamente compatible: en efecto, para que el *VHttpClient* le pueda decir cosas al *VirtualPlayer,* aquel debe conocer a este. Y, probablemente, más adelante nos daremos cuenta de que el *WSClient* también debe conocer a su usuario virtual asociado. Para curarnos en salud, haremos que ambos clientes conozcan al usuario al que están ofreciendo servicios de comunicaciones: nótese la sutil pero importante diferencia en la Figura 783, que implica que el código del constructor de *VirtualUser* deba modificarse (Figura 784).

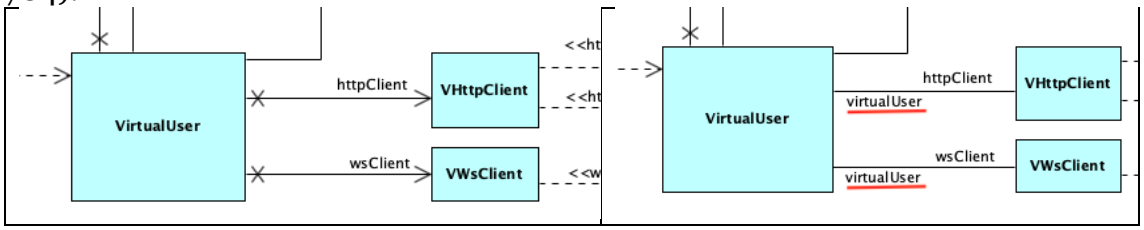


Figura 783. Los clientes conocen a su usuario (lado derecho)

```
public VirtualUser(Match match) {
   this.match = match;
   this.httpClient = new VHttpClient(this);
   this.wsClient = new VWsClient(this);
}
public VHttpClient(VirtualUser virtualUser) {
   this.virtualUser = virtualUser;
}
public VWsClient(VirtualUser virtualUser) {
   this.virtualUser = virtualUser;
}
```

Figura 784. El VirtualUser *informa sus dos clientes de comunicación del usuario para el que trabajan*

Ahora, cuando el *VHttpClient* recibe la respuesta a la petición enviada a *games/play* (la cual contiene un JSON con la información de la partida), sí puede decirle al cliente que inicie la conexión al *WebSocket* del servidor:

```
public void playMatch(String gameName) {
  String url = this.gamesUrl + "games/play?gameName=" + gameName + "&userId=" + this.lolaId;
  this.send("get", url, null, new Callback() {

    @Override
    public void onResponse(Call arg0, Response response) throws IOException {
      JSONObject jsoMatch = new JSONObject(response.body().string());
      VHttpClient.this.virtualUser.buildWebSocket(VHttpClient.this.httpSessionId, jsoMatch);
    }

    @Override
    public void onFailure(Call arg0, IOException arg1) {
      System.err.println(arg1.toString());
    }
  });
}
```

Figura 785. En onResponse *implementamos los mensajes 6 y 7 de la Figura 782*

Se marca un error de compilación en la llamada a *buildWebSocket*, ya que es un método que todavía no hemos implementado en *VirtualPlayer:* pues vamos al tema.

7.1. Conexión del *WebSocket*

El método *buildWebSocket* en el usuario virtual recibe como parámetro la información de la partida a la que Lola se acaba de incorporar. Al ejecutar este método, el *VirtualUser* debe establecer la conexión con *WSGames,* para lo que podemos recordar el diagrama de secuencia de la Figura 514 (página 320): al recibir la respuesta de *games/play,* abríamos la conexión *ws* y, cuando *WSGames* contestaba, en el manejador *onopen* le enviábamos el mensaje PLAYER READY.

Bien, pues podemos comenzar a hacer exactamente esto: en *buildWebSocket* simplemente conectamos el *wsClient*:

```
public void buildWebSocket(String httpSessionId, JSONObject jsoMatch) {
   this.wsClient.open(httpSessionId);
}
```

Figura 786. buildWebSocket *en* VirtualUser

Como la lectora recordará, pusimos bastante cuidado en la implementación del *WSClient* del subsistema de test (Capítulo 22, página 307): un cliente de *WebScoket* recibe mensajes que normalmente pasa a algún otro objeto para que los procese. Entre la Figura 492 y la Figura 494 se plantearon varias alternativas para que el *WSClient* pudiese pasar los mensajes recibidos a cualquier otro

objeto, y por eso creamos una interfaz *IWSReceiver* que independiza al cliente del objeto que brega con los mensajes recibidos.

En la situación actual, nos sirve perfectamente el *WSClient* que construimos en aquel capítulo, y simplemente deberíamos hacer que el *VirtualUser* pudiese procesar los mensajes recibidos por el cliente de *WebSocket*. Quiere esto decir que podemos:

1) Eliminar con tranquilidad la clase *VWsClient* que hemos añadido al subsistema de producción (Figura 787, izquierda).
2) Mover los ficheros *WSClient* e *IWSReceiver* desde el paquete *edu.uclm.esi.juegos.multi* del subsistema de *test* hasta el paquete, por ejemplo, *edu.uclm.esi.juegos.ws* del subsistema de producción (Figura 787, centro).
3) Corregir los errores de compilación y adaptar el código, cosa que ya no ilustramos a las alturas que estamos, como cantaban Los Chichos, «ni más ni menos» que en el Capítulo 32. A este respecto, simplemente apuntaremos que el *wsClient* espera un receptor, por lo que añadiremos la sentencia *this.wsClient.setReceiver(this)* al constructor del *VirtualUser:* el lado derecho de la Figura 787 nos da suficientes pistas sobre lo que debemos hacer.

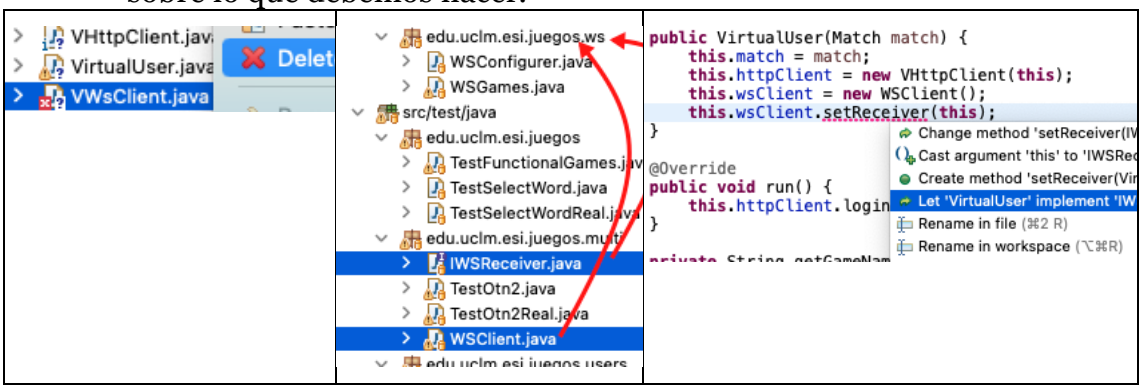


Figura 787. Usaremos el WSClient *que construimos en el Capítulo 22*

7.2. Solo falta seguir...

A partir de este punto, solo falta implementar muy poquito para que el sistema funcione. Mucho ánimo, querido o querida estudiante.

Capítulo 33. Dibujo de gráficos: formato SVG

1. Cálculo de π con el método de Montecarlo

Sabemos que la superficie de un círculo es $\pi \cdot r^2$. Si el radio del círculo es 1, entonces la superficie del círculo será directamente π. Si inscribimos un círculo de radio 1 en un cuadrado de lado 2, la figura resultante será del estilo de la siguiente:

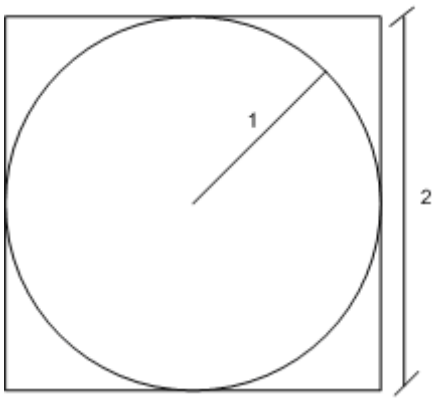


Figura 788. Circunferencia de radio 1 inscrita en un cuadrado de lado 2

Supongamos ahora que podemos lanzar dardos a ese cuadrado: si la cantidad de dardos que lanzamos es muy-muy grande, la proporción entre el número de dardos que caen dentro del círculo y el número total de dardos que hemos lanzado será muy parecida a la proporción existente entre el área del círculo y el área del cuadrado.

Como sabemos que el área del círculo es π (puesto que su radio es 1) y que la del cuadrado es 2·2=4, podemos asumir que es cierto que:

$$\frac{A_{círculo}}{A_{cuadrado}} = \frac{\pi}{4} \approx \frac{dardos_{dentro\ del\ círculo}}{dardos\ totales}$$

Si sabemos cuántos dardos hemos tirado y cuántos han caído dentro, entonces podemos calcular un valor aproximado para π:

$$\pi \approx 4 \cdot \frac{dardos_{dentro\ del\ círculo}}{dardos\ totales}$$

El número de dardos totales es tan fácil de calcular como establecer una variable a un determinado valor. Para contar el número de dardos que caen dentro del círculo podemos aplicar el Teorema de Pitágoras: si la suma de los cuadrados de las coordenadas del lugar en el que ha caído del dardo es menor que 1, entonces el dardo se ha clavado dentro de la diana; si no, ha ido fuera.

2. Implementación del cálculo

Generaremos un componente *pi* (*ng generate component pi*) en el que, de momento, declararemos un campo *shots* que indica el número de dados que vamos a tirar, el cual elegimos en la vista (véase la directiva *[(ngModel)]* en la Figura 789) y otro campo *pi* (de momento sin valor, de ahí la interrogación) al que asignaremos el valor que se calcule.

```
<input type="number" [(ngModel)]="shots" placeholder="Shots">
<br>
<button (click)="calculate()">Calculate</button>
Pi = {{ pi }}
```

```
export class PiComponent implements OnInit {
  shots : number = 1000
  pi? : number

  constructor() { }

  ngOnInit(): void {
  }

  calculate() {

  }
}
```

Figura 789. Vista y modelo del componente Pi

Cuando el usuario el botón *Calculate,* el modelo debe lanzar tantos disparos aleatorios como *shots* se hayan indicado, y aplicar entonces las fórmulas de la sección anterior.

Realmente, la implementación es sencillísima (Figura 790): por cada dardo se generan sus coordenadas *x* e *y,* se calcula el cuadrado de la hipotenusa (*h2)*, la hipotenusa *(h)* y se incrementa el número de dardos que han caído en la diana *(inside)* según el valor de *h*. Luego se calcula el valor de *this.pi*.

```
calculate() {
  let inside = 0
  for (var i=0; i<this.shots; i++) {
    let x = Math.random()
    let y = Math.random()
    let h2 = x*x + y*y
    let h = Math.sqrt(h2)
    if (h<=1)
      inside++
  }

  this.pi = 4 * inside / this.shots
}
```

Figura 790. Cálculo aproximado de π

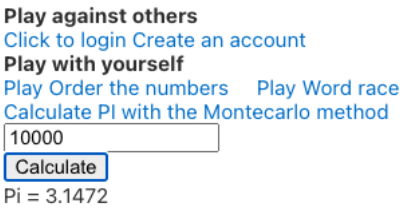


Figura 791. Valor calculado para π con 10.000 dardos

3. Tirando y dibujando los dardos

SVG (*Schalar Vector Graphics)* es un formato para generar y almacenar gráficos vectoriales. Muy resumidamente, si queremos almacenar una recta de color rojo sobre fondo blanco, en formato BMP necesitaríamos guardar todos los bytes que componen tanto el fondo como los puntos de la recta; SVG se basa en operaciones matemáticas, bastándole con recordar poco más que las coordenadas de origen y de fin y el color.

A modo de ejemplo, en la Figura 792 se muestra el código que permite dibujar el triángulo de color rojo que aparece a la derecha:

1) Se precisa un elemento de tipo *<svg>*.
2) Dentro de este, definimos un camino (*path)*. La definición de un camino puede ser bastante compleja y, afortunadamente, no nos ocuparemos de eso en este libro. Por lo que a nosotros respecta, el atributo *d="M0 0 L100 20 L50 60 Z"* hace lo siguiente:
    a. *M0 0* coloca el primer punto de dibujo en las coordenadas (0, 0). Ojo, porque SVG es sensible a mayúsculas y *M* no es lo mismo que *m*.
    b. *L100 20* dibuja una línea desde el punto actual (en este caso, el *(0, 0)* determinado por *M*) hasta las coordenadas *(100, 20)*. Es por esto por lo que el segundo punto se desplaza a la derecha (100 pixeles) y baja un poquito (20 pixeles).
    c. *L50 60* traza una línea desde el punto actual (ahora es *(100, 20)* hasta *(50, 60)*.
    d. El comando *Z* cierra el polígono.

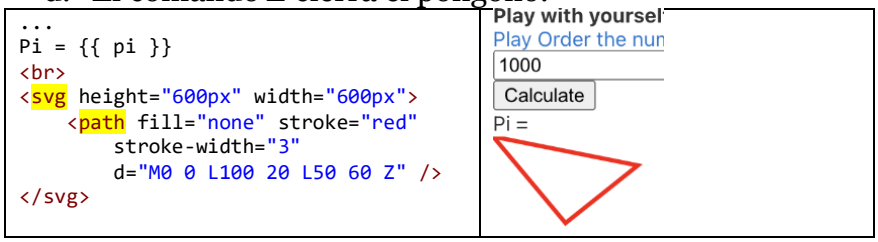


Figura 792. Un triángulo ocupa muy poquito en SVG

3.1. Dibujo del cuadrado de 2x2

En la figura anterior mostramos que el elemento *<svg>* tiene unas dimensiones de 600x600 pixeles. Para dibujar un cuadrado que ocupe todo el tamaño posible, sustituimos el *<path>* anterior por el de la Figura 793, que pinta un polígono que pasa por los puntos (0, 0), (600, 0), (600, 600), (0, 600) y termina cerrándose.

```
<svg height="600px" width="600px">
    <path fill="none" stroke="red"
        stroke-width="3"
        d="M0 0 L600 0 L600 600 L0 600 Z" />
</svg>
```

Figura 793. Dibujo del cuadrado

3.2. Dibujo del círculo de radio 1

SCG ofrece la etiqueta *<circle>*, que admite los parámetros *cx, cy* y *r* (respectivamente coordenadas del centro en *x,* en *y* y longitud del radio). Nuestro círculo estará centrado en nuestro cuadrado de 600x600 (estará en (300, 300)) y tendrá radio 300.

```
<svg height="600px" width="600px">
    <path fill="none" stroke="red"
        stroke-width="3"
        d="M0 0 L600 0 L600 600 L0 600 Z" />
    <circle cx="300" cy="300" r="300"
        stroke="red" fill="none"></circle>
</svg>
```

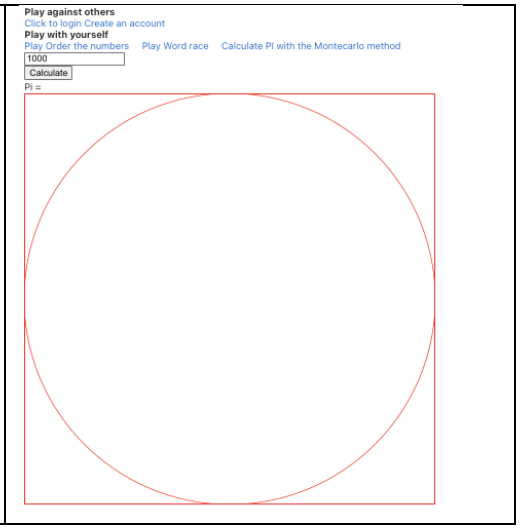


Figura 794. Dibujo del círculo

3.3. Dibujo de los dardos

Podemos dibujar cada dardo como un círculo de radio 1 en las coordenadas que le correspondan, multiplicadas por 600. Para dibujar un dardo debemos añadir al elemento <*svg*> un nodo hijo de tipo <*circle*>. Para localizar el <*svg*> con facilidad, le asignamos un *id* en el *html*.

```
calculate() {
  let inside = 0
  for (var i=0; i<this.shots; i++) {
    let x = Math.random()
    let y = Math.random()
    this.addPoint(x, y)
    let h2 = x*x + y*y
    let h = Math.sqrt(h2)
    if (h<=1)
      inside++
  }

  this.pi = 4 * inside / this.shots
}

addPoint(x: number, y: number) {
  let point : any = document.createElementNS("http://www.w3.org/2000/svg", "circle")
  x = x * 600
  y = y * 600
  point.setAttribute("cx", x)
  point.setAttribute("cy", y)
  point.setAttribute("r", 1)
  document.getElementById("lienzo")!.appendChild(point)
}
```

```
<svg height="600px" width="600px" id="lienzo">
    <path fill="none" stroke="red"
        stroke-width="3"
        d="M0 0 L600 0 L600 600 L0 600 Z" />
    <circle cx="300" cy="300" r="300"
        stroke="red" fill="none"></circle>
</svg>
```

Figura 795. addPoint *pinta el disparo*

Capítulo 34. Manejo de objetos binarios

1. Los usuarios, con foto

Supondremos que los usuarios pueden subir, si lo desean, una foto al registrarse. Almacenaremos la fotografía como una columna más de la tabla *users* del sistema *auth,* para lo que añadiremos un campo *picture* a la entidad *AUser*. El tipo del campo y de la columna ha de permitir almacenar el objeto, que puede ser muy grande. Optaremos por almacenar la foto como una cadena en base64, que ya sabemos que es fácilmente desencriptable. Si declaramos el campo como *String* y, además del *get* y del *set,* le añadimos las dos anotaciones resaltadas, Spring añadirá la columna a la tabla asociada a esta entidad. Nótese que no nos serviría un *varchar,* cuya longitud máxima es de 255 caracteres.

```
public class AUser {
    @Id @Column(length = 36)
    private String id;
    ...
    @Lob
    @Column(columnDefinition = "longtext")
    private String picture;
    ...
```

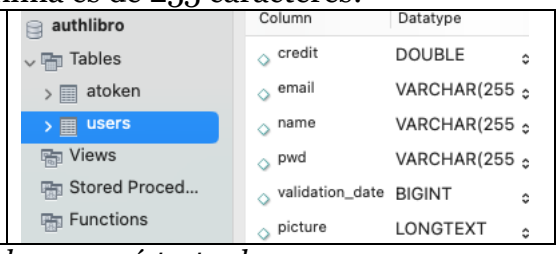


Figura 796. picture *almacenará textos largos*

2. Tratamiento del binario en el *front*

Para que el usuario pueda seleccionar la foto que desea subir, colocaremos un *widget* con *type="file"* en la vista del componente *register* (es decir, en *register.component.html*), que debemos bindear a un campo *picture* del *RegisterComponent*.

Además, para que le foto pueda previsualizarse antes de subirla, colocaremos también un *widget* de tipo *<img>*.

El código de la vista y el modelo se muestra en la Figura 797, en donde se resaltan los elementos que se han añadido:

1) En la vista, se ha programado el manejador del evento *onchange* en el *<input type="file">* mediante la función *setPicture* del modelo, y se ha añadido el *<img>* con su campo *src* enlazado al campo *picture* del modelo.
2) En el modelo, el método *register* añade la foto en la *payload* del mensaje que envía al servidor.
3) Por último, el método *setPicture* maneja el evento *onchange* del *file* de la vista. El evento que se recibe tiene un campo *target* que hace referencia al elemento sobre el que se ha producido el propio evento. Como se trata de un *file,* podemos acceder al fichero seleccionado mediante *files[0]*.
   Este manejador declara un objeto de clase *FileReader,* que es un objeto estandarizado que incluye varios métodos para leer ficheros (entre ellos el *readAsBinaryString)* y un manejador *onload*.
   La función asociada a este manejador se ejecuta cuando *readAsBinaryString* ha terminado de leer el contenido del fichero, que se coloca en el campo *result* del *reader*. En el ejemplo, el contenido se

convierte a base64 y se le coloca el prefjo ** (acabado en coma), con lo que se le instruye el navegador para que sepa que la imagen está codificada en este formato.

```
<input type="text" [(ngModel)]="name" placeholder="User's name">
...
Picture: <input type="file" (change)="setPicture($event)">
<br>
<img [src]="picture" *ngIf="picture">
<br>
<button (click)="register()">Create account</button>
...
```

```
import ...

declare let btoa : any

@Component(...)
export class RegisterComponent implements OnInit {
  name? : string = "Pepe"
  ...
  picture? : string
  error? : string

  @Output() onUserRegistered : EventEmitter<any> = new EventEmitter()

  constructor(private userService : UserService) { }
   ...
  register() {
    let user = {
      name : this.name, pwd1 : this.pwd1,
      pwd2 : this.pwd2, email : this.email,
      picture : this.picture
    }
    this.userService.register(user).subscribe({
      ...
    })
  }

  setPicture(event: any) {
    let self = this
    let file : File = event.target.files[0]
    let reader = new FileReader()
    reader.onload = function () {
      let base64 = btoa(reader.result)
      base64 = "" + base64
      self.picture = base64
    }
    reader.readAsBinaryString(file);
  }
}
```

Figura 797. Vista y modelo del componente de registro

El lado izquierdo de la siguiente figura muestra el aspecto del navegador cuando Pepe va a crearse una cuenta y selecciona su fotografía (en este caso es una foto bajada de *thispersondoesnotexist.com)*. A la derecha, se lee el campo *picture* que ha enviado el cliente en la *payload* y se pasa al *register* del *UserService*, para que la guarde también en la base de datos.

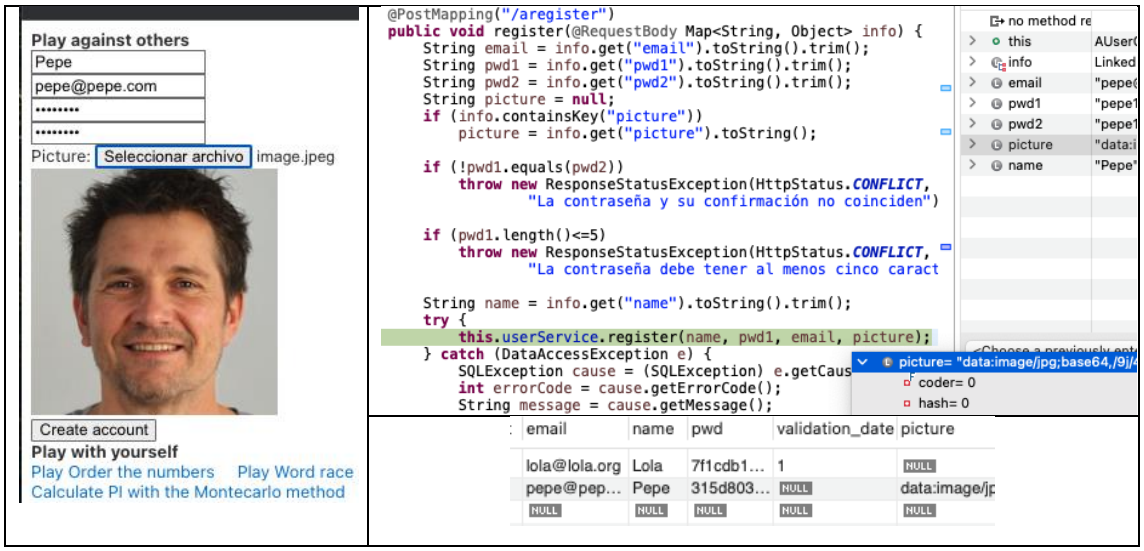


Figura 798. Pepe añade su fotografía y el servidor, que la guarda en forma de cadena

3. Tratamiento del binario en el *back*

Si solo queremos almacenar y recuperar la fotografía, poco más tenemos que hacer en el servidor de lo que hemos hecho. Supondremos que queremos almacenar la fotografía original que ha subido el usuario, y una versión de, por ejemplo, 100x100 pixeles para mostrar cuando el usuario haga login.

Hay muchas librerías para hacer tratamiento de imágenes. Para escalar podemos añadir al *pom.xml* la siguiente dependencia:

```
<dependency>
    <groupId>org.imgscalr</groupId>
    <artifactId>imgscalr-lib</artifactId>
    <version>4.2</version>
</dependency>
```

Figura 799. Una dependencia para escalar imágenes

Crearemos un nuevo cambo *thumbnail* en *AUser*, también *String* y, en esta ocasión, lo anotaremos con *@Column(columnDefinition = "mediumtext")*. En el método *register* del *UserService* utilizaremos la librería recién descargada para construir una versión del tamaño deseado:

```
@Transactional
public void register(String name, String pwd, String email, String picture) throws Exception {
   AUser user = new AUser();
   user.setName(name);
   user.setPwd(pwd);
   user.setEmail(email);

   if (picture!=null && picture.length()>0) {
      user.setPicture(picture);
      String thumbnail = PictureCompressor.compress(picture, 100);
      user.setThumbnail(thumbnail);
   }
   ...
```

Figura 800. Creación de la foto en miniatura

La clase *PictureCompressor* la hemos escrito nosotros y es un *wrapper* que nos facilita el manejo de la dependencia: en efecto, la dependencia trabaja directamente con imágenes en formato binario y no con cadenas, por lo que debemos, en primer lugar, quitarle a la cadena el prefijo *data:image/...*, luego decodificarla para pasarla a bytes, escalarla, recodificarla a base64 y añadirle el prefijo.

```java
package edu.uclm.esi.auth.auxi;

import java.awt.image.BufferedImage;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.util.Base64;

import javax.imageio.ImageIO;

import org.imgscalr.Scalr;

public class PictureCompressor {
  private static final int SIZE = 10_000;

  public static String compress(String original, int thumbSize) throws Exception {
     if (original.length()<SIZE)
          return original;

     String thumbnail = original.substring(22);
     BufferedImage before =
          ImageIO.read(new ByteArrayInputStream(Base64.getDecoder().decode(thumbnail)));

     ByteArrayOutputStream baos = new ByteArrayOutputStream();
     BufferedImage after = Scalr.resize(before, Scalr.Method.AUTOMATIC, thumbSize);
     ImageIO.write(after, "png", baos);
     thumbnail=""+Base64.getEncoder().encodeToString(baos.toByteArray());
     return thumbnail;
  }
}
```

Figura 801. Escalado de la imagen

A modo de prueba, y para ver el resultado en el cliente, el método *login* debe añadir a su respuesta (actualmente el *httpSessionId* y el *userId)* las cadenas en base64 contenidas en los campos *picture* y *thumbnail*. Cuando recibamos la respuesta en el *front* (tenemos el código en la Figura 698, página 427), debemos modificar la emisión del evento *onUserLogged* para que emita no solo el nombre del usuario, sino también sus dos fotos:

```typescript
login() {
  let user = {
    name : this.name,
    pwd : this.pwd
  }

  this.userService.login(user).subscribe({
    next : (response : any) => {
      sessionStorage.setItem("userId", response.userId)
      sessionStorage.setItem("httpSessionId", response.httpSessionId)
      this.error = undefined
      this.onUserLogged.emit({
        name : this.name,
        picture : response.picture,
        thumbnail : response.thumbnail
      })
    },
    error : (result : any) => {
      console.log(result.error.error)
      this.error = result.error.error
    }
  })
}
```

Figura 802. El evento envía un JSON con el nombre y ambas fotos

El evento *onUserLogged* lo maneja el método *showMultiplayer* que teníamos en el *AppComponent:* en este método debemos recoger ahora los dos campos adicionales que nos llegan, asignarlos a dos campos y mostrarlos en la vista:

```
showMultiplayer(user : any) {
  this.userName = user.userName
  this.picture = user.picture
  this.thumbnail = user.thumbnail
  ...
}
```

```
<div *ngIf="showingMultiplayer">
    Logged as {{ userName }}
    <br>
    <img [src]="picture">
    <img [src]="thumbnail">
    <app-multiplayer ...></app-
multiplayer>
  </div>
```

Figura 803. La foto original y su miniatura

ÍNDICE DE TÉRMINOS

?, 139
@After, 158
@AutoConfigureMockMvc, 87
@Autowired, 87, 93, 286, 462
@Bean, 445
@Before, 158
@BeforeAll, 184
@BeforeEach, 104, 158, 183
@Column, 217
@Component, 128
@Configuration, 445
@CrossOrigin, 145
@CrossOrigins, 210, 272
@CsvSource, 223, 227
@DeleteMapping, 230
@DisplayName, 221
@Entity, 217
@GetMapping, 89, 109
@Id, 96, 122, 217
@Index, 220
@OnClose, 298
@OnError, 298
@OnMessage, 298
@OnOpen, 298
@Order, 162
@Param, 177
@ParameterizedTest, 222
@PutMapping, 228
@Query, 177
@RequestBody, 211
@RequestMapping, 89, 211
@RequestParam, 89
@RestController, 89
@ServerEndPoint, 298
@SpringBootTest, 86
@Table, 220
@Test, 86, 104, 132, 158
@TestInstance, 184
@TestMethodOrder, 162, 223
@Transactional, 380
@Transient, 100, 122
@WebListener, 320
**ngFor*, 66, 164
**ngIf*, 74
101, 299, 301
200, 33, 93, 104, 107, 108, 131, 147, 167, 189, 240
304, 384
401, 39, 442
403, 39, 214, 228, 240, 277
404, 108, 131, 146, 167
406, 389
409, 221, 240
500, 148, 222, 240, 272
Access-Control-Allow-Credentials, 274
Access-Control-Allow-Origin, 145, 274
Access-Control-Expose-Headers, 279
Access-Control-Request-Method, 276
actor, 209
actores, 201
Adamo, Salvatore, 219
adaptadores de entrada, 24
adaptadores de salida, 24
afterConnectionClosed, 319
afterConnectionEstablished, 298, 300, 304
  problemas con dos servidores, 433
Alert, 262
All combinations, 245
Allow, 275
Alternativas de diseño, 199, 200, 285, 292, 358, 409, 427, 457, 468, 471, 474
  en herencia, 327
andExpect, 105, 107
andReturn, 107
Angular, 53, 133
  ?, 139
  (click), 68
  *ngFor, 66, 164
  *ngIf, 74
  app.module.ts, 136
  componentes, 271
  Componentes, 58, 138
  Conexión por *ws*, 357
  EventEmitter, 355

eventos, 355, 362, 363
FormsModule, 258
HttpClient, 135
interrogación, 139
inyección, 141
Inyección, 137
ngModel, 260
ngOnInit, 139
observable, 140
Observables, 151
observe, 280
paso de mensajes, 365
Servicio, 141
servicios, 134, 259
Servicios, 140
suscripción a observables, 141
any, 71, 140, 152, 279, 356
anyRequest, 446
apache, 454
Apache, 212
Apache Tomcat, 33
Apache validator, 213
aplicaciones SPA, 21
app.component.ts, 133
app.module.ts, 136
application.properties, 78, 87, 103, 119, 183, 441, 460
ApplicationFilterChain, 403
arquetipo, 76
arquitectura, 15, 75, 108, 409
imposición de un estilo arquitectónico, 108
Arquitectura, 113
Arquitectura cliente-servidor, 17, 19
en web, 19
Arquitectura de *pipes and filters*, 25
Arquitectura de puertos y adaptadores. *Véase* Arquitectura hexagonal
Arquitectura hexagonal, 23
Arquitectura Modelo-Vista-Controlador, 22
arquitectura monolítica, 409
Arquitectura multicapa, 22
Arquitectura MVC, 22
artifactId, 82
ASP.NET_SessionId, 230
assertEquals, 227
assertTrue, 160
Authorization, 440, 460
authorizeHttpRequests, 446
Azure Data Studio, 117, 129, 176, 219, 226
Azure Studio, 190
backend, 41, 75, 134, 197
servicios, 215
bad smells, 473
Baeldung, 212
Base de datos
en memoria, 103
base64, 440, 482
en WebSockets, 298
Base64, 278
base64 (en WebSockets), 299
bases de datos, 102
Bases de datos
Código de error, 224, 417
dialectos, 176
Pool de conexiones, 171
protección, 438
url de conexión, 103, 412
BEARER, 460, 462
binding, 55, 57, 58
Binding, 65
body, 32
búsqueda binaria, 192
cabecera
origin, 273
cabecera (en cuanto a cookies), 42
cabeceras, 32, 230, 278
Cabeceras, 144
Access-Control-Allow-Credentials, 274
Access-Control-Allow-Origin, 274
Access-Control-Expose-Headers, 279
Access-Control-Request-Method, 276
Allow, 275
Authorization, 460
Connection, 298
content-type, 275
Cookie, 309
en los Websockets, 433
origin, 437
Sec-WebSocket-Accept, 299
Sec-WebSocket-Key, 298
Set-Cookie, 276
Upgrade, 298
caché, 178, 181, 192
cada valor, 240
cadena mágica, 298
callback, 457, 472
Callback, 454
capa de aplicación
en Arquitectura hexagonal, 24
Capa de dominio, 22
Capa de negocio, 22
Capa de persistencia, 22
Capa de presentación, 22
caso de uso abstracto, 203
casos de prueba
a partir de máquinas de estado, 248
Casos de uso, 79
cast, 333
chunked, 34
cierre de sesiones, 319
Clases abstractas, 327
Clases de equivalencia, 239
Class, 290
clave alternativa, 190
clave principal, 97
Clave principal, 96
clave publicable, 385
clave secreta, 385
Claves API, 378
cliente http en el backend, 382
cliente-servidor, 197

CloseableHttpClient, 454
cobertura, 237
Cobertura
  Todas las transiciones, 249
  todos los caminos interesantes, 249
  Todos los estados, 249
  Todos los pares de entrada y salida, 249
Código
  401, 442
Código 101, 299, 301
código 200, 93, 147
Código 200, 33, 104, 108, 131, 167, 189
Código 304, 384
Código 401, 39
Código 403, 39, 214, 228
código 404, 131, 146
Código 404, 108, 167
Código 406, 389
Código 409, 221
Código 500, 148, 222, 272
Código de error de a base de datos, 224
código de respuesta, 32, 105
Códigos de respuesta, 38
combinaciones, 241
Common Vulnerabilities and Exposures, 438
Componentes en Angular, 58
ConcurrentHashMap, 287
Conjetura de errores, 239
Connection, 298
contenedor, 20
content-type, 275
Content-Type, 35
controlador, 89, 211
controladores, 82
Controladores, 75
Convenciones de nombrado en Spring, 112
cookie, 231, 274
Cookie, 309
COOKIE, 434
cookies, 42
Cookies de terceros, 43
Correo electrónico
  Formato, 212
CORS, 144, 271, 274, 401
coste de las pruebas, 254
CountDownLatch, 316
Coverage, 237
Create, 40
create table, 103, 121
Create, Read, Update and Delete, 40
Cross-Origin Resource Sharing, 144, 274
CRUD, 40
CrudRepository, 111, 233
cssSelector, 157
cuerpo del mensaje, 32
CURL, 378, 437
CVE, 438
dao, 89
DAO, 80
Data Access Object, 80
Data Definition Language, 103, 118
DataAccessException, 224
DataIntegrityViolationException, 223, 224
datos de prueba, 240
  cobertura, 240
Datos distribuidos, 19
DB2, 176
DDL, 103, 118, 120, 219
ddl-auto, 103, 121, 129, 219, 226
DefaultSecurityFilterChain, 445, 446
delete, 229
Delete, 40
DELETE, 272
deleteById, 233
delimiter, 222
Dependencias, 82, 84
deprecated, 148, 213
depuración, 91
Depuración, 143, 267, 318, 342, 453
  de varios procesos, 267, 453
  Step over, 269
Depuración en el navegador, 64
dialecto, 120
dialecto de SQL, 120
diseño arquitectónico, 15
DispatcherServlet, 404
docker, 115, 117, 125, 169, 237, 341
Docker Desktop, 115
Document Object Model, 139, 164
doFilter, 402
doFilterInternal, 403
DOM, 139, 164
dominio, 22
driven adapters, 24
driver, 155, 158, 263
drivers, 155
driving adapters, 24
Each use, 240
Eclemma Code Coverage, 237
Eclipse, 76, 455
  Depuración, 267
  errores, 85, 87, 121
  Errores, 143, 158, 178, 210, 219, 220, 317
  Market Place, 237
  Procesos, 267
  Recursos, 131
  Refactorización, 216
  Referencias a un elemento, 455
Encriptación, 217
endpoint, 168, 297
entidades, 89
Entidades, 75
entities, 89
Entity, 121
Errores, 143, 369
  por sesiones http diferentes, 426
escarabajo, 267
Estereotipos, 200, 201
estilo arquitectónico, 15
EventEmitter, 355, 484

Excel, 123
excepción, 121
excepciones, 110
Excepciones, 347
  Excepciones personalizadas, 333
  propagación, 223
  throws, 223
expresiones regulares, 212, 214
fail, 310
Field (en reflexión), 290
FileReader, 481
FilterChain, 402
Filtros, 401
  manipulación, 405
Filtros de seguridad, 446
findAll, 99, 174, 175
findById, 98
Flujo de un pago, 386
  con dos servidores, 429
forbidden, 277
FormsModule, 258
front, 108
front-end, 89, 93, 133, 197, 205
GameRuleViolationException, 333
get, 108
getAttribute, 232
getClass, 290
getContentAsString, 106, 107
getDeclaredField, 290
getDeclaredMethod, 290
getErrorCode, 224
getHandshakeHeaders, 434
h2, 103, 120
handleBinaryMessage, 298
handleTextMessage, 298, 300, 347
handshake, 169, 298, 434
hardcodear, 382
headers, 32
Herencia, 325
  regla del 100%, 359
  Regla del *is a*, 359
hilo, 467
HttpClient, 135, 140, 454
https, 20
HttpSecurity, 445
HttpServletRequest, 281
HttpServletResponse, 278
HttpSession, 230, 271, 304
  Descripción, 232
HttpSessionListener, 320
IBM, 176
IDE, 53, 156, 173
Idempotencia, 106
IIS, 20
índice único, 190
Índices, 220
índices en tablas, 190
insert into, 125
Integración de correo electrónico, 377
Integración de servicios de pago, 385
Integrated Development Environment, 53
interceptores, 20, 81, 91, 102, 301
Internet Information Server, 20, 232
interrogación, 139
inyección, 89, 141, 445
Inyección, 137, 231, 427
  de un JSON en un Map, 211
  de un objeto, 427
IOException, 110
ip, 89
is a, 359
is4xxClientError, 108
isNotFound, 108
isPresent, 111
jakarta, 76, 96, 231
Java Development Kit, 76
Java Persistence API, 87
java.awt.Point (para almacenar un par de números), 343
JavascriptExecutor, 157
javax, 96
javax.websocket, 298, 303, 304
JBoss, 20, 81, 232
JDK, 76
JMeter, 167, 185
  $, 186
  creación de variables, 186
  JSON Extractor, 186
  Latencia, 169
  lectura de respuestas, 186
  Listener, 169
  Loop controller, 189
  plan de pruebas, 188
  Plan de pruebas, 167
  Results tree, 171
  Sampler result, 169
  Summary report, 170
  Thread group, 167, 170
  uso con un profiler, 175
jpa, 87
JpaRepository, 97, 111, 218
JSESSIONID, 304, 309, 435
JSON, 21, 32
JSON Extractor, 186
JUnit, 77, 82, 92
  @After, 158
  @AfterEach, 158
  @Before, 158
  @BeforeAll, 184
  @BeforeEach, 104, 158, 183
  @CsvSource, 223, 227
  @DisplayName, 221
  @Order, 162
  @ParameterizedTest, 222
  @Test, 104, 132, 158
  @TestInstance, 184
  @TestMethodOrder, 162, 223
  assertEquals, 227
  assertTrue, 160
  errores, 233
  Errores, 158, 164, 220
  fail, 310

LifeCycle, 184
oráculos, 160
Parametrización de casos, 222
veredicto, 183
Veredicto, 164
Júpiter, 77, 82
Kruchten, 16, 197
Latencia, 169
línea de estado, 32
Listener, 169
localStorage, 279
Lógica distribuida, 18
Loop controller, 189
Los Chichos, 476
Luis Enrique, 439
malos olores, 473
Manager, 127, 179, 278, 292, 311, 372, 426
manejador, 148
manejador de eventos, 29, 38
Map y JSON, 211
máquinas de estado, 248
Matcher, 214
Maven, 76
application.properties, 78
arquetipo, 76
carpetas, 78
dependencies, 82
pom.xml, 77
recursos, 78
repositorios, 84
MC/DC, 246
Method (en reflexión), 290
método (en http), 32
Mi gran noche, 219
Microarquitectura, 61
Microservicios, 409
MIME, 35, 356
mock, 86
MockHttpServletResponse, 106
MockHttpSession, 272, 295, 309
MockMvc, 86, 183, 234, 258, 266, 303
MockMvcRequestBuilders, 88, 105
Modelo-Vista-Controlador, 55
Modified Condition-Decision Coverage, 246
monkey test, 185
multicapa, 22
Multipurpose Internet Mail Extensions (MIME), 35
MVC, 22, 55
MvcResult, 106
mvnrepository, 77
MySQL, 122, 176, 411
Creación de una base de datos, 413
Creación de usuarios, 413
Instalación, 412
Workbench, 412
nativeQuery, 177
Navegador
Depuración, 143, 159, 275
Inspeccionar, 159, 363
Network, 275
Recuperar el XPATH, 159
Network, 275
newInstance, 290, 291, 431
next, 149, 355
ng, 53, 259
ng serve, 56
ngModel, 260
ngOnInit, 139, 141, 150
Ni más ni menos
Los Chichos, 476
NodeJS, 21
NullPointerException, 233
objetos binarios, 481
observable, 140
observe, 280
obsolescencia, 148, 213
Obtención de todas las salidas, 240
OkHttpClient, 454
OncePerRequestFilter, 401
onchange, 30
onclose, 297
onerror, 297
onload, 481
onmessage, 297, 299
onopen, 297, 299, 357, 371
onreadystatechange, 38
Open JDK, 76
Optional, 110
OPTIONS, 275, 276, 404
Oracle, 176
oráculo, 107
oráculos, 210, 250
orégano, 41
origen, 144, 231, 273, 401
origin, 144, 231, 273, 437
OSI, 34
PagingAndSortingRepository, 98
Pairwise, 241
Parametrización de casos de prueba, 222, 227
Partes y contrapartes, 36
partición, 239
Paso de mensajes, 93
patch, 108
Patrones
Estado, 365
MVC, 55
Singleton, 127
State, 361
Template- method, 323
Wrapper, 310
pattern, 214
payload, 32, 108, 144, 210, 228, 272
para WebSockets, 301
persistencia, 22
petición asíncrona, 30
Peticiones
asíncronas, 452
síncronas, 451
pipes and filters, 25
Plain Old Java Object, 99

plan de pruebas, 188
Plan de pruebas, 167
POJO, 99, 238
pom.xml, 77, 82, 83, 85, 96, 102, 119, 155
Pool de conexiones, 171
post, 108
PostgreSQL, 176
preflight, 275, 276, 277, 401, 403
Presentación distribuida, 18
Presentación remota, 17
primary key, 122
Proceso de Unificado de Desarrollo, 15
profiler, 167, 173
Profiling
  con JMeter, 175
programación reflexiva, 471
Programación reflexiva, 289, 430
protocolo como un idioma, 33
protocolo *ws*, 297
Protocolo *ws*, 44
proxy, 474
pruebas de regresión, 183
Pruebas de regresión, 254, 277
Pruebas de rendimiento, 167, 185
Pruebas re regresión, 220
puerto, 89, 117
Puerto 443, 20
Puerto 80, 20
puertos
  en arquitectura hexagonal, 24
put, 108
QueryByExampleExecutor, 98
Raphael, 219
rar, 122
Read, 40
readAsBinaryString, 481
readyState, 38, 297
redireccionamiento, 384
Reejecución, 183, 220, 254, 277
refactorización, 177
Refactorización, 70, 175, 190, 216, 250
  de la base de datos, 190
reflexión, 471
Reflexión, 289, 430
regla del 100%, 359
Reglase de integridad, 220
renderizador, 29
renderizar, 19, 34
rendimiento, 167
repositories, 84
repositorio, 95
repositorios, 89
Repositorios, 75
request, 32, 108, 280
  cabeceras, 280
Request headers, 230
Request Headers, 273
requestMatchers, 446
response, 32, 104, 106, 144, 227
  getContentAsString, 106
Response headers, 230
Response Headers, 273
ResponseStatusException, 110
respuesta, 144
Restricciones de integridad, 223, 245
ResultActions, 105
Results tree, 171
RFC 5322, 212
rmi, 24
router, 203, 205
run(), 452
Runnable, 452, 467
RuntimeException, 110, 224
Salvatore Adamo. *Véase* Adamo, Salvatore
Sampler result, 169
Sánchez Crespo, Luis Enrique, 438
scope, 119
script, 145, 155
Sec-WebSocket-Accept, 299
Sec-WebSocket-Key, 298
Selenium, 155, 183, 262, 293
  Alert, 262
  cobertura, 237
  dependencia, 155
  Diálogos, 262
  driver, 263
  Errores, 164
  eventos, 159
  switchTo, 263
  tiempo, 164
  WebElement, 159
Selnium
  alert, 263
send (en WebSocket cliente), 297
sendinblue, 378
Serializable, 232
services, 89
servicio, 134
servicios, 89
Servicios, 75
Servidor de aplicaciones, 81
  Internet Information Server, 232
  Tomcat, 90
servidor de aplicaciones web, 20
servidor web, 20
sessionDestroyed, 320
sessionStorage, 279, 355, 357, 422, 426
setAttribute, 232
Set-Cookie, 230, 276
setMaxInactiveInterval, 321
setUp, 157, 158, 163, 184, 308
SHA-512, 217, 229, 240
síncrona, 451
Sincronización, 318, 343
Single Page Application, 21
Singleton, 127
Sistema de datos remoto, 18
SMTP, 257, 378
socket, 29
span, 260
split, 146
Spring

@Autowired, 93, 95
@Column, 217
@Component, 128
@CrossOrigin, 145
@DeleteMapping, 230
@Entity, 96, 217
@Id, 96, 217
@Index, 220
@Param en repositorios, 177
@PutMapping, 228
@Query, 177
@RequestBody, 211
@RequestMapping, 89, 211
@RequestParam, 89
@RestController, 89
@Service, 95
@Table, 220
@Transactional, 380
@Transient, 100
@WebListener, 320
Controladores, 75
Convenciones de nombrado en Spring, 112
CrossOrigins, 210
Dependencias, 82
entidad, 95
Entidades, 75
Errores, 210
funcionamiento, 101
interceptores, 81, 102
inyección, 89, 231
JPA, 87
JpaRepository, 97
nativeQuery, 177
recursos, 103, 382, 417
repositorio, 95
repositorios, 97
Repositorios, 75
Servicios, 75
Spring Boot, 75
arranque, 94
spring.security.user.name, 460
spring.security.user.password, 460
SpringApplication, 128
SQL, 112
SQL Server, 115, 169, 176, 217
SQLException, 224
SQLServerDriver, 120
SQLServerException, 223, 224
stackoverflow, 213
StackOverflow, 148
stakeholders, 15, 199
start, 452
State, 361
stateless, 42
Step over, 269
storages, 279
subscribe, 148
subtipos, 325
Summary report, 170
SUT, 239
SVG, 477
Switching protocols, 299
switchTo, 263
synchronized, 318, 343
System Under Test, 239
tabla *hash*, 286
tablas de pares, 241
TCP, 33
TDD, 209, 257
tearDown, 158, 163
telnet, 33
Template- method, 323
Template-method, 328
Template-Method, 345
Test Driven Development, 209
Test-Driven Development, 75, 257
texto plano, 217
Thread, 452, 467
Thread group, 167
Throwable, 224
throws, 110, 336
timeout, 454
todas las combinaciones, 245
Todas las transiciones, 249
todos los caminos interesantes, 249
Todos los estados, 249
todos los pares, 241
Todos los pares de entrada y salida, 249
Tomcat, 20, 33, 81, 90, 232
Transfer-Encoding, 34
TypeScript, 53, 57, 62, 68
any, 71, 140, 279, 356
UML, 15, 127
Unicode, 212
Update, 40
Upgrade, 298
user agent, 19
user-agent, 29, 81, 117
UTF-8, 124
UUID, 217
Valores límite, 239
valores *null*, 96
varchar, 96, 217
Vary, 275
verbo
(en http), 32
veredicto, 164, 183
Vista de desarrollo, 16
vista de despliegue, 16
vista de escenarios, 197
Vista de escenarios, 17
Vista de procesos, 16
Vista física, 16
vista funcional, 197
Vista lógica, 16
Visual Studio Code, 53, 62, 117
Búqueda, 149
errores, 150
Quick fix, 70
Refactorización, 70
void, 150

W3C, 30
WebDriver, 157
WebElement, 159
WebSocket, 433
  @OnClose, 298
  @OnMessage, 298
  @OnOpen, 298
  @ServerEndPoint, 298
  conexión, 303
  creación de un WebSocket cliente en el *back*, 304
  especificación, 298
  Establecimiento de la conexión, 298
  handshake, 298
WebSocket en Angular, 367
WebSockets, 297
  cierre de sesiones, 319
  Errores, 317
  getHandshakeHeaders, 434
  handshake, 434
  lectura de cabeceras, 434
WebSocketSession, 298, 301, 311, 322
widget, 29, 157, 260
withCredentials, 274, 356, 401
World Wide Web Consortium, 30
wrapper, 319
Wrapper, 310
ws, 44, 297
WWW-Authenticate, 39
XMLHttpRequest, 30
xpath, 159
YourKit Java Profiler, 173